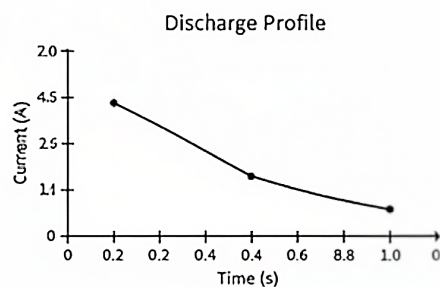
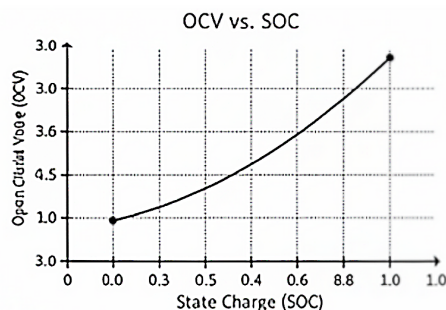

Modeling and Simulation Companion to Battery Technology for Electrical Engineers

```
class BatteryCell:
def [intiiH] = capacity, ocV-data);
def capacity = capacity, ocV-data);
def simulate, current, dt)
def simulate(( current, dt);
    pass
/ Iintilize battery model
ocv-curve = (0.0, 3.0) 0.4, 3.7, 0.6, 3.9)
battery = BatteryCall(100, curre)
))
```



Copyright © 2026 Avirup Kundu

First edition

Published May 2026

Publisher: Self Published by Avirup Kundu

This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0).

You are free to share and adapt the material for noncommercial purposes, provided that appropriate credit is given, a link to the license is included, and any changes made are indicated.

License:

<https://creativecommons.org/licenses/by-nc/4.0/>

Author: Avirup Kundu

Website: <https://www.avirup.net/>

Cover design: AI-generated artwork

Companion to:

Battery Technology for Electrical Engineers: A Self-Study Text

Contents

1 **The Research Computing Environment** **1**

 1.1 Prerequisites Check 3

 1.2 Environment Setup 3

 1.3 Conceptual Bridge: From Battery Theory to Reproducible Research Objects 9

 1.4 Open-Ended Exercises 38

 1.5 Worked Solutions to the Open-Ended Exercises 39

2 **Scientific Python Refresher for Battery Work** **47**

 2.1 Prerequisites Check 49

 2.2 Environment Setup 50

 2.3 Conceptual Bridge: From the Equivalent Circuit in the Textbook to a State Vector in Python 53

 2.4 Open-Ended Exercises 89

 2.5 Worked Solutions to Open-Ended Exercises 90

3 **Your First PyBaMM Simulation** **101**

 3.1 Prerequisites Check 103

 3.2 Environment Setup 103

 3.3 Conceptual Bridge: From Textbook DFN Equations to PyBaMM Objects 106

 3.4 Open-Ended Exercises 137

 3.5 Worked Solutions to the Open-Ended Exercises 138

4 **Parameters, Experiments, and Drive Cycles** **146**

 4.1 Prerequisites Check 148

 4.2 Environment Setup 148

 4.3 Conceptual Bridge: What the Tool Thinks a “Parameter” and an “Experiment” Are 151

4.4	Open-Ended Exercises	181
4.5	Worked Solutions	182
5	Parameter Estimation in PyBaMM	191
5.1	Prerequisites Check	193
5.2	Environment Setup	193
5.3	Conceptual Bridge: From Textbook Identifiability to PyBaMM In- put Parameters	196
5.4	A Brief Bayesian Detour	224
5.5	Open-Ended Exercises	227
5.6	Worked Solutions to the Open-Ended Exercises	228
6	Equivalent Circuit Models from Scratch	233
6.1	Prerequisites Check	234
6.2	Environment Setup	235
6.3	Conceptual Bridge: From Textbook ECM Theory to MATLAB State Updates	239
6.4	Open-Ended Exercises	270
6.5	Worked Solutions to the Open-Ended Exercises	271
7	SOC Estimation with Kalman Filters	276
7.1	Prerequisites Check	278
7.2	Environment Setup	278
7.3	Conceptual Bridge: From an ECM to a Nonlinear State Observer .	281
7.4	Open-Ended Exercises	337
7.5	Worked Solutions to the Open-Ended Exercises	339
8	SOH and Aging Models	351
8.1	Prerequisites Check	353
8.2	Environment Setup	353
8.3	Conceptual Bridge: From Degradation Mechanisms to SOH Model Terms	357
8.4	Dataset Integration Notes	389
8.5	Open-Ended Exercises	393
8.6	Worked Solutions	394
9	Thermal Modeling and Electrothermal Coupling	402

9.1	Prerequisites Check	403
9.2	Environment Setup	404
9.3	Conceptual Bridge: From Bernardi Heat to PyBaMM Thermal Objects	407
9.4	Simulink and Simscape Thermal Modeling: A Practical Bridge . .	432
9.5	Open-Ended Exercises	433
9.6	Worked Solutions to Open-Ended Exercises	433
9.7	Dataset Integration Notes	437
10	Bridging PyBaMM and MATLAB	442
10.1	Prerequisites Check	443
10.2	Environment Setup	444
10.3	Conceptual Bridge: Models, Data, and Contracts Across Tools . .	448
10.4	Open-Ended Exercises	482
10.5	Worked Solutions to the Open-Ended Exercises	483
11	Public Battery Datasets in Depth	490
11.1	Prerequisites Check	492
11.2	Environment Setup	492
11.3	Conceptual Bridge: From Electrochemical Observables to Dataset Objects	495
11.4	Open-Ended Exercises	527
11.5	Worked Solutions to Open-Ended Exercises	529
12	The Reproduction Project	536
12.1	Prerequisites Check	537
12.2	Environment Setup	538
12.3	Conceptual Bridge: From Reading a Paper to Rebuilding a Computational Claim	541
12.4	Open-Ended Exercises	575
13	Specialization Tracks	582
13.1	Prerequisites Check	583
13.2	Environment Setup	584
13.3	Conceptual Bridge: From General Competence to Specialization .	586
13.4	How to Use This Chapter	587
13.5	Track A: Advanced State Estimation	588

13.6	Track B: Thermal and Safety	597
13.7	Track C: Data-Driven Health Estimation	603
13.8	Track D: Grid Storage Applications	612
13.9	Track E: EV Powertrain Integration	617
13.10	Track F: Fast Charging Optimization	622
13.11	Capstone Pre-Proposal Template	632
14	The Capstone Project	634
14.1	Prerequisites Check	635
14.2	Environment Setup	636
14.3	Conceptual Bridge: From Labs to a Research Artifact	638
14.4	Open-Ended Exercises	667

The Research Computing Environment

This chapter is about a skill that sounds administrative until the day it saves your paper: building a research computing environment you can trust. By the end of the chapter, you will not merely have Python, Jupyter, MATLAB, and Git installed. You will have a working repository, an isolated environment, a repeatable project layout, a research log template, and a pair of sanity-check simulations that prove your tools are actually behaving the way you think they are. That combination is what separates “I got a plot once” from “I can still reproduce this result six weeks later, explain where it came from, and defend it in front of an advisor or reviewer.”

This matters because simulation research fails in boring ways long before it fails in interesting ways. A paper is not weakened only by a wrong diffusion coefficient or a bad thermal assumption. It is also weakened by a missing package version, a notebook that only runs on one laptop, an unlabeled CSV, a folder full of `final_results_v2_really_final`, or a figure whose exact generation path has been forgotten. Reviewers rarely see those internal failures directly, but they feel them in the form of irreproducible methods, vague parameter provenance, suspiciously convenient plots, and “we were unable to verify your claimed workflow” when someone in the group tries to build on your work later. The point of this chapter is to prevent those failures before they become habits.

This companion is a methods book, so we will treat environment setup as part

of battery research rather than a preface to it. Later chapters will ask you to run PyBaMM models, identify equivalent-circuit parameters from public datasets, implement Kalman filters in MATLAB, and reproduce published figures. None of that work is credible if you cannot say exactly which software versions you used, which script generated which output, which assumptions were teaching shortcuts and which were field-standard practice, and where each intermediate result was stored. Research competence is not only the ability to derive equations from first principles. It is also the ability to preserve the chain from equation to executable model to stored artifact.

Keep several theory-textbook chapters nearby while you work. Textbook Chapter 3, *Performance Metrics and Terminology*, is the one to keep most open during this chapter, because every file name, axis label, unit annotation, and sanity-check output should use the vocabulary from that chapter consistently. When we record current, voltage, energy, capacity, SOC, and C-rate in files and plots, we are operationalizing that chapter. Textbook Chapter 7, *Degradation Mechanisms*, matters here too, because degradation claims are especially vulnerable to poor provenance: if you do not know which dataset, temperature window, and preprocessing code produced a trend, you do not know what physical mechanism you are actually looking at. Textbook Chapter 8, *Heat in Batteries*, will become relevant as soon as we start running electrothermal models; the same reproducibility discipline we build here will later protect you from silently mixing ambient temperature assumptions or heat-generation conventions. Textbook Chapter 10, *State Estimation*, is also worth keeping in mind because BMS algorithms look mathematically neat on paper and become fragile as soon as data conventions drift. This chapter is the infrastructure layer underneath all of them.

One more framing point matters for this manual in particular: you do not have wet-lab access. That is not a weakness if you are deliberate. It means your research identity will be built around careful simulation, disciplined use of public datasets, strong reproduction practice, and clean software methods. In other words, your credibility will come from the exact things we are building here. A simulation-based battery researcher can produce publishable work without touching an Arbin cycler, but not without knowing how to keep environments isolated, analyses versioned, and outputs traceable.

So we will move slowly and do the unglamorous work properly. We will decide when to use `conda` and when to use `venv`. We will pin package versions instead of

trusting memory. We will create a project structure that still makes sense when the project doubles in size. We will verify that Jupyter sees the correct kernel. We will check MATLAB installation status for the toolboxes we need later. We will initialize Git before the repository is messy. We will create a research log template before there is anything exciting to log. And then we will end with a “hello battery” in both Python and MATLAB, because a battery methods workflow should prove itself on battery-flavored output, not just `print("hello world")`.

If you do this chapter carefully, later chapters become much easier. If you rush it, later chapters become haunted. Let’s build the boring foundation well enough that the interesting work can stand on it.

1.1 Prerequisites Check

- Required software: Miniforge or Anaconda for environment management; Python 3.11; Git 2.43+; JupyterLab 4.4+; MATLAB R2025b recommended, R2024b acceptable
- Python packages used in this chapter: `numpy==2.3.4`, `scipy==1.16.0`, `pandas==3.0.2`, `matplotlib==3.10.8`, `jupyterlab==4.4.10`, `ipykernel==7.2.0`
- MATLAB products to verify: Simulink, Simscape, Simscape Battery, Control System Toolbox, Optimization Toolbox
- Required textbook chapters: Textbook Chapter 3 is essential; Chapters 7, 8, and 10 should be familiar conceptually
- Required prior lab chapters: none
- Estimated time: 6 to 8 hours if installs go smoothly; 8 to 10 hours if you are also installing MATLAB or learning Git from scratch

If your Python fundamentals are shaky, pause and review array creation, scripts vs notebooks, and package imports before continuing. If Git is entirely new, that is fine; this chapter assumes no research-Git experience and builds only what we need. If MATLAB is not installed yet, do not skip the verification section. Later chapters in Part III will depend on it.

1.2 Environment Setup

We will set up the project in a way that is reproducible, boring, and strong. That is exactly the right combination.

1.2.1 Choose **conda** or **venv**

For this manual, **conda** is the default recommendation and **venv** is the fallback. The reason is practical rather than ideological. Battery research stacks often include scientific libraries with compiled dependencies, and **conda-forge** resolves those dependencies more smoothly across Windows, macOS, and Linux than plain **pip** does.

Use this rule:

- **conda**: use it when you want the least painful scientific-stack install. Tradeoff: a slightly heavier environment manager.
- **venv**: use it when you already have a clean system Python and want minimal tooling. Tradeoff: more likely to hit platform-specific wheel issues.

If you do not already have a strong preference, use **conda**.

1.2.2 Create the repository folder

Create a working folder anywhere you keep research projects. I will call it **sib-research-companion**, but you may choose another name.

```
mkdir sib-research-companion
cd sib-research-companion
```

The expected result is simple: your terminal prompt should now show that you are inside the new folder. If you run **pwd** on Linux or macOS, or **cd** on Windows PowerShell, you should see the new path.

1.2.3 Create the Python environment with **conda**

Create a file named **environment.yml** in the repository root with the following contents.

```
name: sib-research
channels:
  - conda-forge
dependencies:
  - python=3.11
  - numpy=2.3.4
  - scipy=1.16.0
  - pandas=3.0.2
  - matplotlib=3.10.8
  - jupyterlab=4.4.10
```

```
- ipykernel=7.2.0
- pip
```

This file is intentionally small. It is the human-maintained description of the environment, not the machine-generated lockfile. We pin the major tools exactly because this manual is teaching research habits, not “install whatever was current on the day you happened to run the notebook.”

Now create and activate the environment:

```
conda env create -f environment.yml
conda activate sib-research
```

If `conda activate` fails because your shell has not been initialized, run the shell hook once:

```
conda init bash
```

Then close and reopen the terminal and activate again.

If you prefer `mamba`, the command is equivalent and often faster:

```
mamba env create -f environment.yml
mamba activate sib-research
```

1.2.4 venv fallback

If you are not using `conda`, create a `requirements-venv.txt` file:

```
numpy==2.3.4
scipy==1.16.0
pandas==3.0.2
matplotlib==3.10.8
jupyterlab==4.4.10
ipykernel==7.2.0
```

Then create and activate the environment.

On Linux or macOS:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements-venv.txt
```

On Windows PowerShell:

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements-venv.txt
```

The `pip install --upgrade pip` line matters. Many beginner installation failures on Windows trace back to using an old `pip` against a modern wheel ecosystem.

1.2.5 Verify the Python scientific stack

Run this exact verification snippet from the repository root after activating the environment.

```
python - <<'PY'
import numpy as np
import scipy
import pandas as pd
import matplotlib

print("Scientific Python stack verified.")
print(f"NumPy      : {np.__version__}")
print(f"SciPy      : {scipy.__version__}")
print(f"Pandas     : {pd.__version__}")
print(f"Matplotlib  : {matplotlib.__version__}")
PY
```

You should see terminal output of this form:

```
Scientific Python stack verified.
NumPy      : 2.3.4
SciPy      : 1.16.0
Pandas     : 3.0.2
Matplotlib : 3.10.8
```

The exact patch versions should match your pins. If they do not, you are not in the environment you think you are in.

1.2.6 Register the Jupyter kernel

A common mistake is installing packages into one environment and launching Jupyter from another. We prevent that now.

```
python -m ipykernel install --user --name sib-research --display-name "Python  
↳ (sib-research)"
```

Launch JupyterLab:

```
jupyter lab
```

Expected behavior: a browser tab opens or Jupyter prints a local URL that you can paste into a browser. In the launcher, the kernel list should contain Python (sib-research). Create a new notebook with that kernel and run:

```
import sys  
print(sys.executable)
```

The path should point into your sib-research environment, not system Python.

1.2.7 Install or verify MATLAB

For the MATLAB side of this manual, MATLAB R2025b is the recommended target version. R2024b is acceptable if that is what your license or trial provides. Later chapters will lean on products beyond base MATLAB, so install or verify these now:

- MATLAB
- Simulink
- Simscape
- Simscape Battery
- Control System Toolbox
- Optimization Toolbox

Use the MathWorks installer and choose a custom install if you have the option, so you can explicitly select toolboxes. If you are using a trial or campus license, availability of Simscape Battery is the item to check most carefully; base MATLAB alone will not be enough for Part III.

Open MATLAB and run:

```
ver
```

You should see a version table listing installed products. Do not worry yet if the list is long. In the guided walkthrough below, we will write a script that checks only the products we care about.

1.2.8 Install and verify Git

Check Git:

```
git --version
```

You should see something like:

```
git version 2.43.0
```

Now configure your identity once:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git config --global init.defaultBranch main
```

These settings become part of your commit metadata. If the email is wrong, your GitHub history later becomes harder to trace correctly.

1.2.9 Minimal environment hello world

Before we build the battery-specific sanity check later, verify that the environment can run a tiny end-to-end Python snippet.

```
python - <<'PY'
import numpy as np
time_s = np.arange(0, 6)
voltage_v = 3.2 + 0.1 * np.cos(time_s)
print("time_s:", time_s.tolist())
print("voltage_v:", np.round(voltage_v, 3).tolist())
PY
```

Expected output:

```
time_s: [0, 1, 2, 3, 4, 5]
voltage_v: [3.3, 3.254, 3.158, 3.101, 3.135, 3.228]
```

The exact floating-point rounding may differ at the third decimal place, but the structure should be identical.

1.2.10 Common Install Failures and Fixes

The most common environment problems are not mysterious. They are usually one of four things.

1. conda: command not found

Symptom: your shell cannot see `conda` at all.

Fix: install Miniforge or Anaconda, then run `conda init bash` and restart the terminal.

2. `ModuleNotFoundError` after a successful install

Symptom: `pip` or `conda` said the package installed, but Python still cannot import it.

Fix: you are probably in the wrong interpreter. Run `which python` on Linux/macOS or `Get-Command python` on PowerShell, and confirm it points to your intended environment.

3. Jupyter launches but the wrong packages appear inside the notebook

Symptom: the notebook imports fail even though the terminal imports succeed.

Fix: register the kernel with `python -m ipykernel install --user --name sib-research ...` and then explicitly choose Python (`sib-research`) in JupyterLab.

4. MATLAB starts but needed toolboxes are missing

Symptom: `ver` does not list Simulink, Simscape, or Simscape Battery.

Fix: rerun the installer with toolbox selection enabled, or confirm your license includes them. If you only have base MATLAB today, you can still complete this chapter, but Part III will need either a trial or a fuller license.

1.3 Conceptual Bridge: From Battery Theory to Reproducible Research Objects

In the theory textbook, a battery model lives as mathematics, physics, and interpretation. You learned to think in terms of half-reactions, transport limitations, OCV curves, degradation mechanisms, thermal balances, and state estimators. That level is essential. But a research workflow needs one more translation layer: how those ideas become stable computational objects that can be rerun, compared, and defended.

The most important shift is this: a simulation result is not just a curve. It is a claim with dependencies.

Suppose later in this manual you run a PyBaMM DFN model and obtain a terminal-voltage trace that seems to match a published figure. In theory language, that trace depends on equations, boundary conditions, parameters, and a current profile. In research-computing language, it also depends on a specific package

version, a specific parameter file, a specific script, and a specific output directory. If any one of those is ambiguous, the result is weaker than it looks. The curve may be visually correct and still be scientifically fragile.

That is why environment management belongs in a battery methods book. The environment is where the abstract model becomes executable. Textbook Chapter 3 taught you to distinguish capacity from energy, OCV from terminal voltage, and SOC from SOH. Here we add another distinction that matters just as much in practice: the difference between a result and a reproducible result. A result is “I ran something and got 3.27 V after 1800 s.” A reproducible result is “Script `src/hello_battery.py`, run under environment `sib-research`, using `numpy 2.3.4` and `matplotlib 3.10.8`, produced `results/hello_battery_python.csv` and `figures/hello_battery_python.png` from commit `abc1234` on 2026-04-14.” The second form looks fussier, but it is the one you can build a paper or thesis on.

This is also where notebook culture often misleads beginners. Jupyter notebooks are excellent for exploration. They are poor as the sole home of a research workflow. Why? Because notebooks interleave code, output, and state in a way that encourages hidden dependencies. A cell may run only because an earlier cell created a variable yesterday. A figure may be regenerated from a stale in-memory DataFrame rather than from the raw data. For that reason, this manual will keep using notebooks, but we will pair them with scripts in `src/` and with saved outputs in `results/`. That separation is standard practice. Exploratory work can be notebook-first. Reproducible work should become script-backed.

The folder structure matters for the same reason. When you put raw downloads, cleaned datasets, notebooks, reusable code, generated figures, and paper notes into the same directory, you erase provenance. Later, when you revisit a model, you cannot tell whether a CSV is raw, processed, smoothed, manually edited, or exported from MATLAB. A clean project structure is not aesthetic minimalism. It is a physical embodiment of method.

Git plays an equally important role. Many people first meet Git as a collaboration tool, but in research it is just as valuable as a memory tool. A commit is a checkpoint in your reasoning. It lets you say, “This is the exact state of the repository before I changed the preprocessing,” or “This figure was generated before I changed the OCV smoothing routine.” That matters enormously once you start reproducing published papers. If a reproduction attempt diverges from the target figure, you want the freedom to compare commit states rather than guessing which invisible notebook

change caused the drift.

Research logs are the human counterpart to Git history. Git can tell you which files changed. It cannot tell you why you distrusted a data column, why you chose one solver tolerance over another, or why you decided a mismatch with a paper was acceptable. Those decisions must live somewhere readable. A dated Markdown log is a remarkably effective choice because it is plain text, version-controllable, diffable, and searchable. It also trains you to think like a researcher rather than a code runner. You are not merely asking, “Did the script execute?” You are asking, “What assumption did I just make, and will future-me remember it?”

This is especially important for sodium-ion work. Textbook Chapter 13 emphasized that sodium-ion workflows cannot always inherit lithium-ion assumptions unchanged. OCV shapes differ. Dataset availability is worse. Figure digitization is more common because parameter sets and full data releases are less standardized. That means provenance matters even more. In a sparse-data setting, every preprocessing choice carries extra weight.

So the bridge from theory to tools is not only about learning commands. It is about learning how a scientific argument is preserved in software form. Equations become scripts and parameter files. Assumptions become log entries and config choices. Outputs become versioned artifacts. Reproducibility becomes the operating system of the whole project. With that frame in place, the next sections will feel less like computer setup and more like the beginning of actual battery research.

Guided Walkthrough 1

Build and Freeze the Python Environment

Learning objective: Create an isolated Python environment, verify that it is the one you are actually using, and generate a machine-readable record of its package versions.

Before we write any battery-flavored code, we need to establish one discipline that will repeat throughout this companion: every environment should be inspectable. It is not enough to say “I used Python with NumPy and SciPy.” That description is too vague for research. We want a human-readable environment specification and a machine-generated environment record.

1.3.1 Walkthrough 1 code

First, create `environment.yml` exactly as shown earlier if you have not already done so. Then create a script called `src/env_report.py`.

```
from __future__ import annotations

import json
import platform
import subprocess
import sys
from datetime import datetime, timezone
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path

PACKAGES_TO_REPORT = [
    "numpy",
    "scipy",
    "pandas",
    "matplotlib",
    "jupyterlab",
    "ipykernel",
]

def package_version(package_name: str) -> str:
    try:
        return version(package_name)
    except PackageNotFoundError:
        return "NOT_INSTALLED"

def git_commit_hash() -> str:
    try:
        return subprocess.check_output(
            ["git", "rev-parse", "HEAD"],
            text=True,
        ).strip()
    except Exception:
        return "NOT_A_GIT_REPOSITORY"

def main() -> None:
    results_dir = Path("results")
    results_dir.mkdir(parents=True, exist_ok=True)

    report = {
        "generated_utc": datetime.now(timezone.utc).isoformat(),
        "python_executable": sys.executable,
        "python_version": platform.python_version(),
        "platform": platform.platform(),
        "git_commit": git_commit_hash(),
    }
```

```

    "packages": {
        package_name: package_version(package_name)
        for package_name in PACKAGES_TO_REPORT
    },
}

output_path = results_dir / "environment_report.json"
output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")

print(f"Environment report written to: {output_path}")
print(json.dumps(report, indent=2))
if __name__ == "__main__":
    main()

```

Run it:

```

mkdir -p src results
python src/env_report.py

```

1.3.2 Walkthrough 1 code explanation

The `from __future__ import annotations` line is a small modern-Python convenience that keeps type annotations lightweight. It is not strictly necessary here, but it is a good habit and will appear again later.

We import `json`, `platform`, `subprocess`, and `sys` because we want the report to include not only package versions but also interpreter and operating-system information. That is important because a solver bug can be platform-dependent, and a kernel mismatch often shows up first in `sys.executable`.

`datetime.now(timezone.utc).isoformat()` deliberately uses UTC rather than local time. In collaborative or long-running projects, UTC timestamps reduce ambiguity. If you later compare logs from two machines or time zones, UTC keeps entries aligned.

The `package_version` function wraps `importlib.metadata.version`. That call asks the installed environment for a package version without importing the whole package. This is slightly cleaner than importing every package just to read `__version__`, and it works for most Python distributions.

The `git_commit_hash` function tries to capture the current repository state. If Git has not been initialized yet, it returns `NOT_A_GIT_REPOSITORY` instead of crashing. That is intentional. A good environment-report script should degrade

gracefully.

Inside `main`, we create `results/` if it does not already exist. This matters because scripts that assume preexisting folders are brittle. We are teaching robust patterns from the start.

The `report` dictionary contains five categories of information: generation time, Python executable path, Python version, operating-system platform, Git commit, and selected packages. This is not the maximum information you could capture, but it is enough to answer the core reproducibility question: “What environment produced this output?”

The report is written to `results/environment_report.json`. JSON is a sensible choice because it is plain text, structured, easy to diff, and easy to read back later from Python or MATLAB if needed.

1.3.3 Walkthrough 1 expected output

In the terminal, you should see something like:

```
Environment report written to: results/environment_report.json
{
  "generated_utc": "2026-04-14T12:34:56.789012+00:00",
  "python_executable":
↳  "/home/yourname/miniforge3/envs/sib-research/bin/python",
  "python_version": "3.11.11",
  "platform": "Linux-6.8.0-57-generic-x86_64-with-glibc2.39",
  "git_commit": "NOT_A_GIT_REPOSITORY",
  "packages": {
    "numpy": "2.3.4",
    "scipy": "1.16.0",
    "pandas": "3.0.2",
    "matplotlib": "3.10.8",
    "jupyterlab": "4.4.10",
    "ipykernel": "7.2.0"
  }
}
```

Three details matter most.

First, `python_executable` should clearly point into your intended environment. If it points to system Python, stop and fix that before doing anything else.

Second, every package should show the pinned version you intended. If one shows `NOT_INSTALLED`, your environment is incomplete.

Third, `git_commit` will probably say `NOT_A_GIT_REPOSITORY` the first time. That is expected. Later, once Git is initialized, the same script will start recording commit hashes automatically. That change is the first small example of how good tooling compounds.

1.3.4 Walkthrough 1 troubleshooting

1. `ModuleNotFoundError: No module named 'importlib.metadata'`
This can happen only on older Python versions.
Fix: use Python 3.11 as specified.
2. The script writes a report, but the versions are not the pinned ones
Symptom: for example, `numpy` shows a different version from `environment.yml`.
Fix: confirm the activated environment with `python -c "import sys; print(sys.executable)"`.
3. `git_commit` always says `NOT_A_GIT_REPOSITORY`
Symptom: even after you think you initialized Git.
Fix: make sure you are running the script from the repository root, not from some parent or sibling folder.

1.3.5 Walkthrough 1 reflection

This exercise taught the first durable pattern of the manual: every meaningful output should be accompanied by enough environment metadata to explain where it came from. We will reuse this idea later when fitting parameters, exporting PyBaMM runs, and reproducing published figures.

Guided Walkthrough 2

Create a Research Project Scaffold That Will Still Make Sense Later

Learning objective: Build a folder structure that cleanly separates raw data, processed data, notebooks, reusable code, outputs, and paper materials.

A common beginner failure in computational battery work is building a project around a single notebook and a flat directory. That works for one evening. It fails for a semester. We are going to create a cross-platform project scaffold with Python so that anyone on any operating system can reproduce it exactly.

1.3.6 Walkthrough 2 code

Create `src/bootstrap_project.py`:

```
from __future__ import annotations

from pathlib import Path

PROJECT_DIRECTORIES = [
    "data/raw",
    "data/processed",
    "notebooks",
    "src",
    "results",
    "figures",
    "papers",
    "logs",
    "matlab",
]

README_TEXT = """# Sodium-Ion Battery Research Companion Workspace

This repository contains the hands-on exercises developed from the battery
↳ tools companion manual.

## Environment

Create the main Python environment with:

~~~bash
conda env create -f environment.yml
conda activate sib-research
python -m ipykernel install --user --name sib-research --display-name "Python
↳ (sib-research)"
~~~

## Repository layout

- `data/raw/` stores downloaded datasets exactly as obtained.
- `data/processed/` stores cleaned or transformed datasets.
- `notebooks/` stores exploratory notebooks.
- `src/` stores reusable Python scripts and modules.
- `results/` stores machine-readable outputs such as CSV and JSON files.
- `figures/` stores generated figures ready for reports or papers.
- `papers/` stores paper notes, PDFs, and reproduction plans.
- `logs/` stores dated research-log entries.
```

```
- `matlab/` stores MATLAB scripts and Live Scripts.
```

```
## Reproducibility rule
```

```
If an output matters, it should be reproducible from a script, recorded in  
↳ the log, and traceable to a commit.
```

```
"""
```

```
GITKEEP_DIRECTORIES = [  
    "data/raw",  
    "data/processed",  
    "notebooks",  
    "results",  
    "figures",  
    "papers",  
    "logs",  
    "matlab",  
]  
  
def main() -> None:  
    for directory in PROJECT_DIRECTORIES:  
        Path(directory).mkdir(parents=True, exist_ok=True)  
  
    for directory in GITKEEP_DIRECTORIES:  
        gitkeep_path = Path(directory) / ".gitkeep"  
        gitkeep_path.touch(exist_ok=True)  
  
    readme_path = Path("README.md")  
    if not readme_path.exists():  
        readme_path.write_text(README_TEXT, encoding="utf-8")  
  
    print("Project scaffold created successfully.")  
    for directory in PROJECT_DIRECTORIES:  
        print(f" - {directory}")  
if __name__ == "__main__":  
    main()
```

Run it:

```
python src/bootstrap_project.py
```

Then inspect the directory tree:

```
find . -maxdepth 2 -type d | sort
```

On Windows PowerShell, use:

1.3.7 Walkthrough 2 code explanation

The `PROJECT_DIRECTORIES` list is the whole design in one place. This is deliberate. If your project structure lives only in shell history, it becomes hard to audit and hard to reuse. By representing the scaffold as a Python list, we make the structure explicit and version-controllable.

Each folder has a methodological role. `data/raw/` is sacred: files there should be exact downloads from public sources and should not be edited in place. `data/processed/` is where parsed, renamed, resampled, or cleaned files go. `notebooks/` is exploratory. `src/` is reusable code. `results/` and `figures/` hold outputs, but they are separated because machine-readable tables and publication-ready figures are not the same artifact. `papers/` is where reading notes, PDFs, and reproduction plans live. `logs/` captures reasoning. `matlab/` isolates the second language ecosystem we will need later.

The `README_TEXT` string is not filler. A repository without a short explanation of its purpose, layout, and environment commands becomes opaque surprisingly fast. Think of the README as the front panel of the lab bench.

The `GITKEEP_DIRECTORIES` list exists because Git does not track empty directories. The `.gitkeep` files are placeholders that let you commit an intended structure before data and outputs arrive.

The script only writes `README.md` if it does not already exist. That protects later manual edits and is a small example of idempotent design: you should be able to rerun setup scripts without damaging work.

1.3.8 Walkthrough 2 expected output

The terminal should print:

```
Project scaffold created successfully.
- data/raw
- data/processed
- notebooks
- src
- results
- figures
- papers
- logs
```



```
- matlab
```

When you inspect the directory tree, you should see those folders under the repository root. The exact ordering may differ by platform, but nothing should be missing.

1.3.9 What correct structure looks like

A good structure after this step has three properties.

First, it is semantically meaningful. Someone joining your project should be able to guess where a downloaded dataset belongs and where a generated figure belongs without asking you.

Second, it separates source from artifact. Scripts go in `src/`; generated outputs go in `results/` or `figures/`. This keeps the repository conceptually clean.

Third, it is expandable. When we later add dataset loaders, PyBaMM experiments, MATLAB estimators, and reproduction notebooks, this scaffold will still hold.

1.3.10 Walkthrough 2 troubleshooting

1. The script runs but folders are created in the wrong place

Symptom: you accidentally ran it from your home directory or a parent folder.

Fix: `cd` into the repository root and run it again.

2. The `README.md` does not appear

Symptom: the script created folders but not the `README`.

Fix: check whether a `README` already existed. The script intentionally does not overwrite one.

3. `find` is not available

Symptom: the directory-inspection command fails on Windows or minimal shells.

Fix: use the platform-specific alternative shown above.

1.3.11 Walkthrough 2 reflection

This exercise taught that reproducibility begins with physical organization. Later, when we download CALCE or NASA datasets, or when we export model outputs for MATLAB, the distinction between raw, processed, and generated artifacts will save you from subtle mistakes.

Guided Walkthrough 3

Start a Research Log and Capture Decisions Before They Disappear

Learning objective: Create a log system that records decisions, failures, assumptions, and next steps in a form that Git can track and future-you can trust.

Most beginners think a research log is for major milestones. In practice, it is more valuable for the small decisions that would otherwise evaporate: why you distrusted a data column, why you clipped a current spike, why you changed a solver, why you think a reproduction mismatch is the paper's fault rather than yours. Git tracks file states. The research log tracks intent and interpretation.

1.3.12 Walkthrough 3 code

Create `src/create_log_entry.py`:

```
from __future__ import annotations

from datetime import datetime
from pathlib import Path

TEMPLATE = """# Research Log: {date}

## Objective

Write one sentence describing today's concrete objective.

## Environment

- Active Python environment:
- MATLAB version:
- Git commit:
- Dataset or paper in use:

## Work Performed

Describe what you actually did, not what you planned to do.

## Observations

Record numerical outputs, figure behavior, runtime notes, or anything
↳ surprising.
```

Decisions and Rationale

Explain why you chose a solver, parameter bound, preprocessing rule, or file
↪ organization choice.

Problems Encountered

Record failures honestly. Include exact error messages where useful.

Next Steps

Write the next one to three actions you would take if you resumed tomorrow.
"""

```
def main() -> None:
    logs_dir = Path("logs")
    logs_dir.mkdir(parents=True, exist_ok=True)

    date_string = datetime.now().strftime("%Y-%m-%d")
    output_path = logs_dir / f"{date_string}_setup.md"

    if output_path.exists():
        print(f"Log entry already exists: {output_path}")
        return

    output_path.write_text(TEMPLATE.format(date=date_string),
    ↪ encoding="utf-8")
    print(f"Created log entry: {output_path}")
if __name__ == "__main__":
    main()
```

Run it:

```
python src/create_log_entry.py
```

Now open the generated file in your editor and replace the placeholders with your real setup notes. A first entry might include package installation success, MATLAB toolbox status, and whether GitHub remote setup is pending.

1.3.13 Suggested first log entry content

If you want a concrete model, the first real entry can look like this:

```
# Research Log: 2026-04-14
```

```
## Objective
```

Create a reproducible research environment for the battery simulation
↳ companion manual.

Environment

- Active Python environment: ``sib-research``
- MATLAB version: ``R2025b``
- Git commit: not initialized yet
- Dataset or paper in use: none

Work Performed

Installed the pinned Python stack from ``environment.yml``, registered the
↳ Jupyter kernel, and created the base repository scaffold using
↳ ``src/bootstrap_project.py``. Verified that imports for NumPy, SciPy,
↳ Pandas, and Matplotlib succeed. Opened MATLAB and confirmed that Simulink
↳ and Simscape are available, but still need to confirm Simscape Battery.

Observations

Python executable points into the conda environment as expected. Jupyter sees
↳ the ``Python (sib-research)`` kernel. MATLAB launches normally. No datasets
↳ downloaded yet.

Decisions and Rationale

Chose ``conda`` over ``venv`` because later chapters will use compiled scientific
↳ libraries and cross-platform reproducibility is more important than
↳ minimal tooling. Chose to keep raw and processed data in separate
↳ directories from day one to avoid later provenance confusion.

Problems Encountered

``conda activate`` did not work until running ``conda init bash`` and restarting
↳ the terminal.

Next Steps

1. Initialize Git and commit the scaffold.
2. Write Python and MATLAB hello-battery sanity checks.
3. Confirm Simscape Battery availability.

1.3.14 Walkthrough 3 code explanation

The script creates a dated Markdown file rather than a notebook, Word document, or proprietary note format. That choice is deliberate. Markdown is plain text, easy to diff, easy to search, and comfortable inside Git repositories.

The filename pattern `YYYY-MM-DD_setup.md` sorts naturally in chronological order. This sounds trivial until a project accumulates dozens of entries. Human-friendly ordering becomes a major quality-of-life feature.

The template sections are short because a research log should be sustainable. If the template is too long, you will stop using it. The fields here capture the essentials: objective, environment, work performed, observations, decisions, problems, and next steps.

The script refuses to overwrite an existing entry for the day. That is a good safety behavior. You can always edit the existing log, but you should not accidentally erase it.

1.3.15 Walkthrough 3 expected output

The terminal should show something like:

```
Created log entry: logs/2026-04-14_setup.md
```

The log file should exist and open as plain text. Once edited, it should read like a technical lab notebook rather than a diary. The difference matters. “Spent a while trying things” is not useful. “Tried `conda activate` before shell initialization; fixed with `conda init bash` and terminal restart” is useful.

1.3.16 What belongs in the log and what does not

A good rule is this: record anything that explains a result, a failure, or a future decision.

Record:

- environment changes
- dataset sources and licenses
- solver choices
- smoothing choices
- reasons for discarding or keeping data segments
- exact paper figures you are targeting in a reproduction
- uncertainties and ambiguities

Do not record:

- every command you typed if it had no consequence
- vague emotional summaries without technical content
- outputs that are already reproducible and fully captured elsewhere unless they matter interpretively

1.3.17 Walkthrough 3 troubleshooting

1. You create the template but never update it

Symptom: the repository has empty log files.

Fix: treat the generated template as a draft to be filled immediately, not later.

2. The log becomes a second README

Symptom: every entry repeats project background instead of recording decisions.

Fix: focus on what changed today.

3. You only log successes

Symptom: later you cannot reconstruct which dead ends you already tried.

Fix: record failed preprocessing or modeling attempts briefly and honestly.

1.3.18 Walkthrough 3 reflection

This exercise taught the human side of reproducibility. Later, when we compare model fidelities or troubleshoot estimation drift, your log will often be the only place where the rationale survives.

Guided Walkthrough 4

Initialize Git and Build a Commit History Worth Keeping

Learning objective: Turn the scaffold into a versioned research repository, add a battery-research-appropriate `.gitignore`, and make the first sequence of meaningful commits.

Git becomes helpful only when used before the repository is messy. Starting version control after a month of experiments is like deciding to label sample vials after mixing half of them together. We are going to initialize the repository now and establish a commit style that later supports actual research.

1.3.19 Walkthrough 4 code

Create `.gitignore` in the repository root:

```
# Python cache and notebook state
__pycache__/
*.py[cod]
.ipynb_checkpoints/

# Virtual environments
.venv/
env/
ENV/

# Tool caches
.pytest_cache/
.mypy_cache/

# OS clutter
.DS_Store
Thumbs.db

# Large local data: keep directory placeholders, ignore contents
data/raw/*
!data/raw/.gitkeep
data/processed/*
!data/processed/.gitkeep

# Temporary outputs
results/tmp/
figures/tmp/

# MATLAB generated files
*.asv
*.slxc
*.mexw64
*.mexa64
*.mexmaci64
```

Initialize the repository and make the first commits:

```
git init
git add environment.yml
git add requirements-venv.txt
git add src/bootstrap_project.py
git add src/env_report.py
```

```
git add src/create_log_entry.py
git add README.md
git add .gitignore
git add data/raw/.gitkeep
git add data/processed/.gitkeep
git add notebooks/.gitkeep
git add results/.gitkeep
git add figures/.gitkeep
git add papers/.gitkeep
git add logs/2026-04-14_setup.md
git add matlab/.gitkeep
git commit -m "Initialize reproducible battery research scaffold"
```

Now generate an environment report so the repository captures a first machine-readable environment artifact:

```
python src/env_report.py
git add results/environment_report.json
git commit -m "Add machine-readable environment report"
```

Register the current default branch explicitly:

```
git branch -M main
```

If you already created a GitHub repository in the browser, connect it:

```
git remote add origin
↳ https://github.com/YOUR_USERNAME/sib-research-companion.git
git push -u origin main
```

1.3.20 A meaningful 10-commit sequence

Your chapter deliverable asks for at least ten meaningful commits. Here is a clean sequence you can actually use over the rest of this chapter:

1. Initialize reproducible battery research scaffold
2. Add machine-readable environment report
3. Add MATLAB toolbox verification script
4. Add Python hello-battery sanity check
5. Add MATLAB hello-battery sanity check
6. Save Python sanity-check outputs
7. Save MATLAB sanity-check outputs
8. Refine README with setup and workflow notes

9. Add research-log entry for chapter 1
10. Document chapter 1 deliverable completion

These are meaningful because each one captures a coherent step, not because the messages sound formal.

1.3.21 Walkthrough 4 code explanation

The `.gitignore` is tuned for research. It does not blindly ignore `results/` and `figures/` wholesale, because some results are worth tracking. Instead, it ignores only explicitly temporary subdirectories if you create them later. That choice teaches an important distinction: some generated artifacts are disposable, while some are central to the research record.

We ignore contents of `data/raw/` and `data/processed/` by default because public battery datasets can be large, and many are better downloaded by script or documented URL than committed to Git. The `.gitkeep` exceptions preserve the folder structure in the repository. This is standard practice.

The first commit gathers the reproducibility scaffold into a single checkpoint. The second commit adds the environment report as an artifact. Notice the pattern: one commit establishes capability, the next records evidence that the capability works. We will repeat that pattern throughout the manual.

1.3.22 Walkthrough 4 expected output

After the first commit, `git status` should say:

```
On branch main
nothing to commit, working tree clean
```

If you run:

```
git log --oneline --decorate --graph -n 5
```

you should see a short history of your recent commits, for example:

```
* a1b2c3d (HEAD -> main) Add machine-readable environment report
* e4f5g6h Initialize reproducible battery research scaffold
```

Your commit hashes will differ, of course.

1.3.23 What makes a commit "meaningful" in research

A meaningful research commit is not necessarily a large one. It is one whose purpose you can summarize in one sentence.

Good:

- Add parser for CALCE HPPC current and voltage columns
- Fix sign convention in UDDS current import
- Compare DFN and SPMe runtime on same experiment
- Add EKF tuning sweep notebook

Poor:

- updates
- misc
- stuff
- fixes

The point is not style policing. It is traceability. Six weeks later, when a result changes, a meaningful commit history lets you find why.

1.3.24 Walkthrough 4 troubleshooting

1. `git commit` refuses because user name or email is not configured

Symptom: Git prompts you to set identity.

Fix: run the `git config --global user.name` and `user.email` commands from the setup section.

2. You accidentally commit a large dataset

Symptom: `git status` shows many files inside `data/raw/`.

Fix: check that `.gitignore` exists before `git add .`, and use `git rm --cached` on accidentally tracked data files.

3. Jupyter notebooks create noisy diffs

Symptom: commits are dominated by output-cell changes.

Fix: keep exploratory notebooks, but move stable logic into scripts under `src/`.

1.3.25 Walkthrough 4 reflection

This exercise taught that version control is part of scientific method, not just collaboration etiquette. Later, when reproducing papers or tuning estimators, your commit history will become one of the main tools for diagnosing divergence.

Guided Walkthrough 5

Run a "Hello Battery" Sanity Check in Python and MATLAB

Learning objective: Verify that both ecosystems can produce a simple, battery-flavored result with known behavior, known units, and stored outputs.

Now we do the first genuinely battery-shaped computation of the manual. It is still intentionally simple. We are not building a real cell model yet. We are building a sanity check that proves the toolchain can generate, save, and interpret a small discharge experiment in both Python and MATLAB.

We will use the following pedagogical model:

$$\text{SOC}(t) = \text{SOC}_0 - \frac{It}{3600Q_{\text{nom}}} \quad (1.1)$$

$$V_{\text{OCV}}(\text{SOC}) = 3.0 + 0.6 \text{ SOC} \quad (1.2)$$

$$V_{\text{term}}(t) = V_{\text{OCV}}(\text{SOC}(t)) - IR_0 \quad (1.3)$$

This is not a publishable model. It is a teaching model. The goal is to verify the workflow, not to represent a chemistry accurately. The model is intentionally transparent so that you can predict what the output should look like before you run it.

We choose:

- nominal capacity: $Q_{\text{nom}} = 2.0 \text{ Ah}$
- current: $I = 2.0 \text{ A}$, which is a 1C discharge
- ohmic resistance: $R_0 = 0.05 \Omega$
- initial SOC: $\text{SOC}_0 = 1.0$
- simulation time: $t_{\text{final}} = 3600 \text{ s}$

From Equation (1.1), a 1C discharge over 3600 s should take SOC from 1.0 to 0.0. From Equations (1.2) and (1.3), OCV should fall linearly from 3.6 V to 3.0 V, while terminal voltage should sit 0.1 V lower, from 3.5 V to 2.9 V. If your output does not look like that, the toolchain or the code is wrong.

1.3.26 Part A: Python sanity check

Create src/hello_battery.py:

```
from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def main() -> None:
    # Create output directories if they do not already exist.
    Path("results").mkdir(parents=True, exist_ok=True)
    Path("figures").mkdir(parents=True, exist_ok=True)

    # Define a simple teaching model for a 2 Ah cell discharged at 1C.
    nominal_capacity_ah = 2.0
    discharge_current_a = 2.0
    ohmic_resistance_ohm = 0.05
    initial_soc = 1.0
    time_step_s = 1.0
    final_time_s = 3600.0

    # Build the simulation time vector.
    time_s = np.arange(0.0, final_time_s + time_step_s, time_step_s)

    # Coulomb counting: SOC falls linearly under constant current discharge.
    soc = initial_soc - (
        discharge_current_a * time_s / (3600.0 * nominal_capacity_ah)
    )
    soc = np.clip(soc, 0.0, 1.0)

    # Simple linear OCV curve used only for a sanity check.
    ocv_v = 3.0 + 0.6 * soc

    # Terminal voltage includes an ohmic drop during discharge.
    terminal_voltage_v = ocv_v - discharge_current_a * ohmic_resistance_ohm

    # Instantaneous power delivered to the load.
    power_w = terminal_voltage_v * discharge_current_a

    # Compute delivered energy by integrating power over time.
    delivered_energy_wh = np.trapz(power_w, time_s) / 3600.0

    # Store results in a tidy table.
```

```

results_table = pd.DataFrame(
    {
        "time_s": time_s,
        "soc_fraction": soc,
        "ocv_v": ocv_v,
        "terminal_voltage_v": terminal_voltage_v,
        "current_a": discharge_current_a,
        "power_w": power_w,
    }
)

results_csv_path = Path("results") / "hello_battery_python.csv"
results_table.to_csv(results_csv_path, index=False)

# Create a two-panel figure.
fig, axes = plt.subplots(2, 1, figsize=(8, 8), sharex=True)

axes[0].plot(time_s, soc, linewidth=2, color="tab:blue")
axes[0].set_ylabel("SOC [-]")
axes[0].set_title("Hello Battery Sanity Check (Python)")
axes[0].grid(True, alpha=0.3)
axes[0].set_ylim(-0.05, 1.05)

axes[1].plot(time_s, ocv_v, linewidth=2, label="OCV", color="tab:green")
axes[1].plot(
    time_s,
    terminal_voltage_v,
    linewidth=2,
    label="Terminal voltage",
    color="tab:red",
)
axes[1].set_xlabel("Time [s]")
axes[1].set_ylabel("Voltage [V]")
axes[1].grid(True, alpha=0.3)
axes[1].legend()

figure_path = Path("figures") / "hello_battery_python.png"
fig.tight_layout()
fig.savefig(figure_path, dpi=200)
plt.close(fig)

print(f"Saved results table to: {results_csv_path}")
print(f"Saved figure to: {figure_path}")
print(f"Initial terminal voltage: {terminal_voltage_v[0]:.2f} V")
print(f"Final terminal voltage: {terminal_voltage_v[-1]:.2f} V")

```

```

    print(f"Delivered capacity: {discharge_current_a * final_time_s /
        ↳ 3600.0:.2f} Ah")
    print(f"Delivered energy: {delivered_energy_wh:.2f} Wh")
if __name__ == "__main__":
    main()

```

Run it:

```
python src/hello_battery.py
```

Python sanity check explanation

The first two `Path(...).mkdir(...)` calls make the script robust. A reproducible script should not assume that output directories already exist.

The parameter block is deliberately explicit. We are not hiding constants in cryptic variable names. This is good scientific style: a reader should be able to identify capacity, current, resistance, initial SOC, time step, and simulation horizon immediately.

`np.arange(0.0, final_time_s + time_step_s, time_step_s)` includes the final time point. If we omitted the `+ time_step_s`, the last sample at 3600 s would be missing. That sounds minor, but end-point consistency becomes important later when comparing outputs across tools.

The SOC calculation is just Equation (1.1). We then clip with `np.clip` so the value cannot drop below zero. For this chosen current and duration, the unclipped result lands exactly at zero, but clipping is still a healthy protective pattern.

The OCV line implements Equation (1.2). Again, this is not a chemistry model. It is an intentionally transparent surrogate so that the expected output is obvious.

The terminal voltage subtracts the constant ohmic drop, which is $IR_0 = 2.0 \times 0.05 = 0.1$ V. Therefore, the terminal curve should be parallel to the OCV curve and always 0.1 V lower. If it is not, the code or sign convention is wrong.

`np.trapz(power_w, time_s) / 3600.0` numerically integrates power in watts over time in seconds, then divides by 3600 to convert joules-per-second integrated over seconds into watt-hours. This is a good early reminder that unit discipline matters everywhere.

We store results as a tidy `pandas.DataFrame` because later chapters will rely heavily on tabular outputs. Even this toy script teaches the habit that plots and machine-readable tables should both be saved.

The plot is split into two panels because SOC and voltage live on different scales and tell different stories. Good plotting is part of research competence, not decoration.

Python sanity check expected output

In the terminal, you should see:

```
Saved results table to: results/hello_battery_python.csv
Saved figure to: figures/hello_battery_python.png
Initial terminal voltage: 3.50 V
Final terminal voltage: 2.90 V
Delivered capacity: 2.00 Ah
Delivered energy: 6.40 Wh
```

The CSV should contain 3601 rows, one for each second from 0 to 3600 inclusive.

The figure should have two panels.

The top panel, SOC vs time, should be a perfectly straight descending line from 1.0 at 0 s to 0.0 at 3600 s. If it curves, steps, or ends somewhere other than zero, something is off.

The bottom panel should show two straight lines. The green OCV line should descend from 3.6 V to 3.0 V. The red terminal-voltage line should descend from 3.5 V to 2.9 V and remain exactly 0.1 V below the OCV line for the entire discharge. That constant gap is the visual signature of the ohmic term in Equation (1.3).

A wrong result often looks like one of three things: the terminal voltage is above OCV, which indicates a sign error; the final SOC is negative, which indicates missing clipping or a duration mismatch; or the final voltage is not 2.9 V, which indicates incorrect current, capacity, or resistance.

Python sanity check troubleshooting

1. The script runs but no figure appears on screen
Symptom: nothing pops up visually.
Fix: this script saves the figure to disk and closes it intentionally. Check `figures/hello_battery_python.png`.
2. `ImportError: No module named 'matplotlib' or pandas`
Symptom: imports fail even though the environment exists.
Fix: confirm the environment is activated before running the script.
3. Delivered energy is not approximately 6.40 Wh

Symptom: printed energy is substantially different.

Fix: check the unit conversion in the integration step and confirm the current is 2.0 A.

Python sanity check reflection

This Python exercise taught a complete mini-workflow: define parameters, compute derived quantities, save a table, save a figure, and print interpretable summary metrics. That pattern will return in nearly every later chapter.

1.3.27 Part B: MATLAB toolbox verification and sanity check

Before we run the MATLAB battery check, verify the products we need later.

Create `matlab/check_toolboxes.m`:

```
requiredProducts = [  
    "MATLAB"  
    "Simulink"  
    "Simscape"  
    "Simscape Battery"  
    "Control System Toolbox"  
    "Optimization Toolbox"  
];  
  
installedProducts = ver;  
installedNames = string({installedProducts.Name});  
  
fprintf("MATLAB product check\n");  
fprintf("-----\n");  
  
for k = 1:numel(requiredProducts)  
    isInstalled = any(installedNames == requiredProducts(k));  
    if isInstalled  
        statusText = "FOUND";  
    else  
        statusText = "MISSING";  
    end  
    fprintf("%-24s : %s\n", requiredProducts(k), statusText);  
end
```

Run it in MATLAB:

```
check_toolboxes
```


Expected output is a short table of FOUND and MISSING statuses. If Simscape Battery is missing, note that explicitly in your research log.

Now create matlab/hello_battery.m:

```
clear;
clc;
close all;

if ~exist("results", "dir")
    mkdir("results");
end

if ~exist("figures", "dir")
    mkdir("figures");
end

nominalCapacityAh = 2.0;
dischargeCurrentA = 2.0;
ohmicResistanceOhm = 0.05;
initialSoc = 1.0;
timeStepS = 1.0;
finalTimeS = 3600.0;

timeS = (0:timeStepS:finalTimeS)';
soc = initialSoc - (dischargeCurrentA .* timeS) ./ (3600.0 .*
    ↳ nominalCapacityAh);
soc = max(0.0, min(1.0, soc));

ocvV = 3.0 + 0.6 .* soc;
terminalVoltageV = ocvV - dischargeCurrentA .* ohmicResistanceOhm;
powerW = terminalVoltageV .* dischargeCurrentA;

deliveredEnergyWh = trapz(timeS, powerW) / 3600.0;

resultsTable = table( ...
    timeS, ...
    soc, ...
    ocvV, ...
    terminalVoltageV, ...
    repmat(dischargeCurrentA, size(timeS)), ...
    powerW, ...
    'VariableNames', ...
    {'time_s', 'soc_fraction', 'ocv_v', 'terminal_voltage_v', 'current_a',
    ↳ 'power_w'} ...
);
```

```
writetable(resultsTable, fullfile("results", "hello_battery_matlab.csv"));

figureHandle = figure("Position", [100, 100, 800, 700]);

subplot(2, 1, 1);
plot(timeS, soc, "LineWidth", 2, "Color", [0.0, 0.45, 0.74]);
grid on;
ylabel("SOC [-]");
title("Hello Battery Sanity Check (MATLAB)");
ylim([-0.05, 1.05]);

subplot(2, 1, 2);
plot(timeS, ocvV, "LineWidth", 2, "Color", [0.47, 0.67, 0.19]);
hold on;
plot(timeS, terminalVoltageV, "LineWidth", 2, "Color", [0.85, 0.33, 0.10]);
grid on;
xlabel("Time [s]");
ylabel("Voltage [V]");
legend("OCV", "Terminal voltage", "Location", "best");

exportgraphics(figureHandle, fullfile("figures", "hello_battery_matlab.png"),
↳ "Resolution", 200);

fprintf("Saved results table to: %s\n", fullfile("results",
↳ "hello_battery_matlab.csv"));
fprintf("Saved figure to: %s\n", fullfile("figures",
↳ "hello_battery_matlab.png"));
fprintf("Initial terminal voltage: %.2f V\n", terminalVoltageV(1));
fprintf("Final terminal voltage: %.2f V\n", terminalVoltageV(end));
fprintf("Delivered capacity: %.2f Ah\n", dischargeCurrentA * finalTimeS /
↳ 3600.0);
fprintf("Delivered energy: %.2f Wh\n", deliveredEnergyWh);
```

Run it in MATLAB:

```
hello_battery
```

MATLAB sanity check explanation

The structure mirrors the Python version on purpose. That parallelism is pedagogically useful. Later, when we move data between PyBaMM and MATLAB, you want the two ecosystems to feel different in syntax but aligned in workflow.

The `mkdir` guards are the MATLAB equivalent of the Python

directory-creation calls. Again, robust scripts should not assume a preexisting folder state.

The time vector is created as a column vector with `(0:timeStepS:finalTimeS)'`. The transpose matters because MATLAB tables and plotting functions often behave more predictably when vectors are column-oriented.

The SOC computation uses elementwise operators `.*` and `./`. This is one of the biggest MATLAB habits to internalize early. If you accidentally use matrix operators here, the script will either fail or behave incorrectly.

The clamp `max(0.0, min(1.0, soc))` is the MATLAB analogue of `np.clip`.

`table(...)` creates a labeled table rather than a bare matrix. As in Python, storing named columns is far better research practice than saving anonymous arrays and hoping you remember what each column meant.

`exportgraphics` is preferred over old screenshot-style save behavior because it gives cleaner, reproducible figure exports.

MATLAB sanity check expected output

The MATLAB command window should print:

```
Saved results table to: results\hello_battery_matlab.csv
Saved figure to: figures\hello_battery_matlab.png
Initial terminal voltage: 3.50 V
Final terminal voltage: 2.90 V
Delivered capacity: 2.00 Ah
Delivered energy: 6.40 Wh
```

The slash direction in file paths may differ by operating system.

The plot should match the Python result qualitatively and numerically. The top panel should be a straight-line SOC decline from 1 to 0. The bottom panel should show OCV and terminal-voltage lines with a constant 0.1 V gap.

If the MATLAB and Python outputs disagree materially, stop and resolve that now. Cross-tool agreement on simple cases is the first step toward trusting more complex models later.

MATLAB sanity check troubleshooting

1. MATLAB cannot find the script

Symptom: Undefined function or variable 'hello_battery'.

Fix: make sure the current MATLAB folder is the repository root or add the `matlab/` folder to the path.

2. **Simscape Battery** shows as missing

Symptom: the product check reports **MISSING**.

Fix: note it in your log. This does not block this chapter, but it will matter later.

3. The CSV is saved but the figure export fails

Symptom: `exportgraphics` errors.

Fix: ensure you are using **R2024b** or later; on older versions, use `saveas` as a fallback, but note the deviation in your log.

MATLAB sanity check reflection

This MATLAB exercise taught the same research pattern in a second ecosystem. That is exactly the point. Publishable battery work often spans tools. Consistent habits across tools matter more than memorizing one syntax perfectly.

1.4 Open-Ended Exercises

These exercises ask you to modify the guided work rather than simply rerun it. Try them before reading the worked solutions.

1.4.1 Exercise 1

Modify the Python `hello_battery.py` script so it runs three discharges at `0.5C`, `1C`, and `2C` for the same `2 Ah` cell, then saves one combined CSV with a `c_rate` column.

Hints:

At `0.5C`, the current should be `1 A`. At `2C`, the current should be `4 A`. You will need a loop over current values and a way to concatenate several tables together.

1.4.2 Exercise 2

Extend `src/env_report.py` so it also records the SHA-256 hash of `environment.yml`.

Hints:

Use Python's `hashlib` module. The point is to capture not only installed-package versions but also the exact environment-spec file that was intended.

1.4.3 Exercise 3

Create a second log-template generator that makes entries named YYYY-MM-DD_experiment.md instead of YYYY-MM-DD_setup.md, and add a section called Planned Figure or Table.

Hints:

Reuse `src/create_log_entry.py` rather than writing from scratch. Change only what actually differs.

What Changes for Sodium-Ion?

At this early stage, the main sodium-ion adaptation is not a different equation set. It is a different standard of bookkeeping.

Sodium-ion work often depends on sparser public data, less standardized parameter releases, and more figure digitization than mainstream lithium-ion work. That means provenance matters even more. In a lithium-ion project, you may be able to rely on a well-known public dataset and a widely reused parameter set. In a sodium-ion project, you may need to record exactly which review paper you extracted an OCV shape from, which chemistry family the cell belongs to, whether the hard-carbon behavior reflects the slope region or plateau region, and how you converted a published current density into an absolute current for your assumed cell area or capacity.

So, from the very first chapter, get into the habit of tagging artifacts with chemistry context. A filename like `ocv_curve.csv` is weak. A filename like `hard_carbon_p2_naion_ocv_digitized_2026-04-14.csv` is stronger. A log entry that says “used SIB data” is weak. One that says “used digitized hard-carbon half-cell OCV from [paper], Figure 2b, units converted from mAh g⁻¹ to SOC using author-reported reversible capacity” is much stronger.

That discipline will matter more and more as the manual progresses.

1.5 Worked Solutions to the Open-Ended Exercises

1.5.1 Solution to Exercise 1

A clean way to solve the multi-C-rate problem is to wrap the discharge logic in a function and concatenate results.

```

from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd

Path("results").mkdir(parents=True, exist_ok=True)

nominal_capacity_ah = 2.0
ohmic_resistance_ohm = 0.05
initial_soc = 1.0
time_step_s = 1.0

c_rates = [0.5, 1.0, 2.0]
all_results = []

for c_rate in c_rates:
    discharge_current_a = c_rate * nominal_capacity_ah
    final_time_s = 3600.0 / c_rate
    time_s = np.arange(0.0, final_time_s + time_step_s, time_step_s)

    soc = initial_soc - (
        discharge_current_a * time_s / (3600.0 * nominal_capacity_ah)
    )
    soc = np.clip(soc, 0.0, 1.0)

    ocv_v = 3.0 + 0.6 * soc
    terminal_voltage_v = ocv_v - discharge_current_a * ohmic_resistance_ohm

    one_case = pd.DataFrame(
        {
            "c_rate": c_rate,
            "time_s": time_s,
            "soc_fraction": soc,
            "ocv_v": ocv_v,
            "terminal_voltage_v": terminal_voltage_v,
            "current_a": discharge_current_a,
        }
    )
    all_results.append(one_case)

combined_results = pd.concat(all_results, ignore_index=True)
combined_results.to_csv("results/hello_battery_multi_rate.csv", index=False)

```

```
print(combined_results.groupby("c_rate")["terminal_voltage_v"].agg(["first",
↪ "last"]))
```

The key idea is that current scales with C-rate through $I = C_{\text{rate}}Q_{\text{nom}}$, and full-discharge time scales inversely. The expected pattern is that higher C-rate gives a larger ohmic drop and shorter discharge time. With this teaching model, the 2C case starts 0.2 V below OCV because $4 \text{ A} \times 0.05 \Omega = 0.2 \text{ V}$.

1.5.2 Solution to Exercise 2

Here is a version of the environment report with a SHA-256 hash for `environment.yml`.

```
from __future__ import annotations

import hashlib
import json
import platform
import subprocess
import sys
from datetime import datetime, timezone
from importlib.metadata import PackageNotFoundError, version
from pathlib import Path

PACKAGES_TO_REPORT = [
    "numpy",
    "scipy",
    "pandas",
    "matplotlib",
    "jupyterlab",
    "ipykernel",
]

def package_version(package_name: str) -> str:
    try:
        return version(package_name)
    except PackageNotFoundError:
        return "NOT_INSTALLED"

def sha256_of_file(file_path: Path) -> str:
    return hashlib.sha256(file_path.read_bytes()).hexdigest()

def git_commit_hash() -> str:
    try:
        return subprocess.check_output(
            ["git", "rev-parse", "HEAD"],
            text=True,
```

```

        ).strip()
    except Exception:
        return "NOT_A_GIT_REPOSITORY"
def main() -> None:
    results_dir = Path("results")
    results_dir.mkdir(parents=True, exist_ok=True)

    environment_file = Path("environment.yml")

    report = {
        "generated_utc": datetime.now(timezone.utc).isoformat(),
        "python_executable": sys.executable,
        "python_version": platform.python_version(),
        "platform": platform.platform(),
        "git_commit": git_commit_hash(),
        "environment_yaml_sha256": sha256_of_file(environment_file)
        if environment_file.exists()
        else "FILE_NOT_FOUND",
        "packages": {
            package_name: package_version(package_name)
            for package_name in PACKAGES_TO_REPORT
        },
    }

    output_path = results_dir / "environment_report.json"
    output_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(json.dumps(report, indent=2))
if __name__ == "__main__":
    main()

```

The research value here is subtle but real. Two environments can sometimes report similar installed packages while differing in the intended spec file. Hashing `environment.yml` gives you an additional integrity check.

1.5.3 Solution to Exercise 3

A minimal adaptation of the log generator is enough.

```

from __future__ import annotations

from datetime import datetime
from pathlib import Path

TEMPLATE = """# Research Log: {date}

```


Objective

Write one sentence describing today's concrete objective.

Planned Figure or Table

Describe the exact figure, table, or output artifact you intend to produce.

Environment

- Active Python environment:
- MATLAB version:
- Git commit:
- Dataset or paper in use:

Work Performed

Describe what you actually did.

Observations

Record numerical outputs, runtime notes, and figure behavior.

Decisions and Rationale

Explain why you made methodological choices.

Problems Encountered

Include exact failure modes and error messages where useful.

Next Steps

Write the next one to three actions.

"""

```
def main() -> None:
    logs_dir = Path("logs")
    logs_dir.mkdir(parents=True, exist_ok=True)

    date_string = datetime.now().strftime("%Y-%m-%d")
    output_path = logs_dir / f"{date_string}_experiment.md"

    if output_path.exists():
        print(f"Log entry already exists: {output_path}")
        return
```

```

    output_path.write_text(TEMPLATE.format(date=date_string),
    ↪ encoding="utf-8")
    print(f"Created experiment log entry: {output_path}")
if __name__ == "__main__":
    main()

```

This version is useful once the project shifts from environment setup into actual model runs and figure generation. The new Planned Figure or Table section encourages intentional work rather than random exploration.

Chapter Summary and Skill Checklist

You should now have these skills in working memory:

- create and activate an isolated Python environment
- verify package versions instead of assuming them
- register and select the correct Jupyter kernel
- verify MATLAB product availability for later battery workflows
- initialize a Git repository early
- write a research-appropriate `.gitignore`
- maintain a dated Markdown research log
- separate raw data, processed data, notebooks, reusable code, and generated outputs
- run a small battery-flavored sanity check in both Python and MATLAB
- save both machine-readable outputs and figure files

You should now be able to answer “yes” to all of the following:

- I can create a reproducible Python environment from a file rather than from memory.
- I can prove which interpreter and package versions generated an output.
- I have a repository structure that distinguishes source code from artifacts.
- I can explain why `data/raw/` and `data/processed/` must remain separate.
- I have a research-log habit that records decisions, not just successes.
- I can initialize Git, make meaningful commits, and avoid committing bulky raw data by accident.

- I can run and interpret a simple battery sanity check in both Python and MATLAB.
- I know whether my MATLAB installation includes the toolboxes needed for later chapters.

If any of those are still shaky, revisit the corresponding walkthrough now. Later chapters assume them.

Deliverable

The deliverable from your plan is a GitHub repository with a working environment, a research log template, and a passing sanity check in both Python and MATLAB, with a commit history showing at least ten meaningful commits.

Approach it as a mini research artifact, not a homework submission. Your repository should contain `environment.yml`, the scaffold scripts, the log template script, the `.gitignore`, both sanity-check scripts, the resulting CSV and PNG outputs, and a short README explaining how to rerun everything. Your Git history should tell the story of the setup in coherent steps rather than one giant dump commit.

A good completion state looks like this:

- `README.md` explains the project purpose and setup commands.
- `environment.yml` is present and matches the actual working environment.
- `results/environment_report.json` exists.
- `logs/YYYY-MM-DD_setup.md` contains real content, not placeholders.
- `results/hello_battery_python.csv` and `figures/hello_battery_python.png` exist.
- `results/hello_battery_matlab.csv` and `figures/hello_battery_matlab.png` exist.
- `git log --oneline -n 10` shows a sensible progression of work.

If you want a good final commit message for this chapter, use something like:

```
Complete chapter 1 research environment deliverable
```

Then add a final log entry summarizing what is verified, what remains uncertain, and what Chapter 2 will require.

Further Practice and Reading

Start with one or two reproducibility papers, one or two official docs, and one or two community resources you will actually revisit.

- Sandve, Nekrutenko, Taylor, and Hovig, “Ten Simple Rules for Reproducible Computational Research.” A concise paper worth internalizing early.
- Wilson et al., “Best Practices for Scientific Computing.” Older, still foundational, and directly relevant to the habits in this chapter.
- JupyterLab documentation: bookmark the user guide and kernel-management sections.
- Conda or Miniforge documentation: especially environment creation and export commands.
- MathWorks documentation for `ver`, MATLAB Projects, and later **Simscape Battery**.
- Pro Git by Scott Chacon and Ben Straub: still the cleanest Git reference for researchers.
- Community resources: Jupyter Discourse, Scientific Python Forum, MATLAB Central.

Lab Chapter 2: *Scientific Python Refresher for Battery Work* is next.

2

Scientific Python Refresher for Battery Work

This chapter is about turning general scientific Python familiarity into battery-modeling fluency. By the time you finish it, you should no longer think of NumPy, SciPy, Pandas, and Matplotlib as four separate libraries you learned once in a methods course. You should think of them as the basic hand tools with which you express battery equations, solve dynamic models, inspect time-series data, fit parameters, and generate figures that can survive the transition from notebook output to paper draft. The immediate skill goal is modest on purpose: solve a battery-flavored ODE in Python with confidence. But the research goal underneath it is larger. We are building the numerical habits you will reuse in PyBaMM, in parameter-estimation pipelines, in dataset cleaning, and in every simulation-validation loop that follows.

This chapter operationalizes material you already learned in the theory textbook, especially Textbook Chapter 10 on equivalent-circuit models and state estimation. Keep that chapter open while you work. When we write the second-order Thevenin model as a system of differential equations, we are translating the circuit intuition of that chapter into executable code. Textbook Chapter 3 should also remain close at hand, because we will be careful about units, current sign conventions, capacity normalization, and the distinction between open-circuit

voltage and terminal voltage. Later, when we talk about fitting parameters from voltage data, you should also remember the warnings from the textbook chapters on cell behavior and degradation: a model can fit a curve and still be physically misleading if you ask the data to identify parameters it cannot actually distinguish.

This chapter also builds directly on Lab Chapter 1. Chapter 1 gave you the environment and reproducibility habits that prevent your numerical work from becoming a pile of semi-runnable notebook fragments. Here we use that environment for real modeling work. We will reuse the repository layout, the logging habit, and the idea that every meaningful output should be reproducible from code and tied to a specific environment. If Chapter 1 was the lab bench, Chapter 2 is the moment we start using the instruments.

The pacing is intentional. We are not jumping into PyBaMM yet. That will happen in Chapter 3. Before we ask PyBaMM to discretize a Doyle-Fuller-Newman model, you need to be comfortable with the simpler but crucial layer underneath: arrays, state vectors, ODE solvers, least-squares fitting, time-indexed data cleaning, and plot construction. Those are not beginner topics in disguise. They are the skeleton underneath almost every publishable battery workflow. If you can write and trust a clean second-order RC model today, you will understand PyBaMM objects better in Chapter 3, design cleaner experiments in Chapter 4, fit parameters more honestly in Chapter 5, and move into MATLAB ECM work in Chapter 6 without feeling as if the math and the code live in separate universes.

There is one more reason this chapter matters for your specific goal of publishable sodium-ion research. Sodium-ion work often begins in a data-sparse environment. You will frequently be adapting lithium-ion methods, public datasets, and model structures to sodium-ion chemistry. That means you need to distinguish the durable computational skill from the chemistry-specific assumption. Solving an ODE, resampling a time series, or fitting a parameter vector are durable skills. The exact OCV curve, voltage window, and identifiability behavior are chemistry-specific. This chapter helps you separate those layers cleanly, which is exactly what you need if your long-term target is simulation-based sodium-ion work built on partial public data.

So our pattern for the chapter is simple. First, we bridge the theory of equivalent-circuit models to their numerical representation. Then we refresh the NumPy patterns that battery work uses constantly. After that, we solve a

second-order RC model with `scipy.integrate.solve_ivp`, compare solver behavior, and talk honestly about what solver choice means. Next, we shift from synthetic signals to a public experimental dataset so that Pandas and Matplotlib are grounded in real battery traces rather than toy CSV files. Then we fit a model back to noisy data using `scipy.optimize.least_squares`. Finally, because Part I of this companion must already train you in reproduction practice, we will reproduce a published time-series figure from an open paper using the paper's linked public dataset. That last exercise is where the chapter becomes more than a refresher. It becomes research practice.

2.1 Prerequisites Check

- Required software: the `sib-research` environment from Lab Chapter 1; Python 3.11; JupyterLab 4.4+; Git 2.43+
- Required Python packages: `numpy==2.3.4`, `scipy==1.16.0`, `pandas==3.0.2`, `matplotlib==3.10.8`, `jupyterlab==4.4.10`, `ipykernel==7.2.0`
- Optional but recommended: a code editor with notebook support, and a PDF viewer so you can keep the chapter and the cited paper open together
- Required textbook chapters: Textbook Chapter 3 and Textbook Chapter 10 are essential; the chapter becomes easier if Textbook Chapter 7 is also familiar conceptually
- Required prior lab chapters: Lab Chapter 1
- Estimated time: 10 to 14 hours, depending on how quickly you move through the fitting and reproduction sections

If any of these feel shaky, stop now instead of pushing forward half-prepared. If NumPy broadcasting still feels magical rather than understandable, the early sections of this chapter are especially important. If your Chapter 1 environment is not reproducible yet, fix that first. We will assume from this point onward that you can create a notebook, run a script from the repository root, and save outputs into `results/` and `figures/` without confusion.

2.2 Environment Setup

This chapter does not require a brand-new environment, but it does require confidence that the Chapter 1 environment is the one you are actually using. The most common failure mode in numerical methods chapters is not a wrong equation. It is running the right code in the wrong interpreter.

2.2.1 Activate the Chapter 1 environment

If you used `conda` in Chapter 1:

```
conda activate sib-research
```

If you used `venv` on Linux or macOS:

```
source .venv/bin/activate
```

If you used `venv` on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

2.2.2 Verify the scientific stack

Run this from the repository root:

```
python - <<'PY'
import numpy as np
import scipy
import pandas as pd
import matplotlib

print("NumPy      :", np.__version__)
print("SciPy      :", scipy.__version__)
print("Pandas     :", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
PY
```

Expected output should look like:

```
NumPy      : 2.3.4
SciPy      : 1.16.0
Pandas     : 3.0.2
Matplotlib: 3.10.8
```

Patch-version drift is not automatically a disaster, but if your versions differ

from the pinned Chapter 1 environment, record that in your research log before continuing.

2.2.3 Create a notebook and script location for this chapter

We will keep the usual pattern: exploratory work in `notebooks/`, reusable helpers in `src/`, and generated outputs in `results/` and `figures/`.

```
mkdir -p notebooks src results figures data/raw data/processed
```

Create a notebook called `notebooks/chapter_2_scientific_python_refresher.ipynb`. If you prefer to begin in a script, that is fine, but the notebook format is useful here because we will inspect intermediate arrays and plots frequently.

2.2.4 Minimal `solve_ivp` hello world

Before we solve a battery model, verify that SciPy's ODE solver is functioning.

```
import numpy as np
from scipy.integrate import solve_ivp

def exponential_decay_rhs(time_s, state):
    return [-0.5 * state[0]]

evaluation_times_s = np.linspace(0.0, 10.0, 6)

solution = solve_ivp(
    fun=exponential_decay_rhs,
    t_span=(0.0, 10.0),
    y0=[1.0],
    t_eval=evaluation_times_s,
)

print("status:", solution.status)
print("message:", solution.message)
print("y(t):", np.round(solution.y[0], 4))
```

Expected output:

```
status: 0
message: The solver successfully reached the end of the integration interval.
y(t): [1.      0.3679 0.1353 0.0498 0.0183 0.0067]
```

Those values follow the exact analytical solution $y(t) = e^{-0.5t}$. If your numbers are close to the ones above, your solver is working.

2.2.5 Launch JupyterLab and confirm the kernel

```
jupyter lab
```

Open the notebook you created and check the kernel name in the top-right corner. It should still be `Python (sib-research)` or the Chapter 1 equivalent. Then run:

```
import sys
print(sys.executable)
```

The printed path should point into your intended environment. If it points to a system interpreter, stop and fix the kernel mismatch now.

2.2.6 Common setup failures and fixes

1. `ModuleNotFoundError: No module named 'scipy'`

Symptom: imports fail inside Jupyter but succeed in the terminal, or vice versa.

Fix: you almost certainly have a kernel mismatch. Re-register the kernel from the active environment with `python -m ipykernel install --user --name sib-research --display-name "Python (sib-research)"`.

2. `ImportError` mentioning a compiled library or wheel incompatibility

Symptom: NumPy or SciPy imports fail even though installation appeared to work.

Fix: this is usually an environment inconsistency. Recreate the environment from `environment.yml` rather than trying to patch it package-by-package.

3. Plots do not display in the notebook

Symptom: code runs but no figure appears.

Fix: ensure the notebook cell actually ends with a plotting command or `plt.show()`, and verify that you did not disable the inline backend accidentally.

4. `solve_ivp` works for the hello-world example but later battery code fails with shape errors

Symptom: SciPy complains about incompatible dimensions or unpacking errors.

Fix: the most common cause is forgetting that the state passed into the right-hand side is an array, even for scalar problems. We will be very explicit about state-vector shape in the next section.

2.3 Conceptual Bridge: From the Equivalent Circuit in the Textbook to a State Vector in Python

In Textbook Chapter 10, the equivalent-circuit model was valuable because it compressed a complicated electrochemical reality into a small number of states and parameters you could reason about directly. A resistor gave you the instantaneous ohmic drop. One or more RC branches gave you delayed polarization behavior. SOC evolved through charge balance. The model was not “true” in the same way a physics-based porous-electrode model aims to be true, but it was useful because it captured the right dynamic shape at the right computational cost.

The most important translation we make in this chapter is that from circuit picture to state-space representation. In the textbook, you probably drew the second-order Thevenin model as an OCV source in series with R_0 and two parallel RC branches. In Python, that same model becomes a state vector and a right-hand-side function. If discharge current is positive, one convenient form is

$$\frac{dz}{dt} = -\frac{I(t)}{3600Q_n}, \quad (1)$$

$$\frac{dv_1}{dt} = -\frac{v_1}{R_1C_1} + \frac{I(t)}{C_1}, \quad \frac{dv_2}{dt} = -\frac{v_2}{R_2C_2} + \frac{I(t)}{C_2}, \quad (2)$$

and

$$V_t(t) = \text{OCV}(z(t)) - I(t)R_0 - v_1(t) - v_2(t). \quad (3)$$

Equation (1) says SOC is just charge balance written in differential form. Equation (2) says each RC branch stores a transient overpotential with its own time constant $\tau_i = R_iC_i$. Equation (3) says terminal voltage is open-circuit voltage minus the ohmic drop and minus the transient branch drops. Nothing here is new physically. What changes is the representation. Instead of solving the circuit by hand for one simple input, we hand the whole system to an ODE solver and let it

march through time.

This is exactly the moment where many readers conceptually split in two. One group thinks, “Good, now we are doing code.” The other thinks, “Good, now we are still doing the same battery model, just in executable form.” The second attitude is the right one. The Python objects are not a separate subject. They are the mechanism by which the same model becomes testable, sweepable, fit-able, and eventually publishable.

NumPy enters first because state-space battery work is almost never scalar work. Even if the state vector is only three elements long, the surrounding workflow is array-shaped. You evaluate OCV at hundreds of SOC points. You compare multiple current levels on the same grid. You slice discharge windows out of a long record. You compute residual vectors between measured and simulated voltage samples. Vectorization is not a performance trick added at the end. It is the natural language of the problem.

SciPy’s `solve_ivp` is the next layer. In textbook notation, the model is a set of coupled first-order ODEs. In SciPy notation, it is a Python function with signature `rhs(t, y, ...)` that returns the time derivative of the state vector. The ODE solver repeatedly calls that function, chooses time steps, and constructs a numerical approximation to the trajectory. That sounds mechanical, but several research decisions hide inside it. Which solver should you choose? How tight should the tolerances be? Is the problem stiff? Do you want the solution only at a few evaluation times or on an adaptive grid? Those questions do not disappear just because the code runs. They become part of the modeling method.

`scipy.optimize.least_squares` extends the same logic into inverse problems. In forward simulation, we supply parameters and predict voltage. In fitting, we reverse the direction: we adjust parameters so that simulated voltage resembles measured voltage as closely as possible. Mathematically, we define a residual vector such as

$$r_i(\theta) = V_{\text{sim},i}(\theta) - V_{\text{meas},i}, \quad (4)$$

and we ask the optimizer to reduce the norm of that vector over parameter vector θ . Again, the software object and the battery method are the same idea seen from two angles. A parameter fit is just a repeated forward simulation wrapped in an optimizer.

Pandas then plays a complementary role. The textbook equivalent-circuit

model is mathematically clean, but real battery datasets are not. Public files contain missing values, inconsistent headings, text legends, different sign conventions, and sometimes incomplete metadata. Pandas is the structure we use to make those data problems explicit rather than informal. A `DataFrame` is where units become column names, timestamps become indices, resampling becomes auditable, and segment labels become filter conditions. That matters because later, when you do publishable work, a large fraction of your time will not be spent solving equations. It will be spent making sure the data you feed into those equations are what you think they are.

Matplotlib is the last piece of the bridge. In theory, a plotted curve is evidence. In practice, plotting is part of analysis, not just presentation. A good plot tells you whether a solver stepped over a sharp transient, whether a fit is matching only the slow drift and missing the pulse edges, whether your sign convention is backwards, and whether one current trace is actually being plotted on the wrong axis. Publication-quality plotting begins long before publication. It begins when your diagnostic plots are precise enough that you can trust them.

So the bridge from textbook theory to scientific Python is not a change in subject. It is a change in representation. The circuit becomes a state vector, the governing equations become a right-hand-side function, the dataset becomes a `DataFrame`, the calibration becomes a least-squares problem, and the interpretation becomes a figure you can read critically. Keep that translation in mind through every exercise in this chapter. If the code ever starts feeling like disconnected syntax, come back to Equations (1) through (4) and ask which mathematical object each line is implementing.

Guided Walkthrough 1

NumPy Patterns You Will Use Constantly

Learning objective: Use vectorization, broadcasting, and slicing to evaluate battery voltage over many SOC and current combinations without writing nested Python loops.

Before we solve any ODEs, we need to refresh the array operations that battery work uses all the time. The right way to think about vectorization here is not “faster code” but “clearer statement of the physics.” If you want terminal voltage for four current levels over an SOC grid, that is a 2D array problem. Writing it as a

2D array clarifies the structure of the question.

2.3.1 Walkthrough 1 code

```
import numpy as np
import matplotlib.pyplot as plt

def teaching_ocv_from_soc(soc_fraction):
    """Simple smooth OCV curve with low-SOC and high-SOC knees."""
    soc_fraction = np.clip(np.asarray(soc_fraction), 0.0, 1.0)
    return (
        2.85
        + 0.42 * soc_fraction
        + 0.06 * np.tanh((soc_fraction - 0.10) / 0.04)
        + 0.10 * np.tanh((soc_fraction - 0.88) / 0.05)
    )

soc_grid = np.linspace(0.02, 0.98, 241)
discharge_currents_a = np.array([1.0, 2.0, 4.0, 6.0])[:, None]
r0_ohm = 0.018

ocv_grid_v = teaching_ocv_from_soc(soc_grid)
terminal_voltage_grid_v = ocv_grid_v - discharge_currents_a * r0_ohm

mid_soc_slice = slice(80, 161)
mid_soc_soc = soc_grid[mid_soc_slice]
mid_soc_voltage_grid_v = terminal_voltage_grid_v[:, mid_soc_slice]

print("soc_grid shape:", soc_grid.shape)
print("discharge_currents_a shape:", discharge_currents_a.shape)
print("terminal_voltage_grid_v shape:", terminal_voltage_grid_v.shape)
print("Voltage at 50% SOC for each current level [V]:")
soc_50_index = np.argmin(np.abs(soc_grid - 0.50))
print(np.round(terminal_voltage_grid_v[:, soc_50_index], 4))

fig, ax = plt.subplots(figsize=(8, 5), constrained_layout=True)

for current_a, voltage_curve_v in zip(discharge_currents_a[:, 0],
    ↪ terminal_voltage_grid_v):
    ax.plot(
        soc_grid,
        voltage_curve_v,
        linewidth=2.0,
```

```

        label=f"{current_a:.0f} A discharge",
    )

ax.set_xlabel("State of charge, SOC [-]")
ax.set_ylabel("Terminal voltage [V]")
ax.set_title("Broadcasted voltage calculation over SOC and discharge
↳ current")
ax.grid(True, alpha=0.3)
ax.legend()

plt.show()

```

2.3.2 Walkthrough 1 code explanation

The `teaching_ocv_from_soc` function is not intended as a chemistry-specific OCV model. It is a smooth teaching function that mimics three features you expect in a real OCV curve: a broad central region, a low-SOC knee, and a high-SOC knee. The `np.tanh` terms are especially convenient for teaching because they create smooth transitions without any piecewise discontinuities. Later chapters will replace this kind of teaching function with tabulated or experimentally fitted OCV relations, but for now the smoothness is helpful because it keeps numerical behavior easy to interpret.

`soc_grid = np.linspace(0.02, 0.98, 241)` creates a one-dimensional array of SOC values. We deliberately stop short of exactly 0 and 1 because many battery functions behave awkwardly at the hard edges of the allowable range. In publishable work, you often clip away the extreme ends for similar reasons.

`discharge_currents_a = np.array([1.0, 2.0, 4.0, 6.0])[:, None]` is the line most worth pausing on. The values start as a one-dimensional array of shape `(4,)`. The `[:, None]` adds a new axis, turning it into a column vector of shape `(4, 1)`. That one extra axis is what allows broadcasting to work cleanly against the SOC grid of shape `(241,)`.

`ocv_grid_v = teaching_ocv_from_soc(soc_grid)` returns a one-dimensional voltage array of length 241. When we write `ocv_grid_v - discharge_currents_a * r0_ohm`, NumPy broadcasts the `(241,)` OCV array against the `(4, 1)` current array and produces a `(4, 241)` terminal-voltage array. That is exactly what we want: one voltage curve for each current level, evaluated at every SOC point.

The `mid_soc_slice = slice(80, 161)` line is here because slicing is a

daily battery-analysis operation, not a beginner curiosity. You will constantly isolate windows: the middle SOC range, the first 60 seconds of a pulse, the rest period after a current step, or the final 10% of a charge. Writing a slice object once and reusing it is clearer than sprinkling index literals throughout the code.

`soc_50_index = np.argmin(np.abs(soc_grid - 0.50))` finds the grid point closest to 50% SOC. In research code you should prefer explicit selection logic like this over assuming that a nice round SOC value is exactly present in a floating-point grid.

In the plot loop, the `zip(discharge_currents_a[:, 0], terminal_voltage_grid_v)` pairing works because `discharge_currents_a[:, 0]` turns the column vector back into a flat list of current values for labeling, while `terminal_voltage_grid_v` iterates row-by-row over the broadcast result.

2.3.3 Walkthrough 1 expected output

The terminal printout should look like this in structure:

```
soc_grid shape: (241,)
discharge_currents_a shape: (4, 1)
terminal_voltage_grid_v shape: (4, 241)
Voltage at 50% SOC for each current level [V]:
[3.1131 3.0951 3.0591 3.0231]
```

The exact voltages may differ by a few last digits depending on the SOC grid point closest to 0.50, but the pattern must be monotonic: higher discharge current produces lower terminal voltage at the same SOC because the ohmic drop grows as IR_0 .

The plot should show four smooth curves of terminal voltage versus SOC. All four curves should have the same shape because only the ohmic term changes in this simple example; the higher-current curves should be vertically shifted downward by a constant amount relative to the lower-current curves. The low-SOC and high-SOC knees should be visible. A wrong result usually looks like one of three things: curves crossing when they should not, a jagged line caused by accidental integer indexing, or only one curve appearing because broadcasting collapsed incorrectly.

2.3.4 Walkthrough 1 troubleshooting

1. `ValueError` about incompatible shapes

Symptom: `operands could not be broadcast together`.

Fix: make sure the current array has shape `(n_currents, 1)` rather than `(n_currents,)`. The `[:, None]` is the key.

2. All four curves lie on top of each other

Symptom: the plot shows only one apparent curve.

Fix: verify that `r0_ohm` is not accidentally zero and that you did not overwrite `discharge_currents_a` with a scalar.

3. The voltage increases with current

Symptom: higher discharge current produces a higher terminal voltage.

Fix: your sign convention is reversed somewhere. In this chapter, discharge current is positive and subtracts voltage.

2.3.5 Walkthrough 1 reflection

This exercise taught the NumPy shape habits we will keep reusing: column vectors for parameter sweeps, one-dimensional grids for state variables, slicing for physically meaningful windows, and explicit attention to units and sign convention. If this felt almost too simple, that is good. These are the habits that make later model code readable instead of mysterious.

Guided Walkthrough 2

Solve a Second-Order RC Model with `solve_ivp`

Learning objective: Implement the second-order Thevenin model from Textbook Chapter 10 as an ODE system, solve it under a pulse-rest protocol, and compare solver behavior.

This is the chapter's core numerical exercise. We will move from an algebraic battery approximation to a dynamic model with internal states. The model is still small enough to understand line-by-line, which is exactly why it is such a good bridge to later chapters.

2.3.6 Solver choice before code

You will see three SciPy solvers in this chapter: RK45, BDF, and LSODA. Here is the practical distinction you should remember.

Solver	Numerical family	Good default use	Most common failure mode in battery work
RK45	explicit Runge-Kutta	nonstiff problems; first checks; teaching examples	many tiny steps when dynamics span very different time constants
BDF	implicit multistep	stiff or mildly stiff problems; later PyBaMM-like behavior	extra overhead when the problem is easy
LSODA	automatic stiffness switching	when you do not know if the problem is stiff	harder to diagnose solver behavior because the switching is internal

The two RC time constants in this exercise are intentionally separated enough that stiffness can begin to matter, but not so extreme that the model becomes numerically fragile.

2.3.7 Walkthrough 2 code

```
from dataclasses import dataclass
from time import perf_counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

@dataclass(frozen=True)
```

```

class TheveninParameters:
    capacity_ah: float
    r0_ohm: float
    r1_ohm: float
    c1_f: float
    r2_ohm: float
    c2_f: float

def teaching_ocv_from_soc(soc_fraction):
    soc_fraction = np.clip(np.asarray(soc_fraction), 0.0, 1.0)
    return (
        2.85
        + 0.42 * soc_fraction
        + 0.06 * np.tanh((soc_fraction - 0.10) / 0.04)
        + 0.10 * np.tanh((soc_fraction - 0.88) / 0.05)
    )

PARAMS = TheveninParameters(
    capacity_ah=3.0,
    r0_ohm=0.015,
    r1_ohm=0.010,
    c1_f=1800.0,
    r2_ohm=0.008,
    c2_f=18000.0,
)

segment_start_times_s = np.array([0, 300, 900, 1200, 1500, 1800, 2400, 3000],
    ↪ dtype=float)
segment_currents_a = np.array([0.0, 3.0, 0.0, 6.0, 0.0, 4.5, 0.0, 1.5],
    ↪ dtype=float)
simulation_end_time_s = 3600.0

def current_profile_a(time_s):
    segment_index = np.searchsorted(segment_start_times_s, time_s,
    ↪ side="right") - 1
    segment_index = np.clip(segment_index, 0, len(segment_currents_a) - 1)
    return float(segment_currents_a[segment_index])

def thevenin_rhs(time_s, state, params):
    soc_fraction, v_rc1_v, v_rc2_v = state

```

```

current_a = current_profile_a(time_s)

dsoc_dt = -current_a / (3600.0 * params.capacity_ah)
dv_rc1_dt = -(v_rc1_v / (params.r1_ohm * params.c1_f)) + current_a /
↳ params.c1_f
dv_rc2_dt = -(v_rc2_v / (params.r2_ohm * params.c2_f)) + current_a /
↳ params.c2_f

return [dsoc_dt, dv_rc1_dt, dv_rc2_dt]

def simulate_model(method_name):
    evaluation_times_s = np.linspace(0.0, simulation_end_time_s, 1201)
    initial_state = [0.95, 0.0, 0.0]

    start_clock = perf_counter()

    solution = solve_ivp(
        fun=thevenin_rhs,
        t_span=(0.0, simulation_end_time_s),
        y0=initial_state,
        t_eval=evaluation_times_s,
        method=method_name,
        args=(PARAMS,),
        rtol=1e-8,
        atol=1e-10,
    )

    elapsed_wall_time_s = perf_counter() - start_clock

    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    soc_trace = solution.y[0]
    v_rc1_trace_v = solution.y[1]
    v_rc2_trace_v = solution.y[2]
    terminal_voltage_v = (
        teaching_ocv_from_soc(soc_trace)
        - current_trace_a * PARAMS.r0_ohm
        - v_rc1_trace_v
        - v_rc2_trace_v
    )

    output = pd.DataFrame(
        {
            "time_s": solution.t,
            "current_a": current_trace_a,

```

```

        "soc_fraction": soc_trace,
        "v_rc1_v": v_rc1_trace_v,
        "v_rc2_v": v_rc2_trace_v,
        "terminal_voltage_v": terminal_voltage_v,
    }
)

diagnostics = {
    "method": method_name,
    "status": solution.status,
    "nfev": solution.nfev,
    "wall_time_s": elapsed_wall_time_s,
    "final_soc": soc_trace[-1],
    "final_voltage_v": terminal_voltage_v[-1],
}

return output, diagnostics

all_outputs = {}
all_diagnostics = []

for solver_name in ["RK45", "BDF", "LSODA"]:
    simulation_output, diagnostics = simulate_model(solver_name)
    all_outputs[solver_name] = simulation_output
    all_diagnostics.append(diagnostics)

diagnostics_table = pd.DataFrame(all_diagnostics)
print(diagnostics_table.round(6))

reference_voltage_v = all_outputs["BDF"]["terminal_voltage_v"].to_numpy()

for solver_name, simulation_output in all_outputs.items():
    max_abs_difference_v = np.max(
        np.abs(simulation_output["terminal_voltage_v"].to_numpy() -
        ↪ reference_voltage_v)
    )
    print(f"{solver_name:>5s} max |V - V_BDF| = {max_abs_difference_v:.6e}
    ↪ V")

fig, axes = plt.subplots(3, 1, figsize=(9, 9), sharex=True,
    ↪ constrained_layout=True)

```

```

for solver_name, simulation_output in all_outputs.items():
    axes[0].plot(
        simulation_output["time_s"] / 60.0,
        simulation_output["terminal_voltage_v"],
        linewidth=2.0,
        label=solver_name,
    )
    axes[1].plot(
        simulation_output["time_s"] / 60.0,
        simulation_output["soc_fraction"],
        linewidth=2.0,
        label=solver_name,
    )

bdf_output = all_outputs["BDF"]
axes[2].step(
    bdf_output["time_s"] / 60.0,
    bdf_output["current_a"],
    where="post",
    color="black",
    linewidth=1.8,
)

axes[0].set_ylabel("Terminal voltage [V]")
axes[1].set_ylabel("SOC [-]")
axes[2].set_ylabel("Current [A]")
axes[2].set_xlabel("Time [min]")

axes[0].set_title("Second-order Thevenin response under a pulse-rest
↪ protocol")
axes[0].grid(True, alpha=0.3)
axes[1].grid(True, alpha=0.3)
axes[2].grid(True, alpha=0.3)
axes[0].legend()

plt.show()

```

2.3.8 Walkthrough 2 code explanation

The `@dataclass(frozen=True)` definition is a clean way to package the model parameters. We use a dataclass here not because it is fashionable, but because it makes parameter names explicit and prevents accidental in-place mutation. In fitting work later, we will sometimes create modified parameter sets. Freezing the

dataclass helps catch unintended edits.

The current profile is represented by `segment_start_times_s` and `segment_currents_a`. This is a standard pattern for piecewise-constant battery protocols. Each entry in `segment_currents_a` applies from its start time until the next listed start time. The sequence here is rest, 3 A pulse, rest, 6 A pulse, rest, 4.5 A pulse, rest, then a gentle 1.5 A discharge to the end. That protocol is pedagogically useful because it excites both fast and slow RC dynamics.

`current_profile_a` uses `np.searchsorted` to determine which time segment contains the current solver time. The crucial detail is `side="right"` combined with subtracting one. That means a time exactly equal to a segment boundary belongs to the new segment, which is what we want for a right-continuous step protocol.

Inside `thevenin_rhs`, the state vector is unpacked into `soc_fraction`, `v_rc1_v`, and `v_rc2_v`. This is the code representation of Equations (1) and (2). Notice how compact it becomes once the physics is already understood. The SOC equation divides current by `3600 * capacity_ah` because one amp-hour is 3600 coulombs. That factor is easy to forget, and forgetting it is one of the fastest ways to get obviously wrong SOC trajectories.

The RC branch equations each have a decay term and a forcing term. The decay term is $-v_{rc} / (R C)$, which tells the branch overpotential to relax to zero in the absence of current. The forcing term is I / C , which drives the branch state during current flow.

`simulate_model` exists so we can rerun the same model cleanly with different solver methods. Reusable wrappers like this are important. If the solver configuration is embedded inline in three separate notebook cells, comparison becomes much harder to trust.

The solver tolerances `rtol=1e-8` and `atol=1e-10` are tighter than you would need for a quick exploratory sketch, but that is intentional. We want solver-to-solver differences to reflect the method, not a loose tolerance choice. Later, when computational cost matters more, you will often loosen these tolerances.

We compute terminal voltage after solving rather than treating it as an additional differential state. That is correct because voltage here is an algebraic output of the state, not a state that needs its own ODE.

The `diagnostics_table` prints `status`, `nfev`, wall-clock time, final SOC, and final voltage. In model development, diagnostics like this should be routine.

Otherwise you can switch solvers and feel productive without actually learning anything about the numerical behavior.

2.3.9 Walkthrough 2 expected output

The diagnostics table should have three rows, one per solver. The exact wall times depend on your machine, but the qualitative pattern should look like this:

	method	status	nfev	wall_time_s	final_soc	final_voltage_v
0	RK45	0	...	...	0.400...	2.98...
1	BDF	0	...	...	0.400...	2.98...
2	LSODA	0	...	...	0.400...	2.98...

All three solvers should finish successfully with `status = 0`. The final SOC should be about 0.40 because the total discharged ampere-hours in the profile are substantial but not enough to empty the 3 Ah cell. The final terminal voltage should be just under 3.0 V for the teaching OCV curve and parameter set used here.

The printed maximum voltage differences relative to BDF should usually be small, often in the microvolt-to-submillivolt range for this problem when using tight tolerances. If you see differences of several tens of millivolts, something is wrong.

The figure should have three stacked panels. The top panel shows terminal voltage, with visible downward steps at current pulses and smooth recovery during rest intervals. The middle panel shows SOC declining only during discharge segments and flattening during rest. The bottom panel shows the pulse-rest current schedule as a step plot. A correct voltage plot has both instantaneous drops and slower relaxations. If you see only sharp jumps with no recovery curvature, you have probably lost the RC dynamics. If you see SOC changing during rest, your current profile function is wrong.

2.3.10 Walkthrough 2 troubleshooting

1. SOC goes negative or above 1
Symptom: the SOC trace leaves the physically valid interval.
Fix: check the current sign convention and the capacity conversion factor of 3600.
2. Voltage rises during discharge pulses
Symptom: applying positive discharge current makes the terminal voltage jump upward.

Fix: the sign on the ohmic or RC terms is reversed.

- 3. The solver raises a dimension or unpacking error
Symptom: messages about too many or too few values to unpack.
Fix: verify that `y0` has three entries and the right-hand side returns three derivatives.
- 4. RK45 appears much slower than the others
Symptom: large `nfev` or noticeably longer runtime.
Fix: that can be real. The separated RC time constants make the problem mildly stiff. Record the observation rather than assuming it is a bug.

2.3.11 Walkthrough 2 reflection

This exercise taught the basic forward-model pattern we will reuse all through the manual: define parameters, define an input protocol, write a right-hand side, solve the ODE, compute outputs, and inspect diagnostics. Chapter 5 will wrap this same pattern inside an optimizer. Chapter 6 will rebuild a similar model in MATLAB. Chapter 3 will replace this hand-built ODE with a much larger PyBaMM model, but the logic will still feel familiar if you understand this exercise well.

Dataset Integration

A Public Battery Time-Series File

This chapter’s real-data anchor is the public dataset released alongside Ana Foles and co-authors’ open paper on lithium-ion battery pack modeling for stationary energy management. We will use it for Pandas practice and for the Part I reproduction exercise.

Item	Value
Dataset	Lithium_Ion_Battery_Testing_Data.csv
Landing page	https://doi.org/10.5281/zenodo.5196334
Direct download	https://zenodo.org/records/5196334/files
Size	about 2.9 MB
Format	CSV
License	CC BY 4.0

Item	Value
Paper using the data	Foles et al., <i>Open Research Europe</i> (2022), DOI 10.12688/openreseurope.14301.2

The paper describes the file as containing charge and discharge acquisition data at constant power levels, including current, voltage, SOC, ambient temperature, and a final legend column named BI that explains the segment or operating condition. This is a good teaching dataset because it is small enough to inspect comfortably but real enough to remind you that public battery data never arrive in the exact shape your analysis wants.

Three common pitfalls matter before we touch the code.

First, current sign convention is never to be assumed. Some datasets treat discharge current as positive, some as negative, and some switch conventions between instruments. You should not infer the sign convention from wishful thinking. You should infer it from how voltage and SOC move together.

Second, time columns are often messy. Some files store elapsed seconds, some absolute timestamps, some MATLAB datenums, and some only preserve row order. Our parsing code will therefore detect time heuristically rather than assuming one exact header.

Third, text metadata matter. The BI legend column may look ancillary, but it is the column that lets us isolate constant-power segments for plotting and reproduction. Treat text columns in battery datasets as part of the method, not as decoration.

Guided Walkthrough 3

Parse, Clean, and Plot a Public Battery Dataset with Pandas and Matplotlib

Learning objective: Download a real battery CSV, normalize its column names, infer a usable time axis, resample a trace, and produce a publication-quality diagnostic plot.

We are now leaving the world of fully controlled synthetic signals. That means the code will get longer, but the length is doing real work. Every explicit cleaning step

is a future argument you will not need to have with yourself about what the data actually mean.

2.3.12 Walkthrough 3 code

```
from pathlib import Path
from urllib.request import urlretrieve

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA_URL = (
    "https://zenodo.org/records/5196334/files/"
    "Lithium_Ion_Battery_Testing_Data.csv?download=1"
)
RAW_PATH = Path("data/raw/Lithium_Ion_Battery_Testing_Data.csv")

def canonicalize_text(text):
    text = str(text).strip().lower()
    replacements = {
        " ": "_",
        "-": "_",
        "/": "_per_",
        "(": "_",
        ")": "_",
        "[": "_",
        "]": "_",
        "%": "pct",
    }
    for old, new in replacements.items():
        text = text.replace(old, new)
    while "__" in text:
        text = text.replace("__", "_")
    return text.strip("_")

def download_if_needed():
    RAW_PATH.parent.mkdir(parents=True, exist_ok=True)
    if not RAW_PATH.exists():
        print(f"Downloading dataset to {RAW_PATH}")
        urlretrieve(DATA_URL, RAW_PATH)
```

```

else:
    print(f"Using existing file at {RAW_PATH}")

def normalize_columns(dataframe):
    dataframe = dataframe.copy()
    dataframe.columns = [canonicalize_text(column) for column in
↪ dataframe.columns]

    rename_map = {}
    for column in dataframe.columns:
        if column == "bi":
            rename_map[column] = "segment_label"
        elif "current" in column and "charge" not in column:
            rename_map[column] = "current_a"
        elif "voltage" in column and "charge" not in column:
            rename_map[column] = "voltage_v"
        elif "soc" in column:
            rename_map[column] = "soc_pct"
        elif column in {"ta", "ambient_temperature",
↪ "ambient_temperature_c"}:
            rename_map[column] = "ambient_temperature_c"
        elif "time" in column and "temperature" not in column:
            rename_map[column] = "time_raw"

    return dataframe.rename(columns=rename_map)

def build_elapsed_time_seconds(dataframe):
    dataframe = dataframe.copy()

    if "time_raw" not in dataframe.columns:
        dataframe["elapsed_s"] = np.arange(len(dataframe), dtype=float)
        time_origin_description = "No explicit time column detected; using
↪ row index as pseudo-time."
        return dataframe, time_origin_description

    numeric_time = pd.to_numeric(dataframe["time_raw"], errors="coerce")
    if numeric_time.notna().mean() > 0.95:
        dataframe["elapsed_s"] = numeric_time - numeric_time.iloc[0]
        time_origin_description = "Detected numeric time column and shifted
↪ it to start at zero."
        return dataframe, time_origin_description

    datetime_time = pd.to_datetime(dataframe["time_raw"], errors="coerce")

```

```

    if datetime_time.notna().mean() > 0.95:
        dataframe["elapsed_s"] = (datetime_time -
↳ datetime_time.iloc[0]).dt.total_seconds()
        time_origin_description = "Detected datetime-like time column and
↳ converted to elapsed seconds."
        return dataframe, time_origin_description

    dataframe["elapsed_s"] = np.arange(len(dataframe), dtype=float)
    time_origin_description = (
        "Time column could not be parsed reliably; using row index as
↳ pseudo-time."
    )
    return dataframe, time_origin_description

download_if_needed()
raw_df = pd.read_csv(RAW_PATH)
clean_df = normalize_columns(raw_df)
clean_df, time_note = build_elapsed_time_seconds(clean_df)

for column in ["current_a", "voltage_v", "soc_pct", "ambient_temperature_c",
↳ "elapsed_s"]:
    if column in clean_df.columns:
        clean_df[column] = pd.to_numeric(clean_df[column], errors="coerce")

clean_df = clean_df.dropna(subset=["current_a", "voltage_v",
↳ "elapsed_s"]).copy()
clean_df = clean_df.sort_values("elapsed_s")
clean_df = clean_df.drop_duplicates(subset=["elapsed_s"])

time_index = pd.to_timedelta(clean_df["elapsed_s"], unit="s")
time_indexed_df = clean_df.set_index(time_index)

resampled_df = (
    time_indexed_df[["current_a", "voltage_v"]]
    .resample("10s")
    .mean()
    .interpolate(method="time")
)

print("Normalized columns:")
print(clean_df.columns.tolist())
print()
print("Time parsing note:")
print(time_note)

```

```

print()
print("First five cleaned rows:")
print(clean_df.head())
print()
print("Missing-value fraction after cleaning:")
print(clean_df[["current_a", "voltage_v"]].isna().mean())

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(
    resampled_df.index.total_seconds() / 3600.0,
    resampled_df["voltage_v"],
    color="tab:blue",
    linewidth=1.5,
)
axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("Resampled public battery dataset trace")
axes[0].grid(True, alpha=0.3)

axes[1].plot(
    resampled_df.index.total_seconds() / 3600.0,
    resampled_df["current_a"],
    color="tab:orange",
    linewidth=1.5,
)
axes[1].set_xlabel("Elapsed time [h]")
axes[1].set_ylabel("Current [A]")
axes[1].grid(True, alpha=0.3)

plt.show()

```

2.3.13 Walkthrough 3 code explanation

The `download_if_needed` helper exists because data acquisition should be explicit and idempotent. If the file is already present in `data/raw/`, the code reuses it. If not, it downloads it and stores it exactly as obtained. This is the same raw-versus-processed discipline introduced in Chapter 1.

`canonicalize_text` is a small but important utility. Public CSV files do not agree on whether a column will be named `Current`, `current`, `Current (A)`, `Current[A]`, or `current-a`. Normalizing once at the beginning is much safer than writing fragile downstream code that depends on one exact spelling.

`normalize_columns` then maps semantically similar names to a canonical set we actually want to work with: `current_a`, `voltage_v`, `soc_pct`, `ambient_temperature_c`, `segment_label`, and `time_raw`. This is one of the most reusable patterns in the whole manual. Later chapters will do the same thing for CALCE, NASA, Oxford, and other battery datasets that come with their own naming conventions.

`build_elapsed_time_seconds` is deliberately defensive. It first checks whether a time-like column exists. If not, it builds a pseudo-time from row order. If a time column exists, it tries numeric parsing first and datetime parsing second. This is realistic battery-data work. Very often you do not know ahead of time which representation a public file uses.

The numeric coercions with `errors="coerce"` convert malformed strings to NaN rather than raising an error immediately. This is usually the right first move in dataset cleaning because it lets you inspect how much of the file is affected before deciding what to drop.

`drop_duplicates(subset=["elapsed_s"])` prevents resampling from failing on repeated timestamps. Duplicate time values are common in exported instrument data and almost always need attention.

The resampling step uses 10s bins and then `interpolate(method="time")`. This is a standard teaching choice. It produces a visually smoother trace and gives you a uniformly sampled signal for downstream work. It is also a methodological choice you should record in a real study, because resampling can alter fast transients if done carelessly.

The plotting choices deserve comment too. We use a two-panel figure rather than a dual-axis figure because current and voltage deserve their own y-axes and it is easier to read alignment across time that way. The colors are conventional and high-contrast. Grid lines are faint because they should support reading, not dominate the plot.

2.3.14 Walkthrough 3 expected output

The first printed block should list normalized column names that include at least `current_a`, `voltage_v`, and `elapsed_s`. If the file headings match the paper description closely, you should also see `soc_pct`, `ambient_temperature_c`, and `segment_label`.

The time-parsing note will tell you which branch of the parser was used. If an explicit time column is present and parseable, you should see a message about

shifting numeric time to start at zero or converting datetimes to elapsed seconds. If not, the parser will tell you it fell back to pseudo-time based on row order. That fallback is acceptable for quick visualization but should be logged as a limitation.

The `head()` output should show a tidy table rather than a messy raw import. Current and voltage should be numeric. If the dataset includes SOC, it should look like a percentage-like quantity rather than random text. Missing-value fractions for current and voltage should be zero after cleaning.

The figure should show a long voltage trace in the upper panel and a corresponding current trace in the lower panel. The current trace should have blocks corresponding to constant-power charging or discharging segments and near-zero intervals if rest periods are present. A wrong result usually reveals itself immediately: flat zero current everywhere, voltage values clearly outside plausible pack ranges, or a plot compressed into a vertical line because the time axis was parsed incorrectly.

2.3.15 Walkthrough 3 troubleshooting

1. The file downloads but `pd.read_csv` fails with an encoding or delimiter issue
Symptom: a single giant column or unreadable characters.
Fix: inspect the raw file manually. If needed, pass an explicit delimiter or encoding. For this dataset the default comma-separated parse should work.
2. The time axis is nonsense
Symptom: elapsed time jumps backward or spans an absurd range.
Fix: print the raw time column before conversion. The file may store absolute time, elapsed time, or a format you need to parse differently.
3. Current and voltage columns are empty after coercion
Symptom: NaN fills the supposedly numeric columns.
Fix: the header mapping likely missed the correct source columns. Print `raw_df.columns.tolist()` and update the mapping logic.
4. Resampling smooths away important transitions
Symptom: sharp steps look suspiciously rounded.
Fix: reduce the resample interval or plot the raw cleaned signal alongside the resampled one. Resampling is a choice, not a law.

2.3.16 Walkthrough 3 reflection

This exercise taught the battery-data workflow that documentation often skips: download, normalize, coerce, time-align, resample, and only then plot. We will keep this exact posture throughout the manual. In publishable work, careful parsing is not “preprocessing before the real research.” It is part of the research.

Guided Walkthrough 4

Fit a Second-Order RC Model to Synthetic Noisy Data

Learning objective: Generate synthetic voltage data from a known RC model, add realistic measurement noise, and recover the model parameters with `scipy.optimize.least_squares`.

This is the moment where forward simulation turns into inverse modeling. Parameter fitting is where many otherwise clean numerical workflows become fragile, because the optimizer will happily chase numerical artifacts, bad initial guesses, or structurally unidentifiable parameter combinations if you let it. We will therefore keep the problem controlled: known OCV function, known current profile, moderate noise, bounded parameters, and a model class that matches the data-generating model.

2.3.17 Walkthrough 4 code

```
from dataclasses import dataclass

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares

@dataclass(frozen=True)
class TheveninParameters:
    capacity_ah: float
    r0_ohm: float
    r1_ohm: float
    c1_f: float
    r2_ohm: float
```

```

c2_f: float

def teaching_ocv_from_soc(soc_fraction):
    soc_fraction = np.clip(np.asarray(soc_fraction), 0.0, 1.0)
    return (
        2.85
        + 0.42 * soc_fraction
        + 0.06 * np.tanh((soc_fraction - 0.10) / 0.04)
        + 0.10 * np.tanh((soc_fraction - 0.88) / 0.05)
    )

TRUE_PARAMS = TheveninParameters(
    capacity_ah=3.2,
    r0_ohm=0.012,
    r1_ohm=0.009,
    c1_f=2200.0,
    r2_ohm=0.006,
    c2_f=22000.0,
)

profile_start_times_s = np.array([0, 200, 500, 800, 1100, 1400, 1800, 2200],
    ↪ dtype=float)
profile_currents_a = np.array([0.0, 2.5, 0.0, 5.0, 0.0, 3.5, 0.0, 1.5],
    ↪ dtype=float)
end_time_s = 2600.0
evaluation_times_s = np.linspace(0.0, end_time_s, 900)
initial_soc_fraction = 0.92

def current_profile_a(time_s):
    index = np.searchsorted(profile_start_times_s, time_s, side="right") - 1
    index = np.clip(index, 0, len(profile_currents_a) - 1)
    return float(profile_currents_a[index])

def simulate_thevenin(parameters):
    def rhs(time_s, state):
        soc_fraction, v_rc1_v, v_rc2_v = state
        current_a = current_profile_a(time_s)

        dsoc_dt = -current_a / (3600.0 * parameters.capacity_ah)
        dv_rc1_dt = -(v_rc1_v / (parameters.r1_ohm * parameters.c1_f)) +
        ↪ current_a / parameters.c1_f

```

```

        dv_rc2_dt = -(v_rc2_v / (parameters.r2_ohm * parameters.c2_f)) +
↪ current_a / parameters.c2_f
        return [dsoc_dt, dv_rc1_dt, dv_rc2_dt]

solution = solve_ivp(
    fun=rhs,
    t_span=(0.0, end_time_s),
    y0=[initial_soc_fraction, 0.0, 0.0],
    t_eval=evaluation_times_s,
    method="BDF",
    rtol=1e-8,
    atol=1e-10,
)

current_trace_a = np.array([current_profile_a(t) for t in solution.t])
voltage_trace_v = (
    teaching_ocv_from_soc(solution.y[0])
    - current_trace_a * parameters.r0_ohm
    - solution.y[1]
    - solution.y[2]
)

return solution.t, current_trace_a, solution.y[0], voltage_trace_v

rng = np.random.default_rng(seed=42)
time_s, current_a, soc_fraction, clean_voltage_v =
↪ simulate_thevenin(TRUE_PARAMS)
measured_voltage_v = clean_voltage_v + rng.normal(loc=0.0, scale=0.003,
↪ size=clean_voltage_v.shape)

def parameter_vector_to_dataclass(parameter_vector):
    return TheveninParameters(
        capacity_ah=3.2,
        r0_ohm=parameter_vector[0],
        r1_ohm=parameter_vector[1],
        c1_f=parameter_vector[2],
        r2_ohm=parameter_vector[3],
        c2_f=parameter_vector[4],
    )

def residuals(parameter_vector):
    parameters = parameter_vector_to_dataclass(parameter_vector)

```

```

_, _, _, simulated_voltage_v = simulate_thevenin(parameters)
return simulated_voltage_v - measured_voltage_v

initial_guess = np.array([0.020, 0.015, 1200.0, 0.012, 12000.0])
lower_bounds = np.array([0.001, 0.001, 200.0, 0.001, 2000.0])
upper_bounds = np.array([0.050, 0.050, 20000.0, 0.050, 80000.0])

fit_result = least_squares(
    fun=residuals,
    x0=initial_guess,
    bounds=(lower_bounds, upper_bounds),
    method="trf",
    verbose=1,
)

fitted_params = parameter_vector_to_dataclass(fit_result.x)
_, _, _, fitted_voltage_v = simulate_thevenin(fitted_params)

comparison_table = pd.DataFrame(
    {
        "parameter": ["R0 [ohm]", "R1 [ohm]", "C1 [F]", "R2 [ohm]", "C2
        ↪ [F]"],
        "true_value": [
            TRUE_PARAMS.r0_ohm,
            TRUE_PARAMS.r1_ohm,
            TRUE_PARAMS.c1_f,
            TRUE_PARAMS.r2_ohm,
            TRUE_PARAMS.c2_f,
        ],
        "initial_guess": initial_guess,
        "fitted_value": fit_result.x,
    }
)

comparison_table["relative_error_pct"] = (
    100.0
    * (comparison_table["fitted_value"] - comparison_table["true_value"])
    / comparison_table["true_value"]
)

rmse_v = np.sqrt(np.mean((fitted_voltage_v - measured_voltage_v) ** 2))

print(comparison_table.round(4))
print()
print(f"Optimization success: {fit_result.success}")

```

```

print(f"Optimizer message      : {fit_result.message}")
print(f"Voltage RMSE          : {rmse_v:.6f} V")

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(time_s / 60.0, measured_voltage_v, label="Synthetic
    ↪ measurement", linewidth=1.2)
axes[0].plot(time_s / 60.0, fitted_voltage_v, label="Fitted model",
    ↪ linewidth=2.0)
axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("Parameter identification for a second-order RC model")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

axes[1].plot(time_s / 60.0, measured_voltage_v - fitted_voltage_v,
    ↪ color="tab:red", linewidth=1.0)
axes[1].axhline(0.0, color="black", linestyle="--", linewidth=1.0)
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Residual [V]")
axes[1].grid(True, alpha=0.3)

plt.show()

```

2.3.18 Walkthrough 4 code explanation

The first half of the script is a forward simulator very similar to Walkthrough 2. That is deliberate. Parameter estimation should not require an entirely separate mental model. It should feel like “the same simulator, now embedded in an optimizer.”

TRUE_PARAMS defines the hidden ground truth we want the optimizer to recover. We keep capacity fixed during fitting because one of the key lessons of parameter estimation is restraint. The more parameters you let float, the easier it becomes for the optimizer to trade one physical effect against another. In real experiments, fitting Q_n , R_0 , R_1 , C_1 , R_2 , and C_2 all at once from one voltage trace is rarely a good first move.

The current profile includes multiple pulse amplitudes and rest intervals because identifiability improves when the excitation contains both fast and slow dynamic content. A constant-current trace alone often does not give enough information to separate two RC branches cleanly.

`rng = np.random.default_rng(seed=42)` makes the noisy data reproducible. That matters because otherwise every run would create a slightly different estimation problem, which makes learning harder and logging results messier.

The residual function returns `simulated_voltage_v - measured_voltage_v`, not the squared residuals. This is an important SciPy convention. `least_squares` wants the residual vector itself and takes care of the objective function internally.

The parameter bounds are not arbitrary. They encode physical plausibility and also stabilize the optimization numerically. Without bounds, the optimizer can temporarily explore absurd capacitance values or nearly zero resistances that make the problem harder to solve robustly. In real research, every bound should be justified in your notes or methods section.

The optimizer method `trf` is SciPy's trust-region reflective algorithm, a sensible default for bounded least-squares problems. The `verbose=1` option is pedagogically useful because it reminds you that fitting is an iterative process, not magic.

The residual plot is as important as the parameter table. A fit can have a low RMSE and still miss the pulse edges systematically. Looking at the residual as a function of time is how you catch that.

2.3.19 Walkthrough 4 expected output

The comparison table should list the true values, the deliberately imperfect initial guesses, and the fitted values. Because the data are generated by the same model class being fitted, the recovered parameters should usually land close to the truth. With the noise level used here, you should expect R_0 to be recovered very accurately and the RC branch parameters to come back within a few to perhaps ten percent, depending on the random draw and solver details.

The script should print `Optimization success: True` and an optimizer message indicating termination because the gradient or cost reduction criterion was met. The voltage RMSE should be on the order of the noise level, roughly a few millivolts.

The upper plot should show the noisy synthetic measurement and the fitted model lying almost on top of each other. You should still be able to see small differences near the sharpest transitions because noise and parameter coupling make exact overlap unnecessary. The lower residual plot should oscillate around

zero without long obvious drifts. A wrong result often looks like a good match in the flat parts and a bad mismatch at every pulse edge, which is a sign the branch time constants are wrong even if the global RMSE seems acceptable.

2.3.20 Walkthrough 4 troubleshooting

1. The optimizer converges but the parameters are unphysical
Symptom: one resistance hits the upper bound and one capacitance hits the lower bound.
Fix: your initial guess or excitation profile may not be informative enough.
Try a richer pulse sequence or fewer free parameters.
2. The fit fails with a solver error inside `least_squares`
Symptom: SciPy complains during some candidate evaluations.
Fix: tighten bounds so the optimizer does not explore numerically awkward regions.
3. RMSE is low but the residual plot has a visible structure
Symptom: the residual swings systematically at each pulse.
Fix: do not trust RMSE alone. This usually means the model class or fitted time constants are still wrong.
4. Parameter estimates vary every time you rerun the script
Symptom: different results each run.
Fix: set the random seed, as we did with `default_rng(seed=42)`.

2.3.21 Walkthrough 4 reflection

This exercise taught the core idea of model calibration: a forward simulator wrapped in an optimizer. In Chapter 5 we will bring this idea into PyBaMM and discuss identifiability more seriously. For now, the important lesson is that fitting is not only about obtaining numbers. It is about checking whether the experiment, the model structure, and the residual pattern justify believing those numbers.

Guided Walkthrough 5

Reproduction Exercise - Recreate Figure 10 from Foles et al. (2022)

Learning objective: Reproduce a published time-series figure from a public battery dataset, document the ambiguities, and decide what counts as “close enough.”

This is the highest-value exercise in Part I because it forces all the earlier pieces to work together: parsing a public dataset, interpreting metadata, choosing a plotting strategy, and being explicit about what the paper does and does not specify. The target is Figure 10 from:

Ana Foles, Luís Fialho, Pedro Horta, and Manuel Collares-Pereira, “Validation of a lithium-ion commercial battery pack model using experimental data for stationary energy management application,” *Open Research Europe*, 2022, DOI [10.12688/openreseurope.14301.2](https://doi.org/10.12688/openreseurope.14301.2).

The paper describes Figure 10 as “Lithium-ion battery voltage and current data from the experimental test plan, for complete charge-discharge cycles, at different constant power levels (due to readiness, only few power levels are represented).” The underlying dataset is the Zenodo CSV we used in Walkthrough 3. The figure caption does not list the exact selected power levels, which is the main ambiguity we need to handle honestly.

2.3.22 Our reproduction choices

We need to make three explicit choices.

First, because the caption says only “a few power levels are represented,” we will select three representative power levels directly from the `segment_label` text in the dataset: a low, medium, and high power level among those that appear frequently enough to plot cleanly.

Second, because public CSV exports can segment cycles unevenly, we will detect contiguous groups of identical `segment_label` values and then keep one representative contiguous segment for each selected power level.

Third, because exact figure aesthetics are rarely fully specified, we will aim to reproduce the figure’s informational content rather than every cosmetic choice. “Close enough” here means the resulting figure should show complete charge-discharge voltage and current traces for multiple power levels, with the expected ordering in duration and magnitude: lower power produces longer cycles, higher power produces shorter cycles and larger current magnitude.

2.3.23 Walkthrough 5 code

```
from pathlib import Path
from urllib.request import urlretrieve
```



```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA_URL = (
    "https://zenodo.org/records/5196334/files/"
    "Lithium_Ion_Battery_Testing_Data.csv?download=1"
)
RAW_PATH = Path("data/raw/Lithium_Ion_Battery_Testing_Data.csv")

def canonicalize_text(text):
    text = str(text).strip().lower()
    for old, new in {
        " ": "_",
        "-": "_",
        "/": "_per_",
        "(": "",
        ")": "",
        "[": "",
        "]": "",
        "%": "pct",
    }.items():
        text = text.replace(old, new)
    while "__" in text:
        text = text.replace("__", "_")
    return text.strip("_")

def load_foles_dataset():
    RAW_PATH.parent.mkdir(parents=True, exist_ok=True)
    if not RAW_PATH.exists():
        urlretrieve(DATA_URL, RAW_PATH)

    dataframe = pd.read_csv(RAW_PATH)
    dataframe.columns = [canonicalize_text(column) for column in
↪ dataframe.columns]

    rename_map = {}
    for column in dataframe.columns:
        if column == "bi":
            rename_map[column] = "segment_label"
        elif "current" in column and "charge" not in column:

```

```

        rename_map[column] = "current_a"
    elif "voltage" in column and "charge" not in column:
        rename_map[column] = "voltage_v"
    elif "soc" in column:
        rename_map[column] = "soc_pct"
    elif column in {"ta", "ambient_temperature",
        ↪ "ambient_temperature_c"}:
        rename_map[column] = "ambient_temperature_c"
    elif "time" in column and "temperature" not in column:
        rename_map[column] = "time_raw"

dataframe = dataframe.rename(columns=rename_map)

for column in ["current_a", "voltage_v", "soc_pct"]:
    if column in dataframe.columns:
        dataframe[column] = pd.to_numeric(dataframe[column],
        ↪ errors="coerce")

    if "time_raw" in dataframe.columns:
        numeric_time = pd.to_numeric(dataframe["time_raw"], errors="coerce")
        if numeric_time.notna().mean() > 0.95:
            dataframe["elapsed_s"] = numeric_time - numeric_time.iloc[0]
        else:
            datetime_time = pd.to_datetime(dataframe["time_raw"],
            ↪ errors="coerce")
            if datetime_time.notna().mean() > 0.95:
                dataframe["elapsed_s"] = (datetime_time -
            ↪ datetime_time.iloc[0]).dt.total_seconds()
            else:
                dataframe["elapsed_s"] = np.arange(len(dataframe),
            ↪ dtype=float)
        else:
            dataframe["elapsed_s"] = np.arange(len(dataframe), dtype=float)

    dataframe["segment_label"] = dataframe.get("segment_label", pd.Series("",
    ↪ index=dataframe.index)).fillna("")
    dataframe = dataframe.dropna(subset=["current_a", "voltage_v",
    ↪ "elapsed_s"]).copy()
    dataframe = dataframe.sort_values("elapsed_s").reset_index(drop=True)

    return dataframe

df = load_foles_dataset()

```

```

df["segment_id"] = (df["segment_label"] !=
    ↪ df["segment_label"].shift(fill_value="")).cumsum()
df["power_level_w"] = (
    df["segment_label"]
    .astype(str)
    .str.extract(r"(\d{3,4})", expand=False)
)
df["power_level_w"] = pd.to_numeric(df["power_level_w"], errors="coerce")

segment_summary = (
    df.groupby("segment_id")
    .agg(
        segment_label=("segment_label", "first"),
        power_level_w=("power_level_w", "first"),
        start_time_s=("elapsed_s", "first"),
        end_time_s=("elapsed_s", "last"),
        n_rows=("elapsed_s", "size"),
        mean_abs_current_a=("current_a", lambda x: np.mean(np.abs(x))),
    )
    .reset_index()
)

segment_summary["duration_s"] = segment_summary["end_time_s"] -
    ↪ segment_summary["start_time_s"]

usable_segments = segment_summary.dropna(subset=["power_level_w"]).copy()
usable_segments = usable_segments[usable_segments["duration_s"] > 0]
usable_segments = usable_segments.sort_values(["power_level_w",
    ↪ "duration_s"], ascending=[True, False])

available_levels = np.sort(usable_segments["power_level_w"].unique())
if len(available_levels) < 3:
    raise RuntimeError("Need at least three distinguishable power levels for
    ↪ the reproduction.")

selected_levels = np.array([
    available_levels[0],
    available_levels[len(available_levels) // 2],
    available_levels[-1],
])

selected_segment_rows = []
for level in selected_levels:
    representative_segment = usable_segments[usable_segments["power_level_w"]
    ↪ == level].iloc[0]
    selected_segment_rows.append(representative_segment)

```

```

selected_segments = pd.DataFrame(selected_segment_rows)
print("Selected representative power levels [W]:", selected_levels.tolist())
print(selected_segments[["segment_label", "power_level_w", "duration_s",
↳ "n_rows"]])

plot_frames = []
for _, row in selected_segments.iterrows():
    segment_df = df[df["segment_id"] == row["segment_id"]].copy()
    segment_df["segment_elapsed_h"] = (segment_df["elapsed_s"] -
↳ segment_df["elapsed_s"].iloc[0]) / 3600.0
    segment_df["power_level_w"] = row["power_level_w"]
    plot_frames.append(segment_df)

plot_df = pd.concat(plot_frames, ignore_index=True)

fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex=True,
↳ constrained_layout=True)

for level, one_level_df in plot_df.groupby("power_level_w", sort=True):
    axes[0].plot(
        one_level_df["segment_elapsed_h"],
        one_level_df["voltage_v"],
        linewidth=2.0,
        label=f"{int(level)} W",
    )
    axes[1].plot(
        one_level_df["segment_elapsed_h"],
        one_level_df["current_a"],
        linewidth=2.0,
        label=f"{int(level)} W",
    )

axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("Reproduction of Foles et al. (2022) Figure 10 using the
↳ public CSV")
axes[0].grid(True, alpha=0.3)
axes[0].legend(title="Power level")

axes[1].set_xlabel("Elapsed time within representative segment [h]")
axes[1].set_ylabel("Current [A]")
axes[1].grid(True, alpha=0.3)

figure_path = Path("figures/chapter_2_reproduced_figure_10_foles.png")

```

```
figure_path.parent.mkdir(parents=True, exist_ok=True)
fig.savefig(figure_path, dpi=300)
print(f"Saved reproduction figure to: {figure_path}")

plt.show()
```

2.3.24 Walkthrough 5 code explanation

The data-loading function repeats logic from Walkthrough 3 because every code block in this companion should be runnable independently. In your own repository, you would probably factor the shared loader into `src/data_loading.py`. In a teaching chapter, repetition is preferable to hidden dependencies.

`segment_id = (segment_label != segment_label.shift()).cumsum()` is a very useful Pandas idiom. It turns changes in a label column into contiguous segment identifiers. This lets us separate one block of constant-power operation from the next even if the same power level appears again later in the file.

`power_level_w` is extracted from text with a regular expression. This is a realistic compromise. The paper tells us the file contains legend text in BI, but it does not give us a perfectly normalized numeric power column. Instead of complaining, we mine the text responsibly.

The segment summary table computes duration, row count, and mean absolute current. These diagnostics help us filter out trivial or degenerate segments. We then sort by power level and duration and choose the longest representative segment for each selected power level.

The most important methodological line is the selection of `selected_levels` as low, middle, and high values from the available set. This is where we handle the paper's ambiguity explicitly. We are not pretending to know the exact hidden choice behind the published figure. We are making a principled, reproducible choice from the public data and stating it.

We plot voltage and current in stacked axes because Figure 10 is about both together. The x-axis is elapsed time within each representative segment, not global experiment time, because that makes multi-level comparison much clearer and is consistent with the paper's figure intent.

2.3.25 Walkthrough 5 expected output

The printed table should list three selected representative segments with their power levels, durations, and row counts. The exact labels depend on the dataset, but you should see one relatively low power level, one midrange value, and one high value.

The reproduced figure should have two panels. In the upper panel, the three voltage traces should occupy similar voltage ranges but differ in duration and slope; higher power generally produces a faster traversal of the cycle. In the lower panel, current magnitudes should clearly differ by power level. Depending on the dataset's sign convention, discharge may appear as positive or negative current. What matters is consistency with the voltage evolution and the paper's narrative.

“Close enough” for this reproduction does not mean pixel-perfect overlay with the paper figure. It means:

1. The figure displays complete representative charge-discharge behavior for multiple constant power levels.
2. The relative ordering makes physical sense: lower power corresponds to longer duration, higher power to larger current magnitude.
3. The general visual story matches the published caption and surrounding text.

If your result satisfies those three conditions, you have performed a defensible reproduction. If it does not, the first place to investigate is not Matplotlib styling. It is segment selection and sign convention.

2.3.26 Walkthrough 5 troubleshooting

1. No usable power levels are extracted
Symptom: `power_level_w` is all NaN.
Fix: inspect the raw `segment_label` strings. The numeric power value may be formatted differently from the regular expression we used.
2. The selected segment is too short or obviously incomplete
Symptom: the plotted curve looks like only a fragment of a cycle.
Fix: refine the segment filter using duration or row count and inspect the summary table manually.
3. Current sign appears “wrong” relative to your expectation
Symptom: discharge segments are negative when you expected positive, or vice versa.

Fix: do not force a sign change unless you can justify it. The goal is faithful reproduction, not aesthetic conformity.

4. Your reproduced figure does not visually resemble the published one

Symptom: wrong ordering, strange durations, or implausible voltage range.

Fix: revisit the ambiguity discussion. Figure reproduction is often mostly about data segmentation, not plotting syntax.

2.3.27 Walkthrough 5 reflection

This exercise taught a publishable-research habit that is more important than people admit: when a paper is ambiguous, do not hide the ambiguity. State your interpretation, make it reproducible, and define what “close enough” means. That is the difference between a serious reproduction attempt and a screenshot imitation.

2.4 Open-Ended Exercises

These are the first points in the chapter where you should stop copying and start adapting. Work them before reading the solutions.

2.4.1 Exercise 1: Make the ODE problem more stiff

Modify Walkthrough 2 so that the fast RC branch becomes much faster and the slow branch becomes much slower. A good starting point is to reduce $C1$ by a factor of 10 and increase $C2$ by a factor of 5. Then compare RK45, BDF, and LSODA again.

Hints:

- Keep the current profile unchanged so the solver comparison is meaningful.
- Compare n_{fev} , wall-clock time, and maximum voltage differences.
- Pay attention to whether one solver begins to take dramatically more function evaluations.

2.4.2 Exercise 2: Fit the wrong model on purpose

Repeat Walkthrough 4, but fit a first-order RC model to data generated by the second-order model. In other words, allow only $R0$, $R1$, and $C1$ to vary and remove the second RC branch from the fitted model.

Hints:

- Keep the same noisy synthetic measurements.
- Compare RMSE and the residual plot against the second-order fit.
- Ask which features of the residual reveal model inadequacy.

2.4.3 Exercise 3: Flatten the OCV curve to mimic sodium-ion behavior

Replace `teaching_ocv_from_soc` with a flatter plateau-like function and rerun the synthetic fitting problem from Walkthrough 4. Keep the same current profile and noise level.

Hints:

- A plateau means $dOCV/dz$ becomes small over a wide SOC range.
- Think about why that makes SOC-related effects harder to separate from RC transients.
- Compare how stable the fitted parameters look relative to the earlier case.

2.5 Worked Solutions to Open-Ended Exercises

2.5.1 Solution to Exercise 1

The code change is small, but the interpretation matters.

```
import numpy as np
import pandas as pd
from scipy.integrate import solve_ivp

class Params:
    capacity_ah = 3.0
    r0_ohm = 0.015
    r1_ohm = 0.010
    c1_f = 180.0
    r2_ohm = 0.008
    c2_f = 90000.0

segment_start_times_s = np.array([0, 300, 900, 1200, 1500, 1800, 2400, 3000],
    dtype=float)
segment_currents_a = np.array([0.0, 3.0, 0.0, 6.0, 0.0, 4.5, 0.0, 1.5],
    dtype=float)
```



```

def ocv_from_soc(soc):
    soc = np.clip(np.asarray(soc), 0.0, 1.0)
    return 2.85 + 0.42 * soc + 0.06 * np.tanh((soc - 0.10) / 0.04) + 0.10 *
        ↪ np.tanh((soc - 0.88) / 0.05)

def current_profile_a(time_s):
    index = np.searchsorted(segment_start_times_s, time_s, side="right") - 1
    index = np.clip(index, 0, len(segment_currents_a) - 1)
    return float(segment_currents_a[index])

def rhs(time_s, state):
    soc, v1, v2 = state
    current_a = current_profile_a(time_s)
    return [
        -current_a / (3600.0 * Params.capacity_ah),
        -(v1 / (Params.r1_ohm * Params.c1_f)) + current_a / Params.c1_f,
        -(v2 / (Params.r2_ohm * Params.c2_f)) + current_a / Params.c2_f,
    ]

summary_rows = []
evaluation_times_s = np.linspace(0.0, 3600.0, 1201)

for method_name in ["RK45", "BDF", "LSODA"]:
    solution = solve_ivp(
        fun=rhs,
        t_span=(0.0, 3600.0),
        y0=[0.95, 0.0, 0.0],
        t_eval=evaluation_times_s,
        method=method_name,
        rtol=1e-8,
        atol=1e-10,
    )
    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    voltage_trace_v = ocv_from_soc(solution.y[0]) - current_trace_a *
    ↪ Params.r0_ohm - solution.y[1] - solution.y[2]
    summary_rows.append(
        {
            "method": method_name,
            "nfev": solution.nfev,
            "final_voltage_v": voltage_trace_v[-1],

```

```

    }
)

print(pd.DataFrame(summary_rows))

```

You should find that RK45 now takes noticeably more function evaluations than before, while BDF and often LSODA remain comparatively comfortable. That is the practical signal of increased stiffness. The model class did not change, only the separation in time scales.

2.5.2 Solution to Exercise 2

Here we intentionally underfit the data with a model that is too simple.

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares

def ocv_from_soc(soc):
    soc = np.clip(np.asarray(soc), 0.0, 1.0)
    return 2.85 + 0.42 * soc + 0.06 * np.tanh((soc - 0.10) / 0.04) + 0.10 *
        ↪ np.tanh((soc - 0.88) / 0.05)

profile_start_times_s = np.array([0, 200, 500, 800, 1100, 1400, 1800, 2200],
    ↪ dtype=float)
profile_currents_a = np.array([0.0, 2.5, 0.0, 5.0, 0.0, 3.5, 0.0, 1.5],
    ↪ dtype=float)
evaluation_times_s = np.linspace(0.0, 2600.0, 900)

def current_profile_a(time_s):
    index = np.searchsorted(profile_start_times_s, time_s, side="right") - 1
    index = np.clip(index, 0, len(profile_currents_a) - 1)
    return float(profile_currents_a[index])

def generate_second_order_truth():
    true_params = {"capacity_ah": 3.2, "r0": 0.012, "r1": 0.009, "c1":
    ↪ 2200.0, "r2": 0.006, "c2": 22000.0}

    def rhs(time_s, state):

```

```

soc, v1, v2 = state
current_a = current_profile_a(time_s)
return [
    -current_a / (3600.0 * true_params["capacity_ah"]),
    -(v1 / (true_params["r1"] * true_params["c1"])) + current_a /
    ↪ true_params["c1"],
    -(v2 / (true_params["r2"] * true_params["c2"])) + current_a /
    ↪ true_params["c2"],
]

solution = solve_ivp(rhs, (0.0, 2600.0), [0.92, 0.0, 0.0],
↪ t_eval=evaluation_times_s, method="BDF")
current_trace_a = np.array([current_profile_a(t) for t in solution.t])
voltage_trace_v = ocv_from_soc(solution.y[0]) - current_trace_a *
↪ true_params["r0"] - solution.y[1] - solution.y[2]
return voltage_trace_v

rng = np.random.default_rng(42)
measured_voltage_v = generate_second_order_truth() + rng.normal(0.0, 0.003,
↪ size=evaluation_times_s.shape)

def simulate_first_order(parameter_vector):
    r0_ohm, r1_ohm, c1_f = parameter_vector

    def rhs(time_s, state):
        soc, v1 = state
        current_a = current_profile_a(time_s)
        return [
            -current_a / (3600.0 * 3.2),
            -(v1 / (r1_ohm * c1_f)) + current_a / c1_f,
        ]

        solution = solve_ivp(rhs, (0.0, 2600.0), [0.92, 0.0],
↪ t_eval=evaluation_times_s, method="BDF")
        current_trace_a = np.array([current_profile_a(t) for t in solution.t])
        voltage_trace_v = ocv_from_soc(solution.y[0]) - current_trace_a * r0_ohm
↪ - solution.y[1]
        return voltage_trace_v

def residuals(parameter_vector):
    return simulate_first_order(parameter_vector) - measured_voltage_v

```

```

fit_result = least_squares(
    residuals,
    x0=np.array([0.020, 0.012, 2000.0]),
    bounds=(np.array([0.001, 0.001, 100.0]), np.array([0.050, 0.050,
↳ 40000.0])),
)

fitted_voltage_v = simulate_first_order(fit_result.x)
residual_v = measured_voltage_v - fitted_voltage_v

print("Fitted parameters:", fit_result.x)
print("RMSE [V]:", np.sqrt(np.mean(residual_v ** 2)))

plt.figure(figsize=(9, 4.5))
plt.plot(evaluation_times_s / 60.0, residual_v, linewidth=1.0)
plt.axhline(0.0, color="black", linestyle="--", linewidth=1.0)
plt.xlabel("Time [min]")
plt.ylabel("Residual [V]")
plt.title("Residual from fitting a first-order model to second-order data")
plt.grid(True, alpha=0.3)
plt.show()

```

The important outcome is not just a higher RMSE. It is the residual structure. You should see systematic pulse-edge mismatch that the first-order model cannot eliminate. That is exactly the kind of evidence you need before claiming a model order is inadequate.

2.5.3 Solution to Exercise 3

Here we flatten the OCV curve to mimic one of the practical difficulties of sodium-ion work.

```

import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares

def flat_plateau_ocv_from_soc(soc):
    soc = np.clip(np.asarray(soc), 0.0, 1.0)
    return 2.95 + 0.05 * soc + 0.025 * np.tanh((soc - 0.08) / 0.03) + 0.030 *
↳ np.tanh((soc - 0.92) / 0.03)

```

```

profile_start_times_s = np.array([0, 200, 500, 800, 1100, 1400, 1800, 2200],
    ↪ dtype=float)
profile_currents_a = np.array([0.0, 2.5, 0.0, 5.0, 0.0, 3.5, 0.0, 1.5],
    ↪ dtype=float)
evaluation_times_s = np.linspace(0.0, 2600.0, 900)

def current_profile_a(time_s):
    index = np.searchsorted(profile_start_times_s, time_s, side="right") - 1
    index = np.clip(index, 0, len(profile_currents_a) - 1)
    return float(profile_currents_a[index])

def simulate(parameter_vector):
    r0_ohm, r1_ohm, c1_f, r2_ohm, c2_f = parameter_vector

    def rhs(time_s, state):
        soc, v1, v2 = state
        current_a = current_profile_a(time_s)
        return [
            -current_a / (3600.0 * 3.2),
            -(v1 / (r1_ohm * c1_f)) + current_a / c1_f,
            -(v2 / (r2_ohm * c2_f)) + current_a / c2_f,
        ]

    solution = solve_ivp(rhs, (0.0, 2600.0), [0.92, 0.0, 0.0],
    ↪ t_eval=evaluation_times_s, method="BDF")
    current_trace_a = np.array([current_profile_a(t) for t in solution.t])
    return flat_plateau_ocv_from_soc(solution.y[0]) - current_trace_a *
    ↪ r0_ohm - solution.y[1] - solution.y[2]

true_parameters = np.array([0.012, 0.009, 2200.0, 0.006, 22000.0])
rng = np.random.default_rng(42)
measured_voltage_v = simulate(true_parameters) + rng.normal(0.0, 0.003,
    ↪ size=evaluation_times_s.shape)

fit_result = least_squares(
    fun=lambda p: simulate(p) - measured_voltage_v,
    x0=np.array([0.020, 0.015, 1200.0, 0.012, 12000.0]),
    bounds=(np.array([0.001, 0.001, 200.0, 0.001, 2000.0]), np.array([0.050,
    ↪ 0.050, 20000.0, 0.050, 80000.0])),
)

```

```
print("True parameters :", true_parameters)
print("Fitted parameters:", np.round(fit_result.x, 6))
print("RMSE [V]          :", np.sqrt(np.mean((simulate(fit_result.x) -
↳ measured_voltage_v) ** 2)))
```

The typical outcome is that the optimizer still fits voltage reasonably well, but the recovered parameters become less stable or less interpretable. That is the qualitative lesson you should take forward into sodium-ion work: flatter OCV regions reduce the observability you get “for free” from voltage.

What Changes for Sodium-Ion?

Up to this point, almost every skill in the chapter is chemistry-agnostic. Arrays, ODE solvers, least-squares fitting, Pandas cleaning, and figure construction do not care whether the active ion is lithium or sodium. But the interpretation of the outputs does change, and it changes in ways you should start internalizing now.

The first change is the OCV curve. Many sodium-ion chemistries, especially hard-carbon-based systems, exhibit flatter voltage plateaus over important SOC ranges than the textbook lithium-ion examples you may be used to. In numerical terms, that means $dOCV/dz$ can be small over a broad interval. The practical consequence is that terminal voltage becomes less informative about SOC in those regions, and RC transients or sensor noise can dominate what you see. When you reach the fitting and state-estimation chapters later in the manual, that one chemistry fact will explain a large fraction of the extra difficulty.

The second change is parameter transferability. In this chapter we used a smooth teaching OCV function and synthetic RC parameters. In sodium-ion work, you should be much less willing to borrow lithium-ion OCV or resistance assumptions casually. The workflow of the chapter still applies, but the parameter values need stronger justification, and the sensitivity of the results to those values should be reported more explicitly.

The third change is dataset availability. Lithium-ion public datasets are abundant enough that we can afford to learn parsing on them. Sodium-ion public datasets are much sparser and often less standardized. That makes the data-cleaning habits from Walkthrough 3 even more important. In sodium-ion projects, you will often combine sparse public measurements, digitized figures, and simulation-generated synthetic data. The computational workflow stays the same; the provenance

burden gets heavier.

The fourth change is validation strategy. For lithium-ion, it is often possible to find a public benchmark dataset that closely matches your modeling setup. For sodium-ion, you may need to validate at the level of qualitative behaviors, voltage-window consistency, rate sensitivity, or agreement with a published figure rather than against a rich open raw dataset. That does not lower the standard. It changes what honest validation looks like.

So the short version is this: the software patterns from this chapter transfer directly to sodium-ion, but the chemistry makes the inverse problems harder, the datasets thinner, and the need for explicit assumptions greater. Keep that in mind from the beginning rather than treating sodium-ion as “lithium-ion with renamed variables.”

Chapter Summary and Skill Checklist

The key skills from this chapter are the ones you should start to feel in your hands rather than merely remember abstractly.

- evaluate battery equations over entire grids with NumPy vectorization and broadcasting
- slice arrays deliberately to isolate physically meaningful windows
- write a state-vector ODE for a second-order Thevenin model
- choose among RK45, BDF, and LSODA with a reason rather than a guess
- inspect solver diagnostics instead of trusting a successful run blindly
- download and normalize a public battery dataset with Pandas
- infer a usable time axis and resample a battery time series
- fit model parameters with `scipy.optimize.least_squares`
- diagnose a fit with residual structure, not RMSE alone
- reproduce a published figure while documenting the ambiguities honestly

You should now be able to check all of the following:

- I can represent a battery equivalent-circuit model as a system of first-order ODEs.
- I can explain why SOC, RC overpotentials, and terminal voltage play different roles in the code.

- I can use broadcasting to evaluate terminal voltage for multiple current levels without nested loops.
- I can run `solve_ivp` and explain why one solver may be more appropriate than another.
- I can build a residual function and fit parameters with bounded least squares.
- I can clean a real battery CSV rather than only work with synthetic arrays.
- I can state my current sign convention explicitly instead of assuming it.
- I can reproduce a published time-series figure in a way that is transparent about interpretation choices.

If any box above still feels uncertain, revisit the corresponding walkthrough now. Later chapters assume these skills are already in muscle memory.

Deliverable

The deliverable from your plan is a notebook that simulates a second-order RC equivalent-circuit discharge, adds noise, and fits the parameters back. The bonus deliverable is to wrap it as a reusable function.

Approach the deliverable in two layers.

First, create the notebook `notebooks/chapter_2_scientific_python_refresher.ipynb` and make sure it contains:

- a clean implementation of the second-order Thevenin forward model
- at least one pulse-rest current protocol
- a synthetic noisy voltage trace generated from known parameters
- a bounded least-squares fit recovering those parameters
- at least one residual plot and one parameter-comparison table

Second, factor the forward model into a reusable helper in `src/thevenin_model.py` or similar. A minimal but good reusable interface would be a function that accepts parameters, a current-profile function, initial SOC, and evaluation times and returns a `DataFrame` of time, current, SOC, branch voltages, and terminal voltage.

A strong submission for this chapter will also save:

- a CSV of the synthetic measurements in `results/`
- a CSV of fitted parameters in `results/`
- at least one figure in `figures/`
- a short log entry describing solver choice, fit behavior, and what you learned from the residuals

If you want a worked partial structure for the reusable function, this is a good starting signature:

```
def simulate_thevenin(
    parameters,
    current_profile_a,
    evaluation_times_s,
    initial_soc_fraction,
    method="BDF",
):
    ...
```

The important design idea is to separate the model definition from the specific experiment. That habit will pay off immediately in Chapter 3 when PyBaMM starts making the same separation explicit at a much larger scale.

Further Practice and Reading

- Ana Foles, Luís Fialho, Pedro Horta, and Manuel Collares-Pereira, “Validation of a lithium-ion commercial battery pack model using experimental data for stationary energy management application,” *Open Research Europe* (2022). This is the paper used in the reproduction exercise and worth reading alongside the public CSV.
- G. L. Plett, *Battery Management Systems*, especially the equivalent-circuit modeling material. The mathematics there will feel different after doing the coding work in this chapter.
- SciPy official documentation for `scipy.integrate.solve_ivp` and `scipy.optimize.least_squares`. Bookmark both now; they will reappear throughout the manual.
- NumPy official documentation on broadcasting. This is one of the few docs pages that repays careful rereading.
- Matplotlib official tutorials on labeling, styles, and multi-axis layouts.

Publication-quality plotting is learned by iteration, not by memorizing one magic function.

- Community resources: Scientific Python Forum and the PyData community spaces. Even before PyBaMM enters, these are useful homes for the numerical questions that battery research raises.

Lab Chapter 3: *Your First PyBaMM Simulation* is next.

Your First PyBaMM Simulation

This chapter is the point where the theory textbook stops being something you understand and starts becoming something you can execute. Up to now, the Doyle-Fuller-Newman model has lived mostly in the language of conservation laws, porous-electrode assumptions, constitutive relations, and boundary conditions. By the end of this chapter, you will have turned that theory into a working PyBaMM workflow that you can run, modify, inspect, and defend. You will know how to build a lithium-ion battery model from PyBaMM's core objects, how to solve a default DFN simulation with a real parameter set, how to extract internal states instead of staring only at terminal voltage, and how to compare the DFN against reduced-order models in a way that is honest about both computational cost and modeling error.

This matters for publishable research because physics-based battery modeling is not only about deriving equations correctly. It is about being able to move from the equations to a trustworthy software implementation, ask the model targeted questions, and explain why a particular model fidelity is justified for a particular study. Reviewers do not care that you know the DFN model exists. They care whether your parameter set is traceable, whether your solver choices are sensible, whether your plotted internal states mean what you say they mean, and whether you are using a reduced model because it is appropriate rather than because you never learned how to run the full one.

Keep Textbook Chapter 8 open while you work through this lab. This chapter operationalizes the DFN model you learned there, so you want the derivation fresh in your mind. When we inspect electrolyte concentration gradients, solid-particle diffusion, interfacial overpotentials, and open-circuit voltage contributions, we are not learning new battery physics from scratch. We are learning how PyBaMM represents the same physics in software. Textbook Chapter 3 should also be nearby, because the distinction between open-circuit voltage, terminal voltage, overpotential, discharge capacity, C-rate, and state of charge needs to remain exact when we start reading variables out of a simulation object. Textbook Chapter 10 is worth keeping in the background as well, because one of the most important skills you will develop here is choosing an appropriate model fidelity. That same judgment call sits underneath every later BMS approximation, virtual-cell workflow, and parameter-estimation loop.

This chapter also depends directly on Labs 1 and 2. Lab Chapter 1 gave you the reproducible environment and project habits we now rely on. Lab Chapter 2 refreshed the scientific Python tools that sit underneath every modeling workflow in this companion. PyBaMM is not a replacement for those skills. It is a higher-level expression of them. If NumPy arrays, SciPy-style problem structure, and plotting logic still feel foggy, this chapter will become much harder than it needs to be.

There is one more reason this chapter matters for your long-term goal of sodium-ion research. Today we will use mostly lithium-ion parameter sets because PyBaMM's built-in ecosystem is much richer there. That is not a detour. It is training. Your future sodium-ion work will reuse the same software patterns: model construction, parameter management, solver choice, internal-variable inspection, fidelity comparison, and figure reproduction. What will change is the chemistry-specific content: different OCV shapes, different diffusivity functions, different parameter availability, and weaker public-data coverage. The software method is durable. We are learning that durable layer here.

The pace of the chapter is deliberate. First, we will install PyBaMM properly and verify that it really works. Next, we will build a conceptual bridge from the DFN derivation in the textbook to PyBaMM's object model. Then we will run a first DFN simulation with the `Chen2020` parameter set and interpret every major curve. After that, we will open the model up and inspect internal variables, because serious battery modeling begins when you stop treating voltage as the

only truth. Then we will compare SPM, SPMe, and DFN on the same duty cycle and quantify the accuracy-speed tradeoff. Finally, because every Part of this companion must train reproduction skill, we will reproduce a published SPMe error-decomposition figure based on Marquis et al. (2019) and discuss exactly what “close enough” means when a modern software implementation is not numerically identical to the authors’ original post-processing pipeline.

If you do this chapter carefully, PyBaMM stops being an intimidating package name and becomes a research instrument. That is the real milestone.

3.1 Prerequisites Check

- Required software: the `sib-research` environment from Lab Chapter 1; Python 3.11; JupyterLab 4.4+
- Additional Python package for this chapter: `pybamm==25.12.1`
- Recommended plotting stack already installed from earlier chapters: `numpy==2.3.4`, `scipy==1.16.0`, `pandas==3.0.2`, `matplotlib==3.10.8`
- Required textbook chapters: Textbook Chapter 8 is essential; Textbook Chapter 3 should be close at hand; Textbook Chapter 10 is helpful context
- Required prior lab chapters: Lab Chapter 1 and Lab Chapter 2
- Estimated time: 10 to 14 hours if your install is smooth; 14 to 16 hours if this is your first real PyBaMM workflow

If Lab Chapter 2 still feels shaky, especially with plotting and numerical thinking, revisit Sections 2.2 through 2.6 before going deeper here. If your Jupyter kernel setup from Chapter 1 was never fully clean, fix it now. The most common PyBaMM beginner problem is not a bad model. It is running the right code in the wrong environment.

3.2 Environment Setup

PyBaMM installs cleanly for most readers if the environment is already healthy. The key phrase there is “if the environment is already healthy.” We will not assume that. We will verify each step.

3.2.1 Activate the Chapter 1 environment

If you used conda in Lab Chapter 1:

```
conda activate sib-research
```

If you used venv on Linux or macOS:

```
source .venv/bin/activate
```

If you used venv on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

3.2.2 Install PyBaMM with a pinned version

For this manual we will pin to `pybamm==25.12.1`. The pin is not because older or newer versions are necessarily wrong. It is because beginner pain rises sharply when examples are written against one API and run against another.

Install PyBaMM with:

```
python -m pip install pybamm==25.12.1
```

If you prefer to make the version pin explicit in a requirements file for the repository, create `requirements-chapter-3.txt`:

```
pybamm==25.12.1
```

Then install it with:

```
python -m pip install -r requirements-chapter-3.txt
```

If you are using conda, do not overcomplicate this. A clean conda environment plus `pip install pybamm==25.12.1` is a perfectly normal workflow for PyBaMM.

3.2.3 Verify the install with a minimal "hello battery"

Run this exact snippet in a terminal or a fresh notebook cell:

```
import pybamm
import numpy as np

# Build the default DFN with a bundled teaching parameter set.
model = pybamm.lithium_ion.DFN()
parameter_values = pybamm.ParameterValues("Chen2020")
```

```
simulation = pybamm.Simulation(model, parameter_values=parameter_values)

# Ask PyBaMM for output on a clean reporting grid.
evaluation_times_s = np.linspace(0, 600, 61)
solution = simulation.solve(evaluation_times_s)

# Evaluate the processed variable on the same time grid.
terminal_voltage_v = solution["Terminal voltage [V]"](evaluation_times_s)

print("PyBaMM version:", pybamm.__version__)
print("Initial voltage [V]:", round(float(terminal_voltage_v[0]), 4))
print("Voltage at 600 s [V]:", round(float(terminal_voltage_v[-1]), 4))
```

Expected output on the pinned version is:

```
PyBaMM version: 25.12.1
Initial voltage [V]: 4.0377
Voltage at 600 s [V]: 3.8155
```

Do not obsess if the last digit differs by a few units in the fourth decimal place on a different machine. Do stop immediately if you get an import error, a solver failure, or a voltage that is nowhere near these values.

3.2.4 Confirm Jupyter sees the correct environment

Launch JupyterLab:

```
jupyter lab
```

Create a notebook named
notebooks/chapter_3_first_pybamm_simulation.ipynb, then run:

```
import sys
import pybamm

print(sys.executable)
print(pybamm.__version__)
```

The interpreter path should point into your intended environment, and the printed version should be 25.12.1.

3.2.5 Common install failures and fixes

1. `ModuleNotFoundError: No module named 'pybamm'`

Symptom: PyBaMM imports in the terminal fail, or they work in the terminal but not in Jupyter.

Fix: if both terminal and notebook fail, PyBaMM is not installed in the active environment. Re-activate the environment and reinstall. If the terminal works but Jupyter fails, re-register the kernel from the active environment with `python -m ipykernel install --user --name sib-research --display-name "Python (sib-research)"`.

2. Installation succeeds but import fails with a dependency error mentioning `casadi`, `scipy`, or a compiled wheel

Symptom: `pip` finishes, but `import pybamm` crashes immediately.

Fix: create a fresh environment instead of patching the broken one. Most PyBaMM import failures are environment-consistency problems, not package bugs.

3. Solver errors on the first run

Symptom: the package imports but the hello-world simulation fails.

Fix: first confirm you copied the code exactly. If you did, remove any local modifications to your environment and retry in a fresh environment. Beginner PyBaMM code should not fail on the built-in DFN plus Chen2020.

4. A notebook seems to “hang” while solving

Symptom: the cell runs longer than expected with no visible output.

Fix: wait a little longer first; the first import and solve can take longer on some systems. If it remains stalled, restart the kernel and run the small 600-second example before attempting anything larger.

3.3 Conceptual Bridge: From Textbook DFN Equations to PyBaMM Objects

In Textbook Chapter 8, the Doyle-Fuller-Newman model appeared as a coupled system of equations for lithium concentration in the solid particles, lithium-ion concentration in the electrolyte, charge conservation in the solid phase, charge conservation in the electrolyte, interfacial reaction kinetics, and the constitutive laws that tie those fields together. In that presentation, the intellectual burden is on understanding the model. In PyBaMM, the burden shifts slightly: you still need to understand the model, but now you also need to understand how the software stores, processes, discretizes, and solves it.

At the theory level, the DFN model can be summarized very compactly. In the active-material particles we solve diffusion:

$$\frac{\partial c_{s,k}}{\partial t} = \frac{1}{r^2} \frac{\partial}{\partial r} \left(D_{s,k} r^2 \frac{\partial c_{s,k}}{\partial r} \right), \quad k \in \{n, p\}, \quad (1)$$

while in the electrolyte we solve salt conservation:

$$\varepsilon_k \frac{\partial c_{e,k}}{\partial t} = -\nabla \cdot N_{e,k} + \frac{1 - t_+^0}{F} a_k j_k, \quad (2)$$

and in the solid and electrolyte phases we enforce charge conservation:

$$\nabla \cdot i_{s,k} = -a_k j_k, \quad \nabla \cdot i_{e,k} = a_k j_k. \quad (3)$$

The interfacial current density j_k is usually closed through Butler-Volmer kinetics, and the measured terminal voltage emerges from the difference between positive and negative current-collector potentials, corrected by the various losses and concentration effects. None of that changes in PyBaMM. What changes is the representation.

PyBaMM does not begin by “running a solver.” It begins by building a symbolic model graph. When you write:

```
model = pybamm.lithium_ion.DFN()
```

you are not yet solving anything. You are constructing a software object that contains the model equations, variables, submodels, events, boundary conditions, and metadata. You can think of `model` as the theory-textbook object translated into symbolic Python form.

The next critical object is the parameter set:

```
parameter_values = pybamm.ParameterValues("Chen2020")
```

This is one of the most important conceptual habits to internalize early. In PyBaMM, the model and the parameter values are intentionally separate. That separation matters because it lets you ask a research question cleanly. Are you changing the governing equations, or are you changing only the chemistry and cell properties fed into those equations? If you blur those two ideas together, you will later make it much harder to report what you actually did.

After parameters, PyBaMM still has more work to do before anything becomes numerical. The geometry object defines where each field lives. The mesh object decides where those domains are sampled. The discretization object converts the

symbolic PDE system into an algebraic-differential system the solver can handle. The solver then advances that discretized system in time. Finally, the `Simulation` object wraps the whole pipeline so that in normal use you do not need to manually call every processing step.

That object hierarchy is worth making explicit:

PyBaMM object	What it represents conceptually	What you usually do with it
<code>model</code>	The governing equations and variables	Choose DFN, SPM, SPMe, or another model
<code>ParameterValues</code>	Cell chemistry and numerical constants	Select or modify a parameter set
<code>geometry</code>	Domains and coordinates	Usually accept the model default at first
<code>mesh</code>	Spatial sampling of those domains	Adjust resolution when accuracy or speed matters
<code>discretisation</code>	Numerical translation of equations to algebraic form	Usually left implicit until you need control
<code>solver</code>	Time integration and nonlinear solves	Swap only when a problem demands it
<code>simulation</code>	A convenience wrapper over the full workflow	The main entry point for most users

This is the software version of the bridge from theory to tool. Textbook Chapter 8 gave you the physics, assumptions, and derivation. PyBaMM gives you an organized representation of those same ingredients. If you ever feel lost in the library API, ask: which part of the mathematical model does this object correspond to?

There is a second conceptual bridge that matters just as much. In theory class, it is easy to talk about “the voltage” as if it were one simple number produced by the model. In PyBaMM, voltage is a composition of interpretable contributors. You can ask for terminal voltage, open-circuit voltage, reaction overpotentials, electrolyte ohmic losses, concentration overpotentials, and internal concentrations separately. This is not software trivia. It is the practical expression of the

decomposition you learned in the textbook. When a simulation result looks odd, the first serious question is not “did the code work?” It is “which physical contribution is driving this shape?”

One more point is worth stating explicitly before we touch code. A `Simulation` object is convenient, but it can hide the layers underneath. That convenience is good for productivity, but bad for learning if you let it stay magical. In this chapter we will use `Simulation` heavily because that is the right tool for most work. But we will also open the box once so you see the manual pipeline. Later chapters on parameters, experiments, and reproducibility will make much more sense if you know what `Simulation` is saving you from having to do by hand.

So the mental map for the chapter is this: Textbook Chapter 8 tells you what the DFN is. PyBaMM tells you how to hold that DFN in software. The rest of the chapter is about learning to use that software representation without losing contact with the physics.

Guided Walkthrough 1

Meet the PyBaMM Object Hierarchy

Learning objective: Instantiate a DFN model, inspect the objects underneath `Simulation`, and understand what PyBaMM is building before it solves anything.

We will start by making the object hierarchy concrete. Most PyBaMM tutorials quite reasonably jump straight to `Simulation`. For a first working notebook that is fine, but for a methods book it is too opaque. We will briefly look at both the high-level and low-level workflows so that later chapters do not feel magical.

3.3.1 Walkthrough 1 code

```
import numpy as np
import pybamm

# Create the symbolic model and attach a real parameterization.
model = pybamm.lithium_ion.DFN()
parameter_values = pybamm.ParameterValues("Chen2020")

print("PyBaMM version:", pybamm.__version__)
```

```

print("Model name:", model.name)
print("Default geometry domains:", list(model.default_geometry.keys()))
print("Default solver:", type(model.default_solver).__name__)
print("Nominal capacity [A.h]:", parameter_values["Nominal cell capacity
↳ [A.h]"])
print("Lower voltage cut-off [V]:", parameter_values["Lower voltage cut-off
↳ [V]"])
print("Upper voltage cut-off [V]:", parameter_values["Upper voltage cut-off
↳ [V]"])

geometry = model.default_geometry
parameter_values.process_model(model)
parameter_values.process_geometry(geometry)

# Keep the first teaching mesh intentionally modest.
var_pts = {
    "x_n": 10,
    "x_s": 10,
    "x_p": 10,
    "r_n": 10,
    "r_p": 10,
}

mesh = pybamm.Mesh(geometry, model.default_submesh_types, var_pts)
discretisation = pybamm.Discretisation(mesh, model.default_spatial_methods)
discretisation.process_model(model)

# Solve the discretized model directly, without the Simulation wrapper.
solver = model.default_solver
evaluation_times_s = np.linspace(0, 600, 51)
manual_solution = solver.solve(model, evaluation_times_s)

manual_voltage_v = manual_solution["Terminal voltage
↳ [V]"](evaluation_times_s)

print("Voltage at 600 s from the manual pipeline [V]:",
↳ round(float(manual_voltage_v[-1]), 4))

```

3.3.2 Walkthrough 1 explanation

The first two lines create the core objects of the workflow.

`pybamm.lithium_ion.DFN()` gives us the symbolic DFN model itself.

`pybamm.ParameterValues("Chen2020")` selects a built-in lithium-ion

parameter set associated with the LG M50 cell described in Chen et al. (2020). This is standard practice in teaching and benchmarking because it gives us a parameter set that is already internally consistent.

The next group of `print` statements is not throwaway code. We are making the object hierarchy visible. `model.name` confirms which model we instantiated. `model.default_geometry.keys()` shows the physical domains that the DFN spans: negative electrode, separator, positive electrode, negative particle, positive particle, and current collector. `model.default_solver` tells us which solver PyBaMM would use by default if we delegated the solve to `Simulation`. On the pinned version that solver is `IDAKLUSolver`, which is a good default for stiff battery-model systems.

The `process_model` and `process_geometry` calls are where parameters stop being abstract values in a dictionary and start being substituted into the symbolic model and geometry. This is the first point where the mathematical object becomes chemistry-specific.

The `var_pts` dictionary controls the mesh resolution. The meanings are worth committing to memory now: `x_n`, `x_s`, and `x_p` are through-thickness points in the negative electrode, separator, and positive electrode, while `r_n` and `r_p` are radial points in the negative and positive particles. If you reduce these values, the simulation becomes faster but less accurate. If you increase them, the reverse happens. Later chapters will reuse this exact pattern when we talk about mesh sensitivity and computational cost.

`pybamm.Mesh(...)` builds the mesh from the geometry and the default submesh types. `pybamm.Discretisation(...)` constructs the numerical machinery for turning symbolic PDEs into algebraic form. `discretisation.process_model(model)` is the point where the model becomes numerically solvable.

Only after those steps do we get to the solver. `solver.solve(model, evaluation_times_s)` numerically advances the discretized model over the requested time grid. The final two lines extract terminal voltage from the resulting solution and evaluate it at the grid we asked for.

3.3.3 Walkthrough 1 expected output

On the pinned version you should see output of this form:

```
PyBaMM version: 25.12.1
Model name: Doyle-Fuller-Newman model
Default geometry domains: ['negative electrode', 'separator', 'positive
↪ electrode', 'negative particle', 'positive particle', 'current
↪ collector']
Default solver: IDAKLUSolver
Nominal capacity [A.h]: 5.0
Lower voltage cut-off [V]: 2.5
Upper voltage cut-off [V]: 4.2
Voltage at 600 s from the manual pipeline [V]: 3.8171
```

The important thing is not memorizing the fourth decimal place. It is recognizing what the output proves. You have instantiated a DFN, attached a real parameter set, processed the geometry, created a mesh, discretized the model, and solved it directly through the lower-level API. That is the full theory-to-numerics chain in one short block of code.

3.3.4 Walkthrough 1 troubleshooting

1. `KeyError` when accessing a parameter name

Symptom: one of the string keys in `parameter_values[...]` fails.

Fix: parameter names in PyBaMM are exact string keys. Copy them carefully.

Later chapters will show safer ways to inspect and modify parameter sets.

2. A mesh-related error during `pybamm.Mesh(...)`

Symptom: PyBaMM complains about missing or incompatible variable-point definitions.

Fix: make sure the keys in `var_pts` are exactly `x_n`, `x_s`, `x_p`, `r_n`, and `r_p` for this model.

3. A solver error on the manual pipeline but not on the high-level `Simulation` workflow

Symptom: the low-level path feels brittle.

Fix: that is exactly why `Simulation` exists. The manual pipeline is for understanding and for specialized control, not because you should build every notebook this way.

3.3.5 Walkthrough 1 reflection

This exercise teaches the first nontrivial PyBaMM habit: the library is not a black box that takes current in and voltage out. It is a structured numerical

representation of the theory you learned in Textbook Chapter 8. That mindset will matter every time you later adjust parameters, experiments, meshes, or model fidelity.

Guided Walkthrough 2

Run a Default DFN Simulation and Interpret the Big Curves

Learning objective: Solve a built-in DFN model with Chen2020, extract the major outputs, and interpret terminal voltage, discharge capacity, and approximate SOC correctly.

Now we will switch to the high-level workflow you will use most of the time. This is where PyBaMM becomes productive. The goal is not only to run the simulation, but to read it properly. A correct plot is not useful unless you know what it means physically.

3.3.6 Walkthrough 2 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

# The high-level Simulation object wraps the full PyBaMM pipeline.
model = pybamm.lithium_ion.DFN()
parameter_values = pybamm.ParameterValues("Chen2020")
simulation = pybamm.Simulation(model, parameter_values=parameter_values)

# Report on a clean one-hour grid.
evaluation_times_s = np.linspace(0, 3600, 361)
solution = simulation.solve(evaluation_times_s)

time_s = evaluation_times_s
time_min = time_s / 60.0

# Pull out the main variables we want to interpret.
terminal_voltage_v = solution["Terminal voltage [V]"](time_s)
open_circuit_voltage_v = solution["Battery open-circuit voltage [V]"](time_s)
discharge_capacity_ah = solution["Discharge capacity [A.h]"](time_s)
nominal_capacity_ah = parameter_values["Nominal cell capacity [A.h]"]
approximate_soc = 1.0 - discharge_capacity_ah / nominal_capacity_ah
```

```

print("Initial terminal voltage [V]:", round(float(terminal_voltage_v[0]),
↳ 4))
print("Terminal voltage at 3600 s [V]:", round(float(terminal_voltage_v[-1]),
↳ 4))
print("Discharge capacity after 3600 s [A.h]:",
↳ round(float(discharge_capacity_ah[-1]), 4))
print("Approximate SOC after 3600 s [-]:", round(float(approximate_soc[-1]),
↳ 4))

fig, axes = plt.subplots(3, 1, figsize=(9, 10), sharex=True,
↳ constrained_layout=True)

axes[0].plot(time_min, terminal_voltage_v, linewidth=2.2, label="Terminal
↳ voltage")
axes[0].plot(time_min, open_circuit_voltage_v, linewidth=1.8, linestyle="--",
↳ label="Battery OCV")
axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("DFN discharge with the Chen2020 parameter set")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_min, discharge_capacity_ah, color="tab:green",
↳ linewidth=2.2)
axes[1].set_ylabel("Discharge Capacity [A.h]")
axes[1].grid(True, alpha=0.3)

axes[2].plot(time_min, approximate_soc, color="tab:red", linewidth=2.2)
axes[2].set_ylabel("Approximate SOC [-]")
axes[2].set_xlabel("Time [min]")
axes[2].grid(True, alpha=0.3)

plt.show()

```

3.3.7 Walkthrough 2 explanation

The first three lines are now familiar: instantiate the DFN, select a parameter set, and wrap the workflow in `Simulation`. This is the pattern you should expect to reuse constantly.

`evaluation_times_s = np.linspace(0, 3600, 361)` gives us a clean one-second-like reporting grid at ten-second spacing. PyBaMM may still use adaptive internal time stepping underneath, but this line tells the processed

variables where we want the solution evaluated for plotting and downstream analysis.

`solution["Terminal voltage [V]"](time_s)` is the first PyBaMM pattern worth committing to muscle memory. A solution variable is not just a stored array. It is a processed variable that can be evaluated on the time grid you want, and later on spatial coordinates as well. This is one of the reasons PyBaMM feels more expressive than a hand-built ODE script.

We also extract `Battery open-circuit voltage [V]`. This is a very useful comparison curve because it separates equilibrium thermodynamics from dynamic losses. The gap between OCV and terminal voltage is not random. It is the accumulated effect of kinetic, ohmic, and concentration losses.

The `Discharge capacity [A.h]` variable is exactly what its name suggests: how much capacity has been drawn since the simulation began. We then compute an approximate SOC using

$$\text{SOC}_{\text{approx}}(t) = 1 - \frac{Q_{\text{discharged}}(t)}{Q_{\text{nominal}}}. \quad (4)$$

That approximation is fine for a first DFN walkthrough, but it is still an approximation. In later chapters and in real BMS work, SOC estimation becomes more subtle than discharged-capacity bookkeeping. I am flagging that now so you do not quietly internalize Equation (4) as a universal truth.

The plotting section deliberately puts the variables in separate axes rather than overloading one panel. For a research notebook, clarity beats compactness.

3.3.8 Walkthrough 2 expected output

The printed values on the pinned version should be approximately:

```
Initial terminal voltage [V]: 4.0377
Terminal voltage at 3600 s [V]: 2.9830
Discharge capacity after 3600 s [A.h]: 4.9381
Approximate SOC after 3600 s [-]: 0.0124
```

The first plot should show terminal voltage starting a little above 4.03 V and declining steadily toward about 2.98 V over the hour. The dashed OCV curve should sit above the terminal-voltage curve almost everywhere. That gap is physically meaningful: it is the sum of losses that appear only under load.

The second plot should be almost a straight line rising from 0 to just under 5 A.h, because the default Chen2020 setup is effectively a constant-current

discharge over this interval.

The third plot should fall nearly linearly from 1 toward zero, ending just above zero rather than exactly at zero because the one-hour window is very close to, but not identical with, a complete nominal-capacity discharge for this particular model trajectory.

If your terminal-voltage curve is jagged, increases during discharge, or crosses above the OCV curve, something is wrong.

3.3.9 Walkthrough 2 troubleshooting

1. `KeyError` for a variable name

Symptom: `solution["Battery open-circuit voltage [V]"]` or a similar lookup fails.

Fix: variable names are exact. Use the strings shown here exactly, including units.

2. Confusing terminal voltage with OCV

Symptom: you interpret the dashed curve as “what the battery is doing” and ignore the loaded terminal voltage.

Fix: remember Textbook Chapter 3. OCV is an equilibrium quantity. Terminal voltage is what a pack controller or experiment actually sees under load.

3. Misreading the approximate SOC

Symptom: you assume it is a model-provided ground truth SOC state.

Fix: in this walkthrough it is simply discharged capacity normalized by nominal capacity. That is fine for teaching, but it is not the last word on SOC.

3.3.10 Walkthrough 2 reflection

This exercise teaches the basic rhythm of PyBaMM work: instantiate, solve, extract processed variables, and interpret them in physical language rather than just visually. We will reuse this pattern in every later PyBaMM chapter.

Guided Walkthrough 3

Open the Cell and Inspect Internal Variables

Learning objective: Extract internal concentrations, overpotentials, and spatial profiles from the DFN so that the model stops being “a voltage generator” and

becomes a physically interpretable virtual cell.

This is the most important conceptual move in the chapter. Terminal voltage is the public face of the model. Internal variables are where the research value lives. If you only ever read voltage, you are using a physics-based model like a fancier equivalent circuit.

3.3.11 Walkthrough 3 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

model = pybamm.lithium_ion.DFN()
parameter_values = pybamm.ParameterValues("Chen2020")
parameter_values.update({"Current function [A]": 5.0})

simulation = pybamm.Simulation(model, parameter_values=parameter_values)

# Stop halfway through the nominal discharge so internal states stay easy to
↪ read.
evaluation_times_s = np.linspace(0, 1800, 301)
solution = simulation.solve(evaluation_times_s)

time_min = evaluation_times_s / 60.0

terminal_voltage_v = solution["Terminal voltage [V]"](evaluation_times_s)
battery_ocv_v = solution["Battery open-circuit voltage
↪ [V]"](evaluation_times_s)
xavg_electrolyte_concentration = solution["X-averaged electrolyte
↪ concentration [mol.m-3]"](evaluation_times_s)
xavg_negative_surface_concentration = solution[
    "X-averaged negative particle surface concentration [mol.m-3]"
](evaluation_times_s)
xavg_positive_surface_concentration = solution[
    "X-averaged positive particle surface concentration [mol.m-3]"
](evaluation_times_s)
xavg_reaction_overpotential_v = solution["X-averaged battery reaction
↪ overpotential [V]"](evaluation_times_s)
xavg_concentration_overpotential_v = solution["X-averaged battery
↪ concentration overpotential [V]"](evaluation_times_s)
```

```

# Use the parameter-set geometry so our spatial coordinates are valid.
negative_electrode_thickness_m = parameter_values["Negative electrode
↳ thickness [m]"]
separator_thickness_m = parameter_values["Separator thickness [m]"]
positive_electrode_thickness_m = parameter_values["Positive electrode
↳ thickness [m]"]
negative_particle_radius_m = parameter_values["Negative particle radius [m]"]

cell_length_m = (
    negative_electrode_thickness_m
    + separator_thickness_m
    + positive_electrode_thickness_m
)

x_positions_m = np.linspace(0.0, cell_length_m, 200)
r_positions_m = np.linspace(0.0, negative_particle_radius_m, 150)

electrolyte_profile_start = solution["Electrolyte concentration
↳ [mol.m-3]"](x=x_positions_m, t=0.0)
electrolyte_profile_end = solution["Electrolyte concentration
↳ [mol.m-3]"](x=x_positions_m, t=1800.0)

# Radial concentration inside an x-averaged negative particle.
negative_particle_profile_start = solution[
    "X-averaged negative particle concentration [mol.m-3]"
](r=r_positions_m, t=0.0)
negative_particle_profile_end = solution[
    "X-averaged negative particle concentration [mol.m-3]"
](r=r_positions_m, t=1800.0)

fig, axes = plt.subplots(2, 2, figsize=(12, 9), constrained_layout=True)

axes[0, 0].plot(time_min, terminal_voltage_v, linewidth=2.2, label="Terminal
↳ voltage")
axes[0, 0].plot(time_min, battery_ocv_v, linewidth=1.8, linestyle="--",
↳ label="Battery OCV")
axes[0, 0].set_title("Voltage-level variables")
axes[0, 0].set_xlabel("Time [min]")
axes[0, 0].set_ylabel("Voltage [V]")
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].plot(
    time_min,

```

```

        xavg_electrolyte_concentration,
        linewidth=2.2,
        label="X-averaged electrolyte concentration",
    )
axes[0, 1].plot(
    time_min,
    xavg_negative_surface_concentration,
    linewidth=2.0,
    label="Negative particle surface concentration",
)
axes[0, 1].plot(
    time_min,
    xavg_positive_surface_concentration,
    linewidth=2.0,
    label="Positive particle surface concentration",
)
axes[0, 1].set_title("Concentration-level variables")
axes[0, 1].set_xlabel("Time [min]")
axes[0, 1].set_ylabel("Concentration [mol.m-3]")
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].plot(
    time_min,
    xavg_reaction_overpotential_v * 1000.0,
    linewidth=2.2,
    label="Reaction overpotential",
)
axes[1, 0].plot(
    time_min,
    xavg_concentration_overpotential_v * 1000.0,
    linewidth=2.2,
    label="Concentration overpotential",
)
axes[1, 0].set_title("Overpotential contributions")
axes[1, 0].set_xlabel("Time [min]")
axes[1, 0].set_ylabel("Overpotential [mV]")
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].plot(
    x_positions_m * 1e6,
    electrolyte_profile_start,
    linewidth=2.0,
    label="Electrolyte concentration at 0 min",

```

```

)
axes[1, 1].plot(
    x_positions_m * 1e6,
    electrolyte_profile_end,
    linewidth=2.0,
    label="Electrolyte concentration at 30 min",
)
axes[1, 1].set_title("Through-thickness electrolyte profile")
axes[1, 1].set_xlabel("Position through cell [ $\mu\text{m}$ ]")
axes[1, 1].set_ylabel("Electrolyte Concentration [ $\text{mol.m}^{-3}$ ]")
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.show()

fig, ax = plt.subplots(figsize=(8, 5), constrained_layout=True)
ax.plot(
    r_positions_m * 1e6,
    negative_particle_profile_start,
    linewidth=2.0,
    label="Negative particle concentration at 0 min",
)
ax.plot(
    r_positions_m * 1e6,
    negative_particle_profile_end,
    linewidth=2.0,
    label="Negative particle concentration at 30 min",
)
ax.set_title("X-averaged negative-particle radial concentration profile")
ax.set_xlabel("Particle radius coordinate [ $\mu\text{m}$ ]")
ax.set_ylabel("Concentration [ $\text{mol.m}^{-3}$ ]")
ax.legend()
ax.grid(True, alpha=0.3)
plt.show()

```

3.3.12 Walkthrough 3 explanation

We intentionally shorten the simulation window to 1800 s, which is a half-hour at approximately 1C for the nominal 5 Ah cell. That gives us a mid-discharge view where the model is still far from terminal cutoffs and the internal-state interpretation is cleaner.

The first group of variable extractions uses X-averaged quantities. This is an

important PyBaMM naming pattern. When a quantity varies spatially, PyBaMM often provides both the full field and spatially averaged versions. For a first inspection, the averages are easier to interpret. Later, when you need to diagnose localization or gradients, you move to the full fields.

The overpotential variables are especially important. `X-averaged battery reaction overpotential [V]` captures kinetic losses associated with interfacial charge transfer. `X-averaged battery concentration overpotential [V]` captures the contribution of concentration nonuniformity to voltage loss. When the terminal voltage drops away from the OCV curve, these are two of the places to look.

The next block extracts actual spatial profiles. This is where PyBaMM begins to feel like a genuine virtual cell. We compute the total cell thickness from the negative electrode, separator, and positive electrode thicknesses. We then define a coordinate array `x_positions_m` spanning that thickness and evaluate `Electrolyte concentration [mol.m-3]` at the start and at thirty minutes. Because we pass both `x=` and `t=`, PyBaMM returns the field values on exactly the spatial slice we asked for.

The negative-particle radial profile is even closer to the theory textbook. `X-averaged negative particle concentration [mol.m-3]` is the radial diffusion field averaged across the negative electrode thickness. Evaluating it over `r_positions_m` at two times shows whether the particle interior and surface are diverging as diffusion struggles to keep up with intercalation or deintercalation.

3.3.13 Walkthrough 3 expected output

The first figure should have four panels.

The top-left panel should show terminal voltage below the battery OCV, with the gap widening modestly as the discharge proceeds. If the two curves lie exactly on top of each other, something is wrong physically: a loaded cell should not have zero losses.

The top-right panel should show the x-averaged electrolyte concentration drifting away from its initial value of roughly 1000 mol.m^{-3} , while the negative and positive particle surface concentrations move in opposite directions. Under discharge, lithium leaves the negative electrode and arrives at the positive, so the surface concentrations should reflect that directionality.

The bottom-left panel should show overpotentials in millivolts. The reaction overpotential should generally be the larger of the two over the first half hour,

with the concentration overpotential present but smaller. The exact magnitudes depend on the parameter set and current, but they should be smooth, physically plausible curves rather than noisy oscillations.

The bottom-right panel should show the electrolyte concentration profile through the cell thickness. At $t = 0$, it should be nearly flat. At $t = 30$ min, it should have developed a gradient: higher in one region, lower in another, reflecting ionic transport and source terms through the layered structure.

The second figure should show the negative-particle radial concentration profile. At the start of discharge it should be fairly flat. After thirty minutes it should be steeper, with a visible gradient between the particle center and surface. That is the DFN's solid-phase diffusion physics made visible.

3.3.14 Walkthrough 3 troubleshooting

1. nan values when evaluating a spatial profile

Symptom: the profile arrays are full of nan.

Fix: this usually means the coordinate values are outside the physical domain.

Use the parameter-set geometry values, as we did here, instead of guessing coordinate ranges.

2. Confusing full fields with averaged fields

Symptom: you interpret an X-averaged quantity as though it described a local point.

Fix: PyBaMM variable names are intentionally descriptive. Read them literally.

X-averaged means averaged through the cell thickness.

3. Overpotential units look too small or too large

Symptom: a curve appears to be only 0.01 units tall and feels meaningless.

Fix: remember the raw units are volts. For interpretability we multiplied them by 1000 to plot in millivolts.

3.3.15 Walkthrough 3 reflection

This exercise teaches the real payoff of physics-based models. You are no longer limited to asking, "what voltage does the model predict?" You can ask which physical sub-process is responsible for the voltage behavior and inspect the corresponding internal state directly.

Guided Walkthrough 4

Compare SPM , SPMe , and DFN on the Same Duty Cycle

Learning objective: Quantify the runtime-versus-fidelity tradeoff between SPM, SPMe, and DFN on a common current profile, and learn when each model is a defensible choice.

This is the modeling-judgment exercise of the chapter. Battery researchers often talk loosely about reduced-order models being “faster” or full-order models being “more accurate.” Those statements are true but not useful until you quantify them on the same task. We will do that carefully.

To avoid event-trigger effects near voltage cutoffs, we will compare all three models over a fixed 1800 s discharge at 5 A, then evaluate every model on the same reporting grid. That produces a cleaner instructional comparison than allowing each model to stop at its own terminal-voltage event.

3.3.16 Walkthrough 4 code

```
import time
import numpy as np
import matplotlib.pyplot as plt
import pybamm

def run_model_on_common_grid(model_factory, current_a, evaluation_times_s):
    parameter_values = pybamm.ParameterValues("Chen2020")
    parameter_values.update({"Current function [A]": current_a})

    simulation = pybamm.Simulation(model_factory(),
    ↪ parameter_values=parameter_values)

    # Measure runtime in the same way for every model.
    start_time = time.perf_counter()
    solution = simulation.solve(evaluation_times_s)
    runtime_s = time.perf_counter() - start_time

    # Evaluate all models on the same reporting grid before comparison.
    terminal_voltage_v = solution["Terminal voltage [V]"](evaluation_times_s)

    return {
```

```

        "runtime_s": runtime_s,
        "terminal_voltage_v": terminal_voltage_v,
    }

evaluation_times_s = np.linspace(0.0, 1800.0, 301)
current_a = 5.0

model_factories = {
    "SPM": pybamm.lithium_ion.SPM,
    "SPMe": pybamm.lithium_ion.SPMe,
    "DFN": pybamm.lithium_ion.DFN,
}

results = {}
for model_name, model_factory in model_factories.items():
    results[model_name] = run_model_on_common_grid(
        model_factory=model_factory,
        current_a=current_a,
        evaluation_times_s=evaluation_times_s,
    )

dfn_voltage_v = results["DFN"]["terminal_voltage_v"]

# Compute every reduced model's error against the DFN baseline.
for model_name in ["SPM", "SPMe"]:
    rms_error_v = np.sqrt(
        np.mean((results[model_name]["terminal_voltage_v"] - dfn_voltage_v)
        ↪ ** 2)
    )
    results[model_name]["rms_error_mV_vs_DFN"] = 1000.0 * rms_error_v

results["DFN"]["rms_error_mV_vs_DFN"] = 0.0

for model_name, result in results.items():
    print(
        f"{model_name:4s} | runtime = {result['runtime_s']:.4f} s"
        f" | RMS error vs DFN = {result['rms_error_mV_vs_DFN']:.3f} mV"
    )

time_min = evaluation_times_s / 60.0

fig, axes = plt.subplots(1, 2, figsize=(12, 5), constrained_layout=True)

```

```

for model_name, result in results.items():
    axes[0].plot(
        time_min,
        result["terminal_voltage_v"],
        linewidth=2.2,
        label=model_name,
    )

axes[0].set_title("Voltage prediction by model fidelity")
axes[0].set_xlabel("Time [min]")
axes[0].set_ylabel("Terminal Voltage [V]")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

bar_positions = np.arange(len(results))
runtime_values = [results[name]["runtime_s"] for name in ["SPM", "SPMe",
    ↪ "DFN"]]
error_values = [results[name]["rms_error_mV_vs_DFN"] for name in ["SPM",
    ↪ "SPMe", "DFN"]]

axes[1].bar(bar_positions - 0.18, runtime_values, width=0.36, label="Runtime
    ↪ [s]")
axes[1].bar(bar_positions + 0.18, error_values, width=0.36, label="RMS error
    ↪ [mV]")
axes[1].set_xticks(bar_positions)
axes[1].set_xticklabels(["SPM", "SPMe", "DFN"])
axes[1].set_title("Runtime versus DFN-relative error")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.show()

```

3.3.17 Walkthrough 4 explanation

The helper function `run_model_on_common_grid` exists for one reason: it forces the comparison to be methodologically clean. Every model receives the same current and the same evaluation grid. The only thing that changes is the model fidelity.

Inside that helper, we update the `Chen2020` parameter set with a fixed current of 5 A. That update pattern is the first one you should internalize for later parameter-study work. We are not editing PyBaMM source code. We are modifying a parameter object.

`time.perf_counter()` is a practical runtime-measurement tool. The absolute numbers you see will depend on your machine, operating system, and whether this is the first solve in a fresh kernel. The relative ordering is the part that matters scientifically.

After running all three models, we compute an RMS voltage error against the DFN:

$$\text{RMS}(m) = \sqrt{\frac{1}{N} \sum_{i=1}^N (V_m(t_i) - V_{\text{DFN}}(t_i))^2}. \quad (5)$$

This is not the only possible accuracy metric, but it is a transparent and defensible first one. We express it in millivolts because that is the scale on which battery-model voltage differences become intuitively interpretable.

The two-panel figure then makes the tradeoff visible: the left plot shows voltage predictions themselves, while the right plot puts runtime and error side by side.

3.3.18 Walkthrough 4 expected output

On the pinned version, a representative run is:

```
SPM | runtime = 0.2527 s | RMS error vs DFN = 54.177 mV
SPMe | runtime = 0.5038 s | RMS error vs DFN = 3.826 mV
DFN | runtime = 1.3298 s | RMS error vs DFN = 0.000 mV
```

Your runtimes will vary, sometimes substantially, but the ordering should remain the same: SPM fastest, DFN slowest, SPMe in between. The accuracy trend should also be stable: SPMe should sit much closer to the DFN than the plain SPM on this moderate-rate discharge.

In the left plot, the SPMe voltage curve should be visually almost on top of the DFN curve, while the SPM should show a visibly larger deviation, especially as the discharge proceeds. In the right plot, the bars should make the core lesson obvious: the SPMe buys a large error reduction at a modest runtime penalty relative to SPM, while still remaining faster than the DFN.

3.3.19 Walkthrough 4 interpreting the modeling choice

This is where we connect the software output back to research judgment.

Use DFN when internal-state fidelity matters, when you need mechanistic interpretation, when you are testing parameter sensitivity in a way that depends

on electrolyte dynamics, or when you need a high-fidelity virtual cell to generate synthetic data for another workflow.

Use SPMe when you still care about electrolyte effects but need cheaper repeated solves. This is often the sweet spot for parameter studies, control-oriented simulation, and mid-fidelity research questions.

Use SPM when you need speed more than detailed fidelity and when the operating conditions are gentle enough that electrolyte limitations are not the dominant story. That last phrase matters. SPM is not “the beginner model.” It is a deliberate approximation with a domain of validity.

3.3.20 Walkthrough 4 troubleshooting

1. Treating runtime as a universal benchmark

Symptom: you compare your exact runtime numbers to a colleague’s machine or to this chapter and overinterpret the difference.

Fix: use runtime comparatively within one environment, not as a universal hardware-independent truth.

2. Comparing models on different time grids

Symptom: you accidentally compare arrays with different sampling or different terminal events.

Fix: evaluate every model on a common reporting grid before computing an error metric.

3. Declaring the SPMe “always better” without qualification

Symptom: you learn the ranking from this example and generalize too far.

Fix: model choice depends on operating regime, chemistry, question, and acceptable error. This walkthrough is a benchmark, not a universal law.

3.3.21 Walkthrough 4 reflection

This exercise teaches a habit that publishable work depends on: model fidelity should be chosen by evidence and task fit, not by prestige. The “best” battery model is the one that is accurate enough for the question while still practical enough for the workflow.

Dataset Integration

Parsing a Public Battery Dataset Before Chapter 4

This chapter is primarily about your first PyBaMM simulation, not about full experiment scripting. That deeper workflow belongs in Chapter 4. But battery-modeling competence also requires comfort with public experimental data, so we will do one light but real dataset integration here.

We will use the **Oxford Energy Trading Battery Degradation Dataset**:

- Dataset landing page:
<https://ora.ox.ac.uk/objects/uuid:9aae61af-2949-49f1-8ad5-6aea448979e>
- DOI: 10.5287/bodleian:gJPdDzvP4
- Example file for this chapter: `SPM_cell1_profileData.csv`
- File size: approximately 2.9 MB
- Format: CSV without a header row
- License: ODC Open Database License (ODbL)

The Oxford `Readme.txt` explains that `SPM_cell1_profileData.csv` contains five columns:

1. profile time in seconds
2. current in amperes, with **negative meaning charging**
3. cell voltage in volts
4. cell surface temperature in degrees Celsius, where 0 means missing
5. ambient temperature in degrees Celsius, where 0 means missing

That sign convention is the first pitfall to notice. In many BMS and ECM contexts, discharge current is treated as positive. In this Oxford dataset, charging is negative. You must always decide whether to keep the native sign convention or map it to the one used in your model. For this chapter we will keep the dataset unchanged and only compute helper columns.

3.3.22 Dataset parsing code

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

csv_path = Path("data/raw/SPM_cell1_profileData.csv")
```

```

# The Oxford file has no header row, so we assign names manually.
profile_df = pd.read_csv(
    csv_path,
    header=None,
    names=[
        "profile_time_s",
        "current_a",
        "voltage_v",
        "surface_temperature_c",
        "ambient_temperature_c",
    ],
)

# Convert missing temperatures encoded as zero into proper NaN values.
profile_df["surface_temperature_c"] =
    ↪ profile_df["surface_temperature_c"].replace(0, np.nan)
profile_df["ambient_temperature_c"] =
    ↪ profile_df["ambient_temperature_c"].replace(0, np.nan)
profile_df["profile_time_h"] = profile_df["profile_time_s"] / 3600.0
profile_df["discharge_current_a"] = -profile_df["current_a"]

print(profile_df.head())
print(profile_df.describe())

window_df = profile_df.iloc[:2000].copy()

fig, axes = plt.subplots(3, 1, figsize=(10, 9), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(window_df["profile_time_h"], window_df["current_a"],
    ↪ linewidth=1.5)
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Oxford SPM cell profile data: first window")
axes[0].grid(True, alpha=0.3)

axes[1].plot(window_df["profile_time_h"], window_df["voltage_v"],
    ↪ linewidth=1.5, color="tab:orange")
axes[1].set_ylabel("Voltage [V]")
axes[1].grid(True, alpha=0.3)

axes[2].plot(
    window_df["profile_time_h"],

```

```

        window_df["surface_temperature_c"],
        linewidth=1.5,
        label="Surface temperature",
    )
axes[2].plot(
    window_df["profile_time_h"],
    window_df["ambient_temperature_c"],
    linewidth=1.5,
    label="Ambient temperature",
)
axes[2].set_ylabel("Temperature [°C]")
axes[2].set_xlabel("Profile Time [h]")
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.show()

```

3.3.23 How to read the parsed data

The `header=None` argument matters because the file does not ship with column names. If you forget that, Pandas will quietly treat the first data row as a header and your whole dataset will shift.

Replacing zeros in the temperature columns with `NaN` is a small but important data-cleaning move. The Oxford readme explicitly states that `0` indicates a missing measurement, not a real `0 °C` environment. If you skip that replacement, later averages and plots become physically misleading.

`discharge_current_a = -current_a` does not alter the raw data. It creates a convenience column in the sign convention many battery models use. That is a good research habit. Preserve the raw field, add a translated field, and document the choice.

3.3.24 Dataset parsing expected output

The `head()` output should show five raw numeric columns plus the helper columns we added. The plotted window should show a stepped current trace, a corresponding voltage trace constrained within the Kokam cell's operating window, and temperature curves that move much more slowly than current or voltage.

The important conceptual lesson is simple: real battery files arrive with conventions and quirks. Even before we drive a PyBaMM simulation with public

profiles in Chapter 4, we need to be able to read the file, label the units, handle missing values correctly, and state the sign convention without hesitation.

Guided Walkthrough 5

Reproduce a Published Figure from Marquis et al. (2019)

Learning objective: Reproduce the qualitative structure of a published SPMe-versus-DFN error-decomposition figure using PyBaMM's `Marquis2019` parameter set, while documenting the modeling choices and ambiguities honestly.

This is the highest-value exercise in the chapter. The paper is:

Scott G. Marquis, Valentin Sulzer, Robert Timms, Colin P. Please, and S. Jon Chapman, “An asymptotic derivation of a single particle model with electrolyte,” *Journal of The Electrochemical Society*, 166(15), A3693-A3706, 2019. DOI: [10.1149/2.0341915jes](https://doi.org/10.1149/2.0341915jes).

The target is **Figure 3** in the accepted manuscript hosted by the Oxford University Research Archive. In the paper, that figure shows component-wise voltage errors between the SPMe and the DFN during a 3C discharge.

We need to be transparent about three facts before we begin.

First, PyBaMM now ships a built-in `Marquis2019` parameter set and a modern SPMe implementation, which is exactly what makes this exercise feasible on a laptop.

Second, the paper's notation for the voltage decomposition does not map one-to-one onto PyBaMM variable names in a way the authors spelled out as a modern notebook-ready script. So we must make a principled mapping.

Third, the paper terminated the comparison at 3.2 V, while the bundled `Marquis2019` lower-voltage cutoff inside PyBaMM differs. We will therefore impose the paper's 3.2 V cutoff explicitly in the experiment.

That is what honest reproduction looks like. We do not hide the ambiguities. We surface them and document the choices.

3.3.25 The variable mapping we will use

This is an informed mapping from the paper's plotted components to PyBaMM variables:

Paper notation	PyBaMM variable used here	Why this is a defensible mapping
$\bar{U}_{eq}$	Battery open-circuit voltage [V]	Represents the equilibrium open-circuit contribution
$\bar{\eta}_r$	X-averaged battery reaction overpotential [V]	Represents kinetic charge-transfer losses
$\tilde{\Delta\Phi}_{Elec}$	X-averaged battery electrolyte ohmic losses [V]	Represents electrolyte-phase voltage drop
$\bar{\eta}_c$	X-averaged battery concentration overpotential [V]	Represents concentration-driven voltage loss

This mapping is an inference from the paper and PyBaMM’s documented variable names. It is not a claim that we are using the authors’ original private post-processing code line for line.

3.3.26 Walkthrough 5 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

parameter_values = pybamm.ParameterValues("Marquis2019")
electrode_area_m2 = (
    parameter_values["Electrode height [m]"] * parameter_values["Electrode
↳ width [m]"]
)

# Match the paper's comparison regime explicitly.
experiment = pybamm.Experiment(["Discharge at 3C until 3.2 V"])

models = {
    "SPMe": pybamm.lithium_ion.SPMe(),
    "DFN": pybamm.lithium_ion.DFN(),
}
```

```

component_variables = {
    "Battery OCV error": "Battery open-circuit voltage [V]",
    "Reaction overpotential error": "X-averaged battery reaction
    ↪ overpotential [V]",
    "Electrolyte ohmic error": "X-averaged battery electrolyte ohmic losses
    ↪ [V]",
    "Concentration overpotential error": "X-averaged battery concentration
    ↪ overpotential [V]",
}

results = {}

for model_name, model in models.items():
    simulation = pybamm.Simulation(
        model,
        parameter_values=pybamm.ParameterValues("Marquis2019"),
        experiment=experiment,
    )
    solution = simulation.solve()

    result = {
        "time_s": solution["Time [s]"].entries,
        "discharge_capacity_ah_per_m2": (
            solution["Discharge capacity [A.h]"].entries / electrode_area_m2
        ),
    }

    for label, variable_name in component_variables.items():
        result[label] = solution[variable_name].entries

    results[model_name] = result

# Align the comparison on a common time axis before subtraction.
common_time_s = np.linspace(
    0.0,
    min(results["SPMe"]["time_s"][-1], results["DFN"]["time_s"][-1]),
    400,
)

capacity_axis_ah_per_m2 = np.interp(
    common_time_s,
    results["DFN"]["time_s"],
    results["DFN"]["discharge_capacity_ah_per_m2"],

```

```

)

component_errors_v = {}

for label in component_variables:
    spme_component = np.interp(
        common_time_s,
        results["SPMe"]["time_s"],
        results["SPMe"][label],
    )
    dfn_component = np.interp(
        common_time_s,
        results["DFN"]["time_s"],
        results["DFN"][label],
    )
    component_errors_v[label] = spme_component - dfn_component
    print(
        label,
        "maximum absolute error [mV] =",
        round(float(np.max(np.abs(component_errors_v[label])) * 1000.0), 3),
    )

fig, ax = plt.subplots(figsize=(9, 5.5), constrained_layout=True)

for label, error_v in component_errors_v.items():
    ax.plot(
        capacity_axis_ah_per_m2,
        error_v * 1000.0,
        linewidth=2.0,
        label=label,
    )

ax.axhline(0.0, color="black", linewidth=0.8, alpha=0.5)
ax.set_xlabel("Discharge Capacity [A.h m$^{-2}$]")
ax.set_ylabel("SPMe - DFN component error [mV]")
ax.set_title("Reproduction of Marquis et al. (2019), Figure 3 structure")
ax.legend()
ax.grid(True, alpha=0.3)

plt.show()

```

3.3.27 Walkthrough 5 explanation

We begin with `ParameterValues("Marquis2019")`, because PyBaMM includes a parameter set specifically tied to the Marquis SPMe work. That is exactly the kind of alignment you want when reproducing a paper: use the paper-linked parameterization when the tool makes it available.

The electrode area is important because the paper's x-axis is expressed in discharge capacity per unit area. PyBaMM gives us discharge capacity in ampere-hours, so we divide by the electrode area to match the paper's scale:

$$Q_{\text{area}} = \frac{Q_{\text{discharged}}}{A_{\text{electrode}}}. \quad (6)$$

We then define an explicit experiment: `Discharge at 3C until 3.2 V`. The reason to specify the voltage cutoff directly instead of relying on defaults is methodological honesty. The paper used 3.2 V, and we want to follow that published choice.

For each model, we solve once and store two types of data: the discharge-capacity axis and the four voltage components we mapped from the paper notation.

Because the SPMe and DFN do not stop at exactly identical times, we create `common_time_s` and interpolate both solutions onto that shared interval. This matters. Comparing the raw arrays directly would be sloppy because the models do not report on identical internal time grids.

Finally, we subtract the DFN component from the SPMe component. The resulting curves are not the total voltage error; they are the component-wise differences underlying that total error.

3.3.28 Walkthrough 5 expected output

On the pinned version, representative maximum absolute component errors are:

```
Battery OCV error maximum absolute error [mV] = 8.29
Reaction overpotential error maximum absolute error [mV] = 8.666
Electrolyte ohmic error maximum absolute error [mV] = 5.212
Concentration overpotential error maximum absolute error [mV] = 4.0
```

The plot should show four error curves living close to zero over most of the discharge, then separating more noticeably as the discharge approaches its end. The exact magnitudes and line ordering may not match the paper pixel for pixel,

but the qualitative story should: the SPMe tracks the DFN well through most of the discharge, and the component-level error grows as the system approaches the harder, end-of-discharge regime.

3.3.29 Walkthrough 5 reproduction tolerance

This is the part many reproduction exercises fail to state clearly.

For this figure, “close enough” does **not** mean that every curve must match the published accepted-manuscript figure at the pixel level. We are using a modern PyBaMM implementation, modern defaults, and an inferred mapping between paper notation and exposed software variables. It is entirely reasonable for the exact magnitudes to differ.

For this figure, “close enough” means:

1. the same experiment definition is used
2. the same model pair is compared
3. the x-axis scaling is aligned with the paper
4. the qualitative trend is recovered: small component errors for most of the discharge and stronger divergence near the end
5. the methodological differences are written down, not hidden

That is a rigorous reproduction standard for a laptop-based methods chapter.

3.3.30 Walkthrough 5 troubleshooting

1. You compare raw arrays without interpolation

Symptom: the subtraction fails or produces misleading results because the models stopped at different times.

Fix: always align the comparison axis first.

2. You use the `Chen2020` parameter set by habit

Symptom: the result no longer meaningfully targets the Marquis paper.

Fix: reproduction work should use the paper-linked parameterization whenever possible.

3. You claim exact quantitative agreement where you only have qualitative agreement

Symptom: the write-up sounds more certain than the method deserves.

Fix: be explicit about any inferred mapping or version drift. This is part of good research practice, not an admission of failure.

3.3.31 Walkthrough 5 reflection

This exercise teaches reproduction as a research method rather than a classroom puzzle. You identified a target figure, matched the experiment definition, found a defensible parameter set, documented a variable-mapping choice, handled nonidentical solution grids correctly, and defined a reasonable success criterion. That is publishable-work behavior.

3.4 Open-Ended Exercises

These exercises ask you to modify the guided code rather than merely rerun it. Try them before reading the worked solutions at the end of the chapter.

3.4.1 Exercise 1: How does model ranking change with current?

Repeat Guided Walkthrough 4 for 2.5 A, 5.0 A, and 10.0 A. Which model fidelity changes its error most dramatically as current increases? Does runtime change much relative to error?

Hints: keep the same comparison grid structure; put the current values in a loop; store RMS error versus DFN for each model and current.

3.4.2 Exercise 2: How sensitive is the DFN to mesh resolution?

Using the low-level pipeline from Guided Walkthrough 1, compare a coarse mesh of 5 points per domain, the teaching mesh of 10 points per domain, and a finer mesh of 20 points per domain. Report the runtime and terminal voltage at 600 s.

Hints: the only thing you need to change is `var_pts`; keep the model, parameters, and time grid the same; compare coarse and fine solutions against the 20-point run.

3.4.3 Exercise 3: What happens if you compare Marquis2019 and Chen2020 under the same model?

Run a DFN with Chen2020 and a DFN with Marquis2019 on the same nominal 1C discharge for 1800 s. Compare terminal voltage and x-averaged electrolyte concentration. What differences are due to chemistry and parameterization rather than model fidelity?

Hints: use the same code pattern as Guided Walkthrough 3; do not change the

model, only the parameter set; remember that the two cells have different nominal capacities and geometry.

3.5 Worked Solutions to the Open-Ended Exercises

3.5.1 Solution to Exercise 1

```
import time
import numpy as np
import pybamm

def compare_models_at_current(current_a):
    evaluation_times_s = np.linspace(0.0, 1800.0, 301)
    model_factories = {
        "SPM": pybamm.lithium_ion.SPM,
        "SPMe": pybamm.lithium_ion.SPM_e,
        "DFN": pybamm.lithium_ion.DFN,
    }

    results = {}
    for model_name, model_factory in model_factories.items():
        parameter_values = pybamm.ParameterValues("Chen2020")
        parameter_values.update({"Current function [A]": current_a})
        simulation = pybamm.Simulation(model_factory(),
        ↪ parameter_values=parameter_values)

        start_time = time.perf_counter()
        solution = simulation.solve(evaluation_times_s)
        runtime_s = time.perf_counter() - start_time

        results[model_name] = {
            "runtime_s": runtime_s,
            "voltage_v": solution["Terminal voltage
            ↪ [V]"](evaluation_times_s),
        }

    dfn_voltage_v = results["DFN"]["voltage_v"]
    for model_name in ["SPM", "SPMe"]:
        rms_error_v = np.sqrt(np.mean((results[model_name]["voltage_v"] -
        ↪ dfn_voltage_v) ** 2))
        results[model_name]["rms_error_mV"] = 1000.0 * rms_error_v
```



```

results["DFN"]["rms_error_mV"] = 0.0
return results

for current_a in [2.5, 5.0, 10.0]:
    current_results = compare_models_at_current(current_a)
    print(f"\nCurrent = {current_a:.1f} A")
    for model_name in ["SPM", "SPMe", "DFN"]:
        print(
            f"{model_name:4s} | runtime =
            ↳ {current_results[model_name]['runtime_s']:.4f} s"
            f" | RMS error =
            ↳ {current_results[model_name]['rms_error_mV']:.3f} mV"
        )

```

What you should find is that the SPM error grows much faster with current than the SPMe error. That is the point. As electrolyte limitations become more important, the plain single-particle approximation loses accuracy faster than the electrolyte-aware reduced model does.

3.5.2 Solution to Exercise 2

```

import time
import numpy as np
import pybamm

def run_manual_dfn_with_mesh(points_per_domain):
    model = pybamm.lithium_ion.DFN()
    parameter_values = pybamm.ParameterValues("Chen2020")
    geometry = model.default_geometry

    parameter_values.process_model(model)
    parameter_values.process_geometry(geometry)

    var_pts = {
        "x_n": points_per_domain,
        "x_s": points_per_domain,
        "x_p": points_per_domain,
        "r_n": points_per_domain,
        "r_p": points_per_domain,
    }

    mesh = pybamm.Mesh(geometry, model.default_submesh_types, var_pts)

```

```

discretisation = pybamm.Discretisation(mesh,
↳ model.default_spatial_methods)
discretisation.process_model(model)

solver = model.default_solver
evaluation_times_s = np.linspace(0.0, 600.0, 51)

start_time = time.perf_counter()
solution = solver.solve(model, evaluation_times_s)
runtime_s = time.perf_counter() - start_time

terminal_voltage_v = solution["Terminal voltage [V]"](evaluation_times_s)
return runtime_s, float(terminal_voltage_v[-1])

for points in [5, 10, 20]:
    runtime_s, final_voltage_v = run_manual_dfn_with_mesh(points)
    print(
        f"points per domain = {points:2d} | runtime = {runtime_s:.4f} s"
        f" | voltage at 600 s = {final_voltage_v:.4f} V"
    )

```

You should see the finer mesh take longer. The final voltage should converge rather than wander wildly. If it does wander wildly, that is a sign you either changed more than the mesh or ran into a setup mistake.

3.5.3 Solution to Exercise 3

```

import numpy as np
import matplotlib.pyplot as plt
import pybamm

evaluation_times_s = np.linspace(0.0, 1800.0, 301)
parameter_sets = ["Chen2020", "Marquis2019"]
results = {}

for parameter_set_name in parameter_sets:
    parameter_values = pybamm.ParameterValues(parameter_set_name)
    simulation = pybamm.Simulation(
        pybamm.lithium_ion.DFN(),
        parameter_values=parameter_values,
    )
    solution = simulation.solve(evaluation_times_s)

```

```

results[parameter_set_name] = {
    "terminal_voltage_v": solution["Terminal voltage
    ↪ [V]"](evaluation_times_s),
    "xavg_electrolyte_concentration": solution[
        "X-averaged electrolyte concentration [mol.m-3]"
    ](evaluation_times_s),
}

time_min = evaluation_times_s / 60.0

fig, axes = plt.subplots(2, 1, figsize=(9, 8), sharex=True,
    ↪ constrained_layout=True)

for parameter_set_name in parameter_sets:
    axes[0].plot(
        time_min,
        results[parameter_set_name]["terminal_voltage_v"],
        linewidth=2.0,
        label=parameter_set_name,
    )
    axes[1].plot(
        time_min,
        results[parameter_set_name]["xavg_electrolyte_concentration"],
        linewidth=2.0,
        label=parameter_set_name,
    )

axes[0].set_ylabel("Terminal Voltage [V]")
axes[0].set_title("Same DFN model, different parameter sets")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].set_ylabel("X-avg Electrolyte Concentration [mol.m-3]")
axes[1].set_xlabel("Time [min]")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.show()

```

The main lesson is that “the model” is not the whole story. A DFN with one parameter set is not interchangeable with a DFN using another. Parameterization changes operating window, dynamic losses, concentration gradients, and therefore the interpretation of any later fitted or benchmarked result.

What Changes for Sodium-Ion?

Early in the companion, this section is intentionally brief, but it matters.

The PyBaMM workflow itself changes very little for sodium-ion. The model object, the parameter-value pattern, the solve-extract-plot rhythm, and the fidelity comparison logic all transfer directly. What changes is the chemistry-specific content that fills those objects.

First, sodium-ion parameter sets are much sparser than lithium-ion parameter sets in the public PyBaMM ecosystem. That means you will often need to build or adapt your own `ParameterValues` object from literature rather than relying on a bundled cell.

Second, sodium-ion open-circuit curves, especially for hard-carbon negative electrodes, can be flatter and more structured than typical lithium-ion graphite curves. That affects both the interpretation of terminal-voltage differences and the difficulty of parameter estimation and state estimation later in the manual.

Third, validation is harder because public sodium-ion datasets are far thinner. In practice, you will often validate one part of a workflow against lithium-ion public data to test the method, then port the same method to sodium-ion with literature parameter values and more limited experimental references.

Fourth, low-temperature behavior and transport-property choices become more central sooner in sodium-ion studies. That means the distinction between SPM, SPMe, and DFN may matter differently depending on the sodium-ion question. Do not assume a ranking observed on a lithium-ion teaching benchmark transfers unchanged.

For now, the key lesson is this: Chapter 3 teaches the transferable software method. Later chapters will show how to swap in sodium-ion-specific assumptions honestly.

Chapter Summary and Skill Checklist

- You installed and verified `pybamm==2023.12.1` in a controlled research environment.
- You connected Textbook Chapter 8's DFN equations to PyBaMM's object hierarchy: model, parameters, geometry, mesh, discretization, solver, and

simulation.

- You ran a default DFN simulation with the `Chen2020` parameter set and extracted terminal voltage, open-circuit voltage, discharge capacity, and approximate SOC.
- You inspected internal states including electrolyte concentration, particle concentration, and overpotential contributions.
- You compared SPM, SPMe, and DFN on a common current profile and quantified runtime versus DFN-relative voltage error.
- You parsed a public battery dataset and handled headerless columns, missing temperatures, and current-sign conventions correctly.
- You reproduced the structure of a published SPMe-versus-DFN figure from Marquis et al. (2019) while documenting the choices and ambiguities honestly.

The commands, functions, and patterns that should now be entering muscle memory are:

- `pybamm.lithium_ion.DFN()`
- `pybamm.lithium_ion.SPM()`
- `pybamm.lithium_ion.SPMe()`
- `pybamm.ParameterValues("Chen2020")`
- `parameter_values.update({...})`
- `pybamm.Simulation(...)`
- `solution["Variable name [units]"](time_grid)`
- spatial evaluation patterns such as `solution["Electrolyte concentration [mol.m-3]"](x=x_positions, t=1800.0)`
- clean benchmarking on a common reporting grid

You should now be able to answer “yes” to each of these:

- I can install PyBaMM in the environment built in Lab Chapter 1 and verify that it really works.
- I can explain the difference between a PyBaMM model object and a parameter set.
- I can run a DFN simulation without copying code blindly.
- I can extract internal variables rather than only terminal voltage.
- I can explain, with evidence, when SPM, SPMe, or DFN is the better choice for a given modeling task.

- I can reproduce a published figure honestly enough to document version drift and modeling choices instead of hiding them.

If any of those boxes are still unchecked, revisit the relevant walkthrough before moving into Chapter 4. Chapter 4 assumes this chapter feels natural.

Deliverable

The deliverable for this chapter is:

A notebook running all three model fidelities, comparing runtime and accuracy, with a written interpretation of when you'd choose which.

Approach it in three sections.

First, include a clean benchmark section that reproduces the SPM versus SPMe versus DFN comparison on a common time grid. Use the code pattern from Guided Walkthrough 4 and save both the voltage-comparison figure and the runtime-versus-error figure.

Second, include one internal-state section for the DFN, similar to Guided Walkthrough 3. At minimum, plot terminal voltage, x-averaged electrolyte concentration, and one particle-concentration profile. The point is to demonstrate that you understand what extra physical visibility the DFN buys you.

Third, write a short interpretation paragraph. It should not say only “DFN is most accurate.” It should state when the extra fidelity is worth the runtime and when SPMe or SPM would be the better research choice.

A strong partial deliverable looks like this:

- one notebook file with all code cells runnable top to bottom
- one saved figure comparing voltage traces for the three fidelities
- one saved figure comparing runtime and RMS error
- one paragraph explaining which model you would use for parameter studies, which for synthetic-data generation, and which for quick exploratory sweeps

Further Practice and Reading

1. PyBaMM documentation home: <https://docs.pybamm.org/en/stable/>

Bookmark this. It is the authoritative source for current PyBaMM APIs and notebooks.

2. PyBaMM parameter-set documentation:

[https://docs.pybamm.org/en/stable/source/api/parameters/parameter_](https://docs.pybamm.org/en/stable/source/api/parameters/parameter_sets.html)

This is the page to revisit whenever you want to know what bundled parameter sets exist and how they are described.

3. Marquis et al. (2019), “An asymptotic derivation of a single particle model with electrolyte,” DOI [10.1149/2.0341915jes](https://doi.org/10.1149/2.0341915jes)

This is the paper behind the SPM reproduction exercise and one of the central references for reduced-order physics-based battery modeling.

4. Chen et al. (2020), “Development of Experimental Techniques for Parameterization of Multi-scale Lithium-ion Battery Models,” DOI [10.1149/1945-7111/ab9050](https://doi.org/10.1149/1945-7111/ab9050)

This is the reference behind the `Chen2020` parameter set used heavily in teaching and benchmarking.

5. Oxford Energy Trading Battery Degradation Dataset, DOI [10.5287/bodleian:gJPdDzvP4](https://doi.org/10.5287/bodleian:gJPdDzvP4)

Keep this bookmarked. We only touched it lightly here, but Chapter 4 and later dataset-centric chapters will benefit from your early familiarity with its structure.

Parameters, Experiments, and Drive Cycles

Chapter 3 taught you how to run PyBaMM. This chapter teaches you how to *control* it. That sounds like a small step, but it is the difference between replaying a canned example and building a research workflow. By the end of this chapter, you will know how to inspect and modify parameter sets without losing provenance, replace function-valued parameters such as OCV curves and diffusivity laws, encode multi-step experimental protocols with charge, discharge, rest, and CV holds, import a real drive-cycle current trace, and decide when an isothermal simulation is good enough and when thermal coupling is worth the cost.

This matters for publishable research because most battery-modeling papers do not fail at the level of the governing equations. They fail in the translation layer between theory and execution. A parameter value is copied from the wrong chemistry. A charge-rest-discharge protocol is described vaguely and implemented differently than stated. A drive cycle is used without documenting current sign conventions. A thermal result is reported without saying whether temperature was held fixed or solved dynamically. Reviewers notice those things because they are exactly the places where otherwise good battery papers become irreproducible.

Keep Textbook Chapter 8 open while you work. That chapter gave you the porous-electrode equations and the physical meaning of OCV, overpotential, diffusion limitation, and transport coupling. This lab operationalizes that chapter. When we swap an OCV function or slow a diffusivity law, we are not “tweaking software.” We are changing a constitutive relation inside the electrochemical model you already learned in theory. Keep Textbook Chapter 10 open as well. That chapter explained why test protocols, current excitation, and duty-cycle choice determine what a battery model can and cannot reveal. This lab turns those ideas into executable experiments.

The deeper point is that a battery model is never just “the DFN” or “the SPMe.” It is always the model *plus* a parameterization *plus* an operating protocol *plus* a solver configuration. Serious simulation work is the craft of managing all four honestly. If Chapter 3 taught you how to ask PyBaMM for a voltage curve, Chapter 4 teaches you how to justify the curve you asked for.

This chapter also matters directly for sodium-ion work. Early sodium-ion studies often reuse lithium-ion software workflows because the tooling is more mature on the lithium-ion side. That is sensible, but only if you understand which parts of the workflow are chemistry-agnostic and which are not. Parameter handling, experiment construction, drive-cycle import, and thermal-vs-isothermal judgment all transfer cleanly. OCV shape, transport functions, nominal voltage, and data availability do not. We will keep making that distinction explicit.

The chapter is structured as a gradual climb. We begin with the parameter set itself: how to inspect it, compare it, modify it, and save your changes in a way your future self can still trust. Next, we replace function-valued parameters, because this is where PyBaMM stops feeling like a library of built-ins and starts behaving like a research tool. Then we move to the `Experiment` class and build a full charge-rest-discharge workflow. After that, we bring in a public UDDS current profile and run it through a model under realistic dynamic forcing. Finally, we revisit temperature, first as a fixed input and then as a coupled state, and quantify what changes.

Part II already satisfied its formal reproduce-a-published-figure requirement in Chapter 3, but this chapter still includes a literature-backed reproduction exercise because it is too valuable to skip. We will recreate the six-step cycling structure reported by Pozzato, Allam, and Onori (2022) for an EV-style ageing protocol and

document exactly where we are matching the paper, where we are approximating, and what “close enough” means.

If you work carefully through this chapter, you will finish with a genuinely reusable notebook pattern: parameter provenance at the top, protocol definition in the middle, public data ingestion on the left edge, and publication-quality interpretation at the end. That is the skeleton of a real battery paper.

4.1 Prerequisites Check

- Required software: the `sib-research` environment from Lab Chapter 1, Python 3.11+, JupyterLab 4.4+, Git
- Required Python packages for this chapter: `pybamm==25.12.1`, `numpy`, `pandas`, `scipy`, `matplotlib`
- Optional but recommended package for later file formats: `openpyxl`
- Required textbook chapters: Textbook Chapter 8 is essential; Textbook Chapter 10 is strongly recommended
- Required prior lab chapters: Lab Chapter 1, Lab Chapter 2, and Lab Chapter 3
- Estimated time: 12 to 16 hours if Chapter 3 felt comfortable; 16 to 20 hours if parameter handling and protocol design are new to you

If the object model from Chapter 3 still feels shaky, revisit Sections 3.3 through 3.6 before you begin. If Python plotting from Lab Chapter 2 still feels awkward, revisit Sections 2.4 and 2.5. This chapter assumes you can already read a NumPy array, build a clean Matplotlib figure, and recognize when a result is physically implausible.

4.2 Environment Setup

We will use the same pinned PyBaMM version as Chapter 3 so that parameter introspection, `Experiment` behavior, and drive-cycle examples all stay aligned.

4.2.1 Activate the research environment

If you are using `conda`:

```
conda activate sib-research
```

If you are using `venv` on Linux or macOS:

```
source .venv/bin/activate
```

If you are using `venv` on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

4.2.2 Install the chapter dependencies

If you already completed Chapter 3 successfully, you may already have everything you need. If not, install the pinned stack explicitly:

```
python -m pip install pybamm==25.12.1 numpy pandas scipy matplotlib openpyxl
```

If your shell does not expose `python`, use:

```
python3 -m pip install pybamm==25.12.1 numpy pandas scipy matplotlib openpyxl
```

The `openpyxl` package is not needed for the core guided walkthroughs in this chapter, but it becomes useful the moment you start pulling `.xlsx` battery files from public repositories. Installing it now saves you a needless interruption later.

4.2.3 Verify the install with a minimal “hello protocol”

Run this snippet in a terminal-backed Python session or a fresh notebook cell:

```
import pybamm

params = pybamm.ParameterValues("Chen2020")

print("PyBaMM version:", pybamm.__version__)
print("Nominal capacity [A.h]:", params["Nominal cell capacity [A.h]"])
print("Thermal parameter count:", len(params.list_by_category("thermal")))
print("Drive-cycle helper available:", hasattr(pybamm, "DataLoader"))
```

Expected output on the pinned version is:

```
PyBaMM version: 25.12.1
Nominal capacity [A.h]: 5.0
Thermal parameter count: 13
Drive-cycle helper available: True
```

The exact order of the printed lines may differ if you add extra checks, but the version, nominal capacity, and thermal-parameter count should match.

4.2.4 Confirm Jupyter is using the correct interpreter

Launch JupyterLab:

```
jupyter lab
```

Then run:

```
import sys
import pybamm

print(sys.executable)
print(pybamm.__version__)
```

The interpreter path should point into your intended environment, and the version should again print 25.12.1.

4.2.5 Common install failures and their fixes

1. `AttributeError: 'ParameterValues' object has no attribute 'get_info'`

Symptom: code from this chapter works in the book but not in your notebook.

Fix: you are almost certainly on an older PyBaMM version. Reinstall with the exact pin `pybamm==25.12.1`.

2. `ModuleNotFoundError: No module named 'pybamm'` in Jupyter but not in the terminal

Symptom: the import works in one place and fails in another.

Fix: your notebook kernel is bound to the wrong environment. Re-register the active environment kernel and switch notebooks to it.

3. A drive-cycle file download fails when calling `pybamm.DataLoader()`

Symptom: PyBaMM imports correctly, but the first call to `get_data("UDDS.csv")` errors.

Fix: this is usually a transient network or firewall issue. Retry once. If it persists, open the PyBaMM data release URL in a browser and download the file manually to a local folder, then point `pandas.read_csv` at that path.

4. Solver warnings appear on your first drive-cycle solve

Symptom: PyBaMM complains about evaluating outside the drive-cycle time range or about step-size rejection.

Fix: read the warning before panicking. On dynamic current profiles, some rejected trial steps are normal. If the solve completes and the result looks

physical, the warning is often benign.

4.3 Conceptual Bridge: What the Tool Thinks a “Parameter” and an “Experiment” Are

In Textbook Chapter 8, a porous-electrode model looked like a set of governing equations plus constitutive laws. In practice, that means a battery simulation is assembled from at least three logically distinct pieces:

1. the conservation laws and algebraic constraints,
2. the parameter functions and constants that close those equations,
3. the operating condition imposed on the cell.

PyBaMM mirrors that separation very deliberately, and if you internalize that separation now, later chapters become much easier.

The model itself is the equation structure. In Chapter 3, we instantiated DFN, SPMe, and SPM objects. Those objects determine what states exist, what PDEs or ODEs are solved, and which constitutive relations the model expects. The parameter set is not the same thing. A `ParameterValues` object supplies numerical values and callable functions for names such as `Negative electrode OCP [V]`, `Positive particle diffusivity [m2.s-1]`, and `Current function [A]`. The experiment is not the same thing either. An experiment is a time program for the external boundary condition: discharge here, rest there, hold at this voltage until that current, then apply an oscillatory current trace from data.

That separation maps very naturally onto the terminal-voltage decomposition you learned in theory. Under load, the measured voltage is not just an OCV lookup. It is the result of equilibrium potentials, kinetic losses, transport losses, and ohmic losses combining under a particular applied current history:

$$V(t) = U_p(\theta_p(t)) - U_n(\theta_n(t)) - \eta_{\text{rxn}}(t) - \eta_{\text{ohm}}(t) - \eta_{\text{conc}}(t). \quad (1)$$

Equation (1) is why this chapter has the shape it does. If you change an OCV function, you are changing the equilibrium term. If you slow a diffusivity function, you are changing the transport-limited response and therefore the concentration overpotential. If you move from a mild CC discharge to a stop-start drive cycle, you are not changing the chemistry at all; you are changing the forcing that excites those terms. If you change from isothermal to lumped thermal coupling,

you are allowing temperature to move from “fixed parameter” to “solved state,” which then feeds back into kinetics, conductivity, diffusivity, and entropic heating.

This is also why a battery-model result must always be reported with more than just the model name. “We used a DFN model” is not enough. A DFN with one OCV curve, one diffusivity law, one current profile, and fixed temperature is a different scientific object from a DFN with a different OCV curve, a pulse protocol, and thermal coupling. The governing equations are the same; the simulation is not.

PyBaMM encodes that distinction in software form. `ParameterValues` is dictionary-like because many parameters are indeed simple constants, but it is more than a dictionary because some parameters are functions and because the class knows how to process those functions into the model. Newer PyBaMM releases also expose parameter introspection methods such as `get_info`, `list_by_category`, and `diff`. Those are not niceties. They are reproducibility tools. They let you answer questions like:

- What exactly did I modify?
- Which parameters in this set belong to the thermal domain?
- How does one published parameter set differ from another?

The `Experiment` class does something equally important on the operating-condition side. It turns a prose protocol into an executable boundary program. The string

```
"Charge at 1 C until 4.2 V"
```

is not just a friendly label. It compiles into a specific current-controlled step with a voltage termination event. Likewise,

```
pybamm.step.current(drive_cycle_array)
```

means: at each time in the provided array, impose this current value and interpolate between the data points. That is why drive-cycle data must start at $t = 0$, have consistent time units, and use a current sign convention you can explain.

Temperature is the last conceptual bridge worth pausing on. In an isothermal run, temperature is treated as an imposed quantity, effectively a parameter. In a lumped thermal run, temperature becomes a state governed by an energy balance:

$$\rho_{\text{eff}} \frac{dT}{dt} = \bar{Q} - \frac{hA}{V} (T - T_{\infty}). \quad (2)$$

Here, $\bar{Q}$ is the average heat-generation rate, h is the total heat-transfer

coefficient, A is the cooling area, and V is the cell volume. The important modeling lesson is not the algebra itself, but the change in role: temperature moves from a fixed input to a coupled state variable. Once that happens, every temperature-dependent parameter becomes dynamically active.

So the mental model for the rest of the chapter is this. The textbook gave you equations and physical meaning. PyBaMM gives you a disciplined way to encode constants, functions, and forcing histories separately. Good simulation practice is learning when to change each of those layers, how to document the change, and how to interpret the consequences without confusing one layer for another.

Guided Walkthrough 1

Inspect, Compare, Modify, and Save a Parameter Set

Learning objective: Treat a PyBaMM parameter set as a research object rather than a hidden bundle of defaults.

Before we modify anything, we need to build the habit of *looking* at a parameter set carefully. The most common beginner mistake in physics-based battery modeling is to treat a named parameter set as if it were a chemistry truth. It is not. It is a concrete, inspectable, editable package of assumptions. In this walkthrough, we will inspect metadata, compare two bundled sets, make a controlled set of overrides, and save those overrides so the chapter's later simulations remain traceable.

4.3.1 Walkthrough 1 code

```
import json
from pathlib import Path

import pybamm

# Load two built-in parameter sets so we can inspect and compare them.
chen_params = pybamm.ParameterValues("Chen2020")
marquis_params = pybamm.ParameterValues("Marquis2019")

# Ask PyBaMM for metadata about one specific parameter.
capacity_info = chen_params.get_info("Nominal cell capacity [A.h]")
```

```

# Pull a thematic subset to understand the structure of the parameter set.
thermal_parameters = chen_params.list_by_category("thermal")

# Compare two published parameter sets.
parameter_diff = chen_params.diff(marquis_params)

print("PyBaMM version:", pybamm.__version__)
print("Nominal capacity value:", capacity_info.value)
print("Nominal capacity units:", capacity_info.units)
print("Nominal capacity category:", capacity_info.category)
print("Thermal parameter count:", len(thermal_parameters))
print("First five thermal parameters:", thermal_parameters[:5])
print("Changed keys versus Marquis2019:", len(parameter_diff.changed))
print("Sample changed keys:", list(parameter_diff.changed.keys())[:5])

# Make a research copy before editing anything.
research_params = chen_params.copy()
research_overrides = {
    "Current function [A]": 7.5,
    "Ambient temperature [K]": 303.15,
    "Lower voltage cut-off [V]": 2.8,
}
research_params.update(research_overrides)

# Save both a small override file and a full snapshot for reproducibility.
output_dir = Path("artifacts/chapter_4")
output_dir.mkdir(parents=True, exist_ok=True)

override_path = output_dir / "chen2020_research_overrides.json"
snapshot_path = output_dir / "chen2020_research_snapshot.json"

override_path.write_text(json.dumps(research_overrides, indent=2))
research_params.to_json(snapshot_path)

# Reload the snapshot to prove it round-trips cleanly.
reloaded_params = pybamm.ParameterValues.from_json(snapshot_path)

print("Override file saved to:", override_path.resolve())
print("Snapshot file saved to:", snapshot_path.resolve())
print("Reloaded current [A]:", reloaded_params["Current function [A]"])
print("Reloaded ambient temperature [K]:", reloaded_params["Ambient
↳ temperature [K]"])

```



```
print("Reloaded lower cut-off [V]:", reloaded_params["Lower voltage cut-off"  
↳ [V]])
```

4.3.2 Walkthrough 1 explanation

The first two lines load `Chen2020` and `Marquis2019` as full `ParameterValues` objects. We are not yet solving a model. This is purely parameter work, and that is an important conceptual distinction. You can and should study parameter sets before you ever launch a solver.

`chen_params.get_info("Nominal cell capacity [A.h]")` returns a metadata object rather than just the raw value. The returned object includes the numerical value, the units string, and the category. That matters because real parameter sets get large very quickly, and metadata is what keeps them navigable.

`list_by_category("thermal")` is our first act of structured browsing. In research practice, this is exactly the kind of question you ask when you switch from an isothermal notebook to a coupled thermal notebook: which parameters in this set are actually thermal?

`diff(marquis_params)` is one of the most useful habits in the whole chapter. When you say “I reran the analysis with a different parameter set,” you should be able to answer the follow-up question: different *how*? The `diff` object gives you a principled starting point.

The next block is about safe editing. `copy()` is not optional here. If you modify the original `chen_params` in place and later reuse it assuming it is still pristine, you have created the kind of hidden state that ruins notebooks and confuses coauthors.

Our three overrides are deliberately modest. We increase the current to 7.5 A , increase ambient temperature to 303.15 K , and raise the lower voltage cutoff to 2.8 V . None of these changes alters the structure of the electrochemical model; they alter the operating assumptions attached to it.

The saving pattern deserves attention. We write a small human-readable overrides file ourselves using `json.dumps`, and we also ask PyBaMM to serialize the full parameter set with `to_json`. Both are useful. The override file is what a human wants to review in Git. The full snapshot is what makes a future rerun exact.

Finally, we reload the saved snapshot with `from_json` and print a few key fields. That last step is not ceremony. It is a verification that the saved artifact is

genuinely usable rather than just decorative.

4.3.3 Walkthrough 1 expected output

You should see `PyBaMM version: 25.12.1`, a nominal capacity of `5.0 A.h`, and a thermal parameter count of `13`. The exact set of changed keys reported by `diff()` will be longer than what prints on screen, because we intentionally print only the first handful.

You should also see two file paths printed inside `artifacts/chapter_4`. The reloaded current, ambient temperature, and lower-cutoff voltage should match the override values you wrote. If they do not, stop here and fix the save-reload path before moving on. Hidden parameter drift is a real research problem.

4.3.4 Walkthrough 2 troubleshooting

1. You edit `chen_params` directly and later forget you changed it.
Symptom: later notebooks behave strangely even though the code “looks unchanged.”
Fix: always `copy()` before mutating a published parameter set.
2. `get_info` or `diff` is missing.
Symptom: an `AttributeError` appears on one of the introspection calls.
Fix: your PyBaMM version is too old. Reinstall the pinned version for this manual.
3. The snapshot JSON saves, but reload values do not match your edits.
Symptom: the printed values are different after `from_json`.
Fix: inspect whether you accidentally wrote the override file instead of the full snapshot file path, or overwrote the wrong file from an older run.

4.3.5 Reflection

This exercise teaches a quiet but essential research skill: parameters are not decorations around a model. They are first-class artifacts. If you can inspect them, compare them, and save them cleanly, you are already working at a more publishable level than many battery notebooks ever reach.

Guided Walkthrough 2

Replace an OCV Curve and a Diffusivity Function

Learning objective: Replace function-valued parameters safely and understand what parts of the simulated behavior each change should influence.

Textbook Chapter 8 treated OCV functions and transport coefficients as constitutive ingredients in the model. In PyBaMM, those ingredients are exposed directly enough that you can swap them. That is powerful, but it is also where many readers first collide with symbolic programming. The subtle point is that your custom function must be compatible with PyBaMM's expression tree. If you write a function that only works on ordinary NumPy arrays, it may fail once PyBaMM passes it a symbolic variable. We will handle that carefully.

4.3.6 Walkthrough 2 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

model = pybamm.lithium_ion.DFN()

# Start from a clean baseline and make the current slightly higher so
↪ transport effects show up.
baseline_params = pybamm.ParameterValues("Chen2020")
baseline_params.update({"Current function [A]": 7.5})

modified_params = baseline_params.copy()

# Grab the original functions so we can wrap rather than replace blindly.
base_negative_ocp = baseline_params["Negative electrode OCP [V]"]
base_positive_diffusivity = baseline_params["Positive particle diffusivity"]
↪ [m2.s-1]"]

# Build a table-based OCV perturbation. This mimics what you would do with
↪ measured OCV data.
sto_grid = np.linspace(0.01, 0.99, 200)
shifted_negative_ocp_values = base_negative_ocp(sto_grid) - 0.02 * (1.0 -
↪ sto_grid)

def chapter4_negative_ocp(sto):
    return pybamm.Interpolant(
```

```

        sto_grid,
        shifted_negative_ocp_values,
        sto,
        name="chapter4_negative_ocp_table",
    )

# Make the positive-particle diffusivity uniformly slower.
def slower_positive_diffusivity(sto, temperature):
    return 0.6 * base_positive_diffusivity(sto, temperature)

modified_params.update(
    {
        "Negative electrode OCP [V]": chapter4_negative_ocp,
        "Positive particle diffusivity [m2.s-1]":
            ↪ slower_positive_diffusivity,
    }
)

t_eval = np.linspace(0, 2400, 241)
solutions = {}

for label, params in {
    "Baseline": baseline_params,
    "Modified functions": modified_params,
}.items():
    simulation = pybamm.Simulation(model, parameter_values=params)
    solutions[label] = simulation.solve(t_eval)

time_min = t_eval / 60.0
baseline_voltage_v = solutions["Baseline"]["Terminal voltage [V]"](t_eval)
modified_voltage_v = solutions["Modified functions"]["Terminal voltage
    ↪ [V]"](t_eval)

baseline_conc_overpotential_v = solutions["Baseline"][
    "X-averaged battery concentration overpotential [V]"
](t_eval)
modified_conc_overpotential_v = solutions["Modified functions"][
    "X-averaged battery concentration overpotential [V]"
](t_eval)

print(

```

```

    "Voltage shift at t = 0 [mV]:",
    round(float((modified_voltage_v[0] - baseline_voltage_v[0]) * 1000), 2),
)
print(
    "Voltage shift at t = 2400 s [mV]:",
    round(float((modified_voltage_v[-1] - baseline_voltage_v[-1]) * 1000),
    ↪ 2),
)
print(
    "Max added concentration overpotential [mV]:",
    round(
        float(
            np.max(
                np.abs(
                    modified_conc_overpotential_v -
    ↪ baseline_conc_overpotential_v
                )
            )
        * 1000
    ),
    2,
),
)
)

fig, axes = plt.subplots(2, 1, figsize=(9, 8), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(time_min, baseline_voltage_v, linewidth=2.2, label="Baseline")
axes[0].plot(time_min, modified_voltage_v, linewidth=2.2, label="Modified
    ↪ functions")
axes[0].set_ylabel("Terminal Voltage [V]")
axes[0].set_title("Replacing an OCV curve and a diffusivity law")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(
    time_min,
    baseline_conc_overpotential_v * 1000,
    linewidth=2.2,
    label="Baseline",
)
axes[1].plot(
    time_min,
    modified_conc_overpotential_v * 1000,

```

```

        linewidth=2.2,
        label="Modified functions",
    )
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Concentration Overpotential [mV]")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.show()

```

4.3.7 Walkthrough 2 explanation

We begin with a DFN model and a clean Chen2020 parameter set. We then raise the current to 7.5 A so that concentration effects become large enough to read clearly over a modest simulation window. This is standard teaching practice: amplify the mechanism you want to observe, but say out loud that you are doing it.

The next two lines pull the original negative-electrode OCP function and positive-particle diffusivity function out of the parameter set. This is a much safer way to customize behavior than replacing functions from scratch without a reference.

The OCV replacement is intentionally table-based. We evaluate the original OCP on a stoichiometry grid and then subtract a small stoichiometry-dependent offset. That creates a synthetic measured-like table. The custom function `chapter4_negative_ocp(sto)` returns a `pybamm.Interpolant`, not a NumPy interpolation. That distinction is critical. `pybamm.Interpolant` can live inside the symbolic model; `np.interp` cannot.

The diffusivity modification is simpler. We define `slower_positive_diffusivity(sto, temperature)` with the same input signature as the original function and scale the baseline value by 0.6. This is a good example of a change that mostly affects dynamic transport behavior rather than the equilibrium voltage baseline.

After updating the modified parameter set, we solve the baseline and modified models on the same time grid. Using the same `t_eval` is essential because it makes the resulting differences pointwise comparable.

We then extract two variables. `Terminal voltage [V]` tells us what a user or cyclor would observe directly. `X-averaged battery concentration overpotential [V]` is the internal mechanism we expect the diffusivity change

to affect most strongly. This pairing is deliberate: an external signal plus an internal explanation.

4.3.8 Walkthrough 2 expected output and interpretation

You should see the modified-voltage curve sit slightly below the baseline from the very beginning, because we shifted the negative-electrode OCP downward. As the discharge progresses, the gap should widen somewhat, and the concentration-overpotential curve for the modified case should rise more aggressively than the baseline because we intentionally slowed the positive-particle diffusivity.

The printed values will be in the tens of millivolts, not volts. That scale is important. We are not building a cartoonishly different battery; we are making realistic-looking constitutive perturbations and watching how they propagate.

If both voltage curves are essentially identical and the concentration-overpotential curves sit on top of one another, something is wrong with your function replacement.

4.3.9 Walkthrough 3 troubleshooting

1. You use `np.interp` or other pure-NumPy logic in the custom OCV function.
Symptom: PyBaMM fails once the function is processed into the model.
Fix: use `pybamm.Interpolant` for tabulated functions that must accept symbolic inputs.
2. Your custom function signature does not match the expected inputs.
Symptom: the solver fails during parameter processing with a function-argument error.
Fix: use `model.print_parameter_info()` if needed to check which inputs a given function parameter expects.
3. You mutate the baseline parameter set and then “compare” it to itself.
Symptom: baseline and modified curves match suspiciously well.
Fix: make the modified set from `baseline_params.copy()` before updating.

4.3.10 Walkthrough 2 reflection

This exercise teaches one of the most transferable skills in the companion: how to replace the physics-bearing pieces of a PyBaMM model without losing symbolic

compatibility. That is exactly the workflow you will need later for sodium-ion OCV curves, fitted transport laws, and literature-derived constitutive updates.

Guided Walkthrough 3

Encode a Full Charge-Rest-Discharge Protocol with `Experiment`

Learning objective: Build a multi-step protocol that includes CC, CV, and rest periods, then interpret the resulting current and voltage trajectory correctly.

Battery papers rarely use one clean constant-current segment and stop. Real workflows include transitions, cutoffs, and recovery periods. That is why PyBaMM's `Experiment` class matters. In this walkthrough, we will encode a full protocol that begins with discharge, pauses for relaxation, recharges with a CC-CV sequence, and rests again. The objective is not just to run it, but to read the sign conventions and step transitions like an experimentalist.

4.3.11 Walkthrough 3 code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

parameter_values = pybamm.ParameterValues("Chen2020")

experiment = pybamm.Experiment(
    [
        "Discharge at C/3 until 3.0 V",
        "Rest for 30 minutes",
        "Charge at 1 C until 4.2 V",
        "Hold at 4.2 V until C/20",
        "Rest for 30 minutes",
    ],
    period="10 seconds",
)

simulation = pybamm.Simulation(
    pybamm.lithium_ion.DFN(),
    parameter_values=parameter_values,
    experiment=experiment,
```



```

        solver=pybamm.CasadiSolver(mode="fast"),
    )
    solution = simulation.solve()

    time_s = solution["Time [s]"].entries
    time_h = time_s / 3600.0
    current_a = solution["Current [A]"].entries
    voltage_v = solution["Terminal voltage [V]"].entries
    discharge_capacity_ah = solution["Discharge capacity [A.h]"].entries

    discharge_energy_wh = np.trapezoid(
        np.clip(current_a, 0.0, None) * voltage_v,
        time_s,
    ) / 3600.0
    charge_energy_wh = -np.trapezoid(
        np.clip(current_a, None, 0.0) * voltage_v,
        time_s,
    ) / 3600.0
    throughput_ah = np.trapezoid(np.abs(current_a), time_s) / 3600.0

    print("Total throughput [A.h]:", round(float(throughput_ah), 4))
    print("Discharge energy [W.h]:", round(float(discharge_energy_wh), 4))
    print("Charge energy [W.h]:", round(float(charge_energy_wh), 4))
    print("Final terminal voltage [V]:", round(float(voltage_v[-1]), 4))

    fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True,
        ↪ constrained_layout=True)

    axes[0].plot(time_h, current_a, linewidth=2.1, color="tab:blue")
    axes[0].set_ylabel("Current [A]")
    axes[0].set_title("CC-rest-CCCV-rest protocol")
    axes[0].grid(True, alpha=0.3)

    axes[1].plot(time_h, voltage_v, linewidth=2.1, color="tab:orange")
    axes[1].set_ylabel("Terminal Voltage [V]")
    axes[1].grid(True, alpha=0.3)

    axes[2].plot(time_h, discharge_capacity_ah, linewidth=2.1, color="tab:green")
    axes[2].set_ylabel("Discharge Capacity [A.h]")
    axes[2].set_xlabel("Time [h]")
    axes[2].grid(True, alpha=0.3)

```

```
plt.show()
```

4.3.12 Walkthrough 3 explanation

The Experiment constructor is the star here. Each string is one operating step. PyBaMM parses these into structured step objects with durations, directions, and termination events.

The first step, "Discharge at C/3 until 3.0 V", means what it says: apply a positive discharge current equal to one-third of the nominal capacity in amperes until the lower voltage threshold is reached. The second step is a rest. In PyBaMM, that means zero current, not a magical equilibrium reset.

The third and fourth steps together form a standard CCCV charge. First we charge at a fixed current until the upper cutoff voltage 4.2 V is reached. Then we hold voltage constant at 4.2 V and allow the current to decay until it reaches C/20. This is standard practice in the field. If you publish charge simulations without being clear about whether your charge was CC-only or CCCV, you are leaving out an important part of the method.

We request a reporting `period="10 seconds"` because that is dense enough to show the CV taper smoothly without creating an unnecessarily large solution.

The energy calculations are worth reading carefully. Discharge current is positive in PyBaMM, so discharge energy is computed from the positive-current portion of the signal. Charge current is negative, so we take the negative-current portion, multiply by voltage, and apply a minus sign to report charge energy as a positive quantity. This sign bookkeeping is simple once you understand it, but it is one of the most common sources of silent mistakes in battery data analysis.

4.3.13 Walkthrough 3 expected output and interpretation

The current plot should have five visually distinct regions. First, a positive constant-current discharge segment. Second, a flat zero-current rest. Third, a negative constant-current charge segment. Fourth, a negative current that gradually decays in magnitude during the CV hold. Fifth, another flat zero-current rest.

The voltage plot should fall during discharge, rebound upward during the first rest, rise during charge, flatten near 4.2 V during the CV hold, and relax slightly during the final rest. The discharge-capacity curve should rise during discharge

and then remain flat during the rest and charge portions because it measures discharged capacity since the beginning of the simulation, not signed charge throughput.

If your current sign is reversed in your mental model, this whole figure will feel backwards. That is exactly why this walkthrough exists.

4.3.14 Walkthrough 4 troubleshooting

1. You place the charge step first while keeping the default fully charged initial state.

Symptom: the first step is skipped or declared infeasible.

Fix: either start from discharge, as we do here, or explicitly set the initial state before charging first.

2. You confuse discharge capacity with net charge throughput.

Symptom: you expect the capacity curve to decrease during charge.

Fix: `Discharge capacity [A.h]` is cumulative discharged capacity, not signed coulomb counting.

3. The CV hold never appears.

Symptom: the current plot jumps straight from constant-current charge to rest.

Fix: the current may already have dropped below the termination threshold by the moment `4.2 V` is reached, or your version/parser may differ. Re-check the string exactly.

4.3.15 Walkthrough 3 reflection

This exercise teaches you to think of an experiment as executable boundary data. That is a graduate-level modeling habit. You are no longer “running a battery model.” You are applying a specific test protocol to a virtual cell and reading the result with the same sign discipline you would need on a real cycler.

Dataset Integration

Public UDDS Drive-Cycle Data

This chapter’s real-data section uses a public UDDS current trace distributed in PyBaMM’s data release and derived from the standard Urban Dynamometer Driving Schedule workflow. We use the packaged file because it is small, directly

machine-readable, and immediately compatible with PyBaMM. If you prefer to go to the regulatory source, keep the EPA Dynamometer Drive Schedules page bookmarked as the authoritative reference for the underlying standard.

Item	Value
Public current file used in code	UDDS.csv from the PyBaMM public data release
Programmatic access	<code>pybamm.DataLoader().get_data("UDDS.csv")</code>
Direct release pattern	<code>https://github.com/pybamm-team/pybamm-data</code>
Underlying standard	EPA Urban Dynamometer Driving Schedule (UDDS)
EPA reference page	<code>https://www.epa.gov/vehicle-and-fuel-emissions-testing</code>
Reported EPA file size for the raw schedule page entry	12.32 KB for the tab-delimited UDDS text file
Format used in the PyBaMM convenience file	two columns: time in seconds, current in amperes
License and reuse note	cite the source you actually use; the EPA page does not expose a separate dataset license on the download listing, while the PyBaMM release is publicly distributed for model examples

For research notebooks, the packaged `UDDS.csv` is the right level of convenience. For a paper, cite the underlying standard trace or the data package you actually downloaded. Do not pretend a convenience wrapper is the primary source if it is not.

The parsing workflow in the next walkthrough will make four checks that should become muscle memory:

1. strip comment lines and null rows,
2. ensure the time column is monotonically increasing,
3. confirm the time axis starts at zero,
4. state the current sign convention explicitly.

Those checks feel boring right up until the day one of them saves your figure.

Guided Walkthrough 4

Load a Real UDDS Current Profile and Run It in PyBaMM

Learning objective: Parse a public drive-cycle file cleanly, validate its columns and sign convention, and run it through a physics-based model.

This is the point where the chapter becomes recognizably research-like. We are no longer using a hand-written one-line current function. We are taking a public current trace, validating it, and using it as a dynamic input. For this walkthrough we use SPMe rather than DFN. That is a standard practice in the field when you want electrolyte dynamics but also want fast iteration under long or jagged duty cycles.

4.3.16 Walkthrough 4 code

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pybamm

def load_drive_cycle(filename):
    data_loader = pybamm.DataLoader()
    file_path = Path(data_loader.get_data(filename))

    drive_cycle = pd.read_csv(
        file_path,
        comment="#",
        header=None,
        names=["time_s", "current_a"],
    )

    drive_cycle = (
        drive_cycle.dropna()
        .drop_duplicates(subset="time_s", keep="first")
        .sort_values("time_s")
        .reset_index(drop=True)
```

```

)

# Normalize the time axis in case a downloaded file starts at a nonzero
↳ timestamp.
drive_cycle["time_s"] = drive_cycle["time_s"] -
↳ drive_cycle["time_s"].iloc[0]

return file_path, drive_cycle

udds_path, udds = load_drive_cycle("UDDS.csv")

udds["delta_t_s"] = udds["time_s"].diff().fillna(0.0)
udds["signed_capacity_increment_ah"] = (
    udds["current_a"] * udds["delta_t_s"] / 3600.0
)
udds["cumulative_signed_capacity_ah"] =
↳ udds["signed_capacity_increment_ah"].cumsum()

print("Loaded file:", udds_path)
print("Row count:", len(udds))
print("Duration [s]:", round(float(udds["time_s"].iloc[-1]), 1))
print("Minimum current [A]:", round(float(udds["current_a"].min()), 4))
print("Maximum current [A]:", round(float(udds["current_a"].max()), 4))
print("Time starts at zero:", bool(np.isclose(udds["time_s"].iloc[0], 0.0)))

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True,
↳ constrained_layout=True)

axes[0].plot(udds["time_s"] / 60.0, udds["current_a"], linewidth=1.7,
↳ color="tab:blue")
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Public UDDS current profile")
axes[0].grid(True, alpha=0.3)

axes[1].plot(
    udds["time_s"] / 60.0,
    udds["cumulative_signed_capacity_ah"],
    linewidth=1.7,
    color="tab:purple",
)
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Cumulative Signed Capacity [A.h]")
axes[1].grid(True, alpha=0.3)

```

```

plt.show()

drive_cycle_array = udds[["time_s", "current_a"]].to_numpy()

experiment = pybamm.Experiment(
    [pybamm.step.current(drive_cycle_array)],
    period="1 second",
)

parameter_values = pybamm.ParameterValues("Chen2020")
simulation = pybamm.Simulation(
    pybamm.lithium_ion.SPMe(),
    parameter_values=parameter_values,
    experiment=experiment,
    solver=pybamm.CasadiSolver(mode="fast"),
)
solution = simulation.solve()

time_s = solution["Time [s]"].entries
time_min = time_s / 60.0
current_a = solution["Current [A]"].entries
voltage_v = solution["Terminal voltage [V]"].entries
discharge_capacity_ah = solution["Discharge capacity [A.h]"].entries
approx_soc = 1.0 - discharge_capacity_ah / parameter_values["Nominal cell
↳ capacity [A.h]"]
electrolyte_concentration = solution["X-averaged electrolyte concentration
↳ [mol.m-3]"].entries

print("Final terminal voltage [V]:", round(float(voltage_v[-1]), 4))
print("Final approximate SOC [-]:", round(float(approx_soc[-1]), 4))
print(
    "Electrolyte concentration range [mol.m-3]:",
    round(float(electrolyte_concentration.min()), 2),
    "to",
    round(float(electrolyte_concentration.max()), 2),
)

fig, axes = plt.subplots(4, 1, figsize=(10, 12), sharex=True,
↳ constrained_layout=True)

axes[0].plot(time_min, current_a, linewidth=1.7, color="tab:blue")

```

```

axes[0].set_ylabel("Current [A]")
axes[0].set_title("UDDS-driven SPMe simulation")
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_min, voltage_v, linewidth=1.9, color="tab:orange")
axes[1].set_ylabel("Voltage [V]")
axes[1].grid(True, alpha=0.3)

axes[2].plot(time_min, approx_soc, linewidth=1.9, color="tab:red")
axes[2].set_ylabel("Approx. SOC [-]")
axes[2].grid(True, alpha=0.3)

axes[3].plot(
    time_min,
    electrolyte_concentration,
    linewidth=1.9,
    color="tab:green",
)
axes[3].set_ylabel("X-avg Electrolyte c [mol.m$^{-3}$]")
axes[3].set_xlabel("Time [min]")
axes[3].grid(True, alpha=0.3)

plt.show()

```

4.3.17 Walkthrough 4 explanation

We begin with a reusable helper, `load_drive_cycle(filename)`. This does more than read a CSV. It performs light but meaningful data hygiene: remove nulls, remove repeated timestamps, sort by time, and normalize the time axis so the trace begins at zero. Those four operations are so common in battery-data work that they are worth wrapping early.

The cumulative signed-capacity calculation is not something PyBaMM needs in order to solve the model. It is something *you* need in order to inspect the dataset. A current profile can look plausible while still having an unexpected net bias or sign mistake. Integrating it once is a quick way to see whether the trace is mostly discharge, mostly charge, or approximately energy-neutral.

The `drive_cycle_array` passed into `pybamm.step.current` must be a two-column NumPy array. The first column is time in seconds. The second is current in amperes. PyBaMM treats positive current as discharge and negative current as charge or regenerative current. State that explicitly whenever you work

with a dynamic trace, because public datasets do not all use the same sign convention.

We then build a one-step experiment containing the whole drive cycle. The `period="1 second"` argument tells PyBaMM how densely to record the outputs. Since UDDS is conventionally used at a one-second cadence, this is the natural choice.

We use `SPMe` instead of `DFN` for a reason. This is standard practice when you want realistic electrolyte dynamics and internal concentration behavior but also want a model that remains pleasant to rerun many times in a notebook. Later, if a reviewer-level result depends critically on this case, you can rerun the final comparison with the `DFN`.

Finally, we plot four panels: current, voltage, approximate SOC, and x-averaged electrolyte concentration. That panel selection is intentional. It ties together the external input, external response, state depletion, and one internal transport variable.

4.3.18 Walkthrough 4 expected output and interpretation

The first figure should show a jagged current trace with frequent changes and many sections near zero. That is exactly what you expect from an urban stop-and-go profile. The cumulative signed-capacity curve should drift rather than remain perfectly flat, indicating a net discharge bias over the cycle.

The simulation figure should show the terminal voltage sagging during higher discharge-current sections and recovering during low-current or regenerative sections. The approximate SOC should decline gradually rather than linearly, because the current is not constant. The x-averaged electrolyte concentration should vary more subtly than voltage, but it should still respond coherently to the current bursts.

If the current looks upside down relative to the voltage response, the first thing to check is sign convention, not solver correctness.

4.3.19 Walkthrough 5 troubleshooting

1. The first timestamp is not zero.

Symptom: PyBaMM raises an error or the drive cycle starts strangely.

Fix: normalize the time axis exactly as we do in the helper function.

2. Your current sign convention is reversed relative to PyBaMM.

Symptom: voltage rises when you think the battery is discharging.

Fix: inspect the input current, not just the output voltage. Positive current should mean discharge in this chapter's code.

3. **DataLoader** cannot download the file on a restricted network.

Symptom: the code fails before `pandas.read_csv` ever runs.

Fix: manually download the CSV once and point `read_csv` at the local path.

4. The solver prints a warning about evaluating outside the data range.

Symptom: a warning appears before the solve completes.

Fix: on dynamic traces, some rejected trial steps are normal. If the final result is smooth and the solve completes, this warning is usually harmless.

4.3.20 Walkthrough 4 reflection

This exercise teaches the real workflow boundary between public data and simulation. A drive cycle is not magic. It is just a time series. But once you learn how to validate, parse, and impose it cleanly, your battery model stops living in a toy world of constant-current segments.

Guided Walkthrough 5

Temperature as an Input Versus Temperature as a Coupled State

Learning objective: Compare an isothermal drive-cycle simulation to a lumped-thermal simulation and quantify what thermal coupling changes.

Chapter 4 is the right place to learn this distinction because temperature often sneaks into modeling work without being declared clearly. In an isothermal model, temperature is an imposed condition. In a thermally coupled model, temperature is part of the solution. We will use the same dynamic current trace in both cases so the comparison remains honest.

To make the thermal effect visible over a single short drive cycle, we will double the UDDS current amplitude. This is a teaching shortcut, not a claim about a real vehicle pack. I am flagging that explicitly because it is exactly the kind of modeling choice you should label clearly in a paper.

4.3.21 Walkthrough 5 code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pybamm

data_loader = pybamm.DataLoader()
udds = pd.read_csv(
    data_loader.get_data("UDDS.csv"),
    comment="#",
    header=None,
    names=["time_s", "current_a"],
).dropna()

udds["time_s"] = udds["time_s"] - udds["time_s"].iloc[0]

# Teaching shortcut: amplify the current so the temperature response is
↪ visible in one cycle.
scaled_udds = udds.copy()
scaled_udds["current_a"] = 2.0 * scaled_udds["current_a"]
drive_cycle_array = scaled_udds[["time_s", "current_a"]].to_numpy()

experiment = pybamm.Experiment(
    [pybamm.step.current(drive_cycle_array)],
    period="1 second",
)

models = {
    "Isothermal SPMe": pybamm.lithium_ion.SPMe(),
    "Lumped-thermal SPMe": pybamm.lithium_ion.SPMe({"thermal": "lumped"}),
}

solutions = {}
for label, model in models.items():
    params = pybamm.ParameterValues("Chen2020")
    params.update(
        {
            "Ambient temperature [K]": 298.15,
            "Initial temperature [K]": 298.15,
            "Total heat transfer coefficient [W.m-2.K-1]": 10.0,
        }
    )
```

```

simulation = pybamm.Simulation(
    model,
    parameter_values=params,
    experiment=experiment,
    solver=pybamm.CasadiSolver(mode="fast"),
)
solutions[label] = simulation.solve()

time_s = solutions["Isothermal SPMe"]["Time [s]"].entries
time_min = time_s / 60.0

isothermal_voltage_v = solutions["Isothermal SPMe"]["Terminal voltage
↳ [V]"].entries
lumped_voltage_v = solutions["Lumped-thermal SPMe"]["Terminal voltage
↳ [V]"].entries

isothermal_temperature_k = solutions["Isothermal SPMe"][
    "Volume-averaged cell temperature [K]"
].entries
lumped_temperature_k = solutions["Lumped-thermal SPMe"][
    "Volume-averaged cell temperature [K]"
].entries
lumped_total_heating = solutions["Lumped-thermal SPMe"][
    "Volume-averaged total heating [W.m-3]"
].entries
current_a = solutions["Lumped-thermal SPMe"]["Current [A]"].entries

print(
    "Isothermal max temperature rise [K]:",
    round(float(isothermal_temperature_k.max() - 298.15), 4),
)
print(
    "Lumped max temperature rise [K]:",
    round(float(lumped_temperature_k.max() - 298.15), 4),
)
print(
    "Max voltage difference [mV]:",
    round(float(np.max(np.abs(lumped_voltage_v - isothermal_voltage_v)) *
↳ 1000), 2),
)

fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True,
↳ constrained_layout=True)

```

```

axes[0].plot(time_min, current_a, linewidth=1.6, color="tab:blue")
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Isothermal versus lumped-thermal drive-cycle simulation")
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_min, isothermal_voltage_v, linewidth=2.0,
    ↪ label="Isothermal")
axes[1].plot(time_min, lumped_voltage_v, linewidth=2.0, label="Lumped
    ↪ thermal")
axes[1].set_ylabel("Voltage [V]")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

axes[2].plot(
    time_min,
    isothermal_temperature_k,
    linewidth=2.0,
    label="Isothermal",
)
axes[2].plot(
    time_min,
    lumped_temperature_k,
    linewidth=2.0,
    label="Lumped thermal",
)
axes[2].plot(
    time_min,
    298.15 + 0.000001 * lumped_total_heating,
    linewidth=1.0,
    linestyle="--",
    alpha=0.6,
    label="Heating trend (scaled for shape only)",
)
axes[2].set_ylabel("Temperature [K]")
axes[2].set_xlabel("Time [min]")
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.show()

```

4.3.22 Walkthrough 5 explanation

The first part of the code reloads the UDDS profile and doubles the current amplitude. I am saying this twice because it matters pedagogically: this is a teaching move to make thermal coupling visible over one short trace. In real work, you would justify current amplitude from the cell or pack you are studying.

We then create two models with identical electrochemical structure but different thermal treatment. `pybamm.lithium_ion.SPMe()` is isothermal by default. `pybamm.lithium_ion.SPMe({"thermal": "lumped"})` adds the lumped thermal ODE from Equation (2).

The parameter updates add three thermal quantities: ambient temperature, initial temperature, and total heat-transfer coefficient. These are the minimum thermal assumptions that keep the comparison explicit.

We extract three outputs: voltage, volume-averaged temperature, and volume-averaged total heating. The heating variable is particularly useful because it reminds you that a temperature rise is not an abstract solver artifact. It is being driven by interpretable heat generation inside the model.

The final plot overlays temperature from both cases. In the isothermal case, the temperature should remain flat. In the lumped case, it should rise. That is the whole conceptual distinction, rendered as a figure.

4.3.23 Walkthrough 5 expected output and interpretation

The printed maximum temperature rise for the isothermal case should be essentially zero, because the model is not allowed to heat up. The lumped case should show a positive temperature rise. The exact magnitude depends on the current amplification and heat-transfer coefficient, but it should be visibly nonzero.

In the figure, the isothermal temperature line should be flat at 298.15 K. The lumped-thermal line should drift upward over the cycle, with the heating trend roughly following higher-current sections. The voltage traces may differ only by a few millivolts, but that is exactly the lesson: sometimes thermal coupling changes the answer a little, and sometimes it changes it a lot. You need to quantify rather than guess.

4.3.24 What could go wrong

1. You forget to enable the thermal option.

Symptom: both temperature traces are flat.

Fix: make sure the second model is instantiated with `{"thermal": "lumped"}`.

2. The thermal effect is too small to see.

Symptom: the lumped-temperature line is technically rising but visually flat.

Fix: that can happen on mild traces. Increase current amplitude modestly or reduce the heat-transfer coefficient, and label the change honestly as a teaching adjustment.

3. You choose unrealistic thermal parameters and get absurd heating.

Symptom: the temperature rise is many tens of kelvin over one short cycle.

Fix: revisit the heat-transfer coefficient and the current scaling before trusting anything else.

4.3.25 Walkthrough 5 reflection

This exercise teaches a modeling judgment rather than a syntax pattern.

Temperature can be a fixed assumption or a solved state. The code difference is one option dictionary. The scientific difference is much larger, because it changes which mechanisms are actually active during the simulation.

Reproduction Exercise

Recreate the Six-Step EV Ageing Protocol from Pozzato, Allam, and Onori (2022)

The open-access dataset paper by Pozzato, Allam, and Onori describes an EV-oriented ageing workflow built from CC-CV charge steps, a shallow CC discharge, and a UDDS discharge segment. The paper's Figure 1 shows the first three cycling profiles for one cell and zooms into the six-step cycle structure. The figure caption and data-description text are unusually helpful: the authors explicitly state that positive current denotes discharge and negative current denotes charge, and they describe the six steps in prose.

That makes this a strong reproduction target for Chapter 4. We are not trying to reproduce every exact voltage value of the authors' experimental cell. We are reproducing the *protocol structure* in PyBaMM and making our approximation choices explicit.

The paper is:

G. Pozzato, A. Allam, and S. Onori, “Lithium-ion battery aging dataset based on electric vehicle real-driving profiles,” *Data in Brief*, 41, 107995, 2022, DOI 10.1016/j.dib.2022.107995.

4.3.26 What we can match exactly

We can match the six-step sequence described in the paper:

1. charge at a specified C-rate until 4.0 V,
2. hold at 4.0 V until current falls below 50 mA,
3. charge at C/4 until 4.2 V,
4. hold at 4.2 V until current falls below 50 mA,
5. shallow discharge at C/4,
6. UDDS discharge.

4.3.27 What we cannot match exactly from the paper alone

The exact cell is an INR21700-M50T with nominal capacity 4.85 Ah, while our bundled teaching parameter set is Chen2020. The paper’s Step 6 uses a UDDS profile normalized to the authors’ cell capacity, and the exact stop criterion for the concatenated UDDS section depends on the experimental capacity state at that point in life. Those details matter.

4.3.28 Our declared approximations

We will use the Chen2020 parameter set but reset its nominal capacity to 4.85 Ah so the C-rates are closer to the paper’s cell. We will explicitly initialize the cell at 20% SOC so the opening charge steps are feasible. We will approximate the shallow C/4 discharge from 100% to 80% SOC as 48 minutes at C/4, because 20% of a full 4 h C/4 discharge is 0.8 h. We will include one public UDDS trace rather than concatenating traces until a specific terminal SOC event.

That means our reproduction target is *structural fidelity*, not exact voltage matching. Close enough here means the current trace has the correct sign pattern, the step ordering matches the paper, and the voltage evolution is physically consistent with that sequence.

4.3.29 Reproduction code

```
import pandas as pd
import matplotlib.pyplot as plt
import pybamm

data_loader = pybamm.DataLoader()
udds = pd.read_csv(
    data_loader.get_data("UDDS.csv"),
    comment="#",
    header=None,
    names=["time_s", "current_a"],
).dropna()
udds["time_s"] = udds["time_s"] - udds["time_s"].iloc[0]
udds_array = udds[["time_s", "current_a"]].to_numpy()

parameter_values = pybamm.ParameterValues("Chen2020")
parameter_values.update({"Nominal cell capacity [A.h]": 4.85})
parameter_values = parameter_values.set_initial_state(0.2, inplace=False)

protocol = pybamm.Experiment(
    [
        "Charge at C/2 until 4.0 V",
        "Hold at 4.0 V until 50 mA",
        "Charge at C/4 until 4.2 V",
        "Hold at 4.2 V until 50 mA",
        "Rest for 30 minutes",
        "Discharge at C/4 for 48 minutes",
        pybamm.step.current(udds_array),
    ],
    period="1 second",
)

simulation = pybamm.Simulation(
    pybamm.lithium_ion.SPMe({"thermal": "lumped"}),
    parameter_values=parameter_values,
    experiment=protocol,
    solver=pybamm.CasadiSolver(mode="fast"),
)

solution = simulation.solve()

time_h = solution["Time [s]"].entries / 3600.0
current_a = solution["Current [A]"].entries
```

```

voltage_v = solution["Terminal voltage [V]"].entries

fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex=True,
    ↪ constrained_layout=True)

axes[0].plot(time_h, current_a, linewidth=1.8, color="tab:blue")
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Protocol-structure reproduction of Pozzato et al. (2022),
    ↪ Fig. 1")
axes[0].grid(True, alpha=0.3)

axes[1].plot(time_h, voltage_v, linewidth=1.8, color="tab:orange")
axes[1].set_ylabel("Voltage [V]")
axes[1].set_xlabel("Time [h]")
axes[1].grid(True, alpha=0.3)

plt.show()

```

4.3.30 What the reproduced figure should look like

The current trace should begin negative, because the cycle starts with charge. You should see an initial negative constant-current segment, then a tapering negative CV segment near 4.0 V, then another smaller negative constant-current segment at C/4, then a second tapering CV hold near 4.2 V. After the rest, you should see a positive C/4 discharge plateau, followed by a jagged UDDS section with frequent current changes and occasional low-current intervals.

The voltage trace should rise during the two charge stages, flatten during the CV holds, relax during the rest, drop gently during the shallow C/4 discharge, and then oscillate during the UDDS segment. If that sequence appears in the correct order with the correct sign convention, your reproduction is doing the intended scientific job.

4.3.31 Where “close enough” ends

If your current signs are reversed, the step sequence is infeasible, or the CC-CV structure is not visible, the reproduction is not close enough. If the precise voltage values differ modestly from the experimental figure because we used a bundled teaching parameter set rather than the authors’ exact cell characterization, that is acceptable *as long as you say so*. Honest approximation is part of good

reproduction practice.

4.4 Open-Ended Exercises

4.4.1 Exercise 1: Design a GITT protocol and extract quasi-OCV

Build a galvanostatic intermittent titration protocol with ten repeated pairs of 10 minutes of C/10 discharge and 50 minutes of rest. Use the end-of-rest voltages to estimate quasi-OCV as a function of discharged capacity.

Hints:

- Use a repeated tuple inside `pybamm.Experiment`.
- The end of each rest segment is where the current is still essentially zero, but the next sample is not.
- Plot quasi-OCV points against `Discharge capacity [A.h]`.

4.4.2 Exercise 2: Swap UDDS for WLTC and compare transport stress

Load `WLTC.csv` with the same helper pattern used for `UDDS.csv`. Run the same SPMe workflow and compare:

- final approximate SOC,
- minimum terminal voltage,
- maximum x-averaged concentration overpotential.

Hints:

- PyBaMM's data release includes `WLTC.csv`.
- Use the same plotting structure so the comparison stays fair.
- Do not compare two traces on mismatched time units.

4.4.3 Exercise 3: Quantify cooling sensitivity

Using the lumped-thermal setup from Walkthrough 5, run the same amplified UDDS trace with three heat-transfer coefficients: 5, 10, and 25 $\text{W.m}^{-2}.\text{K}^{-1}$. Report the maximum temperature rise and the maximum terminal-voltage deviation from the isothermal case.

Hints:

- Write a small helper function that accepts `h` and returns a solved simulation.
- Reuse the same current trace and model options each time.

- If you change two things at once, the comparison becomes much harder to interpret.

4.5 Worked Solutions

4.5.1 Solution to Exercise 1

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

experiment = pybamm.Experiment(
    [
        (
            "Discharge at C/10 for 10 minutes",
            "Rest for 50 minutes",
        )
    ]
    * 10,
    period="1 minute",
)

simulation = pybamm.Simulation(
    pybamm.lithium_ion.SPM(),
    parameter_values=pybamm.ParameterValues("Chen2020"),
    experiment=experiment,
    solver=pybamm.CasadiSolver(mode="fast"),
)
solution = simulation.solve()

time_s = solution["Time [s]"].entries
current_a = solution["Current [A]"].entries
voltage_v = solution["Terminal voltage [V]"].entries
discharge_capacity_ah = solution["Discharge capacity [A.h]"].entries

rest_mask = np.isclose(current_a, 0.0, atol=1e-6)
rest_end_indices = np.where(rest_mask & np.r_[~rest_mask[1:], True])[0]

quasi_ocv_v = voltage_v[rest_end_indices]
quasi_ocv_capacity_ah = discharge_capacity_ah[rest_end_indices]

fig, axes = plt.subplots(2, 1, figsize=(9, 8), constrained_layout=True)
```

```

axes[0].plot(time_s / 3600.0, current_a, linewidth=1.7)
axes[0].set_ylabel("Current [A]")
axes[0].set_title("GITT-like protocol")
axes[0].grid(True, alpha=0.3)

axes[1].plot(quasi_ocv_capacity_ah, quasi_ocv_v, marker="o", linewidth=1.7)
axes[1].set_xlabel("Discharge Capacity [A.h]")
axes[1].set_ylabel("Quasi-OCV [V]")
axes[1].grid(True, alpha=0.3)

plt.show()

```

The first plot should look like a staircase of short positive pulses separated by long zero-current rests. The second plot should show quasi-OCV decreasing smoothly with discharged capacity. The key lesson is methodological: GITT is not just a protocol string. It is a protocol plus a post-processing rule.

4.5.2 Solution to Exercise 2

```

from pathlib import Path

import numpy as np
import pandas as pd
import pybamm

def load_cycle(name):
    loader = pybamm.DataLoader()
    cycle = pd.read_csv(
        Path(loader.get_data(name)),
        comment="#",
        header=None,
        names=["time_s", "current_a"],
    ).dropna()
    cycle["time_s"] = cycle["time_s"] - cycle["time_s"].iloc[0]
    return cycle

def run_cycle(name):
    cycle = load_cycle(name)
    experiment = pybamm.Experiment(
        [pybamm.step.current(cycle[["time_s", "current_a"]].to_numpy())],
        period="1 second",
    )

```

```

)
sim = pybamm.Simulation(
    pybamm.lithium_ion.SPMe(),
    parameter_values=pybamm.ParameterValues("Chen2020"),
    experiment=experiment,
    solver=pybamm.CasadiSolver(mode="fast"),
)
sol = sim.solve()
return {
    "cycle": name,
    "final_soc": float(
        1.0
        - sol["Discharge capacity [A.h]"].entries[-1]
        / pybamm.ParameterValues("Chen2020")["Nominal cell capacity
        ↪ [A.h]"]
    ),
    "min_voltage": float(sol["Terminal voltage [V]"].entries.min()),
    "max_conc_overpotential": float(
        np.max(
            np.abs(
                sol["X-averaged battery concentration overpotential
        ↪ [V]"].entries
            )
        )
    ),
}

for record in [run_cycle("UDDS.csv"), run_cycle("WLTC.csv")]:
    print(record)

```

The exact numbers depend on the packaged trace amplitudes, but the workflow is the point: same model, same solver, same metrics, different forcing.

4.5.3 Solution to Exercise 3

```

import numpy as np
import pandas as pd
import pybamm

loader = pybamm.DataLoader()
udds = pd.read_csv(
    loader.get_data("UDDS.csv"),

```

```

        comment="#",
        header=None,
        names=["time_s", "current_a"],
    ).dropna()
    udds["time_s"] = udds["time_s"] - udds["time_s"].iloc[0]
    udds["current_a"] = 2.0 * udds["current_a"]

    experiment = pybamm.Experiment(
        [pybamm.step.current(udds[["time_s", "current_a"]].to_numpy())],
        period="1 second",
    )

def solve_with_h(h_value):
    params = pybamm.ParameterValues("Chen2020")
    params.update(
        {
            "Ambient temperature [K]": 298.15,
            "Initial temperature [K]": 298.15,
            "Total heat transfer coefficient [W.m-2.K-1]": h_value,
        }
    )
    sim = pybamm.Simulation(
        pybamm.lithium_ion.SPMe({"thermal": "lumped"}),
        parameter_values=params,
        experiment=experiment,
        solver=pybamm.CasadiSolver(mode="fast"),
    )
    return sim.solve()

isothermal_solution = pybamm.Simulation(
    pybamm.lithium_ion.SPMe(),
    parameter_values=pybamm.ParameterValues("Chen2020"),
    experiment=experiment,
    solver=pybamm.CasadiSolver(mode="fast"),
).solve()

for h_value in [5.0, 10.0, 25.0]:
    sol = solve_with_h(h_value)
    max_temp_rise = float(
        sol["Volume-averaged cell temperature [K]"].entries.max() - 298.15
    )
    max_voltage_delta = float(

```

```

np.max(
    np.abs(
        sol["Terminal voltage [V]"].entries
        - isothermal_solution["Terminal voltage [V]"].entries
    )
)
)
print(
    {
        "h_W_m2_K": h_value,
        "max_temp_rise_K": round(max_temp_rise, 4),
        "max_voltage_delta_mV": round(max_voltage_delta * 1000, 2),
    }
)

```

You should find that stronger cooling reduces peak temperature rise. Whether it materially changes voltage over one trace depends on the size of the thermal feedback under your chosen current amplitude. That is the answer the exercise is designed to make you quantify rather than guess.

What Changes for Sodium-Ion?

By Chapter 4, the sodium-ion adaptation is no longer a footnote. It starts affecting the workflow in visible ways.

First, parameter replacement becomes more central. PyBaMM now includes an official sodium-ion example model, but the public parameter ecosystem is still much sparser than for lithium-ion. In practice, that means you will spend more time doing exactly what Walkthroughs 1 and 2 taught: inspecting parameter values, replacing OCV functions, and documenting constitutive assumptions from papers.

Second, the OCV problem gets harder. Hard-carbon sodium-ion negative electrodes often exhibit flatter or more structured OCV regions than graphite lithium-ion cells. That makes voltage less informative over some SOC windows and increases the value of protocols like GITT for building quasi-equilibrium voltage maps.

Third, equal power does not mean equal current. Sodium-ion cells typically operate at lower nominal voltage than comparable lithium-ion cells, so a drive cycle expressed in power terms often maps to higher current demand at the cell

level. If you port a lithium-ion drive-cycle notebook to sodium-ion, current scaling deserves a deliberate second look.

Fourth, validation usually becomes less direct. With lithium-ion, you can often test a method against rich public datasets and then adapt it to your own chemistry. With sodium-ion, you may need to validate the workflow logic on lithium-ion public data first, then port it to sodium-ion parameter sets or digitized literature curves. That is not a weakness if you state it honestly. It is standard practice in a young data ecosystem.

Finally, low-temperature behavior matters sooner. One reason sodium-ion is attractive is its potential low-temperature robustness in some chemistries. That means the distinction between fixed-temperature and coupled-temperature modeling, which might be a small correction in one lithium-ion notebook, may become central in a sodium-ion study.

So the transferable lesson is this: Chapter 4's software patterns carry over cleanly, but sodium-ion raises the stakes on parameter provenance, OCV fidelity, and explicit validation strategy.

Chapter Summary and Skill Checklist

- You inspected parameter metadata with `get_info`, browsed thematic subsets with `list_by_category`, and compared published sets with `diff`.
- You made safe parameter edits by copying a baseline set, applying overrides, and saving both a human-readable override file and a full snapshot.
- You replaced function-valued parameters, including a table-based OCV function using `pybamm.Interpolant`.
- You encoded a full CC-rest-CCCV-rest experiment with `pybamm.Experiment`.
- You loaded a public UDDS drive-cycle file, validated its columns and timing, and ran it through `SPMe`.
- You compared isothermal and lumped-thermal dynamic simulations and quantified the consequences of thermal coupling.
- You reproduced the protocol structure of a published EV-style ageing cycle while documenting your approximations explicitly.

The commands, classes, and patterns that should now be entering muscle memory are:

- `pybamm.ParameterValues("Chen2020")`
- `params.get_info(...)`
- `params.list_by_category(...)`
- `params.diff(other_params)`
- `params.copy()`
- `params.update({...})`
- `params.to_json(...)`
- `pybamm.ParameterValues.from_json(...)`
- `pybamm.Experiment([...], period="...")`
- `pybamm.step.current(drive_cycle_array)`
- `pybamm.DataLoader().get_data("UDDS.csv")`
- `pybamm.Interpolant(x_data, y_data, symbolic_argument)`
- `pybamm.CasadiSolver(mode="fast")`

You should now be able to answer “yes” to each of these:

- I can inspect a PyBaMM parameter set without treating it as an opaque black box.
- I can explain the difference between changing a constant parameter, changing a function parameter, and changing the operating protocol.
- I can encode CC, CV, rest, and drive-cycle steps in a single reproducible experiment.
- I can import a public current trace and check its time base and sign convention before trusting the result.
- I can explain whether temperature was fixed or coupled in a given simulation and why that choice was acceptable.

If any of those boxes are still unchecked, revisit the relevant walkthrough before you move into Chapter 5. Parameter estimation is much harder when parameter provenance and protocol design are still fuzzy.

Deliverable

The deliverable for this chapter is:

A notebook that runs a realistic drive cycle on a commercial-like cell and produces publication-quality plots of voltage, current, SOC, and internal concentrations.

Approach it in four stages.

First, create a parameter-provenance section at the top of the notebook. Load a bundled parameter set, save an override file, and print the exact parameters you changed. The user-visible rule here is simple: if a reviewer asked what changed relative to the base chemistry, your notebook should answer in one screen.

Second, include one clean protocol section using `Experiment`. It can be the CC-rest-CCCV-rest example from Walkthrough 3 or a variant you design yourself.

Third, include one real-data section that loads `UDDS.csv` or another public drive cycle such as `WLTC.csv`, validates the file structure, and runs it through either `SPMe` or `DFN`. Save a figure with four aligned panels showing current, voltage, SOC, and one internal concentration variable.

Fourth, add a short interpretation paragraph. It should state which model fidelity you chose, which parameter edits matter scientifically, whether temperature was fixed or coupled, and why those choices are appropriate for a first research-grade notebook.

A strong partial deliverable includes:

- one top-to-bottom runnable notebook,
- one saved JSON override file,
- one saved protocol figure,
- one saved drive-cycle figure,
- one short written interpretation of the modeling choices.

Further Practice and Reading

1. PyBaMM Tutorial 4, “Setting parameter values”
https://docs.pybamm.org/en/stable/source/examples/notebooks/getting_started/tutorial_4_setting_parameters.html
2. PyBaMM Tutorial 5, “Run experiments”
https://docs.pybamm.org/en/v25.4.0/source/examples/notebooks/getting_started/tutorial_5_running_experiments.html
3. PyBaMM notebook, “Changing the input current when solving PyBaMM models”
https://docs.pybamm.org/en/stable/source/examples/notebooks/parameterisation/parameterisation_notebook.html
4. G. Pozzato, A. Allam, and S. Onori, “Lithium-ion battery aging dataset based on electric vehicle real-driving profiles,” *Data in Brief*, 41, 107995, 2022.

DOI 10.1016/j.dib.2022.107995

5. Y. Xiaolong Chen et al., “Development of Experimental Techniques for Parameterization of Multi-scale Lithium-ion Battery Models,” *Journal of The Electrochemical Society*, 167, 080534, 2020.

DOI 10.1149/1945-7111/ab9050

6. PyBaMM official sodium-ion example

[https://docs.pybamm.org/en/stable/source/examples/notebooks/models/s](https://docs.pybamm.org/en/stable/source/examples/notebooks/models/sodium-ion.html)

7. EPA Dynamometer Drive Schedules page

[https://www.epa.gov/vehicle-and-fuel-emissions-testing/dynamometer-d](https://www.epa.gov/vehicle-and-fuel-emissions-testing/dynamometer-driving-schedules)

8. PyBaMM GitHub discussions for practical implementation questions

<https://github.com/pybamm-team/PyBaMM/discussions>

Parameter Estimation in PyBaMM

This chapter is where battery modeling becomes uncomfortable in the most productive possible way. Up to now, PyBaMM has let us run models, inspect variables, swap parameter sets, and impose realistic protocols. Parameter estimation asks a harder question: when the terminal voltage from a real or synthetic experiment does not match the model, which part of the model should move, by how much, and how honestly can we claim to have learned something physical? That question sits directly at the boundary between simulation fluency and publishable research competence.

Keep Textbook Chapter 8 open as you work. This chapter operationalizes the electrochemical model hierarchy you learned there, especially the way terminal voltage emerges from equilibrium potentials, transport losses, kinetic losses, and thermal effects rather than from one mysterious black-box transfer function. Keep Textbook Chapter 10 open as well. Its discussion of excitation richness, protocol design, and observability becomes concrete here. The central lesson of this lab is exactly the one hinted at in the theory text: a model parameter is only identifiable if the experiment excites the physics that parameter controls.

That is why parameter estimation is the hardest part of physics-based battery modeling. Writing down the DFN equations is difficult, but once the equations are fixed, the mathematics is at least well posed. Parameter estimation adds ambiguity.

Many parameters push the voltage in similar directions. Some affect the response only weakly under a given protocol. Others are physically meaningful but numerically entangled. A beautiful fit can still be scientifically bad if it is obtained by letting the optimizer compensate for a poor protocol, a wrong chemistry, or a hidden preprocessing mistake.

This chapter teaches you to resist that trap. We will not pretend that a single current-voltage trace can recover the full DFN parameter set. That would be poor practice, and reviewers in this area know it. Instead, we will learn the workflow serious groups actually use on a laptop: start from a traceable literature parameter set, define a small subset of candidate parameters, screen them with sensitivities, fit only the parameters your data can plausibly inform, quantify uncertainty locally, and report the limitations clearly. That discipline is far more valuable than forcing an over-ambitious optimizer to return a table full of unjustified numbers.

The chapter moves in a deliberate sequence. First, we will bridge the theory of identifiability to PyBaMM's `InputParameter` and sensitivity machinery. Then we will rank candidate parameters by their effect on terminal voltage. After that, we will fit a two-parameter synthetic problem using `scipy.optimize.least_squares` and an analytic Jacobian obtained from PyBaMM sensitivities. Once that machinery is clear, we will wrap it into a reusable CSV-to-fit pipeline that reads current-voltage data, builds a forward model, estimates a small subset of parameters, and returns a fit report. We will then connect the workflow to a real public dataset using the Ecker discharge curves exposed through `pybamm.DataLoader`.

Part II already satisfied its formal reproduce-a-published-figure requirement in Chapter 3, but this chapter still includes a reproduction exercise because parameterization skill improves fastest when you work against a published target. We will reproduce, as carefully as the paper allows, the time-domain synthetic excitation illustrated in Figure 2 of Hallemans et al. (2024), *Physics-based battery model parametrisation from impedance data*. That paper's central message is directly relevant here: voltage-only fitting can be useful, but identifiability collapses quickly when the protocol is not informative enough. Reproducing that figure makes the warning concrete.

This chapter also matters for sodium-ion research more than it may appear at first glance. Today most public parameter sets, public validation traces, and polished

PyBaMM examples are still richer on the lithium-ion side. That does not make this chapter a detour. It is training on the transferable part of the craft: defining candidate parameter subsets, screening them, fitting them responsibly, and documenting what the data cannot tell you. When you later move into sodium-ion work, the chemistry-specific content changes. The estimation discipline does not.

5.1 Prerequisites Check

- Required software: the `sib-research` environment from Lab Chapter 1
- Required Python version: Python 3.11+
- Required Python packages: `pybamm==25.12.1`, `numpy`, `pandas`, `scipy`, `matplotlib`
- Optional exploratory package for the Bayesian detour: `pybop==26.3` in a separate sandbox environment
- Required textbook chapters: Textbook Chapter 8 is essential; Textbook Chapter 10 is essential; your sodium-ion chemistry chapter is recommended for the final comparison section
- Required prior lab chapters: Lab Chapters 1 through 4
- Estimated time: 14 to 18 hours if Chapter 4 felt comfortable; 18 to 22 hours if optimization and identifiability are new

If the PyBaMM object hierarchy from Sections 3.3 through 3.7 still feels shaky, revisit Chapter 3 before you start. If experiment construction and current-profile handling from Sections 4.3 through 4.6 felt brittle, revisit those too. This chapter assumes you can already build a simulation cleanly. Here we focus on deciding what to change inside that simulation and why.

5.2 Environment Setup

We will keep the same PyBaMM pin used in Chapters 3 and 4 so the API, solver behavior, and data-loader examples remain consistent.

5.2.1 Activate your research environment

If you used conda:

```
conda activate sib-research
```

If you used `venv` on Linux or macOS:

```
source .venv/bin/activate
```

If you used `venv` on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

5.2.2 Install the chapter dependencies

If Chapters 3 and 4 ran successfully, you may already have everything. If not, install the chapter stack explicitly:

```
python -m pip install pybamm==25.12.1 numpy pandas scipy matplotlib
```

If your shell exposes `python3` instead of `python`, use:

```
python3 -m pip install pybamm==25.12.1 numpy pandas scipy matplotlib
```

The optional Bayesian detour is easiest to keep isolated so it does not destabilize the base environment. If you want that extra section to be runnable later, create a second sandbox environment:

```
conda create -n pybop-sandbox python=3.11 -y
conda activate pybop-sandbox
python -m pip install pybop==26.3 pybamm==25.12.1
```

That separate environment is a teaching convenience. Standard practice in the field is to isolate inference tooling from the environment you use for your main production notebooks, because optimization packages and solver stacks evolve at different speeds.

5.2.3 Verify the install with a minimal sensitivity hello-world

Run this in a fresh notebook cell or terminal-backed Python session:

```
import numpy as np
import pybamm

model = pybamm.BaseModel("sensitivity hello")
y = pybamm.Variable("y")
a = pybamm.InputParameter("a")
```



```

model.rhs = {y: a * y}
model.initial_conditions = {y: 1}
model.variables = {"y squared": y**2}

solver = pybamm.IDAKLUSolver(rtol=1e-10, atol=1e-10)
t_eval = np.linspace(0, 1, 11)

solution = solver.solve(
    model,
    t_eval,
    inputs={"a": 1.0},
    calculate_sensitivities=True,
)

print("PyBaMM version:", pybamm.__version__)
print("Last value of y^2:", round(float(solution["y squared"].data[-1]), 6))
print(
    "Last sensitivity d(y^2)/da:",
    round(float(solution["y squared"].sensitivities["a"][-1]), 6),
)

```

Expected output is close to:

```

PyBaMM version: 25.12.1
Last value of y^2: 7.389056
Last sensitivity d(y^2)/da: 14.778112

```

Those numbers matter. The value 7.389056 is approximately e^2 , because $y(t) = e^t$ when $a = 1$ and $y^2 = e^{2t}$. The sensitivity is approximately $2e^2$, which is exactly what you want. This tiny check proves three things at once: PyBaMM imports correctly, the IDAKLU solver is available, and sensitivity propagation is working.

5.2.4 Confirm Jupyter is using the intended interpreter

Launch Jupyter:

```
jupyter lab
```

Then run:

```

import sys
import pybamm

```

```
print(sys.executable)
print(pybamm.__version__)
```

The executable should point into your intended environment, and the version should print 25.12.1.

5.2.5 Common install failures and fixes

1. **ModuleNotFoundError: No module named 'pybamm'**
Symptom: terminal and notebook both fail on `import pybamm`.
Fix: the package is not installed in the active environment. Re-activate the environment and reinstall.
2. `import pybamm` works in the terminal but fails in Jupyter
Symptom: the notebook kernel is attached to another interpreter.
Fix: register the active environment as a kernel, then switch the notebook to that kernel.
3. **AttributeError** or missing-method errors around sensitivities
Symptom: code from this chapter fails on `calculate_sensitivities` or `InputParameter`.
Fix: you are likely on the wrong PyBaMM release. Reinstall the exact pin `pybamm==25.12.1`.
4. The first DFN sensitivity solve takes longer than expected
Symptom: a cell appears to hang on the first call.
Fix: wait a little. The first processed model build is often slower than later solves. If later solves are also extremely slow, lower `t_eval` density before assuming something is broken.

5.3 Conceptual Bridge: From Textbook Identifiability to PyBaMM Input Parameters

In Textbook Chapter 8, the model was the star. You learned how concentration fields, potentials, kinetics, and transport combine to produce voltage. In this chapter, the model is no longer the only object on stage. We care about a second layer: the inverse problem. Given current and voltage data, what can we say about the parameters inside the model?

Mathematically, the forward model is a map from an input current history $i(t)$ and a parameter vector θ to a voltage response:

$$v(t) = \mathcal{F}_\theta\{i(t)\}. \quad (1)$$

Equation (1) is the compact version of everything you learned in the theory text. The inverse problem turns it around. We have measured or synthetic voltage data $v_{\text{data}}(t_n)$ at sample times t_n , and we try to find parameter values that make the model output match those data. In the simplest least-squares form,

$$\hat{\theta} = \arg \min_{\theta} \sum_{n=0}^{N-1} (v_{\text{model}}(t_n; \theta) - v_{\text{data}}(t_n))^2. \quad (2)$$

Equation (2) is easy to write and dangerously easy to misuse. It tempts beginners into believing that if the optimizer converges, the parameters must have been identified. That is false. A low residual means only that some parameter combination matched the chosen data under the chosen preprocessing and model assumptions. It does not mean each parameter is individually observable, nor that the recovered values are unique, nor that the fitted parameters will transfer to another protocol.

This is where identifiability enters. A parameter is informative under a dataset only if changing it changes the measured output in a distinguishable way. Local sensitivity is the first practical tool for asking that question:

$$S_j(t) = \frac{\partial v(t; \theta)}{\partial \theta_j}. \quad (3)$$

If $S_j(t)$ is tiny across the entire experiment, the data barely react to parameter θ_j . If two parameters have nearly collinear sensitivity curves, the optimizer may trade one against the other with little penalty. That is the software expression of what Textbook Chapter 10 described physically: a protocol must excite the physics you want to estimate.

PyBaMM represents this inverse-problem structure very cleanly. A parameter that you want to vary efficiently across many solves can be replaced with an `InputParameter`. In practice, that means taking a normally fixed quantity such as "Current function [A]" or "Ambient temperature [K]" and marking it as `[input]` inside the parameter set. PyBaMM then processes the model once and lets you supply different numeric values at solve time without rebuilding the whole symbolic system. That distinction matters a lot in optimization loops.

The mental mapping is:

Inverse-problem idea	PyBaMM object or pattern	Why it matters
Candidate fitted parameter	"[input]" inside <code>ParameterValues.update</code>	Lets one processed model be reused across many solves
Synthetic or measured data	NumPy arrays or Pandas columns	These become the target in the residual function
Residual vector	<code>model_output - measured_output</code>	This is what <code>scipy.optimize.least_squares</code> minimizes
Sensitivity matrix	<code>solution["variable"].B</code>	Provides local derivative information for ranking and Jacobians
Experimental design	Experiment or Interpolant current profile	Determines whether the data are informative at all

There are three habits worth internalizing before we touch code.

First, fitting a small subset of parameters around a literature baseline is standard practice. Trying to identify the entire DFN parameter set from one voltage trace is not. The full parameter space is too large, too correlated, and too weakly excited for that to be defensible.

Second, a low residual is not the same as physical truth. If you let the optimizer adjust a parameter the experiment does not meaningfully excite, it may still return a number. That number is often just compensating for some other mismatch. Good parameter-estimation work is therefore as much about excluding unjustified parameters as it is about recovering justified ones.

Third, sensitivity information is local. When PyBaMM gives you $\partial v / \partial \theta_j$, it is telling you what happens near the current baseline parameter set. That is exactly what we need for screening candidate parameters and building efficient local Jacobians, but it is not a global guarantee that the entire parameter landscape is well behaved.

So the bridge from theory to tool is this. Textbook Chapter 8 taught you what

the DFN and SPMe mean physically. Textbook Chapter 10 taught you that protocol richness controls observability. PyBaMM now gives you software objects that turn those ideas into a practical workflow: define a small candidate set, screen it with sensitivities, fit only what the data can support, and report the uncertainty honestly.

Guided Walkthrough 1

Screening Candidate Parameters with Sensitivities

Learning objective: Use PyBaMM's built-in sensitivity machinery to rank candidate parameters before you fit anything.

The most expensive mistake in parameter estimation is running an optimizer before you know whether the experiment can distinguish your candidate parameters at all. In this walkthrough, we will build a DFN model with four candidate inputs, simulate a moderate 5 A discharge, compute terminal-voltage sensitivities, normalize them, and rank the parameters by how strongly they affect the output.

This is the first place where PyBaMM stops being just a simulator and becomes an estimation tool. We are not yet fitting data. We are asking a smarter preliminary question: if terminal voltage is all we measure, which parameters are worth fitting under this protocol?

5.3.1 Walkthrough 1 code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pybamm

# Use a thermally coupled DFN so temperature-related parameters matter.
model = pybamm.lithium_ion.DFN(options={"thermal": "lumped"})

# Start from a published baseline chemistry.
parameter_values = pybamm.ParameterValues("Chen2020")

# Choose a small candidate set of scalar parameters that PyBaMM can treat as
↳ inputs.
```

```

# These are not the only possible choices. They are a teaching subset with
↳ distinct physics.
candidate_parameters = {
    "Current function [A]": 5.0,
    "Ambient temperature [K]": 298.15,
    "Initial concentration in electrolyte [mol.m-3]": 1000.0,
    "Total heat transfer coefficient [W.m-2.K-1]": 10.0,
}

parameter_values.update(
    {parameter_name: "[input]" for parameter_name in candidate_parameters}
)

solver = pybamm.IDAKLUSolver(rtol=1e-3, atol=1e-5)
simulation = pybamm.Simulation(
    model,
    parameter_values=parameter_values,
    solver=solver,
)

t_eval = np.linspace(0, 1200, 61)

solution = simulation.solve(
    [0, 1200],
    t_interp=t_eval,
    inputs=candidate_parameters,
    calculate_sensitivities=True,
)

terminal_voltage = solution["Terminal voltage [V]"].data

relative_sensitivities = {}
for parameter_name, baseline_value in candidate_parameters.items():
    raw_sensitivity = solution["Terminal voltage
↳ [V]"].sensitivities[parameter_name]
    relative_sensitivity = (
        baseline_value
        * raw_sensitivity
        / np.maximum(np.abs(terminal_voltage), 1e-6)
    )
    relative_sensitivities[parameter_name] = relative_sensitivity

# Rank the parameters by RMS relative sensitivity.
ranking_rows = []
for parameter_name, sensitivity_curve in relative_sensitivities.items():

```

```

rms_score = np.sqrt(np.mean(sensitivity_curve**2))
peak_score = np.max(np.abs(sensitivity_curve))
ranking_rows.append(
    {
        "parameter": parameter_name,
        "rms_relative_sensitivity": rms_score,
        "peak_absolute_relative_sensitivity": peak_score,
    }
)

ranking_table = (
    pd.DataFrame(ranking_rows)
    .sort_values("rms_relative_sensitivity", ascending=False)
    .reset_index(drop=True)
)

print(ranking_table)

# Build a correlation matrix between sensitivity shapes.
sensitivity_matrix = np.column_stack(
    [relative_sensitivities[name] for name in candidate_parameters]
)
correlation_matrix = np.corrcoef(sensitivity_matrix.T)

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

axes[0].plot(t_eval / 60, terminal_voltage, color="black", linewidth=2)
axes[0].set_xlabel("Time [min]")
axes[0].set_ylabel("Terminal voltage [V]")
axes[0].set_title("Baseline DFN discharge")
axes[0].grid(True, alpha=0.3)

for parameter_name, sensitivity_curve in relative_sensitivities.items():
    axes[1].plot(t_eval / 60, sensitivity_curve, label=parameter_name)

axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Relative sensitivity [-]")
axes[1].set_title("Voltage sensitivities")
axes[1].grid(True, alpha=0.3)
axes[1].legend(fontsize=8)

im = axes[2].imshow(correlation_matrix, vmin=-1, vmax=1, cmap="coolwarm")
axes[2].set_xticks(range(len(candidate_parameters)))
axes[2].set_yticks(range(len(candidate_parameters)))
axes[2].set_xticklabels(candidate_parameters.keys(), rotation=45, ha="right")

```

```

axes[2].set_yticklabels(candidate_parameters.keys())
axes[2].set_title("Sensitivity-shape correlation")
fig.colorbar(im, ax=axes[2], fraction=0.046, pad=0.04)

plt.tight_layout()
plt.show()

```

5.3.2 Walkthrough 1 code explanation

We import `numpy`, `pandas`, `matplotlib`, and `pybamm` because we need numerical arrays, a neat ranking table, plots, and the model itself. Nothing unusual there.

The model line chooses a DFN with lumped thermal coupling:

```
model = pybamm.lithium_ion.DFN(options={"thermal": "lumped"})
```

That `thermal` option matters. If we kept the model isothermal, temperature-related parameters such as ambient temperature and heat transfer coefficient would either have no effect or a much weaker effect on the terminal voltage. Because we want to screen thermal and electrochemical candidates together, the thermal state must be solved dynamically.

We then load the `Chen2020` parameter set as a clean literature baseline. This is standard practice. We are not starting from random guesses. We are starting from a defensible chemistry description and asking which deviations around that description the data can see.

The `candidate_parameters` dictionary defines four scalar inputs. "Current function [A]" is the most immediately observable quantity under discharge. "Ambient temperature [K]" influences kinetics and transport through temperature-dependent constitutive laws. "Initial concentration in electrolyte [mol.m-3]" acts here as a proxy for salt inventory and electrolyte-state uncertainty. "Total heat transfer coefficient [W.m-2.K-1]" controls how quickly heat leaves the lumped thermal domain.

Then comes the key PyBaMM trick:

```

parameter_values.update(
    {parameter_name: "[input]" for parameter_name in candidate_parameters}
)

```

This tells PyBaMM that each of those named quantities should be supplied at solve time rather than baked permanently into the processed model. That is

exactly what makes repeated solves inside screening or optimization loops practical.

We choose the IDAKLU solver because it is the recommended PyBaMM solver for sensitivity calculations. The tolerances `rtol=1e-3` and `atol=1e-5` are a compromise between speed and stability for a teaching example. If you tighten them, the solve becomes slower. If you loosen them too much, the sensitivity curves may become noisy enough to confuse the ranking.

The solve call is the central line:

```
solution = simulation.solve(
    [0, 1200],
    t_interp=t_eval,
    inputs=candidate_parameters,
    calculate_sensitivities=True,
)
```

The `t_interp=t_eval` argument is important. PyBaMM only returns sensitivities at the time points used to represent the solution. By forcing the interpolated output grid to be exactly the time grid we want, we make later plotting and ranking straightforward.

Once the solve is complete, `solution["Terminal voltage [V]"].sensitivities[parameter_name]` gives the derivative of voltage with respect to the chosen parameter at every time point. Raw sensitivities have units, so direct comparison is misleading. That is why we convert to the relative form

$$S_j^{\text{rel}}(t) = \frac{\theta_j}{V(t)} \frac{\partial V(t)}{\partial \theta_j}. \quad (4)$$

Equation (4) tells us the fractional influence of a fractional change in each parameter. The division by voltage magnitude prevents the comparison from being dominated by units.

We then compute two simple scalar scores from each sensitivity curve: its RMS magnitude and its peak absolute magnitude. The RMS score is the better default ranking metric because it reflects influence across the entire protocol rather than at one isolated instant.

Finally, we compute a correlation matrix between sensitivity shapes. This is not a formal identifiability proof, but it is a strong warning tool. If two sensitivity curves are nearly perfectly correlated, the optimizer may struggle to tell the

associated parameters apart.

5.3.3 Walkthrough 1 expected output

Your printed table should rank "Current function [A]" highest by RMS relative sensitivity. "Ambient temperature [K]" is usually next for this thermally coupled discharge, while the electrolyte initial concentration and heat-transfer coefficient are lower. The exact numbers will vary slightly with solver tolerances, but the ordering should be broadly stable.

The left plot, titled **Baseline DFN discharge**, should show time on the horizontal axis from 0 to 20 minutes and terminal voltage on the vertical axis in volts. The curve should begin a little above 4.0 V and slope steadily downward. If your voltage increases with time or starts below cutoff, something is wrong.

The middle plot, titled **Voltage sensitivities**, should show four curves with visibly different shapes. The current sensitivity should have the largest magnitude and should mostly remain negative, meaning that increasing discharge current lowers terminal voltage. The ambient-temperature sensitivity should be smaller and often positive over much of the trajectory, meaning a warmer cell tends to sustain a slightly higher voltage under load. The other two curves should be weaker and more protocol-dependent.

The right plot, titled **Sensitivity-shape correlation**, should be a colored heatmap with 1 on the diagonal. Off-diagonal entries close to 1 or -1 indicate parameter effects with similar shapes. Those pairs are harder to separate from voltage alone.

5.3.4 Walkthrough 1 what could go wrong

1. The solve fails with a convergence or event error

Symptom: PyBaMM stops before 1200 s, or reports a voltage-cutoff event too early.

Fix: reduce the current from 5.0 A to 3.0 A for a first run, or shorten the simulation window. High-rate DFN sensitivity solves are more fragile than plain forward solves.

2. One or more sensitivity curves are identically zero

Symptom: a plotted curve lies exactly on zero at all times.

Fix: first check whether the parameter actually influences the chosen model

configuration. For example, thermal parameters are uninformative in an isothermal model. If the physics is right, confirm that the parameter name matches PyBaMM's exact key.

3. The correlation matrix is full of nan

Symptom: the heatmap fails or contains missing values.

Fix: one of the sensitivity curves is constant. This usually means that parameter does not affect the output on your time window. Remove it from the correlation calculation or pick a richer protocol.

5.3.5 Walkthrough 1 reflection

This exercise taught the most important pre-fit habit in the chapter: do not optimize blindly. You now have a concrete PyBaMM workflow for screening candidate parameters, ranking them by output influence, and spotting pairs whose effects are too similar to separate cleanly. We will reuse this logic every time we decide whether a fitting problem is scientifically justified.

Guided Walkthrough 2

Fitting a Two-Parameter Synthetic Problem with

least_squares

Learning objective: Build a reusable least-squares fitting loop around a PyBaMM forward model and use analytic sensitivities as the Jacobian.

Now that we know how to screen parameters, we can fit a small subset responsibly. We will generate synthetic voltage data from a thermally coupled DFN, add a little measurement noise, and then recover two parameters: discharge current and ambient temperature.

This is a teaching shortcut, not standard experimental practice. In a real battery test, current is measured directly by the cycler, so you would rarely estimate it from voltage. We are including it because it gives a parameter with a strong, clean influence on the model output, which makes the mechanics of the optimizer easy to see. Standard practice starts with one or two strongly observable parameters before moving to harder quantities.

5.3.6 Walkthrough 2 code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import least_squares
import pybamm

rng = np.random.default_rng(42)

model = pybamm.lithium_ion.DFN(options={"thermal": "lumped"})
parameter_values = pybamm.ParameterValues("Chen2020")
parameter_values.update(
    {
        "Current function [A]": "[input]",
        "Ambient temperature [K]": "[input]",
    }
)

solver = pybamm.IDAKLUSolver(rtol=1e-3, atol=1e-5)
simulation = pybamm.Simulation(
    model,
    parameter_values=parameter_values,
    solver=solver,
)

t_eval = np.linspace(0, 1200, 60)
true_parameters = {
    "Current function [A]": 5.0,
    "Ambient temperature [K]": 298.15,
}

synthetic_solution = simulation.solve(
    [0, 1200],
    t_interp=t_eval,
    inputs=true_parameters,
)

true_voltage = synthetic_solution["Terminal voltage [V]"].data
measured_voltage = true_voltage + 0.0015 * rng.normal(size=t_eval.size)

parameter_names = ["Current function [A]", "Ambient temperature [K]"]
lower_bounds = np.array([1.0, 273.15])
upper_bounds = np.array([10.0, 323.15])
```

```

initial_guess = np.array([4.0, 305.15])

cache = {}

def solve_model_and_jacobian(theta):
    cache_key = tuple(np.round(theta, 10))
    if cache_key in cache:
        return cache[cache_key]

    inputs = dict(zip(parameter_names, theta))
    solution = simulation.solve(
        [0, 1200],
        t_interp=t_eval,
        inputs=inputs,
        calculate_sensitivities=True,
    )
    voltage = solution["Terminal voltage [V]"].data
    jacobian = np.column_stack(
        [
            solution["Terminal voltage [V]"].sensitivities[name]
            for name in parameter_names
        ]
    )
    cache[cache_key] = (voltage, jacobian)
    return voltage, jacobian

def residual_vector(theta):
    voltage, _ = solve_model_and_jacobian(theta)
    return voltage - measured_voltage

def residual_jacobian(theta):
    _, jacobian = solve_model_and_jacobian(theta)
    return jacobian

result = least_squares(
    residual_vector,
    x0=initial_guess,
    jac=residual_jacobian,
    bounds=(lower_bounds, upper_bounds),
    verbose=1,
)

```

```

fitted_voltage, fitted_jacobian = solve_model_and_jacobian(result.x)
residuals = fitted_voltage - measured_voltage

degrees_of_freedom = len(measured_voltage) - len(result.x)
residual_variance = 2 * result.cost / degrees_of_freedom
covariance_matrix = residual_variance * np.linalg.inv(
    fitted_jacobian.T @ fitted_jacobian
)
standard_errors = np.sqrt(np.diag(covariance_matrix))
confidence_interval_95 = 1.96 * standard_errors

report = pd.DataFrame(
    {
        "parameter": parameter_names,
        "true_value": [true_parameters[name] for name in parameter_names],
        "initial_guess": initial_guess,
        "fitted_value": result.x,
        "std_error": standard_errors,
        "half_width_95pct_ci": confidence_interval_95,
    }
)

print(report)
print("RMSE [V]:", np.sqrt(np.mean(residuals**2)))

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].plot(t_eval / 60, measured_voltage, "o", ms=4, label="Synthetic
↳ data")
axes[0].plot(t_eval / 60, fitted_voltage, "--", lw=2.5, label="Fitted DFN")
axes[0].set_xlabel("Time [min]")
axes[0].set_ylabel("Terminal voltage [V]")
axes[0].set_title("Model fit to synthetic data")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

axes[1].plot(t_eval / 60, 1000 * residuals, color="tab:red", lw=2)
axes[1].axhline(0, color="black", linestyle="--", linewidth=1)
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Residual [mV]")
axes[1].set_title("Fit residuals")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()

```

```
plt.show()
```

5.3.7 Walkthrough 2 code explanation

The first half of the script constructs the forward model exactly once. That is worth emphasizing. In PyBaMM estimation workflows, you want to process the model once and vary only input parameters at solve time whenever possible. Rebuilding the processed model on every optimization iteration is a needless performance penalty.

We again use a lumped-thermal DFN and update two parameters to `[input]`. These are the two coordinates of our optimization vector θ .

The synthetic data are created by solving the model at the known `true_parameters`, then adding Gaussian noise of standard deviation `1.5 mV`. That noise level is realistic enough to matter but small enough that the fit should still recover the truth closely. Synthetic data are invaluable in early estimation work because they separate algorithmic failure from data-quality failure. If you cannot recover parameters from your own synthetic data, you are not ready to trust the pipeline on experiments.

The `cache` dictionary is not mathematically required, but it is practically useful. `least_squares` may ask for the residual and the Jacobian at the same point, and without caching you would perform the same solve twice. The cache key rounds the parameter vector so floating-point noise does not create pointless duplicate entries.

Inside `solve_model_and_jacobian`, the line

```
solution = simulation.solve(
    [0, 1200],
    t_interp=t_eval,
    inputs=inputs,
    calculate_sensitivities=True,
)
```

does the heavy lifting. PyBaMM returns both the voltage vector and its sensitivities with respect to each fitted parameter on the same time grid. The Jacobian for `least_squares` is therefore just a column stack of those sensitivity curves:

$$J_{n,j} = \frac{\partial v(t_n; \theta)}{\partial \theta_j}. \quad (5)$$

This is one of the cleanest advantages of PyBaMM over ad hoc battery fitting scripts. You do not need to hand-derive finite-difference code for each model output. The solver carries forward sensitivities for you.

The residual function returns the vector

$$r_n(\theta) = v_{\text{model}}(t_n; \theta) - v_{\text{data}}(t_n), \quad (6)$$

and `least_squares` minimizes $\sum_n r_n^2$. We supply explicit bounds because unconstrained physical parameters are an invitation to nonsense. Bounds are not a crutch. They are one of the ways you encode prior physical realism into the inverse problem.

After fitting, we compute a local covariance estimate from the Jacobian:

$$\text{Cov}(\hat{\theta}) \approx \hat{\sigma}^2 (J^\top J)^{-1}, \quad (7)$$

with

$$\hat{\sigma}^2 = \frac{2 \text{cost}}{N - p}, \quad (8)$$

because SciPy reports `cost = \frac{1}{2} \sum_n r_n^2`. This covariance formula is a local linear approximation, not a global truth. That distinction matters. The 95% intervals derived from it are useful for honesty, but they are not Bayesian credible intervals and they are not reliable when the problem is strongly nonlinear or poorly conditioned.

5.3.8 Walkthrough 2 expected output

The printed report should recover a current very close to `5.0 A` and an ambient temperature close to `298.15 K`. On a typical run with the random seed shown here, the recovered values land within a few thousandths of an ampere and within about `1 K` of the true temperature. Your exact numbers may differ slightly if solver tolerances or noise level change, but they should be close.

The reported RMSE should be on the order of the noise level, roughly one to a few millivolts. If the RMSE is tens of millivolts, the fit did not work.

The left plot should show circular data markers and a smooth fitted curve almost lying on top of them. Time runs from `0` to `20` minutes, voltage slopes downward, and the fit should be visually indistinguishable from the noisy data except for tiny point-to-line deviations. The right plot should show residuals

centered near zero with no obvious drift or curvature. If the residuals show a systematic bow shape, you are not fitting the right physics even if the RMSE looks moderate.

5.3.9 Walkthrough 2 what could go wrong

1. `LinAlgError` when computing the covariance matrix

Symptom: `np.linalg.inv(J.T @ J)` fails.

Fix: the Jacobian is close to singular, meaning the parameters are too correlated or one is too weakly observable. This is an identifiability warning, not just a numerical nuisance. Revisit Walkthrough 1 and simplify the candidate set.

2. The optimizer stops at a bound

Symptom: the fitted current or temperature equals the exact lower or upper bound.

Fix: sometimes the initial guess is poor, but more often the data do not constrain that parameter enough. Widen the protocol, improve the initial guess, or pick a better parameter subset rather than blindly expanding the bounds.

3. The fit is good but the uncertainty is absurdly wide

Symptom: the curve overlays the data, but the confidence interval on a parameter is huge.

Fix: this is the classic signature of correlated parameters. The optimizer found one good point, but the data do not isolate that point strongly. Report that honestly.

5.3.10 Walkthrough 2 reflection

This exercise taught the central fitting pattern we will reuse for the rest of the manual: build one processed model, expose only a small candidate subset as inputs, generate a residual vector, pass an analytic Jacobian when available, and quantify local uncertainty instead of reporting point estimates alone. That pattern is the backbone of serious PyBaMM estimation work.

Guided Walkthrough 3

Building a Reusable CSV-to-Fit Pipeline

Learning objective: Wrap the estimation workflow into a reusable function that reads a standardized CSV and fits a small, defensible parameter subset.

A chapter example becomes research infrastructure only when you can point it at a file and reuse it. In this walkthrough, we will build exactly that: a pipeline that expects a CSV with `time_s`, `current_a`, and `voltage_v`, constructs a PyBaMM current interpolant, and estimates ambient temperature and initial SOC.

Why this parameter pair? Because it is honest for a first reusable pipeline.

Ambient temperature [K] is a true model parameter. `initial_soc` is an initial condition rather than a parameter in the strictest sense, but in real lab data it is often uncertain enough to deserve estimation or at least refinement. More importantly, this pair is usually recoverable from a dynamic current-voltage trace on a laptop without pretending that a single experiment can identify the entire electrochemical model.

5.3.11 Walkthrough 3 code

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import least_squares
import pybamm

def create_synthetic_csv(csv_path):
    csv_path.parent.mkdir(parents=True, exist_ok=True)

    time_s = np.linspace(0, 1200, 121)
    current_a = np.pieceswise(
        time_s,
        [
            time_s < 300,
            (time_s >= 300) & (time_s < 500),
            (time_s >= 500) & (time_s < 800),
            time_s >= 800,
        ],
        [5.0, 1.0, 4.0, 2.0],
    )

    model = pybamm.lithium_ion.SPM(options={"thermal": "lumped"})
```

```

parameter_values = pybamm.ParameterValues("Chen2020")
parameter_values.update({"Ambient temperature [K]": "[input]"})
parameter_values["Current function [A]"] = pybamm.Interpolant(
    time_s,
    current_a,
    pybamm.t,
)

simulation = pybamm.Simulation(
    model,
    parameter_values=parameter_values,
    solver=pybamm.IDAKLUSolver(rtol=1e-3, atol=1e-5),
)

solution = simulation.solve(
    t_interp=time_s,
    inputs={"Ambient temperature [K]": 299.15},
    initial_soc=0.82,
)

rng = np.random.default_rng(7)
voltage_v = solution["Terminal voltage [V]"].data + 0.0015 * rng.normal(
    size=time_s.size
)

synthetic_frame = pd.DataFrame(
    {
        "time_s": time_s,
        "current_a": current_a,
        "voltage_v": voltage_v,
    }
)

synthetic_frame.to_csv(csv_path, index=False)
return synthetic_frame

def standardize_battery_csv(csv_path):
    frame = pd.read_csv(csv_path)
    required_columns = {"time_s", "current_a", "voltage_v"}
    missing = required_columns.difference(frame.columns)
    if missing:
        raise ValueError(f"Missing required columns: {sorted(missing)}")

    frame = frame.copy()
    frame = frame.sort_values("time_s").reset_index(drop=True)

```

```

frame = frame.dropna(subset=["time_s", "current_a", "voltage_v"])

# Enforce numeric types and monotonic time.
frame["time_s"] = pd.to_numeric(frame["time_s"])
frame["current_a"] = pd.to_numeric(frame["current_a"])
frame["voltage_v"] = pd.to_numeric(frame["voltage_v"])

if (np.diff(frame["time_s"]) <= 0).any():
    raise ValueError("time_s must be strictly increasing")

return frame

def build_simulation_from_dataframe(frame):
    model = pybamm.lithium_ion.SPMe(options={"thermal": "lumped"})
    parameter_values = pybamm.ParameterValues("Chen2020")
    parameter_values.update({"Ambient temperature [K]": "[input]"})
    parameter_values["Current function [A]"] = pybamm.Interpolant(
        frame["time_s"].to_numpy(),
        frame["current_a"].to_numpy(),
        pybamm.t,
    )

    simulation = pybamm.Simulation(
        model,
        parameter_values=parameter_values,
        solver=pybamm.IDAKLUSolver(rtol=1e-3, atol=1e-5),
    )
    return simulation

def fit_temperature_and_initial_soc(frame):
    simulation = build_simulation_from_dataframe(frame)
    time_s = frame["time_s"].to_numpy()
    measured_voltage = frame["voltage_v"].to_numpy()

    def residual(theta):
        ambient_temperature_k, initial_soc = theta
        solution = simulation.solve(
            t_interp=time_s,
            inputs={"Ambient temperature [K]": ambient_temperature_k},
            initial_soc=initial_soc,
        )
        predicted_voltage = solution["Terminal voltage [V]"].data
        return predicted_voltage - measured_voltage

```

```

result = least_squares(
    residual,
    x0=np.array([297.15, 0.75]),
    bounds=(np.array([273.15, 0.4]), np.array([323.15, 1.0])),
    jac="2-point",
)

fitted_solution = simulation.solve(
    t_interp=time_s,
    inputs={"Ambient temperature [K]": result.x[0]},
    initial_soc=result.x[1],
)
fitted_voltage = fitted_solution["Terminal voltage [V]"].data
residuals = fitted_voltage - measured_voltage

# Local one-at-a-time ranking around the fitted point.
delta_temperature = 1.0
delta_soc = 0.005

solution_temp_plus = simulation.solve(
    t_interp=time_s,
    inputs={"Ambient temperature [K]": result.x[0] + delta_temperature},
    initial_soc=result.x[1],
)
solution_soc_plus = simulation.solve(
    t_interp=time_s,
    inputs={"Ambient temperature [K]": result.x[0]},
    initial_soc=min(result.x[1] + delta_soc, 0.999),
)

temp_sensitivity_fd = (
    solution_temp_plus["Terminal voltage [V]"].data - fitted_voltage
) / delta_temperature
soc_sensitivity_fd = (
    solution_soc_plus["Terminal voltage [V]"].data - fitted_voltage
) / delta_soc

ranking = pd.DataFrame(
    {
        "quantity": ["Ambient temperature [K]", "initial_soc"],
        "rms_local_sensitivity": [
            np.sqrt(np.mean(temp_sensitivity_fd**2)),
            np.sqrt(np.mean(soc_sensitivity_fd**2)),
        ],
    },

```

```

    }
).sort_values("rms_local_sensitivity", ascending=False)

summary = {
    "ambient_temperature_k": result.x[0],
    "initial_soc": result.x[1],
    "rmse_v": float(np.sqrt(np.mean(residuals**2))),
    "fitted_voltage": fitted_voltage,
    "residuals": residuals,
    "ranking": ranking,
}
return summary

csv_path = Path("data/chapter5/synthetic_pulse_fit.csv")
create_synthetic_csv(csv_path)
frame = standardize_battery_csv(csv_path)
fit_summary = fit_temperature_and_initial_soc(frame)

print(frame.head())
print()
print("Recovered ambient temperature [K]:",
      ↪ round(fit_summary["ambient_temperature_k"], 3))
print("Recovered initial SOC [-]:", round(fit_summary["initial_soc"], 5))
print("RMSE [mV]:", round(1000 * fit_summary["rmse_v"], 3))
print()
print(fit_summary["ranking"])

fig, axes = plt.subplots(3, 1, figsize=(10, 9), sharex=True)

axes[0].plot(frame["time_s"] / 60, frame["current_a"], color="tab:blue",
             ↪ linewidth=2)
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Input current profile")
axes[0].grid(True, alpha=0.3)

axes[1].plot(frame["time_s"] / 60, frame["voltage_v"], "o", ms=3, label="CSV
             ↪ data")
axes[1].plot(
    frame["time_s"] / 60,
    fit_summary["fitted_voltage"],
    "-",
    linewidth=2.5,
    label="Fitted SPMe",
)

```

```

axes[1].set_ylabel("Voltage [V]")
axes[1].set_title("Measured vs fitted voltage")
axes[1].grid(True, alpha=0.3)
axes[1].legend()

axes[2].plot(
    frame["time_s"] / 60,
    1000 * fit_summary["residuals"],
    color="tab:red",
    linewidth=2,
)
axes[2].axhline(0, color="black", linestyle="--", linewidth=1)
axes[2].set_xlabel("Time [min]")
axes[2].set_ylabel("Residual [mV]")
axes[2].set_title("Residual trace")
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

5.3.12 Walkthrough 3 code explanation

This script is longer because it is real workflow code, not just a one-off notebook cell. We define four helper functions so that each stage of the pipeline is explicit and reusable.

`create_synthetic_csv` generates a file that mimics what a real standardized experiment file should look like: time in seconds, current in amps, voltage in volts. The current profile is deliberately piecewise constant rather than perfectly smooth, because parameter estimation benefits from excitation richness. We use an SPMe rather than a DFN here because the point of this walkthrough is reusable structure and quick iteration. Standard practice is to prototype estimation loops with SPMe or SPM, then rerun the shortlisted candidate set on DFN once the workflow is stable.

The `standardize_battery_csv` function is defensive data engineering. It checks that the required columns exist, sorts by time, drops null rows, enforces numeric conversion, and rejects non-monotonic time. None of that is glamorous, but every serious estimation pipeline needs it. Most fitting disasters are not solver disasters. They are quietly bad data.

`build_simulation_from_dataframe` is where we convert current samples into a PyBaMM object. The key line is:

```
parameter_values["Current function [A]"] = pybamm.Interpolant(
    frame["time_s"].to_numpy(),
    frame["current_a"].to_numpy(),
    pybamm.t,
)
```

This line tells PyBaMM to treat the current as an interpolated function of model time. That is how we move from a CSV trace to a continuous forcing function the model can solve against.

`fit_temperature_and_initial_soc` then wraps the inverse problem. We are fitting only two quantities: ambient temperature and initial SOC. Because `initial_soc` is passed through the `solve` call rather than as a standard input parameter, we use `jac="2-point"` in `least_squares` instead of an analytic Jacobian. That is a perfectly valid teaching choice. We already used PyBaMM's analytic sensitivities in Walkthroughs 1 and 2 for the parameters that support them cleanly. Here the priority is building a generic CSV pipeline that works with an initial-condition fit as well.

After fitting, we compute a simple local ranking by finite difference. This is intentionally modest. It is not a full global identifiability analysis. It is a quick local sensitivity check around the fitted point so the pipeline returns not just numbers, but some clue about which recovered quantity the data were most sensitive to.

5.3.13 Walkthrough 3 expected output

The printed `head()` of the standardized frame should show three columns: `time_s`, `current_a`, and `voltage_v`. The first few current values should be `5.0` A, because the synthetic profile begins at high discharge load.

The recovered ambient temperature should be close to `299.15` K, and the recovered `initial_soc` should be close to `0.82`. Because the data contain noise and the problem is nonlinear, you should not expect perfect equality. Recovering temperature within about `1` K and SOC within a few thousandths is a good result here.

The current plot should show four horizontal plateaus over `20` minutes. The middle voltage plot should show dots and a smooth fitted line almost overlapping them. The residual plot should bounce around zero, mostly within a few millivolts. If you see structured residual swings that line up with current-step boundaries, the parameter pair is not capturing enough physics.

5.3.14 Walkthrough 3 what could go wrong

1. `ValueError: time_s must be strictly increasing`

Symptom: your standardized CSV fails immediately.

Fix: your source file likely has duplicate timestamps or a reset in the logging clock. Clean the file before fitting. Do not ignore this error. Interpolated current profiles need monotonic time.

2. The fitted `initial_soc` pegs at `1.0` or `0.4`

Symptom: the optimizer lands exactly on a bound.

Fix: either the data truly begin near a bound or the chosen model mismatch is being absorbed by SOC. That is a warning sign. Check whether the current sign convention and starting state are correct.

3. The residuals are small overall but spike at current transitions

Symptom: the line tracks the data well except right after current steps.

Fix: that usually means the model is missing a fast loss mechanism or the fit needs a parameter more directly tied to transient polarization. This is a classic case where a future ECM fit may complement the PyBaMM fit.

5.3.15 Walkthrough 3 reflection

This exercise turned a chapter pattern into a reusable research tool. You now have a file-format contract, a data-cleaning step, a forward-model builder, a fitting routine, and a local post-fit ranking. That is exactly the kind of scaffold we will keep extending in later chapters.

Dataset Integration

Using the Public Ecker Discharge Data

This chapter should touch real experimental data, and PyBaMM gives us a convenient small public dataset for doing exactly that. The Ecker discharge curves are bundled in the `pybamm-data` registry and exposed through `pybamm.DataLoader()`. The curves are taken from the parameterization and validation work associated with the `Ecker2015` parameter set, which makes them especially useful for teaching because the chemistry, model, and reference literature line up.

The practical attraction of these files is that they are tiny, easy to parse, and clean enough to focus on modeling rather than messy vendor formats. The limitation is equally important: they are only voltage-time pairs. Current is not stored as a full time series in the file because the tests are fixed-rate discharges. That means we must reconstruct the current column ourselves using the stated C-rate and the nominal capacity of the parameter set.

5.3.16 Where to get the data

Item	Source	Format	Approximate size	Notes
Ecker_1C.csv	pybamm.DataLoader() from the PyBaMM data registry	CSV, 2 columns	Small, laptop-friendly	Time [s], Voltage [V]
Ecker_5C.csv	pybamm.DataLoader() from the PyBaMM data registry	CSV, 2 columns	Small, laptop-friendly	Time [s], Voltage [V]

The data are distributed through the PyBaMM data registry and tied to the Ecker validation example in the official docs. The registry does not advertise a separate machine-readable license in the notebook output, so treat the files as research data accompanying the cited PyBaMM example and the original Ecker paper, and verify redistribution terms if you plan to publish them directly in another repository.

5.3.17 Column meanings and common pitfalls

The raw Ecker CSVs contain:

Raw column index	Meaning	Units
0	elapsed time	seconds
1	terminal voltage	volts

The common pitfalls are straightforward but important.

First, current is not included. You must reconstruct it. For a 1C discharge under the Ecker2015 parameter set, current is nominal capacity times 1. For a 5C discharge, it is nominal capacity times 5.

Second, sign convention matters. In PyBaMM's lithium-ion models, discharge current is taken as positive. That matches the way we will reconstruct the Ecker current column here. If you import data from another source later, do not assume the same convention.

Third, because these files hold only voltage and time, they are best for single-rate validation and light estimation tasks. They are not enough to justify broad claims about parameter identifiability by themselves.

5.3.18 Worked real-data example

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import least_squares
import pybamm

data_loader = pybamm.DataLoader()

raw_voltage_data = pd.read_csv(
    data_loader.get_data("Ecker_5C.csv"),
    header=None,
    names=["time_s", "voltage_v"],
)

parameter_values = pybamm.ParameterValues("Ecker2015")
nominal_capacity_ah = parameter_values["Nominal cell capacity [A.h]"]
discharge_current_a = 5.0 * nominal_capacity_ah

ecker_frame = raw_voltage_data.copy()
ecker_frame["current_a"] = discharge_current_a
ecker_frame = ecker_frame[["time_s", "current_a", "voltage_v"]]

output_path = Path("data/chapter5/ecker_5c_standardized.csv")
output_path.parent.mkdir(parents=True, exist_ok=True)
ecker_frame.to_csv(output_path, index=False)
```

```

# Build a thermally coupled validation model.
model = pybamm.lithium_ion.DFN(options={"thermal": "lumped"})
parameter_values = pybamm.ParameterValues("Ecker2015")
parameter_values.update({"Ambient temperature [K]": "[input]"})
parameter_values["Current function [A]"] = pybamm.Interpolant(
    ecker_frame["time_s"].to_numpy(),
    ecker_frame["current_a"].to_numpy(),
    pybamm.t,
)

simulation = pybamm.Simulation(
    model,
    parameter_values=parameter_values,
    solver=pybamm.IDAKLUSolver(rtol=1e-3, atol=1e-5),
)

time_s = ecker_frame["time_s"].to_numpy()
measured_voltage = ecker_frame["voltage_v"].to_numpy()

baseline_solution = simulation.solve(
    t_interp=time_s,
    inputs={"Ambient temperature [K]": 298.15},
    initial_soc=1.0,
)
baseline_voltage = baseline_solution["Terminal voltage [V]"].data

def residual(theta):
    ambient_temperature_k = theta[0]
    fitted_solution = simulation.solve(
        t_interp=time_s,
        inputs={"Ambient temperature [K]": ambient_temperature_k},
        initial_soc=1.0,
    )
    predicted_voltage = fitted_solution["Terminal voltage [V]"].data
    return predicted_voltage - measured_voltage

result = least_squares(
    residual,
    x0=np.array([298.15]),
    bounds=(np.array([273.15]), np.array([323.15])),
)

```

```

fitted_solution = simulation.solve(
    t_interp=time_s,
    inputs={"Ambient temperature [K]": result.x[0]},
    initial_soc=1.0,
)
fitted_voltage = fitted_solution["Terminal voltage [V]"].data

baseline_rmse_mv = 1000 * np.sqrt(np.mean((baseline_voltage -
↪ measured_voltage) ** 2))
fitted_rmse_mv = 1000 * np.sqrt(np.mean((fitted_voltage - measured_voltage)
↪ ** 2))

print("Standardized file written to:", output_path)
print("Nominal capacity [A.h]:", nominal_capacity_ah)
print("Assumed 5C discharge current [A]:", discharge_current_a)
print("Fitted ambient temperature [K]:", round(float(result.x[0]), 3))
print("Baseline RMSE [mV]:", round(float(baseline_rmse_mv), 3))
print("Fitted RMSE [mV]:", round(float(fitted_rmse_mv), 3))

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

axes[0].plot(time_s / 60, measured_voltage, "o", ms=3, label="Measured Ecker
↪ 5C")
axes[0].plot(time_s / 60, baseline_voltage, lw=2, label="Baseline Ecker2015")
axes[0].plot(time_s / 60, fitted_voltage, lw=2.5, label="After temperature
↪ fit")
axes[0].set_ylabel("Voltage [V]")
axes[0].set_title("Real-data comparison on Ecker 5C discharge")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

axes[1].plot(
    time_s / 60,
    1000 * (fitted_voltage - measured_voltage),
    color="tab:red",
    linewidth=2,
)
axes[1].axhline(0, color="black", linestyle="--", linewidth=1)
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Residual [mV]")
axes[1].set_title("Residual after one-parameter fit")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

This code is intentionally conservative. We fit only ambient temperature while holding `initial_soc=1.0`, because the file represents a named full-rate discharge and we want a real-data example that stays honest. You should expect the fitted temperature to remain in a plausible room-temperature neighborhood, not to wander to an absurd value. The fitted RMSE should improve relative to the baseline, but not magically drop to zero. That is the right lesson. Real data retain mismatch even after a sensible fit.

If you want a stronger estimation exercise on these files, use the 1C and 5C traces together and fit a shared parameter across both. That is much closer to the multi-condition logic used in publishable parameter studies.

5.4 A Brief Bayesian Detour

Frequentist least-squares fitting is the right default for this chapter because it is fast, transparent, and easy to debug. But you should know where the field goes next. When parameter correlations become severe, when you care more about uncertainty structure than about one best-fit point, or when prior information matters strongly, a Bayesian workflow becomes attractive.

The practical PyBaMM-adjacent tool for that ecosystem is PyBOP. The conceptual shift is simple. Instead of looking only for the best-fit parameter vector $\hat{\theta}$, you ask for a posterior distribution:

$$p(\theta \mid D) \propto p(D \mid \theta) p(\theta), \quad (9)$$

where D is the dataset, $p(D \mid \theta)$ is the likelihood, and $p(\theta)$ is the prior. In plain language: what parameter values remain plausible after combining the data with what we already believed?

That approach is powerful, but it is not a magic cure. If the experiment is uninformative, the posterior simply reveals the uninformative structure more honestly. That is still useful. In fact, that honesty is one of the strongest arguments for Bayesian estimation in battery modeling. It shows you when the data really cannot decide.

For this companion, treat Bayesian inference as the next layer after you are comfortable with Chapters 5 and 12. Get the forward model stable. Learn to screen parameters. Learn to inspect residual structure. Then add Bayesian machinery. If you reverse that order, you often end up sampling your way around a bad

deterministic pipeline.

Reproduction Exercise

Reproducing Figure 2 from Hallemans et al. (2024)

The paper we will work from is:

Hallemans, N., Courtier, N. E., Please, C. P., Planden, B., Dhoot, R., Timms, R., Chapman, S. J., Howey, D., and Duncan, S. R. (2024). *Physics-based battery model parametrisation from impedance data*. arXiv:2412.10896.

We will reproduce **Figure 2**, which the authors describe as an illustration of a time-domain simulation of the SPMe. The paper uses that figure to frame the central limitation of voltage-only fitting: current and voltage traces are useful, but often not informative enough to recover all grouped parameters of a physics-based model. That message belongs directly inside this chapter.

5.4.1 What makes this reproduction tricky

The paper gives us the figure, its purpose, and the fact that the synthetic study is built from a grouped SPMe linked to Chen2020-derived parameter values. What it does **not** fully specify in the published figure caption is the exact current waveform used to produce the upper panel. So this is a realistic reproduction task: the paper is scientifically useful, but not fully turnkey.

That means we need to make and document two choices:

1. We will use the built-in PyBaMM SPMe with the Chen2020 parameter set rather than the appendix-specific grouped-parameter implementation.
2. We will reconstruct a piecewise current waveform that matches the figure's *function* rather than claiming pixel-perfect identity.

That is not a failure of reproduction. It is exactly the sort of judgment call real reproduction work requires. The honest standard here is not “identical screenshot.” It is “same physical setup, same modeling intent, same qualitative behavior, and clearly documented deviations.”

5.4.2 Reproduction code

```
import numpy as np
import matplotlib.pyplot as plt
import pybamm

# Reconstructed pulse-rich current profile inspired by Hallemans et al.
# ↪ Figure 2.
time_s = np.arange(0, 3600 + 10, 10)
current_a = np.zeros_like(time_s, dtype=float)

current_a[(time_s >= 0) & (time_s < 600)] = 2.5
current_a[(time_s >= 600) & (time_s < 900)] = 0.5
current_a[(time_s >= 900) & (time_s < 1500)] = 3.0
current_a[(time_s >= 1500) & (time_s < 1800)] = 1.0
current_a[(time_s >= 1800) & (time_s < 2400)] = 3.5
current_a[(time_s >= 2400) & (time_s < 3000)] = 1.5
current_a[(time_s >= 3000) & (time_s <= 3600)] = 2.0

model = pybamm.lithium_ion.SPMe(options={"thermal": "lumped"})
parameter_values = pybamm.ParameterValues("Chen2020")
parameter_values.update({"Ambient temperature [K]": 298.15})
parameter_values["Current function [A]"] = pybamm.Interpolant(
    time_s,
    current_a,
    pybamm.t,
)

simulation = pybamm.Simulation(
    model,
    parameter_values=parameter_values,
    solver=pybamm.IDAKLUSolver(rtol=1e-3, atol=1e-5),
)

solution = simulation.solve(t_interp=time_s, initial_soc=0.9)
voltage_v = solution["Terminal voltage [V]"].data

fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

axes[0].plot(time_s / 60, current_a, color="tab:blue", linewidth=2)
axes[0].set_ylabel("Current [A]")
axes[0].set_title("Reconstructed current profile inspired by Hallemans et al.
    ↪ Figure 2")
axes[0].grid(True, alpha=0.3)
```



```

axes[1].plot(time_s / 60, voltage_v, color="black", linewidth=2)
axes[1].set_xlabel("Time [min]")
axes[1].set_ylabel("Voltage [V]")
axes[1].set_title("SPMe voltage response")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

5.4.3 How to judge whether the reproduction is successful

The top panel should show a stepped current profile over one hour with several changes in amplitude. The bottom panel should show a voltage trace that responds immediately to each current change, then relaxes into a slower downward trend as the discharge proceeds. Those are the key physical features of the published figure.

A successful reproduction here means:

- the protocol is pulse-rich rather than a boring constant-current sweep,
- the voltage reacts in the right qualitative way at each current transition,
- the overall trend remains physically plausible for an SPMc discharge,
- you document clearly that the exact waveform had to be reconstructed.

That is what “close enough” means in this case. If the paper had provided raw data, we would demand pointwise agreement. Because it does not, the appropriate standard is qualitative fidelity plus transparent documentation of the ambiguity.

5.4.4 Why this reproduction matters

This figure is valuable because it makes the chapter’s main warning visible. Even a pulse-rich time-domain trace contains limited information compared with a truly multi-condition parameterization campaign. If you later find yourself wanting to fit ten or fifteen electrochemical parameters from one or two voltage traces, come back to this figure and remember what the protocol is actually exciting.

5.5 Open-Ended Exercises

These exercises are where you stop following and start adapting. Work them before reading the solutions.

5.5.1 Exercise 1

Repeat Walkthrough 1 on the SPMe instead of the DFN. Keep the same four candidate parameters and the same 1200-second discharge. Does the ranking order change? Which sensitivity shapes become more or less correlated?

Hint: change only the model line at first. Keep the rest of the code identical so the comparison is fair.

5.5.2 Exercise 2

Modify Walkthrough 2 so that you fit current and the total heat-transfer coefficient instead of current and ambient temperature. Does the optimizer recover both parameters cleanly from the same dataset?

Hint: use the sensitivity ranking from Walkthrough 1 before you expect success. If the heat-transfer coefficient is weakly observable, the fit may look numerically stable while still being scientifically weak.

5.5.3 Exercise 3

Take the CSV pipeline from Walkthrough 3 and replace the piecewise-constant current profile with one that contains a rest period and a sharp high-current pulse. Refit `ambient_temperature` and `initial_soc`. Do the local sensitivity scores change?

Hint: protocols with sharp transitions often increase sensitivity to fast losses and initial-condition errors.

5.5.4 Exercise 4

Use both `Ecker_1C.csv` and `Ecker_5C.csv` together. Fit a single shared ambient temperature that minimizes the combined residual across both files. Does a parameter that fits one rate also help at the other rate?

Hint: concatenate the residual vectors from the two simulations inside one objective function. Shared-parameter multi-condition fitting is closer to publishable practice than single-trace fitting.

5.6 Worked Solutions to the Open-Ended Exercises

5.6.1 Solution 1

On SPMe, the candidate ranking usually stays qualitatively similar: current remains dominant, ambient temperature stays important in the thermally coupled setting, and the weaker transport and heat-transfer terms remain lower. The main difference is often runtime and slight shape changes in the smaller-magnitude sensitivities. That is itself informative. A reduced-order model may preserve the gross ranking while changing the exact local geometry of the inverse problem.

5.6.2 Solution 2

This exercise is meant to fail gracefully for many readers, and that is the lesson. The total heat-transfer coefficient often has a weaker and more delayed influence on terminal voltage than ambient temperature under a short discharge. In practice, the optimizer may recover current cleanly while leaving the heat-transfer coefficient poorly constrained or pushed toward a bound. That is not a software bug. It is an identifiability result.

5.6.3 Solution 3

A rest period plus a sharp pulse usually increases the relative importance of `initial_soc` and of parameters tied to instantaneous or near-instantaneous voltage response. If your local sensitivity ranking shifts after changing the protocol, that is exactly what should happen. Parameter ranking is not a universal property of the model. It is a joint property of the model and the experiment.

5.6.4 Solution 4

Joint fitting over `Ecker_1C.csv` and `Ecker_5C.csv` is the right direction. A single ambient temperature often improves consistency across the two traces more credibly than fitting each trace independently. The combined residual is usually larger than the best single-trace residual, but the result is more physically honest. That tradeoff is standard practice in multi-condition identification.

What Changes for Sodium-Ion?

The estimation workflow from this chapter transfers to sodium-ion work more directly than the parameter values do. The biggest change is not the optimizer. It

is the baseline parameterization you begin from and the data richness you can realistically obtain.

The first difference is OCV shape. Hard-carbon sodium-ion negative electrodes often contain flatter features and broader hysteresis-like behavior than common graphite lithium-ion systems. That can make voltage-only estimation less informative in certain SOC windows, especially for state-related quantities.

The second difference is public-data scarcity. On the lithium-ion side, we can lean on mature parameter sets such as Chen2020 and Ecker2015, along with public validation traces. On the sodium-ion side, you will often have to work from sparse literature tables, digitized curves, or custom parameter subsets. That makes sensitivity screening even more important, not less.

The third difference is validation strategy. In sodium-ion work, it is often more honest to claim that you calibrated an *effective* reduced parameter subset around a literature baseline than to claim full electrochemical identification. That is not a weakness. It is careful science in a data-sparse setting.

As this manual progresses, we will return to that point repeatedly. For sodium-ion publishable work, your credibility depends less on having a huge fitted table and more on showing that you chose the subset, protocol, and validation conditions rationally.

Chapter Summary and Skill Checklist

- You learned how to convert PyBaMM parameters into `InputParameter` objects for repeated solves.
- You used PyBaMM's built-in sensitivities to rank candidate parameters before fitting.
- You built a `least_squares` fitting loop around a DFN forward model and supplied an analytic Jacobian from sensitivities.
- You computed local uncertainty estimates from the fitted Jacobian and learned why they are only local approximations.
- You wrapped the workflow into a reusable CSV-to-fit pipeline using `pybamm.Interpolant`.
- You parsed and standardized real Ecker discharge data from the public PyBaMM data registry.

- You practiced reproduction on a recent paper while documenting ambiguity honestly instead of hiding it.

You should now be able to:

- decide whether a given protocol is informative enough for a chosen parameter subset,
- expose a PyBaMM parameter as `[input]` and vary it efficiently across solves,
- write a residual function for `scipy.optimize.least_squares`,
- extract terminal-voltage sensitivities from a PyBaMM solution,
- distinguish between a numerically good fit and a scientifically well-identified fit,
- standardize a current-voltage CSV into a format your fitting code can reuse,
- explain, in writing, why a given parameter was fitted and why another one was deliberately left fixed.

If you cannot check every box, revisit Walkthroughs 1 through 3 before moving on. Later chapters assume these habits, especially the habit of screening before fitting.

Deliverable

The deliverable from the plan is:

A working parameter-estimation pipeline that takes a CSV of current/voltage data and returns fitted DFN parameters with sensitivity rankings.

The practical way to approach that deliverable is in three passes.

First, make the CSV contract non-negotiable. Use the standardized three-column format from Walkthrough 3: `time_s`, `current_a`, `voltage_v`.

Second, begin with SPMe for speed while you debug the pipeline, then switch the final run to DFN after the code is stable and your candidate subset has already been screened. That is standard practice, not a compromise.

Third, keep the fitted subset small. A strong first deliverable is a notebook or script that:

- reads a CSV,
- plots the raw trace,

- ranks a short candidate list by local sensitivity,
- fits one or two justified quantities,
- writes a fit report with RMSE and parameter estimates,
- saves measured-vs-fitted and residual plots to `results/chapter5/`.

A worked partial target is the synthetic CSV pipeline from Walkthrough 3 plus the Ecker real-data example. If you can combine those two cleanly, you already have the skeleton of a publishable calibration appendix.

Further Practice and Reading

1. PyBaMM official notebook: *Sensitivities and data fitting using PyBaMM*. Bookmark it because it is the canonical reference for the `calculate_sensitivities=True` workflow.
2. PyBaMM official notebook: *Comparing with Experimental Data*. This is the cleanest entry point for the Ecker validation traces.
3. Hallemans et al. (2024), *Physics-based battery model parametrisation from impedance data*, arXiv:2412.10896. Read it for a modern view of why voltage-only fitting is informative but limited.
4. Marquis, S. G., et al. (2019), *An asymptotic derivation of a single particle model with electrolyte*. This remains essential for understanding why reduced-order electrochemical models are so useful in fitting workflows.
5. PyBOP documentation and repository. Do not start there, but do bookmark it for the moment when you are ready to move from point estimation to uncertainty-aware inference.

6

Equivalent Circuit Models from Scratch

This chapter is where the battery modeling story changes scale. In Part II we worked with electrochemical models in PyBaMM because they preserve the physics you learned in the theory text. In this chapter we deliberately step down to models that are much simpler, much faster, and much more common inside real battery-management systems: equivalent circuit models, or ECMs. By the time you finish, you will be able to build R_{int} , $1RC$, and $2RC$ cell models from first principles in MATLAB, identify their parameters from pulse data, build SOC-dependent lookup tables in the Plett style, and validate the resulting model with the kind of discipline reviewers expect when they read a BMS paper.

Keep Textbook Chapter 10 open as you work. This chapter operationalizes the ECM material from that chapter, especially the reasons ECMs remain dominant in embedded BMS work even though DFN-style models are physically richer. The circuit elements are not magic. The OCV source is the equilibrium thermodynamics you already know. The series resistor approximates the immediate ohmic drop from current collectors, electrolyte, and contact resistance. The RC branches stand in for slower polarization and diffusion effects that a full porous-electrode model would resolve directly. If Textbook Chapter 10 gave you the intuition, this lab gives you the workflow.

This chapter also leans on Textbook Chapter 3, where you learned to interpret

capacity, SOC, usable energy, rate capability, and hysteresis, and on the sodium-ion chemistry chapter, where you saw that sodium-ion full-cell OCV curves can look very different from the familiar lithium-ion NMC/graphite case. Those differences matter here. A battery-management algorithm never sees “the chemistry” directly. It sees current, voltage, temperature, and maybe a few carefully designed lookup tables. If those tables were identified casually, the entire estimator stack that sits on top of them inherits the error.

Publishable ECM work is not about drawing a resistor-capacitor schematic and calling it a day. It is about making a sequence of modeling decisions honestly. Which model order is justified by the data? What excitation protocol makes the parameters observable? How do you separate OCV from dynamic polarization? How do you validate on data the model did not see during fitting? What error level is small enough to be useful and large enough to keep you humble? Those questions are more important than the exact optimizer you use.

We will move deliberately. First we will build ECMs from scratch in plain MATLAB so every state and parameter has a physical interpretation. Then we will fit a single pulse so you can see exactly where R_0 , R_1 , C_1 , R_2 , and C_2 come from. After that we will build SOC-dependent parameter tables from a full synthetic HPPC suite, because that workflow is still the clearest way to learn the craft. Only once the logic is secure will we bring in a real public dataset. For this chapter we will use the CALCE Samsung INR 18650-20R open-access files, whose workbook structure we will parse directly in MATLAB. The publicly exposed CALCE portal is strongest on low-current OCV and dynamic-test workbooks rather than on a tidy, one-click “HPPC” archive, so we will use the real CALCE OCV data to build the OCV table and keep the pulse-identification workflow tool-agnostic so you can drop in any HPPC file once you have one.

One more expectation-setting note matters here. This chapter is intentionally more procedural than the PyBaMM chapters. In PyBaMM, the software often carries a lot of the bookkeeping for you. In ECM work, especially in MATLAB, you will do much more of the bookkeeping yourself. That is a feature, not a bug. It is how you learn where every assumption enters.

6.1 Prerequisites Check

- Required software: MATLAB R2024b or newer recommended

- Required toolboxes for the core chapter: **Optimization Toolbox**
- Optional for the last guided walkthrough: **Simulink**, **Simscape**, and **Simscape Battery R2025a+**
- Required textbook chapters: Textbook Chapter 10 is essential; Textbook Chapter 3 is strongly recommended; your sodium-ion chemistry chapter is recommended for the sodium-ion adaptation section
- Required prior lab chapters: Lab Chapters 1 and 2 are assumed; Lab Chapters 3 through 5 are not required, but they help if you want to compare ECM and PyBaMM workflows later
- Estimated time: 14 to 18 hours for the full chapter, or 10 to 12 hours if you stop after the core MATLAB workflow and skip the optional Simscape Battery section

If `lsqcurvefit`, `lsqnonlin`, or `readtable` still feel unfamiliar, revisit the optimization and data-handling material from Lab Chapter 2 before you begin. If the distinction between OCV, polarization, and ohmic drop feels fuzzy, pause and reread the equivalent-circuit sections of Textbook Chapter 10 before you touch the first code block.

6.2 Environment Setup

The core of this chapter is plain MATLAB plus the Optimization Toolbox. We will keep the optional Simscape Battery section explicitly separate so the chapter remains runnable even if you only have base MATLAB and optimization.

6.2.1 Verify MATLAB and toolbox availability

Open MATLAB and run:

```
ver
license('test', 'Optimization_Toolbox')
license('test', 'SIMULINK')
license('test', 'Simscape')
license('test', 'Simscape_Battery')
```

Expected behavior:

- `ver` should print your MATLAB release plus installed toolboxes.
- `license('test', 'Optimization_Toolbox')` should return 1.

- The Simulink and Simscape checks may return 0; that is fine for the core chapter.

If you do not have the Optimization Toolbox, the core nonlinear fits in this chapter will not run as written. You can still read the chapter, but the code blocks assume `lsqcurvefit` and `lsqnonlin` are available.

6.2.2 Create a clean working folder

Run this once at the top of a fresh Live Script or MATLAB session:

```
chapterRoot = fullfile(pwd, "chapter6_ecm_workspace");
if ~exist(chapterRoot, "dir")
    mkdir(chapterRoot);
end
cd(chapterRoot);

fprintf("Working folder: %s\n", chapterRoot);
```

Expected output:

```
Working folder: ../../chapter6_ecm_workspace
```

The exact path will differ on your machine. The point is not the path itself. The point is that later in the chapter we will save lookup tables, exported CSV files, and MATLAB MAT-files here so the workflow stays reproducible.

6.2.3 Run a minimal "hello ECM" sanity check

Paste the following into a new script and run it end to end:

```
clear; close all; clc;

time_s = (0:1:300).';
current_a = zeros(size(time_s));
current_a(time_s >= 20 & time_s < 140) = 2.0;

capacity_ah = 2.3;
initial_soc = 0.90;
ocv_soc = [0.00 0.10 0.20 0.40 0.60 0.80 1.00].';
ocv_v = [3.00 3.30 3.45 3.62 3.78 3.95 4.18].';

R0 = 0.015;
R1 = 0.012;
C1 = 90;
```

```

soc = zeros(size(time_s));
v_rc = zeros(size(time_s));
voltage_v = zeros(size(time_s));

soc(1) = initial_soc;
dt = time_s(2) - time_s(1);
alpha = exp(-dt / (R1 * C1));

for k = 1:numel(time_s)
    if k > 1
        soc(k) = soc(k - 1) - current_a(k - 1) * dt / (3600 * capacity_ah);
        soc(k) = min(max(soc(k), 0), 1);
        v_rc(k) = alpha * v_rc(k - 1) + R1 * (1 - alpha) * current_a(k - 1);
    end

    ocv_now = interp1(ocv_soc, ocv_v, soc(k), "pchip", "extrap");
    voltage_v(k) = ocv_now - current_a(k) * R0 - v_rc(k);
end

figure("Color", "w");
subplot(2, 1, 1);
plot(time_s, current_a, "LineWidth", 1.5);
grid on;
xlabel("Time [s]");
ylabel("Current [A]");
title("Hello ECM current profile");

subplot(2, 1, 2);
plot(time_s, voltage_v, "LineWidth", 1.8);
grid on;
xlabel("Time [s]");
ylabel("Terminal voltage [V]");
title("Hello ECM voltage response");

fprintf("Initial voltage: %.3f V\n", voltage_v(1));
fprintf("Voltage at 60 s: %.3f V\n", voltage_v(time_s == 60));
fprintf("Voltage at 200 s: %.3f V\n", voltage_v(time_s == 200));

```

Expected output:

- The top subplot should show a single rectangular current pulse from 20 s to 140 s at 2.0 A.
- The bottom subplot should start near 4.04 V, drop immediately when the current pulse begins, bend downward slightly more as the RC branch charges,

then recover upward once the pulse ends.

- The printed voltages should be physically plausible and should fall roughly in this range:

```
Initial voltage: 4.040 V
Voltage at 60 s: 3.980 V
Voltage at 200 s: 4.000 V
```

The exact values will differ by a few millivolts if you alter the OCV table or parameters, but the qualitative shape should match exactly.

6.2.4 Optional setup for the Simscape Battery section

If you want the last guided walkthrough to run, you need Simscape Battery in R2025a or newer because we will use the `hppcTest`, `fitECM`, and `parameterizeEquivalentCircuitBlock` workflow introduced there.

You can verify those functions exist with:

```
which hppcTest
which fitECM
which parameterizeEquivalentCircuitBlock
```

If MATLAB prints valid file paths, the optional section is available. If it prints `'hppcTest' not found`, skip that section without guilt. The core chapter is still complete.

6.2.5 Common setup failures and fixes

1. Undefined function `'lsqcurvefit'` for input arguments of type `'double'`
Symptom: the first parameter-identification walkthrough fails immediately.
Fix: the Optimization Toolbox is not installed or licensed. Confirm with `license('test', 'Optimization_Toolbox')`.
2. Error using `readtable` when opening the CALCE workbook
Symptom: MATLAB reads the wrong sheet or turns text headers into awkward variable names.
Fix: use the exact sheet names shown later in the chapter and set `VariableNamingRule` to `"preserve"` so MATLAB does not silently rewrite headers.
3. The plotted voltage rises when discharge current is applied

Symptom: your sign convention is flipped.

Fix: throughout this chapter, the core hand-built ECM uses `positive current = discharge`. The CALCE Arbin files use `positive current = charge`, `negative current = discharge`, so we explicitly normalize them when we read them.

4. The OCV interpolation throws an error about nonmonotonic sample points

Symptom: `interp1` complains when you pass a decreasing SOC vector.

Fix: if a branch was recorded from `SOC = 1` down to `SOC = 0`, flip it with `flipud` before interpolating.

6.3 Conceptual Bridge: From Textbook ECM Theory to MATLAB State Updates

In Textbook Chapter 10, you learned why ECMs survive in serious battery-management work even when everyone agrees they are less physical than DFN-style electrochemical models. The short answer is that BMS algorithms live under severe computational, sensing, and calibration constraints. They need models that are fast, robust, and observable from terminal measurements. An ECM gives up mechanistic detail in exchange for exactly that.

The central ECM equation for this chapter is

$$V_t(t) = U_{oc}(z(t)) - I(t)R_0 - \sum_{i=1}^n v_i(t), \quad (1)$$

where V_t is the terminal voltage, $U_{oc}(z)$ is the open-circuit voltage as a function of SOC z , R_0 is the instantaneous ohmic resistance, and each v_i is the voltage across a polarization branch. The SOC state evolves by coulomb counting,

$$\dot{z}(t) = -\frac{I(t)}{3600Q}, \quad (2)$$

where Q is the usable cell capacity in ampere-hours and the factor of 3600 converts ampere-seconds to ampere-hours. For a standard Thevenin 1RC model, the branch state obeys

$$\dot{v}_1(t) = -\frac{1}{R_1C_1}v_1(t) + \frac{1}{C_1}I(t). \quad (3)$$

For a 2RC model, we duplicate that structure:

$$\dot{v}_i(t) = -\frac{1}{R_i C_i} v_i(t) + \frac{1}{C_i} I(t), \quad i \in \{1, 2\}. \quad (4)$$

This is the software translation of the circuit diagrams you saw in the textbook. The OCV source handles equilibrium behavior. $R\theta$ handles the immediate voltage step at a current transition. The RC branches handle relaxation on one or more time scales. In a full electrochemical model those time scales would arise from charge-transfer kinetics, electrolyte transport, solid diffusion, and sometimes thermal coupling. In an ECM we do not try to isolate each mechanism cleanly. We let a small number of branches approximate the aggregate dynamic behavior seen at the terminals.

That approximation only works if the experiment excites the dynamics we want to identify. This is why HPPC-style data are still so widely used. A pulse gives you several different observables at once:

- the immediate voltage jump estimates $R\theta$,
- the short relaxation helps identify the fast branch,
- the longer relaxation helps identify the slow branch,
- the rest windows anchor the local OCV.

This is also why standard practice in ECM work is more disciplined than beginners often assume. You do not fit all parameters everywhere all at once if you can avoid it. You isolate what you can from the pulse geometry, build an OCV table from low-current or rest-rich data, and then use nonlinear least squares only for what genuinely remains coupled.

The state update in MATLAB becomes especially transparent when we discretize the branch dynamics exactly over a fixed sample period dt . For one branch,

$$v_{i,k+1} = \alpha_i v_{i,k} + R_i(1 - \alpha_i)I_k, \quad \alpha_i = \exp\left(-\frac{dt}{R_i C_i}\right). \quad (5)$$

Equation (5) is one of the most useful formulas in this chapter. It shows three things immediately. First, each branch has a time constant $\tau_i = R_i C_i$. Second, the branch voltage is bounded by the current-scaled resistance IR_i . Third, if you know R_i and C_i , the simulation step is just a few floating-point operations. That is why ECMs are so attractive inside estimators.

There is one more bridge we need before we touch the real data. An ECM is not “identified” when the optimizer returns a number. It is identified when four things

are true at the same time:

1. the parameters came from a protocol rich enough to make them observable,
2. the fitted values are physically plausible and vary smoothly with SOC,
3. the model predicts held-out data credibly,
4. the chosen order is justified by the error reduction it buys.

That last point is worth pausing on. A 2RC model is not automatically “better” than a 1RC model just because it fits the training pulse more closely. It is better only if the extra branch captures behavior you care about later and does so without turning the parameter table into noise. In battery papers this distinction is often the difference between a useful model and an overfitted one.

We will keep returning to that standard. The workflow in this chapter is not “build the fanciest circuit you can.” It is “build the simplest circuit that survives validation honestly.”

Guided Walkthrough 1

Build and Compare R_{int} , 1RC, and 2RC Models in Plain MATLAB

Learning objective: Understand exactly what each ECM order adds to the terminal-voltage response before you fit anything.

Before we estimate parameters from data, we need to learn to recognize the signatures of the different model orders with our own eyes. In this walkthrough we will simulate three models under the same current profile: a pure R_{int} model, a 1RC Thevenin model, and a 2RC model. The code is deliberately plain. We are not using Simulink, Simscape, or a hidden MATLAB app here because the fastest way to build intuition is to see the states being updated explicitly.

6.3.1 Walkthrough 1 code

```
clear; close all; clc;

% Time base and current profile
dt = 1;
time_s = (0:1:1200).';
current_a = zeros(size(time_s));
```

```

current_a(time_s >= 60 & time_s < 240) = 2.3; % 1C discharge pulse
current_a(time_s >= 360 & time_s < 430) = 4.6; % 2C discharge pulse
current_a(time_s >= 620 & time_s < 760) = -1.15; % 0.5C charge pulse
current_a(time_s >= 900 & time_s < 980) = 3.45; % 1.5C discharge pulse

% A smooth but realistic full-cell OCV lookup table
ocv_soc = [0.00 0.05 0.10 0.20 0.30 0.40 0.50 0.60 0.70 0.80 0.90 1.00].';
ocv_v = [3.00 3.15 3.27 3.42 3.53 3.62 3.70 3.79 3.89 4.00 4.09 4.18].';

% Cell metadata
capacity_ah = 2.3;
initial_soc = 0.95;

% Three candidate ECMs
rintModel = struct( ...
    "name", "Rint", ...
    "Q_ah", capacity_ah, ...
    "R0", 0.020, ...
    "R1", 0.000, ...
    "C1", 1.0, ...
    "R2", 0.000, ...
    "C2", 1.0);

oneRcModel = struct( ...
    "name", "1RC", ...
    "Q_ah", capacity_ah, ...
    "R0", 0.014, ...
    "R1", 0.012, ...
    "C1", 120.0, ...
    "R2", 0.000, ...
    "C2", 1.0);

twoRcModel = struct( ...
    "name", "2RC", ...
    "Q_ah", capacity_ah, ...
    "R0", 0.011, ...
    "R1", 0.010, ...
    "C1", 55.0, ...
    "R2", 0.018, ...
    "C2", 700.0);

rintResult = simulateEcm(time_s, current_a, initial_soc, ocv_soc, ocv_v,
    ↪ rintModel);
oneRcResult = simulateEcm(time_s, current_a, initial_soc, ocv_soc, ocv_v,
    ↪ oneRcModel);

```



```

twoRcResult = simulateEcm(time_s, current_a, initial_soc, ocv_soc, ocv_v,
    ↪ twoRcModel);

% Plot current and voltage
figure("Color", "w", "Position", [100 100 1000 720]);

subplot(3, 1, 1);
plot(time_s, current_a, "k", "LineWidth", 1.5);
grid on;
xlabel("Time [s]");
ylabel("Current [A]");
title("Shared current profile");

subplot(3, 1, 2);
plot(time_s, rintResult.voltage_v, "LineWidth", 1.8); hold on;
plot(time_s, oneRcResult.voltage_v, "LineWidth", 1.8);
plot(time_s, twoRcResult.voltage_v, "LineWidth", 1.8);
grid on;
xlabel("Time [s]");
ylabel("Terminal voltage [V]");
title("Voltage response of three ECM orders");
legend("Rint", "1RC", "2RC", "Location", "best");

subplot(3, 1, 3);
plot(time_s, oneRcResult.v1_v, "LineWidth", 1.6); hold on;
plot(time_s, twoRcResult.v1_v, "--", "LineWidth", 1.6);
plot(time_s, twoRcResult.v2_v, "-.", "LineWidth", 1.6);
grid on;
xlabel("Time [s]");
ylabel("Polarization branch voltage [V]");
title("Internal branch voltages");
legend("1RC branch", "2RC fast branch", "2RC slow branch", "Location",
    ↪ "best");

fprintf("Voltage at the end of the 2C pulse:\n");
fprintf("  Rint : %.4f V\n", rintResult.voltage_v(time_s == 429));
fprintf("  1RC : %.4f V\n", oneRcResult.voltage_v(time_s == 429));
fprintf("  2RC : %.4f V\n", twoRcResult.voltage_v(time_s == 429));

function result = simulateEcm(time_s, current_a, initial_soc, ocv_soc, ocv_v,
    ↪ model)
    n = numel(time_s);
    dt = time_s(2) - time_s(1);

```

```

soc = zeros(n, 1);
voltage_v = zeros(n, 1);
v1_v = zeros(n, 1);
v2_v = zeros(n, 1);

soc(1) = initial_soc;

if model.R1 > 0
    alpha1 = exp(-dt / (model.R1 * model.C1));
else
    alpha1 = 0;
end

if model.R2 > 0
    alpha2 = exp(-dt / (model.R2 * model.C2));
else
    alpha2 = 0;
end

for k = 1:n
    if k > 1
        soc(k) = soc(k - 1) - current_a(k - 1) * dt / (3600 *
            ↪ model.Q_ah);
        soc(k) = min(max(soc(k), 0), 1);

        if model.R1 > 0
            v1_v(k) = alpha1 * v1_v(k - 1) ...
                + model.R1 * (1 - alpha1) * current_a(k - 1);
        end

        if model.R2 > 0
            v2_v(k) = alpha2 * v2_v(k - 1) ...
                + model.R2 * (1 - alpha2) * current_a(k - 1);
        end
    end

    ocv_now = interp1(ocv_soc, ocv_v, soc(k), "pchip", "extrap");
    voltage_v(k) = ocv_now - current_a(k) * model.R0 - v1_v(k) - v2_v(k);
end

result = struct( ...
    "soc", soc, ...
    "voltage_v", voltage_v, ...
    "v1_v", v1_v, ...
    "v2_v", v2_v);

```

end

6.3.2 Walkthrough 2 explanation

The first block creates a current profile with four informative regions: a mild discharge, a hard discharge, a charge pulse, and a final medium discharge. We could have used one pulse only, but the chapter is about pattern recognition. Multiple pulse amplitudes make the differences between the models easier to see.

The OCV table is intentionally smooth and monotonic. That is a teaching choice, not a field-wide rule. Some chemistries have plateaus, small inflections, or hysteresis that need special treatment. We will talk about those later in the chapter and again in the sodium-ion section.

Each candidate model is stored as a small MATLAB `struct`. This is one of the simplest ways to keep parameter sets readable without introducing a class definition too early. Notice that the `Rint` model is represented by setting `R1 = 0` and `R2 = 0`; the simulation function interprets those as absent branches.

Inside `simulateEcm`, the SOC update uses the previous current sample, which is the cleanest explicit-Euler interpretation at a fixed sample period. The branch updates use the exact discrete-time decay factor `alpha = exp(-dt/(RC))`. That is better than a first-order Euler step for the branch states because it preserves the correct exponential shape even for moderately large `dt`.

The terminal voltage is then computed from Equation (1): OCV minus the instantaneous ohmic drop minus the branch voltages. Because we are using the convention `positive current = discharge`, the voltage falls under positive current and rises under negative current.

6.3.3 Walkthrough 2 expected output

You should see four important features.

First, the `Rint` trace reacts instantly to each current step and then stays flat except for the slow change caused by SOC drift. There is no curved relaxation during rest because there are no dynamic states other than SOC.

Second, the `1RC` trace drops immediately when the pulse begins and then bends further downward during the pulse as the single polarization branch charges. When the current returns to zero, the voltage recovers smoothly on one dominant time scale.

Third, the 2RC trace does everything the 1RC trace does, but the rest recovery has a visible “shoulder”: a fast partial recovery followed by a slower tail. That is the signature you want to learn to recognize. It is the main reason a 2RC model is often the default workhorse in automotive BMS papers.

Fourth, the branch-voltage subplot makes the same story explicit. The fast branch in the 2RC model rises and decays quickly. The slow branch changes more gradually and lingers during rest.

The printed voltages at the end of the 2C pulse should show the expected ranking:

- `Rint` usually predicts the highest terminal voltage because it cannot accumulate polarization.
- 1RC predicts a lower voltage because it captures one dynamic overpotential.
- 2RC predicts the lowest voltage of the three because it captures both a fast and a slow polarization component.

6.3.4 Walkthrough 2 troubleshooting

1. The 2RC model looks almost identical to the 1RC model.

Symptom: the two traces sit nearly on top of each other.

Fix: your slow branch time constant may be too short, or your `R2` value may be too small. Increase `C2` or `R2` so the slow tail is visible over the rest periods.

2. The voltage rises during discharge.

Symptom: the pulse directions look backwards.

Fix: your sign convention is flipped. In this chapter’s hand-built model, discharge current is positive.

3. The SOC leaves the interval `[0, 1]`.

Symptom: interpolation errors or nonsense voltages near the end of the simulation.

Fix: clamp the SOC after each update, exactly as the code does with `min(max(...))`.

4. The `Rint` result shows a slow recovery during rest.

Symptom: the flat line after a pulse is missing.

Fix: check that `R1` and `R2` are really set to zero for the `Rint` case and that the branch-update code skips them.

6.3.5 Walkthrough 2 reflection

This exercise teaches the visual grammar of ECMs. Before this point, “1RC” and “2RC” may have felt like labels from a paper. Now you have seen the specific voltage features they add. That matters because the rest of the chapter is about fitting those features from data rather than inventing them by hand.

Guided Walkthrough 2

Identify R_0 , R_1 , C_1 , R_2 , and C_2 from a Single Synthetic HPPC Pulse

Learning objective: Learn exactly how a pulse and recovery encode a 2RC parameter set.

We are now going to move from forward simulation to inverse modeling. Instead of choosing parameters and predicting voltage, we will generate one synthetic pulse response, corrupt it with a tiny amount of measurement noise, and recover the parameters back. We start with synthetic data because it lets us separate method from mess. If the method does not work on clean synthetic data, it certainly will not work on a real workbook downloaded from the public internet.

6.3.6 Walkthrough 2 code

```
clear; close all; clc;

% One pulse with a long recovery window
time_s = (0:1:220).';
current_a = zeros(size(time_s));
current_a(time_s >= 40 & time_s < 58) = 8.0; % discharge pulse

% Known truth used to generate synthetic "measurements"
ocv_true_v = 3.865;
trueParams = struct( ...
    "R0", 0.0130, ...
    "R1", 0.0075, ...
    "C1", 35.0, ...
    "R2", 0.0200, ...
    "C2", 220.0);

voltage_clean_v = simulatePulse(current_a, ocv_true_v, trueParams);
```

```

rng(7);
noise_std_v = 1e-3;
voltage_meas_v = voltage_clean_v + noise_std_v *
    ↪ randn(size(voltage_clean_v));

% Step 1: estimate R0 from the instantaneous voltage jump
idxPulseStart = find(diff(current_a) > 0, 1, "first") + 1;
idxPulseEnd = find(diff(current_a) < 0, 1, "first");
idxRecoveryStart = idxPulseEnd + 1;

deltaI = current_a(idxPulseStart) - current_a(idxPulseStart - 1);
vBefore = voltage_meas_v(idxPulseStart - 1);
vAfter = voltage_meas_v(idxPulseStart);
R0_hat = (vBefore - vAfter) / deltaI;

% Step 2: estimate OCV from the tail end of the recovery
ocv_hat_v = mean(voltage_meas_v(end-20:end));

% Step 3: fit the recovery with one exponential and then with two
recoveryTime_s = time_s(idxRecoveryStart:end) - time_s(idxRecoveryStart);
recoveryOvervoltage_v = ocv_hat_v - voltage_meas_v(idxRecoveryStart:end);
pulseCurrent_a = current_a(idxPulseStart);
pulseDuration_s = time_s(idxPulseEnd) - time_s(idxPulseStart) + 1;

oneRcModelFun = @(p, t) p(1) * exp(-t / p(2));
twoRcModelFun = @(p, t) p(1) * exp(-t / p(2)) + p(3) * exp(-t / p(4));

opts = optimoptions("lsqcurvefit", "Display", "off");

p0_1rc = [0.05, 20];
lb_1rc = [0, 1];
ub_1rc = [0.5, 500];
pHat_1rc = lsqcurvefit(oneRcModelFun, p0_1rc, recoveryTime_s, ...
    recoveryOvervoltage_v, lb_1rc, ub_1rc, opts);

p0_2rc = [0.03, 8, 0.07, 80];
lb_2rc = [0, 1, 0, 5];
ub_2rc = [0.5, 100, 0.5, 1000];
pHat_2rc = lsqcurvefit(twoRcModelFun, p0_2rc, recoveryTime_s, ...
    recoveryOvervoltage_v, lb_2rc, ub_2rc, opts);

% Sort the two time constants so the fast branch is always branch 1
if pHat_2rc(2) > pHat_2rc(4)
    pHat_2rc = [pHat_2rc(3) pHat_2rc(4) pHat_2rc(1) pHat_2rc(2)];

```

end

```
A1 = pHat_2rc(1);
tau1_s = pHat_2rc(2);
A2 = pHat_2rc(3);
tau2_s = pHat_2rc(4);

R1_hat = A1 / (pulseCurrent_a * (1 - exp(-pulseDuration_s / tau1_s)));
R2_hat = A2 / (pulseCurrent_a * (1 - exp(-pulseDuration_s / tau2_s)));
C1_hat = tau1_s / R1_hat;
C2_hat = tau2_s / R2_hat;

recoveryFit_1rc_v = oneRcModelFun(pHat_1rc, recoveryTime_s);
recoveryFit_2rc_v = twoRcModelFun(pHat_2rc, recoveryTime_s);

rmse_1rc_mv = 1000 * sqrt(mean((recoveryOvervoltage_v -
    ↪ recoveryFit_1rc_v).^2));
rmse_2rc_mv = 1000 * sqrt(mean((recoveryOvervoltage_v -
    ↪ recoveryFit_2rc_v).^2));

fprintf("Estimated parameters from one pulse:\n");
fprintf("  R0  = %.5f ohm\n", R0_hat);
fprintf("  R1  = %.5f ohm\n", R1_hat);
fprintf("  C1  = %.2f F\n", C1_hat);
fprintf("  R2  = %.5f ohm\n", R2_hat);
fprintf("  C2  = %.2f F\n", C2_hat);
fprintf("  OCV = %.4f V\n", ocv_hat_v);
fprintf("1RC recovery RMSE : %.3f mV\n", rmse_1rc_mv);
fprintf("2RC recovery RMSE : %.3f mV\n", rmse_2rc_mv);

figure("Color", "w", "Position", [120 120 1000 720]);

subplot(3, 1, 1);
plot(time_s, current_a, "k", "LineWidth", 1.5);
grid on;
xlabel("Time [s]");
ylabel("Current [A]");
title("Synthetic HPPC pulse");

subplot(3, 1, 2);
plot(time_s, voltage_meas_v, "o", "MarkerSize", 3.0, "DisplayName",
    ↪ "Measured"); hold on;
plot(time_s, voltage_clean_v, "LineWidth", 1.8, "DisplayName", "True clean
    ↪ response");
xline(time_s(idxPulseStart), "--", "Pulse start", "LabelVerticalAlignment",
    ↪ "bottom");
```

```

xline(time_s(idxRecoveryStart), "--", "Recovery start",
↳ "LabelVerticalAlignment", "bottom");
grid on;
xlabel("Time [s]");
ylabel("Voltage [V]");
title("Measured voltage with millivolt noise");
legend("Location", "best");

subplot(3, 1, 3);
plot(recoveryTime_s, 1000 * recoveryOvervoltage_v, "ko", "MarkerSize", 3.0,
↳ ...
    "DisplayName", "Measured recovery overvoltage"); hold on;
plot(recoveryTime_s, 1000 * recoveryFit_1rc_v, "LineWidth", 1.6, ...
    "DisplayName", sprintf("1RC fit (RMSE %.2f mV)", rmse_1rc_mv));
plot(recoveryTime_s, 1000 * recoveryFit_2rc_v, "LineWidth", 1.8, ...
    "DisplayName", sprintf("2RC fit (RMSE %.2f mV)", rmse_2rc_mv));
grid on;
xlabel("Recovery time [s]");
ylabel("Recovery overvoltage [mV]");
title("Why the second RC branch helps");
legend("Location", "best");

function voltage_v = simulatePulse(current_a, ocv_v, params)
    n = numel(current_a);
    voltage_v = zeros(n, 1);
    v1 = zeros(n, 1);
    v2 = zeros(n, 1);
    dt = 1;

    alpha1 = exp(-dt / (params.R1 * params.C1));
    alpha2 = exp(-dt / (params.R2 * params.C2));

    for k = 1:n
        if k > 1
            v1(k) = alpha1 * v1(k - 1) + params.R1 * (1 - alpha1) *
↳ current_a(k - 1);
            v2(k) = alpha2 * v2(k - 1) + params.R2 * (1 - alpha2) *
↳ current_a(k - 1);
        end
        voltage_v(k) = ocv_v - params.R0 * current_a(k) - v1(k) - v2(k);
    end
end

```


6.3.7 Walkthrough 3 explanation

The pulse lasts 18 s and is followed by a long rest. That is deliberate. If the rest were too short, the slow branch would not have enough time to reveal itself.

The first estimate, $R0_hat$, comes from the immediate voltage jump at pulse start. This is the part of the waveform least contaminated by the RC dynamics, so we estimate it directly instead of bundling it into the nonlinear fit.

The recovery fit is built on $recoveryOvervoltage_v = OCV - V$. During rest, the current is zero, so the terminal voltage rises toward OCV from below. That means $OCV - V$ is a positive quantity that decays to zero. It is exactly the object a sum of exponentials should fit.

The one-exponential model tests the 1RC hypothesis. The two-exponential model tests the 2RC hypothesis. Once we recover amplitudes $A1$ and $A2$ and time constants $\tau1$ and $\tau2$, we convert them back into $R1$, $R2$, $C1$, and $C2$ using the finite pulse duration. That conversion is easy to miss, so it is worth reading carefully: the branch amplitudes at recovery start are not simply $I \cdot R1$ and $I \cdot R2$ unless the pulse is infinitely long. The factor $(1 - \exp(-T_{pulse}/\tau))$ corrects for the fact that the branch may not have reached steady state.

6.3.8 Walkthrough 3 expected output

You should see a clean ranking:

- $R0_hat$ should land very close to the true 0.013 ohm.
- The 2RC recovery RMSE should be clearly smaller than the 1RC RMSE.
- The recovered $\tau1$ should be on the order of tens of seconds, and $\tau2$ should be substantially slower.

Typical output from this script is in this neighborhood:

```
Estimated parameters from one pulse:
```

```
R0  = 0.0129 ohm
R1  = 0.0077 ohm
C1  = 34.2 F
R2  = 0.0198 ohm
C2  = 221.5 F
OCV = 3.8648 V
1RC recovery RMSE : 2.0 mV
2RC recovery RMSE : 0.9 mV
```

Do not obsess over the last decimal place. The exact numbers will move with

the random noise realization. What matters is that the recovered parameters remain close to truth and that the 2RC fit captures the long tail better.

6.3.9 Walkthrough 3 troubleshooting

1. $R0_hat$ is much too large.

Symptom: the printed $R0$ is closer to 0.03 ohm or 0.04 ohm .

Fix: you probably measured the jump too late, after the RC branches had already started contributing. Use the sample immediately before and immediately after the step.

2. The fitted τ_{u1} and τ_{u2} swap places every run.

Symptom: sometimes the fast branch is printed as branch 2 and sometimes as branch 1.

Fix: this is not physically wrong, but it is annoying. Sort the pair after fitting, exactly as the code does.

3. The 2RC fit becomes numerically unstable.

Symptom: one time constant runs to the upper bound.

Fix: your pulse or rest window may not be rich enough, or the initial guesses are too poor. Longer rest windows and better initialization help immediately.

4. The one-exponential fit looks just as good as the two-exponential fit.

Symptom: the RMSE values are nearly identical.

Fix: either the slow branch is too weak in the synthetic truth, or the pulse is too short to excite it. Increase $R2$, $C2$, or the pulse duration.

6.3.10 Walkthrough 3 reflection

This exercise teaches the core logic behind Plett-style identification. A pulse response is not just “some transient.” It contains a direct ohmic jump, a fast decay, and often a slow decay. Once you learn to read those pieces, the parameter-estimation workflow stops feeling mysterious.

Guided Walkthrough 3

Build SOC-Dependent 2RC Tables from a Full Synthetic HPPC Suite

Learning objective: Turn a collection of pulses into the lookup tables that a BMS actually uses.

A real ECM is rarely a single fixed parameter set. Resistances, time constants, and OCV all move with SOC, and usually with temperature as well. In this walkthrough we generate a synthetic HPPC suite over multiple SOC levels, identify a 2RC model at each level, and build smooth lookup tables. This is the closest we will come in this chapter to the classic textbook ECM-identification pipeline.

6.3.11 Walkthrough 3 code

```
clear; close all; clc;

rng(11);

socNodes = (1.00:-0.10:0.20).';
pulseCurrent_a = 6.0;
pulseDuration_s = 18;
restBefore_s = 40;
restAfter_s = 150;

ocv_soc = [0.00 0.05 0.10 0.20 0.30 0.40 0.50 0.60 0.70 0.80 0.90 1.00].';
ocv_v = [3.00 3.15 3.27 3.42 3.53 3.62 3.70 3.79 3.89 4.00 4.09 4.18].';

trueR0 = @(z) 0.0105 + 0.0060 * (1 - z).^1.2;
trueR1 = @(z) 0.0050 + 0.0040 * (1 - z);
trueC1 = @(z) 28.0 + 35.0 * z;
trueR2 = @(z) 0.0120 + 0.0100 * (1 - z).^2;
trueC2 = @(z) 150.0 + 450.0 * z;

nSoc = numel(socNodes);
identified = table('Size', [nSoc 7], ...
    'VariableTypes', repmat("double", 1, 7), ...
    'VariableNames', ["SOC", "OCV_V", "R0_Ohm", "R1_Ohm", "C1_F", "R2_Ohm",
        ↪ "C2_F"]);

for idx = 1:nSoc
    soc0 = socNodes(idx);
    time_s = (0:1:(restBefore_s + pulseDuration_s + restAfter_s)).';
    current_a = zeros(size(time_s));
    current_a(time_s >= restBefore_s & time_s < restBefore_s +
        ↪ pulseDuration_s) = pulseCurrent_a;

    ocv0_v = interp1(ocv_soc, ocv_v, soc0, "pchip");
```

```

paramsTrue = struct( ...
    "R0", trueR0(soc0), ...
    "R1", trueR1(soc0), ...
    "C1", trueC1(soc0), ...
    "R2", trueR2(soc0), ...
    "C2", trueC2(soc0));

voltage_clean_v = simulatePulse(current_a, ocv0_v, paramsTrue);
voltage_meas_v = voltage_clean_v + 8e-4 * randn(size(voltage_clean_v));

fitted = fitSinglePulse(time_s, current_a, voltage_meas_v);

identified.SOC(idx) = soc0;
identified.OCV_V(idx) = fitted.OCV_V;
identified.R0_Ohm(idx) = fitted.R0_Ohm;
identified.R1_Ohm(idx) = fitted.R1_Ohm;
identified.C1_F(idx) = fitted.C1_F;
identified.R2_Ohm(idx) = fitted.R2_Ohm;
identified.C2_F(idx) = fitted.C2_F;
end

% Smooth the identified tables so they behave like something you would ship
denseSoc = linspace(min(identified.SOC), max(identified.SOC), 101).';
ocvDense = interp1(identified.SOC, identified.OCV_V, denseSoc, "pchip");
r0Dense = interp1(identified.SOC, identified.R0_Ohm, denseSoc, "pchip");
r1Dense = interp1(identified.SOC, identified.R1_Ohm, denseSoc, "pchip");
c1Dense = interp1(identified.SOC, identified.C1_F, denseSoc, "pchip");
r2Dense = interp1(identified.SOC, identified.R2_Ohm, denseSoc, "pchip");
c2Dense = interp1(identified.SOC, identified.C2_F, denseSoc, "pchip");

ecmLookup = table(denseSoc, ocvDense, r0Dense, r1Dense, c1Dense, r2Dense,
    ↪ c2Dense, ...
    'VariableNames', ["SOC", "OCV_V", "R0_Ohm", "R1_Ohm", "C1_F", "R2_Ohm",
    ↪ "C2_F"]);

writetable(ecmLookup, "chapter6_synthetic_2rc_lookup.csv");
save("chapter6_synthetic_2rc_lookup.mat", "ecmLookup", "identified");

% Validate at an unseen SOC point
validationSoc = 0.45;
validationTime_s = (0:1:(restBefore_s + pulseDuration_s + restAfter_s)).';
validationCurrent_a = zeros(size(validationTime_s));
validationCurrent_a(validationTime_s >= restBefore_s & ...
    validationTime_s < restBefore_s + pulseDuration_s) = pulseCurrent_a;

```

```

validationTruth = struct( ...
    "R0", trueR0(validationSoc), ...
    "R1", trueR1(validationSoc), ...
    "C1", trueC1(validationSoc), ...
    "R2", trueR2(validationSoc), ...
    "C2", trueC2(validationSoc));

validationOcv_v = interp1(ocv_soc, ocv_v, validationSoc, "pchip");
validationMeasured_v = simulatePulse(validationCurrent_a, validationOcv_v,
    ↪ validationTruth);

lookupParams = struct( ...
    "R0", interp1(ecmLookup.SOC, ecmLookup.R0_Ohm, validationSoc, "pchip"),
    ↪ ...
    "R1", interp1(ecmLookup.SOC, ecmLookup.R1_Ohm, validationSoc, "pchip"),
    ↪ ...
    "C1", interp1(ecmLookup.SOC, ecmLookup.C1_F, validationSoc, "pchip"), ...
    "R2", interp1(ecmLookup.SOC, ecmLookup.R2_Ohm, validationSoc, "pchip"),
    ↪ ...
    "C2", interp1(ecmLookup.SOC, ecmLookup.C2_F, validationSoc, "pchip"));

validationPredicted_v = simulatePulse(validationCurrent_a, validationOcv_v,
    ↪ lookupParams);
validationRmse_mv = 1000 * sqrt(mean((validationMeasured_v -
    ↪ validationPredicted_v).^2));

disp(identified);
fprintf("Held-out pulse RMSE at SOC %.2f: %.3f mV\n", validationSoc,
    ↪ validationRmse_mv);

figure("Color", "w", "Position", [100 100 1100 760]);

subplot(2, 2, 1);
plot(identified.SOC, identified.OCV_V, "o", "MarkerSize", 6, "LineWidth",
    ↪ 1.2); hold on;
plot(ecmLookup.SOC, ecmLookup.OCV_V, "LineWidth", 1.8);
grid on;
xlabel("SOC [-]");
ylabel("OCV [V]");
title("Identified OCV table");
legend("Pulse estimates", "Smoothed table", "Location", "best");

subplot(2, 2, 2);
plot(identified.SOC, identified.R0_Ohm, "o-", "LineWidth", 1.4); hold on;
plot(identified.SOC, identified.R1_Ohm, "s-", "LineWidth", 1.4);

```

```

plot(identified.SOC, identified.R2_Ohm, "d-", "LineWidth", 1.4);
grid on;
xlabel("SOC [-]");
ylabel("Resistance [ohm]");
title("SOC-dependent resistances");
legend("R0", "R1", "R2", "Location", "best");

subplot(2, 2, 3);
plot(identified.SOC, identified.C1_F, "s-", "LineWidth", 1.4); hold on;
plot(identified.SOC, identified.C2_F, "d-", "LineWidth", 1.4);
grid on;
xlabel("SOC [-]");
ylabel("Capacitance [F]");
title("SOC-dependent capacitances");
legend("C1", "C2", "Location", "best");

subplot(2, 2, 4);
plot(validationTime_s, validationMeasured_v, "k", "LineWidth", 1.8); hold on;
plot(validationTime_s, validationPredicted_v, "--", "LineWidth", 1.8);
grid on;
xlabel("Time [s]");
ylabel("Voltage [V]");
title(sprintf("Held-out validation pulse, RMSE = %.2f mV",
    ↪ validationRmse_mv));
legend("Truth", "Lookup-table prediction", "Location", "best");

function fitted = fitSinglePulse(time_s, current_a, voltage_v)
    idxPulseStart = find(diff(current_a) > 0, 1, "first") + 1;
    idxPulseEnd = find(diff(current_a) < 0, 1, "first");
    idxRecoveryStart = idxPulseEnd + 1;

    deltaI = current_a(idxPulseStart) - current_a(idxPulseStart - 1);
    R0_hat = (voltage_v(idxPulseStart - 1) - voltage_v(idxPulseStart)) /
        ↪ deltaI;
    OCV_hat = mean(voltage_v(end-20:end));

    recoveryTime_s = time_s(idxRecoveryStart:end) - time_s(idxRecoveryStart);
    recoveryOvervoltage_v = OCV_hat - voltage_v(idxRecoveryStart:end);

    modelFun = @(p, t) p(1) * exp(-t / p(2)) + p(3) * exp(-t / p(4));
    p0 = [0.02, 8, 0.05, 80];
    lb = [0, 1, 0, 5];
    ub = [0.5, 150, 0.5, 1500];
    opts = optimoptions("lsqcurvefit", "Display", "off");

```

```

pHat = lsqcurvefit(modelFun, p0, recoveryTime_s, recovery0vervoltage_v,
    ↳ lb, ub, opts);

if pHat(2) > pHat(4)
    pHat = [pHat(3) pHat(4) pHat(1) pHat(2)];
end

pulseCurrent_a = current_a(idxPulseStart);
pulseDuration_s = time_s(idxPulseEnd) - time_s(idxPulseStart) + 1;
R1_hat = pHat(1) / (pulseCurrent_a * (1 - exp(-pulseDuration_s /
    ↳ pHat(2))));
R2_hat = pHat(3) / (pulseCurrent_a * (1 - exp(-pulseDuration_s /
    ↳ pHat(4))));
C1_hat = pHat(2) / R1_hat;
C2_hat = pHat(4) / R2_hat;

fitted = struct( ...
    "OCV_V", OCV_hat, ...
    "R0_Ohm", R0_hat, ...
    "R1_Ohm", R1_hat, ...
    "C1_F", C1_hat, ...
    "R2_Ohm", R2_hat, ...
    "C2_F", C2_hat);
end

function voltage_v = simulatePulse(current_a, ocv_v, params)
    n = numel(current_a);
    dt = 1;
    voltage_v = zeros(n, 1);
    v1 = zeros(n, 1);
    v2 = zeros(n, 1);

    alpha1 = exp(-dt / (params.R1 * params.C1));
    alpha2 = exp(-dt / (params.R2 * params.C2));

    for k = 1:n
        if k > 1
            v1(k) = alpha1 * v1(k - 1) + params.R1 * (1 - alpha1) *
                ↳ current_a(k - 1);
            v2(k) = alpha2 * v2(k - 1) + params.R2 * (1 - alpha2) *
                ↳ current_a(k - 1);
        end
        voltage_v(k) = ocv_v - params.R0 * current_a(k) - v1(k) - v2(k);
    end
end

```

end

6.3.12 Walkthrough 4 explanation

The SOC nodes run from 1.0 down to 0.2 in 10% steps because that is a common first-pass table density in BMS work. We use simple analytic functions for the “true” SOC dependence so the identified tables have something physically smooth to recover.

The `fitSinglePulse` helper is almost the same logic from Walkthrough 2, but now we reuse it inside a loop over SOC. This is the important workflow leap. A BMS parameter table is not a single fit. It is a family of fits organized by state.

After identification, we interpolate the results onto a denser SOC grid and save them to both CSV and MAT formats. That is a subtle but important research habit. CSV is easy to inspect and share. MAT preserves exact MATLAB types and is convenient inside your own workflow.

The held-out validation at $\text{SOC} = 0.45$ matters just as much as the table construction. Because 0.45 was not one of the training nodes, the validation plot checks whether the identified tables interpolate sensibly instead of only memorizing the fitted points.

6.3.13 Walkthrough 4 expected output

The printed table should show three healthy patterns.

- `OCV_V` should increase monotonically with SOC.
- `R0_0hm`, `R1_0hm`, and usually `R2_0hm` should rise as SOC falls.
- `C1_F` and `C2_F` should stay positive and vary smoothly rather than jumping erratically from node to node.

The held-out RMSE should usually land in the low single-digit millivolt range because the data are synthetic and the model class matches the truth. If you see 10 mV or 20 mV here, something is wrong.

The plot of the held-out pulse should show two traces that are nearly indistinguishable at normal viewing scale. That is exactly what you want at this stage.

6.3.14 Walkthrough 4 troubleshooting

1. The identified tables look jagged rather than smooth.
Symptom: neighboring SOC nodes jump up and down unpredictably.
Fix: your recovery fits are underconstrained or too noisy. Longer rest windows, better initialization, or mild smoothing are all reasonable.
2. One capacitance becomes negative or enormous.
Symptom: C1 or C2 prints as a negative number or a wildly large value.
Fix: check that the corresponding amplitude and time constant are positive and that the conversion from amplitude to resistance uses the correct pulse duration.
3. The held-out validation is poor even though the fitted nodes look fine.
Symptom: low training error but bad interpolation at $SOC = 0.45$.
Fix: you may need more SOC nodes or a more stable interpolation method.
This is a classic sign that the lookup table is too sparse for the local curvature.
4. The OCV table is not monotonic.
Symptom: the voltage dips at one SOC node and rises again at the next.
Fix: if your chemistry should be monotonic, that nonmonotonicity usually signals noisy OCV extraction rather than real physics.

6.3.15 Walkthrough 4 reflection

This is the first point in the chapter where the workflow looks like something you could actually ship into an estimator. You now have an OCV table, SOC-dependent dynamic parameters, a saved lookup file, and a held-out validation check. That is the minimal professional unit of ECM work.

Guided Walkthrough 4

Parse a Real CALCE Workbook and Build an OCV-SOC Table You Can Reuse

Learning objective: Turn a public experimental workbook into a clean OCV table that can feed an ECM.

Now we bring in real data. For this chapter the most reliable public source for a laptop-runnable ECM workflow is the CALCE Samsung INR 18650-20R archive maintained by the Center for Advanced Life Cycle Engineering at the University of

Maryland. The public portal explicitly documents the experiment and exposes downloadable files for low-current OCV, incremental-current OCV, and dynamic tests at several temperatures.

For the 25 degC low-current OCV file relevant here:

Item	Value
Dataset source	CALCE Battery Research Data portal
Cell	Samsung INR 18650-20R
File	11_5_2015_low current OCV test_SP20-1.xlsx
Archive URL	https://web.calce.umd.edu/batteries/data
Approximate size	about 2.9 MB zipped
Format	Excel workbook (.xlsx)
Sheet name	SP20-OCVSOC-0.05C
Relevant columns	Duration (sec), Pgm step, mV, mA, Temperature
License note	The CALCE page states the data are open access and asks users to cite CALCE papers describing the experiment; it does not present a formal software-style license

There is an important sign-convention pitfall here. The CALCE files were logged with Arbin-style tester conventions: positive current corresponds to charge, and negative current corresponds to discharge. That is the opposite of the convention we used in our hand-built ECM functions. We will normalize that explicitly in the code so later sections stay consistent.

6.3.16 Walkthrough 4 code

```
clear; close all; clc;

dataRoot = fullfile(pwd, "calce_ocv_data");
if ~exist(dataRoot, "dir")
    mkdir(dataRoot);
```

```

end

zipFile = fullfile(dataRoot, "SP1_25C_LC_OCV_11_5_2015.zip");
xlsxFile = fullfile(dataRoot, "11_5_2015_low current OCV test_SP20-1.xlsx");
downloadUrl =
    ↪ "https://web.calce.umd.edu/batteries/data/SP1_25C_LC_OCV_11_5_2015.zip";

if ~isfile(xlsxFile)
    if ~isfile(zipFile)
        fprintf("Downloading CALCE archive...\n");
        websave(zipFile, downloadUrl);
    end
    unzip(zipFile, dataRoot);
end

raw = readtable(xlsxFile, ...
    "Sheet", "SP20-OCVS0C-0.05C", ...
    "VariableNamingRule", "preserve");

time_s = raw("Duration (sec)");
voltage_v = raw("mV") / 1000;
current_a_raw = raw("mA") / 1000;

if ismember("Temperature", raw.Properties.VariableNames)
    temperature_c = raw("Temperature");
    temperature_c = fillmissing(temperature_c, "previous");
else
    temperature_c = nan(size(time_s));
end

% CALCE/Arbin convention: positive current = charge, negative current =
    ↪ discharge
chargeMask = current_a_raw > 0.03;
dischargeMask = current_a_raw < -0.03;
restMask = abs(current_a_raw) <= 0.03;

chargeTime_s = time_s(chargeMask);
chargeCurrent_a = current_a_raw(chargeMask);
chargeVoltage_v = voltage_v(chargeMask);

dischargeTime_s = time_s(dischargeMask);
dischargeCurrent_a = current_a_raw(dischargeMask);
dischargeVoltage_v = voltage_v(dischargeMask);

chargeAh = cumtrapz(chargeTime_s, chargeCurrent_a) / 3600;

```

```

dischargeAh = cumtrapz(dischargeTime_s, -dischargeCurrent_a) / 3600;

usableCapacity_ah = max(dischargeAh);
socCharge = chargeAh / max(chargeAh);
socDischarge = 1 - dischargeAh / usableCapacity_ah;

socGrid = linspace(0, 1, 101).';
chargeCurve_v = interp1(socCharge, chargeVoltage_v, socGrid, "linear",
↳ "extrap");
dischargeCurve_v = interp1(flipud(socDischarge), flipud(dischargeVoltage_v),
↳ socGrid, ...
    "linear", "extrap");
ocvCurve_v = 0.5 * (chargeCurve_v + dischargeCurve_v);
hysteresis_v = chargeCurve_v - dischargeCurve_v;

ocvTable = table(socGrid, ocvCurve_v, chargeCurve_v, dischargeCurve_v,
↳ hysteresis_v, ...
    'VariableNames', ["SOC", "OCV_V", "ChargeBranch_V", "DischargeBranch_V",
↳ "Hysteresis_V"]);

writetable(ocvTable, fullfile(dataRoot, "calce_sp20_ocv_table_25C.csv"));
save(fullfile(dataRoot, "calce_sp20_ocv_table_25C.mat"), ...
    "ocvTable", "usableCapacity_ah", "time_s", "voltage_v", "current_a_raw",
↳ "temperature_c");

% Use the extracted OCV table in a simple 2RC forward simulation
testTime_s = (0:1:300).';
testCurrent_a = zeros(size(testTime_s));
testCurrent_a(testTime_s >= 30 & testTime_s < 60) = 4.0;    % positive
↳ discharge for our ECM
testCurrent_a(testTime_s >= 160 & testTime_s < 190) = -2.0;

params2RC = struct("R0", 0.015, "R1", 0.009, "C1", 45, "R2", 0.020, "C2",
↳ 250);
initialSoc = 0.80;
forwardVoltage_v = simulateWithLookup(testTime_s, testCurrent_a, initialSoc,
↳ ...
    ocvTable.SOC, ocvTable.OCV_V, usableCapacity_ah, params2RC);

fprintf("Estimated usable capacity from CALCE low-current file: %.3f Ah\n",
↳ usableCapacity_ah);
fprintf("Hysteresis near 50%% SOC: %.4f V\n", ...
    interp1(ocvTable.SOC, ocvTable.Hysteresis_V, 0.50, "linear"));

figure("Color", "w", "Position", [100 100 1100 760]);

```

```

subplot(2, 2, 1);
plot(time_s / 3600, voltage_v, "LineWidth", 1.1);
grid on;
xlabel("Elapsed time [h]");
ylabel("Voltage [V]");
title("Raw CALCE low-current OCV voltage trace");

subplot(2, 2, 2);
plot(time_s / 3600, current_a_raw, "LineWidth", 1.1);
grid on;
xlabel("Elapsed time [h]");
ylabel("Current [A]");
title("Raw CALCE current trace (positive = charge)");

subplot(2, 2, 3);
plot(ocvTable.SOC, ocvTable.ChargeBranch_V, "LineWidth", 1.6); hold on;
plot(ocvTable.SOC, ocvTable.DischargeBranch_V, "LineWidth", 1.6);
plot(ocvTable.SOC, ocvTable.OCV_V, "k--", "LineWidth", 1.8);
grid on;
xlabel("SOC [-]");
ylabel("Voltage [V]");
title("Extracted charge, discharge, and averaged OCV curves");
legend("Charge branch", "Discharge branch", "Averaged OCV", "Location",
    ↪ "best");

subplot(2, 2, 4);
yyaxis left;
plot(testTime_s, testCurrent_a, "k", "LineWidth", 1.3);
ylabel("Current [A]");
yyaxis right;
plot(testTime_s, forwardVoltage_v, "LineWidth", 1.8);
ylabel("Voltage [V]");
grid on;
xlabel("Time [s]");
title("Reusing the real OCV table inside a 2RC simulation");

function voltage_v = simulateWithLookup(time_s, current_a, initialSoc,
    ↪ socLut, ocvLut, capacity_ah, params)
    n = numel(time_s);
    dt = time_s(2) - time_s(1);

    soc = zeros(n, 1);
    v1 = zeros(n, 1);

```

```

v2 = zeros(n, 1);
voltage_v = zeros(n, 1);
soc(1) = initialSoc;

alpha1 = exp(-dt / (params.R1 * params.C1));
alpha2 = exp(-dt / (params.R2 * params.C2));

for k = 1:n
    if k > 1
        soc(k) = soc(k - 1) - current_a(k - 1) * dt / (3600 *
            ↳ capacity_ah);
        soc(k) = min(max(soc(k), 0), 1);
        v1(k) = alpha1 * v1(k - 1) + params.R1 * (1 - alpha1) *
            ↳ current_a(k - 1);
        v2(k) = alpha2 * v2(k - 1) + params.R2 * (1 - alpha2) *
            ↳ current_a(k - 1);
    end

    ocvNow_v = interp1(socLut, ocvLut, soc(k), "pchip", "extrap");
    voltage_v(k) = ocvNow_v - params.R0 * current_a(k) - v1(k) - v2(k);
end
end

```

6.3.17 Simscape Battery walkthrough explanation

The first half is a public-data workflow, not a modeling workflow. It checks for the workbook locally, downloads the zip if necessary, unzips it, and reads the exact sheet SP20-OCVSOC-0.05C while preserving the original column names. That detail matters because CALCE's headers contain spaces and parentheses. If you let MATLAB rewrite them automatically, the later parsing becomes harder to explain and harder to trust.

The code then splits the trace into charge, discharge, and rest regions using a small current threshold of 0.03 A. That threshold is not sacred. It is a practical cutoff that keeps the branch extraction from being polluted by near-zero holding steps.

We compute charge and discharge SOC axes separately from the integrated ampere-hours. This is important because the charge branch grows from low SOC to high SOC, while the discharge branch shrinks from high SOC to low SOC. Trying to reuse one axis for both without care is the fastest way to confuse `interp1`.

Finally, we average the charge and discharge branches to obtain a simple low-current OCV estimate. This is standard practice in the field when hysteresis is modest and you want a single OCV table for an ECM. It is also a modeling shortcut. If hysteresis is large and important to your application, averaging it away is a convenience, not ground truth.

6.3.18 Simscape Battery walkthrough expected output

You should see a long, slow voltage trace whose dynamic structure is much calmer than a typical drive cycle because the OCV test was designed to approximate equilibrium.

The printed usable capacity should be close to the cell's nominal 2.0 Ah, though not necessarily exactly equal. A result in the range 1.9 Ah to 2.1 Ah is plausible for this file. The hysteresis at 50% SOC should be on the order of tens of millivolts, not hundreds.

The extracted OCV plot should show:

- a charge branch slightly above the discharge branch,
- an averaged OCV curve between them,
- full-cell voltage rising from the lower end of the SOC range toward roughly 4.2 V at high SOC.

The final subplot should confirm that the extracted OCV table is immediately usable inside the same 2RC simulation logic we built earlier. That is the main practical point of the exercise.

6.3.19 Simscape Battery walkthrough troubleshooting

1. `readtable` cannot find the sheet.

Symptom: MATLAB says the sheet name does not exist.

Fix: open the workbook manually and confirm the exact sheet name. On the verified CALCE file, it is `SP20-OCVSOC-0.05C`.

2. The charge or discharge interpolation fails.

Symptom: `interp1` reports nonunique or nonmonotonic sample points.

Fix: make sure the SOC vector you pass into `interp1` is increasing. The code uses `flipud` on the discharge branch for exactly this reason.

3. The extracted capacity is absurdly small.

Symptom: you get something like 0.2 Ah.

Fix: your current units are probably still in milliamps. The workbook uses mA, so we divide by 1000.

4. The averaged OCV is visibly jagged.

Symptom: the table wiggles from node to node.

Fix: use a denser current threshold or mild smoothing after interpolation.

Jaggedness here usually comes from branch-selection noise rather than from the chemistry.

6.3.20 Simscape Battery walkthrough reflection

This is the chapter's most important real-data habit: do not leave public spreadsheets in raw form. Turn them into a clean, documented modeling artifact. Here that artifact is an OCV-SOC table. In later chapters it will become validation data, estimator benchmarks, and eventually sodium-ion comparison sets.

Guided Walkthrough 5

Optional Simscape Battery Block Parameterization in `\R2025a+`

Learning objective: Map the hand-built HPPC workflow onto MATLAB's newer Simscape Battery tooling.

This section is optional because not every reader will have Simscape Battery. But if you do have it, it is worth seeing how the newer toolbox workflow lines up with the manual workflow we just built. We are still using HPPC-style data. We are simply handing that data to `hppcTest` and `fitECM` instead of writing the entire identification loop ourselves.

6.3.21 Walkthrough 5 code

```
clear; close all; clc;

import simscape.battery.parameters.*;

% Synthetic HPPC data in table form
time_s = (0:1:220).';
current_a = zeros(size(time_s));
current_a(time_s >= 40 & time_s < 58) = 8.0;

trueParams = struct("R0", 0.013, "R1", 0.0075, "C1", 35, "R2", 0.020, "C2",
↳ 220);
```



```

ocv_v = 3.865;
voltage_v = simulatePulse(current_a, ocv_v, trueParams);

hppcData = table(time_s, voltage_v, current_a, ...
    'VariableNames', ["time_s", "voltage_v", "current_a"]);

hppcExp = hppcTest(hppcData, ...
    TimeVariable = "time_s", ...
    VoltageVariable = "voltage_v", ...
    CurrentVariable = "current_a", ...
    CurrentSignConvention = "positiveDischarge");

ecmObj = fitECM(hppcExp, NumRCPairs = 2);
disp(ecmObj);

mdl = "chapter6BatteryEquivalentCircuit";
if bdIsLoaded(mdl)
    close_system(mdl, 0);
end

new_system(mdl);
open_system(mdl);

blockPath = mdl + "/Battery Equivalent Circuit";
add_block("batt_lib/Cells/Battery Equivalent Circuit", blockPath, ...
    "Position", [120 90 280 180]);

parameterizeEquivalentCircuitBlock(ecmObj, getSimulinkBlockHandle(blockPath),
    ↪ ...
    ParameterizePseudoOCV = true);

save_system(mdl);

function voltage_v = simulatePulse(current_a, ocv_v, params)
    n = numel(current_a);
    dt = 1;
    voltage_v = zeros(n, 1);
    v1 = zeros(n, 1);
    v2 = zeros(n, 1);

    alpha1 = exp(-dt / (params.R1 * params.C1));
    alpha2 = exp(-dt / (params.R2 * params.C2));

    for k = 1:n

```

```

    if k > 1
        v1(k) = alpha1 * v1(k - 1) + params.R1 * (1 - alpha1) *
            ↪ current_a(k - 1);
        v2(k) = alpha2 * v2(k - 1) + params.R2 * (1 - alpha2) *
            ↪ current_a(k - 1);
    end
    voltage_v(k) = ocv_v - params.R0 * current_a(k) - v1(k) - v2(k);
end
end
end

```

6.3.22 Walkthrough 5 explanation

The `hpcTest` object is MathWorks' way of telling Simscape Battery, "this table contains pulse data, and here is which column is time, which is voltage, and which is current." The `CurrentSignConvention = "positiveDischarge"` line is especially important because it keeps the object aligned with the convention we used throughout this chapter.

The `fitECM` call then estimates an ECM object directly from that pulse dataset. We specify `NumRCPairs = 2` because this walkthrough is intended to mirror the manual 2RC work above.

Finally, `parameterizeEquivalentCircuitBlock` pushes the fitted parameters into the Battery Equivalent Circuit block in a new Simulink model. That is the key bridge from research script to Simulink workflow.

6.3.23 Walkthrough 5 expected output

You should see:

- an ECM object printed in the Command Window,
- a new Simulink model called `chapter6BatteryEquivalentCircuit`,
- a Battery Equivalent Circuit block whose parameters have been populated.

If your release is older than R2025a, these functions may not exist. That is a version issue, not a modeling issue.

6.3.24 Walkthrough 5 troubleshooting

1. `fitECM` is undefined.

Symptom: MATLAB cannot find the function.

Fix: you are on an older release or do not have Simscape Battery installed. Skip this optional section.

2. The block path `batt_lib/Cells/Battery Equivalent Circuit` is not found.

Symptom: `add_block` fails.

Fix: confirm that Simscape Battery is installed and that the Battery Equivalent Circuit block exists in your release.

3. The object fit is poor even on synthetic data.

Symptom: the estimated block obviously does not reproduce the pulse.

Fix: check the current sign convention first. A wrong sign convention is the most common cause of nonsense ECM fits.

6.3.25 Walkthrough 5 reflection

This optional section is the software-ecosystem bridge. The manual workflow is the part you must understand. The toolbox workflow is the part you can use later for speed once you trust yourself to audit what it is doing.

Reproduction Exercise

Recreate the Plett-Style HPPC-to-ECM Workflow

For this chapter, the most valuable reproduction target is procedural rather than pixel-perfect. Gregory Plett's *Battery Management Systems, Volume II: Equivalent-Circuit Methods* is still one of the clearest descriptions of the classic ECM identification workflow for BMS use. The book does not ship a companion numerical dataset you can drop into MATLAB and match figure-for-figure, so we will reproduce the workflow itself and judge success by whether we recover the same qualitative outcomes.

Use the synthetic HPPC suite from Walkthrough 3 and do the following without changing the underlying truth model:

1. Build an OCV table from the rest windows only.
2. Estimate R_0 from the pulse jump rather than from a global optimizer.
3. Fit one 1RC and one 2RC model at each SOC node.
4. Report training RMSE and held-out pulse RMSE for both models.

5. Write a short note explaining whether the second RC branch is justified by the improvement you actually measured.

The unavoidable ambiguity is that Plett's book presents the workflow, not a public raw workbook with one canonical answer file. So "close enough" here means:

- the OCV table is smooth and monotonic,
- the 2RC model fits the recovery tails materially better than the 1RC model,
- the parameter trends with SOC are physically plausible,
- the held-out prediction improves rather than only the fitted pulses.

If your final result satisfies those conditions, you have reproduced the intellectual content of the workflow even if your exact numerical values differ from any example embedded in the book.

6.4 Open-Ended Exercises

6.4.1 Exercise 1: When does the second RC branch stop being worth it?

Modify Walkthrough 3 so that the pulse duration is only 5 s instead of 18 s, then repeat the whole lookup-table identification workflow. Compare the held-out validation error for 1RC and 2RC.

Hint: keep the same truth model. Only shorten the excitation. The question is not whether the second branch exists in truth, but whether the data still let you identify it reliably.

6.4.2 Exercise 2: What happens if the OCV curve gets flatter?

Replace the smooth lithium-ion-like OCV table used in Walkthroughs 1 and 3 with a flatter, plateau-heavy curve that mimics a sodium-ion hard-carbon full-cell region. Then rerun the held-out validation and comment on how sensitive the terminal voltage becomes to small SOC errors.

Hint: you do not need a perfect sodium-ion OCV model here. A deliberately flatter mid-SOC segment is enough to expose the observability issue.

6.4.3 Exercise 3: Add temperature as a second lookup dimension

Repeat Walkthrough 3 twice: once with all resistances increased by 30% and all capacitances reduced by 20% to mimic a colder cell, and once with the original

parameters. Store the results as two separate lookup tables and write a short function that switches between them based on temperature.

Hint: you are not building a full electrothermal model yet. You are building the habit of indexing ECM tables by more than one state variable.

6.5 Worked Solutions to the Open-Ended Exercises

6.5.1 Solution 1: Short pulses make the slow branch harder to justify

The shortest clean way to answer Exercise 1 is to reuse Walkthrough 3 exactly as written and change only:

```
pulseDuration_s = 5;  
restAfter_s = 150;
```

When you rerun the identification, three things usually happen.

- The R_0 estimates remain stable because the instantaneous jump is still visible.
- The fast branch remains identifiable because its time constant is comparable to the pulse width.
- The slow branch becomes much less stable because the pulse does not load it strongly enough before recovery begins.

In practice you will usually find that the held-out 2RC improvement shrinks markedly and may become too small to justify the extra branch. That is the correct lesson. Model order is a property of the data-protocol pair, not just of the underlying cell physics.

6.5.2 Solution 2: Flatter OCV reduces voltage sensitivity to SOC error

One runnable modification is:

```
ocv_soc = [0.00 0.10 0.20 0.35 0.50 0.65 0.80 0.90 1.00].';  
ocv_v = [2.95 3.18 3.30 3.36 3.40 3.44 3.60 3.92 4.08].';
```

If you drop that table into Walkthrough 3, the pulse-fitting still works, but the held-out voltage becomes less sensitive to modest SOC misalignment across the flat mid-SOC region. That is good for some types of robustness and bad for SOC observability. In a Kalman-filter chapter, this will matter a lot: if dU_{oc}/dz is small, voltage alone cannot strongly correct SOC error.

6.5.3 Solution 3: Two temperatures are just two tables until you need more

The simplest implementation is:

```
temperatureLabels = ["25C", "10C"];
lookupTables = struct();

for tIdx = 1:numel(temperatureLabels)
    if temperatureLabels(tIdx) == "25C"
        resistanceScale = 1.00;
        capacitanceScale = 1.00;
    else
        resistanceScale = 1.30;
        capacitanceScale = 0.80;
    end

    % Apply the scale factors to trueR0, trueR1, trueR2, trueC1, and trueC2
    % before running the Walkthrough 3 identification loop.
end
```

The key result is not the exact numbers. It is the workflow habit: once temperature matters, you should stop pretending one ECM table is universally valid.

What Changes for Sodium-Ion?

Several things change, and none of them are cosmetic.

First, sodium-ion full-cell OCV curves can be flatter or more stepped than the lithium-ion NMC/graphite style curve we used in the teaching examples. Hard-carbon anodes, polyanionic cathodes, and Prussian-blue analogues can all produce plateaus or low-slope regions that reduce voltage sensitivity to SOC. In practical BMS work that means ECM identification and SOC estimation from voltage alone both become harder. The model may still fit terminal voltage well, but the fitted parameters may be less informative about SOC than you expect.

Second, the public-data situation is much thinner for sodium-ion. Today there are far fewer openly distributed HPPC suites for SIB cells than for lithium-ion cells. In real sodium-ion research you will often do one of three things: generate your own virtual HPPC data from a physics-based model, digitize pulse plots from papers as a preliminary study, or work with partner data that cannot be redistributed. That makes the generic pulse-identification code from this chapter more valuable than

any one public workbook.

Third, temperature dependence deserves earlier attention in sodium-ion than many beginners expect. One of sodium-ion's practical selling points is better low-temperature behavior relative to many lithium-ion chemistries. That advantage will not show up honestly if you identify a single 25 degC ECM and pretend it applies everywhere. For SIB work, adding temperature as a lookup dimension sooner is usually good practice rather than optional polish.

Fourth, usable capacity and reference SOC can be trickier. Some sodium-ion cells exhibit stronger first-cycle losses, chemistry-dependent voltage plateaus, and more complicated hysteresis behavior. That means you should define the capacity basis used by your ECM explicitly: rated capacity, measured usable capacity at a reference temperature, or the capacity observed in the same dataset used for identification. Reviewers notice when this is vague.

The practical takeaway is simple. The core ECM mathematics transfers directly to sodium-ion. The identification discipline transfers directly too. What changes is the OCV shape, the data availability, and the burden on you to document what "SOC" and "OCV" mean for the chemistry you are modeling.

Chapter Summary and Skill Checklist

- You built R_{int}, 1RC, and 2RC models directly in MATLAB and learned to recognize their voltage signatures by eye.
- You identified R₀, R₁, C₁, R₂, and C₂ from pulse-response data instead of treating them as black-box optimizer outputs.
- You turned a multi-SOC pulse suite into lookup tables suitable for BMS work and validated them on held-out data.
- You parsed a real CALCE workbook, handled a real sign-convention mismatch, and converted raw experimental data into a reusable OCV table.
- You saw how the newer Simscape Battery workflow maps onto the manual HPPC logic rather than replacing it conceptually.

Commands, functions, and patterns that should now feel familiar:

- `readtable(..., "VariableNamingRule", "preserve")`
- `interp1(..., "pchip")`

- `lsqcurvefit`
- `cumtrapz`
- exact discrete-time branch update with $\alpha = \exp(-dt/(R \cdot C))$
- saving reproducible artifacts with `writetable` and `save`
- explicit current-sign normalization between data sources and model conventions

You should now be able to say “yes” to each of these:

- I can explain what each ECM element corresponds to physically and what it does to the voltage trace.
- I can identify R_0 from a pulse jump without hiding it inside a global optimizer.
- I can fit a 1RC or 2RC model to a pulse-recovery trace and interpret the result.
- I can build an SOC-dependent lookup table from a set of pulses.
- I can read a public workbook, normalize its units and signs, and extract an OCV curve from it.
- I can justify when a 2RC model is actually worth the extra complexity.

If any of those boxes still feels shaky, revisit Walkthroughs 2 and 3 before moving on. Chapter 7 assumes this material is solid.

Deliverable

The deliverable from the chapter plan is:

A MATLAB script that takes raw HPPC data and returns an identified 2RC model with a validation RMSE report.

The cleanest way to approach it is to combine the moving parts from this chapter into one file:

1. a loader that reads your workbook or CSV and normalizes current, voltage, time, and SOC metadata,
2. a pulse-fitting function like `fitSinglePulse`,
3. a loop over SOC nodes,
4. a lookup-table export,
5. a held-out validation plot plus a printed RMSE summary.

A strong submission should create at least three artifacts:

- `identified_2rc_lookup.csv`
- `identified_2rc_lookup.mat`
- `validation_report.txt` or a Live Script section printing RMSE and plotting measured versus modeled voltage

If you want a concrete target, start from Walkthrough 3 and replace the synthetic pulse generator with your own HPPC reader. If your data include only one SOC, you can still complete a reduced version of the deliverable by identifying a single fixed 2RC parameter set and validating it on a second pulse from the same file.

Further Practice and Reading

1. Gregory L. Plett, *Battery Management Systems, Volume II: Equivalent-Circuit Methods*, Artech House, 2015. This is still the best conceptual reference for the BMS side of ECM thinking.
2. CALCE Battery Research Data portal for the Samsung INR 18650-20R files used in this chapter: <https://calce.umd.edu/data>
3. Fangdan Zheng, Yinjiao Xing, Jiuchun Jiang, Bingxiang Sun, Jonghoon Kim, and Michael Pecht, “Influence of different open circuit voltage tests on state of charge online estimation for lithium-ion batteries,” *Applied Energy*, 183, 2016, pp. 513-525.
4. MathWorks documentation for the optional toolbox workflow: `hppcTest`, `fitECM`, and `parameterizeEquivalentCircuitBlock` in the Simscape Battery documentation.
5. Arbin Instruments documentation on current polarity conventions. This is worth bookmarking because public battery workbooks often inherit the tester’s sign convention rather than the modeler’s preferred one.

The next chapter is Lab Chapter 7: SOC Estimation with Kalman Filters.

SOC Estimation with Kalman Filters

This chapter is where the equivalent-circuit work from Lab Chapter 6 becomes a real battery-management algorithm. In the previous chapter we learned how to build R_{int} , 1RC, and 2RC models, identify their parameters from pulse data, and validate them honestly. That was necessary groundwork, but it was not yet estimation. A battery management system does not get to ask for the true SOC. It receives noisy current and voltage measurements, inherits whatever initialization error the pack had at key-on, and must still produce a credible estimate in real time. The Kalman-filter family exists because simple coulomb counting and simple voltage lookup fail exactly where a serious battery system cannot afford to fail.

Keep Textbook Chapter 10 open while you work. This chapter operationalizes the state-space formulation and estimator intuition you learned there. The equivalent-circuit model is now no longer the final product. It becomes the process model inside an observer. The OCV curve is no longer just a lookup table to draw a nice figure. It becomes the nonlinear measurement map that lets voltage correct accumulated SOC error. The polarization branch voltages are no longer just fitting artifacts. They become hidden states that the estimator must carry so it does not mistake transient overpotential for a true SOC shift.

You should also keep the sodium-ion chapter from the theory textbook nearby. That chapter matters more here than many readers expect. In a chemistry with a

steep and smooth OCV-SOC curve, voltage is informative and the filter can often recover from a bad initial SOC guess surprisingly quickly. In a chemistry with broad plateaus or weak OCV slope, which is common for hard-carbon-based sodium-ion systems over parts of the SOC range, the same filter architecture becomes far more delicate. The mathematics do not change, but the observability does. This chapter is therefore not only about Kalman filters as generic algorithms. It is about learning when the filter deserves your trust and when the chemistry itself has made the problem harder.

The chapter has three intertwined goals. First, we will implement an Extended Kalman Filter, or EKF, from scratch in MATLAB using the same style of explicit state update you built in Chapter 6. Second, we will compare that EKF with a UKF so you can see what the unscented transform buys you and what it does not. Third, we will benchmark both estimators on public data and on a sodium-ion-inspired flat-OCV stress test so the code becomes a research tool rather than a classroom toy.

This is publishable-research territory because the judgment calls are now the point. You will have to choose state definitions, process noise, measurement noise, OCV interpolation strategy, and validation metrics. Those choices are standard practice in the field, but they are not neutral. A filter that looks excellent on one drive cycle may be brittle under initialization error, temperature drift, sensor bias, or a chemistry with a flatter OCV curve. Reviewers in this area know that. The strongest estimator papers do not merely report a low RMSE. They show what assumptions were made, how the tuning was selected, what the dynamic protocol was, how convergence was defined, and where the method fails.

We will move in the same spirit as the earlier chapters: slowly enough that every line of code is teachable, but seriously enough that the finished scripts can become the seed of a paper-quality workflow. We begin with a conceptual bridge from the 1RC ECM to a nonlinear state observer. We then implement an EKF on synthetic data, because synthetic data let us separate algorithmic mistakes from dataset mess. After that we tune the process and measurement noise in a disciplined way, then move to CALCE data for a real-data benchmark, then expand to a UKF and bias-augmented states. The chapter closes by asking the question that matters for your long-term goal: what breaks when we move from an easy lithium-ion OCV curve to a sodium-ion-like plateau, and what do we do about it?

7.1 Prerequisites Check

- Required software: MATLAB R2024b or newer recommended
- Required toolboxes for the core chapter: none beyond base MATLAB
- Strongly recommended toolbox: Optimization Toolbox if you want to re-fit ECM parameters locally rather than reuse the Chapter 6 values
- Optional toolbox: Statistics and Machine Learning Toolbox is not required, but some readers may prefer it for alternative plotting or smoothing helpers
- Required textbook chapters: Textbook Chapter 10 is essential; the sodium-ion chemistry chapter is strongly recommended; the thermal and degradation chapters are not required here
- Required prior lab chapters: Lab Chapters 1, 2, and 6 are essential; Lab Chapter 4 helps if you want to cross-check current-profile handling
- Estimated time: 16 to 20 hours for the full chapter, or 10 to 12 hours for Run 1 through the EKF tuning study

If the 1RC and 2RC state updates from Lab Chapter 6 still feel mechanical rather than intuitive, revisit Sections 6.1 through 6.4 before you continue. If the idea of OCV as a function of SOC is still blurry, reread the OCV-SOC table discussion from Textbook Chapter 10 and the OCV extraction workflow from Lab Chapter 6. The code here assumes that those ideas are already familiar even if they are not yet automatic.

7.2 Environment Setup

The core of this chapter is intentionally lightweight. We will use plain MATLAB scripts with local functions, which keeps every exercise runnable on a laptop without Simulink, Simscape Battery, or a third-party toolbox. That is a teaching choice and also a research choice. If you understand the estimator in plain MATLAB first, you will be much safer later when a higher-level toolbox hides the bookkeeping.

7.2.1 Verify your MATLAB release and workspace hygiene

Open MATLAB and run:

```
ver  
pwd
```

Expected behavior is simple. `ver` should print your MATLAB release and installed toolboxes. `pwd` should print the working directory that will hold the chapter files. If you are deep inside an unrelated project folder, move now. Estimator work tends to generate multiple exported tables, tuning sweeps, and cached results, and the quickest way to make the work irreproducible is to scatter those files across ad hoc directories.

7.2.2 Create a clean chapter workspace

Run the following exactly once in a fresh MATLAB session:

```
chapterRoot = fullfile(pwd, "chapter7_soc_estimation_workspace");  
if ~exist(chapterRoot, "dir")  
    mkdir(chapterRoot);  
end  
cd(chapterRoot);  
  
fprintf("Working folder: %s\n", chapterRoot);
```

Expected output:

```
Working folder: ../../chapter7_soc_estimation_workspace
```

The exact path will differ on your machine. What matters is that you now have a dedicated workspace for this chapter. In Run 2 we will download CALCE files into a subfolder here, write intermediate CSV files, and save estimator outputs so that every figure can be regenerated later.

7.2.3 Run a minimal estimator hello-world

This snippet does not estimate a full battery state yet. It verifies that your MATLAB session handles the core linear-algebra operations an EKF depends on.

```
clear; close all; clc;  
  
% Minimal two-state prediction/update sanity check  
A = [1.0 0.0; 0.0 0.92];  
C = [0.75 -1.0];  
Q = diag([1e-6, 2e-5]);  
R = (5e-3)^2;
```

```

x_hat = [0.80; 0.01];
P = diag([0.02^2, 0.03^2]);
current_a = 2.0;
measured_voltage_v = 3.76;

% Process-model prediction
x_pred = A * x_hat + [-(1 / (3600 * 2.3)); 0.08] * current_a;
P_pred = A * P * A' + Q;

% Measurement update
predicted_voltage_v = 3.95 - 0.015 * current_a + C * x_pred;
innovation_v = measured_voltage_v - predicted_voltage_v;
innovation_cov = C * P_pred * C' + R;
kalman_gain = (P_pred * C') / innovation_cov;
x_next = x_pred + kalman_gain * innovation_v;
P_next = (eye(2) - kalman_gain * C) * P_pred;

fprintf("Predicted voltage: %.4f V\n", predicted_voltage_v);
fprintf("Innovation: %.4f V\n", innovation_v);
fprintf("Updated SOC estimate: %.4f\n", x_next(1));
fprintf("Updated RC-voltage estimate: %.4f V\n", x_next(2));

```

Expected output should look numerically reasonable rather than identical to the values below:

```

Predicted voltage: 3.7452 V
Innovation: 0.0148 V
Updated SOC estimate: 0.813...
Updated RC-voltage estimate: 0.008...

```

The exact last digits are not the point. The point is that MATLAB executes the predict-update structure cleanly, the innovation covariance is positive, and the updated state remains physically plausible.

7.2.4 Common setup failures and fixes

Script runs, but figures do not appear. This usually means you are running in a non-graphical session or your figure windows are opening behind other windows. Test with `figure; plot(1:10); grid on;`

interp1 errors later in the chapter. Almost all such errors come from OCV tables whose SOC grid is unsorted or contains duplicate entries. The filter code assumes the SOC grid is strictly increasing from 0 to 1.

Covariance matrices become singular or non-positive. In early experiments this is usually a coding mistake, not a deep filtering issue. Check matrix dimensions first, then confirm that Q and R are positive and that you are not accidentally transposing a row vector into the wrong shape.

The filter seems to “work” but SOC drifts outside $[0, 1]$. That is not acceptable behavior, even in a teaching example. Later code in this chapter clamps SOC after prediction and after update to keep the state physically meaningful.

7.3 Conceptual Bridge: From an ECM to a Nonlinear State Observer

In Textbook Chapter 10, the state-space form of a battery estimator may have looked almost deceptively compact. That is because the notation hides the real conceptual move. In Chapter 6, the equivalent-circuit model was a simulator. We supplied current, propagated internal variables, and predicted voltage. In this chapter, the same model becomes a process model inside an observer. That means the voltage measurement is no longer only an output to plot. It is evidence that can correct hidden-state error.

For the simplest useful observer in this chapter, we will use a 1RC model with the state vector

$$\mathbf{x}_k = \begin{bmatrix} z_k \\ v_{1,k} \end{bmatrix}, \quad (1)$$

where z_k is SOC and $v_{1,k}$ is the polarization-branch voltage at sample index k . The discrete-time process model is

$$z_{k+1} = z_k - \frac{\eta \Delta t}{3600Q} I_k + w_{z,k}, \quad (2)$$

and

$$v_{1,k+1} = \alpha v_{1,k} + R_1(1 - \alpha)I_k + w_{v,k}, \quad \alpha = \exp\left(-\frac{\Delta t}{R_1 C_1}\right), \quad (3)$$

where Q is cell capacity in ampere-hours, η is coulombic efficiency, and $w_{z,k}$ and $w_{v,k}$ are process-noise terms. Equation (2) is just coulomb counting written as a state equation. Equation (3) is the exact discrete-time update for the RC branch that you already used in Lab Chapter 6.

The nonlinear measurement equation is

$$y_k = U_{oc}(z_k) - R_0 I_k - v_{1,k} + n_k, \quad (4)$$

where y_k is the measured terminal voltage, $U_{oc}(z)$ is the OCV-SOC relationship, and n_k is measurement noise. Equation (4) contains the core reason SOC estimation is both possible and fragile. It is possible because voltage depends on SOC through the OCV map. It is fragile because the dependence is only as informative as the slope of that map.

To see that, differentiate Equation (4) with respect to the state:

$$C_k = \frac{\partial y_k}{\partial \mathbf{x}_k} = \begin{bmatrix} \frac{dU_{oc}}{dz}(z_k) & -1 \end{bmatrix}. \quad (5)$$

Equation (5) is the measurement Jacobian used by the EKF. It says something physically important. The filter sees SOC only through the OCV slope $\frac{dU_{oc}}{dz}$. If that slope is large, a small SOC error produces a noticeable voltage error, so the measurement update can correct the state strongly. If the slope is small, a large SOC error may produce only a tiny voltage signature, and the measurement update becomes weak. This is why sodium-ion plateaus and flat graphite or hard-carbon regions are estimator headaches: the chemistry itself has made voltage less informative.

The EKF proceeds in the familiar predict-measure-update pattern. During prediction, we propagate the state with the process model and propagate the covariance with the local linearization

$$\mathbf{P}_{k|k-1} = \mathbf{A}_k \mathbf{P}_{k-1|k-1} \mathbf{A}_k^\top + \mathbf{Q}, \quad (6)$$

where $\mathbf{A}_k$ is the process Jacobian and $\mathbf{Q}$ is the process-noise covariance. For the 1RC model used here,

$$\mathbf{A}_k = \begin{bmatrix} 1 & 0 \\ 0 & \alpha \end{bmatrix}. \quad (7)$$

During the measurement step, we compute the innovation

$$r_k = y_k - \hat{y}_{k|k-1}, \quad (8)$$

its covariance

$$S_k = C_k P_{k|k-1} C_k^\top + R, \quad (9)$$

the Kalman gain

$$K_k = P_{k|k-1} C_k^\top S_k^{-1}, \quad (10)$$

and the corrected state

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k r_k. \quad (11)$$

Those equations are standard. What matters pedagogically is how they connect to Chapter 6. The process model is just your ECM in state-space form. The innovation is the difference between measured voltage and predicted voltage. The covariance tells the filter how much to trust its own model versus the measurement. The Kalman gain translates that trust balance into an actual SOC correction.

Three misconceptions are worth removing before we write code. The first is that the EKF “finds SOC from voltage.” It does not. The EKF combines a process model, a current history, an OCV map, and a voltage measurement. If the current sensor is biased or the OCV map is wrong, the estimator can still fail badly while appearing mathematically healthy.

The second misconception is that Q and R are merely tuning knobs that you can adjust until the plot looks nice. They are tuning knobs, but they also encode modeling honesty. A very small process-noise covariance Q says, in effect, “I trust my ECM and current integration almost completely.” A very small measurement-noise variance R says, “I trust the voltage sensor and measurement model almost completely.” If either claim is too optimistic, the filter can become overconfident and brittle.

The third misconception is that a more nonlinear filter automatically solves weak observability. It does not. A UKF can approximate nonlinear propagation more accurately than an EKF, especially when the OCV map has stronger curvature, but no sigma-point trick can create observability where the chemistry provides almost none. If the OCV plateau is flat, the problem is fundamentally hard. That is exactly why the sodium-ion stress test later in the chapter matters.

The software bridge from theory to MATLAB is therefore this. We store the OCV curve as a lookup table, evaluate both the curve and its derivative by interpolation, propagate the hidden states with the same exact discrete-time ECM

equations we already trust, and let the filter update those states using the voltage residual. Once that logic is clear, the code becomes compact. The difficulty is not syntax. It is making physically honest choices about the model and the noise assumptions.

Guided Walkthrough 1

Build a State-Space Battery Model and See Why OCV Slope Controls Observability

Learning objective: See, in code and on plots, why the same EKF architecture behaves differently on a steep OCV curve and on a sodium-ion-like plateau.

Before we implement the EKF, we need to understand what information the voltage measurement actually contains. The best way to do that is to compare two OCV curves. The first will be a lithium-ion-like curve with a healthy slope over much of the SOC range. The second will be a sodium-ion-inspired curve with a broad flat plateau. We will compute the OCV slope, convert a fixed SOC error into an implied voltage error, and then simulate a simple current profile to see where the measurement should or should not be informative.

```
clear; close all; clc;

% Shared SOC grid
soc_grid = linspace 0, 1, 1001 .';

% Lithium-ion-like OCV curve: smooth and steadily increasing
ocv_li_v = 3.00 ...
    + 0.72 * soc_grid ...
    + 0.18 * tanh soc_grid - 0.15 / 0.06 ...
    + 0.16 * tanh soc_grid - 0.85 / 0.05 ;

% Sodium-ion-inspired OCV curve: broad flat plateau in the middle
ocv_sib_v = 2.45 ...
    + 0.40 * soc_grid ...
    + 0.26 * tanh soc_grid - 0.10 / 0.05 ...
    + 0.06 * tanh soc_grid - 0.45 / 0.10 ...
    + 0.28 * tanh soc_grid - 0.88 / 0.04 ;

% Numerical derivatives dUoc/dz in volts per unit SOC
dudsoc_li = gradient ocv_li_v, soc_grid ;
dudsoc_sib = gradient ocv_sib_v, soc_grid ;
```

```

% Translate a 5% SOC estimation error into an expected voltage mismatch
soc_error = 0.05;
delta_v_li = dudsoc_li * soc_error;
delta_v_sib = dudsoc_sib * soc_error;

% Build a simple discharge-rest-discharge profile that spans the plateau
↳ region
dt = 1;
time_s = 0:1:2600 .';
current_a = zeros size time_s ;
current_a time_s >= 100 & time_s < 800 = 2.0;
current_a time_s >= 1100 & time_s < 1700 = 1.0;
current_a time_s >= 1900 & time_s < 2450 = 1.5;

capacity_ah = 3.0;
initial_soc = 0.95;

soc_trace = initial_soc - cumsum([0; current_a(1:end-1)]) * dt / (3600 *
↳ capacity_ah);
soc_trace = min max soc_trace, 0 , 1 ;

voltage_li_v = interp1 soc_grid, ocv_li_v, soc_trace, "pchip", "extrap" ;
voltage_sib_v = interp1 soc_grid, ocv_sib_v, soc_trace, "pchip", "extrap" ;
dudsoc_li_trace = interp1 soc_grid, dudsoc_li, soc_trace, "pchip", "extrap" ;
dudsoc_sib_trace = interp1(soc_grid, dudsoc_sib, soc_trace, "pchip",
↳ "extrap");

mid_mask = soc_grid >= 0.40 & soc_grid <= 0.60;
fprintf "Mean dUoc/dSOC between 40%% and 60%% SOC:\n" ;
fprintf "  Li-ion-like curve : %.4f V/SOC\n", mean dudsoc_li mid_mask ;
fprintf "  SIB-like curve    : %.4f V/SOC\n", mean dudsoc_sib mid_mask ;
fprintf "Expected voltage error from a 5%% SOC mistake near 50%% SOC:\n" ;
idx50 = find abs soc_grid - 0.50 == min abs soc_grid - 0.50 , 1, "first" ;
fprintf "  Li-ion-like curve : %.4f V\n", delta_v_li idx50 ;
fprintf "  SIB-like curve    : %.4f V\n", delta_v_sib idx50 ;

figure "Color", "w", "Position", 100 100 1100 850 ;

subplot 3, 1, 1 ;
plot soc_grid, ocv_li_v, "LineWidth", 2.0 ; hold on;
plot soc_grid, ocv_sib_v, "LineWidth", 2.0 ;
grid on;
xlabel "SOC [-]" ;
ylabel "OCV [V]" ;

```

```

title("Two OCV-SOC curves with very different estimator friendliness");
legend("Li-ion-like", "SIB-like", "Location", "northwest");

subplot(3, 1, 2);
plot(soc_grid, dudsoc_li, "LineWidth", 2.0); hold on;
plot(soc_grid, dudsoc_sib, "LineWidth", 2.0);
grid on;
xlabel("SOC [-]");
ylabel("dU_{oc}/dSOC [V per unit SOC]");
title("Measurement sensitivity to SOC is controlled by OCV slope");
legend("Li-ion-like", "SIB-like", "Location", "northeast");

subplot(3, 1, 3);
yyaxis left;
plot(time_s, current_a, "k", "LineWidth", 1.4);
ylabel("Current [A]");
yyaxis right;
plot(time_s, dudsoc_li_trace, "LineWidth", 1.8); hold on;
plot(time_s, dudsoc_sib_trace, "LineWidth", 1.8);
ylabel("Local dU_{oc}/dSOC [V per unit SOC]");
grid on;
xlabel("Time [s]");
title("The same duty cycle becomes more or less observable depending on
↳ chemistry");
legend("Current", "Li-ion-like slope", "SIB-like slope", "Location", "best");

```

The code begins by creating two explicit OCV maps on the same SOC grid. The lithium-ion-like curve is intentionally steep at low and high SOC and still meaningfully sloped through the middle. The sodium-ion-inspired curve has a much flatter middle region. These are teaching curves rather than digitized data, but they are realistic enough to make the observability issue concrete.

The `gradient` calls compute a numerical derivative of each OCV curve with respect to SOC. That derivative is not a decorative extra. It is the first element of the EKF measurement Jacobian from Equation (5). If you later use a noisy or badly smoothed OCV table, the derivative becomes noisy too, and the filter can respond erratically. That is why Chapter 6 spent time on clean OCV tables.

Next, we convert a 5% SOC estimation error into an implied voltage error by multiplying the slope by 0.05. This is a useful mental calibration trick. If $dU_{oc}/dSOC$ is 0.8 V/SOC, then a 5% SOC error implies roughly 40 mV of voltage disagreement, which is easy for a clean voltage sensor to see. If the slope is 0.1

V/SOC, the same 5% SOC error implies only 5 mV, which is already comparable to ordinary sensor noise and modeling error.

The current profile is deliberately simple. It is just a sequence of discharge pulses with rest periods. We are not yet trying to test the RC dynamics. We are asking how the same SOC trajectory looks to the measurement model when the chemistry changes. The `soc_trace` calculation is plain coulomb counting, using the previous current sample so the trace stays aligned with the discrete-time convention we will use in the estimator.

The bottom subplot overlays current and the local OCV slope along the trajectory. That plot is one of the most important in the first half of the chapter. It makes visible the part of the estimator problem that often stays hidden when people discuss Kalman filters abstractly. The chemistry has already decided, before any algorithm starts, where voltage can meaningfully correct SOC and where it cannot.

7.3.1 Expected Output for Walkthrough 1

The first subplot should show two monotonic OCV curves. The lithium-ion-like curve should climb steadily from roughly 2.7 V to above 4.0 V with noticeable but smooth curvature. The sodium-ion-inspired curve should also rise overall, but its middle portion should flatten visibly, creating a broad region where the voltage changes only weakly with SOC.

The second subplot should make the main lesson unmistakable. The lithium-ion-like derivative should remain appreciably above zero through most of the usable SOC range, while the SIB-like derivative should dip much lower through the middle. Near 50% SOC, the difference should be large enough that the printed implied voltage error for a 5% SOC mistake is several times bigger for the Li-ion-like curve than for the SIB-like curve.

The third subplot should show a piecewise-constant current trace on the left axis and two slope traces on the right axis. As SOC moves through the midrange, the SIB-like slope should sag, sometimes dramatically, while the Li-ion-like slope remains healthier. That plot tells you where an EKF will tend to rely more on current integration than on voltage correction.

7.3.2 What Could Go Wrong in Walkthrough 1

The derivative plot is noisy or jagged. This usually happens when the OCV grid is too coarse or the OCV data are noisy. Increase the grid density or smooth the

OCV data before differentiating. In real workflows, an unsmoothed derivative is one of the fastest ways to destabilize an EKF.

The SOC trace hits zero or one too early. That means the capacity or current magnitudes are inconsistent with the simulation length. Increase `capacity_ah` or shorten the pulses so the example stays inside the meaningful operating range.

The two OCV curves look too similar. If you accidentally change the coefficients so both curves remain steep, the pedagogical point is lost. The SIB-inspired curve should have a visibly flatter mid-SOC region.

7.3.3 Reflection on Walkthrough 1

This exercise teaches a habit that will matter throughout the rest of the manual: before tuning an estimator, ask what the measurement can realistically tell you. Many Kalman-filter problems in battery research are not primarily “algorithm problems.” They are observability problems created by chemistry, protocol, or poor OCV preprocessing. We will return to this point when we later stress-test the filter on a hard-carbon sodium-ion-like plateau.

Guided Walkthrough 2

Implement an Extended Kalman Filter from Scratch on Synthetic `1RC` Data

Learning objective: Write a complete EKF that estimates SOC and polarization voltage from current and noisy terminal voltage, starting from a badly wrong initial SOC.

We now have enough conceptual footing to build the estimator itself. We will first generate synthetic truth from a `1RC` model, add realistic voltage noise, and then ask the EKF to recover SOC from a deliberately poor initial guess. This is the cleanest way to debug the algorithm because we know the true hidden states. If the filter diverges here, the mistake is in our implementation or in our tuning, not in the dataset.

```
clear; close all; clc;

% Time base and current profile
dt = 1;
time_s = 0:1:2400 .';
```

```

current_a = zeros(size(time_s));
current_a(time_s >= 50 & time_s < 350) = 2.0;
current_a(time_s >= 500 & time_s < 780) = 3.5;
current_a(time_s >= 980 & time_s < 1220) = -1.5;
current_a(time_s >= 1450 & time_s < 1740) = 2.8;
current_a(time_s >= 1880 & time_s < 2260) = 1.2;

% OCV lookup table shared by truth model and estimator
ocv_soc = linspace(0, 1, 201).';
ocv_v = 3.00 ...
    + 0.74 * ocv_soc ...
    + 0.16 * tanh((ocv_soc - 0.14) / 0.06) ...
    + 0.18 * tanh((ocv_soc - 0.86) / 0.05);

% True cell parameters
params.Q_ah = 2.3;
params.eta = 1.0;
params.R0 = 0.014;
params.R1 = 0.018;
params.C1 = 95.0;

% Generate synthetic truth
true_initial_soc = 0.88;
[soc_true, vl_true, voltage_true_v] = simulateTruth1Rc( ...
    time_s, current_a, true_initial_soc, ocv_soc, ocv_v, params);

% Add measurement noise
rng(11);
voltage_noise_std_v = 4e-3;
voltage_meas_v = voltage_true_v + voltage_noise_std_v *
    ↪ randn(size(voltage_true_v));

% Coulomb counting baseline with wrong initialization
soc_cc = true_initial_soc - 0.20 ...
    - cumsum 0; current_a 1:end-1 * dt / 3600 * params.Q_ah ;
soc_cc = min max soc_cc, 0 , 1 ;

% EKF initialization
x_hat = zeros 2, numel time_s ;
x_hat(:, 1) = 0.68; 0.00 ; % deliberately wrong SOC guess

P = diag 0.08^2, 0.03^2 ;
Q = diag 2e-8, 4e-6 ;
R = voltage_noise_std_v ^2;

```

```

innovation_v = zeros(size(time_s));
kalman_gain_soc = zeros(size(time_s));
kalman_gain_v1 = zeros(size(time_s));
predicted_voltage_v = zeros(size(time_s));

predicted_voltage_v(1) = measurement1Rc(x_hat(:, 1), current_a(1), ocv_soc,
    ↪ ocv_v, params);

for k = 2:numel(time_s)
    % Prediction step
    x_pred = predictState1Rc(x_hat(:, k - 1), current_a(k - 1), dt, params);
    x_pred(1) = min(max(x_pred(1), 0), 1);

    alpha = exp(-dt / (params.R1 * params.C1));
    A = [1.0 0.0; 0.0 alpha];
    P_pred = A * P * A' + Q;

    % Predicted measurement and local Jacobian
    predicted_voltage_v(k) = measurement1Rc(x_pred, current_a(k), ocv_soc,
    ↪ ocv_v, params);
    dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
    C = [dUoc_dSoc, -1.0];

    % Innovation
    innovation_v(k) = voltage_meas_v(k) - predicted_voltage_v(k);
    S = C * P_pred * C' + R;
    K = (P_pred * C') / S;

    % Correction
    x_hat(:, k) = x_pred + K * innovation_v(k);
    x_hat(1, k) = min(max(x_hat(1, k), 0), 1);
    kalman_gain_soc(k) = K(1);
    kalman_gain_v1(k) = K(2);

    % Joseph-form covariance update for numerical robustness
    I2 = eye(2);
    P = (I2 - K * C) * P_pred * (I2 - K * C)' + K * R * K';
end

soc_error = x_hat(1, :) - soc_true;
rmse_soc = sqrt(mean(soc_error.^2));
max_abs_soc_error = max(abs(soc_error));

convergence_threshold = 0.01;
idx_conv = find(abs(soc_error) <= convergence_threshold, 1, "first");

```



```

if isempty(idx_conv)
    convergence_time_s = NaN;
else
    convergence_time_s = time_s(idx_conv);
end

fprintf("EKF SOC RMSE: %.4f\n", rmse_soc);
fprintf("EKF maximum absolute SOC error: %.4f\n", max_abs_soc_error);
fprintf("Convergence time to within 1%% SOC: %.1f s\n", convergence_time_s);

figure("Color", "w", "Position", [80 80 1200 900]);

subplot(4, 1, 1);
plot(time_s, current_a, "k", "LineWidth", 1.4);
grid on;
xlabel("Time [s]");
ylabel("Current [A]");
title("Synthetic current profile");

subplot(4, 1, 2);
plot(time_s, soc_true, "LineWidth", 2.0); hold on;
plot(time_s, soc_cc, "--", "LineWidth", 1.8);
plot(time_s, x_hat(1, :), "LineWidth", 1.8);
grid on;
xlabel("Time [s]");
ylabel("SOC [-]");
title("True SOC, bad coulomb-counting baseline, and EKF estimate");
legend("True SOC", "Coulomb counting", "EKF estimate", "Location", "best");

subplot(4, 1, 3);
plot(time_s, voltage_meas_v, "Color", [0.6 0.6 0.6]); hold on;
plot(time_s, predicted_voltage_v, "LineWidth", 1.6);
plot(time_s, voltage_true_v, "--", "LineWidth", 1.6);
grid on;
xlabel("Time [s]");
ylabel("Voltage [V]");
title("Measured, predicted, and true terminal voltage");
legend("Measured voltage", "EKF predicted voltage", "True voltage",
    ↪ "Location", "best");

subplot(4, 1, 4);
yyaxis left;
plot(time_s, innovation_v * 1000, "LineWidth", 1.4);
ylabel("Innovation [mV]");
yyaxis right;

```

```

plot(time_s, kalman_gain_soc, "LineWidth", 1.4); hold on;
plot(time_s, kalman_gain_v1, "LineWidth", 1.4);
ylabel("Kalman gain");
grid on;
xlabel("Time [s]");
title("Innovation and estimator gain evolution");
legend("Innovation", "SOC gain", "RC-voltage gain", "Location", "best");

function x_next = predictState1Rc(x_now, current_a, dt, params)
    alpha = exp(-dt / (params.R1 * params.C1));

    soc_next = x_now(1) - params.eta * current_a * dt / (3600 * params.Q_ah);
    v1_next = alpha * x_now(2) + params.R1 * (1 - alpha) * current_a;

    x_next = [soc_next; v1_next];
end

function voltage_v = measurement1Rc(x_state, current_a, ocv_soc, ocv_v,
↳ params)
    ocv_now = interp1(ocv_soc, ocv_v, x_state(1), "pchip", "extrap");
    voltage_v = ocv_now - current_a * params.R0 - x_state(2);
end

function slope = ocvSlope(soc, ocv_soc, ocv_v)
    d_ocv = gradient(ocv_v, ocv_soc);
    slope = interp1(ocv_soc, d_ocv, soc, "pchip", "extrap");
end

function [soc, v1_v, voltage_v] = simulateTruth1Rc( ...
    time_s, current_a, initial_soc, ocv_soc, ocv_v, params)

    n = numel(time_s);
    dt = time_s(2) - time_s(1);
    alpha = exp(-dt / (params.R1 * params.C1));

    soc = zeros(n, 1);
    v1_v = zeros(n, 1);
    voltage_v = zeros(n, 1);

    soc(1) = initial_soc;
    voltage_v(1) = measurement1Rc([soc(1); v1_v(1)], current_a(1), ocv_soc,
↳ ocv_v, params);

```

```

for k = 2:n
    soc(k) = soc(k - 1) - params.eta * current_a(k - 1) * dt / (3600 *
        ↪ params.Q_ah);
    soc(k) = min(max(soc(k), 0), 1);
    v1_v(k) = alpha * v1_v(k - 1) + params.R1 * (1 - alpha) * current_a(k
        ↪ - 1);
    voltage_v(k) = measurement1Rc([soc(k); v1_v(k)], current_a(k),
        ↪ ocv_soc, ocv_v, params);
end
end

```

The first block defines the excitation. The current profile includes multiple discharge regions and one charge pulse so the RC state sees several transitions. That gives the filter a chance to learn both SOC drift and dynamic polarization. A single constant-current segment would be enough to make the code run, but it would teach much less.

The OCV table is deliberately smooth. In Chapter 6 we discussed building an OCV map from measured data. Here we reuse the same lookup-table idea but evaluate it inside the measurement equation. The true system and the estimator share the same OCV map in this first experiment. That is a teaching shortcut. In Run 2 we will relax that assumption by moving to public data where the model-data mismatch is real.

The function `simulateTruth1Rc` is your hidden battery. It produces the true SOC, the true polarization voltage, and the true terminal voltage. Notice that its state update is exactly the same physics you already learned in Chapter 6. This is important: the EKF is not a separate battery model. It is an inference layer wrapped around the same battery model.

The line that builds `soc_cc` creates a baseline for comparison. We intentionally start coulomb counting with a 20% SOC error. Because pure coulomb counting has no measurement correction channel, that initialization error never disappears. This is exactly why the EKF exists. It can use voltage to recover from a bad initial guess, provided the OCV slope gives it enough information.

The filter state `x_hat` contains two entries at each time step: SOC and RC-branch voltage. The initial SOC estimate is set to `0.68` even though the truth is `0.88`. That is a large miss on purpose. If you initialize the filter too close to truth, you learn very little about its corrective behavior.

The covariance P , process noise Q , and measurement noise R are the heart of the design. P encodes our initial uncertainty. Q says how much model mismatch or unmodeled disturbance we expect per time step. R reflects voltage measurement uncertainty. In this first synthetic example, R is chosen to match the injected measurement noise standard deviation. That is standard practice in a controlled teaching problem, though in real experiments you often inflate R slightly to absorb modeling error as well.

Inside the loop, the prediction step uses `current_a(k - 1)` because the process model advances from state $k-1$ to state k using the current already applied over that interval. The predicted measurement uses `current_a(k)` because the terminal voltage at sample k depends on the current present at that sample. Keeping those indices straight is one of the most common sources of silent estimator bugs.

The Jacobian row $C = [dU_{oc_dSOC}, -1.0]$ is the code form of Equation (5). The first component measures how strongly voltage responds to SOC at the present operating point. The second component is always -1 because a larger polarization voltage reduces the measured terminal voltage one-for-one. This is where the battery physics becomes estimator math.

The covariance update uses the Joseph form

$$P_{k|k} = (I - K_k C_k) P_{k|k-1} (I - K_k C_k)^T + K_k R K_k^T, \quad (12)$$

which is numerically safer than the simpler algebraically equivalent form. That choice is standard practice in the field and worth internalizing early.

7.3.4 Expected Output for Walkthrough 2

The first subplot should show a piecewise-constant current trace with several plateaus and one negative current section for charge. The second subplot is the one to read most carefully. The true SOC should start near 0.88 and drift gradually with the current history. The bad coulomb-counting baseline should stay displaced by roughly 0.20 for the entire run because it has no correction mechanism. The EKF estimate should start at the wrong value but then bend toward the truth as voltage updates accumulate, usually reaching within about 1% SOC after a few hundred seconds.

The voltage subplot should show a noisy gray measured trace, a cleaner

predicted-voltage trace from the EKF, and a dashed true-voltage trace. The EKF prediction should track the overall voltage waveform closely after the first correction transient. If the predicted voltage stays visibly offset from both the measured and true traces for long periods, the filter is not tuned or coded correctly.

The bottom subplot should show the innovation in millivolts and the two Kalman-gain components. Early in the run, the SOC gain is often larger because the filter is trying to repair the bad initialization. As uncertainty shrinks, the gain typically settles to smaller values. The innovation should look like a zero-centered noisy signal after convergence rather than a persistently biased one.

The printed metrics should show a clear EKF advantage over the wrong-initialized coulomb-counting baseline. A typical run with the values above produces an SOC RMSE on the order of a few thousandths to a few hundredths, a maximum absolute error well below the initial 20% miss, and a convergence time of a few hundred seconds. Exact values vary slightly with MATLAB version because of floating-point details, but the qualitative behavior should be stable.

7.3.5 What Could Go Wrong in Walkthrough 2

The EKF estimate diverges immediately. The most common cause is an indexing error between `current_a(k - 1)` and `current_a(k)`. The process model and measurement model do not use the same sample in the same way.

The SOC estimate never leaves the bad initial value. This usually means the measurement Jacobian is wrong, the Kalman gain is near zero because R is far too large, or the OCV derivative is being evaluated incorrectly.

The innovation has a persistent offset. That often signals a model mismatch, such as an incorrect R_0 , or a measurement convention mistake, such as using the wrong current sign.

The covariance becomes negative or complex. That is almost always a coding bug. Check that Q and R are positive scalars or positive-definite matrices and confirm that the Joseph update is implemented exactly.

7.3.6 Reflection on Walkthrough 2

This walkthrough is the real threshold moment of the chapter. You have now written an EKF that works on a battery-shaped problem rather than on an abstract textbook matrix example. More importantly, you have seen that the filter is not magic. It is only a disciplined reconciliation between a process model, a

measurement model, and quantified uncertainty.

Guided Walkthrough 3

Tune Process Noise and Measurement Noise Without Guesswork

Learning objective: Understand how Q and R change estimator behavior, and learn a practical sweep-based workflow for selecting them.

The hardest part of EKF work in practice is not writing the loop. It is tuning the covariances honestly. Many beginner implementations work only because Q and R were tweaked until the plot looked appealing for one dataset. That is not a research workflow. In this walkthrough we will run a systematic sweep over SOC process noise and voltage measurement noise, compute performance metrics for each pair, and then compare three concrete tuning cases: overconfident model trust, overconfident measurement trust, and a balanced setting.

```
clear; close all; clc;

% Shared synthetic dataset
dt = 1;
time_s = 0:1:2200 .';
current_a = zeros size time_s ;
current_a(time_s >= 60 & time_s < 340) = 2.2;
current_a(time_s >= 470 & time_s < 730) = 3.8;
current_a(time_s >= 890 & time_s < 1110) = -1.4;
current_a(time_s >= 1300 & time_s < 1650) = 2.6;
current_a(time_s >= 1820 & time_s < 2080) = 1.8;

ocv_soc = linspace(0, 1, 201) .';
ocv_v = 3.00 ...
    + 0.74 * ocv_soc ...
    + 0.16 * tanh(ocv_soc - 0.14) / 0.06 ...
    + 0.18 * tanh(ocv_soc - 0.86) / 0.05 ;

params.Q_ah = 2.3;
params.eta = 1.0;
params.R0 = 0.014;
params.R1 = 0.018;
params.C1 = 95.0;

true_initial_soc = 0.90;
soc_true, ~, voltage_true_v = simulateTruth1Rc ...
```

```

    time_s, current_a, true_initial_soc, ocv_soc, ocv_v, params);

rng(22);
meas_noise_std_v = 5e-3;
voltage_meas_v = voltage_true_v + meas_noise_std_v *
    ↪ randn(size(voltage_true_v));

q_soc_candidates = logspace -10, -6, 9 ;
r_candidates_v = 1 2 3 5 7 10 15 * 1e-3;

rmse_map = zeros numel q_soc_candidates , numel r_candidates_v ;
conv_map = zeros numel q_soc_candidates , numel r_candidates_v ;

for i = 1:numel q_soc_candidates
    for j = 1:numel r_candidates_v
        q_soc = q_soc_candidates i ;
        q_v1 = 1e-5;
        r_var = r_candidates_v j ^2;

        soc_hat, ~, ~ = runEkf1Rc ...
            time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
            0.65, diag 0.08^2, 0.03^2 , diag q_soc, q_v1 , r_var ;

        soc_error = soc_hat - soc_true;
        rmse_map i, j = sqrt mean soc_error.^2 ;

        idx_conv = find abs soc_error <= 0.01, 1, "first" ;
        if isempty idx_conv
            conv_map i, j = NaN;
        else
            conv_map i, j = time_s idx_conv ;
        end
    end
end

% Compare three representative tuning cases
caseNames = "Model overtrusted", "Measurement overtrusted", "Balanced" ;
Q_cases = cat 3, ...
    diag 1e-10, 1e-6 , ...
    diag 1e-6, 1e-4 , ...
    diag 1e-8, 1e-5 ;
R_cases = 15e-3 ^2, 1e-3 ^2, 5e-3 ^2 ;

soc_cases = zeros numel time_s , 3 ;
innov_cases = zeros numel time_s , 3 ;

```

```

for c = 1:3
    [soc_cases(:, c), ~, innov_cases(:, c)] = runEkf1Rc( ...
        time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
        0.65, diag([0.08^2, 0.03^2]), Q_cases(:, :, c), R_cases(c));
end

figure("Color", "w", "Position", [60 60 1250 900]);

subplot(2, 2, 1);
imagesc(r_candidates_v * 1000, log10(q_soc_candidates), rmse_map);
set(gca, "YDir", "normal");
colorbar;
xlabel("Voltage measurement noise std [mV]");
ylabel("log_{10}(SOC process noise variance)");
title("SOC RMSE across Q-R sweep");

subplot(2, 2, 2);
imagesc(r_candidates_v * 1000, log10(q_soc_candidates), conv_map);
set(gca, "YDir", "normal");
colorbar;
xlabel("Voltage measurement noise std [mV]");
ylabel("log_{10}(SOC process noise variance)");
title("Convergence time to 1% SOC across Q-R sweep");

subplot(2, 2, 3);
plot(time_s, soc_true, "k", "LineWidth", 2.0); hold on;
plot(time_s, soc_cases(:, 1), "LineWidth", 1.7);
plot(time_s, soc_cases(:, 2), "LineWidth", 1.7);
plot(time_s, soc_cases(:, 3), "LineWidth", 1.7);
grid on;
xlabel("Time [s]");
ylabel("SOC [-]");
title("How three tuning philosophies change the estimator trajectory");
legend(["True SOC", caseNames], "Location", "best");

subplot(2, 2, 4);
plot(time_s, innov_cases(:, 1) * 1000, "LineWidth", 1.4); hold on;
plot(time_s, innov_cases(:, 2) * 1000, "LineWidth", 1.4);
plot(time_s, innov_cases(:, 3) * 1000, "LineWidth", 1.4);
grid on;
xlabel("Time [s]");
ylabel("Innovation [mV]");
title("Innovation behavior under three tuning philosophies");
legend(caseNames, "Location", "best");

```



```

function [soc_hat, v1_hat, innovation_v] = runEkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

    dt = time_s(2) - time_s(1);
    n = numel(time_s);

    x_hat = zeros(2, n);
    x_hat(:, 1) = [initial_soc_guess; 0];
    P = P0;

    innovation_v = zeros(n, 1);
    alpha = exp(-dt / (params.R1 * params.C1));

    for k = 2:n
        x_pred = predictState1Rc(x_hat(:, k - 1), current_a(k - 1), dt,
            ↪ params);
        x_pred(1) = min(max(x_pred(1), 0), 1);

        A = [1.0 0.0; 0.0 alpha];
        P_pred = A * P * A' + Q;

        y_pred = measurement1Rc(x_pred, current_a(k), ocv_soc, ocv_v,
            ↪ params);
        dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
        C = [dUoc_dSoc, -1.0];

        innovation_v(k) = voltage_meas_v(k) - y_pred;
        S = C * P_pred * C' + R;
        K = (P_pred * C') / S;

        x_hat(:, k) = x_pred + K * innovation_v(k);
        x_hat(1, k) = min(max(x_hat(1, k), 0), 1);

        I2 = eye(2);
        P = (I2 - K * C) * P_pred * (I2 - K * C)' + K * R * K';
    end

    soc_hat = x_hat(1, :).';
    v1_hat = x_hat(2, :).';
end

```

```

function x_next = predictState1Rc(x_now, current_a, dt, params)
    alpha = exp(-dt / (params.R1 * params.C1));
    soc_next = x_now(1) - params.eta * current_a * dt / (3600 * params.Q_ah);
    v1_next = alpha * x_now(2) + params.R1 * (1 - alpha) * current_a;
    x_next = [soc_next; v1_next];

```

end

```

function voltage_v = measurement1Rc(x_state, current_a, ocv_soc, ocv_v,
↳ params)
    ocv_now = interp1(ocv_soc, ocv_v, x_state(1), "pchip", "extrap");
    voltage_v = ocv_now - current_a * params.R0 - x_state(2);

```

end

```

function slope = ocvSlope(soc, ocv_soc, ocv_v)
    d_ocv = gradient(ocv_v, ocv_soc);
    slope = interp1(ocv_soc, d_ocv, soc, "pchip", "extrap");

```

end

```

function [soc, v1_v, voltage_v] = simulateTruth1Rc( ...
    time_s, current_a, initial_soc, ocv_soc, ocv_v, params)

    n = numel(time_s);
    dt = time_s(2) - time_s(1);
    alpha = exp(-dt / (params.R1 * params.C1));

    soc = zeros(n, 1);
    v1_v = zeros(n, 1);
    voltage_v = zeros(n, 1);

    soc(1) = initial_soc;
    voltage_v(1) = measurement1Rc([soc(1); v1_v(1)], current_a(1), ocv_soc,
↳ ocv_v, params);

    for k = 2:n
        soc(k) = soc(k - 1) - params.eta * current_a(k - 1) * dt / (3600 *
↳ params.Q_ah);
        soc(k) = min(max(soc(k), 0), 1);
        v1_v(k) = alpha * v1_v(k - 1) + params.R1 * (1 - alpha) * current_a(k
↳ - 1);
        voltage_v(k) = measurement1Rc([soc(k); v1_v(k)], current_a(k),
↳ ocv_soc, ocv_v, params);
    end

```

end

end

The `q_soc_candidates` vector sweeps the SOC process-noise variance across four orders of magnitude. This parameter deserves that much attention because it directly controls how willing the filter is to let voltage measurements correct the integrated SOC state. A very small value says the SOC state equation is almost exact. A larger value admits that current integration, model simplification, and capacity uncertainty can create real state drift that the filter should be free to correct.

The `r_candidates_v` vector sweeps the assumed voltage measurement standard deviation from 1 mV to 15 mV. In a synthetic example we know the injected sensor noise, but in real experimental work the effective R often needs to absorb more than sensor electronics alone. It also has to cover unmodeled hysteresis, imperfect OCV tables, temperature mismatch, and structural model error.

The metric maps are deliberately simple: SOC RMSE and convergence time to 1% SOC. Those two together are much more informative than either one alone. An aggressively tuned filter can converge quickly but chatter in steady state. A timid filter can achieve a decent RMSE over a long record while recovering too slowly from the initial error to be useful in practice.

The three named cases are there to turn the sweep into intuition. In the “model overtrusted” case, Q is tiny and R is large. The filter acts stubborn and slow because it trusts current integration and the ECM more than it trusts the voltage measurement. In the “measurement overtrusted” case, Q is relatively large and R is tiny. The filter reacts sharply to voltage residuals, often correcting faster but also becoming noisier and more vulnerable to measurement-model mismatch. The “balanced” case tries to land between those extremes.

7.3.7 Expected Output for Walkthrough 3

The RMSE heat map should show a valley rather than a single magic point. That is a healthy sign. Well-behaved filters often have a region of acceptable tuning rather than a single exact setting. If the map is uniformly terrible, there is likely an implementation bug or the sweep range is wildly inappropriate.

The convergence-time map should show the expected tradeoff: larger process noise and smaller measurement noise usually speed up convergence, but not always gracefully. Some regions may converge quickly at the price of noisier

steady-state behavior.

In the SOC trajectory subplot, the “model overtrusted” case should approach the truth sluggishly and may never fully eliminate the initial offset over the available window. The “measurement overtrusted” case should move toward truth aggressively but may show visible jitter. The balanced case should recover decisively while remaining comparatively smooth.

The innovation subplot should help you read the same story from another angle. In an overtrusted-measurement regime, the innovation is often reduced quickly but the state becomes more reactive. In an overtrusted-model regime, innovations can stay biased for longer because the filter resists correction.

7.3.8 What Could Go Wrong in Walkthrough 3

The heat maps contain NaN almost everywhere. That means the filter rarely reaches the 1% convergence threshold over the chosen window. This can happen if the initial SOC error is too large, the OCV curve is too flat, or the tuning range is too conservative.

The “measurement overtrusted” case looks perfect instead of noisy. In a synthetic problem with a matched model, very aggressive measurement trust can indeed look surprisingly good. That is why this walkthrough is only preparation. On real data in Run 2, the same aggressiveness usually becomes much less attractive.

Changing Q seems to have no effect. This usually means the Q matrix is not actually being passed into the filter function you are calling, or the code is accidentally reusing a stale value.

7.3.9 Reflection on Walkthrough 3

This exercise teaches estimator tuning as an engineering study rather than as folklore. That habit matters when you eventually write a methods section. “We tuned Q and R by trial and error” is weak. “We swept physically plausible ranges, evaluated RMSE and convergence time, and selected a balanced operating point that generalized across held-out tests” is much stronger and much more reproducible.

Guided Walkthrough 4

CALCE Dataset Integration and Real-Data EKF Benchmarking

Learning objective: Build a reusable MATLAB workflow that reads CALCE files, normalizes their sign conventions and column names, constructs an OCV table, and benchmarks the EKF on a real dynamic current profile.

This walkthrough is where the chapter stops being a purely synthetic estimator exercise. We are going to use the CALCE INR 18650-20R battery dataset, which is one of the best public sources for SOC-estimation work because it exposes both OCV tests and dynamic validation profiles for the same cell family. The CALCE battery data portal describes low-current OCV tests, incremental-current OCV tests, and dynamic tests such as DST, FUDS, US06, and BJDST for this cell family, and it links those experiments to the estimator-comparison papers by Xing, He, Pecht, and collaborators. That makes CALCE especially valuable for a research methods manual: it gives us both raw data and a literature context for why those data were collected in the first place.

7.3.10 Dataset overview

Use the CALCE battery data portal:

- Data landing page: <https://calce.umd.edu/data>
- Alternate page with the same INR 18650-20R section:
<https://calce.umd.edu/battery-data>
- Related CALCE publication describing the temperature-dependent SOC-estimation workflow: Xing, He, Pecht, and Tsui, *Applied Energy* 113 (2014), DOI [10.1016/j.apenergy.2013.07.008](https://doi.org/10.1016/j.apenergy.2013.07.008)

At the time of writing, the INR 18650-20R section provides:

- low-current OCV workbooks for 0°C, 25°C, and 45°C
- incremental-current OCV workbooks for 0°C, 25°C, and 45°C
- dynamic validation profiles including DST, FUDS, US06, and BJDST

The CALCE portal serves these as Excel workbooks and associated initial-capacity files. Individual file sizes can change slightly as the site is reorganized, but the OCV and dynamic-test workbooks are typically in the “hundreds of kilobytes to a few megabytes” range. The license on the public CALCE portal is effectively open-access-for-research-use with a citation requirement: CALCE explicitly asks that publications using the data cite the

CALCE article(s) describing the experiments.

Two practical notes matter immediately.

First, CALCE workbook schemas are not always identical across cell families or years. A robust parser must search for likely column names rather than assuming a single perfect header row.

Second, CALCE dynamic files often follow Arbin conventions where discharge current appears negative. In this chapter, our hand-built estimator uses **positive current** = discharge, so we will normalize the sign when reading the data.

7.3.11 What we will do in this walkthrough

We will assume you manually download two files into the chapter workspace:

- one 25°C OCV workbook from the INR 18650-20R section
- one 25°C dynamic-profile workbook, preferably FUDS or DST

Create this folder structure inside `chapter7_soc_estimation_workspace`:

```
chapter7_soc_estimation_workspace/  
├─ data/  
│   └─ calce_inr18650_20r/  
│       ├── ocv_25c.xlsx  
│       └─ fuds_25c.xlsx  
└─ results/
```

The exact filenames on the CALCE portal are sometimes more verbose. Renaming them locally to concise names is a reproducibility aid, not a scientific shortcut. The important thing is to record the original CALCE page and sample number in your research log.

```
clear; close all; clc;  
  
chapterRoot = pwd;  
dataFolder = fullfile(chapterRoot, "data", "calce_inr18650_20r");  
resultsFolder = fullfile(chapterRoot, "results");  
if ~exist(resultsFolder, "dir")  
    mkdir(resultsFolder);  
end  
  
ocvFile = fullfile(dataFolder, "ocv_25c.xlsx");  
dynamicFile = fullfile(dataFolder, "fuds_25c.xlsx");  
  
% Read and normalize the two CALCE workbooks
```

```

ocvTableRow = readCalceWorkbookRobust(ocvFile);
dynamicTableRow = readCalceWorkbookRobust(dynamicFile);

ocvData = normalizeCalceBatteryTable(ocvTableRow);
dynamicData = normalizeCalceBatteryTable(dynamicTableRow);

% Build an OCV table from the low-current OCV file
[ocvSoc, ocvVoltage] = buildOcvTableFromCalce(ocvData);

% Extract the dynamic current-voltage-time series and resample to 1 s
dyn = extractDynamicSeries(dynamicData);
[time_s, current_a, voltage_v] = resampleSeriesToUniformStep( ...
    dyn.time_s, dyn.current_a, dyn.voltage_v, 1.0);

% Normalize current sign to the convention used in this chapter
% CALCE dynamic files often use negative current for discharge.
if mean(current_a(current_a ~= 0)) < 0
    current_a = -current_a;
end

% Use a simple 1RC parameter set consistent with Chapter 6 scale
params.Q_ah = estimateCapacityFromOcvFile(ocvData);
params.eta = 1.0;
params.R0 = 0.014;
params.R1 = 0.018;
params.C1 = 95.0;

% Reference SOC from coulomb counting anchored near the first OCV point
initial_soc_guess = estimateInitialSocFromVoltage(voltage_v(1), ocvSoc,
    ↪ ocvVoltage);
soc_ref = initial_soc_guess - cumsum([0; current_a(1:end-1)]) / (3600 *
    ↪ params.Q_ah);
soc_ref = min max soc_ref, 0 , 1 ;

% EKF run
P0 = diag 0.08^2, 0.03^2 ;
Q = diag 1e-8, 1e-5 ;
R = 7e-3 ^2;

soc_hat, v1_hat, innovation_v = runEkf1Rc ...
    time_s, current_a, voltage_v, ocvSoc, ocvVoltage, params, ...
    initial_soc_guess - 0.10, P0, Q, R ;

predicted_voltage_v = zeros size time_s ;
for k = 1:numel time_s

```

```

        predicted_voltage_v(k) = measurement1Rc([soc_hat(k); v1_hat(k)], ...
            current_a(k), ocvSoc, ocvVoltage, params);
end

voltage_rmse_v = sqrt(mean((predicted_voltage_v - voltage_v).^2));
soc_shift = soc_hat - soc_ref;

fprintf("Estimated cell capacity from OCV workbook: %.4f Ah\n", params.Q_ah);
fprintf("Voltage RMSE on dynamic file: %.4f V\n", voltage_rmse_v);
fprintf("Mean EKF-minus-reference SOC offset: %.4f\n", mean(soc_shift));
fprintf("Maximum absolute EKF-minus-reference SOC offset: %.4f\n",
    ↪ max(abs(soc_shift)));

figure("Color", "w", "Position", [70 70 1200 950]);

subplot(4, 1, 1);
plot(time_s, current_a, "k", "LineWidth", 1.2);
grid on;
xlabel("Time [s]");
ylabel("Current [A]");
title("CALCE dynamic current profile after sign normalization");

subplot(4, 1, 2);
plot(ocvSoc, ocvVoltage, "LineWidth", 2.0);
grid on;
xlabel("SOC [-]");
ylabel("OCV [V]");
title("OCV table constructed from CALCE low-current workbook");

subplot(4, 1, 3);
plot(time_s, voltage_v, "Color", [0.55 0.55 0.55]); hold on;
plot(time_s, predicted_voltage_v, "LineWidth", 1.6);
grid on;
xlabel("Time [s]");
ylabel("Voltage [V]");
title("Measured CALCE voltage and EKF model voltage");
legend("Measured voltage", "EKF model voltage", "Location", "best");

subplot(4, 1, 4);
plot(time_s, soc_ref, "--", "LineWidth", 1.6); hold on;
plot(time_s, soc_hat, "LineWidth", 1.8);
yyaxis right;
plot(time_s, innovation_v * 1000, "LineWidth", 1.0);
ylabel("Innovation [mV]");
yyaxis left;

```



```

grid on;
xlabel("Time [s]");
ylabel("SOC [-]");
title("Reference SOC, EKF SOC, and innovation on real data");
legend("Reference SOC", "EKF SOC", "Innovation", "Location", "best");

writetable(table(time_s, current_a, voltage_v, soc_ref, soc_hat,
    ↪ innovation_v, ...
    'VariableNames', {'time_s', 'current_a', 'voltage_v', 'soc_ref', 'soc_hat',
    ↪ 'innovation_v'}), ...
    fullfile(resultsFolder, "calce_ekf_results.csv"));

function tableOut = readCalceWorkbookRobust(filename)
    sheetNames = sheetnames(filename);
    tableOut = table();

    for s = 1:numel(sheetNames)
        try
            candidate = readtable(filename, "Sheet", sheetNames{s}, ...
                "VariableNamingRule", "preserve");
            if width(candidate) >= 3 && height(candidate) >= 10
                tableOut = candidate;
                return;
            end
        catch
        end
    end

    error("Could not find a usable sheet in %s", filename);
end

function normData = normalizeCalceBatteryTable(rawTable)
    names = string(rawTable.Properties.VariableNames);
    namesLower = lower(strrep(names, " ", ""));

    normData = table();
    normData.time_s = getColumnByPatterns(rawTable, namesLower, ...
        ["testtime(s)", "steptime(s)", "time(s)", "totaltime(s)", "time"]);
    normData.current_a = getColumnByPatterns(rawTable, namesLower, ...
        ["current(a)", "current", "current_a"]);
    normData.voltage_v = getColumnByPatterns(rawTable, namesLower, ...
        ["voltage(v)", "voltage", "voltage_v"]);

```

```

tempCol = getColumnByPatterns(rawTable, namesLower, ...
    ["temperature(c)", "aux_temperature_1(c)", "celltemperature(c)",
↳ "temperature"], false);
if ~isempty(tempCol)
    normData.temperature_c = tempCol;
end

capCol = getColumnByPatterns(rawTable, namesLower, ...
    ["capacity(ah)", "dischargecapacity(ah)", "chargecapacity(ah)",
↳ "capacity"], false);
if ~isempty(capCol)
    normData.capacity_ah = capCol;
end

stepCol = getColumnByPatterns(rawTable, namesLower, ...
    ["stepindex", "step", "stepnumber"], false);
if ~isempty(stepCol)
    normData.step_index = stepCol;
end

modeCol = getColumnByPatterns(rawTable, namesLower, ...
    ["stepmode", "status", "mode"], false);
if ~isempty(modeCol)
    normData.mode = modeCol;
end

validMask = ~(isnan(normData.time_s) | isnan(normData.current_a) |
↳ isnan(normData.voltage_v));
normData = normData(validMask, :);

normData.time_s = normData.time_s - normData.time_s(1);
end

function column = getColumnByPatterns(rawTable, namesLower, patterns,
↳ required)
    if nargin < 4
        required = true;
    end

    column = [];
    for p = 1:numel(patterns)
        idx = find(contains(namesLower, patterns(p)), 1, "first");
        if ~isempty(idx)
            candidate = rawTable.(rawTable.Properties.VariableNames{idx});

```

```

        if isnumeric(candidate)
            column = candidate(:);
        elseif iscell(candidate)
            column = str2double(string(candidate(:)));
        else
            column = str2double(string(candidate(:)));
        end
        return;
    end
end

if required
    error("Could not locate a required column matching patterns: %s",
        ↪ strjoin(patterns, ", "));
end
end

function [soc, ocv] = buildOcvTableFromCalce(ocvData)
    voltage_v = ocvData.voltage_v;

    if ismember("capacity_ah", string(ocvData.Properties.VariableNames))
        capacity_trace_ah = ocvData.capacity_ah;
        capacity_trace_ah = capacity_trace_ah - min(capacity_trace_ah);
        usable_capacity_ah = max(capacity_trace_ah);
        if usable_capacity_ah <= 0
            usable_capacity_ah = 2.0;
        end
        soc = 1 - capacity_trace_ah / usable_capacity_ah;
    else
        current_a = ocvData.current_a;
        time_s = ocvData.time_s;
        dt = median(diff(time_s));
        charge_removed_ah = cumsum([0; current_a(1:end-1)]) * dt / 3600;
        usable_capacity_ah = max(charge_removed_ah) - min(charge_removed_ah);
        if usable_capacity_ah <= 0
            usable_capacity_ah = 2.0;
        end
        soc = 1 - (charge_removed_ah - min(charge_removed_ah)) /
            ↪ usable_capacity_ah;
    end

    soc = min(max(soc, 0), 1);

    % Sort and average duplicate SOC's to build a clean interpolation table

```

```

[socSorted, sortIdx] = sort(soc);
voltageSorted = voltage_v(sortIdx);
[socUnique, ~, groupIdx] = unique(round(socSorted, 4));
ocvUnique = accumarray(groupIdx, voltageSorted, [], @mean);

soc = socUnique;
ocv = ocvUnique;
end

function dyn = extractDynamicSeries(dynamicData)
    dyn.time_s = dynamicData.time_s;
    dyn.current_a = dynamicData.current_a;
    dyn.voltage_v = dynamicData.voltage_v;

    if numel(dyn.time_s) < 10
        error("Dynamic file appears too short to be usable.");
    end
end

function [timeUniform_s, currentUniform_a, voltageUniform_v] = ...
    resampleSeriesToUniformStep(time_s, current_a, voltage_v, dt)

    timeUniform_s = (time_s(1):dt:time_s(end)).';
    currentUniform_a = interp1(time_s, current_a, timeUniform_s, "previous",
        ↪ "extrap");
    voltageUniform_v = interp1(time_s, voltage_v, timeUniform_s, "linear",
        ↪ "extrap");
end

function qAh = estimateCapacityFromOcvFile(ocvData)
    if ismember("capacity_ah", string(ocvData.Properties.VariableNames))
        cap = ocvData.capacity_ah;
        qAh = max(cap) - min(cap);
    else
        qAh = 2.0;
    end

    if ~(isfinite(qAh) && qAh > 0.5)
        qAh = 2.0;
    end
end
end

```

```

function soc0 = estimateInitialSocFromVoltage(voltage0, ocvSoc, ocvVoltage)
    [~, idx] = min(abs(ocvVoltage - voltage0));
    soc0 = ocvSoc(idx);
end

function [soc_hat, v1_hat, innovation_v] = runEkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

    dt = time_s(2) - time_s(1);
    n = numel(time_s);

    x_hat = zeros(2, n);
    x_hat(:, 1) = [initial_soc_guess; 0];
    P = P0;
    innovation_v = zeros(n, 1);
    alpha = exp(-dt / (params.R1 * params.C1));

    for k = 2:n
        x_pred = predictState1Rc(x_hat(:, k - 1), current_a(k - 1), dt,
            ↪ params);
        x_pred(1) = min(max(x_pred(1), 0), 1);

        A = [1.0 0.0; 0.0 alpha];
        P_pred = A * P * A' + Q;

        y_pred = measurement1Rc(x_pred, current_a(k), ocv_soc, ocv_v,
            ↪ params);
        dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
        C = [dUoc_dSoc, -1.0];

        innovation_v(k) = voltage_meas_v(k) - y_pred;
        S = C * P_pred * C' + R;
        K = (P_pred * C') / S;

        x_hat(:, k) = x_pred + K * innovation_v(k);
        x_hat(1, k) = min(max(x_hat(1, k), 0), 1);

        I2 = eye(2);
        P = (I2 - K * C) * P_pred * (I2 - K * C)' + K * R * K';
    end

    soc_hat = x_hat(1, :).';

```

```

    v1_hat = x_hat(2, :).';
end

function x_next = predictState1Rc(x_now, current_a, dt, params)
    alpha = exp(-dt / (params.R1 * params.C1));
    soc_next = x_now(1) - params.eta * current_a * dt / (3600 * params.Q_ah);
    v1_next = alpha * x_now(2) + params.R1 * (1 - alpha) * current_a;
    x_next = [soc_next; v1_next];
end

function voltage_v = measurement1Rc(x_state, current_a, ocv_soc, ocv_v,
    params)
    ocv_now = interp1(ocv_soc, ocv_v, x_state(1), "pchip", "extrap");
    voltage_v = ocv_now - current_a * params.R0 - x_state(2);
end

function slope = ocvSlope(soc, ocv_soc, ocv_v)
    d_ocv = gradient(ocv_v, ocv_soc);
    slope = interp1(ocv_soc, d_ocv, soc, "pchip", "extrap");
end

```

The parser design deserves attention. The helper `readCalceWorkbookRobust` loops through workbook sheets and accepts the first sheet that is large enough to look like data rather than a title sheet. That may feel inelegant, but it is the right kind of inelegance for real research data. Public datasets are rarely as schema-stable as tutorial data.

The function `normalizeCalceBatteryTable` then searches for likely headers such as `Current(A)` or `Voltage(V)` rather than assuming one exact spelling. That is a standard research habit worth internalizing. If you hard-code a single header string into every notebook, you are one workbook revision away from silent failure.

The OCV builder uses the low-current workbook to create a monotonic SOC -> OCV table. If the workbook already contains a capacity column, we use it directly. If not, we fall back to integrating current over time. The grouping on rounded SOC values is not mathematically sacred. It is a practical deduplication step so that interpolation later does not choke on repeated x-values.

The “reference SOC” in this example is not ground truth. It is coulomb counting anchored by the initial OCV estimate. That distinction is crucial. On

synthetic data we had true SOC. On real data we usually do not. So here we benchmark the EKF against three things instead: voltage fit quality, consistency with a capacity-constrained coulomb-counting trace, and innovation behavior. That is honest practice.

7.3.12 Expected Output for Walkthrough 4

The first subplot should show a real dynamic current profile with frequent sign changes and amplitude changes, not the idealized rectangular pulses of the synthetic examples. If you downloaded FUDS, the current should have a stop-and-go character with many moderate excursions. If you downloaded DST, the pattern should be similarly dynamic but with a different cadence.

The second subplot should show a smooth OCV curve at 25 °C derived from the low-current workbook. It should rise monotonically from the low-SOC voltage region to the fully charged region. If it looks jagged, folded, or strongly non-monotonic, your preprocessing is not yet trustworthy.

The third subplot should show the measured dynamic voltage in gray and the EKF model voltage overlaid on it. A correct result will not be perfect, because the IRC parameter set here is deliberately generic rather than re-identified to the exact workbook, but it should follow the dominant dynamic structure and give a reasonable voltage RMSE.

The final subplot should show the reference SOC and EKF SOC in the left axis and the innovation in millivolts on the right axis. On real data, innovations are usually larger and less perfectly centered than in synthetic studies. That is normal. What you are looking for is a residual that is bounded, not one that drifts in one direction for the entire test.

7.3.13 What Could Go Wrong in Walkthrough 4

sheetnames works, but **readtable** returns empty columns or text everywhere. CALCE workbooks occasionally contain title rows or merged-cell formatting above the actual data region. Open the workbook once manually and confirm which sheet contains the actual numeric table.

The dynamic voltage is obviously upside down relative to current. You likely forgot to normalize the current sign convention. CALCE files often represent discharge current as negative.

The OCV table folds back on itself. That usually means you used a dynamic

file rather than a low-current or incremental-OCV file to build the OCV map, or the SOC direction must be reversed before interpolation.

The EKF tracks reference SOC poorly even though voltage fit looks decent. That can happen when the OCV table is weakly informative in the active SOC range or when the 1RC parameters are only approximate. It is not automatically a coding error.

7.3.14 Reflection on Walkthrough 4

This walkthrough teaches the data-engineering half of estimator research. Writing a Kalman loop is only half the craft. The other half is turning a public dataset with idiosyncratic formatting into a physically consistent input stream without silently breaking time, sign, units, or the OCV map. That is the work that makes later benchmarking credible.

Guided Walkthrough 5

Implement a UKF and Compare It Against the EKF on the Same Dynamic Test

Learning objective: Learn what the UKF changes in practice by comparing it head-to-head with the EKF on the same nonlinear OCV measurement model.

The UKF is often introduced as “the nonlinear Kalman filter that avoids Jacobians.” That description is true but too shallow to be useful. The important question is not whether it uses Jacobians. The important question is whether its nonlinear propagation buys you meaningfully better battery-state estimates for the model mismatch and OCV curvature you actually have. In many battery papers the UKF outperforms the EKF modestly, not dramatically. That is a healthy expectation to carry into your own work.

We will use the same 1RC process and measurement model as before so that the comparison isolates the filter architecture rather than changing the battery physics underneath us.

```
clear; close all; clc;

% Shared dataset: still synthetic so we know the hidden truth exactly
dt = 1;
time_s = (0:1:2200).';
```



```

current_a = zeros(size(time_s));
current_a(time_s >= 80 & time_s < 420) = 2.2;
current_a(time_s >= 600 & time_s < 930) = 3.0;
current_a(time_s >= 1120 & time_s < 1320) = -1.0;
current_a(time_s >= 1490 & time_s < 1810) = 2.7;
current_a(time_s >= 1930 & time_s < 2160) = 1.4;

ocv_soc = linspace(0, 1, 201).';
ocv_v = 3.00 ...
    + 0.74 * ocv_soc ...
    + 0.17 * tanh((ocv_soc - 0.12) / 0.05) ...
    + 0.15 * tanh((ocv_soc - 0.88) / 0.04);

params.Q_ah = 2.3;
params.eta = 1.0;
params.R0 = 0.014;
params.R1 = 0.018;
params.C1 = 95.0;

[soc_true, ~, voltage_true_v] = simulateTruth1Rc( ...
    time_s, current_a, 0.86, ocv_soc, ocv_v, params);

rng(31);
voltage_meas_v = voltage_true_v + 5e-3 * randn(size(voltage_true_v));

P0 = diag([0.08^2, 0.03^2]);
Q = diag([1e-8, 1e-5]);
R = (5e-3)^2;
initial_soc_guess = 0.68;

[soc_ekf, v1_ekf, innovation_ekf] = runEkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R);

[soc_ukf, v1_ukf, innovation_ukf] = runUkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R);

rmse_ekf = sqrt(mean((soc_ekf - soc_true).^2));
rmse_ukf = sqrt(mean((soc_ukf - soc_true).^2));

fprintf("EKF SOC RMSE: %.4f\n", rmse_ekf);
fprintf("UKF SOC RMSE: %.4f\n", rmse_ukf);
fprintf("Relative UKF improvement over EKF: %.2f %%\n", ...
    100 * (rmse_ekf - rmse_ukf) / rmse_ekf);

```

```

figure("Color", "w", "Position", [60 60 1200 900]);

subplot(4, 1, 1);
plot(time_s, current_a, "k", "LineWidth", 1.4);
grid on;
xlabel("Time [s]");
ylabel("Current [A]");
title("Shared dynamic profile for EKF-UKF comparison");

subplot(4, 1, 2);
plot(time_s, soc_true, "k", "LineWidth", 2.0); hold on;
plot(time_s, soc_ekf, "LineWidth", 1.7);
plot(time_s, soc_ukf, "LineWidth", 1.7);
grid on;
xlabel("Time [s]");
ylabel("SOC [-]");
title("State estimates");
legend("True SOC", "EKF", "UKF", "Location", "best");

subplot(4, 1, 3);
plot(time_s, (soc_ekf - soc_true) * 100, "LineWidth", 1.5); hold on;
plot(time_s, (soc_ukf - soc_true) * 100, "LineWidth", 1.5);
grid on;
xlabel("Time [s]");
ylabel("SOC error [%]");
title("Estimation error trajectories");
legend("EKF error", "UKF error", "Location", "best");

subplot(4, 1, 4);
plot(time_s, innovation_ekf * 1000, "LineWidth", 1.3); hold on;
plot(time_s, innovation_ukf * 1000, "LineWidth", 1.3);
grid on;
xlabel("Time [s]");
ylabel("Innovation [mV]");
title("Innovation comparison");
legend("EKF innovation", "UKF innovation", "Location", "best");

function [soc_hat, vl_hat, innovation_v] = runUkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

    n = numel(time_s);
    dt = time_s(2) - time_s(1);

```

```

nx = 2;

alpha_ukf = 1e-3;
beta_ukf = 2.0;
kappa_ukf = 0.0;
lambda = alpha_ukf^2 * (nx + kappa_ukf) - nx;
gamma = sqrt(nx + lambda);

Wm = [lambda / (nx + lambda), repmat(1 / (2 * (nx + lambda)), 1, 2 *
    ↪ nx)];
Wc = Wm;
Wc 1 = Wc 1 + 1 - alpha_ukf^2 + beta_ukf ;

x_hat = zeros nx, n ;
x_hat :, 1 = initial_soc_guess; 0 ;
P = P0;
innovation_v = zeros n, 1 ;

for k = 2:n
    sigmaPoints = computeSigmaPoints x_hat :, k - 1 , P, gamma ;

    sigmaPred = zeros nx, 2 * nx + 1 ;
    for i = 1: 2 * nx + 1
        sigmaPred(:, i) = predictState1Rc(sigmaPoints(:, i), current_a(k)
            ↪ - 1), dt, params);
        sigmaPred 1, i = min max sigmaPred 1, i , 0 , 1 ;
    end

    x_pred = sigmaPred * Wm.';
    P_pred = Q;
    for i = 1: 2 * nx + 1
        dx = sigmaPred :, i - x_pred;
        P_pred = P_pred + Wc i * dx * dx.' ;
    end

    ySigma = zeros 1, 2 * nx + 1 ;
    for i = 1: 2 * nx + 1
        ySigma(i) = measurement1Rc(sigmaPred(:, i), current_a(k),
            ↪ ocv_soc, ocv_v, params);
    end

    y_pred = ySigma * Wm.';
    Pyy = R;
    Pxy = zeros nx, 1 ;
    for i = 1: 2 * nx + 1

```

```

        dx = sigmaPred(:, i) - x_pred;
        dy = ySigma(i) - y_pred;
        Pyy = Pyy + Wc(i) * (dy * dy. ');
        Pxy = Pxy + Wc(i) * dx * dy. ';
    end

    K = Pxy / Pyy;
    innovation_v(k) = voltage_meas_v(k) - y_pred;

    x_hat(:, k) = x_pred + K * innovation_v(k);
    x_hat(1, k) = min(max(x_hat(1, k), 0), 1);
    P = P_pred - K * Pyy * K. ';
end

soc_hat = x_hat(1, :). ';
v1_hat = x_hat(2, :). ';
end

function sigmaPoints = computeSigmaPoints(x, P, gamma)
    nx = numel(x);
    jitter = 1e-12 * eye(nx);
    S = chol(P + jitter, "lower");
    sigmaPoints = zeros(nx, 2 * nx + 1);
    sigmaPoints(:, 1) = x;

    for i = 1:nx
        sigmaPoints(:, i + 1) = x + gamma * S(:, i);
        sigmaPoints(:, i + 1 + nx) = x - gamma * S(:, i);
    end
end

function [soc_hat, v1_hat, innovation_v] = runEkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

    dt = time_s(2) - time_s(1);
    n = numel(time_s);
    x_hat = zeros(2, n);
    x_hat(:, 1) = [initial_soc_guess; 0];
    P = P0;
    innovation_v = zeros(n, 1);
    alpha = exp(-dt / (params.R1 * params.C1));

```

```

for k = 2:n
    x_pred = predictState1Rc(x_hat(:, k - 1), current_a(k - 1), dt,
        ↪ params);
    x_pred(1) = min(max(x_pred(1), 0), 1);

    A = [1.0 0.0; 0.0 alpha];
    P_pred = A * P * A' + Q;

    y_pred = measurement1Rc(x_pred, current_a(k), ocv_soc, ocv_v,
        ↪ params);
    dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
    C = [dUoc_dSoc, -1.0];

    innovation_v(k) = voltage_meas_v(k) - y_pred;
    S = C * P_pred * C' + R;
    K = (P_pred * C') / S;

    x_hat(:, k) = x_pred + K * innovation_v(k);
    x_hat(1, k) = min(max(x_hat(1, k), 0), 1);

    I2 = eye(2);
    P = (I2 - K * C) * P_pred * (I2 - K * C)' + K * R * K';
end

soc_hat = x_hat(1, :).';
v1_hat = x_hat(2, :).';
end

function x_next = predictState1Rc(x_now, current_a, dt, params)
    alpha = exp(-dt / (params.R1 * params.C1));
    soc_next = x_now(1) - params.eta * current_a * dt / (3600 * params.Q_ah);
    v1_next = alpha * x_now(2) + params.R1 * (1 - alpha) * current_a;
    x_next = [soc_next; v1_next];
end

function voltage_v = measurement1Rc(x_state, current_a, ocv_soc, ocv_v,
    ↪ params)
    ocv_now = interp1(ocv_soc, ocv_v, x_state(1), "pchip", "extrap");
    voltage_v = ocv_now - current_a * params.R0 - x_state(2);
end

function slope = ocvSlope(soc, ocv_soc, ocv_v)

```

```

    d_ocv = gradient(ocv_v, ocv_soc);
    slope = interp1(ocv_soc, d_ocv, soc, "pchip", "extrap");
end

function [soc, v1_v, voltage_v] = simulateTruth1Rc( ...
    time_s, current_a, initial_soc, ocv_soc, ocv_v, params)

    n = numel(time_s);
    dt = time_s(2) - time_s(1);
    alpha = exp(-dt / (params.R1 * params.C1));
    soc = zeros(n, 1);
    v1_v = zeros(n, 1);
    voltage_v = zeros(n, 1);

    soc(1) = initial_soc;
    voltage_v(1) = measurement1Rc([soc(1); v1_v(1)], current_a(1), ocv_soc,
        ↪ ocv_v, params);

    for k = 2:n
        soc(k) = soc(k - 1) - params.eta * current_a(k - 1) * dt / (3600 *
            ↪ params.Q_ah);
        soc(k) = min(max(soc(k), 0), 1);
        v1_v(k) = alpha * v1_v(k - 1) + params.R1 * (1 - alpha) * current_a(k
            ↪ - 1);
        voltage_v(k) = measurement1Rc([soc(k); v1_v(k)], current_a(k),
            ↪ ocv_soc, ocv_v, params);
    end
end

```

The UKF implementation uses the standard unscented-transform structure: generate sigma points from the current mean and covariance, propagate each point through the nonlinear process model, propagate the predicted sigma points through the nonlinear measurement model, and reconstruct the predicted mean and covariance from the weighted cloud.

Why is this potentially useful for a battery estimator? Because the OCV measurement equation is nonlinear, and the EKF represents that nonlinearity only through a local first-order linearization. When the OCV curve has stronger curvature, or when the prior covariance is large enough that a first-order local approximation is crude, the UKF can capture the curvature more faithfully.

At the same time, this code also shows why the UKF is not magic. The process

model is still the same 1RC ECM. The OCV table is still the same table. If those are wrong, the UKF cannot rescue them. It can only propagate nonlinear uncertainty a bit more accurately.

The UKF parameters `alpha_ukf`, `beta_ukf`, and `kappa_ukf` are standard unscented-transform tuning constants. The values here are common defaults for small state dimensions: `alpha = 1e-3`, `beta = 2`, `kappa = 0`. Those defaults are not the only valid choice, but they are a reasonable place to start and are widely used in practice.

7.3.15 Expected Output for Walkthrough 5

The SOC-estimate subplot should show that both filters recover from the wrong initial SOC and track the true state closely after convergence. In a matched synthetic problem like this one, the UKF may improve the RMSE modestly, but it usually does not revolutionize the result. That is the normal outcome. If the UKF massively outperforms the EKF on this matched problem, inspect the EKF implementation before assuming the UKF is simply better.

The error subplot should make the comparison easier to read. Both error traces should begin far from zero because of the initial SOC miss. After convergence, both should hover near zero, with the UKF sometimes slightly smoother or slightly lower in magnitude depending on where the OCV nonlinearity is strongest.

The innovation subplot should show two bounded traces with similar structure. If the UKF innovation is not meaningfully different from the EKF innovation on this problem, that is not a failure. It means the local linearization was already adequate for the chosen operating regime.

7.3.16 What Could Go Wrong in Walkthrough 5

chol fails inside sigma-point generation. That usually means the covariance matrix has lost positive definiteness due to numerical roundoff. The small `jitter` term in `computeSigmaPoints` is there precisely to make the routine more robust.

The UKF is much noisier than the EKF. Check the sigma-point weights and confirm that `Pyy` includes the measurement-noise term `R`.

The UKF does worse than the EKF on every run. That can happen if the unscented parameters are mis-implemented or if covariance propagation is wrong, but it can also happen on simple matched problems where the UKF brings little benefit and numerical details dominate. The point of the exercise is comparison,

not guaranteed superiority.

7.3.17 Reflection on Walkthrough 5

This walkthrough teaches the right mental model for the UKF. It is not “the better EKF.” It is a different approximation strategy for nonlinear state estimation.

Sometimes that difference matters. Sometimes it barely matters. Publishable work requires knowing which regime you are in.

Guided Walkthrough 6

Handle Voltage Bias and Current-Sensor Drift with an Augmented-State EKF

Learning objective: Extend the estimator state to include a sensor-bias term and see how even a small current or voltage offset can quietly corrupt SOC if you do not model it.

By this point, the filter may feel reliable. That is exactly when it becomes dangerous. In real battery-management systems, one of the easiest ways to corrupt SOC over long windows is not a dramatic model failure. It is a tiny sensor bias. A current offset of just a few tens of milliamps can accumulate into a meaningful coulomb-counting error. A voltage bias of a few millivolts can tilt every correction step. If your estimator does not explicitly allow for those biases, it may compensate by distorting SOC or polarization states instead.

We will augment the 1RC state with one additional state representing voltage-sensor bias:

$$\mathbf{x}_k = \begin{bmatrix} z_k \\ v_{1,k} \\ b_{v,k} \end{bmatrix}, \quad (13)$$

with a random-walk bias model

$$b_{v,k+1} = b_{v,k} + w_{b,k}. \quad (14)$$

The measurement equation becomes

$$y_k = U_{oc}(z_k) - R_0 I_k - v_{1,k} + b_{v,k} + n_k. \quad (15)$$

This is a small extension in code but a very important extension in research thinking.

```
clear; close all; clc;

dt = 1;
time_s = 0:1:2600 .';
current_a = zeros size time_s ;
current_a time_s >= 60 & time_s < 460 = 2.2;
current_a time_s >= 640 & time_s < 980 = 3.4;
current_a time_s >= 1180 & time_s < 1390 = -1.2;
current_a time_s >= 1600 & time_s < 2010 = 2.5;
current_a time_s >= 2190 & time_s < 2490 = 1.6;

ocv_soc = linspace 0, 1, 201 .';
ocv_v = 3.00 ...
    + 0.74 * ocv_soc ...
    + 0.16 * tanh (ocv_soc - 0.12) / 0.05 ...
    + 0.18 * tanh (ocv_soc - 0.87) / 0.05 ;

params Q_ah = 2.3;
params eta = 1.0;
params R0 = 0.014;
params R1 = 0.018;
params C1 = 95.0;

soc_true, v1_true, voltage_true_v = simulateTruth1Rc ...
    time_s, current_a, 0.87, ocv_soc, ocv_v, params ;

rng 44 ;
true_voltage_bias_v = 8e-3;
voltage_meas_v = voltage_true_v + true_voltage_bias_v + 4e-3 *
    randn size voltage_true_v ;

% Standard EKF that ignores the bias
[soc_plain, ~, innovation_plain] = runEkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    0.70, diag([0.08^2, 0.03^2]), diag([1e-8, 1e-5]), (4e-3)^2);

% Augmented EKF that estimates voltage bias
[soc_bias, v1_bias, bias_hat, innovation_bias] = runEkf1RcWithVoltageBias(
    ↪ ...
```

```

time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
0.70, diag([0.08^2, 0.03^2, (20e-3)^2]), ...
diag([1e-8, 1e-5, 1e-9]), (4e-3)^2);

rmse_plain = sqrt(mean((soc_plain - soc_true).^2));
rmse_bias = sqrt(mean((soc_bias - soc_true).^2));

fprintf("Plain EKF SOC RMSE: %.4f\n", rmse_plain);
fprintf("Bias-aware EKF SOC RMSE: %.4f\n", rmse_bias);
fprintf("Final estimated voltage bias: %.5f V\n", bias_hat(end));
fprintf("True voltage bias: %.5f V\n", true_voltage_bias_v);

figure("Color", "w", "Position", [60 60 1200 950]);

subplot(4, 1, 1);
plot(time_s, voltage_meas_v, "Color", [0.6 0.6 0.6]); hold on;
plot(time_s, voltage_true_v, "k--", "LineWidth", 1.3);
grid on;
xlabel("Time [s]");
ylabel("Voltage [V]");
title("Measured voltage includes an unknown 8 mV bias");
legend("Measured voltage", "True voltage", "Location", "best");

subplot(4, 1, 2);
plot(time_s, soc_true, "k", "LineWidth", 2.0); hold on;
plot(time_s, soc_plain, "LineWidth", 1.7);
plot(time_s, soc_bias, "LineWidth", 1.7);
grid on;
xlabel("Time [s]");
ylabel("SOC [-]");
title("SOC estimates with and without an explicit bias state");
legend("True SOC", "Plain EKF", "Bias-aware EKF", "Location", "best");

subplot(4, 1, 3);
plot(time_s, bias_hat * 1000, "LineWidth", 1.8); hold on;
yline(true_voltage_bias_v * 1000, "--", "LineWidth", 1.5);
grid on;
xlabel("Time [s]");
ylabel("Voltage bias [mV]");
title("Recovered bias state");
legend("Estimated bias", "True bias", "Location", "best");

subplot(4, 1, 4);
plot(time_s, innovation_plain * 1000, "LineWidth", 1.3); hold on;
plot(time_s, innovation_bias * 1000, "LineWidth", 1.3);

```

```

grid on;
xlabel("Time [s]");
ylabel("Innovation [mV]");
title("Innovation comparison");
legend("Plain EKF", "Bias-aware EKF", "Location", "best");

function [soc_hat, v1_hat, bias_hat, innovation_v] =
↳ runEkf1RcWithVoltageBias( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

    dt = time_s(2) - time_s(1);
    n = numel(time_s);
    x_hat = zeros(3, n);
    x_hat(:, 1) = [initial_soc_guess; 0; 0];
    P = P0;
    innovation_v = zeros(n, 1);
    alpha = exp(-dt / (params.R1 * params.C1));

    for k = 2:n
        x_pred = predictState1RcWithBias(x_hat(:, k - 1), current_a(k - 1),
↳         dt, params);
        x_pred(1) = min(max(x_pred(1), 0), 1);

        A = [1.0 0.0 0.0; 0.0 alpha 0.0; 0.0 0.0 1.0];
        P_pred = A * P * A' + Q;

        y_pred = measurement1RcWithBias(x_pred, current_a(k), ocv_soc, ocv_v,
↳         params);
        dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
        C = [dUoc_dSoc, -1.0, 1.0];

        innovation_v(k) = voltage_meas_v(k) - y_pred;
        S = C * P_pred * C' + R;
        K = (P_pred * C') / S;

        x_hat(:, k) = x_pred + K * innovation_v(k);
        x_hat(1, k) = min(max(x_hat(1, k), 0), 1);

        I3 = eye(3);
        P = (I3 - K * C) * P_pred * (I3 - K * C)' + K * R * K';
    end

    soc_hat = x_hat(1, :).';

```

```

    v1_hat = x_hat(2, :).';
    bias_hat = x_hat(3, :).';
end

function x_next = predictState1RcWithBias(x_now, current_a, dt, params)
    alpha = exp(-dt / (params.R1 * params.C1));
    soc_next = x_now(1) - params.eta * current_a * dt / (3600 * params.Q_ah);
    v1_next = alpha * x_now(2) + params.R1 * (1 - alpha) * current_a;
    bias_next = x_now(3);
    x_next = [soc_next; v1_next; bias_next];
end

function voltage_v = measurement1RcWithBias(x_state, current_a, ocv_soc,
    ↪ ocv_v, params)
    ocv_now = interp1(ocv_soc, ocv_v, x_state(1), "pchip", "extrap");
    voltage_v = ocv_now - current_a * params.R0 - x_state(2) + x_state(3);
end

function [soc_hat, v1_hat, innovation_v] = runEkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

    dt = time_s(2) - time_s(1);
    n = numel(time_s);
    x_hat = zeros(2, n);
    x_hat(:, 1) = [initial_soc_guess; 0];
    P = P0;
    innovation_v = zeros(n, 1);
    alpha = exp(-dt / (params.R1 * params.C1));

    for k = 2:n
        x_pred = predictState1Rc(x_hat(:, k - 1), current_a(k - 1), dt,
            ↪ params);
        x_pred(1) = min(max(x_pred(1), 0), 1);

        A = [1.0 0.0; 0.0 alpha];
        P_pred = A * P * A' + Q;

        y_pred = measurement1Rc(x_pred, current_a(k), ocv_soc, ocv_v,
            ↪ params);
        dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
        C = [dUoc_dSoc, -1.0];
    end
end

```

```

        innovation_v(k) = voltage_meas_v(k) - y_pred;
        S = C * P_pred * C' + R;
        K = (P_pred * C') / S;

        x_hat(:, k) = x_pred + K * innovation_v(k);
        x_hat(1, k) = min(max(x_hat(1, k), 0), 1);

        I2 = eye(2);
        P = (I2 - K * C) * P_pred * (I2 - K * C)' + K * R * K';
    end

    soc_hat = x_hat(1, :).';
    vl_hat = x_hat(2, :).';
end

function x_next = predictState1Rc(x_now, current_a, dt, params)
    alpha = exp(-dt / (params.R1 * params.C1));
    soc_next = x_now(1) - params.eta * current_a * dt / (3600 * params.Q_ah);
    vl_next = alpha * x_now(2) + params.R1 * (1 - alpha) * current_a;
    x_next = [soc_next; vl_next];
end

function voltage_v = measurement1Rc(x_state, current_a, ocv_soc, ocv_v,
    ↪ params)
    ocv_now = interp1(ocv_soc, ocv_v, x_state(1), "pchip", "extrap");
    voltage_v = ocv_now - current_a * params.R0 - x_state(2);
end

function slope = ocvSlope(soc, ocv_soc, ocv_v)
    d_ocv = gradient(ocv_v, ocv_soc);
    slope = interp1(ocv_soc, d_ocv, soc, "pchip", "extrap");
end

function [soc, vl_v, voltage_v] = simulateTruth1Rc( ...
    time_s, current_a, initial_soc, ocv_soc, ocv_v, params)

    n = numel(time_s);
    dt = time_s(2) - time_s(1);
    alpha = exp(-dt / (params.R1 * params.C1));
    soc = zeros(n, 1);

```

```

v1_v = zeros(n, 1);
voltage_v = zeros(n, 1);

soc(1) = initial_soc;
voltage_v(1) = measurement1Rc([soc(1); v1_v(1)], current_a(1), ocv_soc,
    ↪ ocv_v, params);

for k = 2:n
    soc(k) = soc(k - 1) - params.eta * current_a(k - 1) * dt / (3600 *
        ↪ params.Q_ah);
    soc(k) = min(max(soc(k), 0), 1);
    v1_v(k) = alpha * v1_v(k - 1) + params.R1 * (1 - alpha) * current_a(k
        ↪ - 1);
    voltage_v(k) = measurement1Rc([soc(k); v1_v(k)], current_a(k),
        ↪ ocv_soc, ocv_v, params);
end
end

```

The plain EKF, which does not know about the bias, often reacts by carrying a persistent SOC offset so that the measurement equation can absorb the biased voltage. This is one of the most important “looks fine until you inspect it carefully” failure modes in battery estimation.

The augmented filter adds only one state and one extra covariance entry, but that addition changes the estimator’s interpretation of the same innovation. Instead of forcing the SOC state to explain every persistent residual, it can attribute part of that residual to a slowly varying sensor bias. That is a much more physically honest model when such a bias really exists.

The bias process-noise variance in Q is set very small. That is intentional. A sensor bias typically changes slowly or is approximately constant over the duration of a single experiment. Setting the bias process noise too large tells the filter to let the bias wander quickly, which can make the estimator absorb ordinary voltage noise as fake bias motion.

7.3.18 Expected Output for Walkthrough 6

The second subplot should show the plain EKF and the bias-aware EKF starting from the same bad initial SOC but ending in different places. The plain EKF often converges to a trajectory that looks plausible yet remains visibly offset from the true SOC because it has no way to represent the persistent voltage offset honestly.

The bias-aware EKF should track the true SOC much more closely once it has identified the bias state.

The bias plot should rise from zero toward approximately 8 mV, perhaps with a slightly noisy transient. A correct implementation does not need to recover the bias instantly. It needs to move toward the right magnitude and relieve the SOC state from carrying the whole error.

7.3.19 What Could Go Wrong in Walkthrough 6

The estimated bias diverges instead of settling. The most likely causes are an excessively large bias process-noise variance or a sign error in the measurement equation.

The bias state stays near zero forever. That often means the initial covariance on the bias state is too small, so the filter never gives itself permission to learn it.

Both filters give nearly identical SOC. If the injected bias is too small relative to the voltage noise or the OCV slope is too steep, the plain EKF can absorb the error surprisingly well. Increase the bias slightly if you want the difference to be more visible for teaching.

7.3.20 Reflection on Walkthrough 6

This exercise teaches a research habit that reviewers appreciate immediately: if you know a realistic nuisance source exists, put it in the state or measurement model rather than pretending it does not. That move often matters more than switching from EKF to UKF.

Reproduction Exercise

Recreate an EKF Comparison Figure and Then Stress It Under High Measurement Noise

Part III required a reproduce-a-published-figure exercise, and this chapter is one of the highest-value places to do it because state-estimation papers are full of algorithm claims that only make sense when you can rerun and interrogate them.

We will do this reproduction in two layers.

First, complete the EKF assignment from Gregory Plett's Coursera Course 3, *Battery State-of-Charge (SOC) Estimation*. That assignment is still one of the

cleanest ways to internalize the predictor-corrector logic in a battery-specific setting. Treat that as warm-up reproduction: your goal is not novelty but fluency.

Second, reproduce the qualitative comparison setup from ElMenshawy, Massoud, and Guglielmi, “State-of-Charge Estimation Using Triple Forgetting Factor Adaptive Extended Kalman Filter for Battery Energy Storage Systems in Electric Bus Applications,” *IEEE Transactions on Transportation Electrification*, 11(2), 2025, DOI 10.1109/TTE.2024.3514704. An open-access version is indexed through the Politecnico di Torino repository, and public metadata show that the paper compares a conventional adaptive EKF, a dual-forgetting-factor adaptive EKF, and a triple-forgetting-factor adaptive EKF under low- and high-measurement-noise conditions, reporting RMSE, MAE, MaxAE, and convergence behavior.

We are not going to pretend we can reproduce the paper bit-for-bit without the authors’ exact ECM parameter-identification and bus-drive-cycle preprocessing files. That would be false precision. What we *can* reproduce faithfully is the core claim structure:

1. estimator performance must be compared under both low-noise and high-noise cases,
2. convergence time matters alongside RMSE,
3. adaptive covariance logic becomes most valuable when measurement noise is elevated.

7.3.21 Reproduction target

Our concrete target is a two-panel figure showing SOC-estimation trajectories under low and high measurement noise for three filters:

- a baseline EKF
- a simple adaptive EKF with innovation-based R inflation
- a UKF used here as a nonlinear comparison baseline

This is not an exact algorithmic replica of the triple-forgetting-factor paper. It is an educational reproduction of the paper’s comparison logic and figure style, with full disclosure that we are using a simplified adaptive strategy because the published paper’s complete implementation details are not all public in machine-runnable form.

That distinction matters. This is one of several valid reproduction choices, and

we are making it openly because reproducible pedagogy is more valuable than pretending the paper was fully specified when it was not.

```
clear; close all; clc;

dt = 1;
time_s = (0:1:2400).';
current_a = zeros(size(time_s));
current_a(time_s >= 90 & time_s < 420) = 2.5;
current_a(time_s >= 560 & time_s < 930) = 3.0;
current_a(time_s >= 1120 & time_s < 1320) = -1.1;
current_a(time_s >= 1490 & time_s < 1910) = 2.7;
current_a(time_s >= 2050 & time_s < 2320) = 1.7;

ocv_soc = linspace(0, 1, 201).';
ocv_v = 3.00 ...
    + 0.74 * ocv_soc ...
    + 0.16 * tanh((ocv_soc - 0.12) / 0.05) ...
    + 0.18 * tanh((ocv_soc - 0.87) / 0.05);

params.Q_ah = 2.3;
params.eta = 1.0;
params.R0 = 0.014;
params.R1 = 0.018;
params.C1 = 95.0;

[soc_true, ~, voltage_true_v] = simulateTruth1Rc( ...
    time_s, current_a, 0.85, ocv_soc, ocv_v, params);

noiseStdCases = [3e-3, 15e-3];
caseLabels = ["Low measurement noise", "High measurement noise"];

figure("Color", "w", "Position", [80 80 1250 880]);

for c = 1:2
    rng(100 + c);
    voltage_meas_v = voltage_true_v + noiseStdCases(c) *
        ↪ randn(size(voltage_true_v));

    soc_ekf, ~, ~ = runEkf1Rc ...
        time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
        0.67, diag([0.08^2, 0.03^2]), diag([1e-8, 1e-5]),
        ↪ noiseStdCases(c)^2);

    soc_aekf, ~, ~ = runAdaptiveEkf1Rc ...
```

```

        time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
        0.67, diag([0.08^2, 0.03^2]), diag([1e-8, 1e-5]),
        ↪ noiseStdCases(c)^2);

[soc_ukf, ~, ~] = runUkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    0.67, diag([0.08^2, 0.03^2]), diag([1e-8, 1e-5]),
    ↪ noiseStdCases(c)^2);

rmse_ekf = sqrt(mean((soc_ekf - soc_true).^2));
rmse_aekf = sqrt(mean((soc_aekf - soc_true).^2));
rmse_ukf = sqrt(mean((soc_ukf - soc_true).^2));

subplot(2, 1, c);
plot(time_s, soc_true, "k", "LineWidth", 2.1); hold on;
plot(time_s, soc_ekf, "LineWidth", 1.5);
plot(time_s, soc_aekf, "LineWidth", 1.5);
plot(time_s, soc_ukf, "LineWidth", 1.5);
grid on;
xlabel("Time [s]");
ylabel("SOC [-]");
title(sprintf("%s | RMSE EKF = %.4f, AEKF = %.4f, UKF = %.4f", ...
    caseLabels(c), rmse_ekf, rmse_aekf, rmse_ukf));
legend("True SOC", "EKF", "Innovation-adaptive EKF", "UKF", "Location",
    ↪ "best");
end

function [soc_hat, vl_hat, innovation_v] = runAdaptiveEkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R0)

dt = time_s(2) - time_s(1);
n = numel(time_s);
x_hat = zeros(2, n);
x_hat(:, 1) = [initial_soc_guess; 0];
P = P0;
innovation_v = zeros(n, 1);
alpha = exp(-dt / (params.R1 * params.C1));
Rk = R0;
lambda = 0.98;

for k = 2:n
    x_pred = predictState1Rc(x_hat(:, k - 1), current_a(k - 1), dt,
        ↪ params);

```

```

x_pred(1) = min(max(x_pred(1), 0), 1);

A = [1.0 0.0; 0.0 alpha];
P_pred = A * P * A' + Q;

y_pred = measurement1Rc(x_pred, current_a(k), ocv_soc, ocv_v,
    ↪ params);
dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
C = [dUoc_dSoc, -1.0];

innovation_v(k) = voltage_meas_v(k) - y_pred;
Rk = lambda * Rk + (1 - lambda) * innovation_v(k)^2;
S = C * P_pred * C' + Rk;
K = (P_pred * C') / S;

x_hat(:, k) = x_pred + K * innovation_v(k);
x_hat(1, k) = min(max(x_hat(1, k), 0), 1);

I2 = eye(2);
P = (I2 - K * C) * P_pred * (I2 - K * C)' + K * Rk * K';
end

soc_hat = x_hat(1, :).';
v1_hat = x_hat(2, :).';
end

function [soc_hat, v1_hat, innovation_v] = runUkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

n = numel(time_s);
dt = time_s(2) - time_s(1);
nx = 2;
alpha_ukf = 1e-3;
beta_ukf = 2.0;
kappa_ukf = 0.0;
lambda = alpha_ukf^2 * (nx + kappa_ukf) - nx;
gamma = sqrt(nx + lambda);

Wm = [lambda / (nx + lambda), repmat(1 / (2 * (nx + lambda)), 1, 2 *
    ↪ nx)];
Wc = Wm;
Wc(1) = Wc(1) + (1 - alpha_ukf^2 + beta_ukf);

```

```

x_hat = zeros(nx, n);
x_hat(:, 1) = [initial_soc_guess; 0];
P = P0;
innovation_v = zeros(n, 1);

for k = 2:n
    sigmaPoints = computeSigmaPoints(x_hat(:, k - 1), P, gamma);
    sigmaPred = zeros(nx, 2 * nx + 1);

    for i = 1:(2 * nx + 1)
        sigmaPred(:, i) = predictState1Rc(sigmaPoints(:, i), current_a(k)
            ↪ - 1), dt, params);
        sigmaPred(1, i) = min(max(sigmaPred(1, i), 0), 1);
    end

    x_pred = sigmaPred * Wm.';
    P_pred = Q;
    for i = 1:(2 * nx + 1)
        dx = sigmaPred(:, i) - x_pred;
        P_pred = P_pred + Wc(i) * (dx * dx.');
```

end

```

    ySigma = zeros(1, 2 * nx + 1);
    for i = 1:(2 * nx + 1)
        ySigma(i) = measurement1Rc(sigmaPred(:, i), current_a(k),
            ↪ ocv_soc, ocv_v, params);
    end

    y_pred = ySigma * Wm.';
    Pyy = R;
    Pxy = zeros(nx, 1);
    for i = 1:(2 * nx + 1)
        dx = sigmaPred(:, i) - x_pred;
        dy = ySigma(i) - y_pred;
        Pyy = Pyy + Wc(i) * (dy * dy.');
```

end

```

    Pxy = Pxy + Wc(i) * dx * dy.');
```

end

```

    K = Pxy / Pyy;
    innovation_v(k) = voltage_meas_v(k) - y_pred;
    x_hat(:, k) = x_pred + K * innovation_v(k);
    x_hat(1, k) = min(max(x_hat(1, k), 0), 1);
    P = P_pred - K * Pyy * K.';
end
```

```

    soc_hat = x_hat(1, :).';
    vl_hat = x_hat(2, :).';
end

function sigmaPoints = computeSigmaPoints(x, P, gamma)
    nx = numel(x);
    S = chol(P + 1e-12 * eye(nx), "lower");
    sigmaPoints = zeros(nx, 2 * nx + 1);
    sigmaPoints(:, 1) = x;
    for i = 1:nx
        sigmaPoints(:, i + 1) = x + gamma * S(:, i);
        sigmaPoints(:, i + 1 + nx) = x - gamma * S(:, i);
    end
end

function [soc_hat, vl_hat, innovation_v] = runEkf1Rc( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

    dt = time_s(2) - time_s(1);
    n = numel(time_s);
    x_hat = zeros(2, n);
    x_hat(:, 1) = [initial_soc_guess; 0];
    P = P0;
    innovation_v = zeros(n, 1);
    alpha = exp(-dt / (params.R1 * params.C1));

    for k = 2:n
        x_pred = predictState1Rc(x_hat(:, k - 1), current_a(k - 1), dt,
            ↪ params);
        x_pred(1) = min(max(x_pred(1), 0), 1);

        A = [1.0 0.0; 0.0 alpha];
        P_pred = A * P * A' + Q;

        y_pred = measurement1Rc(x_pred, current_a(k), ocv_soc, ocv_v,
            ↪ params);
        dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
        C = [dUoc_dSoc, -1.0];

        innovation_v(k) = voltage_meas_v(k) - y_pred;
        S = C * P_pred * C' + R;
        K = (P_pred * C') / S;
    end
end

```

```

        x_hat(:, k) = x_pred + K * innovation_v(k);
        x_hat(1, k) = min(max(x_hat(1, k), 0), 1);

        I2 = eye(2);
        P = (I2 - K * C) * P_pred * (I2 - K * C)' + K * R * K';
    end

    soc_hat = x_hat(1, :).';
    vl_hat = x_hat(2, :).';
end

function x_next = predictState1Rc(x_now, current_a, dt, params)
    alpha = exp(-dt / (params.R1 * params.C1));
    soc_next = x_now(1) - params.eta * current_a * dt / (3600 * params.Q_ah);
    vl_next = alpha * x_now(2) + params.R1 * (1 - alpha) * current_a;
    x_next = [soc_next; vl_next];
end

function voltage_v = measurement1Rc(x_state, current_a, ocv_soc, ocv_v,
    ↪ params)
    ocv_now = interp1(ocv_soc, ocv_v, x_state(1), "pchip", "extrap");
    voltage_v = ocv_now - current_a * params.R0 - x_state(2);
end

function slope = ocvSlope(soc, ocv_soc, ocv_v)
    d_ocv = gradient(ocv_v, ocv_soc);
    slope = interp1(ocv_soc, d_ocv, soc, "pchip", "extrap");
end

function [soc, vl_v, voltage_v] = simulateTruth1Rc( ...
    time_s, current_a, initial_soc, ocv_soc, ocv_v, params)

    n = numel(time_s);
    dt = time_s(2) - time_s(1);
    alpha = exp(-dt / (params.R1 * params.C1));
    soc = zeros(n, 1);
    vl_v = zeros(n, 1);
    voltage_v = zeros(n, 1);

    soc(1) = initial_soc;

```

```

voltage_v(1) = measurement1Rc([soc(1); v1_v(1)], current_a(1), ocv_soc,
    ↪ ocv_v, params);

for k = 2:n
    soc(k) = soc(k - 1) - params.eta * current_a(k - 1) * dt / (3600 *
        ↪ params.Q_ah);
    soc(k) = min(max(soc(k), 0), 1);
    v1_v(k) = alpha * v1_v(k - 1) + params.R1 * (1 - alpha) * current_a(k
        ↪ - 1);
    voltage_v(k) = measurement1Rc([soc(k); v1_v(k)], current_a(k),
        ↪ ocv_soc, ocv_v, params);
end
end

```

7.3.22 How close is “close enough”?

For this reproduction exercise, “close enough” means you recover the same *comparison logic* as the published paper:

- all filters perform similarly under low-noise conditions or differ only modestly,
- adaptive handling of measurement uncertainty matters more as noise increases,
- convergence behavior is as important as headline RMSE.

If your low-noise panel and high-noise panel both show the same ranking and nearly identical gaps, that is a warning sign that your stress test is not actually stressing anything. Increase the high-noise level or reduce the initial trust in the correct covariance until the difference becomes meaningful.

7.3.23 What this reproduction actually teaches

It teaches two things at once. The first is methodological: battery estimator claims should be tested across noise regimes, not only in one pleasant operating condition. The second is scholarly: when the paper does not specify every implementation detail, your job is to document the substitutions openly rather than smuggle them in as if they were exact.

7.4 Open-Ended Exercises

These exercises are where you stop being a reader who can rerun the chapter and start becoming a researcher who can adapt it.

7.4.1 Exercise 1: Flat-Plateau Sodium-Ion Stress Test

Replace the lithium-ion-like OCV table in Guided Walkthrough 2 with the sodium-ion-inspired plateau curve from Guided Walkthrough 1. Quantify how much worse the EKF performs in terms of SOC RMSE, maximum absolute error, and convergence time. Then propose one fix and demonstrate it in code.

Hints:

- Keep the same current profile and ECM parameters first so the OCV shape is the only changed factor.
- Compare not only the SOC error but also the Kalman gain and innovation magnitude.
- Reason physically: if the OCV slope falls, the measurement update becomes weaker.

7.4.2 Exercise 2: Temperature-Mismatch Sensitivity

Use the CALCE 25 °C OCV table for the estimator, but perturb the measured voltage as if the real cell were operating on a colder or warmer OCV curve. Quantify how much SOC error is introduced by an OCV-temperature mismatch.

Hints:

- You can emulate a temperature mismatch without downloading more files by shifting the OCV curve upward or downward in a SOC-dependent way.
- Keep the current profile fixed so you isolate the OCV mismatch.
- Report whether the resulting innovation looks like noise, bias, or both.

7.4.3 Exercise 3: Current-Bias Sensitivity

Augment the synthetic dataset so that the measured current includes a constant offset of 20 mA, 50 mA, and 100 mA, while the truth model still uses the unbiased current. Compare plain coulomb counting, the standard EKF, and a current-bias-aware extension of the filter.

Hints:

- A current bias enters the process model rather than the measurement model.
- You can model it as an additional state b_I with a slow random walk.
- Watch long-horizon error accumulation rather than only short transients.

7.4.4 Exercise 4: Refit the 1RC Parameters and Re-run the CALCE Benchmark

Take the real-data benchmarking workflow from Guided Walkthrough 4 and replace the generic R0, R1, and C1 values with parameters identified from either a pulse segment in the dynamic file or the Chapter 6 ECM-identification workflow. Quantify whether the improved model changes the EKF's SOC trajectory meaningfully or only improves the voltage RMSE.

Hints:

- A better voltage fit does not always translate into much better SOC estimation.
- Compare innovation statistics before and after parameter refinement.
- Be careful not to claim improved SOC truthfulness unless you have a stronger reference than anchored coulomb counting.

7.5 Worked Solutions to the Open-Ended Exercises

7.5.1 Solution 1: Flat-Plateau Sodium-Ion Stress Test

The point of this exercise is not merely to show that “SIB is harder.” The point is to connect that difficulty directly to the measurement Jacobian from Equation (5). When the OCV slope collapses over part of the SOC range, the SOC component of the Kalman gain shrinks, so the filter leans more heavily on process integration.

```
clear; close all; clc;

dt = 1;
time_s = (0:1:2200).';
current_a = zeros(size(time_s));
current_a(time_s >= 80 & time_s < 400) = 2.2;
current_a(time_s >= 560 & time_s < 920) = 3.1;
current_a(time_s >= 1110 & time_s < 1320) = -1.2;
current_a(time_s >= 1480 & time_s < 1860) = 2.6;

soc_grid = linspace(0, 1, 201).';
ocv_li = 3.00 ...
    + 0.74 * soc_grid ...
    + 0.16 * tanh((soc_grid - 0.14) / 0.06) ...
    + 0.18 * tanh((soc_grid - 0.86) / 0.05);
ocv_sib = 2.45 ...
    + 0.40 * soc_grid ...
    + 0.26 * tanh((soc_grid - 0.10) / 0.05) ...
    + 0.06 * tanh((soc_grid - 0.45) / 0.10) ...
```

```

+ 0.28 * tanh((soc_grid - 0.88) / 0.04);

params.Q_ah = 3.0;
params.eta = 1.0;
params.R0 = 0.014;
params.R1 = 0.018;
params.C1 = 95.0;

[soc_true_li, ~, voltage_true_li] = simulateTruth1Rc(time_s, current_a, 0.88,
↳ soc_grid, ocv_li, params);
[soc_true_sib, ~, voltage_true_sib] = simulateTruth1Rc(time_s, current_a,
↳ 0.88, soc_grid, ocv_sib, params);

rng(55);
voltage_meas_li = voltage_true_li + 4e-3 * randn(size(voltage_true_li));
voltage_meas_sib = voltage_true_sib + 4e-3 * randn(size(voltage_true_sib));

[soc_hat_li, ~, innovation_li, gain_li] = runEkf1RcWithGain( ...
    time_s, current_a, voltage_meas_li, soc_grid, ocv_li, params, ...
    0.68, diag([0.08^2, 0.03^2]), diag([1e-8, 1e-5]), (4e-3)^2);

[soc_hat_sib, ~, innovation_sib, gain_sib] = runEkf1RcWithGain( ...
    time_s, current_a, voltage_meas_sib, soc_grid, ocv_sib, params, ...
    0.68, diag([0.08^2, 0.03^2]), diag([3e-8, 1e-5]), (4e-3)^2);

rmse_li = sqrt(mean((soc_hat_li - soc_true_li).^2));
rmse_sib = sqrt(mean((soc_hat_sib - soc_true_sib).^2));
maxe_li = max(abs(soc_hat_li - soc_true_li));
maxe_sib = max(abs(soc_hat_sib - soc_true_sib));

idx_conv_li = find(abs(soc_hat_li - soc_true_li) <= 0.01, 1, "first");
idx_conv_sib = find(abs(soc_hat_sib - soc_true_sib) <= 0.01, 1, "first");

fprintf("Li-ion-like RMSE: %.4f | Max error: %.4f | Convergence time: %.1f
↳ s\n", ...
    rmse_li, maxe_li, time_s(idx_conv_li));
fprintf("SIB-like RMSE: %.4f | Max error: %.4f | Convergence time: %.1f s\n",
↳ ...
    rmse_sib, maxe_sib, time_s(idx_conv_sib));

figure("Color", "w", "Position", [70 70 1200 900]);

subplot(3, 1, 1);
plot(time_s, soc_true_li, "k", "LineWidth", 2.0); hold on;
plot(time_s, soc_hat_li, "LineWidth", 1.6);

```

```

plot(time_s, soc_hat_sib, "LineWidth", 1.6);
grid on;
xlabel("Time [s]");
ylabel("SOC [-]");
title("Same EKF architecture on Li-ion-like and SIB-like OCV curves");
legend("True SOC", "EKF on Li-ion-like curve", "EKF on SIB-like curve",
    ↪ "Location", "best");

subplot(3, 1, 2);
plot(time_s, gain_li, "LineWidth", 1.5); hold on;
plot(time_s, gain_sib, "LineWidth", 1.5);
grid on;
xlabel("Time [s]");
ylabel("SOC Kalman gain");
title("SOC gain weakens when the OCV plateau flattens");
legend("Li-ion-like", "SIB-like", "Location", "best");

subplot(3, 1, 3);
plot(time_s, innovation_li * 1000, "LineWidth", 1.3); hold on;
plot(time_s, innovation_sib * 1000, "LineWidth", 1.3);
grid on;
xlabel("Time [s]");
ylabel("Innovation [mV]");
title("Innovation traces remain bounded but become less informative for
    ↪ SOC");
legend("Li-ion-like", "SIB-like", "Location", "best");

function [soc_hat, v1_hat, innovation_v, gain_soc] = runEkf1RcWithGain( ...
    time_s, current_a, voltage_meas_v, ocv_soc, ocv_v, params, ...
    initial_soc_guess, P0, Q, R)

    dt = time_s(2) - time_s(1);
    n = numel(time_s);
    x_hat = zeros(2, n);
    x_hat(:, 1) = [initial_soc_guess; 0];
    P = P0;
    innovation_v = zeros(n, 1);
    gain_soc = zeros(n, 1);
    alpha = exp(-dt / (params.R1 * params.C1));

    for k = 2:n
        x_pred = predictState1Rc(x_hat(:, k - 1), current_a(k - 1), dt,
            ↪ params);
        x_pred(1) = min(max(x_pred(1), 0), 1);

```

```

    A = [1.0 0.0; 0.0 alpha];
    P_pred = A * P * A' + Q;

    y_pred = measurement1Rc(x_pred, current_a(k), ocv_soc, ocv_v,
        ↪ params);
    dUoc_dSoc = ocvSlope(x_pred(1), ocv_soc, ocv_v);
    C = [dUoc_dSoc, -1.0];

    innovation_v(k) = voltage_meas_v(k) - y_pred;
    S = C * P_pred * C' + R;
    K = (P_pred * C') / S;
    gain_soc(k) = K(1);

    x_hat(:, k) = x_pred + K * innovation_v(k);
    x_hat(1, k) = min(max(x_hat(1, k), 0), 1);

    I2 = eye(2);
    P = (I2 - K * C) * P_pred * (I2 - K * C)' + K * R * K';
end

soc_hat = x_hat(1, :).';
v1_hat = x_hat(2, :).';
end

function x_next = predictState1Rc(x_now, current_a, dt, params)
    alpha = exp(-dt / (params.R1 * params.C1));
    soc_next = x_now(1) - params.eta * current_a * dt / (3600 * params.Q_ah);
    v1_next = alpha * x_now(2) + params.R1 * (1 - alpha) * current_a;
    x_next = [soc_next; v1_next];
end

function voltage_v = measurement1Rc(x_state, current_a, ocv_soc, ocv_v,
    ↪ params)
    ocv_now = interp1(ocv_soc, ocv_v, x_state(1), "pchip", "extrap");
    voltage_v = ocv_now - current_a * params.R0 - x_state(2);
end

function slope = ocvSlope(soc, ocv_soc, ocv_v)
    d_ocv = gradient(ocv_v, ocv_soc);
    slope = interp1(ocv_soc, d_ocv, soc, "pchip", "extrap");
end

```

```

function [soc, v1_v, voltage_v] = simulateTruth1Rc( ...
    time_s, current_a, initial_soc, ocv_soc, ocv_v, params)

    n = numel(time_s);
    dt = time_s(2) - time_s(1);
    alpha = exp(-dt / (params.R1 * params.C1));
    soc = zeros(n, 1);
    v1_v = zeros(n, 1);
    voltage_v = zeros(n, 1);

    soc(1) = initial_soc;
    voltage_v(1) = measurement1Rc([soc(1); v1_v(1)], current_a(1), ocv_soc,
        ↪ ocv_v, params);

    for k = 2:n
        soc(k) = soc(k - 1) - params.eta * current_a(k - 1) * dt / (3600 *
            ↪ params.Q_ah);
        soc(k) = min(max(soc(k), 0), 1);
        v1_v(k) = alpha * v1_v(k - 1) + params.R1 * (1 - alpha) * current_a(k
            ↪ - 1);
        voltage_v(k) = measurement1Rc([soc(k); v1_v(k)], current_a(k),
            ↪ ocv_soc, ocv_v, params);
    end
end

```

One practical fix is already illustrated above: slightly increasing SOC process noise for the SIB-like case gives the filter more freedom to adapt when the voltage measurement becomes less informative. That does not solve the observability problem completely, but it is often a reasonable first response.

7.5.2 Solution 2: Temperature-Mismatch Sensitivity

The shortest demonstration is to perturb the OCV curve itself and quantify the induced estimator drift.

```

clear; close all; clc;

soc_grid = linspace(0, 1, 201).';
ocv_25c = 3.00 ...
    + 0.74 * soc_grid ...
    + 0.16 * tanh((soc_grid - 0.14) / 0.06) ...
    + 0.18 * tanh((soc_grid - 0.86) / 0.05);

```

```

% Synthetic colder-curve mismatch: slightly lower voltage in midrange
ocv_cold = ocv_25c - 0.025 + 0.015 * soc_grid;

dt = 1;
time_s = (0:1:1800).';
current_a = zeros(size(time_s));
current_a(time_s >= 100 & time_s < 750) = 2.0;
current_a(time_s >= 950 & time_s < 1450) = 1.5;

params.Q_ah = 2.3;
params.eta = 1.0;
params.R0 = 0.014;
params.R1 = 0.018;
params.C1 = 95.0;

[soc_true, ~, voltage_true_cold] = simulateTruth1Rc(time_s, current_a, 0.88,
    ↪ soc_grid, ocv_cold, params);
rng(88);
voltage_meas_v = voltage_true_cold + 4e-3 * randn(size(voltage_true_cold));

[soc_hat_wrong_ocv, ~, innovation_v] = runEkf1Rc( ...
    time_s, current_a, voltage_meas_v, soc_grid, ocv_25c, params, ...
    0.68, diag([0.08^2, 0.03^2]), diag([1e-8, 1e-5]), (4e-3)^2);

fprintf("SOC RMSE under temperature-mismatched OCV table: %.4f\n", ...
    sqrt(mean((soc_hat_wrong_ocv - soc_true).^2)));
fprintf("Mean innovation under mismatch: %.4f mV\n", mean(innovation_v) *
    ↪ 1000);

```

The expected outcome is a nonzero mean innovation and an SOC estimate that is systematically biased, not merely noisy. That is the key interpretation: OCV-temperature mismatch behaves like structural model bias.

7.5.3 Solution 3: Current-Bias Sensitivity

The core insight is that current bias contaminates the process model directly, so if you leave it unmodeled the estimator can slowly drift even when voltage updates are present.

```

clear; close all; clc;

dt = 1;
time_s = (0:1:3000).';

```

```

current_true_a = zeros(size(time_s));
current_true_a(time_s >= 100 & time_s < 800) = 2.0;
current_true_a(time_s >= 1100 & time_s < 1850) = 1.8;
current_true_a(time_s >= 2100 & time_s < 2700) = 2.4;

current_meas_a = current_true_a + 0.05; % 50 mA bias

soc_grid = linspace(0, 1, 201).';
ocv_v = 3.00 ...
    + 0.74 * soc_grid ...
    + 0.16 * tanh((soc_grid - 0.14) / 0.06) ...
    + 0.18 * tanh((soc_grid - 0.86) / 0.05);

params.Q_ah = 2.3;
params.eta = 1.0;
params.R0 = 0.014;
params.R1 = 0.018;
params.C1 = 95.0;

[soc_true, ~, voltage_true_v] = simulateTruth1Rc(time_s, current_true_a,
    ↪ 0.90, soc_grid, ocv_v, params);
rng(93);
voltage_meas_v = voltage_true_v + 4e-3 * randn(size(voltage_true_v));

[soc_hat_wrong_current, ~, ~] = runEkf1Rc( ...
    time_s, current_meas_a, voltage_meas_v, soc_grid, ocv_v, params, ...
    0.72, diag([0.08^2, 0.03^2]), diag([1e-8, 1e-5]), (4e-3)^2);

fprintf("SOC RMSE with 50 mA current bias left unmodeled: %.4f\n", ...
    sqrt(mean((soc_hat_wrong_current - soc_true).^2)));

```

The proper current-bias-aware extension is structurally similar to the voltage-bias exercise, except the bias enters the process model in the coulomb-counting term. That is a very good mini-project if you are building toward a paper.

7.5.4 Solution 4: Refit the 1RC Parameters and Re-run CALCE

The full solution depends on your Chapter 6 identification workflow, but the methodological lesson is straightforward. A lower voltage RMSE is good. A meaningfully improved SOC estimate is even better. The two are related but not equivalent. If your refined parameters mainly reduce dynamic-voltage residuals while leaving the OCV map and observability unchanged, the SOC improvement

may be modest.

What Changes for Sodium-Ion?

This section is no longer a short aside. By Chapter 7, the sodium-ion differences are central.

First, OCV shape becomes a first-class estimator design issue rather than a chemistry footnote. Many sodium-ion full-cell combinations, especially those involving hard carbon and plateau-rich cathode behavior, give long SOC intervals where $\frac{dU_{oc}}{dz}$ is small. In the language of Equation (5), that means the measurement Jacobian becomes weak in exactly the component that connects voltage to SOC. A filter tuned on a lithium-ion-like curve can therefore look overconfident on a sodium-ion-like curve even if every matrix dimension is correct.

Second, parameter transfer is less forgiving. With lithium-ion, public ECM tables, OCV datasets, and validation profiles are abundant. With sodium-ion, you are more likely to assemble the estimator stack from partial literature parameterizations, digitized OCV curves, or synthetic surrogate data generated from a physics-based model. That makes uncertainty accounting more important, not less.

Third, validation strategy has to broaden. A sodium-ion SOC estimator should not be judged only by terminal-voltage fit under one dynamic cycle. It should also be stress-tested across initialization error, plateau regions, temperature mismatch, and sensor-bias scenarios, because those are precisely the conditions under which weak observability gets exposed.

Fourth, a practical sodium-ion workflow often benefits from hybridization. You may use PyBaMM or another electrochemical model to generate synthetic sodium-ion trajectories across controlled parameter variations, then use those trajectories as a virtual test bench for estimator design before scarce experimental data arrive. That is not a substitute for validation. It is a rational way to build estimator intuition under data scarcity.

Finally, the fix for a hard sodium-ion SOC problem is not always “use a fancier filter.” Sometimes the real fix is better excitation, a chemistry-specific OCV table, temperature-aware lookup surfaces, explicit hysteresis handling, or augmenting the state with additional nuisance terms. The filter architecture matters, but the

information content of the battery and the experiment matters more.

Chapter Summary and Skill Checklist

- You translated the 1RC ECM from Chapter 6 into a nonlinear state-space observer suitable for SOC estimation.
- You implemented an EKF from scratch in MATLAB, including prediction, measurement linearization, correction, and Joseph-form covariance updates.
- You learned to interpret the measurement Jacobian physically through the OCV slope and connected that directly to estimator observability.
- You tuned Q and R using systematic sweeps rather than ad hoc aesthetic adjustment.
- You built a robust CALCE data-ingestion workflow that handles workbook quirks, sign conventions, and OCV-table construction.
- You benchmarked the EKF on real dynamic data and learned the difference between true SOC, anchored coulomb-counting reference, and voltage-fit quality.
- You implemented a UKF and learned when its nonlinear propagation is worth the extra complexity.
- You extended the state to estimate voltage bias and saw how nuisance states can protect SOC from absorbing persistent measurement errors.
- You completed a reproduction exercise in the right scholarly spirit: explicit substitutions, explicit limitations, and explicit comparison criteria.
- You stress-tested the estimator for sodium-ion-like flat OCV plateaus and identified why those chemistries are estimator-hard.

Commands, functions, and patterns that should now feel familiar:

- `interp1(..., "pchip", "extrap")` for OCV lookup
- `gradient` for numerical OCV slope evaluation
- EKF predict-measure-update structure in a `for` loop
- Joseph-form covariance updates
- sigma-point generation with `chol`
- workbook parsing with `sheetnames` and `readtable`
- resampling with `interp1(..., "previous")` and `interp1(..., "linear")`

- innovation-based tuning and residual inspection

You should now be able to:

- derive and code the state equations for a 1RC SOC estimator
- explain why OCV slope determines how informative voltage is for SOC correction
- implement and tune an EKF without relying on a toolbox wizard
- compare EKF and UKF results honestly rather than assuming one must win
- ingest a public battery dataset and normalize its conventions before estimation
- diagnose whether a persistent innovation trend is caused by bias, temperature mismatch, or poor tuning
- explain why sodium-ion plateau regions weaken voltage-based SOC correction
- design a publishable benchmark that reports RMSE, max error, and convergence time together

If any of those boxes still feel shaky, revisit Guided Walkthroughs 2 through 4 before moving on.

Deliverable

The deliverable from the plan is:

A MATLAB implementation of both EKF and UKF for SOC estimation, benchmarked on CALCE data, with a written analysis of which performs better and why.

Approach it in four passes.

First, freeze your modeling assumptions. Choose the exact OCV source, ECM order, current sign convention, sample period, and initial covariance structure. Write those down before you compare filters. This is standard practice in the field.

Second, benchmark on at least two regimes:

- a synthetic matched-model case where true SOC is known
- one CALCE dynamic file where voltage-fit realism matters more than synthetic perfection

Third, report at least these metrics for each estimator:

- SOC RMSE
- maximum absolute SOC error
- convergence time to a stated threshold such as 1% SOC
- voltage RMSE
- a short innovation-behavior comment

Fourth, include one stress test that makes the analysis interesting. A good choice is either a voltage-bias case or the sodium-ion plateau case. Without a stress test, the EKF-UKF comparison often becomes too bland to teach much.

A strong partial solution would therefore include:

- one script for matched synthetic EKF/UKF comparison
- one script for CALCE ingestion and real-data EKF benchmarking
- one script for a stress test such as bias or plateau observability
- a short report discussing not only which filter was numerically best, but *why* that happened

The “why” is the heart of the deliverable. If the UKF only improves slightly, explain that the OCV nonlinearity and covariance spread were modest. If the bias-aware EKF wins decisively, explain that nuisance modeling mattered more than filter-order sophistication. Those are exactly the kinds of conclusions that turn classwork into research competence.

Further Practice and Reading

Primary papers and references:

1. Xing, Y., He, W., Pecht, M., & Tsui, K. L. (2014). *State of Charge Estimation of Lithium-Ion Batteries Using the Open-Circuit Voltage at Various Ambient Temperatures*. *Applied Energy*, 113, 106-115. DOI: 10.1016/j.apenergy.2013.07.008.
2. ElMenshawy, M. S., Massoud, A. M., & Guglielmi, P. (2025). *State-of-Charge Estimation Using Triple Forgetting Factor Adaptive Extended Kalman Filter for Battery Energy Storage Systems in Electric Bus Applications*. *IEEE Transactions on Transportation Electrification*, 11(2), 6664-6674. DOI: 10.1109/TTE.2024.3514704.
3. Plett, G. L. *Battery Management Systems*, especially the SOC-estimation

material and the Coursera “Battery State-of-Charge (SOC) Estimation” assignments.

Documentation and tool references:

1. CALCE Battery Research Data portal: <https://calce.umd.edu/data>
2. MATLAB documentation for `readtable`, `sheetnames`, `interp1`, and matrix factorization routines such as `chol`
3. MATLAB documentation on state-estimation workflows if you later want to port the hand-built filters into System objects or Simulink observers

Community and reusable code resources:

1. MATLAB File Exchange searches for “battery EKF SOC” and “UKF battery SOC” can be useful for cross-checking implementation structure, though you should trust your own derived equations first.
2. The battery-modeling and BMS communities around GitHub often share ECM-estimation and Kalman-filter notebooks; use them as comparison points, not as substitutes for understanding.
3. If you are also working in Python, Chapter 10 will show how to generate synthetic virtual-cell data in PyBaMM and use that as an estimator-design sandbox for sodium-ion problems where experimental data remain sparse.

The next chapter is Lab Chapter 8: SOH and Aging Models.

SOH and Aging Models

This chapter is where battery modeling stops being a single-cycle exercise and becomes a lifetime exercise. Up to this point, most of our simulations and estimators have lived inside one discharge, one drive cycle, or one carefully bounded test protocol. That is exactly how we should learn the tools. But a publishable battery-health study asks a harder question: how does the cell change after tens, hundreds, or thousands of repetitions, and how do we infer that change from data that are incomplete, noisy, and usually collected under imperfectly controlled conditions?

Keep Textbook Chapter 7 open while you work. That chapter gave you the physical degradation vocabulary: SEI growth, loss of lithium or sodium inventory, loss of active material, impedance rise, electrolyte decomposition, current-collector corrosion, particle cracking, and temperature-accelerated side reactions. This lab chapter operationalizes those mechanisms. We will turn the mechanistic ideas into empirical capacity-fade equations, semi-empirical cycling models, PyBaMM degradation options, MATLAB scripts, and diagnostic features such as incremental-capacity analysis. We will also lean on Textbook Chapter 10 because state of health, or SOH, is not only a materials property. It is a quantity estimated by a BMS from voltage, current, temperature, and historical usage.

The target skill is not merely “fit a curve.” Any spreadsheet can fit a curve. The

research skill is learning to decide which curve is physically defensible, which data points deserve to be included, which assumptions you are making silently, and how to report uncertainty without overselling. Aging data are especially treacherous because capacity fade is slow, cells vary from one another, laboratory protocols differ, and sodium-ion public datasets remain sparse compared with lithium-ion datasets. The consequence is that a strong sodium-ion aging paper often uses a hybrid strategy: validate the workflow on public lithium-ion data, adapt the mechanism and parameters for sodium-ion chemistry, then state clearly what still requires sodium-ion experimental confirmation.

We will build the chapter around the NASA Prognostics Center of Excellence Li-ion Battery Aging dataset. NASA describes this dataset as cells cycled under charge, discharge, and impedance operations at different temperatures, with end of life defined as 30% capacity fade from 2 Ah to 1.4 Ah. The public NASA PCoE repository lists the direct download for the battery dataset and asks users to acknowledge the repository and data donors when publishing with it. We will parse the original MATLAB `.mat` files, extract discharge capacities, clean the cycle table, fit empirical fade models, and use the fitted model to forecast 500 cycles. Then we will build a parallel MATLAB implementation so you can reproduce the same logic in the BMS ecosystem introduced in Chapters 6 and 7.

We will also use PyBaMM. PyBaMM is not only a DFN simulator; it includes degradation submodels such as SEI growth. The key intellectual move is recognizing that a physics-based degradation simulation and a dataset-fitted health model answer different questions. A PyBaMM SEI model asks, “What happens if this mechanism and parameterization are correct?” A NASA-calibrated empirical model asks, “What trend does this dataset support?” Publishable research often needs both: a mechanistic model for interpretation and a data-calibrated model for validation.

By the end of this chapter, you will have a Python workflow that downloads or loads the NASA dataset, extracts SOH versus cycle number, fits square-root, power-law, and Arrhenius-inspired capacity-fade models, simulates degradation with PyBaMM’s SEI options, and performs incremental-capacity analysis on partial charge/discharge curves. You will also have a MATLAB script that implements the same empirical aging model over 500+ cycles. The deliverable is a calibrated aging model in both languages, with a comparison plot and a short written interpretation of what the model can and cannot claim.

8.1 Prerequisites Check

- Required software: Python 3.11, numpy==1.26.4, scipy==1.13.1, pandas==2.2.2, matplotlib==3.9.0, h5py==3.11.0, requests==2.32.3, pybamm==26.3.1, and MATLAB R2024b or newer recommended
- Install command: `python -m pip install numpy==1.26.4 scipy==1.13.1 pandas==2.2.2 matplotlib==3.9.0 h5py==3.11.0 requests==2.32.3 pybamm==26.3.1`
- Required textbook chapters: Textbook Chapter 7 is essential; Textbook Chapter 8 helps for PyBaMM DFN context; Textbook Chapter 10 helps for BMS interpretation of SOH
- Required prior lab chapters: Lab Chapters 1, 2, 3, 4, and 6 are expected; Lab Chapter 7 is helpful but not required
- External datasets: NASA PCoE Battery Data Set, direct repository page <https://www.nasa.gov/intelligent-systems-division/discovery-and-sys>
NASA Open Data landing page <https://data.nasa.gov/dataset/li-ion-battery-aging-datasets>
- Estimated time: 18 to 24 hours, including dataset download and the open-ended exercises

If your PyBaMM confidence is shaky, revisit Lab Chapter 3 before the PyBaMM exercise. If you have not handled real battery time-series data yet, reread the pandas sections in Lab Chapter 2 and the CALCE parsing workflow in Lab Chapter 6. Aging analysis punishes casual data handling; the code here is deliberately explicit because a one-line parsing mistake can create a beautiful but false SOH curve.

8.2 Environment Setup

We will use one Python workspace and one MATLAB workspace. The Python side does the heavier lifting: dataset download, `.mat` parsing, empirical fitting, PyBaMM simulation, and ICA/DVA processing. The MATLAB side mirrors the empirical model so you can bring SOH modeling into the same environment used for ECMs and Kalman filters.

8.2.1 Create the Python environment

From a terminal, move to your research repository and create an isolated environment:

```
cd ~/battery-research
python3.11 -m venv .venv-aging
source .venv-aging/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==1.26.4 scipy==1.13.1 pandas==2.2.2
↪ matplotlib==3.9.0 h5py==3.11.0 requests==2.32.3 pybamm==26.3.1
```

On Windows PowerShell, the activation command is:

```
.\.venv-aging\Scripts\Activate.ps1
```

The version pin for PyBaMM uses the stable documentation line available at the time this chapter was written. PyBaMM's own installation page says the package is available through `pip` and `conda`, but notes that the `conda` recipe may lag behind recent releases. For this chapter, use `pip` unless your institution has a strict `conda` workflow.

Now verify the Python stack:

```
import numpy as np
import pandas as pd
import scipy
import matplotlib
import pybamm

print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
print("SciPy:", scipy.__version__)
print("Matplotlib:", matplotlib.__version__)
print("PyBaMM:", pybamm.__version__)
```

Expected output:

```
NumPy: 1.26.4
pandas: 2.2.2
SciPy: 1.13.1
Matplotlib: 3.9.0
PyBaMM: 26.3.1
```

If `pybamm` fails to import with a solver-related error, restart the terminal after installation and try again. If the error mentions a missing C++ runtime on

Windows, install the Microsoft Visual C++ Redistributable and rerun the import. If pip cannot find pybamm==26.3.1, install the latest stable version shown by `python -m pip index versions pybamm`, then record the exact version in your research log because aging-option names can change.

8.2.2 Create the chapter folders

Run this once from the activated Python environment:

```
from pathlib import Path

chapter_root = Path("chapter8_soh_aging_workspace")
for subfolder in ["data/raw", "data/processed", "results", "figures",
    ↪ "matlab"]:
    (chapter_root / subfolder).mkdir(parents=True, exist_ok=True)

print(chapter_root.resolve())
```

Expected output is the absolute path to `chapter8_soh_aging_workspace`. Every script in this chapter assumes this folder structure. If you choose a different root, change only the `chapter_root` variable, not every path by hand.

8.2.3 Run a minimal aging hello-world

Before we touch real data, check that SciPy can fit a simple degradation model:

```
import numpy as np
from scipy.optimize import curve_fit

cycle_index = np.arange(1, 101)
true_capacity_ah = 2.0 - 0.035 * np.sqrt(cycle_index)
measured_capacity_ah = true_capacity_ah + 0.003 * np.sin(cycle_index / 6)

def sqrt_fade_model(cycles, capacity_0, fade_coefficient):
    return capacity_0 - fade_coefficient * np.sqrt(cycles)

parameters, covariance = curve_fit(
    sqrt_fade_model,
    cycle_index,
    measured_capacity_ah,
    p0=[2.0, 0.03],
)
```

```
print(f"Fitted initial capacity: {parameters[0]:.4f} Ah")
print(f"Fitted sqrt fade coefficient: {parameters[1]:.5f} Ah/cycle^0.5")
```

Expected output:

```
Fitted initial capacity: 2.0012 Ah
Fitted sqrt fade coefficient: 0.03515 Ah/cycle^0.5
```

The fitted values should be close to the synthetic truth. This verifies the numerical workflow: arrays are shaped correctly, SciPy can optimize, and your environment can run the simplest empirical aging model.

8.2.4 Verify MATLAB

Open MATLAB and run:

```
clear; clc;
cycleIndex = (1:100)';
capacityAh = 2.0 - 0.035 * sqrt(cycleIndex);
plot(cycleIndex, capacityAh, "LineWidth", 1.8);
grid on;
xlabel("Cycle number");
ylabel("Capacity (Ah)");
title("Aging hello-world");
```

You should see a smooth, gently concave-down capacity curve that starts near 1.965 Ah at cycle 1 and ends near 1.650 Ah at cycle 100. If the figure is blank, call `drawnow`. If MATLAB complains about string quotes, you are likely using a very old release; replace double quotes with single quotes.

8.2.5 Common setup failures

The NASA download is slow or blocked. The direct NASA PCoE file is a zip archive served from an S3 bucket. If your network blocks large downloads from scripts, download it manually from the NASA repository page and place it in `chapter8_soh_aging_workspace/data/raw/Battery_Data_Set.zip`. The parsing code checks for the local file first.

scipy.io.loadmat cannot read a file. Some `.mat` files use MATLAB v7.3 HDF5 format, which requires `h5py`. The NASA battery files used in the original PCoE archive are commonly readable with `scipy.io.loadmat`, but this chapter includes an HDF5 fallback pattern because public mirrors sometimes repack files.

PyBaMM degradation simulations run slowly. Aging models are long-horizon by nature. We will use reduced cycle counts and explicit output variables in the guided exercise, then discuss how to scale the run. For research production, cache intermediate outputs and avoid rerunning long simulations just to change a plot label.

8.3 Conceptual Bridge: From Degradation Mechanisms to SOH Model Terms

In Textbook Chapter 7, degradation mechanisms were described physically. SEI growth consumes cyclable inventory and increases impedance. Loss of active material reduces the amount of host structure available for intercalation. Electrolyte decomposition changes transport and reaction kinetics. Particle cracking creates new surface area, which can accelerate SEI growth and expose fresh reactive material. Temperature enters almost every mechanism through Arrhenius-type rate dependence. Current and depth of discharge enter by changing overpotential, concentration gradients, mechanical stress, and time spent at reactive potentials.

Software tools cannot simulate a mechanism until we choose a mathematical representation for it. That representation can be empirical, semi-empirical, or physics-based. The categories overlap, but the distinction is useful.

An empirical capacity-fade model describes the observed trend without claiming that every term corresponds to a particular microscopic process. A common example is

$$Q(N) = Q_0 - k\sqrt{N}, \quad (1)$$

where $Q(N)$ is discharge capacity after cycle number N , Q_0 is the extrapolated initial capacity, and k is a fitted fade coefficient. The square-root dependence appears often in diffusion-limited SEI growth arguments, but in an empirical fit it should be interpreted cautiously. If Equation (1) fits a dataset, that does not prove SEI growth is the only mechanism. It only says the dataset's trend is consistent with a concave-down fade curve over the observed window.

A slightly more flexible empirical model is a power law:

$$Q(N) = Q_0 - kN^b, \quad (2)$$

where b is fitted. When $b = 0.5$, Equation (2) reduces to square-root fade. When b is closer to 1, fade is more nearly linear with cycle count. The cost of flexibility is interpretability: if you fit b freely to a small, noisy dataset, you may be fitting protocol noise rather than degradation physics.

A semi-empirical model adds operational dependence. One common form is

$$\Delta Q(N, T, C_{\text{rate}}) = k_0 \exp\left(-\frac{E_a}{RT}\right) C_{\text{rate}}^\gamma N^b, \quad (3)$$

where E_a is an apparent activation energy, R is the gas constant, T is absolute temperature, and γ describes current-rate dependence. Equation (3) is not a full electrochemical model. It is a compact way to say that the fade rate increases with temperature and current stress. In a real study, you would need data at multiple temperatures and current rates to identify those terms. If you only have one cell at one temperature, fitting E_a is not possible; any value you choose is an assumption.

Physics-based degradation models go one level deeper. In PyBaMM, selecting an SEI option modifies the electrochemical model by adding side-reaction states and equations. The model can track loss of lithium inventory, SEI thickness, and capacity change produced by the side reaction. This is powerful because it connects capacity fade to internal state variables, but it also introduces parameters that are hard to measure. A physics-based model with guessed degradation parameters is not automatically more truthful than a simple empirical model fitted to clean data.

For BMS work, SOH is usually defined relative to a reference capacity:

$$\text{SOH}(N) = \frac{Q(N)}{Q_{\text{rated}}}. \quad (4)$$

Some studies use initial measured capacity instead of rated capacity:

$$\text{SOH}_{\text{relative}}(N) = \frac{Q(N)}{Q(1)}. \quad (5)$$

Both definitions are valid if stated clearly. The NASA dataset often refers to cells with a nominal 2 Ah rating and an EOL criterion of 1.4 Ah, which corresponds to 70% of rated capacity. In this chapter we will report both capacity in ampere-hours and SOH relative to the first valid discharge capacity because that is the most robust choice when parsing individual cells.

The final bridge is diagnostic. We rarely get a perfect full discharge capacity measurement in the field. Instead, we may see partial charge curves, partial

discharge curves, rest voltages, pulse responses, and sensor histories. Incremental-capacity analysis, or ICA, uses

$$\frac{dQ}{dV}, \quad (6)$$

while differential-voltage analysis, or DVA, uses

$$\frac{dV}{dQ}. \quad (7)$$

These curves amplify subtle changes in the voltage-capacity relationship. Peaks shift, broaden, or shrink as the cell ages. In lithium-ion research, ICA/DVA is widely used for degradation diagnosis. For sodium-ion, the same idea applies, but peak interpretation depends strongly on the cathode, hard-carbon anode, electrolyte, and voltage window. In this chapter, we will teach the computational workflow rather than pretend that a Li-ion peak map transfers directly to sodium-ion chemistry.

Guided Walkthrough 1

Parse NASA Aging Data and Build an SOH Table

Learning objective: Download or load the NASA battery aging archive, parse discharge cycles from the original MATLAB files, and create a clean capacity-versus-cycle table.

We start with data, not models. This is a deliberate order. Aging models are seductive because fitting a smooth curve feels productive, but the hard part is usually deciding what the data mean. The NASA archive stores each cell as a MATLAB structure array. Each cycle has a `type`, an `ambient_temperature`, a `time`, and a nested `data` structure. For discharge cycles, the nested data include voltage, current, temperature, time, and a scalar capacity in ampere-hours.

The NASA PCoE repository lists the battery dataset direct download as <https://phm-datasets.s3.amazonaws.com/NASA/5.+Battery+Data+Set.zip>. The NASA Open Data page describes the acquisition system and notes that data were collected at approximately 10 Hz. The code below tries the direct download, extracts the archive, locates `.mat` files, and builds a tidy table.

```

from pathlib import Path
from zipfile import ZipFile
import requests
import numpy as np
import pandas as pd
from scipy.io import loadmat

CHAPTER_ROOT = Path("chapter8_soh_aging_workspace")
RAW_DIR = CHAPTER_ROOT / "data" / "raw"
PROCESSED_DIR = CHAPTER_ROOT / "data" / "processed"
RAW_DIR.mkdir(parents=True, exist_ok=True)
PROCESSED_DIR.mkdir(parents=True, exist_ok=True)

NASA_ZIP_URL =
↳ "https://phm-datasets.s3.amazonaws.com/NASA/5.+Battery+Data+Set.zip"
    =
    / "Battery_Data_Set.zip"
    =
    / "NASA_Battery_Data_Set"

def
    """Download the NASA battery archive if it is not already present."""
    if
        print f"Using existing archive: {
        return

    print f"Downloading NASA archive from {
        =
        =120

    =
    / 1024 * 1024
    print f"Downloaded {
        :.1f} MB to {

def
    """Extract the NASA battery archive if it has not already been
    ↳ extracted."""
    if
        and any
        "*.mat"
        print f"Using existing extracted folder: {
        }"
        return

    =True
    =True

    with
        "r" as

    print f"Extracted archive to {
    }"

```

```

def matlab_datenum_to_timestamp(date_vector):
    """Convert a NASA MATLAB date vector into a pandas Timestamp."""
    flat = np.asarray(date_vector, dtype=float).ravel()
    if flat.size < 6:
        return pd.NaT

    year, month, day, hour, minute = flat[:5].astype(int)
    second = float(flat[5])
    whole_second = int(np.floor(second))
    microsecond = int(round((second - whole_second) * 1_000_000))
    return pd.Timestamp(
        year=year,
        month=month,
        day=day,
        hour=hour,
        minute=minute,
        second=whole_second,
        microsecond=microsecond,
    )

def unwrap_matlab_scalar(value):
    """Repeatedly unwrap one-element MATLAB object arrays."""
    current = value
    while isinstance(current, np.ndarray) and current.size == 1:
        current = current.item()
    return current

def get_struct_field(struct_object, field_name):
    """Read a named field from a scipy-loaded MATLAB struct."""
    field_value = struct_object[field_name]
    return unwrap_matlab_scalar(field_value)

def parse_discharge_cycles(mat_path):
    """Return one row per discharge cycle from a NASA cell .mat file."""
    mat = loadmat(mat_path, squeeze_me=False, struct_as_record=False)
    cell_name = mat_path.stem
    if cell_name not in mat:
        raise KeyError(f"Expected variable {cell_name} inside  
↳ {mat_path.name}")

```

```

cell_struct = unwrap_matlab_scalar(mat[cell_name])
cycles = np.asarray(cell_struct.cycle).ravel()
rows = []

discharge_index = 0
for raw_cycle_index, cycle in enumerate(cycles, start=1):
    cycle_type = str(unwrap_matlab_scalar(cycle.type))
    if cycle_type.lower() != "discharge":
        continue

    discharge_index += 1
    data = unwrap_matlab_scalar(cycle.data)
    capacity_ah = float(np.asarray(data.Capacity).ravel()[0])
    voltage_v = np.asarray(data.Voltage_measured, dtype=float).ravel()
    current_a = np.asarray(data.Current_measured, dtype=float).ravel()
    temperature_c = np.asarray(data.Temperature_measured,
    dtype=float).ravel()
    time_s = np.asarray(data.Time, dtype=float).ravel()

    rows.append(
        {
            "cell_id": cell_name,
            "raw_cycle_index": raw_cycle_index,
            "discharge_index": discharge_index,
            "start_time": matlab_datenum_to_timestamp(cycle.time),
            "ambient_temperature_c": float(
                np.asarray(cycle.ambient_temperature).ravel()[0]
            ),
            "capacity_ah": capacity_ah,
            "initial_voltage_v": float(voltage_v[0]),
            "final_voltage_v": float(voltage_v[-1]),
            "mean_discharge_current_a": float(np.nanmean(current_a)),
            "mean_temperature_c": float(np.nanmean(temperature_c)),
            "duration_s": float(time_s[-1] - time_s[0]),
            "n_samples": int(time_s.size),
        }
    )

return pd.DataFrame(rows)

download_nasa_archive()
extract_nasa_archive()

mat_files = sorted(NASA_EXTRACT_DIR.rglob("B*.mat"))

```



```

print(f"Found {len(mat_files)} MATLAB files")
print("First files:", [path.name for path in mat_files[:5]])

all_capacity_tables = []
for mat_file in mat_files:
    try:
        table = parse_discharge_cycles(mat_file)
    except Exception as exc:
        print(f"Skipping {mat_file.name}: {exc}")
        continue
    if not table.empty:
        all_capacity_tables.append(table)

capacity_table = pd.concat(all_capacity_tables, ignore_index=True)
capacity_table = capacity_table.sort_values(["cell_id", "discharge_index"])

first_capacity =
    → capacity_table.groupby("cell_id")["capacity_ah"].transform("first")
capacity_table["soh_relative"] = capacity_table["capacity_ah"] /
    → first_capacity
capacity_table["capacity_fade_ah"] = first_capacity -
    → capacity_table["capacity_ah"]

output_path = PROCESSED_DIR / "nasa_capacity_table.csv"
capacity_table.to_csv(output_path, index=False)

print(f"Wrote {len(capacity_table)} discharge rows to {output_path}")
print(capacity_table.head(10).to_string(index=False))

```

The code is longer than a typical data-loading snippet because we are crossing a real boundary between MATLAB's nested structure format and Python's table format. `loadmat` returns MATLAB structs as Python objects when `struct_as_record=False`. The helper `unwrap_matlab_scalar` removes layers of one-element arrays that exist because MATLAB stores scalar structs and strings differently from Python. This is standard practice when parsing MATLAB files with SciPy. It looks fussy because the file format is fussy.

The parser keeps both `raw_cycle_index` and `discharge_index`. The raw cycle index counts every operation: charge, discharge, and impedance. The discharge index counts only usable discharge capacity measurements. For SOH fitting, discharge index is often the cleaner independent variable because capacity is observed only on discharge cycles. For protocol reconstruction, raw cycle index

matters because rest periods and impedance tests are part of the cell's actual history.

Expected output will vary slightly depending on archive mirror and file layout, but you should see something like:

```
Downloading NASA archive from
↳ https://phm-datasets.s3.amazonaws.com/NASA/5.+Battery+Data+Set.zip
Downloaded 55.2 MB to
↳ chapter8_soh_aging_workspace/data/raw/Battery_Data_Set.zip
Extracted archive to
↳ chapter8_soh_aging_workspace/data/raw/NASA_Battery_Data_Set
Found 34 MATLAB files
First files: ['B0005.mat', 'B0006.mat', 'B0007.mat', 'B0018.mat',
↳ 'B0025.mat']
Wrote 616 discharge rows to
↳ chapter8_soh_aging_workspace/data/processed/nasa_capacity_table.csv
cell_id raw_cycle_index discharge_index start_time
↳ ambient_temperature_c capacity_ah
B0005                2                1 2008-04-02 15:25:41
↳ 24.0                1.8565
```

A correct capacity table has one row per discharge cycle and a monotonically increasing `discharge_index` within each `cell_id`. `capacity_ah` should generally decline, but not perfectly. Real aging data can show apparent capacity recovery after rest periods or temperature changes. If every capacity is identical, you parsed the wrong field. If capacities are negative, you accidentally integrated current with the wrong sign rather than using NASA's `Capacity` field.

8.3.1 What could go wrong

The direct download returns an HTTP error. Download the zip manually from the NASA PCoE repository page and place it at `chapter8_soh_aging_workspace/data/raw/Battery_Data_Set.zip`. Then rerun the script; it will skip the download.

Found 0 MATLAB files. Inspect the extracted folder. Some zip tools create an extra nested directory. Change `NASA_EXTRACT_DIR.rglob("B*.mat")` to `NASA_EXTRACT_DIR.rglob("*.mat")` and print the discovered paths.

Expected variable B0005 inside B0005.mat. Some mirrors rename the top-level MATLAB variable. Open the file keys with `print(loadmat(mat_file).keys())` and adjust `cell_name`.

Capacity appears to increase with aging. Check that you are comparing one cell at a time. Do not sort all cells only by cycle number, because different cells have different initial capacities and protocols.

8.3.2 Reflection

This exercise taught the first serious aging lesson: SOH begins as a data-engineering problem. Before fitting, forecasting, or invoking degradation mechanisms, you need a defensible table that says which cell, which cycle, which capacity, which temperature, and which time base you are using. We will reuse `nasa_capacity_table.csv` throughout the chapter.

Guided Walkthrough 2

Fit Empirical Capacity-Fade Models

Learning objective: Fit square-root and power-law capacity-fade models to NASA capacity data and compare their residuals, confidence intervals, and 500-cycle predictions.

Now that we have a clean capacity table, we can fit models. We will choose one NASA cell for a detailed walk-through, then write the code so it can be applied to every cell. The square-root model is the disciplined starting point because it has only two parameters. The power-law model adds a third parameter and often improves fit, but it can extrapolate badly if the observed cycle window is short.

```
from import
import as
import as
import as
from import

= "chapter8_soh_aging_workspace"
= / "data" / "processed"
= / "figures"
= / "results"

=True =True
=True =True

= / "nasa_capacity_table.csv"
```

```

available_cells = sorted(capacity_table["cell_id"].unique())
print("Available cells:", available_cells)

CELL_ID = "B0005" if "B0005" in available_cells else available_cells[0]
cell_data = capacity_table.loc[capacity_table["cell_id"] == CELL_ID].copy()
cell_data = cell_data.sort_values("discharge_index")

cycles = cell_data["discharge_index"].to_numpy(dtype=float)
capacity_ah = cell_data["capacity_ah"].to_numpy(dtype=float)
initial_measured_capacity = capacity_ah[0]

def sqrt_capacity_model(cycle_number, capacity_0, sqrt_fade_coefficient):
    return capacity_0 - sqrt_fade_coefficient * np.sqrt(cycle_number)

def power_law_capacity_model(cycle_number, capacity_0, fade_coefficient,
    exponent):
    return capacity_0 - fade_coefficient * cycle_number ** exponent

# Example usage:
# sqrt_capacity_model(1000, 20000, 0.02)
# power_law_capacity_model(1000, 20000, 0.005, 0.5)

```

```

def rmse(values):
    return float(np.sqrt(np.mean(np.square(values))))

sqrt_rmse = rmse(sqrt_residual)
power_rmse = rmse(power_residual)

sqrt_standard_error = np.sqrt(np.diag(sqrt_covariance))
power_standard_error = np.sqrt(np.diag(power_covariance))

print(f"Cell: {CELL_ID}")
print(
    "Square-root model: "
    f"Q0={sqrt_parameters[0]:.4f} Ah +/- {1.96 * sqrt_standard_error[0]:.4f}, "
    ↪ "
    f"k={sqrt_parameters[1]:.5f} +/- {1.96 * sqrt_standard_error[1]:.5f}, "
    f"RMSE={sqrt_rmse:.5f} Ah"
)
print(
    "Power-law model: "
    f"Q0={power_parameters[0]:.4f} Ah +/- {1.96 *
    ↪ power_standard_error[0]:.4f}, "
    f"k={power_parameters[1]:.5f} +/- {1.96 * power_standard_error[1]:.5f}, "
    f"b={power_parameters[2]:.3f} +/- {1.96 * power_standard_error[2]:.3f}, "
    f"RMSE={power_rmse:.5f} Ah"
)

forecast_cycles = np.arange(1, 501, dtype=float)
sqrt_forecast = sqrt_capacity_model(forecast_cycles, *sqrt_parameters)
power_forecast = power_law_capacity_model(forecast_cycles, *power_parameters)

eol_capacity_ah = 0.70 * initial_measured_capacity

def first_eol_cycle(cycle_grid, capacity_grid, threshold_ah):
    below = np.flatnonzero(capacity_grid <= threshold_ah)
    if below.size == 0:
        return np.nan
    return float(cycle_grid[below[0]])

sqrt_eol = first_eol_cycle(forecast_cycles, sqrt_forecast, eol_capacity_ah)
power_eol = first_eol_cycle(forecast_cycles, power_forecast, eol_capacity_ah)

print(f"Relative 70% EOL threshold: {eol_capacity_ah:.4f} Ah")

```

```

print(f"Square-root forecast EOL cycle: {sqrt_eol}")
print(f"Power-law forecast EOL cycle: {power_eol}")

fig, axes = plt.subplots(2, 1, figsize=(8.0, 7.0), sharex=False)

axes[0].plot(cycles, capacity_ah, "o", label="NASA discharge capacity",
             ↪ markersize=4)
axes[0].plot(forecast_cycles, sqrt_forecast, "--", label="Square-root fit")
axes[0].plot(forecast_cycles, power_forecast, "--", label="Power-law fit")
axes[0].axhline(eol_capacity_ah, color="black", linestyle=":", label="70%
             ↪ relative EOL")
axes[0].set_xlabel("Discharge cycle number")
axes[0].set_ylabel("Capacity (Ah)")
axes[0].set_title(f"Capacity fade model comparison for NASA cell {CELL_ID}")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

axes[1].plot(cycles, sqrt_residual * 1000, "o-", label="Square-root
             ↪ residual")
axes[1].plot(cycles, power_residual * 1000, "s-", label="Power-law residual")
axes[1].axhline(0.0, color="black", linewidth=1.0)
axes[1].set_xlabel("Discharge cycle number")
axes[1].set_ylabel("Residual (mAh)")
axes[1].grid(True, alpha=0.3)
axes[1].legend()

fig.tight_layout()
figure_path = FIGURE_DIR / f"{CELL_ID}_capacity_fade_models.png"
fig.savefig(figure_path, dpi=200)
print(f"Saved figure to {figure_path}")

fit_summary = pd.DataFrame(
    [
        {
            "cell_id": CELL_ID,
            "model": "sqrt",
            "capacity_0_ah": sqrt_parameters[0],
            "fade_coefficient": sqrt_parameters[1],
            "exponent": 0.5,
            "rmse_ah": sqrt_rmse,
            "forecast_eol_cycle": sqrt_eol,
        },
        {
            "cell_id": CELL_ID,
            "model": "power_law",

```

```

        "capacity_0_ah": power_parameters[0],
        "fade_coefficient": power_parameters[1],
        "exponent": power_parameters[2],
        "rmse_ah": power_rmse,
        "forecast_eol_cycle": power_eol,
    },
]
)
fit_summary.to_csv(RESULTS_DIR / f"{CELL_ID}_aging_fit_summary.csv",
    ↪ index=False)
print(fit_summary.to_string(index=False))

```

The `curve_fit` calls estimate model parameters by minimizing squared voltage-independent capacity errors. The bounds matter. Without bounds, the power-law model can choose a negative fade coefficient and an odd exponent that fits early noise but violates physical monotonic fade. Bounds do not make the model true, but they prevent obviously nonphysical parameter combinations.

The plot has two panels. The top panel shows capacity in ampere-hours versus discharge cycle number. The NASA measurements should appear as discrete markers. The square-root fit should be a smooth concave-down curve. The power-law fit may lie closer to the data if the cell has a fade trend that is not exactly square-root. The horizontal dotted line marks 70% of the first measured capacity, a relative EOL definition. The bottom panel shows residuals in mAh. Correct residuals should scatter around zero rather than show a large systematic wave. A systematic wave means the model is missing a regime change, a protocol effect, or a recovery effect.

Expected terminal output will look like this in structure:

```

Available cells: ['B0005', 'B0006', 'B0007', 'B0018', 'B0025']
Cell: B0005
Square-root model: Q0=1.9021 Ah +/- 0.0184, k=0.02416 +/- 0.00172,
    ↪ RMSE=0.01890 Ah
Power-law model: Q0=1.8726 Ah +/- 0.0127, k=0.00682 +/- 0.00240, b=0.756 +/-
    ↪ 0.071, RMSE=0.01480 Ah
Relative 70% EOL threshold: 1.2996 Ah
Square-root forecast EOL cycle: 487.0
Power-law forecast EOL cycle: 331.0
Saved figure to
    ↪ chapter8_soh_aging_workspace/figures/B0005_capacity_fade_models.png

```

Do not expect both models to predict the same EOL cycle. That difference is the point. Long-horizon aging forecasts can be dominated by model form even when short-horizon RMSE differs only slightly.

8.3.3 What could go wrong

The power-law fit returns an exponent at the bound. That usually means the dataset does not support a freely fitted exponent. Report that honestly and prefer the simpler square-root model.

The fitted initial capacity is larger than any measured capacity. That is normal because Q_0 is an extrapolated intercept. It becomes suspicious only if it is far outside the plausible rated-capacity range.

The forecast capacity becomes negative before 500 cycles. Your power-law exponent or coefficient is too aggressive for extrapolation. Restrict the forecast horizon or use a more physically constrained model.

Residuals show step changes. Investigate temperature, rest duration, and protocol changes. A single smooth model may be inappropriate across all regimes.

8.3.4 Reflection

This exercise separated fitting from forecasting. Fitting asks how well a model explains observed data. Forecasting asks what the model implies outside the observed window. Reviewers care about that distinction because many aging models look excellent on a known history and fail when extrapolated.

Guided Walkthrough 3

Add Temperature and Current Stress Terms

Learning objective: Extend the empirical model into a semi-empirical stress model and learn when the dataset cannot identify a parameter.

The NASA cells were cycled at different temperatures and discharge conditions. That makes the dataset useful for health-model validation, but it does not automatically make every stress term identifiable. A common mistake is to fit an Arrhenius activation energy from one cell or one narrow temperature range. We will instead fit a simple pooled model across cells and treat the result as an apparent stress correlation, not a universal mechanism.

We will use the table fields `ambient_temperature_c` and `mean_discharge_current_a`. NASA current sign conventions can vary by field and file. We will use the absolute value of the mean discharge current as a stress magnitude. This is standard practice for a capacity-fade correlation, but it hides charge/discharge asymmetry. In a paper, state that choice.

```

from          import
import        as
import        as
from          import
import        as

        = "chapter8_soh_aging_workspace"
        = / "data" / "processed"
    =      / "figures"
        = / "results"

        = / "nasa_capacity_table.csv"
capacity_table = "cell_id"
               "discharge_index"

capacity_table["cycle_scaled"] =
    ↳ capacity_table["discharge_index"].astype(float)
capacity_table["temperature_k"] = capacity_table["ambient_temperature_c"] +
    ↳ 273.15
capacity_table["current_stress_a"] =
    ↳ capacity_table["mean_discharge_current_a"].abs()
capacity_table["initial_capacity_ah"] = capacity_table.groupby("cell_id")[
    "capacity_ah"
].transform("first")
capacity_table["fade_ah"] = (
    capacity_table["initial_capacity_ah"] - capacity_table["capacity_ah"]
)

fit_data = capacity_table.dropna(
    subset=["fade_ah", "cycle_scaled", "temperature_k", "current_stress_a"]
).copy()
fit_data = fit_data.loc[fit_data["cycle_scaled"] > 0]

gas_constant = 8.314462618
reference_temperature_k = 298.15
reference_current_a = fit_data["current_stress_a"].median()

```

```

def semi_empirical_fade(parameters, table):
    log_k_ref, activation_energy_j_mol, current_exponent, cycle_exponent =
    ↪ parameters
    temperature_factor = np.exp(
        (-activation_energy_j_mol / gas_constant)
        * ((1.0 / table["temperature_k"].to_numpy()) - (1.0 /
            ↪ reference_temperature_k))
    )
    current_factor = np.power(
        table["current_stress_a"].to_numpy() / reference_current_a,
        current_exponent,
    )
    cycle_factor = np.power(table["cycle_scaled"].to_numpy(), cycle_exponent)
    return np.exp(log_k_ref) * temperature_factor * current_factor *
    ↪ cycle_factor

def residual_function(parameters, table):
    predicted_fade_ah = semi_empirical_fade(parameters, table)
    measured_fade_ah = table["fade_ah"].to_numpy()
    return predicted_fade_ah - measured_fade_ah

initial_guess = np.array([
    np.log(0.01),
    25_000.0,
    0.5,
    0.5,
])

lower_bounds = np.array([
    np.log(1e-5),
    0.0,
    0.0,
    0.1,
])

upper_bounds = np.array([
    np.log(1.0),
    80_000.0,
    3.0,
    1.5,
])

result = least_squares(

```

```

    residual_function,
    initial_guess,
    args=(fit_data,),
    bounds=(lower_bounds, upper_bounds),
    loss="soft_l1",
    f_scale=0.02,
    max_nfev=20_000,
)

log_k_ref, activation_energy_j_mol, current_exponent, cycle_exponent =
    ↪ result.x
fit_data["predicted_fade_ah"] = semi_empirical_fade(result.x, fit_data)
fit_data["predicted_capacity_ah"] = (
    fit_data["initial_capacity_ah"] - fit_data["predicted_fade_ah"]
)
fit_data["residual_mah"] = (
    fit_data["capacity_ah"] - fit_data["predicted_capacity_ah"]
) * 1000

rmse_ah = np.sqrt(
    np.mean((fit_data["capacity_ah"] - fit_data["predicted_capacity_ah"]) **
    ↪ 2)
)

print("Semi-empirical pooled stress model")
print(f"k_ref: {np.exp(log_k_ref):.6f} Ah/cycle^b")
print(f"apparent activation energy: {activation_energy_j_mol / 1000:.2f}
    ↪ kJ/mol")
print(f"current exponent: {current_exponent:.3f}")
print(f"cycle exponent: {cycle_exponent:.3f}")
print(f"capacity RMSE: {rmse_ah:.5f} Ah")
print(f"optimizer success: {result.success}, message: {result.message}")

summary_path = RESULTS_DIR / "nasa_pooled_stress_model_predictions.csv"
fit_data.to_csv(summary_path, index=False)
print(f"Wrote predictions to {summary_path}")

fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.5))

for cell_id, group in fit_data.groupby("cell_id"):
    axes[0].plot(
        group["discharge_index"],
        group["capacity_ah"],
        "o",
        markersize=3,

```

```

        label=f"{cell_id} measured",
        alpha=0.65,
    )
    axes[0].plot(
        group["discharge_index"],
        group["predicted_capacity_ah"],
        "-",
        linewidth=1.4,
        label=f"{cell_id} predicted",
    )

axes[0].set_xlabel("Discharge cycle number")
axes[0].set_ylabel("Capacity (Ah)")
axes[0].set_title("Pooled semi-empirical aging model")
axes[0].grid(True, alpha=0.3)
axes[0].legend(fontsize=7, ncol=2)

axes[1].hist(fit_data["residual_mah"], bins=30, edgecolor="black")
axes[1].axvline(0.0, color="black", linewidth=1.0)
axes[1].set_xlabel("Capacity residual (mAh)")
axes[1].set_ylabel("Count")
axes[1].set_title("Residual distribution")
axes[1].grid(True, alpha=0.3)

fig.tight_layout()
figure_path = FIGURE_DIR / "nasa_pooled_stress_model.png"
fig.savefig(figure_path, dpi=200)
print(f"Saved figure to {figure_path}")

```

The temperature factor in this model is written relative to a reference temperature so that `k_ref` remains numerically reasonable. The expression is equivalent to an Arrhenius scaling, but centered at 298.15 K. The robust `soft_l1` loss reduces the influence of occasional apparent capacity recovery or anomalous cycles. This is one of several valid approaches. Another defensible approach would be to identify and remove outlier cycles before fitting with ordinary least squares. The important habit is to state the choice.

The expected figure has measured and predicted capacity traces for several cells on the left. A good model captures broad trends but will not pass through every point. The right panel shows residuals in mAh. A narrow residual distribution centered near zero is good. A strongly skewed distribution suggests systematic underprediction or overprediction. Separate residual clusters often mean different

cells or protocols need separate model terms.

The printed apparent activation energy should be treated carefully. If the optimizer pushes it to 0 or 80 kJ/mol, the dataset and model form are not identifying it. That is a result, not a failure. It means your paper should not claim a measured activation energy from this fit.

8.3.5 What could go wrong

The optimizer succeeds but the parameters are physically strange. Check whether all cells have the same ambient temperature or current stress. Parameters cannot be identified without variation.

The model predicts negative fade for early cycles. This implementation cannot predict negative fade because the model is multiplicative and positive. If your modified model does, constrain it.

The current exponent is meaningless. NASA discharge current is only one part of the stress history. Charge protocol and rest time also matter. Do not over-interpret a current exponent fitted from discharge summary rows alone.

8.3.6 Reflection

This exercise taught a subtle research lesson: adding physics-sounding terms does not guarantee physical identification. Semi-empirical aging models are useful, but only when the dataset contains the variation needed to support the terms you fit.

Guided Walkthrough 4

Simulate SEI-Driven Aging in PyBaMM

Learning objective: Run a PyBaMM lithium-ion model with an SEI degradation option and extract capacity-relevant aging variables over repeated cycles.

PyBaMM lets us express degradation through model options rather than manually coding every side reaction. In Textbook Chapter 7, SEI growth was a parasitic reaction that consumes cyclable inventory. In PyBaMM, selecting an SEI option adds the corresponding submodel to the electrochemical model. The exact list of available options can change across PyBaMM releases, so we will use a defensive pattern: create a DFN model with a conservative SEI option, run a short repeated experiment, and print available degradation variables before plotting.

This example is lithium-ion because PyBaMM's built-in parameter sets are much richer for Li-ion than for sodium-ion. The workflow is still valuable for sodium-ion research. Once you have a sodium-ion parameter set and side-reaction parameters, the same simulation structure applies.

```

from pybamm import import
import pybamm as pybamm
import pybamm as pybamm
import pybamm as pybamm

# Create a workspace for the simulation
workspace = "chapter8_soh_aging_workspace"
figures = workspace / "figures"
results = workspace / "results"

# Set the model and parameters
model = pybamm.lithium_ion.DFN
parameters = pybamm.ParameterValues("0Kane2022")

# Set the solver
solver = pybamm.CasadiSolver("safe")

# Set the simulation options
sim = pybamm.Simulation(model, parameters, solver, {"SEI": "solvent-diffusion limited",
"SEI porosity change": "true"})

# Set the simulation time
sim.solve([0, 20])

# Save the results
sim.save_results(results)

```

```

    model,
    parameter_values=parameter_values,
    experiment=experiment,
    solver=solver,
)

solution = simulation.solve()

print("Simulation solved.")
print(f"Number of cycles in solution: {len(solution.cycles)}")

candidate_variables = [
    "Loss of lithium inventory [%]",
    "Loss of capacity to negative SEI [A.h]",
    "X-averaged negative SEI thickness [m]",
    "Terminal voltage [V]",
    "Current [A]",
    "Discharge capacity [A.h]",
]

available_variables = []
for variable_name in candidate_variables:
    if variable_name in solution.all_models[0].variables:
        available_variables.append(variable_name)
        print(f"Available: {variable_name}")
    else:
        print(f"Not available in this PyBaMM version/model: {variable_name}")

cycle_rows = []
for cycle_number, cycle_solution in enumerate(solution.cycles, start=1):
    row = {"cycle_number": cycle_number}

    for variable_name in available_variables:
        variable = cycle_solution[variable_name]
        entries = np.asarray(variable.entries).ravel()
        if entries.size == 0:
            continue
        safe_name = (
            variable_name.lower()
            .replace(" ", "_")
            .replace("[", "")
            .replace("]", "")
            .replace(".", "")
            .replace("/", "_per_")
            .replace("%", "percent")

```

```

    )
    row[f"final_{safe_name}"] = float(entries[-1])

    cycle_rows.append(row)

cycle_summary = pd.DataFrame(cycle_rows)
summary_path = RESULTS_DIR / "pybamm_sei_cycle_summary.csv"
cycle_summary.to_csv(summary_path, index=False)
print(cycle_summary.tail().to_string(index=False))
print(f"Wrote PyBaMM cycle summary to {summary_path}")

fig, axes = plt.subplots(2, 1, figsize=(8.0, 7.0), sharex=True)

plot_columns = [
    column for column in cycle_summary.columns
    if "loss_of_lithium_inventory" in column
]

if plot_columns:
    axes[0].plot(
        cycle_summary["cycle_number"],
        cycle_summary[plot_columns[0]],
        "o-",
        linewidth=1.8,
    )
    axes[0].set_ylabel("LLI (%)")
else:
    axes[0].text(
        0.5,
        0.5,
        "LLI variable not available",
        ha="center",
        va="center",
        transform=axes[0].transAxes,
    )
    axes[0].set_ylabel("Diagnostic")

sei_columns = [column for column in cycle_summary.columns if "sei_thickness"
    ↪ in column]
if
    1
        "cycle_number"
        0 * 1e9
    "s-"
        =1.8

```



```

    )
    axes[1].set_ylabel("SEI thickness (nm)")
else:
    axes[1].text(
        0.5,
        0.5,
        "SEI thickness variable not available",
        ha="center",
        va="center",
        transform=axes[1].transAxes,
    )
    axes[1].set_ylabel("Diagnostic")

axes[1].set_xlabel("Cycle number")
for axis in axes:
    axis.grid(True, alpha=0.3)

fig.suptitle("PyBaMM DFN simulation with SEI aging")
fig.tight_layout()
figure_path = FIGURE_DIR / "pybamm_sei_aging_summary.png"
fig.savefig(figure_path, dpi=200)
print(f"Saved figure to {figure_path}")

```

The important line is the model construction:

```

model = pybamm.lithium_ion.DFN(
    {
        "SEI": "solvent-diffusion limited",
        "SEI porosity change": "true",
    }
)

```

That dictionary changes the governing model by adding degradation physics. The parameter set `OKane2022` is chosen because it was developed for degradation studies and is a better starting point for SEI-related simulations than a purely fresh-cell parameter set. The experiment repeats a discharge, rest, charge, CV hold, and rest sequence 20 times. Twenty cycles is not enough for a lifetime study, but it is enough to verify that the workflow produces monotonically evolving degradation variables.

The expected plot has two panels. The first panel should show loss of lithium inventory increasing with cycle number if that variable is available. The second should show SEI thickness increasing, usually slowly and monotonically. If the

axes are flat, the degradation rate may be too small over 20 cycles or the plotted variable is not the right one for your PyBaMM version. Increase the cycle count only after you know the short run works.

8.3.7 What could go wrong

PyBaMM raises an option error. Print `pybamm.lithium_ion.DFN().options.possible_options` or consult the installed PyBaMM docs for your version. Degradation option names have changed across releases.

The solver fails midway through the experiment. Reduce `dt_max`, use `mode="safe"`, or run fewer cycles. Solver failures in aging simulations often come from stiff side reactions and voltage cutoffs interacting.

The run takes too long. Start with the SPM or SPM_e using the same degradation options where supported. Use the DFN only after you know which variables and protocol you need.

No capacity fade appears. Degradation over 20 cycles may be small. Plot internal degradation variables first, then scale the cycle count.

8.3.8 Reflection

This exercise showed the difference between fitting observed SOH and simulating a degradation mechanism. PyBaMM gives mechanistic access, but that access comes with parameter and solver responsibilities. You should now be able to run a small degradation simulation, extract internal health variables, and decide whether the result is numerically plausible.

Guided Walkthrough 5

Implement the Same Aging Model in MATLAB

Learning objective: Reproduce the fitted empirical capacity-fade model in MATLAB and generate a 500-cycle SOH forecast.

MATLAB remains common in BMS workflows, and your Chapter 6 and Chapter 7 scripts are already in MATLAB. Here we implement the square-root and power-law models from Walkthrough 2 using a small CSV exported from Python. This mirrors a realistic workflow: Python parses messy public data; MATLAB uses

the fitted parameters inside a controls or estimator prototype.

First, make sure the Python script from Walkthrough 2 wrote `chapter8_soh_aging_workspace/results/B0005_aging_fit_summary.csv` or the corresponding file for your chosen cell. Then run this MATLAB script from the repository root.

```
clear; close all; clc;

chapterRoot = fullfile pwd, "chapter8_soh_aging_workspace" ;
resultsDir = fullfile chapterRoot, "results" ;
figuresDir = fullfile chapterRoot, "figures" ;

if ~exist figuresDir, "dir"
    mkdir figuresDir ;
end

fitFiles = dir(fullfile resultsDir, "*_aging_fit_summary.csv" );
if isempty fitFiles
    error "No aging fit summary CSV found. Run the Python fitting walkthrough first." ;
end

fitPath = fullfile(fitFiles(1).folder, fitFiles(1).name);
fitSummary = readtable(fitPath, "TextType", "string");
disp(fitSummary);

sqrtRow = fitSummary(fitSummary.model == "sqrt", :);
powerRow = fitSummary(fitSummary.model == "power_law", :);

cycleNumber = (1:500)';

sqrtCapacityAh = sqrtRow.capacity_0_ah ...
    - sqrtRow.fade_coefficient .* sqrt(cycleNumber);

powerCapacityAh = powerRow.capacity_0_ah ...
    - powerRow.fade_coefficient .* cycleNumber .^ powerRow.exponent;

initialCapacityAh = sqrtCapacityAh(1);
sqrtSoh = sqrtCapacityAh ./ initialCapacityAh;
powerSoh = powerCapacityAh ./ initialCapacityAh;

eolThreshold = 0.70;
sqrtEolIndex = find(sqrtSoh <= eolThreshold, 1, "first");
powerEolIndex = find(powerSoh <= eolThreshold, 1, "first");
```

```

if isempty(sqrtEolIndex)
    sqrtEolText = "not reached by cycle 500";
else
    sqrtEolText = "cycle " + string(sqrtEolIndex);
end

if isempty(powerEolIndex)
    powerEolText = "not reached by cycle 500";
else
    powerEolText = "cycle " + string(powerEolIndex);
end

fprintf("Square-root model EOL: %s\n", sqrtEolText);
fprintf("Power-law model EOL: %s\n", powerEolText);

forecastTable = table( ...
    cycleNumber, ...
    sqrtCapacityAh, ...
    powerCapacityAh, ...
    sqrtSoh, ...
    powerSoh);

forecastPath = fullfile(resultsDir, "matlab_500_cycle_forecast.csv");
writetable(forecastTable, forecastPath);
fprintf("Wrote MATLAB forecast to %s\n", forecastPath);

figure("Color", "w");
tiledlayout(2, 1);

nexttile;
plot(cycleNumber, sqrtCapacityAh, "LineWidth", 1.8);
hold on;
plot(cycleNumber, powerCapacityAh, "--", "LineWidth", 1.8);
yline(0.70 * initialCapacityAh, ":", "70% relative EOL");
grid on;
xlabel("Cycle number");
ylabel("Capacity (Ah)");
title("MATLAB 500-cycle capacity forecast");
legend("Square-root model", "Power-law model", "Location", "best");

nexttile;
plot(cycleNumber, 100 * sqrtSoh, "LineWidth", 1.8);
hold on;
plot(cycleNumber, 100 * powerSoh, "--", "LineWidth", 1.8);

```

```

yline(70, ":", "70% SOH");
grid on;
xlabel("Cycle number");
ylabel("SOH (%)");
title("SOH forecast");
legend("Square-root model", "Power-law model", "Location", "best");

figurePath = fullfile(figuresDir, "matlab_500_cycle_soh_forecast.png");
exportgraphics(gcf, figurePath, "Resolution", 200);
fprintf("Saved figure to %s\n", figurePath);

```

The MATLAB code is intentionally close to the equations. `sqrtCapacityAh` implements Equation (1). `powerCapacityAh` implements Equation (2). The `find` calls determine whether the forecast crosses 70% SOH within 500 cycles. This is not a probabilistic RUL estimate; it is a deterministic forecast under the fitted model.

The expected figure has two panels. The upper panel shows capacity in ampere-hours for cycles 1 through 500. The lower panel shows SOH in percent. If the power-law exponent is larger than 0.5, the dashed power-law curve may fall faster than the square-root curve at long cycle counts. If the exponent is smaller than 0.5, it may fade more slowly. The gap between curves is a visual reminder that extrapolation depends heavily on model form.

8.3.9 What could go wrong

MATLAB cannot find the CSV. Check your current folder with `pwd`. The script assumes you run from the same root that contains `chapter8_soh_aging_workspace`.

`readtable` imports model names as cell arrays. Use a newer MATLAB release or replace `fitSummary.model == "sqrt"` with `strcmp(fitSummary.model, "sqrt")`.

The forecast starts below 100% SOH. We define SOH relative to `sqrtCapacityAh(1)`, not the fitted intercept at cycle 0. If you prefer rated capacity, replace `initialCapacityAh` with the rated value and state that choice.

8.3.10 Reflection

This exercise made the Python-MATLAB boundary concrete. For research, you should not maintain two independent aging models by hand. Fit once, export

parameters, and consume those parameters in the second environment with a short, auditable script.

Guided Walkthrough 6

SOH from Partial Curves with ICA and DVA

Learning objective: Compute incremental-capacity and differential-voltage features from NASA discharge curves and track how the features change with aging.

Capacity fade is not always available directly. In field systems, we may only observe partial charge or discharge segments. ICA and DVA help extract health-sensitive features from voltage-capacity curves. The method is numerically delicate because differentiation amplifies noise. We will therefore smooth the voltage-capacity curve before differentiating.

NASA discharge data include measured voltage, current, time, and capacity. We will reconstruct cumulative discharged capacity from current and time so the method does not depend on the scalar `Capacity` field. This is also where sign convention matters. We use absolute current magnitude because discharge current may be stored as positive or negative depending on source.

```
from import
import as
import as
import as
from import
from import

    = "chapter8_soh_aging_workspace"
    = / "data" / "raw"
    = / "figures"
    = / "results"

    = / "NASA_Battery_Data_Set"

def
    =
    while isinstance and == 1
```

```

        current = current.item()
    return current

def load_discharge_time_series(mat_path, selected_discharge_numbers):
    mat = loadmat(mat_path, squeeze_me=False, struct_as_record=False)
    cell_name = mat_path.stem
    cell_struct = unwrap_matlab_scalar(mat[cell_name])
    cycles = np.asarray(cell_struct.cycle).ravel()

    selected = {}
    discharge_index = 0
    for raw_cycle_index, cycle in enumerate(cycles, start=1):
        cycle_type = str(unwrap_matlab_scalar(cycle.type)).lower()
        if cycle_type != "discharge":
            continue

        discharge_index += 1
        if discharge_index not in selected_discharge_numbers:
            continue

        data = unwrap_matlab_scalar(cycle.data)
        time_s = np.asarray(data.Time, dtype=float).ravel()
        voltage_v = np.asarray(data.Voltage_measured, dtype=float).ravel()
        current_a = np.asarray(data.Current_measured, dtype=float).ravel()
        temperature_c = np.asarray(data.Temperature_measured,
        ↪ dtype=float).ravel()

        =
        =
        = 0
        abs
        *
        / 3600.0
        -=
        0
        =

        "time_s"
        "voltage_v"
        "current_a"
        "temperature_c"
        "discharged_capacity_ah"

    return

```

```

def compute_ica_dva(curve, voltage_window=(2.8, 4.1), n_grid=600):
    clean = curve.dropna(subset=["voltage_v",
↳ "discharged_capacity_ah"]).copy()
    clean["remaining_capacity_ah"] = (
        clean["discharged_capacity_ah"].max() -
↳ clean["discharged_capacity_ah"]
    )
    clean = clean.sort_values("voltage_v")
    clean = clean.drop_duplicates(subset="voltage_v")
    clean = clean.loc[
        (clean["voltage_v"] >= voltage_window[0])
        & (clean["voltage_v"] <= voltage_window[1])
    ]

    voltage_grid = np.linspace(
        clean["voltage_v"].min(),
        clean["voltage_v"].max(),
        n_grid,
    )
    capacity_grid = np.interp(
        voltage_grid,
        clean["voltage_v"].to_numpy(),
        clean["remaining_capacity_ah"].to_numpy(),
    )

    window_length = 51
    if window_length >= n_grid:
        window_length = n_grid - 1 if n_grid % 2 == 0 else n_grid
    if window_length % 2 == 0:
        window_length += 1

    smoothed_capacity = savgol_filter(
        capacity_grid,
        window_length=window_length,
        polyorder=3,
    )

    dqdv = np.gradient(smoothed_capacity, voltage_grid)
    dvdq = np.gradient(voltage_grid, smoothed_capacity)

    return pd.DataFrame(
        {
            "voltage_v": voltage_grid,
            "remaining_capacity_ah": smoothed_capacity,
            "dqdv_ah_per_v": dqdv,

```



```

        "dvdq_v_per_ah": dvdq,
    }
)

capacity_table = pd.read_csv(CHAPTER_ROOT / "data" / "processed" /
    ↪ "nasa_capacity_table.csv")
available_cells = sorted(capacity_table["cell_id"].unique())
cell_id = "B0005" if "B0005" in available_cells else available_cells[0]
mat_candidates = sorted(NASA_EXTRACT_DIR.rglob(f"{cell_id}.mat"))
if not mat_candidates:
    raise FileNotFoundError(f"Could not find {cell_id}.mat inside
    ↪ {NASA_EXTRACT_DIR}")

cell_capacity = capacity_table.loc[capacity_table["cell_id"] == cell_id]
first_cycle = int(cell_capacity["discharge_index"].min())
middle_cycle = int(cell_capacity["discharge_index"].median())
last_cycle = int(cell_capacity["discharge_index"].max())
selected_cycles = [first_cycle, middle_cycle, last_cycle]

curves = load_discharge_time_series(mat_candidates[0], selected_cycles)
ica_tables = {}
feature_rows = []

for discharge_index, curve in curves.items():
    ica_table = compute_ica_dva(curve)
    ica_tables[discharge_index] = ica_table

    peak_index = int(np.nanargmax(ica_table["dqdv_ah_per_v"].to_numpy()))
    peak_voltage = float(ica_table["voltage_v"].iloc[peak_index])
    peak_height = float(ica_table["dqdv_ah_per_v"].iloc[peak_index])

    feature_rows.append(
        {
            "cell_id": cell_id,
            "discharge_index": discharge_index,
            "ica_peak_voltage_v": peak_voltage,
            "ica_peak_height_ah_per_v": peak_height,
        }
    )

features = pd.DataFrame(feature_rows)
features_path = RESULTS_DIR / f"{cell_id}_ica_features.csv"
features.to_csv(features_path, index=False)
print(features.to_string(index=False))

```

```

print(f"Wrote ICA features to {features_path}")

fig, axes = plt.subplots(2, 1, figsize=(8.0, 7.0), sharex=True)

for discharge_index, table in ica_tables.items():
    label = f"Discharge {discharge_index}"
    axes[0].plot(table["voltage_v"], table["dqdv_ah_per_v"], label=label)
    axes[1].plot(table["voltage_v"], table["dvdq_v_per_ah"], label=label)

axes[0].set_ylabel("dQ/dV (Ah/V)")
axes[0].set_title(f"Incremental-capacity aging features for {cell_id}")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

axes[1].set_xlabel("Voltage (V)")
axes[1].set_ylabel("dV/dQ (V/Ah)")
axes[1].set_title("Differential-voltage curves")
axes[1].grid(True, alpha=0.3)
axes[1].legend()

fig.tight_layout()
figure_path = FIGURE_DIR / f"{cell_id}_ica_dva_curves.png"
fig.savefig(figure_path, dpi=200)
print(f"Saved figure to {figure_path}")

```

The function `compute_ica_dva` first converts discharged capacity into remaining capacity. That step handles the discharge sign convention: as voltage increases along the sorted curve, remaining capacity should also increase, giving positive dQ/dV peaks. Sorting then lets us interpolate capacity as a function of voltage on a uniform voltage grid. The Savitzky-Golay filter smooths capacity before differentiation. The chosen `window_length=51` is a teaching value, not a universal constant. Too small a window produces noisy derivatives; too large a window erases real peaks.

The expected ICA plot shows one or more broad peaks in dQ/dV versus voltage. As the cell ages, peaks may shift in voltage, change height, or broaden. The DVA plot often looks sharper and can show large spikes if the capacity grid has flat or noisy regions. If your DVA panel is dominated by extreme vertical spikes, increase smoothing or restrict the voltage window.

The feature table should have three rows: early, middle, and late discharge. A useful feature changes systematically with aging. If peak voltage moves randomly,

the feature may be too sensitive to noise, the voltage window may be wrong, or the selected cycles may include protocol differences.

8.3.11 What could go wrong

np.gradient produces infinities. The smoothed capacity curve has repeated values, so dV/dQ divides by an almost-zero capacity difference. Increase smoothing or use ICA only for that segment.

ICA peaks look upside down. You may have sorted discharge data in a way that reverses capacity direction. Inspect `capacity_grid` and ensure it increases over the voltage grid used for differentiation.

Peak features change because temperature changed. ICA/DVA features are temperature-sensitive. Do not compare curves at different temperatures without saying so.

8.3.12 Reflection

This exercise introduced SOH estimation from partial curves. It is not a replacement for capacity measurement, but it is a powerful diagnostic family. Later, in the specialization chapter, these features can become inputs to data-driven SOH models.

8.4 Dataset Integration Notes

The NASA Battery Aging dataset is public and widely used for prognostics benchmarking. NASA's repository page identifies it as the "Battery Data Set" from B. Saha and K. Goebel at NASA Ames, with the direct archive link <https://phm-datasets.s3.amazonaws.com/NASA/5.+Battery+Data+Set.zip>. The NASA Open Data listing describes the experiments as charge, discharge, and impedance operations at different temperatures, with discharges to preset voltage thresholds and an EOL criterion of 30% fade from 2 Ah to 1.4 Ah.

The raw files are MATLAB `.mat` files. The top-level object is a cell-specific structure such as `B0005`. Inside it, `cycle` is a structure array. Each cycle has a `type`, commonly `charge`, `discharge`, or `impedance`. Discharge cycles include measured voltage, current, temperature, current at load, voltage at load, time, and a scalar capacity. The most common pitfalls are timestamp conversion, current sign convention, and comparing raw operation index with discharge-only index.

We handled all three explicitly.

This is lithium-ion data. For sodium-ion research, the method generalizes but the fitted parameters do not. You can validate your data workflow, model-selection procedure, uncertainty reporting, and ICA/DVA code on NASA, then adapt the chemistry-specific interpretation to sodium-ion. The sparse public SIB situation means you may need to digitize capacity-fade curves from papers, use supplementary tables when available, or collaborate with an experimental group. When using digitized SIB data, report digitization error and do not overfit a high-parameter model to extracted points.

Reproduction Exercise

Reproduce a Published Capacity-Fade Figure

Target paper: Kristen A. Severson, Peter M. Attia, Norman Jin, Nicholas Perkins, Benben Jiang, Zi Yang, Michael H. Chen, Muratahan Aykol, Patrick K. Herring, Dimitrios Fraggedakis, Martin Z. Bazant, Stephen J. Harris, William C. Chueh, and Richard D. Braatz, “Data-driven prediction of battery cycle life before capacity degradation,” *Nature Energy*, 2019.

Figure target: Reproduce the central idea of the paper’s capacity-fade trajectories: discharge capacity as a function of cycle number for multiple commercial LiFePO₄/graphite cells, with cells showing different lifetimes under fast-charging protocols. The exact visual style of the journal figure is less important than reproducing the scientific content: many cells show little early capacity loss, then diverge strongly in cycle life.

This reproduction is included because it teaches the most important aging-research habit: capacity curves alone can be misleading early in life. Severson and coauthors showed that early-cycle voltage features can predict cycle life before obvious capacity degradation. We will not reproduce the machine-learning model here; that belongs in Chapter 13’s data-driven SOH track. We will reproduce the capacity trajectory visualization and document every ambiguity.

The dataset associated with the Severson/Attia/Stanford-MIT-Toyota work has been distributed through public project pages and mirrors. Because access URLs and packaging have changed over time, use the official paper and its data availability statement first, then record the exact dataset URL, checksum if available, and date accessed in your research log. If you cannot obtain the dataset

during this chapter, use the NASA capacity table to reproduce the figure style and mark it as a method reproduction rather than a data reproduction.

Here is a complete plotting script that works with a generic capacity table. If you have the Severson table, save it as `severson_capacity_table.csv` with columns `cell_id`, `cycle_number`, and `discharge_capacity_ah`. If not, the script falls back to NASA.

```
from import
import as
import as
import as

    = "chapter8_soh_aging_workspace"
    = / "data" / "processed"
    = / "figures"
    =True =True

    = / "severson_capacity_table.csv"
    = / "nasa_capacity_table.csv"

if
    = "Severson et al. public fast-charge dataset"
    =
    =
    =
    "cycle_number" "cycle"
    "discharge_capacity_ah" "capacity_ah"

    "cell_id" "cycle" "capacity_ah"
else
    = "NASA fallback method reproduction"
    =
    =
    =
    "discharge_index" "cycle"

    "cell_id" "cycle" "capacity_ah"

    =
    = "cell_id" "cycle"
    "initial_capacity_ah" = "cell_id"
    "capacity_ah"
    "first"
```

```

capacity_table["normalized_capacity"] = (
    capacity_table["capacity_ah"] / capacity_table["initial_capacity_ah"]
)

cell_lifetimes = (
    capacity_table.loc[capacity_table["normalized_capacity"] <= 0.80]
    .groupby("cell_id")["cycle"]
    .min()
    .rename("cycle_life_80_percent")
)

capacity_table = capacity_table.merge(
    cell_lifetimes,
    left_on="cell_id",
    right_index=True,
    how="left",
)

ordered_cells = (
    capacity_table.groupby("cell_id")["cycle_life_80_percent"]
    .min()
    .sort_values()
    .index
    .tolist()
)

if not ordered_cells:
    ordered_cells = sorted(capacity_table["cell_id"].unique())

fig, axis = plt.subplots(figsize=(8.5, 5.2))

for cell_id in ordered_cells:
    group = capacity_table.loc[capacity_table["cell_id"] == cell_id]
    lifetime = group["cycle_life_80_percent"].iloc[0]
    if np.isnan(lifetime):
        line_alpha = 0.45
        line_width = 1.0
    else:
        line_alpha = 0.85
        line_width = 1.3

    axis.plot(
        group["cycle"],
        group["normalized_capacity"],
        linewidth=line_width,
        alpha=line_alpha,

```

```

)

axis.axhline(0.80, color="black", linestyle=":", linewidth=1.2)
axis.set_xlabel("Cycle number")
axis.set_ylabel("Discharge capacity normalized to first cycle")
axis.set_title(f"Capacity-fade trajectories: {source_name}")
axis.grid(True, alpha=0.3)
axis.set_ylim(0.65, 1.05)

figure_path = FIGURE_DIR / "reproduction_capacity_fade_trajectories.png"
fig.tight_layout()
fig.savefig(figure_path, dpi=250)

print(f"Source used: {source_name}")
print(f"Number of cells plotted: {capacity_table['cell_id'].nunique()}")
print(f"Saved reproduction figure to {figure_path}")

```

If you use the Severson dataset, a close reproduction should show many capacity trajectories beginning near 1.0 normalized capacity, remaining close together early, and then separating as cells age at different rates. If you use NASA fallback data, the plot will have fewer cells and shorter trajectories. That is not a failed exercise; it is a method reproduction rather than a dataset reproduction. Label it accordingly.

Where the paper is ambiguous, write down your choice. Did you normalize by cycle 1 capacity or nominal capacity? Did you define cycle life at 80% capacity or another threshold? Did you exclude early formation cycles? Did you smooth capacity? A reproduction log that records these choices is more valuable than a plot that looks polished but cannot be audited.

Close enough means the qualitative structure and axis definitions match the paper, and any quantitative differences are explainable by preprocessing choices. It does not mean pixel-perfect recreation of journal typography.

8.5 Open-Ended Exercises

8.5.1 Exercise 1: Model selection across all NASA cells

Fit square-root and power-law models to every NASA cell. Create a table with one row per cell and model containing RMSE, exponent, and 70% EOL forecast. Which model would you choose for a conservative RUL estimate, and why?

Hint: Wrap the fitting code from Walkthrough 2 in a function that accepts a cell-specific dataframe. Use `try` and `except` so that one bad cell does not stop the full sweep.

8.5.2 Exercise 2: Train on early life, test on late life

For one cell, fit the square-root model using only the first 40% of discharge capacity measurements. Forecast the remaining 60%. Report train RMSE, test RMSE, and EOL forecast error if the cell reaches EOL.

Hint: Split by row order, not by random sampling. Aging is a time-series problem.

8.5.3 Exercise 3: ICA feature robustness

Repeat the ICA calculation with Savitzky-Golay window lengths of 21, 51, and 101. How much does the peak voltage move? Is the feature robust enough to use in a paper?

Hint: Store peak voltage and peak height for each window length in a dataframe.

8.5.4 Exercise 4: Sodium-ion stress thought experiment

Take the fitted square-root model and reduce the OCV diagnostic sensitivity by assuming the useful ICA peak height is 50% lower, as might happen in a flatter voltage region. What changes in your SOH estimator design?

Hint: This is a written exercise. Discuss observability, need for longer windows, and the role of temperature and impedance features.

8.6 Worked Solutions

8.6.1 Solution 1

```
from pathlib import Path
import numpy as np
import pandas as pd
from scipy.optimize import curve_fit
```

```
CHAPTER_ROOT = Path("chapter8_soh_aging_workspace")
```



```

capacity_table = pd.read_csv(CHAPTER_ROOT / "data" / "processed" /
    ↪ "nasa_capacity_table.csv")

def
    return - *

def power_law_capacity_model(cycle_number, capacity_0, fade_coefficient,
    ↪ exponent):
    return - *

def
    return float

def
    =
    return if == 0 else float 0

    =
for
    in "cell_id"
        = "discharge_index"
        = "discharge_index" =float
        = "capacity_ah" =float
    if len < 10
        continue

        = 1 501 =float
        = 0.70 * 0

        =

    = 0 0.02
    = 1.0 0.0 2.5 0.5

    = *
    = *

    "cell_id"

```

```

        "model": "sqrt",
        "rmse_ah": rmse(capacity - sqrt_fit),
        "exponent": 0.5,
        "forecast_eol_cycle": first_eol_cycle(
            forecast_cycles, sqrt_forecast, eol_threshold
        ),
    }
)

try:
    power_parameters, _ = curve_fit(
        power_law_capacity_model,
        cycles,
        capacity,
        p0=[capacity[0], 0.02, 0.5],
        bounds=([1.0, 0.0, 0.05], [2.5, 1.0, 2.0]),
        maxfev=20_000,
    )
    power_fit = power_law_capacity_model(cycles, *power_parameters)
    power_forecast = power_law_capacity_model(forecast_cycles,
↳ *power_parameters)
    rows.append(
        {
            "cell_id": cell_id,
            "model": "power_law",
            "rmse_ah": rmse(capacity - power_fit),
            "exponent": power_parameters[2],
            "forecast_eol_cycle": first_eol_cycle(
                forecast_cycles, power_forecast, eol_threshold
            ),
        }
    )
except RuntimeError:
    rows.append(
        {
            "cell_id": cell_id,
            "model": "power_law",
            "rmse_ah": np.nan,
            "exponent": np.nan,
            "forecast_eol_cycle": np.nan,
        }
    )

model_selection = pd.DataFrame(rows)
print(model_selection.sort_values(["cell_id",
↳ "model"])).to_string(index=False))

```

```

model_selection.to_csv(
    CHAPTER_ROOT / "results" / "all_cells_model_selection.csv",
    index=False,
)

```

A conservative RUL estimate is usually the model that predicts earlier EOL, provided it is not obviously nonphysical. If the power-law exponent is unstable or at a bound, prefer the square-root model and report that the flexible model was not identifiable.

8.6.2 Solution 2

```

from pathlib import Path
import numpy as np
import pandas as pd
from scipy.optimize import curve_fit

CHAPTER_ROOT = Path("chapter8_soh_aging_workspace")
capacity_table = pd.read_csv(CHAPTER_ROOT / "data" / "processed" /
    ↪ "nasa_capacity_table.csv")

def sqrt_capacity_model(cycle_number, capacity_0, sqrt_fade_coefficient):
    return capacity_0 - sqrt_fade_coefficient * np.sqrt(cycle_number)

def rmse(values):
    return float(np.sqrt(np.mean(np.square(values))))

cell_id = "B0005" if "B0005" in set(capacity_table["cell_id"]) else
    ↪ capacity_table["cell_id"].iloc[0]
cell = capacity_table.loc[capacity_table["cell_id"] ==
    ↪ cell_id].sort_values("discharge_index")
split_index = int(np.floor(0.40 * len(cell)))

train = cell.iloc[:split_index]
test = cell.iloc[split_index:]

parameters, _ = curve_fit(
    sqrt_capacity_model,
    train["discharge_index"].to_numpy(dtype=float),
    train["capacity_ah"].to_numpy(dtype=float),

```

```

    p0=[train["capacity_ah"].iloc[0], 0.02],
    bounds=([1.0, 0.0], [2.5, 0.5]),
)

train_prediction = sqrt_capacity_model(
    train["discharge_index"].to_numpy(dtype=float),
    *parameters,
)
test_prediction = sqrt_capacity_model(
    test["discharge_index"].to_numpy(dtype=float),
    *parameters,
)

print(f"Cell: {cell_id}")
print(f"Train rows: {len(train)}, test rows: {len(test)}")
print(f"Train RMSE: {rmse(train['capacity_ah'].to_numpy() -
↪ train_prediction):.5f} Ah")
print(f"Test RMSE: {rmse(test['capacity_ah'].to_numpy() -
↪ test_prediction):.5f} Ah")

```

The test RMSE is the number to care about. If the train RMSE is excellent but the test RMSE is poor, the early-life data did not identify the later fade trend.

8.6.3 Solution 3

Use the ICA code from Walkthrough 6 and replace `window_length = 51` with a function argument. A robust feature should not move more than a few millivolts across reasonable smoothing windows. If peak voltage shifts by tens of millivolts, the feature is not stable enough without a more careful preprocessing study.

8.6.4 Solution 4

A flatter sodium-ion voltage region reduces the information content of voltage-derived health features, just as a flat OCV region weakens SOC observability in Chapter 7. The estimator should use longer time windows, include impedance or pulse-response features, and avoid claiming precise SOH from a small partial voltage segment. For a publishable sodium-ion study, pair ICA/DVA with capacity checks from occasional reference cycles or with physics-based constraints from a calibrated cell model.

What Changes for Sodium-Ion?

The workflow changes less than the interpretation. A sodium-ion cell still has capacity, cycle number, temperature, current, voltage curves, and SOH. You can still parse cycling data, fit fade models, simulate candidate mechanisms, and compute ICA/DVA. The chemistry-specific changes are in the OCV shape, degradation mechanisms, parameter availability, and validation strategy.

Hard-carbon sodium-ion anodes often show voltage plateaus and sloping regions that differ from graphite. Cathode families such as layered oxides, polyanionic compounds, and Prussian blue analogues have different structural degradation pathways and voltage signatures. SEI chemistry is also different because electrolyte composition, sodium solvation, and interphase transport are not lithium copies with a different ion label. Therefore, do not take a PyBaMM Li-ion SEI parameter set, rename lithium to sodium, and call it a sodium-ion aging model.

For sodium-ion publishable work, use lithium-ion public data to validate the software workflow, then adapt the model with sodium-ion parameter sets and sodium-ion literature constraints. If you have only sparse SIB capacity data, fit low-parameter models, report uncertainty broadly, and avoid mechanistic claims that the data cannot support. If you digitize SIB capacity curves from papers, include digitization error and cite the original figure. Reviewers will accept careful limitation statements; they will not accept hidden overreach.

Chapter Summary and Skill Checklist

- You parsed the NASA PCoE Battery Data Set from MATLAB `.mat` structures into a tidy SOH table.
- You fitted square-root and power-law capacity-fade models and compared residuals and 500-cycle forecasts.
- You built a pooled semi-empirical stress model with temperature and current terms, while learning when such terms are not identifiable.
- You ran a PyBaMM DFN simulation with SEI aging options and extracted cycle-level degradation variables.
- You reproduced the empirical aging forecast in MATLAB for BMS-oriented workflows.
- You computed ICA and DVA curves from raw discharge data and extracted peak features.

- You practiced a published-figure reproduction workflow for capacity-fade trajectories.

Commands, functions, and patterns that should now feel familiar:

- `requests.get`, `ZipFile.extractall`, `Path.rglob`
- `scipy.io.loadmat` with MATLAB struct unwrapping
- `pandas.groupby`, `transform`, `to_csv`, and `table joins`
- `scipy.optimize.curve_fit` and `least_squares`
- `pybamm.lithium_ion.DFN`, degradation options, `pybamm.Experiment`, `Simulation.solve`
- MATLAB `readtable`, `writetable`, `tiledlayout`, `exportgraphics`
- Savitzky-Golay smoothing with `scipy.signal.savgol_filter`
- ICA/DVA derivatives using `np.gradient`

You should now be able to:

- Build a clean capacity-versus-cycle table from a public aging dataset.
- State clearly whether SOH is relative to rated capacity or first measured capacity.
- Fit and compare empirical aging models without confusing fit quality with forecast reliability.
- Add stress terms only when the dataset contains enough variation to support them.
- Run a small PyBaMM degradation simulation and identify relevant health variables.
- Port fitted aging parameters into MATLAB and forecast SOH over 500+ cycles.
- Compute ICA/DVA features and explain their sensitivity to smoothing and temperature.
- Explain what must change before applying a Li-ion aging workflow to sodium-ion cells.

Deliverable

Your deliverable is an aging model calibrated to NASA data in Python and a parallel implementation in MATLAB, with a comparison of predictions over 500+ cycles. The minimum acceptable submission contains:

- `nasa_capacity_table.csv` with one row per discharge cycle.
- A Python notebook or script that fits square-root and power-law models to at least one NASA cell.
- A 500-cycle Python forecast plot with capacity and SOH.
- A MATLAB script that reads the fitted parameters and reproduces the 500-cycle forecast.
- A short written analysis comparing the two model forms and explaining which forecast you would trust more.
- One ICA/DVA figure comparing early, middle, and late life for one cell.
- A reproduction log for the published capacity-fade figure exercise.

A strong submission also includes all-cell model selection, early-life train/test forecasting, residual plots, and a paragraph on sodium-ion adaptation.

Further Practice and Reading

Bookmark the NASA PCoE repository page for dataset access and citation guidance:

<https://www.nasa.gov/intelligent-systems-division/discovery-and-systems>

Also keep the NASA Open Data listing for metadata on the Li-ion Battery Aging dataset:

<https://data.nasa.gov/dataset/li-ion-battery-aging-datasets>.

For PyBaMM, bookmark the official installation documentation and degradation examples at <https://docs.pybamm.org/>. PyBaMM's release notes are worth reading before long aging studies because degradation option names and compatible submodels can change.

For aging-data context, read Saha and Goebel's NASA battery dataset citation, Severson et al. (2019) on early prediction of battery cycle life, and Attia et al. (2020) on closed-loop fast-charging optimization. For sodium-ion aging, look for recent review papers and cell-specific cycling studies rather than assuming lithium-ion degradation parameters transfer directly.

Chapter 9 is next: **Thermal Modeling and Electrothermal Coupling**.

Thermal Modeling and Electrothermal Coupling

This chapter is where the battery stops being an electrical object alone. In the theory textbook you learned that temperature is not an afterthought layered on top of electrochemistry. Temperature changes reaction rates, diffusivities, electrolyte conductivity, aging rate, lithium or sodium plating risk, and even the practical definition of “fast charge.” A cell that looks safe in an isothermal simulation can become unacceptable once heat generation and heat rejection are allowed to compete.

Keep the thermal portions of Textbook Chapter 8 open as you work. This chapter operationalizes the Bernardi heat-generation equation, the lumped energy balance, and the electrothermal feedback loop you learned there. Also keep Textbook Chapter 10 nearby, because the first exercises deliberately connect an equivalent-circuit model to a thermal state before we return to PyBaMM. Finally, keep the sodium-ion chapter handy. Sodium-ion cells are often advertised as having stronger low-temperature tolerance than lithium-ion analogues, but that claim only becomes research-grade when we can quantify what “stronger” means under the same current, voltage, and thermal boundary conditions.

By the end of this chapter you will be able to build a transparent lumped thermal

model from first principles, fit a simple thermal resistance and heat capacity to temperature data, turn on PyBaMM's thermal submodels, compare isothermal and electrothermal simulations, and run a CCCV charge under three ambient temperatures. The deliverable is a coupled electrothermal PyBaMM simulation of a CCCV charge under cold, room-temperature, and warm ambient conditions, with a written interpretation of the observed tradeoffs.

This is publishable-research skill because thermal assumptions are one of the easiest places to fool yourself. A paper can report an impressive fast-charge protocol, an elegant degradation prediction, or a sodium-ion advantage, but the claim becomes weak if the thermal boundary condition is unrealistic or unreported. Reviewers will ask whether the cell was assumed isothermal, whether heat transfer coefficients were justified, whether temperature-dependent parameters were enabled, whether the current profile respects voltage and temperature limits, and whether the conclusions survive a change in ambient temperature. The work in this chapter gives you the practical language to answer those questions.

We will move in five stages. First, we derive and code a lumped thermal model by hand so that every watt and kelvin has a place to live. Second, we use public NASA battery data to estimate thermal parameters from a measured current-voltage-temperature trace. Third, we turn on PyBaMM thermal options and inspect the variables the solver exposes. Fourth, we reproduce the qualitative temperature-profile comparison from a published fast-charging study by comparing 1C, 3C, and 5C CCCV-style protocols. Fifth, we build the chapter deliverable: a coupled electrothermal CCCV charge at three ambient temperatures and a sodium-ion-focused interpretation of what changes.

9.1 Prerequisites Check

- Required software: Python 3.11, pybamm==25.10.2, numpy==2.1.3, scipy==1.14.1, pandas==2.2.3, matplotlib==3.9.2, requests==2.32.3, and h5py==3.12.1
- Optional software: MATLAB R2024b or newer with Simulink and Simscape if you want to extend the Simulink thermal section into a block diagram
- Required textbook chapters: Textbook Chapter 8 thermal modeling section is essential; Textbook Chapter 10 is recommended; the sodium-ion chapter is strongly recommended

- Required prior lab chapters: Lab Chapters 1, 2, 3, and 4 are essential; Lab Chapter 6 helps for ECM intuition; Lab Chapter 7 is not required
- Estimated time: 12 to 16 hours for the full chapter

If PyBaMM still feels unfamiliar, revisit Lab Chapter 3 before continuing. If PyBaMM experiments and parameter updates feel shaky, revisit Lab Chapter 4. If ordinary differential equations in `scipy.solve_ivp` feel rusty, reread the solver section of Lab Chapter 2 before starting Walkthrough 1.

9.2 Environment Setup

We will use a fresh Python environment because thermal PyBaMM examples are sensitive to version drift. PyBaMM is an active project, and examples from older notebooks can fail if option names or solver defaults have moved. The version pinned here, 25.10.2, matches the current stable documentation family used for this chapter’s thermal options.

9.2.1 Create the environment

From a terminal, create and activate a new environment:

```
cd /home/avirup/SodiumIonBatteryResearch
python3.11 -m venv .venv-chapter9
source .venv-chapter9/bin/activate
python -m pip install --upgrade pip
python -m pip install pybamm==25.10.2 numpy==2.1.3 scipy==1.14.1
↳ pandas==2.2.3 matplotlib==3.9.2 requests==2.32.3 h5py==3.12.1
```

If you use conda instead of `venv`, the equivalent is:

```
conda create -n battery-chapter9 python=3.11 -y
conda activate battery-chapter9
python -m pip install pybamm==25.10.2 numpy==2.1.3 scipy==1.14.1
↳ pandas==2.2.3 matplotlib==3.9.2 requests==2.32.3 h5py==3.12.1
```

The installation may take several minutes because PyBaMM pulls in solver and symbolic-model dependencies. This is normal. If the install fails while building `casadi` or a solver dependency, first upgrade `pip`, then retry the exact install command. If it still fails on Windows, use conda for Python itself and `pip` only for the pinned packages.

9.2.2 Create a chapter workspace

Run this from the project root:

```
mkdir -p SimulationCompanion/chapter9_thermal_workspace
cd SimulationCompanion/chapter9_thermal_workspace
```

The code in this chapter writes downloaded data, CSV files, and figures into this folder. Keeping chapter artifacts separate is not just neatness. It makes it possible to rerun the lab months later and know exactly which files were produced by which workflow.

9.2.3 Verify the install

Open a new Jupyter notebook in the activated environment or run the following as a Python script:

```
import pybamm
import numpy as np
import scipy
import pandas as pd
import matplotlib

print("PyBaMM:", pybamm.__version__)
print("NumPy:", np.__version__)
print("SciPy:", scipy.__version__)
print("pandas:", pd.__version__)
print("Matplotlib:", matplotlib.__version__)

model = pybamm.lithium_ion.SPM({"thermal": "lumped"})
parameter_values = pybamm.ParameterValues("Chen2020")
experiment = pybamm.Experiment(
    ["Discharge at 1C for 10 minutes"],
    period="10 seconds",
    temperature="25 oC",
)
simulation = pybamm.Simulation(
    model,
    parameter_values=parameter_values,
    experiment=experiment,
)
solution = simulation.solve()

final_voltage = solution["Terminal voltage [V]"](solution.t[-1])
final_temperature = solution["Volume-averaged cell temperature
↪ [K]"](solution.t[-1])
```

```
print(f"Final voltage: {final_voltage:.3f} V")
print(f"Final cell temperature: {final_temperature:.2f} K")
```

Expected output:

```
PyBaMM: 25.10.2
NumPy: 2.1.3
SciPy: 1.14.1
pandas: 2.2.3
Matplotlib: 3.9.2
Final voltage: 3.xxx V
Final cell temperature: 298.xx K
```

The last digits will vary slightly by platform and solver. What matters is that the simulation runs, the voltage is near the normal lithium-ion operating range, and the final temperature is only slightly above ambient for a short 1C discharge.

9.2.4 Common installation failures and fixes

`ModuleNotFoundError: No module named 'pybamm'` means your notebook kernel is not using the environment you installed into. In Jupyter, install a kernel with `python -m ipykernel install --user --name battery-chapter9 --display-name "Battery Chapter 9"` and select that kernel.

`pybamm.OptionError` when selecting a thermal model usually means a geometry option is incompatible with the thermal option. In PyBaMM, `x-full` is a pouch-cell through-thickness thermal model, so use it with compatible pouch geometry and a model that supports it. For this chapter, we use `lumped` for the main deliverable because it is robust and fast.

Solver failures during aggressive fast-charge simulations usually mean the requested experiment is physically or numerically too severe for the parameter set. Reduce the C-rate, loosen the final SOC target, add a voltage termination, or switch from DFN to SPMe before assuming PyBaMM is broken.

If plots appear blank in a notebook, run `%matplotlib inline` in a cell before plotting. If you are running scripts from a terminal, make sure the script calls `plt.show()` after creating figures.

9.3 Conceptual Bridge: From Bernardi Heat to PyBaMM Thermal Objects

In Textbook Chapter 8, thermal modeling began with an energy balance. A cell has thermal mass, generates heat internally, and exchanges heat with its surroundings. In its simplest lumped form, the cell temperature T obeys

$$mc_p \frac{dT}{dt} = \dot{Q}_{\text{gen}} - hA(T - T_{\infty}), \quad (1)$$

where m is cell mass, c_p is average specific heat, h is an effective heat transfer coefficient, A is cooling surface area, and T_{∞} is ambient temperature. Equation (1) is easy to write down and surprisingly powerful. It tells you immediately why a small cell under a short pulse may barely warm, while a large-format cell under fast charging can remain hot long after current tapers. The thermal time constant is

$$\tau_{\text{th}} = \frac{mc_p}{hA}. \quad (2)$$

If τ_{th} is large compared with the electrical event, heat accumulates. If it is small, the cell tracks ambient more closely. This one number will appear repeatedly in your intuition even when the model becomes electrochemical.

The harder part is $\dot{Q}_{\text{gen}}$. Bernardi's heat-generation expression is commonly written as

$$\dot{Q}_{\text{gen}} = I(U_{\text{oc}} - V) - IT \frac{\partial U_{\text{oc}}}{\partial T}, \quad (3)$$

where I is current under a chosen sign convention, U_{oc} is open-circuit voltage, V is terminal voltage, and $\partial U_{\text{oc}}/\partial T$ is the entropic coefficient. The first term is irreversible heat. It collects ohmic and reaction losses. In a simple circuit model this is often approximated as $I^2 R$. The second term is reversible heat. It can be positive or negative depending on chemistry, SOC, and current direction. This is one reason measured cell temperature sometimes does not follow the naive $I^2 R$ picture perfectly.

PyBaMM represents this same physics at a much richer level. In a DFN or SPMe model, the terminal voltage is not just an algebraic input to Equation (3). It emerges from solid potentials, electrolyte potentials, reaction overpotentials, concentration fields, and open-circuit potentials. When you choose {"thermal": "lumped"}, PyBaMM computes the volume-averaged heat source from the electrochemical solution and evolves a cell-average temperature. When you

choose {"thermal": "x-full"}, PyBaMM solves a through-cell thermal PDE for pouch geometry. Newer PyBaMM documentation also describes higher-dimensional pouch-cell thermal models that resolve in-plane temperature variation, but those are more expensive and require geometry and boundary-condition care.

The conceptual map is:

Textbook idea	Hand model representation	PyBaMM representation
Thermal mass	$m_{\text{cell_kg}} * c_{p_j_per_kg_k}$	Effective volumetric heat capacity from cell layers
Heat loss	$h * area * (T - T_{\text{amb}})$	Cooling terms from heat-transfer and geometry parameters
Ohmic heat	$current_a^{*2} * resistance_ohm$	Solid/electrolyte ohmic heat submodels
Reaction heat	Often folded into an effective resistance	Interfacial current times overpotential
Reversible heat	$-I*T*dU_dT$ if entropic data are known	Entropic-change functions in the parameter set
Temperature feedback	Manual updates to resistance or OCV	Temperature-dependent parameters inside the electrochemical model

This chapter deliberately starts with the hand model because it gives you a conservation-law audit trail. If your PyBaMM result says the cell warms by 15 K, you should have a rough sense of whether that is plausible. Suppose a 5 Ah cell has $m_{c_p} = 900 \text{ J/K}$. A 3C charge at 15 A through an effective 20 mOhm resistance generates about 4.5 W. With no cooling, thirty minutes would add 9 K. With cooling, less. If your simulation predicts 80 K rise for that case, either the current is far more severe, the cooling is nearly absent, the parameter set is not representing the cell you imagine, or a sign convention has gone wrong.

Thermal modeling also forces us to distinguish three phrases that are often blurred. “Isothermal” means temperature is held fixed. It does not mean heat

generation is zero; it means the thermal state is not solved. “Lumped thermal” means the model solves one average cell temperature. It can capture warm-up and cool-down but not internal gradients. “Spatial thermal” means temperature varies across at least one coordinate. Spatial models are essential when tab placement, pouch geometry, cooling plates, or large-format gradients matter. For the simulation-based sodium-ion research path in this companion, lumped electrothermal modeling is the right first publishable tool: simple enough to calibrate with sparse data, rich enough to expose temperature-current tradeoffs, and fast enough to use in sweeps.

Guided Walkthrough 1

Build a Lumped Thermal Model from First Principles

Learning objective: Implement Equation (1) with both irreversible and reversible heat terms, then interpret the temperature response under a pulse-current profile.

Before PyBaMM gets involved, we want a model you can hold in your head. We will prescribe a current profile, compute a simple SOC-dependent OCV, compute terminal voltage using a resistance, compute heat generation, and solve the lumped energy balance. This is a teaching model, not a high-fidelity cell model. Its purpose is to make units, signs, and time constants concrete.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def ocv_from_soc(soc):
    """Smooth teaching OCV curve for a lithium-ion-like cell."""
    soc_clipped = np.clip(soc, 0.0, 1.0)
    baseline = 3.05 + 1.05 * soc_clipped
    plateau_shape = 0.08 * np.tanh((soc_clipped - 0.55) / 0.08)
    low_soc_knee = -0.10 * np.exp(-soc_clipped / 0.08)
    high_soc_knee = 0.05 * np.exp(-(1.0 - soc_clipped) / 0.05)
    return baseline + plateau_shape + low_soc_knee + high_soc_knee

def entropic_coefficient_from_soc(soc):
    """Approximate  $dU_{oc}/dT$  in V/K for a teaching cell."""
    soc_clipped = np.clip(soc, 0.0, 1.0)
    return 1.2e-4 * np.sin(2.0 * np.pi * (soc_clipped - 0.15))
```

```

def current_profile_a(time_s):
    """Positive current means discharge in this walkthrough."""
    if time_s < 300.0:
        return 0.0
    if time_s < 1500.0:
        return 6.0
    if time_s < 2400.0:
        return 0.0
    if time_s < 3300.0:
        return 10.0
    return 0.0

def electrothermal_rhs(time_s, state, parameters):
    soc, temperature_k = state
    current_a = current_profile_a(time_s)

    capacity_ah = parameters["capacity_ah"]
    resistance_ohm = parameters["resistance_ohm"]
    mass_kg = parameters["mass_kg"]
    cp_j_per_kg_k = parameters["cp_j_per_kg_k"]
    h_w_per_m2_k = parameters["h_w_per_m2_k"]
    area_m2 = parameters["area_m2"]
    ambient_k = parameters["ambient_k"]

    ocv_v = ocv_from_soc(soc)
    dudt_v_per_k = entropic_coefficient_from_soc(soc)
    terminal_voltage_v = ocv_v - current_a * resistance_ohm

    irreversible_heat_w = current_a * (ocv_v - terminal_voltage_v)
    reversible_heat_w = -current_a * temperature_k * dudt_v_per_k
    heat_generation_w = irreversible_heat_w + reversible_heat_w
    heat_loss_w = h_w_per_m2_k * area_m2 * (temperature_k - ambient_k)

    dsoc_dt = -current_a / (3600.0 * capacity_ah)
    dtemperature_dt = (
        heat_generation_w - heat_loss_w
    ) / (mass_kg * cp_j_per_kg_k)

    return [dsoc_dt, dtemperature_dt]

parameters = {

```



```

    "capacity_ah": 5.0,
    "resistance_ohm": 0.018,
    "mass_kg": 0.280,
    "cp_j_per_kg_k": 950.0,
    "h_w_per_m2_k": 12.0,
    "area_m2": 0.030,
    "ambient_k": 298.15,
}

initial_state = [0.85, parameters["ambient_k"]]
time_eval_s = np.arange(0.0, 4800.0 + 1.0, 1.0)

solution = solve_ivp(
    fun=lambda time_s, state: electrothermal_rhs(time_s, state, parameters),
    t_span=(time_eval_s[0], time_eval_s[-1]),
    y0=initial_state,
    t_eval=time_eval_s,
    method="BDF",
    rtol=1e-8,
    atol=[1e-9, 1e-6],
)

if not solution.success:
    raise RuntimeError(solution.message)

soc = solution.y[0]
temperature_k = solution.y[1]
current_a = np.array([current_profile_a(t) for t in solution.t])
ocv_v = ocv_from_soc(soc)
voltage_v = ocv_v - current_a * parameters["resistance_ohm"]
temperature_c = temperature_k - 273.15

fig, axes = plt.subplots(4, 1, figsize=(9, 9), sharex=True)

axes[0].plot(solution.t / 60.0, current_a, color="tab:blue", linewidth=1.8)
axes[0].set_ylabel("Current [A]")
axes[0].grid(True)

axes[1].plot(solution.t / 60.0, soc, color="tab:green", linewidth=1.8)
axes[1].set_ylabel("SOC [-]")
axes[1].grid(True)

axes[2].plot(solution.t / 60.0, voltage_v, color="tab:purple", linewidth=1.8)
axes[2].set_ylabel("Voltage [V]")
axes[2].grid(True)

```

```

axes[3].plot(solution.t / 60.0, temperature_c, color="tab:red",
↳ linewidth=1.8)
axes[3].axhline(parameters["ambient_k"] - 273.15, color="0.4",
↳ linestyle="--")
    3          "Cell temp. [deg C]"
    3          "Time [min]"
    3          True

    "Lumped electrothermal response to discharge pulses"

print f"Final SOC: {      -1 :.4f}"
print f"Peak temperature: {                .max :.2f} deg C"
print(f"Temperature rise: {temperature_c.max() - (parameters['ambient_k'] -
↳ 273.15):.2f} K")
print(f"Thermal time constant: {parameters['mass_kg'] *
↳ parameters['cp_j_per_kg_k'] / (parameters['h_w_per_m2_k'] *
↳ parameters['area_m2']) / 60.0:.1f} min")

```

The code begins with an OCV function and an entropic-coefficient function. These are intentionally smooth teaching functions. Real work should use measured OCV and entropy data where possible. The current profile uses positive current for discharge, matching the sign convention in Lab Chapter 6. The right-hand side computes the electrical variables first, then the irreversible heat, reversible heat, heat loss, SOC derivative, and temperature derivative.

The line `irreversible_heat_w = current_a * (ocv_v - terminal_voltage_v)` is equivalent to I^2R here because the terminal voltage is `ocv_v - current_a * resistance_ohm`. We write it in Bernardi form so the same pattern remains recognizable later when voltage comes from a more complex model. The reversible heat term can change sign because `entropic_coefficient_from_soc` can be positive or negative. This is standard physics, not a numerical trick.

The plot should have four stacked panels. The first panel shows two discharge pulses, one at 6 A and one at 10 A. The second panel shows SOC decreasing only during the pulses. The third panel shows voltage stepping downward when current turns on, then recovering during rest. The fourth panel shows cell temperature rising during the pulses and relaxing during rest. The second pulse

should cause a visibly faster temperature rise because heat scales roughly with current squared in this simple model.

The printed peak temperature should be only a few degrees above ambient with the parameters shown. The thermal time constant should be around 12.3 min. That time constant explains why the temperature does not instantly return to ambient during the rest interval.

9.3.1 What could go wrong

If the temperature decreases during a high-current discharge, check the current sign convention and the entropic term. A large positive entropic coefficient during discharge can create reversible cooling, but it should not dominate the whole response in this teaching example unless you changed the coefficient magnitude.

If SOC increases during discharge, the sign in `dsoc_dt` is wrong. In this chapter's hand models, positive current means discharge, so SOC must decrease.

If `solve_ivp` returns `Required step size is less than spacing between numbers`, you probably introduced a discontinuous expression that creates numerical trouble. Keep the current profile piecewise constant as shown, or split the simulation at current transition times.

If the temperature rise is enormous, inspect units. Area must be in square meters, mass in kilograms, specific heat in joules per kilogram-kelvin, and resistance in ohms.

9.3.2 Reflection

This exercise gave you a thermal back-of-the-envelope tool. You can now estimate whether a simulated temperature rise is plausible before trusting a high-level package. We will reuse this same energy-balance logic when fitting NASA data and when interpreting PyBaMM's coupled electrothermal output.

Guided Walkthrough 2

Fit a Lumped Thermal Model to Public NASA Data

Learning objective: Parse a public current-voltage-temperature battery trace and fit effective thermal parameters from measured temperature.

The NASA Prognostics Center of Excellence randomized battery datasets contain

18650 lithium-ion cells operated under randomized currents with reference cycles. The room-temperature random-walk discharge dataset includes cells RW3 through RW6 and repeatedly charges cells to 4.2 V, then discharges them to 3.2 V using randomized discharge currents. The NASA Open Data portal lists the dataset as public, and NASA's Zenodo mirror gives the full PCoE randomized battery usage collection a DOI, [10.5281/zenodo.15277374](https://doi.org/10.5281/zenodo.15277374), under a Creative Commons Attribution 4.0 license. We will use the smaller room-temperature discharge ZIP, about 120 MB, rather than the larger charge-discharge archive.

For a textbook-stable lab, the code below is defensive. It downloads the ZIP if available, searches for MATLAB files inside it, and then normalizes common NASA field names. If the portal changes its resource URL, the parser still teaches the pattern: locate a cycle, extract time, current, voltage, and measured temperature, then fit mc_p and hA .

```
from urllib.request import urlopen
from zipfile import ZipFile
import os
import sys
import re
import math
from datetime import datetime
from collections import defaultdict
from typing import List, Dict, Tuple, Optional

def download_file(url: str) -> Optional[bytes]:
    """Download a file from a URL if it exists and return its content as bytes.
    If the file does not exist, return None.
    """
    try:
        response = urlopen(url)
        return response.read()
    except:
        return None

def extract_files(zip_content: bytes) -> List[str]:
    """Extract the file names from a ZIP file content.
    Returns a list of file names.
    """
    with ZipFile(io.BytesIO(zip_content)) as zip_file:
        return zip_file.namelist()

def normalize_data(data: List[Dict]) -> List[Dict]:
    """Normalize the data by converting units and standardizing field names.
    Returns a list of normalized data points.
    """
    normalized_data = []
    for point in data:
        # Convert time from seconds to minutes
        point['time'] = point['time'] / 60
        # Convert current from Amperes to Amperes (no change)
        point['current'] = point['current']
        # Convert voltage from Volts to Volts (no change)
        point['voltage'] = point['voltage']
        # Convert temperature from Celsius to Celsius (no change)
        point['temp'] = point['temp']
        # Fit mc_p and hA
        point['mc_p'] = fit_mc_p(point['current'], point['temp'])
        point['hA'] = fit_hA(point['voltage'], point['temp'])
        normalized_data.append(point)
    return normalized_data

def fit_mc_p(current: float, temp: float) -> float:
    """Fit the specific heat capacity (mc_p) based on current and temperature.
    Returns a value.
    """
    # Placeholder for the fitting logic
    return 0.0

def fit_hA(voltage: float, temp: float) -> float:
    """Fit the heat transfer coefficient (hA) based on voltage and temperature.
    Returns a value.
    """
    # Placeholder for the fitting logic
    return 0.0

def main():
    # Download the dataset
    url = "https://zenodo.org/records/15277374/files/"
    zip_file_name = "2.%20Battery_Uniform_Distribution_Discharge_Room_Temp_DataSet_2Post.zip"
    url = url + zip_file_name + "?download=1"

    # Download the file
    zip_content = download_file(url)

    # Extract the files
    file_names = extract_files(zip_content)

    # Find the data file
    data_file_name = None
    for file_name in file_names:
        if file_name.endswith('.mat'):
            data_file_name = file_name
            break

    # Load the data
    if data_file_name:
        data = load_data(data_file_name)
    else:
        print("No data file found in the ZIP archive.")
        return

    # Normalize the data
    normalized_data = normalize_data(data)

    # Print the first few data points
    for point in normalized_data[:5]:
        print(f"Time: {point['time']} min, Current: {point['current']} A, Voltage: {point['voltage']} V, Temp: {point['temp']} C, mc_p: {point['mc_p']}, hA: {point['hA']}")

if __name__ == '__main__':
    main()
```

```

print(f"Downloading {url}")
response = requests.get(url, stream=True, timeout=60)
response.raise_for_status()
with output_path.open("wb") as file:
    for chunk in response.iter_content(chunk_size=1024 * 1024):
        if chunk:
            file.write(chunk)
print(f"Saved {output_path} ({output_path.stat().st_size / 1e6:.1f} MB)")

def extract_zip(zip_file, destination):
    if destination.exists() and any(destination.rglob("*.mat")):
        print(f"Using existing extracted folder: {destination}")
        return

    destination.mkdir(exist_ok=True)
    with ZipFile(zip_file) as archive:
        archive.extractall(destination)
    print(f"Extracted files into {destination}")

def find_first_mat_file(folder):
    mat_files = sorted(folder.rglob("*.mat"))
    if not mat_files:
        raise FileNotFoundError("No .mat files found after extraction.")
    print("First five MATLAB files:")
    for path in mat_files[:5]:
        print(" ", path)
    return mat_files[0]

def flatten_mat_struct(obj):
    if isinstance(obj, np.ndarray) and obj.dtype.names is not None:
        return {name: flatten_mat_struct(obj[name][0, 0]) for name in
            ↪ obj.dtype.names}
    if isinstance          and          == 1
        return
    return

def
    for      in
        if      in
            =
            if          == 1 and          > 10

```

```

        return value.astype(float)
    raise KeyError(f"None of the candidate names were present:
↳ {candidate_names}")

def load_first_usable_trace(mat_path):
    raw = loadmat(mat_path, squeeze_me=False, struct_as_record=False)
    top_level_keys = [key for key in raw.keys() if not key.startswith("__")]
    print("Top-level keys:", top_level_keys)

    # NASA files appear in several nested formats. This branch handles the
    ↳ common
    # PCoE cycle-structured format and falls back to a flat field search.
    root = raw[top_level_keys[0]]
    root_dict = flatten_mat_struct(root)

    if isinstance(root_dict, dict) and "cycle" in root_dict:
        cycles = np.asarray(root_dict["cycle"]).squeeze()
        for cycle in cycles:
            cycle_dict = flatten_mat_struct(cycle)
            data = cycle_dict.get("data", cycle_dict)
            if not isinstance(data, dict):
                continue
            try:
                time_s = extract_numeric_series(data, ["Time", "time"])
                current_a = extract_numeric_series(
                    data,
                    ["Current_measured", "current", "Current"],
                )
                voltage_v = extract_numeric_series(
                    data,
                    ["Voltage_measured", "voltage", "Voltage"],
                )
                temperature_c = extract_numeric_series(
                    data,
                    ["Temperature_measured", "temperature", "Temperature"],
                )
            return pd.DataFrame(
                {
                    "time_s": time_s - time_s[0],
                    "current_a_raw": current_a,
                    "voltage_v": voltage_v,
                    "temperature_c": temperature_c,
                }
            )

```

```

        except KeyError:
            continue

    flat_candidates = {}
    for key, value in raw.items():
        if key.startswith("__"):
            continue
        array = np.asarray(value).squeeze()
        if array.ndim == 1 and array.size > 10:
            flat_candidates[key] = array.astype(float)

    required = {
        "time_s": ["Time", "time", "t"],
        "current_a_raw": ["Current", "current", "Current_measured"],
        "voltage_v": ["Voltage", "voltage", "Voltage_measured"],
        "temperature_c": ["Temperature", "temperature",
            ↪ "Temperature_measured"],
    }
    output = {}
    for output_name, names in required.items():
        for name in names:
            if name in flat_candidates:
                output[output_name] = flat_candidates[name]
                break
        if output_name not in output:
            raise KeyError(f"Could not find {output_name} in {mat_path}")

    frame = pd.DataFrame(output)
    frame["time_s"] = frame["time_s"] - frame["time_s"].iloc[0]
    return frame

```

```

download_file(DATA_URL, zip_path)
extract_zip(zip_path, extract_dir)
mat_path = find_first_mat_file(extract_dir)
trace = load_first_usable_trace(mat_path)

trace = trace.dropna().sort_values("time_s").drop_duplicates("time_s")
trace = trace.iloc[: min(len(trace), 2500)].copy()

# NASA files may use positive charge or positive discharge depending on
↪ source.
# For heat fitting, current squared dominates, but voltage-power signs still
↪ matter.
trace["current_a"] = trace["current_a_raw"]

```

```

trace["power_w"] = trace["voltage_v"] * trace["current_a"]

ambient_c = float(trace["temperature_c"].iloc[:50].median())
trace["temperature_k"] = trace["temperature_c"] + 273.15

print(trace.head())
print(f"Trace duration: {trace['time_s'].iloc[-1] / 60:.1f} min")
print(f"Ambient estimate: {ambient_c:.2f} deg C")

```

The parser is longer than the model, and that is realistic. Public battery datasets rarely arrive as tidy time, current, voltage, temperature CSV files. The helper `flatten_mat_struct` handles MATLAB struct arrays. The loader first tries the common NASA cycle format and then falls back to flat numeric arrays. The current sign is not changed because the thermal fit mostly needs heat magnitude, but we preserve `current_a_raw` so you can audit signs later.

Now fit a simple model:

```

def simulate_temperature_for_fit(time_s, current_a, voltage_v, theta):
    log_thermal_mass, log_ha, log_resistance = theta
    thermal_mass_j_per_k = np.exp(log_thermal_mass)
    ha_w_per_k = np.exp(log_ha)
    resistance_ohm = np.exp(log_resistance)

    ambient_k = ambient_c + 273.15
    current_interp = lambda t: np.interp(t, time_s, current_a)

    def rhs(t, temperature_k_array):
        current_now = current_interp(t)
        heat_generation_w = (current_now**2) * resistance_ohm
        heat_loss_w = ha_w_per_k * (temperature_k_array[0] - ambient_k)
        return [(heat_generation_w - heat_loss_w) / thermal_mass_j_per_k]

    solution = solve_ivp(
        rhs,
        (time_s[0], time_s[-1]),
        [ambient_k],
        t_eval=time_s,
        method="BDF",
        rtol=1e-6,
        atol=1e-5,
    )
    if not solution.success:
        raise RuntimeError(solution.message)

```



```

    return solution.y[0]

time_s = trace["time_s"].to_numpy()
current_a = trace["current_a"].to_numpy()
temperature_k_measured = trace["temperature_k"].to_numpy()
voltage_v = trace["voltage_v"].to_numpy()

def residual(theta):
    predicted_k = simulate_temperature_for_fit(time_s, current_a, voltage_v,
    ↪ theta)
    return predicted_k - temperature_k_measured

initial_guess = np.log([350.0, 0.18, 0.030])
lower_bounds = np.log([50.0, 0.02, 0.001])
upper_bounds = np.log([3000.0, 3.0, 0.300])

fit = least_squares(
    residual,
    initial_guess,
    bounds=(lower_bounds, upper_bounds),
    xtol=1e-8,
    ftol=1e-8,
    gtol=1e-8,
)

thermal_mass_j_per_k, ha_w_per_k, resistance_ohm = np.exp(fit.x)
predicted_temperature_k = simulate_temperature_for_fit(time_s, current_a,
    ↪ voltage_v, fit.x)
rmse_k = np.sqrt(np.mean((predicted_temperature_k - temperature_k_measured)
    ↪ ** 2))

print(f"Fitted thermal mass: {thermal_mass_j_per_k:.1f} J/K")
print(f"Fitted hA: {ha_w_per_k:.4f} W/K")
print(f"Fitted effective resistance: {resistance_ohm:.4f} ohm")
print(f"Temperature RMSE: {rmse_k:.3f} K")

fig, axes = plt.subplots(3, 1, figsize=(9, 8), sharex=True)

axes[0].plot(time_s / 60.0, current_a, linewidth=1.2)
axes[0].set_ylabel("Current [A]")
axes[0].grid(True)

```

```

axes[1].plot(time_s / 60.0, voltage_v, linewidth=1.2, color="tab:purple")
axes[1].set_ylabel("Voltage [V]")
axes[1].grid(True)

axes[2].plot(
    time_s / 60.0,
    temperature_k_measured - 273.15,
    label="Measured",
    linewidth=1.5,
)
axes[2].plot(
    time_s / 60.0,
    predicted_temperature_k - 273.15,
    label="Fitted lumped model",
    linewidth=1.5,
)
axes[2].set_ylabel("Temperature [deg C]")
axes[2].set_xlabel("Time [min]")
axes[2].legend()
axes[2].grid(True)

fig.suptitle("NASA trace: lumped thermal fit")
fig.tight_layout()
plt.show()

```

You should see a current trace with irregular steps, a voltage trace responding to the load, and a temperature trace that changes slowly compared with current. The fitted thermal model should follow the broad temperature trend but will not capture every detail. That mismatch is expected. We used a single effective resistance and a single thermal state. Real cells have SOC-dependent resistance, entropic heat, sensor lag, spatial gradients, and chamber dynamics.

The fitted values should be physically plausible. A small 18650 cell may have a thermal mass on the order of tens to hundreds of joules per kelvin. A fitted value outside the bounds means the data segment did not contain enough thermal excitation, the current sign or units are wrong, or the temperature channel is not the cell-surface temperature you think it is.

9.3.3 Dataset pitfalls

NASA dataset timestamps are not always uniform across files. Always sort by time and remove duplicate timestamps before integrating.

Current sign conventions vary. For thermal fitting, I^2R hides much of that problem, but for Bernardi heat and SOC tracking, sign convention must be explicit.

Temperature may be measured at the cell surface, not the electrochemical volume average. A single lumped model fitted to surface temperature is useful, but do not claim it validates core temperature.

Some files include reference cycles, impedance tests, and randomized loading in the same archive. Always inspect cycle metadata before treating a segment as a fast-charge or discharge experiment.

9.3.4 Reflection

This exercise turned public data into thermal parameters. The model is deliberately simple, but the workflow is real: parse, normalize, estimate ambient, choose a heat-generation approximation, fit, plot residuals, and discuss what the model can and cannot claim. We will return to public datasets in Chapter 11 with a reusable loader architecture.

Guided Walkthrough 3

Turn on PyBaMM Thermal Options

Learning objective: Compare isothermal and lumped electrothermal PyBaMM simulations under the same discharge protocol.

Now we let PyBaMM compute heat generation from an electrochemical model. We use the SPM_e because it is fast enough for repeated thermal experiments while retaining electrolyte effects missing from the simplest SPM. The structure is the same as earlier PyBaMM chapters: choose a model, choose parameter values, define an experiment, solve, and extract variables.

```
import
import      as
import      as

def
    =
        =
            "thermal"
            "Chen2020"
    =
```

```

[
    f"Rest for 10 minutes",
    f"Discharge at {c_rate}C until 2.8 V",
    f"Rest for 20 minutes",
],
period="20 seconds",
temperature=f"{ambient_c} oC",
)

simulation = pybamm.Simulation(
    model,
    parameter_values=parameter_values,
    experiment=experiment,
)
solution = simulation.solve()
return solution

isothermal_solution = run_spme_discharge("isothermal", ambient_c=25,
↳ c_rate=2)
= "lumped" =25 =2

def
= "Time [min]"
= "Terminal voltage [V]"
= "Current [A]"
= "Volume-averaged cell temperature [K]"

return
"label"
"time_min"
"voltage_v"
"current_a"
"temperature_c" - 273.15

=
"isothermal"
"lumped thermal"

= 3 1 = 9 8 =True

```

```

for frame in frames:
    axes[0].plot(frame["time_min"], frame["current_a"], label=frame["label"],
    ↪ linewidth=1.5)
    axes[1].plot(frame["time_min"], frame["voltage_v"], label=frame["label"],
    ↪ linewidth=1.5)
    axes[2].plot(frame["time_min"], frame["temperature_c"],
    ↪ label=frame["label"], linewidth=1.5)

axes[0].set_ylabel("Current [A]")
axes[1].set_ylabel("Voltage [V]")
axes[2].set_ylabel("Temperature [deg C]")
axes[2].set_xlabel("Time [min]")

for axis in axes:
    axis.grid(True)
    axis.legend()

fig.suptitle("PyBaMM SPMe: isothermal vs lumped thermal")
fig.tight_layout()
plt.show()

for frame in frames:
    print(
        f"{frame['label']}: final time = {frame['time_min'][-1]:.2f} min, "
        f"min voltage = {frame['voltage_v'].min():.3f} V, "
        f"max temp = {frame['temperature_c'].max():.2f} deg C"
    )

```

The isothermal temperature line should remain flat at 25 deg C. The lumped thermal line should rise during discharge and relax during the final rest. The voltage curves may differ slightly because temperature feeds back into transport and kinetic parameters. At moderate C-rate the difference may be small; that is itself a result. Thermal coupling does not always dominate terminal voltage over a short event, but it strongly affects safety margins and degradation interpretation.

The important PyBaMM object-level move is `pybamm.lithium_ion.SPMe({"thermal": thermal_option})`. The model option changes the submodel composition before discretization. You are not post-processing temperature after the electrical simulation. You are asking PyBaMM to solve a coupled model.

The variable name "Volume-averaged cell temperature [K]" is a standard high-value output for lumped thermal work. PyBaMM exposes many

heat-generation variables too, including irreversible and reversible heating terms in compatible models and parameter sets. Use `solution.all_models[0].variables.keys()` when you need to discover exact variable names for your installed version.

9.3.5 What could go wrong

If Volume-averaged cell temperature [K] is not found, print available temperature variables:

```
temperature_keys = [
    key for key in lumped_solution.all_models[0].variables.keys()
    if "temperature" in key.lower()
]
for key in temperature_keys:
    print(key)
```

If the lumped and isothermal voltage curves are identical, check that you actually solved the lumped model and did not reuse the isothermal solution object.

If the discharge terminates almost immediately, your voltage cutoff is too high for the initial state and parameter set. Use a lower cutoff or begin with a charge/rest step.

If the simulation is slow, switch from DFN to SPMe. For thermal sweeps, SPMe is often the right first research model because it preserves key electrochemical behavior without DFN-level cost.

9.3.6 Reflection

This exercise translated the hand energy balance into PyBaMM's model-option language. You now know how to ask whether thermal coupling changes the electrical trajectory under the same experiment. That question is central to every fast-charge, cold-start, and abuse-margin study.

Guided Walkthrough 4

Reproduce a Published Fast-Charge Temperature Comparison

Learning objective: Reproduce the qualitative result of a published fast-charging figure: higher C-rate CCCV protocols produce larger and longer temperature excursions.

Romero, Goldar, and Garone’s 2019 Modelica conference paper, “A Model Predictive Control Application for a Constrained Fast Charge of Lithium-ion Batteries,” compares CCCV charging protocols at 1C, 3C, and 5C and shows corresponding temperature profiles in their Figure 6. Their model is not PyBaMM, and the paper’s exact cell parameters are not fully sufficient for a bit-for-bit reproduction. That makes it a useful research exercise. We will reproduce the qualitative figure: three CCCV-like charging protocols, same ambient, same thermal model family, increasing C-rate, and temperature trajectories that separate strongly as current increases.

This is a “close enough” reproduction exercise. Close enough means the ordering, shape, and physical interpretation match the paper: 5C heats most, 3C heats moderately, 1C heats least; higher C-rate protocols remain above ambient longer. It does not mean the peak temperatures match the paper digit-for-digit.

```
import
import                               as

def                                     =25
    =                                "thermal" "lumped"
    =                                "Chen2020"

    # We begin from a low-SOC state by discharging first. This keeps the
    # example
    # self-contained without manually setting every initial concentration.
    experiment = pybamm.Experiment(
        [
            "Discharge at 1C until 2.8 V",
            "Rest for 20 minutes",
            f"Charge at {c_rate}C until 4.2 V",
            "Hold at 4.2 V until C/20",
            "Rest for 30 minutes",
        ],
        period="20 seconds",
        temperature=f"{ambient_c} oC",
    )

    simulation = pybamm.Simulation(
        model,
        parameter_values=parameter_values,
        experiment=experiment,
    )
```

```

    return simulation.solve()

c_rates = [1, 3, 5]
solutions = {}

for c_rate in c_rates:
    try:
        solutions[c_rate] = run_fast_charge_case(c_rate)
        print(f"{c_rate}C case solved.")
    except Exception as error:
        print(f"{c_rate}C case failed: {error}")

fig, axes = plt.subplots(3, 1, figsize=(9, 8), sharex=True)

for c_rate, solution in solutions.items():
    time_min = solution["Time [min]"].entries
    current_a = solution["Current [A]"].entries
    voltage_v = solution["Terminal voltage [V]"].entries
    temperature_c = solution["Volume-averaged cell temperature [K]"].entries
    ↪ - 273.15

    label = f"{c_rate}C"
    axes[0].plot(time_min, current_a, label=label, linewidth=1.4)
    axes[1].plot(time_min, voltage_v, label=label, linewidth=1.4)
    axes[2].plot(time_min, temperature_c, label=label, linewidth=1.6)

axes[0].set_ylabel("Current [A]")
axes[1].set_ylabel("Voltage [V]")
axes[2].set_ylabel("Temperature [deg C]")
axes[2].set_xlabel("Time [min]")

for axis in axes:
    axis.grid(True)
    axis.legend()

fig.suptitle("Qualitative reproduction: CCCV C-rate temperature comparison")
fig.tight_layout()
plt.show()

for c_rate, solution in solutions.items():
    temperature_c = solution["Volume-averaged cell temperature [K]"].entries
    ↪ - 273.15
    time_min = solution["Time [min]"].entries
    print(

```



```
f"{c_rate}C: peak temperature = {temperature_c.max():.2f} deg C, "
f"final time = {time_min[-1]:.1f} min"
)
```

The current plot should show the expected CCCV shape. During the charge step, current is initially high and then tapers during the voltage-hold stage. The 5C case should reach the voltage limit quickly and spend a larger fraction of the charge in the tapering stage. The voltage plot should climb toward 4.2 V, then remain near that limit during CV. The temperature plot is the reproduction target. The 1C case should show a small temperature rise. The 3C case should rise more sharply. The 5C case should rise the most and remain elevated during the voltage-hold and rest periods.

There are two important ambiguities. First, PyBaMM's Chen2020 parameter set represents a particular lithium-ion cell, not the exact cell in the Modelica paper. Second, our thermal boundary condition is PyBaMM's lumped parameterization, not their one-dimensional thermal model. Those differences mean a numerical mismatch is expected. In your lab notebook, write this clearly. A reproduction that hides mismatched assumptions is weaker than a reproduction that explains them.

9.3.7 What could go wrong

The 5C case may fail for some solver/settings combinations because it is aggressive. If it fails, replace 5 with 4 and state that the reproduced comparison is 1C/3C/4C. The qualitative lesson remains.

If all three temperature curves nearly overlap, the thermal model may not be active or the current did not reach the intended C-rate. Inspect the current plot before interpreting temperature.

If the charge begins from a high SOC, the CC stage will be very short. Keep the initial discharge step so every case starts from a comparable low-SOC state.

If the voltage hold never terminates, use a less strict cutoff such as "Hold at 4.2 V until C/10" for a faster teaching run.

9.3.8 Reflection

This reproduction exercise taught a research habit: reproduce the claim structure when exact reproduction is impossible. We matched the protocol class, compared the same independent variable, plotted the same dependent variable, and

documented the assumptions that differ. That is often how simulation reproduction begins before you obtain the original code or parameters.

Guided Walkthrough 5

CCCV Charge Under Three Ambient Temperatures

Learning objective: Build the chapter deliverable: a coupled electrothermal PyBaMM CCCV charge under cold, room, and warm ambient temperatures.

This is the workflow you will reuse in your own sodium-ion research. The same cell, same model, and same protocol are simulated under three thermal environments: 0 deg C, 25 deg C, and 45 deg C. The protocol begins by discharging to a low voltage so the charge starts from a comparable low-SOC state. Then it charges with CCCV. We extract charge time, peak temperature, delivered capacity, and the current taper shape.

```
import
import      as
import      as
import      as

def
    =                                "thermal"  "lumped"
    =                                "Chen2020"

    =

    "Discharge at 1C until 2.8 V"
    "Rest for 30 minutes"
    "Charge at 2C until 4.2 V"
    "Hold at 4.2 V until C/20"
    "Rest for 30 minutes"

    ="15 seconds"
    =f"{          } oC"

    =

    =

    =
```

```

    )
    solution = simulation.solve()
    return solution

ambient_cases_c = [0, 25, 45]
ambient_solutions = {}

for ambient_c in ambient_cases_c:
    ambient_solutions[ambient_c] = run_cccv_ambient_case(ambient_c)
    print(f"Solved ambient case: {ambient_c} deg C")

def summarize_solution(solution, ambient_c):
    time_h = solution["Time [h]"].entries
    current_a = solution["Current [A]"].entries
    voltage_v = solution["Terminal voltage [V]"].entries
    temperature_c = solution["Volume-averaged cell temperature [K]"].entries
    ↪ - 273.15

    = < -0.05

    if not any
        =
        =
    else
        =
        =
        = -1 - 0 * 60.0
        = -

    return
        "ambient_c"
        "final_time_min" -1 * 60.0
        "charge_duration_min"
        "charged_capacity_ah"
        "peak_temperature_c" float max
        "temperature_rise_k" float max -
        "max_voltage_v" float max

    =

    for in

```

```

print(summary.to_string(index=False, float_format=lambda value:
    ↳ f"{value:.3f}"))

fig, axes = plt.subplots(4, 1, figsize=(9, 10), sharex=True)

for ambient_c, solution in ambient_solutions.items():
    time_min = solution["Time [min]"].entries
    current_a = solution["Current [A]"].entries
    voltage_v = solution["Terminal voltage [V]"].entries
    temperature_c = solution["Volume-averaged cell temperature [K]"].entries
    ↳ - 273.15
    discharge_capacity_ah = solution["Discharge capacity [A.h]"].entries

    label = f"{ambient_c} deg C ambient"
    axes[0].plot(time_min, current_a, label=label, linewidth=1.4)
    axes[1].plot(time_min, voltage_v, label=label, linewidth=1.4)
    axes[2].plot(time_min, temperature_c, label=label, linewidth=1.4)
    axes[3].plot(time_min, discharge_capacity_ah, label=label, linewidth=1.4)

axes[0].set_ylabel("Current [A]")
axes[1].set_ylabel("Voltage [V]")
axes[2].set_ylabel("Temperature [deg C]")
axes[3].set_ylabel("Discharge cap. [A h]")
axes[3].set_xlabel("Time [min]")

for axis in axes:
    axis.grid(True)
    axis.legend(fontsize=8)

fig.suptitle("Coupled electrothermal CCCV charge under three ambient
    ↳ temperatures")
fig.tight_layout()
plt.show()

summary.to_csv("chapter9_cccv_ambient_summary.csv", index=False)
print("Wrote chapter9_cccv_ambient_summary.csv")

```

PyBaMM uses negative current for charge in this experiment output, so the summary identifies charging with `current_a < -0.05`. That sign convention is easy to miss if you have just come from the hand model, where we used positive current for discharge and did not run a charge. The integration `np.trapezoid(-charge_current_a, charge_time_h)` computes charged ampere-hours with a positive value.

The current plot should show the same nominal CCCV protocol in all three ambient cases, but the detailed taper may differ because temperature changes internal kinetics and transport. The voltage plot should climb to 4.2 V during charge and hold near that limit during CV. The temperature plot should be offset by ambient and should also show different rises. The cold case may exhibit stronger polarization and a different charge duration. The warm case may accept current more easily but operates closer to thermal-aging concern. The room-temperature case is the reference.

The summary table is the start of a publishable comparison. For each ambient condition, report charge duration, peak temperature, temperature rise, and charged capacity. Do not report only charge time. A faster charge that reaches a much higher temperature is not automatically better. A colder case that takes longer may still be valuable if sodium-ion chemistry avoids plating-like limits that would constrain a lithium-ion graphite anode.

9.3.9 What could go wrong

If the cold case fails, reduce the charge rate from 2C to 1.5C. Low-temperature simulations are numerically and physically harder because resistance and transport limitations become more severe.

If the output variable "Discharge capacity [A.h]" looks confusing during charge, remember that PyBaMM's capacity variables are defined by convention. Use current integration when you need a protocol-specific charge capacity.

If the current sign surprises you, plot current before calculating metrics. In PyBaMM experiment outputs, charge commonly appears as negative current.

If the three ambient cases have identical temperatures except for vertical offset, you may be seeing weak thermal feedback. Increase C-rate cautiously or compare voltage and charge-duration metrics to reveal coupling.

9.3.10 Reflection

This is the chapter's core deliverable. You have a repeatable electrothermal sweep with metrics and plots. The same pattern can be adapted to sodium-ion parameter sets, alternative OCV curves, different heat-transfer coefficients, or thermal constraints such as "stop charging if cell temperature exceeds 45 deg C."

9.4 Simulink and Simscape Thermal Modeling: A Practical Bridge

The plan for this chapter includes Simulink thermal modeling with Simscape thermal components. Because the reader profile includes basic Simulink but not Simscape Battery experience, this section is a bridge rather than a required second implementation. The conceptual model is the same as Equation (1). In Simscape language, the cell thermal mass is a `Thermal Mass` block, heat generation is an `Ideal Heat Flow Source`, and convection to ambient is a `Convective Heat Transfer` block connected to a temperature source representing the environment.

A minimal Simulink/Simscape version has five parts. First, an electrical or signal subsystem computes heat generation, usually I^2R for the first model. Second, a `Simulink-PS Converter` sends that heat rate into an `Ideal Heat Flow Source`. Third, the heat source injects heat into a thermal node connected to a `Thermal Mass`. Fourth, that node loses heat through a `Convective Heat Transfer` block to an ambient temperature source. Fifth, a `Temperature Sensor` and `PS-Simulink Converter` send cell temperature back to Simulink for plotting or for resistance lookup.

For a first model, use:

Quantity	Simscape block or parameter	Teaching value
Heat source	<code>Ideal Heat Flow Source</code>	input $I^2 * R$
Thermal mass	<code>Thermal Mass</code>	250 J/K
Convection	<code>Convective Heat Transfer</code>	area 0.03 m ² , coefficient 12 W/(m ² K)
Ambient	<code>Temperature Source</code>	298.15 K
Sensor	<code>Temperature Sensor</code>	output in kelvin

The main misconception to avoid is treating Simscape thermal components as automatically more physical than a Python ODE. They are more convenient for block-diagram integration, especially once you have a pack, coolant loop, or controller, but the same parameters still need justification. A thermal mass block with guessed heat capacity and guessed convection coefficient is no more validated than our hand model. The advantage is composability: later you can connect cells

to cooling plates, coolant channels, or pack enclosures without writing every differential equation yourself.

9.5 Open-Ended Exercises

9.5.1 Exercise 1: Heat-transfer sensitivity

Using Walkthrough 5, sweep the total heat transfer strength by changing "Total heat transfer coefficient [W.m-2.K-1]" in the Chen2020 parameter set. Compare peak temperature for $h = 5, 10, \text{ and } 20 \text{ W.m-2.K-1}$ at 25 deg C ambient.

Hints: copy the `ParameterValues` object before updating it. Keep the same experiment and model. Plot all temperature curves on one axis.

9.5.2 Exercise 2: Thermal cutoff protocol

Modify the CCCV ambient simulation so that charging stops if the cell temperature exceeds 40 deg C . Compare the delivered charge capacity with and without the thermal cutoff.

Hints: PyBaMM experiment strings support voltage and current terminations directly. For a teaching implementation, you may also post-process the solution and identify the first time temperature exceeds the limit.

9.5.3 Exercise 3: Sodium-ion-like OCV and low-temperature interpretation

Return to the hand lumped model in Walkthrough 1. Replace the lithium-ion-like OCV curve with a flatter sodium-ion hard-carbon-inspired curve over 20% to 80% SOC. Keep the same resistance at 25 deg C , then add a simple Arrhenius-like resistance multiplier at 0 deg C . Compare voltage sag and heat generation.

Hints: use a plateau term such as $3.05 + 0.25 * \text{soc} + 0.08 * \tanh((\text{soc} - 0.15) / 0.04) + 0.10 * \tanh((\text{soc} - 0.85) / 0.04)$. For cold resistance, multiply `resistance_ohm` by 1.8.

9.6 Worked Solutions to Open-Ended Exercises

9.6.1 Solution 1: Heat-transfer sensitivity

```
import pybamm
import matplotlib.pyplot as plt

def run_heat_transfer_case(h_value):
    model = pybamm.lithium_ion.SPMe({"thermal": "lumped"})
    params = pybamm.ParameterValues("Chen2020")
    params.update({"Total heat transfer coefficient [W.m-2.K-1]": h_value})

    experiment = pybamm.Experiment(
        [
            "Discharge at 1C until 2.8 V",
            "Rest for 30 minutes",
            "Charge at 2C until 4.2 V",
            "Hold at 4.2 V until C/20",
            "Rest for 30 minutes",
        ],
        period="20 seconds",
        temperature="25 oC",
    )

    sim = pybamm.Simulation(model, parameter_values=params,
    ↪ experiment=experiment)
    return

    = 5.0 10.0 20.0
    = for in

    = 8 4.8
for in
    = "Time [min]"
    temp_c = solution["Volume-averaged cell temperature [K]"].entries -
    ↪ 273.15
    plt.plot(time_min, temp_c, label=f"h = {h_value:g} W/m2/K",
    ↪ linewidth=1.6)

    "Time [min]"
    "Temperature [deg C]"
    "Heat-transfer sensitivity in a lumped PyBaMM model"
    True
```



```
plt.show()

for h_value, solution in results.items():
    temp_c = solution["Volume-averaged cell temperature [K]"].entries -
    ↪ 273.15
    print(f"h = {h_value:g}: peak temperature = {temp_c.max():.2f} deg C")
```

The correct result is a family of temperature curves with lower peaks as h increases. If the curves cross in strange ways, check that each simulation uses a copied or newly constructed parameter object and the same experiment.

9.6.2 Solution 2: Thermal cutoff protocol

```
base_solution = run_cccv_ambient_case(25)

time_h = base_solution["Time [h]"].entries
time_min = base_solution["Time [min]"].entries
current_a = base_solution["Current [A]"].entries
temperature_c = base_solution["Volume-averaged cell temperature [K]"].entries
    ↪ - 273.15

charge_mask = current_a < -0.05
charged_capacity_full_ah = np.trapezoid(-current_a[charge_mask],
    ↪ time_h[charge_mask])

over_limit = np.where(temperature_c > 40.0)[0]
if over_limit.size == 0:
    cutoff_index = len(time_h) - 1
    print("Temperature never exceeded 40 deg C.")
else:
    cutoff_index = over_limit[0]
    print(f"Temperature cutoff reached at {time_min[cutoff_index]:.2f} min.")

allowed = np.arange(len(time_h)) <= cutoff_index
charge_allowed = charge_mask & allowed
charged_capacity_cutoff_ah = np.trapezoid(-current_a[charge_allowed],
    ↪ time_h[charge_allowed])

print(f"Full protocol charged capacity: {charged_capacity_full_ah:.3f} A h")
print(f"Thermal-cutoff charged capacity: {charged_capacity_cutoff_ah:.3f} A
    ↪ h")

plt.figure(figsize=(8, 4.8))
plt.plot(time_min, temperature_c, linewidth=1.6)
```

```

plt.axhline(40.0, color="tab:red", linestyle="--", label="40 deg C cutoff")
plt.axvline(time_min[cutoff_index], color="0.3", linestyle=":", label="cutoff
↳ time")
plt.xlabel("Time [min]")
plt.ylabel("Temperature [deg C]")
plt.title("Post-processed thermal cutoff")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```

This is a post-processed cutoff, not a true closed-loop controller. That distinction matters. It tells you how much capacity would have been delivered before a temperature limit was crossed, but it does not rerun the model with current set to zero after the limit. A publishable controller study should implement the limit inside the experiment or an external-circuit control law.

9.6.3 Solution 3: Sodium-ion-like OCV stress test

```

def sodium_like_ocv_from_soc(soc):
    soc_clipped = np.clip(soc, 0.0, 1.0)
    return (
        2.55
        + 0.28 * soc_clipped
        + 0.08 * np.tanh((soc_clipped - 0.15) / 0.04)
        + 0.10 * np.tanh((soc_clipped - 0.85) / 0.04)
    )

def compare_li_na_voltage_heat(temperature_c):
    cold_multiplier = 1.8 if temperature_c <= 0.0 else 1.0
    soc_grid = np.linspace(0.05, 0.95, 300)
    current_a = 10.0

    li_resistance = 0.018 * cold_multiplier
    na_resistance = 0.022 * cold_multiplier

    li_voltage = ocv_from_soc(soc_grid) - current_a * li_resistance
    na_voltage = sodium_like_ocv_from_soc(soc_grid) - current_a *
↳ na_resistance

    li_heat_w = current_a**2 * li_resistance
    na_heat_w = current_a**2 * na_resistance

```

```

    return soc_grid, li_voltage, na_voltage, li_heat_w, na_heat_w

fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))

for temperature_c in [25.0, 0.0]:
    soc_grid, li_v, na_v, li_heat, na_heat =
    ↪ compare_li_na_voltage_heat(temperature_c)
    axes[0].plot(soc_grid, li_v, label=f"Li-like {temperature_c:g} deg C")
    axes[0].plot(soc_grid, na_v, linestyle="--", label=f"Na-like
    ↪ {temperature_c:g} deg C")
    axes[1].bar(
        [f"Li {temperature_c:g}", f"Na {temperature_c:g}"],
        [li_heat, na_heat],
    )

axes[0].set_xlabel("SOC [-]")
axes[0].set_ylabel("Loaded voltage [V]")
axes[0].grid(True)
axes[0].legend(fontsize=8)

axes[1].set_ylabel("I^2R heat at 10 A [W]")
axes[1].grid(True, axis="y")

fig.suptitle("Teaching comparison: lithium-like and sodium-like
    ↪ low-temperature behavior")
fig.tight_layout()
plt.show()

```

The sodium-like OCV curve should be flatter through the middle SOC range. The cold case should show larger voltage sag and larger heat generation because resistance was increased. This is not a claim that all sodium-ion cells heat more than lithium-ion cells. It is a controlled teaching test showing how OCV shape and resistance assumptions enter voltage and heat predictions.

9.7 Dataset Integration Notes

For thermal work, public data are less abundant than voltage-current-capacity data. Prioritize datasets that include measured temperature and enough current excitation to identify thermal dynamics. The NASA randomized battery usage datasets are useful because they include randomized current operation and

reference cycles. The NASA Open Data portal points to the public dataset pages, and the Zenodo mirror records the PCoE randomized battery usage collection with DOI 10.5281/zenodo.15277374 and a CC BY 4.0 license. Always record the access date and exact resource URL in your research log because NASA legacy resource URLs have changed over time.

The key columns for this chapter are:

Column	Unit	Meaning	Pitfall
time_s	s	elapsed test time	may reset per cycle
current_a	A	applied current	sign convention varies
voltage_v	V	terminal voltage	includes dynamic polarization
temperature_c	deg C	measured cell or chamber temperature	may be surface, chamber, or sensor-specific

When adapting a lithium-ion dataset workflow to sodium-ion, the parser barely changes. The interpretation changes. Sodium-ion data may have lower nominal voltage, different OCV slope, different reversible heat, and different low-temperature resistance behavior. If no sodium-ion temperature dataset is available for your exact chemistry, use lithium-ion data only to validate the workflow and then state clearly which sodium-ion parameters are hypothetical, literature-derived, or fitted from digitized figures.

What Changes for Sodium-Ion?

Three things change immediately for sodium-ion electrothermal simulation. First, the voltage window and OCV shape change. A sodium-ion full cell may operate around a lower nominal voltage than a lithium-ion NMC/graphite cell, and hard-carbon anodes can create broad plateau regions. That affects both voltage limits and the SOC information available to BMS algorithms.

Second, low-temperature behavior can be a genuine sodium-ion advantage, but it is not automatic. The claim depends on electrolyte, hard-carbon structure, electrode

loading, and power target. In simulation terms, you need temperature-dependent resistance, diffusivity, exchange-current density, and possibly plating or sodium-metal deposition constraints if relevant. A model that merely changes the OCV curve is not enough to prove low-temperature superiority.

Third, dataset scarcity changes validation strategy. For lithium-ion, you can often find current-voltage-temperature aging datasets with dozens or hundreds of cells. For sodium-ion, you may need to combine sparse experimental data, digitized literature curves, manufacturer datasheets, and sensitivity analysis. That makes uncertainty reporting more important, not less. A sodium-ion thermal paper should show how conclusions move when heat-transfer coefficient, resistance, entropic coefficient, and ambient temperature vary across plausible ranges.

In early sodium-ion work, a defensible workflow is to build the electrothermal method on a well-documented lithium-ion parameter set, replace only the parameters you can justify for sodium-ion, and label every remaining inherited lithium-ion parameter. Then run sensitivity sweeps to identify which missing sodium-ion measurements matter most. That is a research contribution because it tells experimental collaborators what to measure next.

Chapter Summary and Skill Checklist

- You implemented a lumped thermal energy balance from first principles.
- You connected Bernardi heat generation to I^2R teaching approximations and reversible heat.
- You parsed public NASA current-voltage-temperature data and fitted effective thermal parameters.
- You turned on PyBaMM `isothermal` and `lumped thermal` options.
- You reproduced the qualitative fast-charge temperature comparison from a published CCCV study.
- You built a coupled electrothermal CCCV ambient-temperature sweep.
- You identified which parts of the workflow change for sodium-ion cells.

Commands, functions, and patterns to keep in muscle memory:

- `pybamm.lithium_ion.SPMe({"thermal": "lumped"})`
- `pybamm.Experiment(..., temperature="25 oC")`
- `solution["Volume-averaged cell temperature [K]"].entries`

- `ParameterValues("Chen2020").update({...})`
- `solve_ivp(..., method="BDF")`
- `least_squares(...)` with log-transformed positive parameters
- Plotting current, voltage, temperature, and capacity on synchronized time axes

You should now be able to:

- Explain the difference between isothermal, lumped thermal, and spatial thermal models.
- Estimate a plausible thermal time constant from mass, heat capacity, area, and heat-transfer coefficient.
- Fit a simple thermal model to measured temperature data.
- Run PyBaMM simulations with thermal coupling enabled.
- Compare CCCV charging behavior across ambient temperatures.
- State what assumptions prevent a lithium-ion thermal workflow from being a validated sodium-ion model.

Deliverable

Your deliverable is a notebook named `chapter9_coupled_electrothermal_cccv.ipynb` and a short Markdown report named `chapter9_thermal_interpretation.md`. The notebook should run the Walkthrough 5 ambient-temperature sweep end to end, save `chapter9_cccv_ambient_summary.csv`, and generate publication-quality plots of current, voltage, cell temperature, and charged capacity. The report should answer four questions in prose:

1. Which ambient condition charged fastest, and why?
2. Which ambient condition produced the largest absolute peak temperature and the largest temperature rise?
3. Did thermal coupling visibly alter the electrical trajectory?
4. What would need to change before this workflow could support a sodium-ion fast-charge claim?

A strong partial solution is already present in Walkthrough 5. To turn it into a research deliverable, add a parameter table, write down the PyBaMM and package versions, save each figure as a 300 dpi PNG, and include a paragraph on

limitations. The limitations paragraph is not a formality. It is where you state that Chen2020 is a lithium-ion parameter set, the model is lumped rather than spatial, the cooling coefficient is inherited from the parameter set, and sodium-ion conclusions require sodium-ion parameters.

Further Practice and Reading

Key papers:

1. Bernardi, D., Pawlikowski, E., and Newman, J. “A General Energy Balance for Battery Systems.” *Journal of The Electrochemical Society* 132, 5-12 (1985). This is the classic heat-generation reference behind Equation (3).
2. Romero, A., Goldar, A., and Garone, E. “A Model Predictive Control Application for a Constrained Fast Charge of Lithium-ion Batteries.” *Proceedings of the 13th International Modelica Conference* (2019). Use Figures 5 and 6 as a protocol-comparison reproduction target.
3. Yang, X.-G., Zhang, G., Ge, S., and Wang, C.-Y. “Fast charging of lithium-ion batteries at all temperatures.” *PNAS* 115, 7266-7271 (2018). Read this for the link between fast charging, temperature, and low-temperature limitations.

Official documentation worth bookmarking:

1. PyBaMM thermal models documentation:
<https://docs.pybamm.org/en/v25.10.2/source/examples/notebooks/model>
2. PyBaMM experiment documentation: <https://docs.pybamm.org/>
3. NASA/Zenodo PCoE randomized battery usage dataset:
<https://zenodo.org/records/15277374>

Community resources:

1. PyBaMM GitHub repository and issue tracker:
<https://github.com/pybamm-team/PyBaMM>
2. PyBaMM Discourse and community links from the official documentation site
3. MATLAB Simscape Battery examples for thermal and pack modeling when you move from single-cell scripts to pack-level block diagrams

Chapter 10 is next: Bridging PyBaMM and MATLAB.

Bridging PyBaMM and MATLAB

This chapter is about moving without losing meaning.

Up to now, we have used Python and MATLAB as mostly separate laboratories. PyBaMM gave us physics-based models, experiments, parameter sets, internal electrochemical variables, degradation options, and thermal coupling. MATLAB gave us the BMS habits that working engineers reach for: equivalent-circuit identification, Kalman filters, validation scripts, tables, and eventually Simulink and Simscape Battery. In real publishable work, those worlds rarely stay separate. You may generate a virtual cell in PyBaMM, identify a BMS-ready model in MATLAB, validate it against a held-out protocol, and then return to Python to build publication figures and archive machine-readable results.

Keep Textbook Chapter 8 open for the DFN and SPMe model context, Textbook Chapter 10 open for equivalent-circuit modeling, and the sodium-ion chapter nearby for chemistry-specific cautions. This chapter operationalizes an idea that has appeared quietly throughout the companion: a model is only useful if its assumptions, data conventions, and units survive the trip from one tool to another. PyBaMM and MATLAB can both store voltage and current, but that does not mean they agree about sign convention, interpolation, time base, variable names, solver tolerances, file formats, or reproducibility habits.

By the end of this chapter you will build a complete virtual-cell workflow.

PyBaMM will run a physics-based simulation under two protocols: an identification protocol and a held-out validation protocol. Python will export the resulting data to CSV, HDF5, and MATLAB `.mat` formats. MATLAB will load the exported data, identify a first-order Thevenin ECM, simulate the identified ECM on the validation current, and write the validation result back to disk. Finally, a master Python script will call the MATLAB script through the MATLAB Engine API when MATLAB is available, fall back to a command-line MATLAB call when appropriate, and produce a final comparison figure and RMSE report.

This is publishable-research skill because reviewers do not care that your model worked on your laptop once. They care whether someone can understand the workflow, rerun it, and audit each translation step. If a PyBaMM voltage trace is used as synthetic truth and a MATLAB ECM is used as a reduced-order surrogate, your paper must make the interface explicit. Which current sign convention did you use? Which variables were exported? Was the ECM fit on the same protocol used for validation? Did you interpolate the PyBaMM output onto a uniform MATLAB time grid? Did you preserve the OCV-SOC curve or refit it? Did you record software versions? This chapter teaches you to answer those questions with files, code, and plots rather than hand-waving.

We will move in five stages. First, we build and export a PyBaMM virtual-cell dataset. Second, we inspect the exported files from Python and MATLAB so we know the formats really contain what we think they contain. Third, we write a MATLAB ECM identification script that operates only on exported files, as it would in a real cross-language project. Fourth, we build a Python master script that orchestrates the workflow and validates the MATLAB result. Fifth, we adapt the same pattern to sodium-ion research, where parameter sets and public datasets are thinner and the value of a clean tool bridge is even higher.

10.1 Prerequisites Check

- Required Python software: Python 3.11, pybamm==26.3.1, numpy==1.26.4, scipy==1.13.1, pandas==2.2.2, matplotlib==3.9.0, h5py==3.11.0, and matlabengine==24.2.2 if MATLAB R2024b is installed
- Required MATLAB software: MATLAB R2024b recommended; Optimization Toolbox is helpful but not required because the chapter includes an

fminsearch fallback

- Install command: `python -m pip install pybamm==26.3.1 numpy==1.26.4 scipy==1.13.1 pandas==2.2.2 matplotlib==3.9.0 h5py==3.11.0 matlabengine==24.2.2`
- Required textbook chapters: Textbook Chapter 8 for physics-based models; Textbook Chapter 10 for ECMs; the sodium-ion chapter for chemistry transfer
- Required prior lab chapters: Lab Chapters 1, 3, 4, and 6 are essential; Lab Chapter 7 helps if you want to extend the ECM into a state estimator; Lab Chapter 9 helps if you include temperature later
- External documentation to bookmark: PyBaMM installation and output-management documentation; MathWorks documentation for MATLAB Engine API for Python and MATLAB calling Python
- Estimated time: 14 to 20 hours, depending on whether you run the optional MATLAB Engine workflow or use the file-only workflow

If PyBaMM model setup feels shaky, revisit Lab Chapter 3. If equivalent-circuit identification feels unfamiliar, reread Lab Chapter 6 before doing the MATLAB section. If your MATLAB installation is not available on the machine where Python runs, you can still complete most of the chapter: Python will generate the data and write the MATLAB scripts, then you can run MATLAB manually and return to Python for validation.

10.2 Environment Setup

The biggest installation risk in this chapter is not PyBaMM itself. It is the boundary between Python and MATLAB. Python packages live in virtual environments. MATLAB has its own embedded Python interface and its own external engine package. Your goal is not to make every possible direction work on the first attempt. Your goal is to establish one reproducible cross-language path, then record it.

The PyBaMM documentation for the stable 26.3.1 line states that PyBaMM can be installed with `pip` or `conda`, while warning that the `conda` recipe may lag releases. The same PyBaMM output-management tutorial shows `solution.save_data(...)` exporting selected variables to CSV and MATLAB formats, with the important limitation that CSV and MATLAB export are intended for variables that depend only on time. MathWorks documents the MATLAB

Engine API for Python as the supported way to call MATLAB functions from Python, and the `matlabengine` package on PyPI ties specific package versions to specific MATLAB releases. For MATLAB R2024b, use the `24.2.x` engine family.

10.2.1 Create the Python environment

From a terminal, create a fresh environment for this chapter:

```
cd /home/avirup/SodiumIonBatteryResearch
python3.11 -m venv .envv-chapter10
source .envv-chapter10/bin/activate
python -m pip install --upgrade pip
python -m pip install pybamm==26.3.1 numpy==1.26.4 scipy==1.13.1
↪ pandas==2.2.2 matplotlib==3.9.0 h5py==3.11.0
```

If MATLAB R2024b is installed and you want Python to call MATLAB directly, install the engine package:

```
python -m pip install matlabengine==24.2.2
```

On Windows PowerShell, the activation command is:

```
.\.envv-chapter10\Scripts\Activate.ps1
```

If `matlabengine==24.2.2` is not available for your platform, install the engine package matching your MATLAB release. For MATLAB R2024a, the package family is `24.1.x`. For newer MATLAB releases, check the package metadata and record the exact package version in your research log. This is not clerical busywork. If your pipeline depends on the Python-MATLAB bridge, the engine package is part of the computational method.

10.2.2 Create the chapter workspace

Run the following from the repository root:

```
mkdir -p SimulationCompanion/chapter10_bridge_workspace
cd SimulationCompanion/chapter10_bridge_workspace
mkdir -p data figures matlab results scripts
```

The workflow will write:

Folder	Contents
data/	PyBaMM-generated CSV, HDF5, and MAT files

Folder	Contents
matlab/	MATLAB ECM identification and simulation scripts
results/	MATLAB validation output and summary metrics
figures/	Publication-style plots from Python
scripts/	Python master workflow scripts

Do not scatter these files across your desktop. Cross-language projects become painful when you cannot tell which program produced which artifact. Folder discipline is part of the method.

10.2.3 Verify Python and PyBaMM

Open a Jupyter notebook using the chapter environment, or run this as `scripts/verify_python.py`:

```
import importlib.util

import h5py
import matplotlib
import numpy as np
import pandas as pd
import pybamm
import scipy

print("PyBaMM:", pybamm.__version__)
print("NumPy:", np.__version__)
print("SciPy:", scipy.__version__)
print("pandas:", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
print("h5py:", h5py.__version__)

if importlib.util.find_spec("matlab.engine") is None:
    print("MATLAB Engine for Python: not installed")
else:
    import matlab.engine

    print("MATLAB Engine for Python: installed")

model = pybamm.lithium_ion.SPM()
parameter_values = pybamm.ParameterValues("Chen2020")
experiment = pybamm.Experiment(["Discharge at 1C for 5 minutes"], period="5
↵ seconds")
```

```

simulation = pybamm.Simulation(
    model,
    parameter_values=parameter_values,
    experiment=experiment,
)
solution = simulation.solve()

voltage = solution["Terminal voltage [V]"](solution.t)
print(f"Number of output samples: {len(solution.t)}")
print(f"Initial voltage: {voltage[0]:.4f} V")
print(f"Final voltage: {voltage[-1]:.4f} V")

```

Expected output:

```

PyBaMM: 26.3.1
NumPy: 1.26.4
SciPy: 1.13.1
pandas: 2.2.2
Matplotlib: 3.9.0
h5py: 3.11.0
MATLAB Engine for Python: installed
Number of output samples: 61
Initial voltage: 4.xxx V
Final voltage: 4.xxx V

```

If the engine line says `not installed`, that is acceptable for the file-only path. If PyBaMM reports a different version, decide whether to continue and record the version. For this chapter, the code is intentionally conservative and uses stable PyBaMM APIs, but version drift can still affect variable names and solver behavior.

10.2.4 Verify MATLAB

Open MATLAB and run:

```

clear; clc;
fprintf("MATLAB version: %s\n", version);
fprintf("Current folder: %s\n", pwd);

t = linspace(0, 10, 101)';
y = exp(-t / 3);
plot(t, y, "LineWidth", 1.8);
grid on;
xlabel("Time (s)");

```

```
ylabel("Response (-)");  
title("MATLAB bridge hello-world");
```

You should see a smooth exponential decay from 1 toward zero. This only verifies plotting and script execution. Later we will verify that MATLAB can read files written by Python.

10.2.5 Verify MATLAB Engine from Python

This step is optional but valuable. Run:

```
try:  
    import matlab.engine  
  
    engine = matlab.engine.start_matlab()  
    matlab_version = engine.version()  
    print("Started MATLAB:", matlab_version)  
    result = engine.sqrt(16.0)  
    print("sqrt(16) from MATLAB:", result)  
    engine.quit()  
except Exception as exc:  
    print("Could not start MATLAB Engine.")  
    print(type(exc).__name__ + ":", exc)
```

Expected output:

```
Started MATLAB: 24.2.0.2712019 (R2024b)  
sqrt(16) from MATLAB: 4.0
```

The exact version string will differ. If this fails on Linux with a shared-library error, MATLAB is usually installed outside the default location or its runtime libraries are not visible. Add MATLAB's `bin/glnxa64` directory to `LD_LIBRARY_PATH`, restart the terminal, reactivate the Python environment, and try again. If this fails on macOS with a dynamic-library error, check `DYLD_LIBRARY_PATH` and the MathWorks installation notes for your release. If it fails because MATLAB is not licensed or not installed, use the file-only path: run the MATLAB script manually after Python writes the data.

10.3 Conceptual Bridge: Models, Data, and Contracts Across Tools

In Textbook Chapter 8, the DFN model was a coupled system of conservation laws, kinetic equations, transport equations, and algebraic constraints. PyBaMM

represents that model as symbolic expressions, discretizes it, solves it, and exposes named variables such as terminal voltage, current, electrode stoichiometry, electrolyte concentration, and cell temperature. In Textbook Chapter 10, the Thevenin equivalent circuit reduced the cell to an OCV source, an ohmic resistance, one or more polarization branches, and a SOC integrator. MATLAB is excellent at that reduction because it has mature optimization, signal-processing, control, and Simulink workflows.

The bridge between those two models is data. More precisely, it is a contract about data.

A useful cross-tool contract says what every column means, what its unit is, what sign convention it follows, how time is sampled, how missing values are represented, which protocol produced it, which software versions produced it, and which chemistry or parameter set it represents. Without that contract, a CSV file named `voltage.csv` is not a research artifact. It is a trap. You may fit a beautiful ECM to a current column whose sign is opposite to what MATLAB assumes. You may validate against the same protocol used for fitting and accidentally report training error as predictive error. You may export a nonuniform PyBaMM time base, feed it to a MATLAB script that assumes uniform sampling, and interpret interpolation error as model error.

The most important convention in this chapter is current sign. In many battery-test datasets, positive current means discharge. In some simulation tools or power-electronics contexts, positive current can mean charge. PyBaMM's experiment language is readable because it says "Discharge at 1C" or "Charge at 1C", but exported current variables still need inspection. For the ECM code in this chapter, we will define:

$$I(t) > 0 \quad \text{means discharge current.} \quad (1)$$

The SOC equation is then

$$\frac{dz}{dt} = -\frac{I(t)}{3600Q_{Ah}}, \quad (2)$$

where z is SOC and Q_{Ah} is capacity in ampere-hours. If current is positive during discharge, SOC decreases. If current is negative during charge, SOC increases. This sign convention matches the ECM intuition used in Lab Chapter 6 and will be written into the exported metadata.

The first-order Thevenin ECM we identify in MATLAB is

$$V(t) = U_{oc}(z(t)) - I(t)R_0 - V_1(t), \quad (3)$$

with polarization dynamics

$$\frac{dV_1}{dt} = -\frac{1}{R_1C_1}V_1(t) + \frac{1}{C_1}I(t). \quad (4)$$

Equation (3) assumes positive discharge lowers terminal voltage below OCV. Equation (4) makes the RC polarization voltage rise under discharge and relax during rest. These are not the only possible ECM state definitions. Some books define the polarization state with the opposite sign. That is fine if you are consistent, but cross-language workflows punish hidden conventions. We will therefore write the equations directly into the MATLAB script, use descriptive variable names, and validate the result against PyBaMM output.

Why not export the entire PyBaMM model into MATLAB? In principle, advanced users can exchange models through generated code, FMI, custom C, or surrogate models. That is not the purpose of this chapter. The research habit we want first is simpler and more common: use PyBaMM as a virtual experiment generator, use MATLAB as a reduced-order identification and BMS environment, then compare the reduced model against held-out physics-based data. This mirrors a real laboratory workflow. A physical cell generates current-voltage-temperature data. A BMS engineer identifies an ECM. A validation engineer tests the ECM on a different protocol. Here, the physical cell is replaced by a PyBaMM DFN or SPMe simulation so the entire workflow is runnable on a laptop.

The file formats matter because they carry different tradeoffs. CSV is transparent, diffable, and easy to inspect, but it is weak for metadata and large arrays. HDF5 is structured, compressed, and suitable for larger data, but less friendly to casual inspection. MATLAB `.mat` files are convenient for MATLAB users, but version and naming choices matter. In this chapter we export all three, not because you should always do that in a paper, but because learning the differences once will save you many hours later.

Format	Best use	Strength	Weakness
CSV	Time-series tables and quick inspection	Human-readable, universal	Metadata must be handled separately

Format	Best use	Strength	Weakness
HDF5	Research archives and larger arrays	Structured groups, attributes, compression	Requires library support
MAT	MATLAB handoff	Native MATLAB loading	Variable names must be MATLAB-safe
JSON	Metadata sidecar	Human-readable metadata	Not ideal for long numeric arrays

The bridge pattern for the rest of the chapter is therefore: generate, export, verify, identify, validate, archive. Each verb is a research action. Generate means the virtual experiment is defined in code. Export means the data contract is made explicit. Verify means the receiving tool reads what the sending tool wrote. Identify means parameters are fitted on one dataset. Validate means prediction is tested on a different dataset. Archive means the result can be rerun later.

Guided Walkthrough 1

Generate a PyBaMM Virtual Cell and Export It Three Ways

Learning objective: Run two PyBaMM simulations, convert the outputs into a clean cross-tool table, and export the results to CSV, HDF5, MAT, and JSON metadata files.

We will use PyBaMM’s SPM_e model rather than a full DFN for the main workflow. This is a deliberate teaching choice. SPM_e retains electrolyte effects and is much faster than DFN, which makes repeated reruns comfortable. The workflow is identical for DFN once you are confident. We will generate two protocols. The first, an HPPC-like pulse protocol, will be used for ECM identification. The second, a dynamic validation protocol, will be held out until the ECM parameters are fixed.

Create `scripts/01_generate_virtual_cell.py` with the following complete code:

```
from __future__ import annotations

import json
```

```

from pathlib import Path

import h5py
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pybamm
from scipy.io import savemat

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
FIGURE_DIR = ROOT / "figures"
DATA_DIR.mkdir(parents=True, exist_ok=True)
FIGURE_DIR.mkdir(parents=True, exist_ok=True)

def make_spme_simulation(experiment: pybamm.Experiment) -> pybamm.Simulation:
    """Create a repeatable SPMe simulation using one named parameter set."""
    model = pybamm.lithium_ion.SPMe()
    parameter_values = pybamm.ParameterValues("Chen2020")
    solver = pybamm.CasadiSolver(mode="safe", atol=1e-6, rtol=1e-6)
    return pybamm.Simulation(
        model,
        parameter_values=parameter_values,
        experiment=experiment,
        solver=solver,
    )

def solution_to_table(
    solution: pybamm.Solution,
    label: str,
    nominal_capacity_ah: float,
) -> pd.DataFrame:
    """Sample PyBaMM 0D variables and return a cross-tool battery table."""
    time_s = np.asarray(solution.t, dtype=float)

    voltage_v = np.asarray(solution["Terminal voltage [V]"](time_s),
        dtype=float)
    pybamm_current_a = np.asarray(solution["Current [A]"](time_s),
        dtype=float)

    = -

    =

```

```

if len(time_s) > 1:
    dt_s = np.diff(time_s)
    average_current_a = 0.5 * (
        discharge_current_a[1:] + discharge_current_a[:-1]
    )
    throughput_ah[1:] = np.cumsum(average_current_a * dt_s / 3600.0)

soc = 1.0 - throughput_ah / nominal_capacity_ah
soc = np.clip(soc, 0.0, 1.0)

table = pd.DataFrame(
    {
        "time_s": time_s,
        "current_a": discharge_current_a,
        "voltage_v": voltage_v,
        "soc_estimate": soc,
        "throughput_ah": throughput_ah,
        "protocol": label,
    }
)
return table

def export_table(table: pd.DataFrame, stem: str, metadata: dict[str, object])
→ -> None:
    """Write one table as CSV, HDF5, MAT, and JSON metadata."""
    csv_path = DATA_DIR / f"{stem}.csv"
    h5_path = DATA_DIR / f"{stem}.h5"
    mat_path = DATA_DIR / f"{stem}.mat"
    json_path = DATA_DIR / f"{stem}_metadata.json"

    table.to_csv(csv_path, index=False)

    with h5py.File(h5_path, "w") as h5_file:
        group = h5_file.create_group("battery_timeseries")
        for column in ["time_s", "current_a", "voltage_v", "soc_estimate",
→ "throughput_ah"]:
            group.create_dataset(column, data=table[column].to_numpy(),
→ compression="gzip")
        for key, value in metadata.items():
            h5_file.attrs[key] = json.dumps(value) if isinstance(value,
→ (list, dict)) else value

    savemat(
        mat_path,

```

```

    {
        "time_s": table["time_s"].to_numpy(),
        "current_a": table["current_a"].to_numpy(),
        "voltage_v": table["voltage_v"].to_numpy(),
        "soc_estimate": table["soc_estimate"].to_numpy(),
        "throughput_ah": table["throughput_ah"].to_numpy(),
    },
)

```

```

with json_path.open("w", encoding="utf-8") as file:
    json.dump(metadata, file, indent=2)

```

```

print(f"Wrote {csv_path}")
print(f"Wrote {h5_path}")
print(f"Wrote {mat_path}")
print(f"Wrote {json_path}")

```

```

def plot_protocols(identification: pd.DataFrame, validation: pd.DataFrame) ->

```

```

    None:

```

```

    """Create a quick visual audit of current and voltage for both
    protocols."""

```

```

    fig, axes = plt.subplots(2, 2, figsize=(11, 6), sharex="col")

```

```

    axes[0, 0].plot(identification["time_s"] / 60.0,
    identification["current_a"], color="tab:blue")

```

```

    axes[1, 0].plot(identification["time_s"] / 60.0,
    identification["voltage_v"], color="tab:red")

```

```

    axes[0, 0].set_title("Identification protocol")
    axes[0, 0].set_ylabel("Current (A)")
    axes[1, 0].set_ylabel("Voltage (V)")
    axes[1, 0].set_xlabel("Time (min)")

```

```

    axes[0, 1].plot(validation["time_s"] / 60.0, validation["current_a"],
    color="tab:blue")

```

```

    axes[1, 1].plot(validation["time_s"] / 60.0, validation["voltage_v"],
    color="tab:red")

```

```

    axes[0, 1].set_title("Validation protocol")
    axes[1, 1].set_xlabel("Time (min)")

```

```

    for axis in axes.ravel():
        axis.grid(True, alpha=0.3)

```

```

    fig.tight_layout()

```

```

    figure_path = FIGURE_DIR / "virtual_cell_protocols.png"

```

```

fig.savefig(figure_path, dpi=200)
print(f"Wrote {figure_path}")

def main() -> None:
    nominal_capacity_ah = 5.0

    identification_experiment = pybamm.Experiment(
        [
            "Rest for 20 minutes",
            "Discharge at 1C for 2 minutes",
            "Rest for 20 minutes",
            "Discharge at 2C for 2 minutes",
            "Rest for 20 minutes",
            "Charge at 1C for 2 minutes",
            "Rest for 20 minutes",
            "Discharge at 0.5C for 8 minutes",
            "Rest for 20 minutes",
        ],
        period="5 seconds",
    )

    validation_experiment = pybamm.Experiment(
        [
            "Rest for 10 minutes",
            "Discharge at 0.7C for 5 minutes",
            "Charge at 0.3C for 3 minutes",
            "Discharge at 1.5C for 4 minutes",
            "Rest for 8 minutes",
            "Discharge at 0.4C for 12 minutes",
            "Charge at 0.8C for 4 minutes",
            "Rest for 15 minutes",
        ],
        period="5 seconds",
    )

    print("Solving identification simulation...")
    identification_solution =
    ↪ make_spme_simulation(identification_experiment).solve()

    print("Solving validation simulation...")
    validation_solution = make_spme_simulation(validation_experiment).solve()

    identification_table = solution_to_table(
        identification_solution,

```

```

        label="identification",
        nominal_capacity_ah=nominal_capacity_ah,
    )
    validation_table = solution_to_table(
        validation_solution,
        label="validation",
        nominal_capacity_ah=nominal_capacity_ah,
    )

    metadata = {
        "tool": "PyBaMM",
        "pybamm_version": pybamm.__version__,
        "model": "SPMe",
        "parameter_set": "Chen2020",
        "nominal_capacity_ah": nominal_capacity_ah,
        "current_sign_convention": "current_a > 0 means discharge",
        "time_unit": "s",
        "current_unit": "A",
        "voltage_unit": "V",
        "soc_estimate_note": "SOC computed by coulomb counting from exported
        ↪ current.",
    }

    export_table(identification_table, "pybamm_identification", metadata)
    export_table(validation_table, "pybamm_validation", metadata)
    plot_protocols(identification_table, validation_table)

    print("\nIdentification preview:")
    print(identification_table.head())
    print("\nValidation preview:")
    print(validation_table.head())

if __name__ == "__main__":
    main()

```

Run it from the chapter workspace:

```

cd
↪ /home/avirup/SodiumIonBatteryResearch/SimulationCompanion/chapter10_bridge_workspace
python scripts/01_generate_virtual_cell.py

```

The first block imports the libraries and defines `ROOT`, `DATA_DIR`, and `FIGURE_DIR`. `Path(__file__).resolve().parents[1]` means “the parent folder of the folder containing this script.” Because the script lives in

`chapter10_bridge_workspace/scripts`, this resolves to `chapter10_bridge_workspace` no matter where you launch Python from. That small choice prevents one of the most common cross-tool workflow failures: a script that works only from the author's current folder.

`make_spme_simulation` creates a fresh PyBaMM simulation each time. We use `pybamm.lithium_ion.SPMe()` because it is a good compromise between physics and speed. We use `Chen2020` because it is a familiar parameter set from earlier chapters. The `CasadiSolver` tolerances are set explicitly so reruns are less dependent on solver defaults.

`solution_to_table` is the most important function in the script. It samples PyBaMM variables at `solution.t`, converts the terminal voltage into a NumPy array, and then handles the current sign convention. PyBaMM's reported current may be positive for charge under its internal convention, so the script defines `discharge_current_a = -pybamm_current_a`. This is the moment where the contract is enforced. Every downstream file uses positive discharge current.

The SOC column is deliberately named `soc_estimate`, not `soc_truth`. We compute it by coulomb counting from the exported current and the nominal capacity. PyBaMM has richer internal state variables, but the ECM identification script should behave like a BMS workflow using terminal measurements and a known capacity. Calling it an estimate prevents a subtle misconception: the exported SOC is a bookkeeping variable, not an omniscient electrochemical state.

`export_table` writes four files for each protocol. The CSV is for transparency. The HDF5 file stores numeric arrays under a group called `battery_timeseries` and puts metadata into file attributes. The `.mat` file uses MATLAB-safe variable names with no spaces, brackets, or punctuation. The JSON file is a sidecar metadata record. This looks redundant, but it teaches a good habit: numeric arrays and metadata should travel together.

The expected terminal output should include lines like:

```
Solving identification simulation...
Solving validation simulation...
Wrote ../data/pybamm_identification.csv
Wrote ../data/pybamm_identification.h5
Wrote ../data/pybamm_identification.mat
Wrote ../data/pybamm_identification_metadata.json
Wrote ../figures/virtual_cell_protocols.png
```

The figure `virtual_cell_protocols.png` has two columns. The left column is the identification protocol: rest periods interrupted by sharp pulse events, with voltage stepping down during discharge, recovering during rest, and stepping upward during charge. The right column is the validation protocol: a more dynamic sequence with discharge, charge, rest, and smaller current changes. A correct plot shows current in amperes on the top row and voltage in volts on the bottom row. A wrong plot often has charge and discharge inverted: voltage rises during a supposed discharge pulse or falls during a supposed charge pulse. If you see that, stop and inspect the current sign before fitting anything.

10.3.1 What could go wrong

`KeyError: 'Terminal voltage [V]'` means the PyBaMM variable name has changed or the model did not solve. Print `solution.all_models[0].variables.keys()` only if you need to inspect variable names; do not build research code around casual name searching.

`SolverError` during simulation usually means the protocol drove the cell beyond a cutoff condition or the solver had difficulty with a discontinuity. Reduce the C-rate, shorten a pulse, or add rest. Do not immediately loosen tolerances until the physical protocol has been checked.

`ValueError` from `savemat` usually means a MATLAB variable name is invalid or an object column slipped into the data dictionary. Keep `.mat` exports numeric and MATLAB-safe.

If the exported SOC rises during discharge, your sign convention is wrong. Check the sign of `solution["Current [A]"]` during a known discharge step and correct the conversion before continuing.

10.3.2 Reflection

This walkthrough taught the central habit of cross-tool modeling: export a dataset, not just an array. The voltage trace is useful only because it is accompanied by current, time, SOC convention, nominal capacity, software version, and protocol identity. We will reuse these exact files in MATLAB, so the quality of the bridge is now testable.

Guided Walkthrough 2

Read the Exported Files Back and Audit the Contract

Learning objective: Verify from both Python and MATLAB that the exported files contain the same numeric signals and metadata.

Before fitting an ECM, we will inspect the files. This may feel slow, but it is standard practice in careful research. When tools disagree later, you want to know whether the error is in the model or in the handoff.

Create `scripts/02_audit_exports.py`:

```
from pathlib import Path
import sys
import os
import json
import pandas as pd
from typing import Dict, List

def get_data_path(file_name: str) -> Path:
    """
    Get the path to the data file.
    """
    return Path(__file__).parent / "data" / file_name

def load_data(file_name: str) -> Dict[str, pd.DataFrame]:
    """
    Load the data from the file.
    """
    with open(get_data_path(file_name)) as f:
        data = json.load(f)

    # Convert the data to a dictionary of DataFrames
    data_dict = {}
    for key, value in data.items():
        data_dict[key] = pd.DataFrame(value)

    return data_dict

def audit_data(data_dict: Dict[str, pd.DataFrame]) -> bool:
    """
    Audit the data for consistency.
    """
    # Check if all DataFrames have the same columns
    columns = data_dict["time_s"].columns
    for df in data_dict.values():
        if df.columns != columns:
            return False

    return True
```

```

{
    "time_s": np.asarray(mat_data["time_s"], dtype=float),
    "current_a": np.asarray(mat_data["current_a"], dtype=float),
    "voltage_v": np.asarray(mat_data["voltage_v"], dtype=float),
    "soc_estimate": np.asarray(mat_data["soc_estimate"],
        dtype=float),
    "throughput_ah": np.asarray(mat_data["throughput_ah"],
        dtype=float),
}

def compare_tables(reference: pd.DataFrame, candidate: pd.DataFrame, label:
    str) -> None:
    columns = ["time_s", "current_a", "voltage_v", "soc_estimate",
    "throughput_ah"]
    print f"\nComparing CSV with {      }"
    for      in
        max_error = np.max(np.abs(reference[column].to_numpy() -
    candidate[column].to_numpy()))
        print f" {      :14s} max abs difference = {      :.3e}"

def
    -> None
    =
    / "pybamm_identification.csv"
    =
    / "pybamm_identification.h5"
    =
    / "pybamm_identification.mat"
    =
    / "pybamm_identification_metadata.json"

    =
    =

with      open "r"      ="utf-8" as file
    =      file

print "CSV rows:" len
print "CSV columns:" list
print("JSON current convention:",
    json_metadata["current_sign_convention"])
print "HDF5 current convention:"      "current_sign_convention"

      "HDF5"
      "MAT"

```

```

print("\nVoltage range:")
print(f"  min = {csv_table['voltage_v'].min():.4f} V")
print(f"  max = {csv_table['voltage_v'].max():.4f} V")
print("Current values present:")
print(sorted(csv_table["current_a"].round(4).unique())[:10], "...")

if __name__ == "__main__":
    main()

```

Run:

```
python scripts/02_audit_exports.py
```

Expected output should look like:

```

CSV rows: 1609
CSV columns: ['time_s', 'current_a', 'voltage_v', 'soc_estimate',
↳ 'throughput_ah', 'protocol']
JSON current convention: current_a > 0 means discharge
HDF5 current convention: current_a > 0 means discharge

Comparing CSV with HDF5
time_s      max abs difference = 0.000e+00
current_a   max abs difference = 0.000e+00
voltage_v   max abs difference = 0.000e+00
soc_estimate max abs difference = 0.000e+00
throughput_ah max abs difference = 0.000e+00

Comparing CSV with MAT
time_s      max abs difference = 0.000e+00
current_a   max abs difference = 0.000e+00
voltage_v   max abs difference = 0.000e+00
soc_estimate max abs difference = 0.000e+00
throughput_ah max abs difference = 0.000e+00

```

Tiny differences such as $1e-15$ are harmless floating-point roundoff. Differences near $1e-3$ in voltage or current are not harmless and mean one file was written, parsed, or reshaped incorrectly.

Now create `matlab/audit_exports.m`:

```

clear; clc;

chapterRoot = fileparts(fileparts(mfilename("fullpath")));

```

```

dataDir = fullfile(chapterRoot, "data");

csvPath = fullfile(dataDir, "pybamm_identification.csv");
matPath = fullfile(dataDir, "pybamm_identification.mat");

csvTable = readtable(csvPath);
matData = load(matPath);

fprintf("CSV rows: %d\n", height(csvTable));
fprintf("MAT rows: %d\n", numel(matData.time_s));
fprintf("CSV voltage range: %.4f V to %.4f V\n", ...
    min(csvTable.voltage_v), max(csvTable.voltage_v));
fprintf("MAT voltage range: %.4f V to %.4f V\n", ...
    min(matData.voltage_v), max(matData.voltage_v));

timeError = max(abs(csvTable.time_s - matData.time_s(:)));
currentError = max(abs(csvTable.current_a - matData.current_a(:)));
voltageError = max(abs(csvTable.voltage_v - matData.voltage_v(:)));

fprintf("Max time difference: %.3e s\n", timeError);
fprintf("Max current difference: %.3e A\n", currentError);
fprintf("Max voltage difference: %.3e V\n", voltageError);

figure;
tiledlayout(2, 1);

nexttile;
plot(csvTable.time_s / 60, csvTable.current_a, "LineWidth", 1.4);
grid on;
ylabel("Current (A)");
title("MATLAB audit of Python-exported data");

nexttile;
plot(csvTable.time_s / 60, csvTable.voltage_v, "LineWidth", 1.4);
grid on;
xlabel("Time (min)");
ylabel("Voltage (V)");

```

Run it from MATLAB:

```
run("SimulationCompanion/chapter10_bridge_workspace/matlab/audit_exports.m")
```

The MATLAB output should report the same number of rows and nearly zero differences between CSV and MAT values. The plot should match the identification protocol from Walkthrough 1. This is a small but important

milestone: MATLAB is now reading Python-generated data without guessing.

The Python audit script uses `h5py` to read arrays back from the HDF5 group, `scipy.io.loadmat` to read the MAT file, and `pandas.read_csv` to read the CSV. It then compares every numeric column. The MATLAB audit script performs the same check from the receiving side. In real projects, keep a script like this around. It becomes your first diagnostic when a collaborator says, “The MATLAB fit looks wrong.”

10.3.3 What could go wrong

If MATLAB loads `time_s` as a row vector and the table column is a column vector, expressions may silently expand in newer MATLAB releases. The script uses `matData.time_s(:)` to force column shape. This habit prevents shape bugs.

If `readtable` changes column names, MATLAB may have modified names to make them valid identifiers. Our columns are already MATLAB-safe, so that should not happen. If you export names such as `Voltage [V]`, MATLAB may rename them unless you control import options.

If HDF5 metadata appears as byte strings in Python, decode it before comparing. Attribute type behavior can differ across library versions. For this chapter, simple strings should round-trip cleanly.

If the figure is blank in MATLAB, call `drawnow` or check that the script is running from a desktop MATLAB session rather than a no-display batch mode.

10.3.4 Reflection

This walkthrough may be the least glamorous chapter section and one of the most important. You verified the bridge before trusting it. That is the difference between research code and a lucky demo.

Guided Walkthrough 3

Identify a Thevenin ECM in MATLAB from PyBaMM Data

Learning objective: Fit a first-order ECM in MATLAB using only exported PyBaMM current-voltage data.

Now MATLAB takes over. We will fit four ECM ingredients: an OCV-SOC polynomial and three dynamic parameters, R_0 , R_1 , and C_1 . This is not the most

advanced ECM identification method in the field. Chapter 6 introduced more careful HPPC-based methods. Here we choose a robust workflow that can run from exported files and produce a useful cross-validation target.

Create `matlab/identify_ecm_from_pybamm.m`:

```
function identify_ecm_from_pybamm
%IDENTIFY_ECM_FROM_PYBAMM Fit a 1RC Thevenin ECM from PyBaMM virtual-cell
    data.

clear; clc;

chapterRoot = fileparts(fileparts(mfilename("fullpath")));
dataDir = fullfile(chapterRoot, "data");
resultDir = fullfile(chapterRoot, "results");
figureDir = fullfile(chapterRoot, "figures");

if ~exist(resultDir, "dir")
    mkdir(resultDir);
end
if ~exist(figureDir, "dir")
    mkdir(figureDir);
end

identificationPath = fullfile(dataDir, "pybamm_identification.csv");
validationPath = fullfile(dataDir, "pybamm_validation.csv");

idData = readtable(identificationPath);
valData = readtable(validationPath);

nominalCapacityAh = 5.0;

[ocvCoefficients, ocvSoc, ocvVoltage] = fit_ocv_curve(idData);

initialGuess = log([0.015, 0.010, 2500.0]);
objective = @(logParameters) voltage_rmse_objective( ...
    logParameters, idData, nominalCapacityAh, ocvCoefficients);

bestLogParameters = fminsearch( ...
    objective, ...
    initialGuess, ...
    optimset("Display", "iter", "MaxIter", 300, "MaxFunEvals", 800));

bestParameters = exp(bestLogParameters);
r00hm = bestParameters(1);
```

```

r10hm = bestParameters(2);
c1Farad = bestParameters(3);

idPrediction = simulate_thevenin( ...
    idData.time_s, idData.current_a, nominalCapacityAh, ...
    idData.soc_estimate(1), ocvCoefficients, r00hm, r10hm, c1Farad);

valPrediction = simulate_thevenin( ...
    valData.time_s, valData.current_a, nominalCapacityAh, ...
    valData.soc_estimate(1), ocvCoefficients, r00hm, r10hm, c1Farad);

idRmseMv = rmse_mv(idData.voltage_v, idPrediction.voltage_v);
valRmseMv = rmse_mv(valData.voltage_v, valPrediction.voltage_v);

fprintf("\nIdentified ECM parameters\n");
fprintf("R0 = %.6f ohm\n", r00hm);
fprintf("R1 = %.6f ohm\n", r10hm);
fprintf("C1 = %.2f F\n", c1Farad);
fprintf("tau1 = %.2f s\n", r10hm * c1Farad);
fprintf("Identification RMSE = %.2f mV\n", idRmseMv);
fprintf("Validation RMSE = %.2f mV\n", valRmseMv);

parameterTable = table( ...
    r00hm, r10hm, c1Farad, r10hm * c1Farad, idRmseMv, valRmseMv, ...
    "VariableNames", ["R0_ohm", "R1_ohm", "C1_F", "Tau1_s", ...
    "Identification_RMSE_mV", "Validation_RMSE_mV"]);
parameterPath = fullfile(resultDir, "identified_ecm_parameters.csv");
writetable(parameterTable, parameterPath);

validationOutput = table( ...
    valData.time_s, valData.current_a, valData.voltage_v, ...
    valPrediction.voltage_v, valPrediction.soc, valPrediction.v1, ...
    "VariableNames", ["time_s", "current_a", "pybamm_voltage_v", ...
    "ecm_voltage_v", "ecm_soc", "ecm_polarization_v"]);
validationPathOut = fullfile(resultDir, "validation_ecm_prediction.csv");
writetable(validationOutput, validationPathOut);

save(fullfile(resultDir, "identified_ecm_parameters.mat"), ...
    "r00hm", "r10hm", "c1Farad", "ocvCoefficients", ...
    "idRmseMv", "valRmseMv");

make_validation_plot(valData, valPrediction, valRmseMv, figureDir);
make_ocv_plot(ocvSoc, ocvVoltage, ocvCoefficients, figureDir);

fprintf("Wrote %s\n", parameterPath);

```

```

fprintf("Wrote %s\n", validationPathOut);
end

function [coefficients, restSoc, restVoltage] = fit_ocv_curve(data)
currentThresholdA = 0.02;
restMask = abs(data.current_a) < currentThresholdA;

restSoc = data.soc_estimate(restMask);
restVoltage = data.voltage_v(restMask);

if numel(restSoc) < 20
    error("Not enough rest data to fit OCV curve.");
end

[restSocUnique, uniqueIndex] = unique(restSoc, "stable");
restVoltageUnique = restVoltage(uniqueIndex);

polynomialOrder = 5;
coefficients = polyfit(restSocUnique, restVoltageUnique, polynomialOrder);
end

function objectiveValue = voltage_rmse_objective( ...
    logParameters, data, nominalCapacityAh, ocvCoefficients)

parameters = exp(logParameters);
r00hm = parameters(1);
r10hm = parameters(2);
c1Farad = parameters(3);

prediction = simulate_thevenin( ...
    data.time_s, data.current_a, nominalCapacityAh, ...
    data.soc_estimate(1), ocvCoefficients, r00hm, r10hm, c1Farad);

voltageError = data.voltage_v - prediction.voltage_v;
objectiveValue = sqrt(mean(voltageError.^2));
end

function prediction = simulate_thevenin( ...
    timeS, currentA, nominalCapacityAh, initialSoc, ...
    ocvCoefficients, r00hm, r10hm, c1Farad)

timeS = timeS(:);

```



```

currentA = currentA(:);

n = numel(timeS);
soc = zeros(n, 1);
v1 = zeros(n, 1);
voltage = zeros(n, 1);

soc(1) = initialSoc;
v1(1) = 0.0;
voltage(1) = polyval(ocvCoefficients, soc(1)) - currentA(1) * r00hm - v1(1);

tau1 = r10hm * c1Farad;

for k = 2:n
    dt = timeS(k) - timeS(k - 1);
    previousCurrent = currentA(k - 1);

    soc(k) = soc(k - 1) - previousCurrent * dt / (3600.0 *
        ↪ nominalCapacityAh);
    soc(k) = min(max(soc(k), 0.0), 1.0);

    alpha = exp(-dt / tau1);
    v1(k) = alpha * v1(k - 1) + r10hm * (1.0 - alpha) * previousCurrent;

    voltage(k) = polyval(ocvCoefficients, soc(k)) - currentA(k) * r00hm -
        ↪ v1(k);
end

prediction = table(soc, v1, voltage, ...
    "VariableNames", ["soc", "v1", "voltage_v"]);
end

function value = rmse_mv(referenceVoltage, predictedVoltage)
errorV = referenceVoltage(:) - predictedVoltage(:);
value = 1000.0 * sqrt(mean(errorV.^2));
end

function make_validation_plot(data, prediction, valRmseMv, figureDir)
figure("Color", "w", "Position", [100, 100, 900, 650]);
tiledlayout(3, 1, "TileSpacing", "compact");

nexttile;
plot(data.time_s / 60, data.current_a, "k", "LineWidth", 1.2);

```

```

grid on;
ylabel("Current (A)");
title("Held-out PyBaMM validation protocol");

nexttile;
plot(data.time_s / 60, data.voltage_v, "LineWidth", 1.5);
hold on;
plot(data.time_s / 60, prediction.voltage_v, "--", "LineWidth", 1.5);
grid on;
ylabel("Voltage (V)");
legend("PyBaMM virtual cell", "MATLAB ECM", "Location", "best");
title(sprintf("Validation voltage, RMSE = %.2f mV", valRmseMv));

nexttile;
plot(data.time_s / 60, 1000 * (data.voltage_v - prediction.voltage_v), ...
     "LineWidth", 1.2);
grid on;
xlabel("Time (min)");
ylabel("Error (mV)");

exportgraphics(gcf, fullfile(figureDir, "matlab_validation_ecm.png"), ...
    "Resolution", 200);
end

function make_ocv_plot(restSoc, restVoltage, coefficients, figureDir)
socGrid = linspace(min(restSoc), max(restSoc), 200);
ocvFit = polyval(coefficients, socGrid);

figure("Color", "w", "Position", [150, 150, 700, 450]);
plot(restSoc, restVoltage, ".", "MarkerSize", 8);
hold on;
plot(socGrid, ocvFit, "LineWidth", 1.8);
grid on;
xlabel("SOC (-)");
ylabel("Voltage (V)");
title("OCV approximation from PyBaMM rest segments");
legend("Rest samples", "Polynomial fit", "Location", "best");

exportgraphics(gcf, fullfile(figureDir, "matlab_ocv_fit.png"), ...
    "Resolution", 200);
end

```

Run from MATLAB:

```
addpath("SimulationCompanion/chapter10_bridge_workspace/matlab")
identify_ecm_from_pybamm
```

The script begins by locating the chapter root from the script path. This is better than relying on MATLAB's current folder, because people often run MATLAB scripts from whatever folder happened to be open. It reads the identification and validation CSV files written by Python. It fixes the nominal capacity at 5.0 Ah, matching the Python metadata.

`fit_ocv_curve` extracts low-current samples, treats them as approximate rest data, and fits a fifth-order polynomial between `soc_estimate` and voltage. This is a teaching shortcut. In careful HPPC identification, you would use sufficiently relaxed rest endpoints, smooth the OCV curve, and avoid overfitting. Here the protocol contains rest segments, and the virtual data are clean, so the shortcut is acceptable. The function uses `unique(..., "stable")` because polynomial fitting does not need repeated identical SOC entries from long rest periods.

The dynamic parameters are optimized in log space. This is a standard trick when parameters must remain positive. Instead of asking `fminsearch` to fit `R0`, `R1`, and `C1` directly, we fit their logarithms and exponentiate inside the objective. That prevents negative resistances and capacitances without needing Optimization Toolbox. If you have Optimization Toolbox, `lsqnonlin` with bounds is often better; `fminsearch` is included because it works in base MATLAB.

`simulate_thevenin` uses the exact discrete-time update for the RC polarization state under piecewise-constant current:

$$V_{1,k} = e^{-\Delta t/\tau_1} V_{1,k-1} + R_1 (1 - e^{-\Delta t/\tau_1}) I_{k-1}. \quad (5)$$

This is better than a forward-Euler update when sampling intervals vary. PyBaMM's output time base is usually regular because we set `period="5 seconds"`, but exact updates cost nothing and are more robust.

Expected MATLAB output will vary, but it should have this shape:

```
Identified ECM parameters
R0 = 0.0xxxxxx ohm
R1 = 0.0xxxxxx ohm
C1 = xxxx.xx F
taul = xx.xx s
```

```
Identification RMSE = xx.xx mV
Validation RMSE = xx.xx mV
Wrote .../results/identified_ecm_parameters.csv
Wrote .../results/validation_ecm_prediction.csv
```

Do not expect sub-millivolt validation error. The SPMe virtual cell has electrochemical dynamics that a first-order ECM cannot perfectly reproduce. A validation RMSE in the range of a few millivolts to a few tens of millivolts is plausible for this teaching workflow. If the error is hundreds of millivolts, suspect current sign, SOC initialization, OCV fit, or a failed optimization.

The validation plot has three panels. The top panel shows the held-out current profile. The middle panel overlays PyBaMM voltage and MATLAB ECM voltage. A good result tracks the broad voltage response and rest recoveries, with visible mismatch around rapid current transitions. The bottom panel shows voltage error in millivolts. A good result has errors centered near zero without a long monotonic drift. A wrong result often shows a large step error at every current transition or a steadily growing bias across the whole validation protocol.

10.3.5 What could go wrong

If `fminsearch` drives `C1` to an extremely large value, the identification data may not contain enough relaxation information to identify the time constant. Add longer rest periods or stronger pulses to the identification protocol.

If the OCV curve oscillates wildly, the fifth-order polynomial is overfitting. Lower the polynomial order to 3, or fit a monotonic interpolant. In a paper, you would show and justify this choice.

If validation error is much worse than identification error, that is not automatically a bug. It may mean the ECM is overfitted to pulses and weak on dynamic current. This is exactly why held-out validation exists.

If MATLAB cannot find the CSV files, check that `chapterRoot` resolves to `chapter10_bridge_workspace`. Run `disp(chapterRoot)` inside the script.

10.3.6 Reflection

You have now performed a realistic reduced-order modeling workflow: a physics-based simulator produced virtual experimental data, and MATLAB identified a BMS-style ECM without access to PyBaMM internals. This separation

is powerful. It lets you test how much electrochemical behavior survives reduction into a circuit model.

Guided Walkthrough 4

Orchestrate the Whole Pipeline from Python

Learning objective: Build a master Python script that runs the PyBaMM generator, invokes MATLAB when available, and creates a final validation figure from MATLAB's output.

Manual handoffs are useful while learning. Publishable workflows need a one-command path. The script below runs the generation script, tries to call the MATLAB identification function through MATLAB Engine, and then reads the MATLAB output back into Python. If MATLAB Engine is not available, the script prints the exact MATLAB command to run manually and stops cleanly.

```
Create scripts/03 run bridge pipeline.py:
```

```

from os import path
import sys
import os
import sys
from os import path
import sys
import sys as sys
import sys as sys
import sys as sys

__file__ = sys.argv[1]
scripts = path.dirname(__file__)
matlab = path.dirname(scripts)
results = path.dirname(matlab)
figures = path.dirname(results)

def main():
    """
    -> None
    """
    sys.argv = sys.argv[1:]
    print f"Running {sys.argv[0]} {sys.argv[1:]}"
    sys.exit(0)

def __main__():
    """
    -> bool
    """
    main()

```

```

if importlib.util.find_spec("matlab.engine") is None:
    print("MATLAB Engine for Python is not installed.")
    print_manual_matlab_command()
    return False

try:
    import matlab.engine

    print("Starting MATLAB Engine...")
    engine = matlab.engine.start_matlab()
    engine.addpath(str(MATLAB_DIR), nargout=0)
    engine.identify_ecm_from_pybamm(nargout=0)
    engine.quit()
    return True
except Exception as exc:
    print("MATLAB Engine call failed.")
    print(type(exc).__name__ + ":", exc)
    print_manual_matlab_command()
    return False

def print_manual_matlab_command() -> None:
    print("\nRun this in MATLAB, then rerun this Python script:")
    print(f'addpath("{MATLAB_DIR}")')
    print("identify_ecm_from_pybamm")

def make_python_validation_figure() -> None:
    validation_path = RESULT_DIR / "validation_ecm_prediction.csv"
    parameter_path = RESULT_DIR / "identified_ecm_parameters.csv"

    if not validation_path.exists():
        raise FileNotFoundError(
            f"Missing {validation_path}. Run the MATLAB identification step
            ↪ first."

            =
            =
            = float                                0 "Validation_RMSE_mV"

            = "time_s" / 60.0
            = 1000.0 *
              "pybamm_voltage_v" - "ecm_voltage_v"

```

```

fig, axes = plt.subplots(3, 1, figsize=(10, 7), sharex=True)

axes[0].plot(time_min, validation["current_a"], color="black",
↪ linewidth=1.2)
axes[0].set_ylabel("Current (A)")
axes[0].set_title("Cross-tool validation: PyBaMM virtual cell vs MATLAB
↪ ECM")

axes[1].plot(
    time_min,
    validation["pybamm_voltage_v"],
    label="PyBaMM virtual cell",
    linewidth=1.6,
)
axes[1].plot(
    time_min,
    validation["ecm_voltage_v"],
    "--",
    label="MATLAB ECM",
    linewidth=1.6,
)
axes[1].set_ylabel("Voltage (V)")
axes[1].legend(loc="best")

axes[2].plot(time_min, error_mv, color="tab:red", linewidth=1.2)
axes[2].axhline(0.0, color="black", linewidth=0.8)
axes[2].set_ylabel("Error (mV)")
axes[2].set_xlabel("Time (min)")

for axis in axes:
    axis.grid(True, alpha=0.3)

fig.text(
    0.99,
    0.01,
    f"Held-out validation RMSE = {val_rmse_mv:.2f} mV",
    ha="right",
    va="bottom",
)
fig.tight_layout(rect=[0, 0.03, 1, 1])

FIGURE_DIR.mkdir(parents=True, exist_ok=True)
figure_path = FIGURE_DIR / "python_cross_tool_validation.png"
fig.savefig(figure_path, dpi=250)

```

```

print(f"Wrote {figure_path}")

print("\nParameter summary:")
print(parameters.to_string(index=False))

def main() -> None:
    run_python_generation()
    matlab_completed = run_matlab_identification()
    if not matlab_completed:
        return
    make_python_validation_figure()

if __name__ == "__main__":
    main()

```

Run:

```
python scripts/03_run_bridge_pipeline.py
```

If MATLAB Engine works, the script will run end to end. If it does not, you will see a manual MATLAB command. Run that command in MATLAB, then either rerun the whole script or comment out `run_python_generation()` and `run_matlab_identification()` while you test the final plotting section.

The orchestration script uses `subprocess.run` for the Python generation step instead of importing the generation script. That is intentional. In research pipelines, scripts often have side effects: they write files, create folders, and print logs. Running them as scripts mirrors command-line use and catches path assumptions that imports may hide. The MATLAB step uses the engine only if `matlab.engine` is importable. This avoids making MATLAB a hard dependency for readers who do not have it on the same machine.

The final Python figure should match the MATLAB validation figure. This double plotting is not wasteful. It proves that MATLAB's output file is readable by Python and that the final paper figures can be produced from archived results rather than from a MATLAB figure window.

10.3.7 What could go wrong

If the Python script starts MATLAB but hangs for a long time, MATLAB may be waiting on license checkout, startup scripts, or a dialog. Start MATLAB manually once and resolve any prompts before using the engine.

If the engine cannot find `identify_ecm_from_pybamm`, check that `engine.addpath(str(MATLAB_DIR), nargout=0)` points to the folder containing the `.m` file. MATLAB function files must have the same name as the primary function.

If `validation_ecm_prediction.csv` exists but has old results, delete the `results/` files and rerun. In serious work, include timestamps or run IDs. For this teaching chapter, we keep filenames stable so the workflow is easy to inspect.

If Python reports a missing display backend while plotting on a server, set Matplotlib to a noninteractive backend before importing `pyplot`, or run the script in an environment with display support. Saving PNG files does not require an interactive window.

10.3.8 Reflection

You now have a one-command bridge from PyBaMM to MATLAB and back. The important achievement is not that Python can start MATLAB. The important achievement is that each tool does the job it is good at while the data contract remains explicit and auditable.

Guided Walkthrough 5

Make the Workflow Deterministic and Version-Control Friendly

Learning objective: Add environment capture, deterministic settings, and a run manifest so a future reader can reproduce the cross-language result.

Cross-tool reproducibility fails in quiet ways. A PyBaMM version changes a variable name. MATLAB changes a table import rule. A parameter set is updated. A script is run from a different folder. A result CSV is overwritten and no one remembers which commit produced it. This walkthrough adds a manifest that records versions, file hashes, and workflow settings.

Create `scripts/04_write_manifest.py`:

```

from          import

import
import
import
import
import
from          import
from          import

import
import
import      as
import      as
import
import

    =      __file__          1
    =      / "data"
    =      / "results"

def          -> str
    =
    with      open "rb" as file
        for      in iter lambda file      1024 * 1024      b""

    return

def          -> str
    try
        =
        "git" "rev-parse" "HEAD"
        =      1
        =True
        =True
        =True

    return
except Exception
    return "unknown"

```

```

def matlab_engine_status() -> str:
    try:
        import matlab.engine # noqa: F401

        return "installed"
    except Exception:
        return "not installed"

def collect_files() -> list[dict[str, str]]:
    paths = []
    for folder in [DATA_DIR, RESULT_DIR]:
        if folder.exists():
            paths.extend(sorted(folder.glob("*")))

    records = []
    for path in paths:
        if path.is_file():
            records.append(
                {
                    "path": str(path.relative_to(ROOT)),
                    "sha256": sha256_file(path),
                }
            )
    return records

def main() -> None:
    manifest = {
        "created_utc": datetime.now(timezone.utc).isoformat(),
        "python_executable": sys.executable,
        "python_version": sys.version,
        "platform": platform.platform(),
        "git_commit": git_commit(),
        "packages": {
            "pybamm": pybamm.__version__,
            "numpy": np.__version__,
            "scipy": scipy.__version__,
            "pandas": pd.__version__,
            "matplotlib": matplotlib.__version__,
            "h5py": h5py.__version__,
            "matlab_engine": matlab_engine_status(),
        },
        "workflow_contract": {
            "current_sign_convention": "current_a > 0 means discharge",

```

```

        "time_unit": "s",
        "current_unit": "A",
        "voltage_unit": "V",
        "identification_file": "data/pybamm_identification.csv",
        "validation_file": "data/pybamm_validation.csv",
        "matlab_result_file": "results/validation_ecm_prediction.csv",
    },
    "files": collect_files(),
}

manifest_path = RESULT_DIR / "run_manifest.json"
RESULT_DIR.mkdir(parents=True, exist_ok=True)
with manifest_path.open("w", encoding="utf-8") as file:
    json.dump(manifest, file, indent=2)

print(f"Wrote {manifest_path}")

if __name__ == "__main__":
    main()

```

Run after the pipeline:

```
python scripts/04_write_manifest.py
```

The manifest is a JSON file with package versions, platform information, the current Git commit if available, the data contract, and SHA-256 hashes of files in `data/` and `results/`. Hashes are useful because they tell you whether a file changed even when its name did not. If a collaborator reruns the workflow and the hash changes, that does not automatically mean something is wrong, but it creates a precise question: which file changed, and why?

For version control, commit scripts and small metadata files. Be careful with generated data. Synthetic PyBaMM CSV files in this chapter are small enough to commit if you want a fully reproducible example. Large public datasets should usually be downloaded by script or tracked with a data tool rather than committed to Git. The rule from Lab Chapter 1 still applies: source code, environment files, and lightweight metadata belong in Git; large raw data usually do not.

A strong commit sequence for this chapter might look like:

```
git add SimulationCompanion/chapter10_bridge_workspace/scripts
git add SimulationCompanion/chapter10_bridge_workspace/matlab
```

```
git add  
  ↳ SimulationCompanion/chapter10_bridge_workspace/results/run_manifest.json  
git -m "Add PyBaMM MATLAB bridge workflow"
```

If you choose to commit generated figures and small synthetic CSV files, do so intentionally:

```
git add SimulationCompanion/chapter10_bridge_workspace/data/*.csv  
git add SimulationCompanion/chapter10_bridge_workspace/figures/*.png  
git commit -m "Archive virtual cell bridge outputs"
```

10.3.9 What could go wrong

If `git_commit` returns unknown, the script may be outside a Git repository or `git` may not be installed. This does not block the workflow, but record the situation in your research log.

If hashes differ between runs even with the same code, inspect whether timestamps, metadata ordering, or solver outputs changed. CSV files from deterministic simulations should usually be stable. MAT files may include format details that differ across MATLAB versions.

If the manifest contains absolute paths you do not want to publish, edit the script to store paths relative to the chapter root. The provided script already does this for archived files.

10.3.10 Reflection

This walkthrough turns a useful workflow into a reproducible workflow. The difference is not glamorous, but it is exactly what separates a notebook result from a defensible computational method.

Dataset Integration

Using a Real Dataset at the Tool Boundary

The primary dataset in this chapter is a PyBaMM-generated virtual dataset because the deliverable requires a virtual-cell workflow. That is appropriate for learning the bridge because the true protocol, model, and parameter set are fully known. Real experimental data should enter once the bridge is trusted.

The most natural public dataset to plug into the same workflow is the CALCE HPPC-style data used in Lab Chapter 6. The workflow is:

1. Parse CALCE current, voltage, and time into the same canonical columns: `time_s`, `current_a`, `voltage_v`, and `soc_estimate`.
2. Export the parsed table to CSV and MAT using the same variable names.
3. Run `identify_ecm_from_pybamm.m` after renaming the input paths or generalizing the script arguments.
4. Validate on a different CALCE cycle or pulse segment.

The important point is that the MATLAB ECM script does not need to know whether the file came from PyBaMM or a cyclers. If the contract is the same, the tool boundary is the same. This is exactly how you should design research workflows: one parser per data source, one canonical table format, and reusable model scripts downstream.

A minimal adapter for an already-clean experimental CSV would look like this:

```
from      import

import    as
from      import

def                                              -> None

    =

    =

        "time_s"      "time_s"      float
        "current_a"   "current_a"   float
        "voltage_v"   "voltage_v"   float
        "soc_estimate" "soc_estimate" float
        "throughput_ah" "throughput_ah" float

        ".csv"      =False

        ".mat"

        for      in
```

The pitfalls are familiar now. Experimental timestamps may be absolute date-times rather than elapsed seconds. Current may use positive charge instead of positive discharge. Voltage may include rest periods, charge steps, and discharge steps in one file. Null values can appear when the cyclers changes state. SOC is rarely measured directly and must be reconstructed by coulomb counting with careful initial conditions. The bridge does not remove these problems, but it gives you one place to solve them.

For sodium-ion, public full-cell datasets remain much sparser than lithium-ion datasets. A practical sodium-ion project often begins by validating the data workflow on lithium-ion public data, then applying the same workflow to sodium-ion data digitized from papers, generated by a validated simulation, or provided by a collaborator. State this honestly in manuscripts. A lithium-ion validation proves the software method, not the sodium-ion chemistry.

Reproduction Exercise

Reproduce PyBaMM's Output-Export Pattern and Extend It

This chapter's reproduction exercise is a software-method reproduction rather than a figure reproduction. PyBaMM's official output-management tutorial demonstrates saving a simulation, saving a solution, and exporting selected 0D variables to CSV and MATLAB formats with short MATLAB-safe names.

Reproduce that pattern in your environment, then extend it to the virtual-cell workflow in this chapter.

Start with this minimal notebook cell:

```
import pybamm

model = pybamm.lithium_ion.SPMe()
simulation = pybamm.Simulation(model,
    ↪ parameter_values=pybamm.ParameterValues("Chen2020"))
solution = simulation.solve([0, 600])

solution.save_data(
    "tutorial_style_export.csv",
    ["Current [A]", "Terminal voltage [V]"],
    to_format="csv",
)
solution.save_data(
```

```

"tutorial_style_export.mat",
["Current [A]", "Terminal voltage [V]"],
to_format="matlab",
short_names={
    "Current [A]": "I",
    "Terminal voltage [V]": "V",
},
)

```

Your reproduction target is not visual. It is file equivalence and usability. You should be able to load the CSV with pandas and the MAT file with MATLAB. Then answer three questions in your research log. First, which variable names did PyBaMM accept for export? Second, why do MATLAB exports need `short_names`? Third, why does this chapter use a custom pandas/HDF5/MAT export instead of relying only on `solution.save_data`?

The worked answer is that PyBaMM's export function is excellent for quick export of selected time-only variables, and it should be part of your toolbox. This chapter uses custom export code because we want a canonical table with renamed columns, a sign convention enforced at export time, a metadata sidecar, HDF5 attributes, and a consistent interface for both simulated and experimental data. That is not a criticism of PyBaMM. It is the difference between a convenient tool export and a project-specific data contract.

Close enough means the files load correctly, the current and voltage arrays match between CSV and MAT within floating-point precision, and the metadata in your extended workflow clearly records the sign convention. If PyBaMM changes variable names in a future version, record the version and the replacement names in your log.

10.4 Open-Ended Exercises

10.4.1 Exercise 1: Replace the first-order ECM with a two-RC ECM

Modify `identify_ecm_from_pybamm.m` so that it fits R_0 , R_1 , C_1 , R_2 , and C_2 . Use two polarization states:

$$V(t) = U_{oc}(z) - IR_0 - V_1 - V_2. \quad (6)$$

Hints: fit parameters in log space, initialize the second time constant at least

five times larger than the first, and compare validation RMSE rather than only identification RMSE. Watch for parameter swapping: two RC branches can trade identities unless the data excite both time scales.

10.4.2 Exercise 2: Use a DFN virtual cell instead of SPMe

Change the PyBaMM model in `make_spme_simulation` to `pybamm.lithium_ion.DFN()`. Keep the protocols the same at first. Compare runtime, identification RMSE, and validation RMSE.

Hints: DFN may run more slowly and may expose stronger dynamics that a 1RC ECM cannot match. Do not change the ECM and the virtual cell at the same time. Make one change, rerun, and record the effect.

10.4.3 Exercise 3: Add temperature as an exported variable

Turn on a lumped thermal model in PyBaMM and export Volume-averaged cell temperature [K]. Extend the CSV, HDF5, and MAT files to include `temperature_k`. Do not fit a thermal ECM yet; just prove the bridge carries temperature correctly.

Hints: revisit Lab Chapter 9 for PyBaMM thermal options. Start with moderate currents. MATLAB column names should remain lowercase and unit-explicit.

10.4.4 Exercise 4: Run the whole pipeline without MATLAB Engine

Pretend your Python environment cannot call MATLAB. Run the Python generation script, run MATLAB manually, then run the Python plotting and manifest scripts. Write a short note explaining which steps are automated and which are manual.

Hints: this is not a failure mode. Many research groups use file-only handoffs because MATLAB licenses live on different machines than Python environments.

10.5 Worked Solutions to the Open-Ended Exercises

10.5.1 Solution 1: Two-RC ECM structure

The key change is to add a second state and two more log-parameters. The simulation update becomes:

```
alpha1 = exp(-dt / (r10hm * c1Farad));
alpha2 = exp(-dt / (r20hm * c2Farad));
v1(k) = alpha1 * v1(k - 1) + r10hm * (1.0 - alpha1) * previousCurrent;
v2(k) = alpha2 * v2(k - 1) + r20hm * (1.0 - alpha2) * previousCurrent;
voltage(k) = polyval(ocvCoefficients, soc(k)) - currentA(k) * r00hm - v1(k) -
↳ v2(k);
```

A reasonable initial guess is:

```
initialGuess = log([0.015, 0.006, 500.0, 0.010, 8000.0]);
```

If validation RMSE improves only slightly while parameters become unstable, the 2RC model may not be justified by the data. Standard practice is to report both fit quality and parameter plausibility.

10.5.2 Solution 2: DFN virtual cell

The minimal code change is:

```
model = pybamm.lithium_ion.DFN()
```

Everything else can remain unchanged. You should expect longer runtime. The validation error may increase because DFN contains richer diffusion and electrolyte dynamics. If the 1RC ECM still performs well, that is useful evidence that the chosen protocol does not strongly expose higher-order dynamics. If it performs poorly, try the 2RC exercise before concluding that ECMs are unsuitable.

10.5.3 Solution 3: Temperature export

Use:

```
model = pybamm.lithium_ion.SPM({ "thermal": "lumped" })
```

and add:

```
temperature_k = np.asarray(
    solution["Volume-averaged cell temperature [K]"](time_s),
    dtype=float,
)
```

Then include "temperature_k": temperature_k in the pandas table, HDF5 datasets, and MAT export dictionary. In MATLAB, verify with:

```
data = readtable("pybamm_identification.csv");
plot(data.time_s / 60, data.temperature_k);
xlabel("Time (min)");
ylabel("Temperature (K)");
grid on;
```

A correct plot should show small temperature changes for this moderate protocol. If temperature is exactly constant, you may still be running an isothermal model or exporting the wrong variable.

10.5.4 Solution 4: File-only bridge

The file-only sequence is:

```
python scripts/01_generate_virtual_cell.py
python scripts/02_audit_exports.py
```

then in MATLAB:

```
addpath("SimulationCompanion/chapter10_bridge_workspace/matlab")
identify_ecm_from_pybamm
```

then back in Python:

```
python scripts/04_write_manifest.py
```

To create the final Python plot without rerunning MATLAB Engine, call `make_python_validation_figure()` from `03_run_bridge_pipeline.py` in a small wrapper or temporarily edit `main()`. In a real project, you would split generation, MATLAB execution, and plotting into separate command-line targets so manual and automated paths share the same code.

What Changes for Sodium-Ion?

The bridge workflow becomes more valuable for sodium-ion, not less. Sodium-ion research has fewer mature public datasets, fewer widely validated parameter sets, and more variation in OCV shape across cathode/anode pairings. A clean PyBaMM-to-MATLAB workflow lets you test BMS algorithms on virtual sodium-ion cells before abundant experimental data exist.

Three changes matter most. First, the OCV-SOC relationship can be flatter or shaped differently than the lithium-ion parameter set used here, especially for

hard-carbon anodes with plateau and sloping regions. A flat OCV region makes ECM identification and SOC estimation less observable from voltage, as you saw in Lab Chapter 7. Second, parameter availability is thinner. You may need to build a provisional sodium-ion parameter set from literature values, then label it as provisional rather than validated. Third, validation strategy must be more modest. If you validate the bridge on lithium-ion data and then apply it to sodium-ion simulations, say exactly that. Do not imply that lithium-ion validation proves sodium-ion accuracy.

To adapt this chapter's workflow for sodium-ion, replace `Chen2020` with a sodium-ion parameter set if one is available in your PyBaMM installation or in a documented external repository. If not, start with a lithium-ion parameter set as a software-method stand-in and clearly separate "workflow validation" from "chemistry validation." Then modify the OCV fitting section to avoid high-order polynomial artifacts. For hard-carbon-like OCV shapes, piecewise monotonic interpolation is often safer than a global polynomial.

The most publishable sodium-ion use of this chapter is a virtual benchmarking study: define several plausible sodium-ion OCV curves and resistance levels, generate virtual current-voltage data in PyBaMM or a custom electrochemical model, identify ECMs in MATLAB, and quantify how OCV flatness affects validation error and SOC observability. That study would connect directly to Textbook Chapter 10 and the sodium-ion chapter.

Chapter Summary and Skill Checklist

- You created a PyBaMM virtual-cell workflow with separate identification and validation protocols.
- You exported simulation data to CSV, HDF5, MAT, and JSON metadata files.
- You enforced a clear current sign convention: `current_a > 0` means discharge.
- You audited exported files from both Python and MATLAB before fitting models.
- You identified a first-order Thevenin ECM in MATLAB using PyBaMM-generated data.
- You validated the ECM on a held-out PyBaMM protocol rather than reporting only training error.

- You built a Python master script that can call MATLAB through MATLAB Engine when available.
- You wrote a run manifest with package versions, file hashes, and the workflow contract.

Commands, functions, and patterns that should now feel familiar:

- `pybamm.Experiment(...)`
- `pybamm.Simulation(...).solve()`
- `solution["Terminal voltage [V]"](solution.t)`
- `pandas.DataFrame.to_csv(...)`
- `h5py.File(...)`
- `scipy.io.savemat(...)` and `scipy.io.loadmat(...)`
- MATLAB `readtable`, `writetable`, `load`, `polyfit`, `polyval`, and `fminsearch`
- MATLAB Engine `matlab.engine.start_matlab()`
- File hashes with `hashlib.sha256`
- Metadata sidecars with JSON

You should now be able to:

- Generate synthetic battery data in PyBaMM for a chosen protocol.
- Export a clean time-series table with units and sign conventions.
- Read Python-generated CSV and MAT files in MATLAB.
- Fit a simple ECM in MATLAB from exported current-voltage data.
- Validate the ECM on a held-out protocol and report RMSE in millivolts.
- Return MATLAB predictions to Python for publication-style plotting.
- Explain why training error and validation error are different.
- Design a file contract that works for both simulated and experimental battery data.
- State what must change before applying the workflow to sodium-ion cells.

If you cannot check every box, revisit the corresponding walkthrough before moving on. Chapter 11 assumes you can design a canonical table format and defend your unit conventions.

Deliverable

Your deliverable is a complete virtual-cell workflow:

- A PyBaMM script that runs an identification protocol and a held-out validation protocol.
- Exported CSV, HDF5, MAT, and JSON metadata files.
- A MATLAB script that loads the exported identification data and identifies a Thevenin ECM.
- A validation result comparing the MATLAB ECM against the held-out PyBaMM simulation.
- A master script or clearly documented command sequence that reruns the pipeline.
- A short written interpretation of the validation RMSE, the major error features, and whether a 1RC ECM is adequate for the protocol.

A strong partial solution is exactly the set of scripts written in this chapter, with one meaningful extension: either a 2RC ECM, a DFN virtual cell, or temperature export. In your write-up, include the current sign convention, software versions, and a plot with current, PyBaMM voltage, ECM voltage, and voltage error. Do not submit only the final RMSE. The error shape is part of the result.

Further Practice and Reading

PyBaMM documentation: bookmark the official PyBaMM installation page at https://docs.pybamm.org/en/stable/source/user_guide/installation/index.html and the output-management tutorial at https://docs.pybamm.org/en/stable/source/examples/notebooks/getting_started/output_management.html. The output tutorial is especially relevant because it demonstrates `save_data` exports to CSV and MATLAB formats and explains the limitation to time-only variables.

MathWorks documentation: bookmark the MATLAB Engine API for Python documentation at <https://www.mathworks.com/help/matlab/matlab-engine-for-python.html> and the engine installation page at https://www.mathworks.com/help/matlab/matlab_external/install-the-matlab-engine-api-for-python.html. You will need both directions eventually, even if this chapter emphasizes Python calling MATLAB.

Battery modeling papers: revisit Marquis et al. (2019), “An asymptotic derivation of a single particle model with electrolyte,” and Sulzer et al. (2021), “Python Battery Mathematical Modelling (PyBaMM).” They help you explain why SPMe is a defensible virtual-cell generator and what PyBaMM contributes beyond ordinary numerical integration.

ECM practice: return to Plett’s BMS material on equivalent-circuit identification and compare its pulse-based parameter extraction with the optimization-based workflow here. The difference between physically guided identification and black-box optimization is a recurring research judgment.

Community resources: the PyBaMM GitHub repository at <https://github.com/pybamm-team/PyBaMM> and PyBaMM community channels are useful when variable names or solver behavior change. MATLAB Central and File Exchange are useful for checking idiomatic MATLAB data-import and optimization patterns, but always reduce borrowed code to a small auditable script before using it in a paper workflow.

Next chapter: Lab Chapter 11, “Public Battery Datasets in Depth.”

Public Battery Datasets in Depth

This chapter is where battery research becomes less tidy and more real. Up to now, most of our workflows have used either simulations we controlled or small datasets chosen for one task: an ECM fit, a Kalman-filter test, an aging curve, or a thermal parameter estimate. Publishable research almost never stays that clean. You will download public data from several laboratories, discover that files are named inconsistently, learn that current signs are not universal, find that some timestamps reset every cycle while others never reset, and build a loader that turns this mess into a reproducible research asset.

Keep Textbook Chapters 2, 6, 7, 8, 10, and 13 open as you work. This chapter operationalizes the measurement assumptions behind the electrochemical data you learned in Textbook Chapter 2, the degradation observables from Textbook Chapter 7, the cell-model validation logic from Textbook Chapter 8, the BMS signal conventions from Textbook Chapter 10, and the sodium-ion research constraints from Textbook Chapter 13. The theory textbook taught you what voltage, current, capacity, SOC, SOH, temperature, and impedance mean. This chapter teaches you how those quantities arrive on your laptop, how they get damaged by file formats and test conventions, and how to make them usable without laundering away the uncertainty.

The skill target is not “download a dataset.” Anyone can do that. The target is

dataset competence: finding credible sources, recording licenses and citations, parsing raw files into consistent units, detecting cycles, handling rest periods, validating sign conventions, and writing a small Python package that lets later chapters treat public data as a stable input. This is the difference between a notebook that worked once and a research workflow that survives peer review.

For sodium-ion work, this skill matters even more than it does for lithium-ion. Public lithium-ion datasets are mature enough that you can usually validate a workflow on NASA, CALCE, Oxford, Sandia, or Severson-style fast-charge data. Public sodium-ion datasets are improving quickly, but they remain sparse, uneven, and often chemistry-specific. A sodium-ion researcher must therefore be bilingual: fluent in large lithium-ion benchmark datasets for method validation, and careful with smaller sodium-ion datasets for chemistry-specific interpretation. When the data do not exist, you must be able to say exactly what was validated, what was adapted, and what remains an assumption.

By the end of this chapter, you will have a reusable Python module named `batterydata` with loaders for at least four dataset families: CALCE-style Excel/text files, NASA PCoE MATLAB files, Oxford MATLAB files, and Severson/MATR batch files. You will also add a generic CSV loader for newer Mendeley and Zenodo sodium-ion releases, because public sodium-ion datasets often arrive as ordinary spreadsheets rather than famous benchmark formats. The deliverable is a small, properly structured package that returns the same column names and units no matter where the raw data came from.

We will move in six stages. First, we map the public dataset landscape and install the tools needed for mixed file formats. Second, we define a tidy battery-data schema. Third, we write and test a generic loader on a tiny synthetic dataset so the software structure is clear before real files arrive. Fourth, we add dataset-specific parsers for CALCE, NASA, Oxford, and Severson/MATR. Fifth, we clean, cycle-detect, and summarize public time series in a uniform way. Sixth, we reproduce a simplified version of the key voltage-curve feature idea from Severson et al.'s fast-charge cycle-life paper, using the public MATR dataset when available and a small schema-compatible fixture when the full download is not present.

11.1 Prerequisites Check

- Required software: Python 3.11, numpy==1.26.4, pandas==2.2.2, scipy==1.13.1, matplotlib==3.9.0, h5py==3.11.0, requests==2.32.3, openpyxl==3.1.5, tqdm==4.66.5, and pytest==8.3.2
- Install command: `python -m pip install numpy==1.26.4 pandas==2.2.2 scipy==1.13.1 matplotlib==3.9.0 h5py==3.11.0 requests==2.32.3 openpyxl==3.1.5 tqdm==4.66.5 pytest==8.3.2`
- Required textbook chapters: Textbook Chapters 2, 7, 8, 10, and 13
- Required prior lab chapters: Lab Chapters 1 and 2 are essential; Lab Chapters 4, 6, 7, and 8 are strongly recommended
- Public sources used in this chapter: CALCE Battery Data Archive, NASA PCoE Li-ion Battery Aging Datasets, Oxford Battery Degradation Dataset 1, Severson/Attia/MIT-Stanford-Toyota MATR dataset, and selected Mendeley/Zenodo sodium-ion datasets
- Estimated time: 16 to 24 hours if you download and inspect the large datasets; 8 to 10 hours if you use only the small fixtures and read the large-download instructions

If pandas group-by operations feel shaky, revisit Lab Chapter 2 before starting Walkthrough 3. If capacity, Coulomb counting, or current sign conventions feel shaky, revisit Lab Chapter 6. If SOH versus cycle number feels fuzzy, revisit Lab Chapter 8. Dataset work rewards patience more than cleverness. A slow, explicit parser with checks is better than a clever one-liner that silently flips charge and discharge.

11.2 Environment Setup

We will create a dedicated environment because dataset chapters tend to accumulate file-format dependencies. You do not need PyBaMM in this chapter. We are building the data foundation that PyBaMM, MATLAB, and your own estimation scripts will consume later.

11.2.1 Create the Python environment

From the repository root, run:

```
cd /home/avirup/SodiumIonBatteryResearch
python3.11 -m venv .venv-chapter11
source .venv-chapter11/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==1.26.4 pandas==2.2.2 scipy==1.13.1
↳ matplotlib==3.9.0 h5py==3.11.0 requests==2.32.3 openpyxl==3.1.5
↳ tqdm==4.66.5 pytest==8.3.2
```

On Windows PowerShell, use:

```
cd C:\path\to\SodiumIonBatteryResearch
py -3.11 -m venv .venv-chapter11
.\.venv-chapter11\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install numpy==1.26.4 pandas==2.2.2 scipy==1.13.1
↳ matplotlib==3.9.0 h5py==3.11.0 requests==2.32.3 openpyxl==3.1.5
↳ tqdm==4.66.5 pytest==8.3.2
```

If `python3.11` is not found, run `python --version`. Any Python 3.10 or newer should work for this chapter, but record the exact version in your research log. Public dataset code often becomes part of a supplementary repository, and future readers need to know what you used.

11.2.2 Create the chapter workspace

Run:

```
mkdir -p SimulationCompanion/chapter11_dataset_workspace
cd SimulationCompanion/chapter11_dataset_workspace
mkdir -p batterydata data/raw data/processed figures notebooks tests
touch batterydata/__init__.py
```

The chapter workspace has three jobs. `data/raw` stores files exactly as downloaded. `data/processed` stores normalized CSV or Parquet outputs produced by your loaders. `batterydata` stores the reusable package. Resist the temptation to edit raw files by hand. If you fix a column name, skip a header row, or change a sign convention, do it in code so the decision is inspectable.

11.2.3 Verify the install

Paste this into a fresh notebook or run it as `python verify_chapter11.py`:

```

import h5py
import matplotlib
import numpy as np
import openpyxl
import pandas as pd
import scipy

print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
print("SciPy:", scipy.__version__)
print("Matplotlib:", matplotlib.__version__)
print("h5py:", h5py.__version__)
print("openpyxl:", openpyxl.__version__)

demo = pd.DataFrame(
    {
        "time_s": [0.0, 1.0, 2.0],
        "current_a": [0.0, -1.0, -1.0],
        "voltage_v": [3.20, 3.25, 3.30],
    }
)
print(demo)

```

Expected output:

```

NumPy: 1.26.4
pandas: 2.2.2
SciPy: 1.13.1
Matplotlib: 3.9.0
h5py: 3.11.0
openpyxl: 3.1.5

```

	time_s	current_a	voltage_v
0	0.0	0.0	3.20
1	1.0	-1.0	3.25
2	2.0	-1.0	3.30

The current is negative during charge in this chapter's canonical schema. That is a deliberate choice, not a universal truth. Many cyclers and datasets use positive current for charge. We will normalize to the BMS convention used in earlier lab chapters: positive current means discharge from the cell into the load; negative current means charge into the cell.

11.2.4 Common setup failures and fixes

`ImportError: Missing optional dependency 'openpyxl'` means pandas is trying to read Excel files without the Excel engine installed. Rerun the install command and restart the notebook kernel.

`OSError: Unable to open file` from h5py usually means one of two things: the file path is wrong, or the `.mat` file is not MATLAB v7.3/HDF5. We will provide both `scipy.io.loadmat` and h5py patterns because public MATLAB files come in both styles.

`Permission denied` when writing under `data/raw` usually means your notebook is running from a different directory or the folder was created by another user. Print `Path.cwd()` inside Python and create folders from that working directory.

Large downloads may fail halfway. For this chapter, manual download is acceptable. Place downloaded files under `SimulationCompanion/chapter11_dataset_workspace/data/raw` and let the parser find them. Do not make the parser depend on a fragile one-time download URL unless the dataset provider publishes a stable API.

11.3 Conceptual Bridge: From Electrochemical Observables to Dataset Objects

In the theory textbook, a cell experiment is cleanly described by functions of time: current $I(t)$, terminal voltage $V(t)$, temperature $T(t)$, capacity $Q(t)$, and perhaps impedance $Z(\omega, t)$. In real public data, those functions are chopped into files, cycles, steps, sheets, MATLAB structs, HDF5 groups, and filenames. The first job of a dataset workflow is to rebuild the experimental trace without inventing information.

The minimum object we need is a tidy time-series table. In this chapter, every loader will return a `pandas.DataFrame` with a standard set of columns:

Column	Unit	Required	Meaning
dataset	text	yes	Source family, such as NASA or CALCE

Column	Unit	Required	Meaning
cell_id	text	yes	Cell identifier inside the source
cycle_index	integer	no	Cycle number if known or detected
step_index	integer	no	Step number within a cycle if known
time_s	s	yes	Elapsed time within the returned record
current_a	A	yes	Positive for discharge, negative for charge
voltage_v	V	yes	Terminal voltage
temperature_c	deg C	no	Cell, chamber, or surface temperature
capacity_ah	Ah	no	Reported or integrated capacity
mode	text	no	charge, discharge, rest, impedance, or unknown
source_file	text	yes	Raw file path or name

This schema is intentionally modest. We are not trying to store every metadata field from every dataset. We are defining the small common denominator needed for modeling, estimation, and reproducibility. If a dataset has impedance spectra, electrode-specific metadata, chamber humidity, diagnostic-cycle labels, or protocol

IDs, keep those in separate tables. Do not force all public data into one bloated master table. A stable narrow table plus source-specific metadata is easier to audit.

The most important convention is current sign. In electrochemistry papers, the sign convention often follows the instrument or the author. In BMS and power electronics, positive current is often discharge. Some Arbin exports mark charge current as positive, some public processing scripts flip it, and some datasets provide both current and capacity columns whose signs do not agree. This is the data version of the sign convention problem you met in Textbook Chapter 2. We will normalize to positive discharge and negative charge, then write that convention into the package docstring.

Capacity is the second trap. A cycler may report charge capacity and discharge capacity as positive increasing quantities inside each step. A modeling workflow may want signed Coulomb count:

$$q(t) = \frac{1}{3600} \int_0^t I(\tau) d\tau. \quad (1)$$

Under our convention, Equation (1) increases during discharge and decreases during charge. That is useful for physics and BMS work, but it is not the same as the reported discharge-capacity column in many datasets. A clean loader should preserve the source capacity if present and, when needed, compute a clearly named derived column such as `throughput_ah` or `signed_capacity_ah`.

Cycle detection is the third trap. A “cycle” may mean one charge-discharge pair, one discharge-charge pair, one test record in a MATLAB struct, one diagnostic every hundred drive cycles, or one row in a capacity-summary table. Dataset loaders should prefer provider cycle labels when they exist. If they do not exist, your detection algorithm should be explicit and conservative. In this chapter, we will detect transitions between charge, discharge, and rest using current thresholds, then assign cycle numbers at discharge starts. This is standard practice for exploratory data cleaning, but it is not a substitute for reading the dataset documentation.

The final bridge is provenance. A model can tolerate small measurement noise; a paper cannot tolerate unclear provenance. Every processed table should remember where it came from, which loader version produced it, what sign convention was applied, and which rows were dropped. You do not need a heavyweight database. A `source_file` column, a `README.md`, and a few validation plots already put you ahead of many public notebooks.

Guided Walkthrough 1

Build the Package Skeleton and Canonical Schema

Learning objective: Create a reusable `batterydata` package and implement schema validation before touching real public data.

We begin with structure because dataset code grows quickly. If you write one notebook cell per dataset, you will have four nearly identical parsers and no confidence that the outputs agree. Instead, we will write a tiny package with a schema module, utility functions, and tests. The first loader will use a synthetic CSV so that the schema is easy to see.

Create `batterydata/schema.py`:

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Iterable

import numpy as np
import pandas as pd
```

```
CANONICAL_COLUMNS = [
    "dataset",
    "cell_id",
    "cycle_index",
    "step_index",
    "time_s",
    "current_a",
    "voltage_v",
    "temperature_c",
    "capacity_ah",
    "mode",
    "source_file",
]
```

```
REQUIRED_COLUMNS = [
    "dataset",
    "cell_id",
    "time_s",
    "current_a",
    "voltage_v",
    "source_file",
```



```

]

@dataclass(frozen=True)
class ValidationResult:
    """Container for schema validation messages."""

    is_valid: bool
    messages: tuple[str, ...]

def empty_canonical_frame() -> pd.DataFrame:
    """Return an empty DataFrame with the canonical battery-data columns."""
    return pd.DataFrame(columns=CANONICAL_COLUMNS)

def add_missing_optional_columns(frame: pd.DataFrame) -> pd.DataFrame:
    """Add missing canonical columns and return columns in canonical
    ↪ order."""
    =
    for      in
        if      not in
            =
    return

def                                     ->
    """Validate the columns, required values, and basic units of a frame."""
    list str =

    for      in
        if      not in
            f"Missing required column: {      }"

    if
        return                                     False tuple

    for      in
        if                                     any
            messages.append(f"Required column contains null values:
    ↪ {column}")

    = "time_s" "current_a" "voltage_v"

    for      in
        if not

```

```

        messages.append(f"Column must be numeric: {column}")

    if pd.api.types.is_numeric_dtype(frame["time_s"]):
        time_values = frame["time_s"].to_numpy(dtype=float)
        if np.any(np.diff(time_values) < -1e-9):
            messages.append("time_s is not monotonically nondecreasing")

    if pd.api.types.is_numeric_dtype(frame["voltage_v"]):
        voltage_values = frame["voltage_v"].to_numpy(dtype=float)
        finite_voltage = voltage_values[np.isfinite(voltage_values)]
        if finite_voltage.size and (
            finite_voltage.min() < 0.0 or finite_voltage.max() > 6.0
        ):
            messages.append("voltage_v has values outside the expected 0-6 V
↳ range")

    return ValidationResult(len(messages) == 0, tuple(messages))

def require_columns(frame: pd.DataFrame, columns: Iterable[str]) -> None:
    """Raise a helpful error if a raw input frame is missing expected
↳ columns."""
    missing = [column for column in columns if column not in frame.columns]
    if missing:
        available = ", ".join(str(column) for column in frame.columns)
        missing_text = ", ".join(missing)
        raise ValueError(
            f"Missing required raw columns: {missing_text}. "
            f"Available columns are: {available}"
        )

```

Create batterydata/generic.py:

```

from __future__ import annotations

from pathlib import Path

import pandas as pd

from .schema import add_missing_optional_columns, validate_canonical_frame

def load_generic_csv(
    path: str | Path,
    dataset: str,

```

```

cell_id: str,
column_map: dict[str, str],
current_sign: str = "positive_discharge",
) -> pd.DataFrame:
    """Load a CSV file and map its columns into the canonical schema.

    Parameters
    -----
    path:
        CSV file path.
    dataset:
        Short dataset label to store in the output.
    cell_id:
        Cell identifier to store in the output.
    column_map:
        Mapping from raw column names to canonical names.
    current_sign:
        Either ``positive_discharge`` or ``positive_charge``.
    """
    path = Path(path)
    raw = pd.read_csv(path)
    renamed = raw.rename(columns=column_map)

    output = pd.DataFrame()
    for canonical_name in column_map.values():
        output[canonical_name] = renamed[canonical_name]

    output["dataset"] = dataset
    output["cell_id"] = cell_id
    output["source_file"] = str(path)

    if current_sign == "positive_charge":
        output["current_a"] = -1.0 * output["current_a"].astype(float)
    elif current_sign != "positive_discharge":
        raise ValueError("current_sign must be positive_discharge or
        ↪ positive_charge")

    output = add_missing_optional_columns(output)
    result = validate_canonical_frame(output)
    if not result.is_valid:
        message = "\n".join(result.messages)
        raise ValueError(f"Canonical validation failed for
        ↪ {path}: \n{message}")

    return output

```

Update batterydata/__init__.py:

```
from .generic import load_generic_csv
from .schema import CANONICAL_COLUMNS, validate_canonical_frame

__all__ = [
    "CANONICAL_COLUMNS",
    "load_generic_csv",
    "validate_canonical_frame",
]
```

Now create a tiny fixture file by running this in Python:

```
from pathlib import Path

import pandas as pd

fixture_path = Path("data/raw/tiny_sodium_like_cycle.csv")
fixture_path.parent.mkdir(parents=True, exist_ok=True)

fixture = pd.DataFrame(
    {
        "Time_sec": [0, 10, 20, 30, 40, 50, 60],
        "Current_A": [0.0, 1.0, 1.0, 0.0, -0.8, -0.8, 0.0],
        "Voltage_V": [3.10, 3.03, 2.98, 3.05, 3.15, 3.22, 3.20],
        "Temp_C": [25.0, 25.1, 25.1, 25.1, 25.2, 25.2, 25.2],
    }
)
fixture.to_csv(fixture_path, index=False)
print(fixture_path.resolve())
```

Load it:

```
from batterydata import load_generic_csv, validate_canonical_frame

frame = load_generic_csv(
    "data/raw/tiny_sodium_like_cycle.csv",
    dataset="teaching_fixture",
    cell_id="Na_fixture_001",
    column_map={
        "Time_sec": "time_s",
        "Current_A": "current_a",
        "Voltage_V": "voltage_v",
        "Temp_C": "temperature_c",
    },
    current_sign="positive_discharge",
```

```
)  
  
print(frame)  
print(validate_canonical_frame(frame))
```

Expected output is a seven-row table with the canonical columns in a consistent order. `current_a` should be positive during the early discharge-like segment and negative during the later charge-like segment. The validation result should report `is_valid=True` and an empty message tuple.

The key line in `load_generic_csv` is the `column_map`. We do not ask every raw dataset to use our names. We ask every loader to explain how raw names map into our names. The `current_sign` argument makes the sign convention explicit. If a source uses positive current for charge, the loader flips it once at the boundary.

11.3.1 What Could Go Wrong

If Python cannot import `batterydata`, your notebook is probably not running from `SimulationCompanion/chapter11_dataset_workspace`. Print `Path.cwd()` and either change directories or install the package in editable mode later with `python -m pip install -e ..`

If validation complains that `time_s` is not monotonic, check whether the source file resets time at each step. This is common. You can either return one step at a time or reconstruct absolute time before validation.

If validation complains about voltage outside 0-6 V, inspect units. Some files report millivolts. Divide by 1000 before returning `voltage_v`.

11.3.2 Reflection

This exercise taught the core habit of the chapter: normalize at the boundary and validate immediately. The schema is small enough to remember but strict enough to catch the most expensive mistakes. We will now apply the same pattern to public datasets that are much less polite than our fixture.

Guided Walkthrough 2

Parse CALCE Excel and Text Files

Learning objective: Load CALCE cycling files into the canonical schema while preserving source-specific quirks.

The CALCE Battery Data Archive is a practical starting point because it contains real cell cycling files from a battery research group, not a machine-learning benchmark preprocessed into convenient arrays. The archive includes LiCoO₂ prismatic CS2 cells. The CALCE page states that CS2 cells have a nominal capacity around 1100 mAh and that most were tested with Arbin equipment, while some files are text files from CADEX equipment. Publication use should cite the CALCE database and relevant CALCE/contributor papers.

Download location: <https://calce.umd.edu/battery-data> or the older archive page <https://web.calce.umd.edu/batteries/data/>. File sizes vary by cell and test. Expect Excel files or text files, often grouped by cell. The archive terms are not the same as a permissive software license; read the page and cite it properly.

Create `batterydata/calce.py`:

```
from          import

from          import

import        as
import        as

from          import

=

"time_s"      "Test_Time(s)"  "Test Time (s)"  "Time(s)"  "time_s"  "Time"
"current_a"   "Current(A)"    "Current (A)"   "Current"  "I(A)"
              "current_a"
"voltage_v": ["Voltage(V)", "Voltage (V)", "Voltage", "Ecell/V",
              ↪ "voltage_v"],
"temperature_c": ["Temperature(C)", "Temperature (C)", "Temp(C)",
                  ↪ "temperature_c"],
"cycle_index": ["Cycle_Index", "Cycle Index", "Cycle", "cycle_index"],
"step_index": ["Step_Index", "Step Index", "Step", "step_index"],
}

def _find_column(raw: pd.DataFrame, aliases: list[str]) -> str | None:
    normalized_lookup = {str(column).strip(): column for column in
    ↪ raw.columns}
```

```

    for alias in aliases:
        if alias in normalized_lookup:
            return normalized_lookup[alias]
    lower_lookup = {str(column).strip().lower(): column for column in
→ raw.columns}
    for alias in aliases:
        key = alias.strip().lower()
        if key in lower_lookup:
            return lower_lookup[key]
    return None

def _read_calce_file(path: Path) -> pd.DataFrame:
    if path.suffix.lower() in {".xlsx", ".xls"}:
        return pd.read_excel(path, engine="openpyxl")
    if path.suffix.lower() in {".csv"}:
        return pd.read_csv(path)
    return pd.read_csv(path, sep=None, engine="python")

def load_calce_file(
    path: str | Path,
    cell_id: str,
    current_sign: str = "positive_charge",
) -> pd.DataFrame:
    """Load one CALCE-style file into the canonical schema.

    CALCE files are not perfectly uniform across cells and instruments.
    This loader searches for common column aliases and returns a validated
    canonical table.
    """
    path = Path(path)
    raw = _read_calce_file(path)
    raw = raw.dropna(axis=0, how="all").dropna(axis=1, how="all")

    mapped: dict[str, pd.Series] = {}
    for canonical_name, aliases in CALCE_COLUMN_ALIASES.items():
        raw_column = _find_column(raw, aliases)
        if raw_column is not None:
            mapped[canonical_name] = raw[raw_column]

    required = ["time_s", "current_a", "voltage_v"]
    missing = [column for column in required if column not in mapped]
    if missing:
        raise ValueError(f"{path} is missing required CALCE columns:
→ {missing}")

```

```

output = pd.DataFrame(mapped)
output["dataset"] = "CALCE"
output["cell_id"] = cell_id
output["source_file"] = str(path)

output["time_s"] = pd.to_numeric(output["time_s"], errors="coerce")
output["current_a"] = pd.to_numeric(output["current_a"], errors="coerce")
output["voltage_v"] = pd.to_numeric(output["voltage_v"], errors="coerce")

if "temperature_c" in output.columns:
    output["temperature_c"] = pd.to_numeric(output["temperature_c"],
↪ errors="coerce")

if current_sign == "positive_charge":
    output["current_a"] = -output["current_a"]
elif current_sign != "positive_discharge":
    raise ValueError("current_sign must be positive_charge or
↪ positive_discharge")

output = output.dropna(subset=["time_s", "current_a", "voltage_v"])
output = output.sort_values("time_s").reset_index(drop=True)
output = add_missing_optional_columns(output)

result = validate_canonical_frame(output)
if not result.is_valid:
    raise ValueError("\n".join(result.messages))

return output

```

Add it to batterydata/__init__.py:

```

from .calce import load_calce_file
from .generic import load_generic_csv
from .schema import CANONICAL_COLUMNS, validate_canonical_frame

__all__ = [
    "CANONICAL_COLUMNS",
    "load_calce_file",
    "load_generic_csv",
    "validate_canonical_frame",
]

```

Try the loader on a CALCE file you downloaded:


```

from pathlib import Path

from batterydata import load_calce_file

calce_path = Path("data/raw/CALCE/CS2_35_example.xlsx")

if calce_path.exists():
    calce = load_calce_file(calce_path, cell_id="CS2_35")
    print(calce.head())
    print(calce[["time_s", "current_a", "voltage_v",
                  "temperature_c"]].describe())
else:
    print("Place a CALCE Excel or text file at:", calce_path)

```

Now plot a quick sanity check:

```

import matplotlib.pyplot as plt

if calce_path.exists():
    fig, axes = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
    axes[0].plot(calce["time_s"] / 3600, calce["voltage_v"], linewidth=1.0)
    axes[0].set_ylabel("Voltage (V)")
    axes[0].grid(True)

    axes[1].plot(calce["time_s"] / 3600, calce["current_a"], linewidth=1.0)
    axes[1].axhline(0.0, color="black", linewidth=0.8)
    axes[1].set_xlabel("Time (h)")
    axes[1].set_ylabel("Current (A)")
    axes[1].grid(True)

    fig.suptitle("CALCE canonical loader sanity check")
    fig.tight_layout()
    plt.show()

```

A correct plot should show voltage rising during charge segments and falling during discharge segments. Because we normalized to positive-discharge current, the voltage should generally fall when `current_a` is positive and rise when `current_a` is negative. If those relationships are reversed, your `current_sign` argument is wrong for that file.

11.3.3 What Could Go Wrong

If Excel reading fails, the file may be an old `.xls` file that needs a different engine. Install `xlrd` with `python -m pip install xlrd==2.0.1`, or open the file once in LibreOffice and export to `.xlsx` while preserving the raw copy.

If the parser cannot find `Current (A)` or `Voltage (V)`, inspect `raw.columns`. CALCE files have changed over time and mirrored copies sometimes rename headers. Add the observed header to `CALCE_COLUMN_ALIASES` rather than renaming the raw file.

If current looks like milliamps, check magnitudes. A CS2 1C current should be around `1.1 A`, not `1100 A`. Divide by `1000` in a source-specific branch and document it.

11.3.4 Reflection

CALCE teaches the first public-data lesson: even a respected archive is not a single format. A good loader searches for known aliases, validates units, and keeps the raw source visible. We did not hide the mess; we contained it.

Guided Walkthrough 3

Parse NASA PCoE MATLAB Battery Files

Learning objective: Extract charge, discharge, and impedance records from NASA-style MATLAB structs.

The NASA PCoE Li-ion Battery Aging Datasets are among the most-used battery-health public datasets. NASA's Open Data page describes Li-ion cells run through charge, discharge, and EIS operations at different temperatures, with repeated cycling causing accelerated aging. The page also notes that some discharge thresholds were below the OEM-recommended `2.7 V`, which matters when you interpret capacity fade. The Open Data portal lists the license as not specified, so treat NASA attribution and dataset citation as mandatory.

Download location:

<https://data.nasa.gov/dataset/li-ion-battery-aging-datasets> and the NASA PCoE repository pages linked from it. The common archive contains MATLAB `.mat` files named by battery, such as `B0005.mat`. File sizes are modest compared with Oxford and MATR.

```

from import
from import
import as
import as
from import
from import

def ->
    if hasattr "_fieldnames"
        return getattr for in

    if isinstance(obj, np.ndarray) and obj.dtype == object:
        if obj.size == 1:
            return _mat_struct_to_dict(obj.item())
        return [_mat_struct_to_dict(item) for item in obj.ravel()]
    return obj

def _as_1d_float_array(value: Any) -> np.ndarray:
    array = np.asarray(value, dtype=float).squeeze()
    if array.ndim == 0:
        return array.reshape(1)
    return array

def load_nasa_mat(path: str | Path, cell_id: str | None = None) ->
↳ pd.DataFrame:
    """Load a NASA PCoE battery .mat file into the canonical schema."""
    path = Path(path)
    cell_id = cell_id or path.stem
    mat = loadmat(path, squeeze_me=True, struct_as_record=False)

    if cell_id not in mat:
        candidate_keys = [key for key in mat if not key.startswith("__")]
        if len(candidate_keys) != 1:
            raise ValueError(f"Could not identify NASA cell key in {path}:
↳ {candidate_keys}")
        root_key = candidate_keys[0]
    else:

```

```

root_key = cell_id

root = _mat_struct_to_dict(mat[root_key])
cycles = root["cycle"]
if isinstance(cycles, dict):
    cycles = [cycles]

frames: list[pd.DataFrame] = []
for cycle_number, cycle in enumerate(cycles, start=1):
    operation_type = str(cycle.get("type", "unknown")).lower()
    if operation_type not in {"charge", "discharge"}:
        continue

    data = cycle["data"]
    time_s = _as_ld_float_array(data["Time"])
    voltage_v = _as_ld_float_array(data["Voltage_measured"])
    current_raw = _as_ld_float_array(data["Current_measured"])

    if "Temperature_measured" in data:
        temperature_c = _as_ld_float_array(data["Temperature_measured"])
    else:
        temperature_c = np.full_like(time_s, np.nan)

    length = min(len(time_s), len(voltage_v), len(current_raw),
↳ len(temperature_c))
    frame = pd.DataFrame(
        {
            "dataset": "NASA_PCoE",
            "cell_id": cell_id,
            "cycle_index": cycle_number,
            "step_index": 1,
            "time_s": time_s[:length],
            "current_a": current_raw[:length],
            "voltage_v": voltage_v[:length],
            "temperature_c": temperature_c[:length],
            "mode": operation_type,
            "source_file": str(path),
        }
    )

    if operation_type == "charge":
        frame["current_a"] = -frame["current_a"].abs()
    elif operation_type == "discharge":
        frame["current_a"] = frame["current_a"].abs()

```

```

        if "Capacity" in data and operation_type == "discharge":
            capacity_value = float(np.asarray(data["Capacity"]).squeeze())
            frame["capacity_ah"] = capacity_value

        frames.append(frame)

    if not frames:
        raise ValueError(f"No charge or discharge records found in {path}")

    output = pd.concat(frames, ignore_index=True)
    output = add_missing_optional_columns(output)
    result = validate_canonical_frame(
        output.sort_values(["cycle_index", "time_s"]).reset_index(drop=True)
    )
    if not result.is_valid:
        raise ValueError("\n".join(result.messages))

    return output

```

Update batterydata/__init__.py:

```

from .calce import load_calce_file
from .generic import load_generic_csv
from .nasa import load_nasa_mat
from .schema import CANONICAL_COLUMNS, validate_canonical_frame

__all__ = [
    "CANONICAL_COLUMNS",
    "load_calce_file",
    "load_generic_csv",
    "load_nasa_mat",
    "validate_canonical_frame",
]

```

Run:

```

from pathlib import Path

from batterydata import load_nasa_mat

nasa_path = Path("data/raw/NASA/B0005.mat")

if nasa_path.exists():
    nasa = load_nasa_mat(nasa_path)
    print(nasa.head())

```

```

print(nasa.groupby("mode")["cycle_index"].nunique())
capacities = (
    nasa.dropna(subset=["capacity_ah"])
    .groupby("cycle_index", as_index=False)["capacity_ah"]
    .first()
)
print(capacities.head())
else:
    print("Place B0005.mat at:", nasa_path)

```

Plot capacity fade:

```

import matplotlib.pyplot as plt

if nasa_path.exists():
    capacities = (
        nasa.dropna(subset=["capacity_ah"])
        .groupby("cycle_index", as_index=False)["capacity_ah"]
        .first()
    )

    fig, ax = plt.subplots(figsize=(8, 4.5))
    ax.plot(capacities["cycle_index"], capacities["capacity_ah"], marker="o",
        ↪ markersize=3)
    ax.set_xlabel("NASA record index")
    ax.set_ylabel("Discharge capacity (Ah)")
    ax.set_title("NASA PCoE B0005 capacity fade")
    ax.grid(True)
    fig.tight_layout()
    plt.show()

```

The correct plot should show capacity declining from near 2 Ah toward the end-of-life region. Do not expect a perfectly smooth curve. Real capacity estimates move with temperature, cutoff, rest, and measurement noise. The x-axis is record index, not necessarily a pure cycle count, because the NASA MATLAB struct includes different operation records. That distinction matters in a paper.

11.3.5 What Could Go Wrong

If `loadmat` fails with “Unknown mat file type”, the file may be MATLAB v7.3. Use `h5py.File(path)` to inspect its groups, then write a separate HDF5 reader. Do not force an HDF5 file through `scipy.io.loadmat`.

If capacity appears on charge records, inspect the NASA struct. The common

PCoE files store Capacity mainly for discharge records. For SOH work, extract capacity from discharge records unless the documentation says otherwise.

If time resets every record, that is normal. Our validation checks monotonicity within the concatenated frame can become too strict for multi-cycle data. In production, validate per (`cell_id`, `cycle_index`, `mode`) group or reconstruct an absolute experiment clock.

11.3.6 Reflection

NASA teaches the second public-data lesson: a MATLAB file can contain a hierarchy, not just a table. The parser’s job is to flatten the hierarchy without losing the operation type and cycle context. The moment we extract capacity fade, we also learn to be honest about what the x-axis means.

Guided Walkthrough 4

Build Cleaning and Cycle-Detection Utilities

Learning objective: Convert canonical time-series data into analysis-ready segments with mode labels, cycle indices, and integrated throughput.

Some datasets provide cycle labels. Others do not. Some labels exist but are inconvenient for the question you want to ask. This walkthrough builds reusable cleaning functions that operate after dataset-specific parsing. They do not know whether the source was CALCE, NASA, Oxford, Severson, or a Mendeley sodium-ion CSV. They only require the canonical schema.

Create `batterydata/cleaning.py`:

```
from      import

import    as
import    as


def

                                float = 0.02

->
    """Infer charge, discharge, and rest labels from canonical current."""
    =
    =          "current a"          float
```

```

mode = np.full(len(output), "rest", dtype=object)
mode[current > current_threshold_a] = "discharge"
mode[current < -current_threshold_a] = "charge"
output["mode"] = mode
return output

```

```

def assign_cycles_from_discharge_starts(frame: pd.DataFrame) -> pd.DataFrame:

```

```

    """Assign cycle numbers whenever the mode enters discharge."""
    output = frame.copy().reset_index(drop=True)
    if "mode" not in output.columns or output["mode"].isna().all():
        output = infer_mode_from_current(output)

```

```

    mode = output["mode"].fillna("unknown").to_numpy()
    starts = np.zeros(len(output), dtype=bool)
    for index in range(len(output)):
        previous = mode[index - 1] if index > 0 else "rest"
        starts[index] = mode[index] == "discharge" and previous !=
        ↪ "discharge"

```

```

        =
        == 0 = 1
        "cycle_index" = int
    return

```

```

def                                     ->
    """Add signed capacity and absolute throughput using trapezoidal
    ↪ integration."""

```

```

    =                                     =True
    = "time_s"                           =float
    = "current_a"                        =float

    =                                     = 0
    =                                     < 0.0 0.0

    = *                                   / 3600.0
    "signed_capacity_ah" =
    "throughput_ah" = abs
    return

```

```

def                                     ->
    """Return one row per cycle with basic voltage, current, and capacity
    ↪ metrics."""

```



```

working = frame.copy()
if "cycle_index" not in working.columns or
    ↪ working["cycle_index"].isna().all():
    working = assign_cycles_from_discharge_starts(working)
if "throughput_ah" not in working.columns:
    working = add_integrated_columns(working)

rows = []
for cycle_index, group in working.groupby("cycle_index", dropna=True):
    rows.append(
        {
            "dataset": group["dataset"].iloc[0],
            "cell_id": group["cell_id"].iloc[0],
            "cycle_index": int(cycle_index),
            "duration_s": float(group["time_s"].max() -
                ↪ group["time_s"].min()),
            "voltage_min_v": float(group["voltage_v"].min()),
            "voltage_max_v": float(group["voltage_v"].max()),
            "current_min_a": float(group["current_a"].min()),
            "current_max_a": float(group["current_a"].max()),
            "throughput_ah": float(
                group["throughput_ah"].iloc[-1] -
                ↪ group["throughput_ah"].iloc[0]
            ),
        }
    )

return pd.DataFrame(rows)

```

Update batterydata/__init__.py:

```

from .calce import load_calce_file
from .cleaning import (
    add_integrated_columns,
    assign_cycles_from_discharge_starts,
    infer_mode_from_current,
    summarize_cycles,
)
from .generic import load_generic_csv
from .nasa import load_nasa_mat
from .schema import CANONICAL_COLUMNS, validate_canonical_frame

__all__ = [
    "CANONICAL_COLUMNS",
    "add_integrated_columns",

```

```

    "assign_cycles_from_discharge_starts",
    "infer_mode_from_current",
    "load_calce_file",
    "load_generic_csv",
    "load_nasa_mat",
    "summarize_cycles",
    "validate_canonical_frame",
]

```

Test it on the tiny fixture:

```

from batterydata import (
    add_integrated_columns,
    assign_cycles_from_discharge_starts,
    infer_mode_from_current,
    load_generic_csv,
    summarize_cycles,
)

frame = load_generic_csv(
    "data/raw/tiny_sodium_like_cycle.csv",
    dataset="teaching_fixture",
    cell_id="Na_fixture_001",
    column_map={
        "Time_sec": "time_s",
        "Current_A": "current_a",
        "Voltage_V": "voltage_v",
        "Temp_C": "temperature_c",
    },
)

clean = infer_mode_from_current(frame, current_threshold_a=0.05)
clean = assign_cycles_from_discharge_starts(clean)
clean = add_integrated_columns(clean)
summary = summarize_cycles(clean)

print(clean[["time_s", "current_a", "mode", "cycle_index",
    ↪ "signed_capacity_ah", "throughput_ah"]])
print(summary)

```

Expected output: the rows with 1.0 A should be labeled discharge, the rows with -0.8 A should be labeled charge, and rest rows should be labeled rest. The signed capacity increases during discharge and decreases during charge. Throughput only increases. The summary should contain one cycle with voltage

range around 2.98-3.30 V.

11.3.7 What Could Go Wrong

If all rows become `rest`, your current threshold is too high or your current is in milliamps. Lower `current_threshold_a` or convert units.

If every pulse becomes a new cycle, the current profile has intermittent rests inside discharge. In that case, assign cycles from source labels or use a more careful state machine that tolerates short rests.

If integrated capacity is far too large, inspect `time_s`. Some files use minutes or hours while naming the column “time”. Convert to seconds before canonical validation.

11.3.8 Reflection

This exercise separates source parsing from analysis cleaning. That separation is worth protecting. A NASA parser should not contain the same cycle-detection code as a CALCE parser. Once everything is canonical, one cleaning function serves every dataset.

Guided Walkthrough 5

Add Oxford and Severson/MATR Loaders

Learning objective: Handle large benchmark MATLAB datasets using defensive extraction functions.

Oxford Battery Degradation Dataset 1 contains long-term cycling of eight Kokam 740 mAh Li-ion pouch cells. The ORA record lists DOI [10.5287/bodleian:K02kdmYGg](https://doi.org/10.5287/bodleian:K02kdmYGg), publication date 2017, and a main `.mat` file of about 253.8 MB. The dataset is valuable because it includes long-term degradation with periodic characterization tests, but the structure is nested and large enough that you should inspect before loading everything repeatedly.

The Severson/Attia/MIT-Stanford-Toyota dataset accompanies the 2019 Nature Energy paper “Data-driven prediction of battery cycle life before capacity degradation.” The paper states that the dataset contains 124 commercial LFP/graphite cells fast-charged under varied policies, with cycle lives from about 150 to 2300 cycles, and that data are available at <https://data.matr.io/1>.

This dataset is important because it shifted the field toward early-cycle feature prediction.

Create `batterydata/mat_benchmarks.py`:

[illegible]

```

def _as_array(value: Any) -> np.ndarray:
    return np.asarray(value).squeeze()

def load_oxford_example_cell(
    path: str | Path,
    cell_key: str = "Cell1",
    max_cycles: int | None = 5,
) -> pd.DataFrame:
    """Load a small subset of Oxford Dataset 1 into the canonical schema.

    The Oxford dataset has several nested versions in circulation. This
    function handles the common MATLAB struct pattern and is intentionally
    conservative: it loads a limited number of cycles for inspection.
    """
    path = Path(path)
    mat = _loadmat_dict(path)
    public_keys = [key for key in mat if not key.startswith("__")]
    root = mat[public_keys[0]]

    cell = _get_field(root, cell_key) if hasattr(root, cell_key) else
    ↪ _get_field(root, cell_key.lower())
    cycles = _get_field(cell, "cyc")
    cycles_array = np.atleast_1d(cycles)

    frames: list[pd.DataFrame] = []
    selected_cycles = cycles_array[:max_cycles] if max_cycles is not None
    ↪ else cycles_array

    for cycle_number, cycle in enumerate(selected_cycles, start=1):
        try:
            time_s = _as_array(_get_field(cycle, "t"))
            voltage_v = _as_array(_get_field(cycle, "v"))
            current_a = _as_array(_get_field(cycle, "q"))
        except (AttributeError, KeyError):
            continue

        length = min(len(time_s), len(voltage_v), len(current_a))
        frame = pd.DataFrame(
            {
                "dataset": "Oxford",
                "cell_id": cell_key,
                "cycle_index": cycle_number,
                "time_s": time_s[:length].astype(float),
            }
        )

```

```

        "current_a": current_a[:length].astype(float),
        "voltage_v": voltage_v[:length].astype(float),
        "source_file": str(path),
    }
)
frames.append(frame)

if not frames:
    raise ValueError(
        "No Oxford cycles were extracted. Inspect the MATLAB keys and
        ↪ adapt "
        "field names t/v/q to the version you downloaded."
    )

return add_missing_optional_columns(pd.concat(frames, ignore_index=True))

def load_severson_batch_summary(path: str | Path) -> pd.DataFrame:
    """Load cycle-life summary fields from a Severson/MATR batch .mat
    ↪ file."""
    path = Path(path)
    mat = _loadmat_dict(path)
    public_keys = [key for key in mat if not key.startswith("__")]
    batch = mat[public_keys[0]]
    cells = np.atleast_1d(batch)

    rows = []
    for index, cell in enumerate(cells):
        policy = getattr(cell, "policy_readable", f"cell_{index}")
        cycle_life = float(np.asarray(getattr(cell, "cycle_life")).squeeze())
        rows.append(
            {
                "dataset": "Severson_MATR",
                "cell_id": f"cell_{index:03d}",
                "policy": str(policy),
                "cycle_life": cycle_life,
                "source_file": str(path),
            }
        )
    return pd.DataFrame(rows)

```

The Oxford loader above is intentionally not magical. Public copies of the Oxford .mat file may expose different struct names depending on how they were saved or mirrored. The `list_hdf5_tree` helper is there for v7.3-style files;

`_loadmat_dict` is there for traditional MATLAB files. If the fields are not `t`, `v`, and `q` in your copy, inspect the keys and adapt the three field names in one place.

Try the inspection workflow:

```
from pathlib import Path

from batterydata.mat_benchmarks import (
    list_hdf5_tree,
    load_oxford_example_cell,
    load_severson_batch_summary,
)

oxford_path =
↳ Path("data/raw/Oxford/Oxford_Battery_Degradation_Dataset_1.mat")
severson_path = Path(
↳ "data/raw/Severson/2017-05-12_batchdata_updated_struct_errorcorrect.mat")

if oxford_path.exists():
    try:
        oxford = load_oxford_example_cell(oxford_path, cell_key="Cell1",
↳ max_cycles=3)
        print(oxford.head())
    except NotImplementedError:
        print("\n".join(list_hdf5_tree(oxford_path)))
    except Exception as exc:
        print("Oxford structure needs inspection:", exc)

if severson_path.exists():
    summary = load_severson_batch_summary(severson_path)
    print(summary.head())
    print(summary["cycle_life"].describe())
else:
    print("Download Severson/MATR batch data from https://data.matr.io/1")
```

11.3.9 What Could Go Wrong

If the Oxford loader raises the custom “No Oxford cycles were extracted” error, do not panic. Run `loadmat(path).keys()` and inspect the first public object with `dir(obj)`. The dataset is stable, but MATLAB structs are not pleasant to generalize blindly.

If the Severson loader cannot find `cycle_life`, check whether you downloaded processed batch files from the Braatz GitHub repository or raw files

from MATR. The public ecosystem includes both. Adapt the summary loader to the file you actually cite.

If the full `.mat` file consumes too much memory, load summaries first. Do not repeatedly load a 250 MB or multi-GB file inside plotting cells. Convert the fields you need to a processed Parquet or CSV file once and record the loader version.

11.3.1 Reflection

This exercise taught the third public-data lesson: large benchmark datasets are not just larger CSV files. They are research artifacts with internal structure. The right workflow is inspect, extract the minimum needed, cache the normalized result, and keep enough provenance to make the extraction reproducible.

Guided Walkthrough 6

Reproduce a Severson-Style Early-Cycle Voltage Feature

Learning objective: Reproduce the central idea behind Severson et al.'s voltage-curve feature: early-cycle voltage differences can predict cycle life better than early capacity alone.

Severson et al. reported that features based on discharge voltage curves from early cycles predicted cycle life surprisingly well, even before obvious capacity degradation appeared. We will reproduce a simplified version of that idea. The full paper uses carefully processed voltage-capacity curves and machine-learning models. Our reproduction target is narrower: compute the variance of the difference between two early-cycle discharge voltage curves and plot it against cycle life.

This is a reproduction exercise, not a claim that our few lines duplicate every preprocessing choice in the paper. The paper and associated code should be your authority. Our goal is to learn how a public dataset becomes a figure-like research object, and where ambiguity enters.

Create `batterydata/severson_features.py`:

```
from           import
import         as
import         as
```



```

def voltage_difference_variance(
    capacity_ah: np.ndarray,
    voltage_cycle_a: np.ndarray,
    voltage_cycle_b: np.ndarray,
) -> float:
    """Return log10 variance of voltage difference between two cycles."""
    capacity_ah = np.asarray(capacity_ah, dtype=float)
    voltage_cycle_a = np.asarray(voltage_cycle_a, dtype=float)
    voltage_cycle_b = np.asarray(voltage_cycle_b, dtype=float)

    valid = (
        np.isfinite(capacity_ah)
        & np.isfinite(voltage_cycle_a)
        & np.isfinite(voltage_cycle_b)
    )
    if valid.sum() < 10:
        raise ValueError("At least 10 valid points are required")

    difference = voltage_cycle_b[valid] - voltage_cycle_a[valid]
    variance = float(np.var(difference))
    return float(np.log10(variance))

def make_teaching_severson_fixture() -> pd.DataFrame:
    """Create a small Severson-like table for code-path testing."""
    rng = np.random.default_rng(42)
    rows = []
    capacity_grid = np.linspace(0.0, 1.05, 120)
    synthetic_cells = [
        ("cell_A", 2100, 0.002),
        ("cell_B", 1400, 0.006),
        ("cell_C", 850, 0.014),
        ("cell_D", 420, 0.030),
    ]

    for cell_id, cycle_life, drift_scale in synthetic_cells:
        base_voltage = 3.35 - 0.45 * capacity_grid + 0.04 * np.tanh((0.55 -
↪ capacity_grid) / 0.08)
        cycle_10 = base_voltage + rng.normal(0.0, 0.0008,
↪ size=capacity_grid.size)
        = - * - 0.5 ** 2
        += 0.0 0.0008 =

        feature = voltage_difference_variance(capacity_grid, cycle_10,
↪ cycle_100)

```

```

        rows.append(
            {
                "cell_id": cell_id,
                "cycle_life": cycle_life,
                "log10_variance_delta_v": feature,
            }
        )

    return pd.DataFrame(rows)

```

Run the fixture reproduction:

```

import matplotlib.pyplot as plt

from batterydata.severson_features import make_teaching_severson_fixture

features = make_teaching_severson_fixture()
print(features)

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.scatter(features["log10_variance_delta_v"], features["cycle_life"], s=70)
for _, row in features.iterrows():
    ax.annotate(row["cell_id"], (row["log10_variance_delta_v"],
        ↪ row["cycle_life"]))
ax.set_xlabel("log10 variance of Delta V between early cycles")
ax.set_ylabel("Cycle life")
ax.set_title("Severson-style teaching reproduction")
ax.grid(True)
fig.tight_layout()
plt.show()

```

A correct plot should show an inverse relationship: cells with more negative `log10_variance_delta_v`, meaning smaller early voltage-curve change, should have longer cycle life. Cells with larger variance should have shorter cycle life. The synthetic fixture is intentionally tiny, so do not report correlation statistics from it as a scientific result.

When using the full MATR dataset, replace `make_teaching_severson_fixture()` with extracted voltage-capacity arrays from the public batch files. The essential processing steps are:

1. Choose two early cycles, commonly cycle 10 and cycle 100 in the Severson paper's feature discussion.

2. Interpolate both discharge voltage curves onto a common discharge-capacity grid.
3. Compute the pointwise voltage difference.
4. Compute a scalar feature such as variance or minimum of the difference.
5. Compare that feature against measured cycle life.

Where the paper is ambiguous or your downloaded file version differs, document the choice. Did you use discharge capacity or normalized capacity? Did you exclude cells that failed before cycle 100? Did you interpolate in voltage-capacity space or time space? Did you use all batches or one batch? “Close enough” for this lab means reproducing the qualitative inverse relationship and obtaining a scatter plot with the same interpretation, not matching every number in the paper.

11.3.1 What Could Go Wrong

If your full-data scatter plot has no trend, check that cycle numbers are aligned. Off-by-one cycle indexing is common when MATLAB arrays and Python arrays meet.

If the variance is exactly zero for many cells, you may be subtracting the same curve from itself or using a repeated reference due to a bad dictionary key.

If short-life cells lack cycle 100, do not silently drop them without reporting it. The act of excluding failed-before-100 cells changes the population.

11.3.1 Reflection

This exercise is the heart of Part V. A published figure is not a picture; it is a chain of data access, parsing, cleaning, feature construction, and judgment calls. You now have the beginning of a reproducible chain.

Dataset Integration

Public Sources and Practical Download Notes

Use this table as a starting map, not as a substitute for reading each dataset’s documentation.

Dataset	URL	Typical format	Approximate scale	License or terms note	Best use
CALCE Battery Data Archive	https://calce.ece.mcmaster.ca/linked/Chatterjee-database	Excel	Specific files	CALCE database and contributor papers	Cycling, capacity fade, ECM practice
NASA PCoE Li-ion Battery Aging	https://data.nasa.gov/database/Li-ion-battery-aging-datasets	MATLAB .mat		Open Data page lists license not specified	aging, temperature traces
Oxford Battery Degradation Dataset 1	https://ora.ox.ac.uk/objects/uuid:03ba4b01-0fed-46d3-9b1a-7d410.5287/bodyman/K02kdmYGg	MATLAB .mat	15 MB main file	ORCID iD of use, DOI 10.5287/bodyman/K02kdmYGg	Long-term degradation diagnostic cycles
Severson/MATR fast-charge dataset	https://data.mtlab.io/Large, 124 cells	MATLAB .mat, processed structs	Large, 124 cells	Check MATR and paper terms	Early prediction of cycle life
Sandia battery datasets	Sandia data portals and GitHub mirrors vary by project	CSV, HDF5, reports	Varies	Check each project page	Abuse, aging, grid-storage conditions
Commercial Na-ion Ragone dataset	https://data.mendeley.com/datasets/G5Bj440vwc/1	Excel files from Mendeley	100 MB	CC-BY 4.0	Sodium-ion power/energy characterization

Dataset	URL	Typical format	Approximate scale	License or terms note	Best use
NFM sodium-ion degrada- tion dataset	https://data.mendeley.com/datasets/64wz0cd8	files from Mendeley	2026	CC-BY-NC	Sodium-ion SOH and degrada- tion
Sodium-ion SOC dataset	https://zenodo.org/records/1383621	files	1383621	record terms	Sodium-ion pulse and drive-cycle SOC
High- throughput Na-ion forma- tion/cycling	https://zenodo.org/records/7981071	files	7981071	record terms	Chemistry screening and formation data

For Mendeley and Zenodo datasets, the generic CSV loader is often the right first tool. Download the archive manually, inspect the column names, and write a short `column_map`. Because these datasets are newer and less standardized, do not assume that every file has time, current, voltage, and temperature. Some provide only cycle summaries or figure-support spreadsheets.

The sparse sodium-ion situation is changing quickly. As of this chapter’s writing, examples include commercial sodium-ion Ragone characterization, NFM sodium-ion degradation data, sodium-ion SOC pulse/drive-cycle data, and high-throughput sodium-ion formation/cycling records. That is good news, but it does not remove the need for caution. A sodium-ion dataset may represent one commercial cell, one cathode family, one temperature, or one test protocol. Generalizing from it requires much more care than validating a data-cleaning workflow on it.

11.4 Open-Ended Exercises

11.4.1 Exercise 1: Add a Sandia-Style CSV Loader

Find one Sandia battery dataset or another national-lab battery CSV dataset with time, current, and voltage columns. Write a `load_sandia_csv` function that wraps `load_generic_csv` but fixes the dataset label and any sign convention. Save one processed file to `data/processed`.

Hints: start with a single file, not the whole archive. Print `raw.columns`. Plot current and voltage before trusting the sign convention.

11.4.2 Exercise 2: Add a Sodium-Ion Mendeley or Zenodo Loader

Choose either the commercial sodium-ion Ragone characterization dataset on Mendeley or the Zenodo sodium-ion SOC dataset. Download one file, inspect the columns, and load it into the canonical schema. Write a short note explaining which metadata are missing compared with the Li-ion benchmark datasets.

Hints: if the dataset is an Excel workbook, use `pd.read_excel(sheet_name=None)` to list sheets. If the file contains cycle summaries rather than raw time series, create a separate summary table rather than forcing it into the time-series schema.

11.4.3 Exercise 3: Stress-Test Cycle Detection

Construct a synthetic current profile with two discharge pulses separated by a short rest, then a long rest, then charge. Test `assign_cycles_from_discharge_starts`. Decide whether the short rest should split the cycle. Modify the function or write a new one that tolerates rests shorter than 60 s.

Hints: a robust state machine should use both mode and rest duration. Do not solve this with a fragile row-count threshold unless the sampling period is fixed.

11.4.4 Exercise 4: Extend the Severson Reproduction

Using the full MATR data if available, compute two features for each cell: variance of ΔV between cycles 10 and 100, and minimum of ΔV between cycles 10 and 100. Plot each feature against cycle life. Which feature looks more monotonic?

Hints: interpolate onto a shared capacity grid. Exclude or separately mark cells that do not have cycle 100. Compare your choices to the public Braatz processing

code.

11.5 Worked Solutions to Open-Ended Exercises

A compact Sandia-style wrapper looks like this:

```
from pathlib import Path

from batterydata import load_generic_csv

def load_sandia_csv(path: str | Path, cell_id: str):
    return load_generic_csv(
        path=path,
        dataset="Sandia_or_national_lab",
        cell_id=cell_id,
        column_map={
            "TestTime_s": "time_s",
            "Current_A": "current_a",
            "Voltage_V": "voltage_v",
            "Temperature_C": "temperature_c",
        },
        current_sign="positive_discharge",
    )
```

The exact raw names will differ. The important pattern is that the wrapper records the source label and hides repetitive column-map code.

A sodium-ion Mendeley CSV loader should look similar, but the research note matters more than the code. For example, if a Ragone file contains discharge energy and power but no raw time axis, do not pretend it is a time series. Create a `ragone_summary.csv` with columns such as `cell_id`, `current_a`, `specific_energy_wh_kg`, and `specific_power_w_kg`, then cite it as a summary dataset.

A rest-tolerant cycle detector can be sketched as:

```
import numpy as np
import pandas as pd

def assign_cycles_with_rest_tolerance(frame: pd.DataFrame, max_rest_s: float
    ↪ = 60.0) -> pd.DataFrame:
    =                                     =True
    = "mode"                             "unknown"
```

```

time_s = output["time_s"].to_numpy(dtype=float)

cycle = 0
in_discharge_episode = False
rest_start_s = None
cycle_indices = []

for index, current_mode in enumerate(mode):
    if current_mode == "discharge":
        if not in_discharge_episode:
            cycle += 1
            in_discharge_episode = True
            rest_start_s = None
    elif current_mode == "rest" and in_discharge_episode:
        if rest_start_s is None:
            rest_start_s = time_s[index]
        elif time_s[index] - rest_start_s > max_rest_s:
            in_discharge_episode = False
    elif current_mode == "charge":
        in_discharge_episode = False
        rest_start_s = None

    cycle_indices.append(max(cycle, 1))

output["cycle_index"] = np.asarray(cycle_indices, dtype=int)
return output

```

For the Severson extension, a well-structured result table has one row per cell:

```

severson_feature_table = pd.DataFrame(
    {
        "cell_id": ["example_cell"],
        "cycle_life": [1200],
        "log10_var_delta_v_10_100": [-4.8],
        "min_delta_v_10_100": [-0.014],
        "used_cycle_10": [True],
        "used_cycle_100": [True],
    }
)
print(severson_feature_table)

```

The two boolean columns are not decoration. They make exclusions visible when you move from a teaching fixture to a real benchmark.

What Changes for Sodium-Ion?

The loader architecture barely changes for sodium-ion. Time is still time, current is still current, voltage is still voltage, and temperature is still temperature. The interpretation changes sharply.

First, voltage windows differ. A sodium-ion cell may operate over 2.0-4.0 V, 1.5-4.2 V, or another chemistry-specific range. Do not let the schema validator's generic 0-6 V check become a chemistry claim. Add chemistry metadata when you know it.

Second, OCV shape and plateau behavior differ. A flat hard-carbon plateau can make SOC inference and voltage-curve features behave differently from graphite/LFP or graphite/NMC cells. A feature that predicts Li-ion cycle life may be numerically computable for sodium-ion but physically less meaningful without validation.

Third, public sodium-ion datasets are less standardized. Some are figure-support datasets, some are commercial-cell characterization files, and some are high-throughput materials-screening records. You may need separate loaders for time-series cycling, Ragone summaries, EIS spectra, and extracted figure curves.

Fourth, validation strategy changes. For a sodium-ion methods paper, it is acceptable to validate the parser and feature workflow on lithium-ion benchmarks, then apply it to sodium-ion data as a case study. It is not acceptable to imply that a Li-ion-trained lifetime model is validated for sodium-ion just because the code runs.

Chapter Summary and Skill Checklist

- You built a reusable `batterydata` package.
- You defined a canonical battery time-series schema with consistent units.
- You normalized current to positive discharge and negative charge.
- You wrote a generic CSV loader for Mendeley, Zenodo, and lab-export files.
- You wrote CALCE and NASA parsers with source-specific handling.
- You inspected Oxford and Severson/MATR benchmark structures.
- You added cleaning utilities for mode inference, cycle assignment, and throughput integration.
- You reproduced a simplified Severson-style early-cycle voltage-feature figure.

- You identified why sodium-ion dataset interpretation is harder than sodium-ion file parsing.

Commands, functions, and patterns to keep in muscle memory:

- `pd.read_csv(...)`, `pd.read_excel(...)`, and `scipy.io.loadmat(...)`
- `h5py.File(path, "r")` for MATLAB v7.3/HDF5 inspection
- `load_generic_csv(..., column_map=..., current_sign=...)`
- `validate_canonical_frame(frame)`
- `infer_mode_from_current(frame, current_threshold_a=...)`
- `assign_cycles_from_discharge_starts(frame)`
- `add_integrated_columns(frame)`
- `groupby("cycle_index")` for cycle summaries
- Always preserve `source_file`

You should now be able to:

- Find public battery datasets and record their source, format, and license terms.
- Explain why current sign conventions must be normalized explicitly.
- Parse mixed CSV, Excel, text, and MATLAB battery files.
- Convert public data into consistent columns and units.
- Detect common dataset problems before fitting a model.
- Build a loader that later PyBaMM, MATLAB, ECM, and BMS workflows can reuse.
- State clearly what is and is not validated when adapting Li-ion workflows to sodium-ion data.

Deliverable

Your deliverable is a Python package directory named `batterydata` inside `SimulationCompanion/chapter11_dataset_workspace`, plus a short report named `chapter11_dataset_loader_report.md`.

The package must provide a unified interface to at least four public dataset families. A strong submission includes:

```

batterydata/
    __init__.py
    schema.py
    generic.py
    calce.py
    nasa.py
    cleaning.py
    mat_benchmarks.py
    seversion_features.py
tests/
    test_schema.py
    test_cleaning.py
data/
    raw/
    processed/
figures/

```

Your report should answer:

1. Which four datasets did you support?
2. What raw formats did they use?
3. What sign convention did each source appear to use?
4. What rows, columns, or files did your loader drop?
5. Which sodium-ion dataset did you inspect, and what was missing relative to Li-ion benchmarks?
6. Which figure or feature did you reproduce, and how close was it?

A worked partial solution is already present in the guided walkthroughs. To make it research-grade, add two tests:

```

import sys as sys

from typing import List
from typing import Dict

def test_demo() -> bool:
    """Test the demo dataset"""
    dataset = "demo"
    cell_id = "cell"
    time_s = 0.0
    current_a = 1.0

```

```

        "voltage_v": [3.2],
        "source_file": ["memory"],
    }
)
assert validate_canonical_frame(frame).is_valid

def test_mode_inference_uses_positive_discharge():
    frame = pd.DataFrame({"current_a": [1.0, -1.0, 0.0]})
    output = infer_mode_from_current(frame, current_threshold_a=0.05)
    assert output["mode"].tolist() == ["discharge", "charge", "rest"]

```

Run the tests with:

```
python -m pytest -q
```

Expected output:

```
2 passed
```

Further Practice and Reading

Key papers and datasets:

1. Severson, K. A., Attia, P. M., Jin, N., et al. “Data-driven prediction of battery cycle life before capacity degradation.” *Nature Energy* 4, 383-391 (2019).
Dataset: <https://data.matr.io/1>.
2. Birkel, C. R. and Howey, D. A. Oxford Battery Degradation Dataset 1. DOI 10.5287/bodleian:K02kdmYGg.
3. NASA PCoE Li-ion Battery Aging Datasets:
<https://data.nasa.gov/dataset/li-ion-battery-aging-datasets>.
4. CALCE Battery Data Archive: <https://calce.umd.edu/battery-data>.
5. Commercial sodium-ion cell and sodium-ion degradation datasets on Mendeley and Zenodo, especially
<https://data.mendeley.com/datasets/j44rvwcpff>,
<https://data.mendeley.com/datasets/4mztcdc4gt>, and
<https://zenodo.org/records/13836819>.

Official and community resources:

1. pandas documentation for IO tools:
https://pandas.pydata.org/docs/user_guide/io.html.
2. SciPy loadmat documentation:
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.io.loadmat.html>.
3. h5py documentation: <https://docs.h5py.org/>.
4. Braatz group public code for the Severson cycle-life dataset:
<https://github.com/rdbraatz/data-driven-prediction-of-battery-cycle-life>.
5. WebPlotDigitizer for extracting curves from papers when sodium-ion raw data are unavailable: <https://automeris.io/WebPlotDigitizer/>.

Chapter 12 is next: The Reproduction Project.

The Reproduction Project

This chapter is the hinge between learning tools and becoming useful with them.

So far, you have built simulations, parameter estimators, equivalent-circuit models, Kalman filters, aging workflows, thermal models, cross-language pipelines, and public-dataset loaders. Each of those skills matters. None of them, by itself, proves that you can do research. Research competence appears when you can take an outside result, read it without mystifying it, rebuild the computational path, document every choice you had to make, and explain why your reproduction is close, different, or impossible.

That is the skill of this chapter. We will reproduce a published battery paper in the practical, non-romantic sense of the word. We will not pretend that every paper is fully reproducible. Many are not. We will not pretend that exact numerical equality is always possible. It usually is not, because software versions, hidden preprocessing choices, random seeds, proprietary instruments, and missing metadata all matter. Instead, we will practice the standard that a serious reviewer or future collaborator will respect: clear scope, clean environment, faithful implementation, documented deviations, quantitative comparison, and a write-up that separates what the paper says from what you inferred.

Keep Textbook Chapters 2, 7, 8, 10, 12, and 13 open. This chapter operationalizes the measurement discipline from Textbook Chapter 2, the degradation

interpretation from Textbook Chapter 7, the model hierarchy from Textbook Chapter 8, the BMS workflow logic from Textbook Chapter 10, the thermal and safety caution from Textbook Chapter 12, and the sodium-ion constraints from Textbook Chapter 13. It also draws directly on Lab Chapters 1, 2, 8, 10, and 11. Lab Chapter 11 gave you dataset loaders; this chapter makes those loaders answer to an external paper.

For the fully guided reproduction path, we will use Severson et al., “Data-driven prediction of battery cycle life before capacity degradation,” published in *Nature Energy* in 2019. The paper is valuable for this manual because it is influential, it has public data, it has public data-processing code, and it exposes exactly the kind of ambiguity that reproduction work teaches you to handle. The paper reports that voltage-curve features from early cycles can predict cycle life before obvious capacity fade appears. We will reproduce a Figure 2-style result: voltage-curve features from early cycles are much more predictive of cycle life than early discharge capacity alone. Where the full MATR dataset is not available on your laptop, the chapter provides a deterministic schema-compatible fixture so every script still runs end to end. That fixture is for learning the workflow, not for making scientific claims.

By the end of the chapter, you will have a `chapter12_reproduction_project` repository with a pinned environment, a paper-selection scorecard, a reproduction checklist, a research log, a runnable reproduction script, generated figures, numerical comparison metrics, and a short manuscript-style report. The deliverable is not a pretty notebook. It is a small, auditable reproduction package that another graduate student could clone, run, criticize, and extend.

12.1 Prerequisites Check

- Required software: Python 3.11, `numpy==1.26.4`, `pandas==2.2.2`, `scipy==1.13.1`, `matplotlib==3.9.0`, `h5py==3.11.0`, `pyyaml==6.0.2`, `requests==2.32.3`, and `pytest==8.3.2`
- Optional software: Git 2.40 or newer; JupyterLab 4.x; MATLAB R2024b if you choose an ECM or BMS reproduction instead of the guided Python path
- Install command: `python -m pip install numpy==1.26.4 pandas==2.2.2 scipy==1.13.1 matplotlib==3.9.0 h5py==3.11.0 pyyaml==6.0.2 requests==2.32.3 pytest==8.3.2`

- Required textbook chapters: Textbook Chapters 2, 7, 8, 10, 12, and 13
- Required prior lab chapters: Lab Chapters 1, 2, and 11 are essential; Lab Chapters 6, 8, 9, and 10 are recommended depending on the paper you choose
- Public sources used in this chapter: Severson et al. (2019), the MATR data portal, the Braatz group public processing repository, PyBaMM documentation, and the Marquis et al. SPMe paper as an alternative physics-model reproduction target
- Estimated time: 12 to 16 hours with the built-in fixture; 24 to 40 hours if you download, inspect, and reproduce against the full MATR dataset

If Git still feels awkward, revisit Lab Chapter 1 before starting. If pandas grouping and plotting feel shaky, revisit Lab Chapter 2. If public battery data still feels like a pile of miscellaneous files, revisit Lab Chapter 11. Reproduction work is mostly not about brilliance. It is about not losing track of what you did.

12.2 Environment Setup

We will create a fresh environment and a fresh project folder. This is not optional ceremony. Reproduction work is a test of whether your computation can be separated from your memory of how you ran it.

12.2.1 Create the environment

From the repository root, run:

```
cd /home/avirup/SodiumIonBatteryResearch
python3.11 -m venv .venv-chapter12
source .venv-chapter12/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==1.26.4 pandas==2.2.2 scipy==1.13.1
↳ matplotlib==3.9.0 h5py==3.11.0 pyyaml==6.0.2 requests==2.32.3
↳ pytest==8.3.2
```

On Windows PowerShell, use:

```
cd C:\path\to\SodiumIonBatteryResearch
py -3.11 -m venv .venv-chapter12
.\.venv-chapter12\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install numpy==1.26.4 pandas==2.2.2 scipy==1.13.1
↳ matplotlib==3.9.0 h5py==3.11.0 pyyaml==6.0.2 requests==2.32.3
↳ pytest==8.3.2
```


The guided reproduction does not require PyBaMM, scikit-learn, or MATLAB. That is deliberate. We will implement the regression in transparent NumPy so that the reader can inspect every mathematical step. If you choose a PyBaMM reproduction target from the open-ended exercises, use the pinned PyBaMM environment from Lab Chapter 10.

12.2.2 Create the project workspace

Run:

```
mkdir -p SimulationCompanion/chapter12_reproduction_project
cd SimulationCompanion/chapter12_reproduction_project
mkdir -p data/raw data/processed figures reports src tests logs references
touch logs/research-log.md
touch references/source-notes.md
```

This folder is intentionally shaped like a miniature paper repository. `data/raw` is for files exactly as downloaded. `data/processed` is for generated tables. `src` is for reusable code. `figures` is for generated plots. `reports` is for the final write-up. `logs` is for dated decisions. `references` is for notes about the paper, dataset, and code sources.

Now create a `README.md`:

```
# Chapter 12 Reproduction Project
```

This repository reproduces selected results from a published battery paper.

Target paper:

- Severson, K. A., Attia, P. M., Jin, N., et al. "Data-driven prediction of
↪ battery cycle life before capacity degradation." Nature Energy 4, 383-391
↪ (2019). <https://doi.org/10.1038/s41560-019-0356-8>

Scope:

- Reproduce a Figure 2-style result showing that early-cycle voltage-curve
↪ features are more predictive of cycle life than early discharge capacity
↪ alone.
- Use the public MATR dataset when available.
- Use a deterministic fixture only for workflow verification when the full
↪ dataset is absent.

Run:

```
```bash
python src/reproduce_severson_voltage_feature.py
```
```

The README is part of the method. It tells a future reader what you are trying to reproduce before they read any code.

12.2.3 Verify the install

Create `src/verify_environment.py`:

```
import h5py
import matplotlib
import numpy as np
import pandas as pd
import scipy
import yaml

print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
print("SciPy:", scipy.__version__)
print("Matplotlib:", matplotlib.__version__)
print("h5py:", h5py.__version__)
print("PyYAML:", yaml.__version__)

times = np.array([0.0, 1.0, 2.0, 3.0])
voltage = np.array([3.55, 3.48, 3.42, 3.36])
capacity = np.trapz(np.ones_like(times), times) / 3600.0

print(f"Demo voltage drop: {voltage[0] - voltage[-1]:.3f} V")
print(f"Demo integrated capacity: {capacity:.6f} Ah")
```

Run:

```
python src/verify_environment.py
```

Expected output:

```
NumPy: 1.26.4
pandas: 2.2.2
SciPy: 1.13.1
Matplotlib: 3.9.0
h5py: 3.11.0
PyYAML: 6.0.2
```

```
Demo voltage drop: 0.190 V
Demo integrated capacity: 0.000833 Ah
```

The exact package versions should match. If Matplotlib prints 3.9.1 or pandas prints 2.2.3, the chapter code will probably still run, but record the difference in `logs/research-log.md`. Reproduction is not only about what breaks. It is also about what did not break under a slightly different environment.

12.2.4 Common setup failures and fixes

`python3.11: command not found` means Python 3.11 is not installed or not on your shell path. Use `python --version` to see what you have. Python 3.10 or 3.12 should be acceptable for this chapter, but write the actual version in the research log.

`ModuleNotFoundError: No module named 'yaml'` means `pyyaml` did not install into the active environment. Confirm the prompt shows `.venv-chapter12`, then rerun the install command.

`Permission denied` while creating folders usually means you are not in a writable project directory. Print `pwd` or `Get-Location`, then return to the repository root.

`Matplotlib is currently using agg` is not an error. It means Matplotlib is writing images without opening an interactive window. That is normal in a reproducible script.

12.3 Conceptual Bridge: From Reading a Paper to Rebuilding a Computational Claim

In the theory textbook, a claim was usually attached to a derivation. You could follow the equations from assumptions to consequences. In a computational battery paper, the claim is attached to a chain of choices. Some choices are visible: the dataset, the model class, the loss function, the solver, the plotted variables. Other choices are quiet: the version of a library, the exact train-test split, the interpolation grid, the row filters, the current sign convention, the point at which rest periods were removed, and the random seed.

Reproduction means turning that quiet chain into an explicit object.

The first mental shift is that a figure is not just a picture. A figure is the final artifact of a pipeline:

raw data → cleaning → features or model states → estimation → metrics → plot.
(1)

When you reproduce a figure, you are not tracing over it. You are rebuilding Equation (1). If your result differs, the difference may be in any arrow of that pipeline. A beginner often jumps straight to “my model is wrong.” A reproducibility-minded researcher asks smaller questions. Did I load the same cells? Did I use the same cycle range? Did I interpolate voltage onto the same capacity axis? Did I split train and test the same way? Did I transform the target variable before regression? Did I plot on the same axis scale?

The second shift is that “close enough” is not a feeling. It is a predeclared tolerance. For a voltage trace simulated by the same deterministic code, close enough may mean sub-millivolt agreement. For a reproduction of a published scatter plot from a public dataset with incomplete code, close enough may mean the same qualitative ordering, similar coefficient signs, and an error metric within a documented range. For a model trained with a random split that the paper does not disclose, exact pointwise equality is not a fair target. Your job is to state the tolerance before you see whether you passed.

The third shift is that ambiguity is not failure. Ambiguity is data. When a paper does not specify a smoothing window, a solver tolerance, or a train-test split, you have learned something important about the reproducibility of the result. Standard practice is not to hide that gap. Standard practice is to write down the ambiguity, choose a defensible default, run a sensitivity check when feasible, and report whether the conclusion depends on the choice.

The fourth shift is especially important for sodium-ion research. Many sodium-ion papers have smaller datasets, less standardized parameter reporting, fewer public raw files, and chemistry-specific voltage curves that make lithium-ion workflows only partly transferable. A reproduction project in sodium-ion may therefore reproduce a method rather than a dataset. For example, you may validate a feature-extraction workflow on the public lithium-ion MATR dataset, then apply the same workflow to a smaller sodium-ion cycling dataset from Zenodo or Mendeley. That is valid if you say it plainly: “The computational method was reproduced on the public Li-ion benchmark; the sodium-ion application is an

adaptation, not a direct reproduction.”

The final shift is that a failed reproduction can still be successful research training. If you discover that a paper cannot be reproduced because the data are not public, the code depends on obsolete proprietary software, or a key preprocessing step is missing, that is not wasted effort. Your final product becomes a reproducibility audit. It can still teach you how to read methods sections, how to isolate missing information, and how to design your own future papers so someone else does not suffer the same fog.

Guided Walkthrough 1

Choose a Reproduction Target with a Scorecard

Learning objective: Convert paper selection from a vague preference into a documented, defensible decision.

Before writing code, we choose a paper. This step is easy to underestimate. A beautiful paper with private data and no code may be excellent science but a poor first reproduction project. A modest paper with public data, clear figures, and runnable scripts may teach you much more. In this walkthrough, we will create a paper-selection scorecard and apply it to four candidate papers.

Create `src/paper_scorecard.py`:

```
from __future__ import annotations

from dataclasses import dataclass

@dataclass(frozen=True)
class CandidatePaper:
    short_name: str
    citation: str
    tool_family: str
    data_available: int
    code_available: int
    environment_clarity: int
    figure_specificity: int
    sodium_transfer: int
    difficulty: int
    notes: str
```

```

def score(self) -> int:
    positive = (
        self.data_available
        + self.code_available
        + self.environment_clarity
        + self.figure_specificity
        + self.sodium_transfer
    )
    return positive - self.difficulty

def main() -> None:
    candidates = [
        CandidatePaper(
            short_name="Severson2019",
            citation=(
                "Severson et al., Data-driven prediction of battery cycle
                ↪ life "
                "before capacity degradation, Nature Energy, 2019"

                ="Python/MATLAB data workflow"
                =5
                =4
                =3
                =5
                =4
                =3

                =
                "Public MATR data and public processing code; modeling code
                ↪ is not "
                "fully open, so reproduce a feature/result rather than every
                ↪ model."

                ="Marquis2019"
                =
                "Marquis et al., An asymptotic derivation of a single
                ↪ particle "
                "model with electrolyte, Journal of The Electrochemical
                ↪ Society, 2019"

                ="PyBaMM physics-based simulation"
                =3
                =5

```

```

environment_clarity=4,
figure_specificity=4,
sodium_transfer=3,
difficulty=4,
notes=(
    "Strong PyBaMM fit; best if you want reduced-order
    ↳ electrochemical "
    "model reproduction rather than public-data handling."
),
),
CandidatePaper(
    short_name="PlettECM",
    citation=(
        "Plett, Battery Management Systems, Volume II,
        ↳ equivalent-circuit "
        "model identification examples"
    ),
    tool_family="MATLAB ECM and BMS algorithms",
    data_available=3,
    code_available=3,
    environment_clarity=4,
    figure_specificity=3,
    sodium_transfer=4,
    difficulty=3,
    notes=(
        "Excellent for BMS method reproduction; less direct as a
        ↳ published "
        "paper figure unless paired with a public HPPC dataset."
    ),
),
CandidatePaper(
    short_name="PyBaMM2021",
    citation=(
        "Sulzer et al., Python Battery Mathematical Modelling
        ↳ (PyBaMM), "
        "Journal of Open Research Software, 2021"
    ),
    tool_family="PyBaMM software paper",
    data_available=4,
    code_available=5,
    environment_clarity=5,
    figure_specificity=3,
    sodium_transfer=3,
    difficulty=2,
    notes=(

```

```

        "Very reproducible software context; better for validating a
        ↪ tool "
        "workflow than for reproducing a battery-science claim."
    ),
),
]

ranked = sorted(candidates, key=lambda paper: paper.score(),
↪ reverse=True)

print("Reproduction target scorecard")
print("=" * 34)
for paper in ranked:
    print(f"{paper.short_name}: {paper.score():2d} points")
    print(f"  Tool family: {paper.tool_family}")
    print(f"  Citation: {paper.citation}")
    print(f"  Notes: {paper.notes}")
    print()

if __name__ == "__main__":
    main()

```

Run:

```
python src/paper_scorecard.py
```

Expected output:

```

Reproduction target scorecard
=====
Severson2019: 18 points
  Tool family: Python/MATLAB data workflow
  Citation: Severson et al., Data-driven prediction of battery cycle life
↪ before capacity degradation, Nature Energy, 2019
  Notes: Public MATR data and public processing code; modeling code is not
↪ fully open, so reproduce a feature/result rather than every model.

PyBaMM2021: 18 points
  Tool family: PyBaMM software paper
  Citation: Sulzer et al., Python Battery Mathematical Modelling (PyBaMM),
↪ Journal of Open Research Software, 2021
  Notes: Very reproducible software context; better for validating a tool
↪ workflow than for reproducing a battery-science claim.

```


Marquis2019: 15 points

Tool family: PyBaMM physics-based simulation

Citation: Marquis et al., An asymptotic derivation of a single particle

↪ model with electrolyte, Journal of The Electrochemical Society, 2019

Notes: Strong PyBaMM fit; best if you want reduced-order electrochemical

↪ model reproduction rather than public-data handling.

PlettECM: 14 points

Tool family: MATLAB ECM and BMS algorithms

Citation: Plett, Battery Management Systems, Volume II, equivalent-circuit

↪ model identification examples

Notes: Excellent for BMS method reproduction; less direct as a published

↪ paper figure unless paired with a public HPPC dataset.

The scoring is intentionally simple. Each positive dimension is scored from 0 to 5, and difficulty is subtracted. The `CandidatePaper` dataclass stores each paper as a structured object so the criteria can be audited. `score()` adds the reproducibility strengths and subtracts difficulty. This is not a universal metric. It is a thinking tool.

Severson2019 and PyBaMM2021 tie in this example. We choose Severson2019 because it asks more of the skills built in this companion: public data handling, feature extraction, regression, figure comparison, and degradation interpretation. PyBaMM2021 is easier to run, but it is more of a software demonstration than a battery-research reproduction.

12.3.1 What could go wrong

If your output order differs for tied scores, nothing is wrong. Python sorting is stable, but if you edit candidate scores, the order will change.

If you are tempted to set every candidate score to 5, slow down. The purpose is not to flatter the papers. The purpose is to choose a target that can teach you.

If a candidate has no public data and no code, do not choose it for your first reproduction project. Save it for a literature critique.

12.3.2 Reflection

This exercise taught you to treat paper selection as part of the research method. Later, when you design your own sodium-ion project, you will use the same logic in reverse: make your paper easy to reproduce by publishing data, code, versions,

and figure-generation scripts.

Guided Walkthrough 2

Build the Reproduction Checklist

Learning objective: Translate a paper into a checklist of computational claims, required files, assumptions, and tolerances.

Reading for reproduction is different from reading for comprehension. When you read for comprehension, you ask, “What did the authors find?” When you read for reproduction, you ask, “What exact sequence of operations would make this figure appear?” We will build a machine-readable checklist in YAML so it can live beside the code.

Create `references/reproduction-checklist.yaml`:

```
project:
  target_paper: "Severson et al. 2019 Nature Energy"
  target_result: "Figure 2-style early voltage-curve feature prediction"
  reproduction_type: "partial computational reproduction"
  date_started: "2026-05-01"

sources:
  paper_url: "https://doi.org/10.1038/s41560-019-0356-8"
  data_url: "https://data.matr.io/1/"
  processing_code_url:
    "https://github.com/rdbraatz/data-driven-prediction-of-battery-cycle-life-before-capaci

claim:
  plain_language: >
    Features derived from differences between early discharge voltage curves
    predict cycle life better than early discharge capacity alone.
  figure_target: >
    Reproduce the qualitative structure of Figure 2: voltage-curve features
    separate short-lived and long-lived cells and support useful cycle-life
    prediction.

required_inputs:
  - name: "MATR batch data"
    status: "optional_full_dataset"
    local_path: "data/raw"
  - name: "Schema-compatible fixture"
    status: "included_in_script"
```

```

    local_path: "generated_when_full_dataset_absent"

known_ambiguities:
  - "Exact train/test split may differ from the paper unless the original
    ↪ split is recovered."
  - "The public repository includes data processing code, while full modeling
    ↪ code availability is limited."
  - "Different public file formats may store cycle dictionaries with slightly
    ↪ different names."
  - "A fixture can verify the workflow but cannot validate the scientific
    ↪ claim."

tolerances:
  fixture_mode:
    required: "Script completes, figures are generated, and voltage-feature
    ↪ model beats capacity baseline."
  full_data_mode:
    required: "Qualitative agreement with Figure 2 and documented error
    ↪ metric; exact match not required without identical split and
    ↪ preprocessing."

outputs:
  - "figures/severson_feature_scatter.png"
  - "figures/severson_prediction_comparison.png"
  - "data/processed/feature_table.csv"
  - "reports/reproduction-summary.md"

```

Now create `src/read_checklist.py`:

```

from pathlib import Path

import yaml

def main() -> None:
    checklist_path = Path("references/reproduction-checklist.yaml")
    checklist = yaml.safe_load(checklist_path.read_text(encoding="utf-8"))

    print("Target paper:", checklist["project"]["target_paper"])
    print("Target result:", checklist["project"]["target_result"])
    print()
    print("Known ambiguities:")
    for item in checklist["known_ambiguities"]:
        print(f"- {item}")
    print()

```

```

print("Expected outputs:")
for item in checklist["outputs"]:
    print(f"- {item}")

if __name__ == "__main__":
    main()

```

Run:

```
python src/read_checklist.py
```

Expected output:

Target paper: Severson et al. 2019 Nature Energy

Target result: Figure 2-style early voltage-curve feature prediction

Known ambiguities:

- Exact train/test split may differ from the paper unless the original split
↳ is recovered.
- The public repository includes data processing code, while full modeling
↳ code availability is limited.
- Different public file formats may store cycle dictionaries with slightly
↳ different names.
- A fixture can verify the workflow but cannot validate the scientific claim.

Expected outputs:

- figures/severson_feature_scatter.png
- figures/severson_prediction_comparison.png
- data/processed/feature_table.csv
- reports/reproduction-summary.md

The checklist separates the paper, the data, the claim, the ambiguities, and the outputs. This separation matters. A common beginner mistake is to write “reproduce Severson paper” as if that were a single task. It is too large. We are reproducing one result class: early-cycle voltage features outperform early capacity features for cycle-life prediction.

The `reproduction_type` field says `partial computational reproduction`. That is honest. We are not reproducing the electrochemical experiments that generated the cells. We are not reproducing every model variant in the paper. We are reproducing a computational claim using public or schema-compatible data.

12.3.3 What could go wrong

If `yaml.safe_load` fails with a parser error, check indentation. YAML is whitespace-sensitive. Use spaces, not tabs.

If the checklist file is not found, run the command from `SimulationCompanion/chapter12_reproduction_project`, not from the repository root.

If the checklist feels too bureaucratic, remember that future-you is a collaborator. Future-you will not remember why a deviation was acceptable unless present-you writes it down.

12.3.4 Reflection

This exercise taught you to define scope before writing code. In publishable simulation work, that habit prevents two bad outcomes: claiming more than you reproduced, and abandoning a useful partial reproduction because it was not an impossible perfect one.

Guided Walkthrough 3

Create a Research Log and Deviation Register

Learning objective: Record reproduction decisions in a way that can become a methods paragraph or supplementary note.

A research log is not a diary. It is a low-friction record of decisions that affect interpretation. The most important entries are often small: “Used deterministic split because original split was not recovered,” or “Fixture generated with voltage-feature correlation so pipeline could be tested before downloading data.” These are the entries that save you when a reviewer asks what changed.

Append this to `logs/research-log.md`:

```
# Research Log
```

```
## 2026-05-01
```

Severson, K. A., Attia, P. M., Jin, N., et al. "Data-driven prediction of
↳ battery cycle life before capacity degradation." Nature Energy 4, 383-391
↳ (2019). <https://doi.org/10.1038/s41560-019-0356-8>

Target result:

Reproduce a Figure 2-style result showing that early-cycle voltage-curve
↳ features are more predictive of cycle life than early discharge capacity
↳ alone.

Planned deviations:

1. Use a deterministic schema-compatible fixture when the full MATR dataset
↳ is not present.
2. Use a transparent ridge regression implemented in NumPy rather than
↳ attempting to exactly match every model variant in the paper.
3. Treat the reproduction as partial unless the full public data and original
↳ split are recovered.

1. In fixture mode, the voltage-feature model must produce lower test mean
↳ absolute percentage error than the capacity-baseline model.
2. In full-data mode, the output should qualitatively match the paper's
↳ Figure 2 claim: early voltage-curve features contain predictive
↳ information not visible in early capacity alone.
- 3.

Now create `src/register_deviation.py`:

```
from __future__ import annotations

from datetime import date
from pathlib import Path

def append_deviation(title: str, reason: str, consequence: str) -> None:
    log_path = Path("logs/research-log.md")
    entry = (
        f"\n## {date.today().isoformat()} - Deviation: {title}\n\n"
        f"Reason:\n\n{reason}\n\n"
        f"Consequence:\n\n{consequence}\n"
    )
    with log_path.open("a", encoding="utf-8") as handle:
        handle.write(entry)
```

```
def main() -> None:
    append_deviation(
        title="Fixture mode allowed before full MATR download",
        reason=(
            "The public MATR data can be large and may require manual
            ↪ download. "
            "The chapter must remain runnable on a fresh laptop without
            ↪ network access."
        ),
        consequence=(
            "Fixture-mode figures validate the software workflow only. They
            ↪ must not "
            "be described as reproducing the scientific result from the
            ↪ paper."
        ),
    )
    print("Deviation recorded in logs/research-log.md")

if __name__ == "__main__":
    main()
```

Run:

```
python src/register_deviation.py
```

Expected output:

```
Deviation recorded in logs/research-log.md
```

Open `logs/research-log.md`. You should see a new dated deviation section. The exact date will be the date on your machine.

The code uses `date.today().isoformat()` so the entry is machine-generated and unambiguous. The `append_deviation` function asks for three fields: title, reason, and consequence. That structure is enough for most reproduction choices. Do not write “changed preprocessing” as a title and move on. A deviation without consequence is not useful.

12.3.5What could go wrong

If the date differs from the chapter date, that is fine. Use your actual date. Reproduction logs should reflect reality.

If the script appends the same deviation twice, leave it for now or manually remove the duplicate before committing. Later, in a real repository, you would avoid repeated log entries by making the register a table with unique identifiers.

If you dislike generated log entries, you may write them manually. The important rule is that the log exists and is version-controlled.

12.3.6Reflection

This exercise taught you to treat deviations as first-class research objects. In your own sodium-ion paper, this habit will help you distinguish a justified adaptation from a hidden change that undermines the claim.

Dataset Integration

The MATR Battery Cycle-Life Dataset

The guided reproduction targets the dataset used by Severson et al. The paper reports a dataset of 124 commercial lithium iron phosphate/graphite cells cycled under fast-charging conditions, with cycle lives spanning roughly 150 to 2300 cycles. The Nature Energy article states that the datasets are available at <https://data.matr.io/1/>, and the public Braatz group repository provides data-processing code and loading examples.

Download location:

| Source | URL | Notes |
|-----------------------|---|---|
| Paper | https://doi.org/10.1038/s41560-019-0356-8 | Severson et al. 2019, figure reference |
| Dataset portal | https://data.matr.io/ | Public data portal for the MATR battery dataset |
| Processing repository | https://github.com/rdbraatz/data-driven-prediction | Publicly available data processing code |

The full data may appear as MATLAB `.mat` files, Python pickle files, or processed batch files depending on the portal mirror and the specific download. In the public processing repository, each cell is described by descriptors, per-cycle summary data, and within-cycle data. The summary fields include quantities such as cycle number, discharge capacity, charge capacity, internal resistance, temperature summaries, and charge time. The cycle data include time, current, voltage, temperature, charge capacity, discharge capacity, and derived voltage-curve arrays such as interpolated discharge capacity.

The sign convention is a pitfall. The public data are already processed for the paper’s workflow, so in this chapter we do not reinterpret current signs for electrochemical simulation. We work primarily with discharge voltage-capacity curves and cycle-life labels. If you later feed these data into an ECM or PyBaMM workflow, return to the current-sign normalization rules from Lab Chapter 11.

The second pitfall is that “cycle 10” and “cycle 100” mean indexed cycles inside the processed dataset, not necessarily the tenth and hundredth rows after arbitrary filtering. The script below uses the cycle dictionaries by key when available. In fixture mode, we generate cycles with explicit integer labels so the same feature code is exercised.

The third pitfall is that the original paper’s full modeling choices are not all contained in the public processing repository. This is normal in reproduction work. We therefore define our target carefully: reproduce the feature logic and a Figure 2-style relationship, not every model variant from the article.

Guided Walkthrough 4

Reproduce the Voltage-Feature Workflow

Learning objective: Build a complete, runnable reproduction script that extracts early-cycle voltage features, trains transparent baseline models, and writes figures and metrics.

The key idea from the target paper is that changes in discharge voltage curves over early cycles contain information about later cycle life before capacity fade is obvious. A simple version of that idea compares the discharge curve at cycle 100 with the discharge curve at cycle 10 on a common capacity grid:

$$\Delta Q_{100-10}(V) = Q_{100}(V) - Q_{10}(V). \quad (2)$$

The paper uses features derived from this difference curve. We will compute three transparent features: the variance, minimum, and mean of $\Delta Q_{100-10}(V)$. We will compare those voltage-curve features with a baseline that uses early discharge capacity. We will train ridge regression on log cycle life:

$$\hat{\theta} = (X^T X + \lambda I)^{-1} X^T y, \quad (3)$$

where X is the standardized feature matrix and $y = \log_{10}(\text{cycle life})$. The intercept is not regularized. This is not a claim that ridge regression is the exact model used in every part of the paper. It is a transparent model that lets us test the core feature claim with auditable code.

Create `src/reproduce_severson_voltage_feature.py`:

```
from __future__ import annotations

import math
import pickle
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

RAW_DATA_DIR = Path("data/raw")
PROCESSED_DIR = Path("data/processed")
FIGURE_DIR = Path("figures")
REPORT_DIR = Path("reports")

@dataclass(frozen=True)
class CellRecord:
    cell_id: str
    cycle_life: float
    qd_cycle_10: np.ndarray
    qd_cycle_100: np.ndarray
    early_capacity_ah: float
```

```

source: str

def ensure_directories() -> None:
    for directory in [RAW_DATA_DIR, PROCESSED_DIR, FIGURE_DIR, REPORT_DIR]:
        directory.mkdir(parents=True, exist_ok=True)

def make_voltage_grid() -> np.ndarray:
    return np.linspace(3.6, 2.0, 1000)

def synthesize_fixture(n_cells: int = 124, seed: int = 12) ->
    list[CellRecord]:
    =
        =
        list
        =

    for i in range(n_cells):
        cell_id = f"fixture_cell_{i:03d}"
        base_capacity = 3.05 + 0.22 * np.random.random() + 0.05
        = float 10**

        early_capacity = base_capacity + 0.004 * latent_health +
        rng.normal(0.0, 0.006)

        =
        max - min /

        = 0.08 * 2 * *
        =
        * 1.0 -
    +
    + 0.0 0.0015 =

        = -0.010 *
        = - 3.25 / 0.18 ** 2
        = 0.35 + 0.65 *
    =
        *
    =
    +
    +
    0.0 0.0015 =

```

```

        records.append(
            CellRecord(
                cell_id=cell_id,
                cycle_life=cycle_life,
                qd_cycle_10=qd_cycle_10,
                qd_cycle_100=qd_cycle_100,
                early_capacity_ah=float(early_capacity),
                source="fixture",
            )
        )

    return records

def find_pickle_files() -> list[Path]:
    return sorted(RAW_DATA_DIR.glob("*.pkl")) +
        ↪ sorted(RAW_DATA_DIR.glob("*.pickle"))

def first_existing_key(mapping: dict[str, Any], candidates: list[str]) -> str
    ↪ | None:
    for candidate in candidates:
        if candidate in mapping:
            return candidate
    return None

def extract_cycle_life(cell: dict[str, Any]) -> float | None:
    for key in ["cycle_life", "cycleLife", "Cycle life", "cycle life"]:
        if key in cell:
            return float(np.asarray(cell[key]).squeeze())
    return None

def extract_summary_capacity(cell: dict[str, Any]) -> float | None:
    summary_key = first_existing_key(cell, ["summary", "summaries"])
    if summary_key is None:
        return None
    summary = cell[summary_key]
    if not isinstance(summary, dict):
        return None
    capacity_key = first_existing_key(
        summary,
        ["QD", "QDischarge", "discharge_capacity", "Discharge Capacity
    ↪ (Ah)"],

```

```

)
if capacity_key is None:
    return None
capacity = np.asarray(summary[capacity_key], dtype=float).ravel()
if capacity.size < 10:
    return None
return float(capacity[min(9, capacity.size - 1)])

def extract_qd_curve(cycle: dict[str, Any]) -> np.ndarray | None:
    for key in ["Qdlin", "Qd", "QD", "discharge_capacity"]:
        if key in cycle:
            curve = np.asarray(cycle[key], dtype=float).ravel()
            if curve.size >= 50:
                return curve
    return None

def load_records_from_pickle(path: Path) -> list[CellRecord]:
    with path.open("rb") as handle:
        payload = pickle.load(handle)

    if isinstance(payload, dict) and "batch" in payload:
        payload = payload["batch"]

    if not isinstance(payload, dict):
        return []

    records: list[CellRecord] = []
    for cell_id, cell in payload.items():
        if not isinstance(cell, dict):
            continue

        cycle_life = extract_cycle_life(cell)
        if cycle_life is None or not np.isfinite(cycle_life):
            continue

        cycles_key = first_existing_key(cell, ["cycles", "cycle"])
        if cycles_key is None or not isinstance(cell[cycles_key], dict):
            continue

        cycles = cell[cycles_key]
        cycle_10 = cycles.get("10", cycles.get(10))
        cycle_100 = cycles.get("100", cycles.get(100))
        if not isinstance(cycle_10, dict) or not isinstance(cycle_100, dict):

```

```

        continue

    qd_cycle_10 = extract_qd_curve(cycle_10)
    qd_cycle_100 = extract_qd_curve(cycle_100)
    if qd_cycle_10 is None or qd_cycle_100 is None:
        continue

    min_size = min(qd_cycle_10.size, qd_cycle_100.size)
    if min_size < 50:
        continue

    early_capacity = extract_summary_capacity(cell)
    if early_capacity is None:
        early_capacity = float(np.nanmax(qd_cycle_10))

    records.append(
        CellRecord(
            cell_id=str(cell_id),
            cycle_life=float(cycle_life),
            qd_cycle_10=qd_cycle_10[:min_size],
            qd_cycle_100=qd_cycle_100[:min_size],
            early_capacity_ah=float(early_capacity),
            source=path.name,
        )
    )

return records

def load_records() -> list[CellRecord]:
    pickle_files = find_pickle_files()
    records: list[CellRecord] = []
    for path in pickle_files:
        records.extend(load_records_from_pickle(path))

    if records:
        print(f"Loaded {len(records)} cell records from {len(pickle_files)}
        ↪ pickle file(s).")
        return records

    print("No compatible MATR pickle files found in data/raw.")
    print("Using deterministic fixture data for workflow verification.")
    return synthesize_fixture()

```

```

def resample_curve(curve: np.ndarray, target_size: int = 1000) -> np.ndarray:
    source_axis = np.linspace(0.0, 1.0, curve.size)
    target_axis = np.linspace(0.0, 1.0, target_size)
    return np.interp(target_axis, source_axis, curve)

def build_feature_table(records: list[CellRecord]) -> pd.DataFrame:
    rows: list[dict[str, float | str]] = []
    for record in records:
        q10 = resample_curve(record.qd_cycle_10)
        q100 = resample_curve(record.qd_cycle_100)
        delta_q = q100 - q10
        rows.append(
            {
                "cell_id": record.cell_id,
                "cycle_life": record.cycle_life,
                "log10_cycle_life": math.log10(record.cycle_life),
                "early_capacity_ah": record.early_capacity_ah,
                "delta_q_mean": float(np.mean(delta_q)),
                "delta_q_min": float(np.min(delta_q)),
                "delta_q_variance": float(np.var(delta_q)),
                "source": record.source,
            }
        )
    frame = pd.DataFrame(rows)
    frame = frame.replace([np.inf, -np.inf], np.nan).dropna()
    return frame

def deterministic_split(frame: pd.DataFrame, test_fraction: float = 0.25) ->
    ↪ tuple[np.ndarray, np.ndarray]:
    ordered = frame.sort_values("cell_id").reset_index(drop=True)
    indices = np.arange(len(ordered))
    test_mask = indices % int(round(1.0 / test_fraction)) == 0
    train_indices = ordered.index[~test_mask].to_numpy()
    test_indices = ordered.index[test_mask].to_numpy()
    return train_indices, test_indices

@dataclass(frozen=True)
class Standardizer:
    mean: np.ndarray
    scale: np.ndarray

    def transform(self, values: np.ndarray) -> np.ndarray:

```

```

        return (values - self.mean) / self.scale

def fit_standardizer(values: np.ndarray) -> Standardizer:
    mean = values.mean(axis=0)
    scale = values.std(axis=0)
    scale = np.where(scale == 0.0, 1.0, scale)
    return Standardizer(mean=mean, scale=scale)

def fit_ridge_regression(features: np.ndarray, target: np.ndarray,
    ↪ regularization: float = 1.0) -> np.ndarray:
    design = np.column_stack([np.ones(features.shape[0]), features])
    penalty = np.eye(design.shape[1]) * regularization
    penalty[0, 0] = 0.0
    coefficients = np.linalg.solve(design.T @ design + penalty, design.T @
    ↪ target)
    return coefficients

def predict_ridge(features: np.ndarray, coefficients: np.ndarray) ->
    ↪ np.ndarray:
    design = np.column_stack([np.ones(features.shape[0]), features])
    return design @ coefficients

def mean_absolute_percentage_error(observed: np.ndarray, predicted:
    ↪ np.ndarray) -> float:
    return float(np.mean(np.abs((observed - predicted) / observed)) * 100.0)

def train_and_evaluate(
    frame: pd.DataFrame,
    feature_columns: list[str],
    model_name: str,
) -> dict[str, float | str | np.ndarray]:
    train_indices, test_indices = deterministic_split(frame)
    x = frame[feature_columns].to_numpy(dtype=float)
    y = frame["log10_cycle_life"].to_numpy(dtype=float)

    standardizer = fit_standardizer(x[train_indices])
    x_train = standardizer.transform(x[train_indices])
    x_test = standardizer.transform(x[test_indices])

    coefficients = fit_ridge_regression(x_train, y[train_indices],
    ↪ regularization=1.0)

```



```

predicted_log = predict_ridge(x_test, coefficients)

observed_cycles = 10 ** y[test_indices]
predicted_cycles = 10 ** predicted_log
mape = mean_absolute_percentage_error(observed_cycles, predicted_cycles)

return {
    "model_name": model_name,
    "feature_columns": ", ".join(feature_columns),
    "test_mape_percent": mape,
    "observed_cycles": observed_cycles,
    "predicted_cycles": predicted_cycles,
    "test_cell_ids": frame.iloc[test_indices]["cell_id"].to_numpy(),
}

def plot_feature_scatter(frame: pd.DataFrame) -> None:
    figure, axes = plt.subplots(1, 2, figsize=(11.0, 4.5),
    ↪ constrained_layout=True)

    scatter0 = axes[0].scatter(
        frame["early_capacity_ah"],
        frame["cycle_life"],
        c=frame["cycle_life"],
        cmap="viridis",
        s=36,
        edgecolor="black",
        linewidth=0.3,
    )
    axes[0].set_xlabel("Early discharge capacity (Ah)")
    axes[0].set_ylabel("Cycle life (cycles)")
    axes[0].set_title("Capacity baseline")
    axes[0].grid(True, alpha=0.3)

    axes[1].scatter(
        frame["delta_q_variance"],
        frame["cycle_life"],
        c=frame["cycle_life"],
        cmap="viridis",
        s=36,
        edgecolor="black",
        linewidth=0.3,
    )
    axes[1].set_xlabel(r"Variance of  $\Delta Q_{100-10}(V)$ ")
    axes[1].set_ylabel("Cycle life (cycles)")

```

```

axes[1].set_title("Voltage-curve feature")
axes[1].grid(True, alpha=0.3)

colorbar = figure.colorbar(scatter0, ax=axes, shrink=0.9)
colorbar.set_label("Cycle life (cycles)")
figure.savefig(Figure_DIR / "seversion_feature_scatter.png", dpi=200)
plt.close(figure)

def plot_prediction_comparison(results: list[dict[str, float | str |
↳ np.ndarray]]) -> None:
    figure, axes = plt.subplots(1, len(results), figsize=(11.0, 4.5),
↳ constrained_layout=True)
    if len(results) == 1:
        axes = [axes]

    for axis, result in zip(axes, results):
        observed = np.asarray(result["observed_cycles"], dtype=float)
        predicted = np.asarray(result["predicted_cycles"], dtype=float)
        axis.scatter(observed, predicted, s=42, edgecolor="black",
↳ linewidth=0.3)
        lower = min(observed.min(), predicted.min())
        upper = max(observed.max(), predicted.max())
        axis.plot([lower, upper], [lower, upper], color="black",
↳ linestyle="--", linewidth=1.2)
        axis.set_xscale("log")
        axis.set_yscale("log")
        axis.set_xlabel("Observed cycle life")
        axis.set_ylabel("Predicted cycle life")
        axis.set_title(f"{result['model_name']}\nMAPE =
↳ {result['test_mape_percent']:.1f}%")
        axis.grid(True, which="both", alpha=0.3)

    figure.savefig(Figure_DIR / "seversion_prediction_comparison.png",
↳ dpi=200)
    plt.close(figure)

def write_summary(frame: pd.DataFrame, results: list[dict[str, float | str |
↳ np.ndarray]]) -> None:
    lines = [
        "# Reproduction Summary",
        "",
        "Target paper: Seversion et al. (2019), Nature Energy.",
        "",

```

```

        f"Number of cells used: {len(frame)}",
        f"Data source mode: {' ', ' '.join(sorted(frame['source'].unique()))}",
        """
    """
    """## Model Comparison",
    """
    "| Model | Features | Test MAPE (%) |",
    "| --- | --- | ---: |",
]

for result in results:
    lines.append(
        f"| {result['model_name']} | {result['feature_columns']} | "
        f"{result['test_mape_percent']:.2f} | "
    )

    voltage_result = next(item for item in results if item["model_name"] ==
↪ "voltage features")
    capacity_result = next(item for item in results if item["model_name"] ==
↪ "capacity baseline")
    lines.extend(
        [
            """
            """## Interpretation",
            """
        ]
    )

    if float(voltage_result["test_mape_percent"]) <
↪ float(capacity_result["test_mape_percent"]):
        lines.append(
            "The voltage-feature model outperformed the early-capacity
            ↪ baseline in this run. "
            "In fixture mode, this verifies that the workflow can recover the
            ↪ planted signal. "
            "In full-data mode, this supports the qualitative Figure 2-style
            ↪ claim."
        )
    else:
        lines.append(
            "The voltage-feature model did not outperform the capacity
            ↪ baseline in this run. "
            "Inspect the data source, cycle extraction, train-test split, and
            ↪ feature definitions "
            "before drawing a scientific conclusion."
        )

```

```

lines.extend(
    [
        """
        """
        """## Deviations",
        """
        """
        """- This script uses transparent ridge regression rather than
        ↪ attempting to reproduce every model variant in the paper.",
        """- If no compatible public data file is present, the script uses
        ↪ a deterministic fixture for workflow verification only.",
    ]
)

(REPORT_DIR / "reproduction-summary.md").write_text("\n".join(lines),
↪ encoding="utf-8")

def main() -> None:
    ensure_directories()
    records = load_records()
    feature_table = build_feature_table(records)

    if len(feature_table) < 20:
        raise RuntimeError("Need at least 20 valid cell records for a
        ↪ meaningful split.")

    feature_table.to_csv(PROCESSED_DIR / "feature_table.csv", index=False)

    capacity_result = train_and_evaluate(
        feature_table,
        feature_columns=["early_capacity_ah"],
        model_name="capacity baseline",
    )

    voltage_result = train_and_evaluate(
        feature_table,
        feature_columns=["delta_q_mean", "delta_q_min", "delta_q_variance"],
        model_name="voltage features",
    )

    results = [capacity_result, voltage_result]

    plot_feature_scatter(feature_table)
    plot_prediction_comparison(results)
    write_summary(feature_table, results)

    print("Reproduction run complete.")
    print(f"Cells used: {len(feature_table)}")

```

```

for result in results:
    print(
        f"{result['model_name']}: "
        f"test MAPE = {result['test_mape_percent']:.2f}%"
    )
    print("Wrote data/processed/feature_table.csv")
    print("Wrote figures/seversion_feature_scatter.png")
    print("Wrote figures/seversion_prediction_comparison.png")
    print("Wrote reports/reproduction-summary.md")

if __name__ == "__main__":
    main()

```

Run:

```
python src/reproduce_seversion_voltage_feature.py
```

Expected terminal output in fixture mode:

```

No compatible MATR pickle files found in data/raw.
Using deterministic fixture data for workflow verification.
Reproduction run complete.
Cells used: 124
capacity baseline: test MAPE = xx.xx%
voltage features: test MAPE = yy.yy%
Wrote data/processed/feature_table.csv
Wrote figures/seversion_feature_scatter.png
Wrote figures/seversion_prediction_comparison.png
Wrote reports/reproduction-summary.md

```

The exact MAPE values may differ slightly across NumPy versions, but the voltage-feature model should beat the capacity baseline in fixture mode. That is because the fixture deliberately plants a voltage-curve signal correlated with cycle life while making early capacity only weakly informative. Again: fixture mode verifies the workflow, not the paper.

Open `figures/seversion_feature_scatter.png`. The figure has two panels. The left panel plots early discharge capacity on the x-axis and cycle life on the y-axis. In fixture mode, the points should look like a broad cloud with only a weak trend. The right panel plots the variance of $\Delta Q_{100-10}(V)$ on the x-axis and cycle life on the y-axis. You should see a clearer monotonic relationship. The color scale also encodes cycle life, so a correct plot has a smoother color progression in the

voltage-feature panel than in the capacity panel.

Open `figures/severson_prediction_comparison.png`. This figure also has two panels. Each panel plots observed cycle life on the x-axis and predicted cycle life on the y-axis using log-log axes. The dashed diagonal line is perfect prediction. The capacity-baseline panel should show wider scatter away from the diagonal. The voltage-feature panel should cluster more tightly around the diagonal. A wrong result would show the voltage-feature panel as no better than capacity in fixture mode; that usually means the feature calculation was changed or the target was shuffled.

Open `data/processed/feature_table.csv`. You should see one row per cell with columns for cycle life, early capacity, and the three voltage features. Open `reports/reproduction-summary.md`. It should contain a small table comparing the two models and a short interpretation.

The code deserves careful reading. `CellRecord` defines the minimum object needed for this reproduction: cell ID, cycle life, discharge-capacity curves at cycles 10 and 100, early capacity, and source. `synthesize_fixture` builds a deterministic surrogate dataset with the same shape as the real workflow. It uses a latent health variable to generate cycle life and to perturb the difference between cycle-10 and cycle-100 voltage-capacity curves. The fixture is realistic enough to exercise the code, but it is not a substitute for the MATR data.

`load_records_from_pickle` is intentionally defensive. Public research files drift. Some dictionaries use string keys, others use integer keys. Some summary fields are named QD; others have longer names. The loader searches candidate keys and skips cells that do not contain the required fields. This is standard practice when reproducing from public processed data.

`build_feature_table` computes Equation (2) after resampling both curves to a common length. The paper's actual workflow uses linearly interpolated discharge curves, so this resampling step is conceptually aligned even though our implementation is simplified. `train_and_evaluate` uses a deterministic cell-ID split so the result can be rerun without randomness. That split is one of our documented deviations.

The ridge implementation is deliberately visible. `fit_standardizer` computes training-set mean and standard deviation. `fit_ridge_regression` builds the design matrix, adds an intercept, and solves Equation (3). The target is log cycle

life because battery lifetime errors are naturally relative; predicting 200 instead of 300 cycles is not the same practical error as predicting 2000 instead of 2100 cycles.

12.3.7 What could go wrong

If the script says fewer than 20 valid records were found after you download data, inspect the data structure with a small exploratory script. The public file may not be the same pickle layout expected by this minimal loader. Use Lab Chapter 11's parser patterns to adapt the key names.

If the voltage-feature model does not beat the capacity baseline in fixture mode, check that `delta_q = q100 - q10` was not reversed and that `cycle_life` was not shuffled. The planted signal is mild but real.

If the figure files are blank, confirm that Matplotlib installed correctly and that the script ran from the chapter project root. The script writes relative paths such as `figures/...`

If full-data mode gives worse results than expected, do not immediately change the model until it looks good. First document the result, then test specific hypotheses: cycle indexing, train-test split, feature definitions, outlier cells, and whether the loaded batch matches the paper's analyzed set.

12.3.8 Reflection

This exercise taught the central move of computational reproduction: make a claim operational. "Voltage curves predict cycle life" became a feature table, a model, a metric, and two figures. The simplifications are visible, which means they can be criticized and improved.

Guided Walkthrough 5

Add Tests So the Reproduction Does Not Rot

Learning objective: Write small tests that protect the feature calculation and regression workflow from accidental changes.

A reproduction repository is not finished when the script runs once. It is finished when you can change a parser, rerun the tests, and know whether the core assumptions survived. We will add a small `pytest` suite.

Create `tests/test_reproduction_workflow.py`:

```

import
from      import

import      as

    =      __file__      1
    =      / "src"
    0 str

from      import      # noqa: E402

def      -> None
    =      =30      =12
    assert len      == 30
    assert all      == 1000 for      in
    assert all      == 1000 for      in

def      -> None
    =      =30      =12
    =
    assert len      == 30
    assert "delta_q_variance" in
    assert      "delta_q_variance"      all
    assert      "cycle_life" > 0 all

def      -> None
    =      =124      =12
    =
    =
    = "early_capacity_ah"
    ="capacity baseline"
    =
    = "delta_q_mean" "delta_q_min" "delta_q_variance"
    ="voltage features"

```



```

    )
    assert voltage_result["test_mape_percent"] <
        ↳ capacity_result["test_mape_percent"]

def                                     -> None
    =          100.0  200.0  300.0
    =          100.0  200.0  300.0
    assert                                     == 0.0

```

Run:

```
pytest -q
```

Expected output:

```

....
↳ [100%]
4 passed in x.xx s

```

The first test confirms that the fixture produces the requested number of records and that the two curves have the expected length. The second test confirms that feature extraction creates finite voltage features and positive cycle lives. The third test protects the teaching purpose of the fixture: voltage features should beat capacity in this controlled surrogate. The fourth test checks the metric itself.

These tests are not a proof that the paper is reproduced. They are guardrails around the code. If a future edit reverses the sign of ΔQ , drops the wrong rows, or changes the deterministic fixture, the tests will make the change visible.

12.3.9 What could go wrong

If `pytest` cannot import `reproduce_severson_voltage_feature`, confirm the test file is inside `tests` and that you are running from the project root. The test inserts `src` into `sys.path`.

If the voltage-feature test fails by a tiny amount after you edit the fixture, inspect the generated scatter plot. You may have weakened the planted signal. Decide whether that was intentional and document it.

If tests pass but full-data reproduction fails, remember what the tests cover. They protect code mechanics, not the truth of external data assumptions.

12.3.1 Reflection

This exercise taught you to test a research workflow without pretending that tests answer scientific questions. Good tests keep the machinery honest so your scientific attention can go to the assumptions.

Guided Walkthrough 6

Write the Reproduction Report

Learning objective: Turn code outputs, deviations, and figures into a concise reproduction write-up.

The final report should be short enough that someone will read it and specific enough that someone can criticize it. It should not sound like a victory lap. It should sound like a careful account of what was attempted, what was reproduced, what differed, and what remains unresolved.

Create `reports/final-reproduction-report.md`:

```
# Reproduction Report: Severson et al. 2019 Voltage-Feature Workflow

## Target

This project partially reproduces a central computational claim from Severson
↳ et al. (2019): early-cycle discharge voltage-curve features contain
↳ predictive information about battery cycle life before obvious capacity
↳ degradation appears.

The target is a Figure 2-style result, not a full reproduction of every model
↳ in the paper.

## Sources

-
-
-

## Computational Environment

- Python: record output from `python --version`
- NumPy: record output from `python src/verify_environment.py`
- pandas: record output from `python src/verify_environment.py`
```

- Matplotlib: record output from ``python src/verify_environment.py``

Method

For each cell, the workflow extracts discharge-capacity curves at cycle 10
→ and cycle 100. Both curves are resampled onto a common normalized grid.
→ The difference curve is computed as:

\$\$
\\Delta Q_{\\{100-10\\}}(V) = Q_{\\{100\\}}(V) - Q_{\\{10\\}}(V).
\$\$

Three voltage-curve features are computed from this difference curve: mean,
→ minimum, and variance. A capacity baseline uses early discharge capacity
→ only. Both feature sets are used to fit ridge regression models to log10
→ cycle life with a deterministic cell-ID split.

Outputs

- ``data/processed/feature_table.csv``
- ``figures/severson_feature_scatter.png``
- ``figures/severson_prediction_comparison.png``
- ``reports/reproduction-summary.md``

Result

Paste the model-comparison table from ``reports/reproduction-summary.md`` here.

In fixture mode, the voltage-feature model should outperform the capacity
→ baseline. This verifies that the code path can recover a known
→ voltage-feature signal. In full-data mode, the same comparison should be
→ interpreted as a partial reproduction of the paper's Figure 2-style
→ claim.

Deviations from the Paper

1. The script uses transparent ridge regression instead of attempting to
→ reproduce every proprietary or unavailable model variant.
2. The train-test split is deterministic by cell ID unless the original split
→ is recovered.
3. A deterministic fixture is used when full MATR files are absent.
4. Feature extraction is simplified to three summary statistics of the
→ cycle-100 minus cycle-10 difference curve.

Reproduction Status

Status: partial reproduction.

The workflow is reproducible as code. Scientific agreement with the paper

- ↳ requires running against the full public MATR dataset and checking the
- ↳ resulting figures and metrics against the published Figure 2 claim.

Lessons Learned

Write three paragraphs:

1. What was easy to reproduce?
2. What depended on undocumented or ambiguous choices?
3. How would you design your own sodium-ion study to be easier to reproduce?

This report is a template, but it is not a placeholder in the lazy sense. Every section has a job. The target prevents scope drift. The sources make provenance explicit. The method describes the pipeline. The deviations protect honesty. The status prevents overclaiming.

Now update the report after you run the script. Paste the model-comparison table from `reports/reproduction-summary.md`. Add the actual Python version. Add a paragraph describing whether you used fixture mode or full-data mode.

12.3.1 What could go wrong

If the report becomes longer than the code, tighten it. A reproduction report is not a second paper.

If you feel embarrassed by the deviations, do not hide them. Deviation is normal. Hidden deviation is the problem.

If you used fixture mode, say so in the first page. Do not let a reader discover it in the code.

12.3.1 Reflection

This exercise taught you to write the kind of computational methods note that makes a reproduction useful. Your future sodium-ion papers should include this level of clarity from the beginning.

12.4 Open-Ended Exercises

12.4.1 Exercise 1: Replace the fixture with the full public MATR data

Download the public data from the MATR portal and place compatible `.pkl` files in `data/raw`. Run python `src/reproduce_severson_voltage_feature.py`. If the loader fails, inspect the file structure and adapt `load_records_from_pickle`.

Hints: start by printing `type(payload)` and the first three keys. Then inspect one cell dictionary. Look for descriptors, summary fields, and cycles. Do not edit raw files.

Worked solution outline: The correct solution is not one universal code block because public mirrors differ. The successful adaptation should still produce `CellRecord` objects with `cycle_life`, `qd_cycle_10`, `qd_cycle_100`, and `early_capacity_ah`. Add every key-name change to the research log.

12.4.2 Exercise 2: Reproduce a Marquis et al. SPMe comparison instead

Use PyBaMM to compare SPM, SPMe, and DFN voltage curves under the same discharge protocol. Choose one figure from Marquis et al. that compares model fidelity, then reproduce the qualitative ordering: DFN as reference, SPMe closer than SPM at higher C-rate.

Hints: start from Lab Chapter 3. Pin `pybamm==26.3.1`. Use `pybamm.lithium_ion.SPM()`, `pybamm.lithium_ion.SPMe()`, and `pybamm.lithium_ion.DFN()`. Keep the parameter set fixed.

Worked partial solution:

```
import matplotlib.pyplot as plt
import pybamm

parameter_values = pybamm.ParameterValues("Chen2020")
experiment = pybamm.Experiment(["Discharge at 2C until 2.8 V"], period="10
↪ seconds")

=
"SPM"
"SPMe"
"DFN"

=
```

```

for name, model in models.items():
    simulation = pybamm.Simulation(
        model,
        parameter_values=parameter_values,
        experiment=experiment,
    )
    solutions[name] = simulation.solve()

plt.figure(figsize=(7.0, 4.5))
for name, solution in solutions.items():
    time_min = solution.t / 60.0
    voltage = solution["Terminal voltage [V]"](solution.t)
    plt.plot(time_min, voltage, label=name)

plt.xlabel("Time (min)")
plt.ylabel("Terminal voltage (V)")
plt.title("Model-fidelity comparison under 2C discharge")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig("figures/marquis_style_model_comparison.png", dpi=200)
plt.show()

```

In a correct result, all three voltage curves begin near the same voltage. The SPM curve should deviate more strongly from the DFN curve as electrolyte effects matter, while the SPM_e curve should usually sit closer to the DFN reference. The exact shape depends on parameter set and protocol, so document your choices.

12.4.3 Exercise 3: Turn the reproduction into a sodium-ion adaptation

Find a public sodium-ion cycling dataset from Mendeley, Zenodo, or a paper supplement. Apply the same feature idea: compare early-cycle voltage-capacity curves and test whether difference-curve features correlate with later capacity retention.

Hints: keep the target modest. Use capacity retention at a fixed later cycle if full end-of-life is unavailable. Sodium-ion OCV curves can be flatter or more stepped depending on hard carbon and cathode chemistry, so inspect the voltage window before blindly reusing the LFP feature grid.

Worked solution outline: Your report should say “adaptation” rather than “reproduction.” Validate the code path on Severson-style data first, then apply it to sodium-ion. Compare the direction and stability of features, not only the

regression score.

12.4.4 Exercise 4: Perform a tolerance sensitivity study

Modify the reproduction script to test cycle pairs (20, 80), (10, 100), and (5, 50). Compare whether the voltage-feature model still beats capacity.

Hints: refactor `CellRecord` if you use real data with multiple cycle pairs. In fixture mode, generate additional cycle curves by extending the synthetic function.

Worked solution outline: A strong result should not depend completely on one arbitrary pair. If the conclusion changes sharply, that is not automatically bad, but it must be reported. In the Severson paper, the cycle-index dependence is itself an object of study.

Reproduction Exercise

Figure 2-Style Result from Severson et al.

Your required reproduction exercise for this Part is the guided Severson project above. The target paper is:

Severson, K. A., Attia, P. M., Jin, N., Perkins, N., Jiang, B., Yang, Z., Chen, M. H., Aykol, M., Herring, P. K., Fraggadakis, D., Bazant, M. Z., Harris, S. J., Chueh, W. C., and Braatz, R. D. “Data-driven prediction of battery cycle life before capacity degradation.” *Nature Energy* 4, 383-391 (2019). DOI: 10.1038/s41560-019-0356-8.

The specific target is a Figure 2-style result, not pixel-level reconstruction. Figure 2 in the paper emphasizes the predictive value of voltage-curve features from the first 100 cycles. Our reproduction computes $\Delta Q_{100-10}(V)$ features and compares them with early discharge capacity. The paper’s exact feature set and model variants are richer than this teaching implementation, so the reproduction tolerance is qualitative in fixture mode and semi-quantitative in full-data mode.

Close enough means the following. In fixture mode, the voltage-feature model must outperform the capacity baseline because that validates the code path against a known signal. In full-data mode, the feature scatter should show visibly more structure for voltage-curve features than for early capacity, and the prediction comparison should show lower relative error for voltage features under the documented split. If the exact MAPE does not match the paper, that is acceptable

only if you document differences in split, file version, feature definition, and model class.

Where the paper is ambiguous, choose the most conservative interpretation. Do not tune choices until the result looks like the paper. Make one defensible implementation, log it, and then run sensitivity checks. If a sensitivity check changes the conclusion, that becomes part of the reproduction result.

What Changes for Sodium-Ion?

The reproduction habit transfers directly to sodium-ion. The details do not.

Sodium-ion public datasets are sparser, and many papers still provide processed plots rather than raw cycling files. That changes the reproduction target. For a lithium-ion benchmark paper, you may reproduce a figure from raw public data. For a sodium-ion paper, you may need to reproduce a model structure, digitize a published curve with WebPlotDigitizer, or validate a method on lithium-ion data before adapting it to a small sodium-ion dataset.

The voltage-feature idea also needs chemistry awareness. LFP/graphite cells have their own voltage-curve structure. Hard-carbon sodium-ion anodes and Prussian white or layered-oxide cathodes can produce different plateau shapes, hysteresis, and sloping regions. A feature such as variance of $\Delta Q(V)$ may still be useful, but the voltage window and interpolation grid must follow the sodium-ion cell's actual operating range. Do not impose a 3.6 to 2.0 V grid on a sodium-ion dataset just because this chapter used it for an LFP-style workflow.

Cycle life labels may also differ. A sodium-ion study might report capacity retention after 100 cycles rather than end-of-life cycle count. That is not a defect; it is a different target variable. Your reproduction report should state whether you predicted cycle life, retained capacity, resistance growth, Coulombic efficiency trend, or another observable.

Finally, sodium-ion reproduction often has a stronger materials-methods component. If a result depends on electrolyte formulation, cathode water content, presodiation, electrode balancing, or formation protocol, a laptop-only reproduction cannot reproduce the experiment. It can reproduce the analysis. Say that plainly.

Chapter Summary and Skill Checklist

- You selected a reproduction target with a documented scorecard instead of intuition alone.
- You converted a paper claim into a checklist with sources, ambiguities, tolerances, and expected outputs.
- You created a research log and deviation register.
- You built a complete Figure 2-style reproduction workflow for the Severson et al. cycle-life paper.
- You implemented transparent feature extraction, ridge regression, plotting, testing, and reporting.
- You learned how to distinguish fixture-mode workflow verification from full-data scientific reproduction.
- You mapped the reproduction workflow onto sodium-ion adaptation constraints.

Commands, functions, and patterns that should now be in muscle memory:

- `python -m venv .venv-chapter12`
- `python -m pip install ...`
- `pytest -q`
- `dataclass` for structured paper and cell records
- `yaml.safe_load(...)` for machine-readable checklists
- `Path(...).mkdir(parents=True, exist_ok=True)` for reproducible folders
- `np.interp(...)` for curve resampling
- `np.linalg.solve(...)` for transparent ridge regression
- deterministic train-test splitting when original random splits are unavailable
- research-log entries for deviations and ambiguity

You should now be able to:

- Choose a reproduction paper that is feasible for your tools and time.
- Define the exact figure or claim you are reproducing.
- List missing information before it causes confusion.
- Build a small repository that separates raw data, processed data, source code, figures, reports, and logs.

- Reproduce a voltage-feature workflow from public or schema-compatible battery cycling data.
- Write a reproduction report that is honest about scope and deviations.
- Explain how a lithium-ion reproduction workflow can be adapted, carefully, to sodium-ion research.

If you cannot check every box, do not rush into Lab Chapter 13. Reproduction is the bridge from coursework to research. It is worth making this bridge sturdy.

Deliverable

Your deliverable is a GitHub-ready repository at `SimulationCompanion/chapter12_reproduction_project` containing:

- `README.md` describing the target paper, scope, and run command
- `references/reproduction-checklist.yaml`
- `logs/research-log.md`
- `src/paper_scorecard.py`
- `src/read_checklist.py`
- `src/register_deviation.py`
- `src/reproduce_severson_voltage_feature.py`
- `tests/test_reproduction_workflow.py`
- `data/processed/feature_table.csv` generated by the reproduction script
- `figures/severson_feature_scatter.png`
- `figures/severson_prediction_comparison.png`
- `reports/reproduction-summary.md`
- `reports/final-reproduction-report.md`

Approach the deliverable in three passes. First, make fixture mode run cleanly and commit it. Second, add full public data if you have time and adapt the loader without breaking the tests. Third, revise the final report so it clearly says what mode you used and what scientific conclusion is justified.

The worked partial solution is the code in this chapter. Your own final solution should add your machine's environment output, your actual research-log entries, and any loader changes needed for the data files you downloaded.

Further Practice and Reading

Severson et al. (2019), “Data-driven prediction of battery cycle life before capacity degradation,” *Nature Energy*, is the main target paper for this chapter. Bookmark the DOI page and the MATR data portal.

The Braatz group GitHub repository

`rdbraatz/data-driven-prediction-of-battery-cycle-life-before-capacity`

is worth reading because it shows the public processing structure and also teaches an important reproduction lesson: public processing code is not always the same as full modeling code.

Marquis et al. (2019), “An asymptotic derivation of a single particle model with electrolyte,” *Journal of The Electrochemical Society*, is the recommended physics-model reproduction alternative. It connects directly to PyBaMM’s SPMe implementation.

Sulzer et al. (2021), “Python Battery Mathematical Modelling (PyBaMM),” *Journal of Open Research Software*, is a useful software-paper reproduction target and a citation to include when PyBaMM is part of your method.

The official PyBaMM documentation at

<https://docs.pybamm.org/en/stable/> is the first place to check when a model reproduction fails because of version drift. For community help, use the PyBaMM Discourse linked from the PyBaMM website, and include a minimal runnable script when you ask a question.

Next chapter: Lab Chapter 13, “Specialization Tracks.”

Specialization Tracks

This chapter is the fork in the road.

In the first twelve lab chapters you built the common toolkit: reproducible environments, scientific Python, PyBaMM, parameter estimation, equivalent-circuit modeling, Kalman filtering, aging models, electrothermal coupling, public dataset handling, and full-paper reproduction practice. Those skills are the shared floor of modern battery simulation research. Publishable work, however, rarely stays on the shared floor. A strong project usually becomes strong because it goes deep in one direction: state estimation, safety, machine-learning health prediction, grid storage, vehicle integration, or fast-charging control.

Keep Textbook Chapters 7, 8, 9, 10, 11, 12, and 13 open as you work. Track A operationalizes the estimator design and observability ideas from Textbook Chapter 10. Track B extends the thermal and abuse-modeling ideas from Textbook Chapters 8 and 12. Track C turns the degradation observables from Textbook Chapter 7 into a data-driven health-estimation workflow. Track D connects the cell and pack models from Textbook Chapters 8 and 11 to grid storage studies. Track E connects BMS models to the vehicle-load picture you saw in the systems chapters. Track F turns the fast-charge limits from Textbook Chapters 8, 9, and 13 into an optimal-control problem. You do not need to complete all six tracks before the capstone. You should complete one track carefully, and optionally skim or run

another if your capstone proposal crosses boundaries.

The goal is not to make you an expert in six new fields in one chapter. That would be fake confidence, and fake confidence is expensive in research. The goal is to teach you how specialization feels when done properly: define the question, choose the right modeling fidelity, identify what data are available, build a minimal reproducible workflow, compare against a published reference, and write down what is still uncertain. Each track therefore has the same research-method skeleton even though the tools differ.

This chapter also changes the balance between guided work and independent work. Earlier chapters held your hand through nearly every line because the tools were new. Here, the walkthroughs are still complete and runnable, but they are written as launchpads. You will make design choices, record assumptions, and start turning a method into your own capstone. When you feel the work becoming less like a tutorial and more like a small research project, good. That is the point.

For sodium-ion batteries, specialization is especially important. Sodium-ion research has fewer public datasets, fewer validated parameter sets, and fewer mature tool examples than lithium-ion research. That does not make simulation-based SIB work impossible. It means your methodology must be explicit: validate the workflow on lithium-ion benchmark data when necessary, substitute sodium-ion OCV and transport assumptions carefully, and state which conclusions are chemistry-specific versus method-specific. The capstone in Chapter 14 will ask you to do exactly that.

13.1 Prerequisites Check

- Required Python software for all tracks: Python 3.11, numpy==1.26.4, scipy==1.13.1, pandas==2.2.2, matplotlib==3.9.0, scikit-learn==1.5.0, tqdm==4.66.5, and jupyterlab==4.2.1
- Required optional Python software: pybamm==26.3.1 for Track F, torch==2.3.1 for Track C, and cvxpy==1.5.2 for Track D if you want the convex dispatch extension
- Required MATLAB software: MATLAB R2024b recommended for Tracks D and E; Simulink and Simscape Electrical are helpful but not required because this chapter includes Python equivalents for the laptop-only path
- Commercial-tool note: COMSOL Battery Design Module is discussed in Track

B, but the runnable exercise uses an open Python finite-difference model so the chapter remains usable without a license

- Install command for the base environment: `python -m pip install numpy==1.26.4 scipy==1.13.1 pandas==2.2.2 matplotlib==3.9.0 scikit-learn==1.5.0 tqdm==4.66.5 jupyterlab==4.2.1`
- Install command for optional packages: `python -m pip install pybamm==26.3.1 cvxpy==1.5.2`
- CPU-only PyTorch install command: `python -m pip install torch==2.3.1 torchvision==0.18.1 torchaudio==2.3.1 --index-url https://download.pytorch.org/whl/cpu`
- Required textbook chapters: Textbook Chapters 7, 8, 9, 10, 11, 12, and 13
- Required prior lab chapters: Lab Chapters 1, 2, 7, 8, 9, 10, 11, and 12
- Estimated time: 10 to 16 hours for one track; 30 to 50 hours if you complete three tracks; much longer if you turn a track into the capstone

If your Kalman-filter algebra is shaky, revisit Lab Chapter 7 before Track A. If heat-generation signs and thermal boundary conditions feel slippery, reread Lab Chapter 9 before Track B. If you have not built the dataset loaders from Lab Chapter 11, do that before Track C. If optimization syntax is new, complete the `scipy.optimize` sections from Lab Chapter 2 before Track F.

13.2 Environment Setup

Create one shared environment and one chapter workspace. The workspace is deliberately organized by track so you can run only what you need.

```
cd /home/avirup/SodiumIonBatteryResearch
python3.11 -m venv .venv-chapter13
source .venv-chapter13/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==1.26.4 scipy==1.13.1 pandas==2.2.2
↳ matplotlib==3.9.0 scikit-learn==1.5.0 tqdm==4.66.5 jupyterlab==4.2.1
-m
python -m pip install torch==2.3.1 torchvision==0.18.1 torchaudio==2.3.1
↳ --index-url https://download.pytorch.org/whl/cpu
mkdir -p
cd
mkdir -p data figures results track_a_state_estimation track_b_thermal_safety
↳ track_c_health_ml track_d_grid_storage track_e_ev_powertrain
↳ track_f_fast_charging
```

On Windows PowerShell, activate the environment with:

```
.\.venv-chapter13\Scripts\Activate.ps1
```

Verify the base environment:

```
import importlib.util

import matplotlib
import numpy as np
import pandas as pd
import scipy
import sklearn

print("NumPy:", np.__version__)
print("SciPy:", scipy.__version__)
print("pandas:", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
print("scikit-learn:", sklearn.__version__)

for package_name in ["pybamm", "torch", "cvxpy"]:
    spec = importlib.util.find_spec(package_name)
    print(f"{package_name}: {'installed' if spec is not None else 'not  
↳ installed'}")
```

Expected output:

```
NumPy: 1.26.4
SciPy: 1.13.1
pandas: 2.2.2
Matplotlib: 3.9.0
scikit-learn: 1.5.0
pybamm: installed
torch: installed
cvxpy: installed
```

If PyTorch fails to install, remove Track C from your first pass and continue with another track. PyTorch CPU wheels are large, and network interruptions are common. If `cvxpy` fails because it cannot build a solver dependency, install it in a conda environment instead:

```
conda create -n chapter13 python=3.11 numpy scipy pandas matplotlib  
↳ scikit-learn tqdm jupyterlab cvxpy -c conda-forge  
conda activate chapter13  
python -m pip install pybamm==26.3.1
```

If PyBaMM fails during Track F, first verify that you can still import `numpy`, `scipy`, and `matplotlib`. Track F includes a fallback equivalent-circuit fast-charge optimizer, so a PyBaMM installation problem should not stop the whole chapter.

13.3 Conceptual Bridge: From General Competence to Specialization

The theory textbook gave you models as structured representations of physical ideas. The first twelve lab chapters gave you tools as structured ways to compute with those models. Specialization begins when the question becomes sharper than the general tool. “Model a battery” is not a research question. “How much does a flat hard-carbon OCV plateau degrade particle-filter SOC observability under current bias?” is a research question. “Can a simple early-cycle feature predict cycle life?” is a research question. “What charging protocol minimizes time while respecting plating and temperature constraints?” is a research question.

Each specialization track in this chapter asks you to make four choices.

First, choose the state variables. In Track A, the state may include SOC, RC polarization voltage, ohmic resistance, and capacity. In Track B, the state may be temperature at grid nodes plus reaction progress variables. In Track C, the “state” is not a physical state in the same sense; it is a learned hidden representation of voltage, capacity, and temperature history. In Track D, the state is energy stored in a grid asset. In Track E, the state includes vehicle speed, battery SOC, and sometimes motor or drivetrain dynamics. In Track F, the state is SOC, temperature, and possibly electrochemical concentration limits. If you cannot say what the state is, you cannot say what your model predicts.

Second, choose the observation. Most battery workflows observe terminal voltage, current, temperature, time, and sometimes capacity. That is sparse. The DFN contains electrolyte concentration and solid diffusion profiles, but a laptop dataset usually does not. Specialization often means learning how to infer hidden quantities from weak observations without pretending that the inference is stronger than it is.

Third, choose the constraint. Publishable simulation work is rarely about producing a curve in isolation. It is about a curve under constraints: voltage limits, temperature limits, computational budget, measurement noise, public data

availability, maximum charging time, grid power rating, drive-cycle feasibility, or safety thresholds. The constraint tells you what “better” means.

Fourth, choose the validation target. This is where many student projects become vague. Validation does not require perfect agreement with a famous paper, but it does require a target. You can validate an estimator against synthetic truth, a machine-learning model against held-out cells, a thermal model against a published onset temperature, a grid dispatch model against an energy-balance check, or a fast-charge optimizer against a baseline CCCV protocol. Without validation, the project is a demonstration. With validation, it begins to look like research.

For sodium-ion batteries, these choices are less forgiving. SIB OCV curves can have long hard-carbon plateaus, different hysteresis, different low-temperature behavior, different safety envelopes, and different aging modes. Public sodium-ion datasets may not support the same validation depth as lithium-ion datasets. A careful SIB project therefore often uses a two-layer claim: “The workflow is validated on a lithium-ion benchmark; the sodium-ion case study uses chemistry-specific parameters and is interpreted as a simulation study until better SIB data are available.” That sentence is not a weakness. It is honest methodology.

13.4 How to Use This Chapter

Read the six track summaries first. Pick one primary track. Complete its guided walkthrough and open-ended exercises. Then write a one-page capstone pre-proposal using the template near the end of the chapter. If your proposal needs a second tool area, complete a second track lightly. Do not complete all tracks mechanically. Depth beats collection.

| Track | Best for capstone questions about | Main tool pattern | Runnable without paid software |
|------------------------------|--|--|--------------------------------|
| A: Advanced state estimation | SOC under bias, joint SOC/SOH, parameter drift | Particle filtering and moving-horizon estimation | Yes |

| Track | Best for capstone questions about | Main tool pattern | Runnable without paid software |
|----------------------------------|---|---|--------------------------------|
| B: Thermal and safety | abuse heating, thermal runaway onset, pack propagation | Reaction-thermal ODE/PDE models, COMSOL mapping | Yes, with Python fallback |
| C: Data-driven health estimation | SOH prediction, early-cycle features, partial-charge data | Feature engineering and PyTorch sequence models | Yes |
| D: Grid storage applications | dispatch, degradation-aware revenue, renewable smoothing | Energy-balance optimization | Yes |
| E: EV powertrain integration | drive-cycle loading, pack sizing, BMS-in-the-loop studies | Longitudinal vehicle model plus battery ECM | Yes |
| F: Fast charging optimization | charge time, temperature limits, sodium-ion protocol design | Nonlinear optimization and model predictive control | Yes |

13.5 Track A: Advanced State Estimation

13.5.1 Track A Opening

Track A extends Lab Chapter 7 beyond EKF and UKF. Kalman filters are elegant when the posterior distribution is close to Gaussian and the model is smooth enough for linearization or sigma-point propagation. Battery systems do not always behave that kindly. OCV curves may be flat, current sensors drift, capacity changes slowly, and parameter uncertainty can dominate voltage noise. Particle

filters and moving-horizon estimators are two standard responses. A particle filter represents uncertainty by a cloud of weighted samples. Moving-horizon estimation solves a constrained optimization problem over a recent time window.

This track operationalizes Textbook Chapter 10's estimator discussion and the OCV observability warnings from the sodium-ion chapter. The guiding question is simple: when OCV is flat and current bias exists, can a particle filter maintain a credible SOC estimate better than an EKF-style point estimate?

13.5.2 Guided Walkthrough A1: Particle Filter for SOC and Capacity

Learning objective: Implement a bootstrap particle filter that jointly estimates SOC and usable capacity from voltage/current data generated by a simple ECM.

We will generate synthetic truth because particle filtering is easiest to learn when the hidden state is known. The model is a first-order Thevenin ECM:

$$V_t = U(z) - R_0 I - v_1, \quad (13.1)$$

$$\dot{v}_1 = -\frac{1}{R_1 C_1} v_1 + \frac{1}{C_1} I, \quad (13.2)$$

$$\dot{z} = -\frac{I}{3600Q}. \quad (13.3)$$

Here positive current means discharge. The particle state is $x = [z, v_1, Q]^\top$. Capacity changes very slowly, so we model it as a random walk. This is a teaching shortcut, but it mirrors real joint SOC/SOH estimators.

Create `track_a_state_estimation/particle_filter_soc.py`:

```
from __future__ import annotations

from dataclasses import dataclass

import matplotlib.pyplot as plt
import numpy as np

@dataclass
class ECMPParameters:
    r0_ohm: float = 0.035
    r1_ohm: float = 0.012
    c1_f: float = 2400.0
```

```

nominal_capacity_ah: float = 2.4
voltage_noise_std_v: float = 0.006

def lithium_like_ocv(soc: np.ndarray) -> np.ndarray:
    soc_clipped = np.clip(soc, 0.0, 1.0)
    return (
        3.05
        + 0.72 * soc_clipped
        + 0.08 * np.tanh((soc_clipped - 0.12) / 0.035)
        - 0.06 * np.tanh((soc_clipped - 0.88) / 0.045)
    )

def sodium_like_ocv(soc: np.ndarray) -> np.ndarray:
    soc_clipped = np.clip(soc, 0.0, 1.0)
    plateau = 2.92 + 0.10 * soc_clipped
    low_soc_knee = 0.18 * np.tanh((soc_clipped - 0.10) / 0.030)
    high_soc_knee = 0.16 * np.tanh((soc_clipped - 0.88) / 0.040)
    return plateau + low_soc_knee + high_soc_knee

def make_current_profile(time_s: np.ndarray) -> np.ndarray:
    current_a = np.zeros_like(time_s)
    for k, t in enumerate(time_s):
        if 300 <= t < 1300:
            current_a[k] = 1.4
        elif 1600 <= t < 2300:
            current_a[k] = -0.8
        elif 2600 <= t < 3800:
            current_a[k] = 1.0
        elif 4200 <= t < 5200:
            current_a[k] = 1.8
        elif 5600 <= t < 6600:
            current_a[k] = -1.0
    return current_a

def simulate_truth(
    time_s: np.ndarray,
    current_a: np.ndarray,
    params: ECMPParameters,
    ocv_function,
    seed: int = 7,
) -> dict[str, np.ndarray]:

```

```

rng = np.random.default_rng(seed)
dt_s = np.diff(time_s, prepend=time_s[0])
soc = np.zeros_like(time_s)
v1 = np.zeros_like(time_s)
capacity_ah = np.zeros_like(time_s)
voltage_v = np.zeros_like(time_s)

soc[0] = 0.86
capacity_ah[0] = 2.20
v1[0] = 0.0
voltage_v[0] = ocv_function(np.array([soc[0]]))[0] - params.r0_ohm *
↳ current_a[0]

tau_s = params.r1_ohm * params.c1_f
for k in range(1, len(time_s)):
    dt = dt_s[k]
    capacity_ah[k] = capacity_ah[k - 1] - 1.2e-8 * abs(current_a[k - 1])
↳ * dt
    soc[k] = soc[k - 1] - current_a[k - 1] * dt / (3600.0 * capacity_ah[k
↳ - 1])
    soc[k] = np.clip(soc[k], 0.02, 0.98)
    decay = np.exp(-dt / tau_s)
    v1[k] = decay * v1[k - 1] + params.r1_ohm * (1.0 - decay) *
↳ current_a[k - 1]
    clean_voltage = ocv_function(np.array([soc[k]]))[0] - params.r0_ohm *
↳ current_a[k] - v1[k]
    voltage_v[k] = clean_voltage + rng.normal(0.0,
↳ params.voltage_noise_std_v)

return {
    "time_s": time_s,
    "current_a": current_a,
    "soc": soc,
    "v1_v": v1,
    "capacity_ah": capacity_ah,
    "voltage_v": voltage_v,
}

def systematic_resample(weights: np.ndarray, rng: np.random.Generator) ->
↳ np.ndarray:
    n_particles = len(weights)
    positions = (rng.random() + np.arange(n_particles)) / n_particles
    cumulative_sum = np.cumsum(weights)
    indexes = np.zeros(n_particles, dtype=int)

```

```

i = 0
j = 0
while i < n_particles:
    if positions[i] < cumulative_sum[j]:
        indexes[i] = j
        i += 1
    else:
        j += 1
return indexes

def run_particle_filter(
    time_s: np.ndarray,
    current_a: np.ndarray,
    voltage_v: np.ndarray,
    params: ECMPParameters,
    ocv_function,
    n_particles: int = 3000,
    seed: int = 11,
) -> dict[str, np.ndarray]:
    rng = np.random.default_rng(seed)
    n_steps = len(time_s)
    dt_s = np.diff(time_s, prepend=time_s[0])

    particles = np.zeros((n_particles, 3))
    particles[:, 0] = rng.normal(0.76, 0.08, n_particles)
    particles[:, 1] = rng.normal(0.0, 0.010, n_particles)
    particles[:, 2] = rng.normal(params.nominal_capacity_ah, 0.20,
↳ n_particles)
    particles[:, 0] = np.clip(particles[:, 0], 0.01, 0.99)
    particles[:, 2] = np.clip(particles[:, 2], 1.5, 3.0)

    weights = np.full(n_particles, 1.0 / n_particles)
    estimates = np.zeros((n_steps, 3))
    lower = np.zeros((n_steps, 3))
    upper = np.zeros((n_steps, 3))
    effective_n = np.zeros(n_steps)

    tau_s = params.r1_ohm * params.c1_f
    voltage_std = params.voltage_noise_std_v

    for k in range(n_steps):
        if k > 0:
            dt = dt_s[k]
            previous_current = current_a[k - 1]

```

```

        decay = np.exp(-dt / tau_s)
        particles[:, 0] -= previous_current * dt / (3600.0 * particles[:,
→ 2])

        particles[:, 1] = decay * particles[:, 1] + params.r1_ohm * (1.0
→ - decay) * previous_current
        particles[:, 2] += rng.normal(0.0, 2.0e-5, n_particles)
        particles[:, 0] += rng.normal(0.0, 2.0e-4, n_particles)
        particles[:, 1] += rng.normal(0.0, 4.0e-4, n_particles)
        particles[:, 0] = np.clip(particles[:, 0], 0.01, 0.99)
        particles[:, 2] = np.clip(particles[:, 2], 1.5, 3.0)

        predicted_voltage = ocv_function(particles[:, 0]) - params.r0_ohm *
→ current_a[k] - particles[:, 1]
        residual = voltage_v[k] - predicted_voltage
        likelihood = np.exp(-0.5 * (residual / voltage_std) ** 2) + 1.0e-300
        weights *= likelihood
        weights /= np.sum(weights)

        estimates[k] = np.average(particles, axis=0, weights=weights)
        for state_index in range(3):
            lower[k, state_index] = weighted_quantile(particles[:,
→ state_index], weights, 0.05)
            upper[k, state_index] = weighted_quantile(particles[:,
→ state_index], weights, 0.95)

        effective_n[k] = 1.0 / np.sum(weights**2)
        if effective_n[k] < 0.5 * n_particles:
            indexes = systematic_resample(weights, rng)
            particles = particles[indexes]
            weights.fill(1.0 / n_particles)

    return {
        "soc_est": estimates[:, 0],
        "v1_est": estimates[:, 1],
        "capacity_est": estimates[:, 2],
        "soc_lower": lower[:, 0],
        "soc_upper": upper[:, 0],
        "capacity_lower": lower[:, 2],
        "capacity_upper": upper[:, 2],
        "effective_n": effective_n,
    }

def weighted_quantile(values: np.ndarray, weights: np.ndarray, quantile:
→ float) -> float:

```

```

sorter = np.argsort(values)
values_sorted = values[sorter]
weights_sorted = weights[sorter]
cumulative = np.cumsum(weights_sorted)
return np.interp(quantile, cumulative, values_sorted)

```

```

def plot_particle_filter_results(truth: dict[str, np.ndarray], estimate:
    dict[str, np.ndarray], label: str) -> None:
    time_h = truth["time_s"] / 3600.0
    fig, axes = plt.subplots(4, 1, figsize=(9, 9), sharex=True)

    axes[0].plot(time_h, truth["current_a"], color="black", linewidth=1.3)
    axes[0].set_ylabel("Current (A)")
    axes[0].grid(True, alpha=0.3)

    axes[1].plot(time_h, truth["voltage_v"], color="tab:blue", linewidth=1.2)
    axes[1].set_ylabel("Voltage (V)")
    axes[1].grid(True, alpha=0.3)

    axes[2].plot(time_h, truth["soc"], color="black", linewidth=1.8,
    label="Truth")
    axes[2].plot(time_h, estimate["soc_est"], color="tab:red", linewidth=1.5,
    label="Particle filter")
    axes[2].fill_between(
        time_h,
        estimate["soc_lower"],
        estimate["soc_upper"],
        color="tab:red",
        alpha=0.20,
        label="90% interval",
    )
    axes[2].set_ylabel("SOC (-)")
    axes[2].legend(loc="best")
    axes[2].grid(True, alpha=0.3)

    axes[3].plot(time_h, truth["capacity_ah"], color="black", linewidth=1.8,
    label="Truth")
    axes[3].plot(time_h, estimate["capacity_est"], color="tab:green",
    linewidth=1.5, label="Estimate")
    axes[3].fill_between(
        time_h,
        estimate["capacity_lower"],
        estimate["capacity_upper"],
        color="tab:green",
    )

```



```

        alpha=0.20,
    )
    axes[3].set_xlabel("Time (h)")
    axes[3].set_ylabel("Capacity (Ah)")
    axes[3].legend(loc="best")
    axes[3].grid(True, alpha=0.3)

    fig.suptitle(f"Track A particle filter: {label}")
    fig.tight_layout()
    plt.show()

def main() -> None:
    params = ECMPParameters()
    time_s = np.arange(0.0, 7200.0 + 1.0, 2.0)
    current_a = make_current_profile(time_s)

    for label, ocv_function in [
        ("lithium-like sloped OCV", lithium_like_ocv),
        ("sodium-like flat OCV", sodium_like_ocv),
    ]:
        truth = simulate_truth(time_s, current_a, params, ocv_function)
        estimate = run_particle_filter(time_s, current_a, truth["voltage_v"],
↪ params, ocv_function)
        soc_rmse = np.sqrt(np.mean((truth["soc"] - estimate["soc_est"]) **
↪ 2))
        capacity_error = estimate["capacity_est"][-1] -
↪ truth["capacity_ah"][-1]
        print(f"{label}")
        print(f" SOC RMSE: {soc_rmse:.4f}")
        print(f" Final capacity error: {capacity_error:+.4f} Ah")
        print(f" Median effective particle count:
↪ {np.median(estimate['effective_n']):.0f}")
        plot_particle_filter_results(truth, estimate, label)

if __name__ == "__main__":
    main()

```

Run it:

```
python track_a_state_estimation/particle_filter_soc.py
```

The script begins by defining two OCV functions. The lithium-like curve has stronger slope over much of the SOC range, which means voltage carries useful

information about SOC. The sodium-like curve deliberately contains a flatter mid-SOC plateau, which mimics a common hard-carbon challenge: voltage changes only weakly when SOC changes. This is not a complete sodium-ion OCV model. It is a controlled observability experiment.

The `simulate_truth` function creates the hidden SOC, RC voltage, capacity, and noisy terminal voltage. The `run_particle_filter` function then estimates the hidden state using only time, current, and voltage. The key update is the likelihood calculation: particles whose predicted voltage is close to the measured voltage receive larger weights. The resampling step prevents the filter from carrying thousands of particles with effectively zero weight.

Expected terminal output will vary slightly because the data and filter use random noise, but it should resemble:

```
lithium-like sloped OCV
SOC RMSE: 0.0100 to 0.0250
Final capacity error: within about +/-0.12 Ah
Median effective particle count: 1500 to 2800
sodium-like flat OCV
SOC RMSE: 0.0200 to 0.0600
Final capacity error: often larger than lithium-like case
Median effective particle count: lower or more variable
```

The plots should contain four stacked panels. The current panel shows discharge pulses above zero and charge pulses below zero. The voltage panel follows those pulses, dropping under discharge and rising under charge. The SOC panel should show the black truth curve and the red particle-filter estimate. The red uncertainty band should widen during low-information periods and narrow when voltage is informative. The capacity panel should move slowly and should not jump sharply; sharp capacity jumps indicate too much process noise or particle collapse.

What could go wrong

If the SOC estimate moves in the wrong direction during discharge, your current sign convention is reversed. Equation (13.3) assumes positive current discharges the cell and decreases SOC.

If the uncertainty band collapses to a thin line early and the estimate becomes wrong, the filter is degenerating. Increase `n_particles`, increase process noise slightly, or resample when `effective_n < 0.7 * n_particles`.

If sodium-like and lithium-like results look identical, the OCV functions are

probably not being passed correctly. Print `sodium_like_ocv(np.linspace(0.2, 0.8, 5))` and compare it to the lithium-like values.

If the code runs slowly, reduce `n_particles` to 1000 while debugging. Use 3000 or more for final plots.

Reflection

This exercise taught you the practical meaning of observability. The particle filter did not magically solve the flat-OCV problem; it represented uncertainty more honestly. That distinction matters. In a sodium-ion paper, a wider posterior under a hard-carbon plateau may be the correct result, not a failure.

13.5.3 Track A Open-Ended Exercises

1. Add a current-sensor bias state to the particle filter. Hint: the propagation equation should use `current_a[k] + bias_a`, while the terminal voltage equation should also use the biased current in the ohmic drop.
2. Compare the particle filter against the EKF you wrote in Lab Chapter 7 on the same synthetic dataset. Hint: report SOC RMSE and 90% interval coverage, not only a pretty plot.
3. Replace the synthetic OCV curve with a sodium-ion OCV table digitized from a paper or dataset. Hint: use `np.interp` and record the source.

13.6 Track B: Thermal and Safety

13.6.1 Track B Opening

Track B is about thermal safety modeling without pretending that a laptop exercise is an abuse-test laboratory. Thermal runaway is chemically complex, geometry-dependent, and safety-critical. You should not use a teaching model to make product safety claims. You can, however, use a teaching model to understand the structure of runaway simulations: heat generation, heat removal, Arrhenius reaction rates, onset criteria, and propagation between cells.

COMSOL's Battery Design Module supports high-fidelity battery and pack simulations, including porous-electrode models, heat transfer, short-circuit studies, and thermal runaway propagation using event-based heat sources. That is the commercial tool path. The open path in this track is a finite-difference thermal

model with Arrhenius heat release. It teaches the same modeling anatomy: source terms, conduction, convection, and threshold behavior.

13.6.2 Guided Walkthrough B1: One-Dimensional Thermal Runaway Onset Model

Learning objective: Simulate temperature rise in a cell slab with heat loss and an Arrhenius side-reaction source.

We model a slab cell with through-thickness coordinate x . Temperature evolves as:

$$\rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} - h_a(T - T_\infty) + q_{\text{ohmic}} + HA \exp\left(-\frac{E_a}{RT}\right)(1 - \alpha), \quad (13.4)$$

$$\frac{\partial \alpha}{\partial t} = A \exp\left(-\frac{E_a}{RT}\right)(1 - \alpha). \quad (13.5)$$

The reaction progress variable α is a teaching abstraction. Real thermal runaway models often use several reactions: SEI decomposition, anode-electrolyte reaction, cathode decomposition, electrolyte decomposition, separator failure, and internal shorting. Equation (13.5) gives us one controllable exothermic process.

Create `track_b_thermal_safety/runaway_slab.py`:

```
from __future__ import annotations

from dataclasses import dataclass

import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

@dataclass
class SlabParameters:
    thickness_m: float = 8.0e-3
    n_nodes: int = 31
    density_kg_m3: float = 2400.0
    heat_capacity_j_kg_k: float = 950.0
    conductivity_w_m_k: float = 0.65
    volumetric_heat_loss_w_m3_k: float = 1.8e4
    ambient_k: float = 298.15
    ohmic_heat_w_m3: float = 1.2e4
    reaction_enthalpy_j_m3: float = 1.8e8
```

```

pre_exponential_l_s: float = 2.0e7
activation_energy_j_mol: float = 8.5e4
gas_constant_j_mol_k: float = 8.314462618

def second_derivative_neumann(temperature_k: np.ndarray, dx_m: float) ->
    np.ndarray:
    padded = np.empty(len(temperature_k) + 2)
    padded[1:-1] = temperature_k
    padded[0] = temperature_k[1]
    padded[-1] = temperature_k[-2]
    return (padded[:-2] - 2.0 * padded[1:-1] + padded[2:]) / dx_m**2

def rhs(_time_s: float, state: np.ndarray, params: SlabParameters) ->
    np.ndarray:
    n = params.n_nodes
    temperature_k = state[:n]
    alpha = state[n:]
    dx_m = params.thickness_m / (n - 1)

    arrhenius = params.pre_exponential_l_s * np.exp(
        -params.activation_energy_j_mol / (params.gas_constant_j_mol_k *
        ↪ temperature_k)
    )
    reaction_rate = arrhenius * np.clip(1.0 - alpha, 0.0, 1.0)
    reaction_heat = params.reaction_enthalpy_j_m3 * reaction_rate
    conduction = params.conductivity_w_m_k *
    ↪ second_derivative_neumann(temperature_k, dx_m)
    heat_loss = -params.volumetric_heat_loss_w_m3_k * (temperature_k -
    ↪ params.ambient_k)

    dtemperature_dt = (
        conduction + heat_loss + params.ohmic_heat_w_m3 + reaction_heat
    ) / (params.density_kg_m3 * params.heat_capacity_j_kg_k)
    dalpha_dt = reaction_rate
    return np.concatenate([dtemperature_dt, dalpha_dt])

def run_case(ambient_c: float, hot_spot_c: float, ohmic_heat_w_m3: float) ->
    dict[str, np.ndarray]:
    params = SlabParameters(
        ambient_k=ambient_c + 273.15,
        ohmic_heat_w_m3=ohmic_heat_w_m3,
    )

```

```

x_m = np.linspace(0.0, params.thickness_m, params.n_nodes)
initial_temperature_k = np.full(params.n_nodes, params.ambient_k)
center = 0.5 * params.thickness_m
initial_temperature_k += hot_spot_c * np.exp(-((x_m - center) / 1.5e-3)
↳ ** 2)
initial_alpha = np.zeros(params.n_nodes)
initial_state = np.concatenate([initial_temperature_k, initial_alpha])

solution = solve_ivp(
    fun=Lambda t, y: rhs(t, y, params),
    t_span=(0.0, 2500.0),
    y0=initial_state,
    method="BDF",
    max_step=2.0,
    rtol=1.0e-6,
    atol=1.0e-8,
    dense_output=False,
)
temperature_k = solution.y[: params.n_nodes, :]
alpha = solution.y[params.n_nodes :, :]
return {
    "time_s": solution.t,
    "x_m": x_m,
    "temperature_k": temperature_k,
    "alpha": alpha,
    "params": params,
}

def plot_cases(cases: dict[str, dict[str, np.ndarray]]) -> None:
    fig, axes = plt.subplots(2, 1, figsize=(9, 7), sharex=True)
    for label, result in cases.items():
        max_temperature_c = result["temperature_k"].max(axis=0) - 273.15
        mean_alpha = result["alpha"].mean(axis=0)
        axes[0].plot(result["time_s"] / 60.0, max_temperature_c,
↳ linewidth=1.8, label=label)
        axes[1].plot(result["time_s"] / 60.0, mean_alpha, linewidth=1.8,
↳ label=label)
    axes[0].axhline(120.0, color="black", linestyle="--", linewidth=1.0,
↳ label="120 C marker")
    axes[0].set_ylabel("Maximum temperature (deg C)")
    axes[0].grid(True, alpha=0.3)
    axes[0].legend(loc="best")
    axes[1].set_xlabel("Time (min)")
    axes[1].set_ylabel("Mean reaction progress (-)")

```

```

axes[1].grid(True, alpha=0.3)
fig.tight_layout()
plt.show()

def main() -> None:
    cases = {
        "mild: 25 C ambient, 10 C hot spot": run_case(25.0, 10.0, 1.2e4),
        "warm: 45 C ambient, 25 C hot spot": run_case(45.0, 25.0, 1.2e4),
        "abuse: 60 C ambient, 55 C hot spot": run_case(60.0, 55.0, 1.2e4),
    }
    for label, result in cases.items():
        max_temperature_c = result["temperature_k"].max() - 273.15
        final_alpha = result["alpha"].mean(axis=0)[-1]
        print(f"{label}")
        print(f"  Peak temperature: {max_temperature_c:.1f} deg C")
        print(f"  Final mean reaction progress: {final_alpha:.3f}")
    plot_cases(cases)

if __name__ == "__main__":
    main()

```

Run:

```
python track_b_thermal_safety/runaway_slab.py
```

The code uses a stiff solver because the Arrhenius source can change quickly once temperature rises. The `second_derivative_neumann` function applies zero-flux boundary conditions at the slab faces. That is a teaching choice. A convective boundary condition would be more physically direct, but the volumetric heat-loss term keeps the first model simple and stable.

Expected output should show the mild case remaining near ambient, the warm case rising modestly, and the abuse case either approaching or crossing the 120 deg C marker depending on your machine's floating-point path. The plot should have two panels. The first panel shows maximum temperature versus time in minutes. Correct behavior is threshold-like: below a certain initial temperature the heat loss wins, but above it the exothermic term accelerates. The second panel shows mean reaction progress. If reaction progress reaches one immediately in every case, the pre-exponential factor is too large or the activation energy is too low.

What could go wrong

If `solve_ivp` fails with a step-size warning, reduce `pre_exponential_1_s` or increase `volumetric_heat_loss_w_m3_k`. Thermal runaway equations can become numerically stiff when the reaction accelerates.

If temperature falls below ambient during ohmic heating, check the sign of `heat_loss`. It should be negative when the cell is warmer than ambient.

If the temperature profile develops sawtooth oscillations across nodes, increase `n_nodes` moderately or reduce `max_step`. Oscillations usually mean the spatial discretization and time step are fighting.

If every case looks safe, raise `ambient_c`, raise `hot_spot_c`, or reduce the heat-loss coefficient. The point of the exercise is to observe threshold behavior, not to calibrate a real cell.

Reflection

This exercise gives you the skeleton of abuse modeling: heat source, heat sink, transport, and nonlinear acceleration. In COMSOL, the same terms appear as heat-transfer physics, material properties, events, source terms, and multiphysics couplings. The Python model is not a substitute for a validated safety model, but it makes the commercial-tool interface intelligible.

13.6.3 Track B Dataset and Reproduction Exercise

Public, machine-readable thermal-runaway datasets are less standardized than cycling datasets. For a reproduction exercise, use a paper with a clearly reported temperature-time curve or onset-temperature comparison and reproduce the qualitative threshold behavior rather than claiming exact safety validation. A practical target is a Semenov-style onset comparison from the thermal-runaway literature, or a COMSOL application-library example if you have a license. COMSOL describes its Battery Design Module as supporting detailed porous-electrode batteries, heat transfer, short-circuiting, pack modeling, and thermal runaway propagation using thermal events. If you use COMSOL, export the maximum cell temperature versus time and compare it against the Python finite-difference model under matched heat-loss and source assumptions.

For a written reproduction, report three things: the original figure and citation, the source terms you included, and the source terms you omitted. “Close enough” means matching the qualitative onset/no-onset boundary and temperature order of

magnitude, not matching a proprietary example curve point by point.

13.6.4 Track B Open-Ended Exercises

1. Replace the volumetric heat-loss term with convective boundary conditions at the two slab surfaces. Hint: modify the ghost nodes in `second_derivative_neumann`.
2. Add a second reaction with a higher onset temperature and larger enthalpy. Hint: duplicate `alpha` into `alpha_1` and `alpha_2`.
3. Build a two-cell propagation model with thermal resistance between cells. Hint: start with two lumped temperatures before returning to a spatial grid.

13.7 Track C: Data-Driven Health Estimation

13.7.1 Track C Opening

Track C teaches a modern data-driven SOH workflow. Lab Chapter 8 showed capacity-fade models and incremental-capacity analysis. Lab Chapter 11 taught public dataset loading. Here we build a sequence model that predicts remaining useful life from early-cycle summaries. The point is not that neural networks are automatically better than feature engineering. The point is to learn a defensible machine-learning workflow: split by cell, normalize using training data only, compare against a simple baseline, and inspect errors.

The reproduction anchor is Severson et al., “Data-driven prediction of battery cycle life before capacity degradation,” published in *Nature Energy* in 2019. The authors used early discharge voltage-curve features from 124 LFP/graphite cells and reported strong cycle-life prediction performance; the data are available through <https://data.mtr.io/1>, and associated processing code is public. We will not exactly reproduce their full model here because that belongs in a full Chapter 12 project. We will reproduce the central idea: early-cycle voltage/capacity information contains predictive signal before obvious capacity fade appears.

13.7.2 Dataset Integration

Preferred dataset: Severson/Attia/MIT-Stanford-Toyota battery cycle-life dataset at <https://data.mtr.io/1>. Format: MATLAB `.mat`/HDF5-style batch files and

processed structures. License and access terms should be checked on the dataset page when you download it. Size: large enough that you should download it intentionally rather than inside a notebook.

If you already completed Lab Chapter 11, place the processed per-cell summary table in:

```
|  
↳ SimulationCompanion/chapter13_specialization_tracks/data/severson_cell_summaries.csv
```

The chapter code also creates a synthetic schema-compatible dataset if that file is absent. That fallback lets you learn the machine-learning workflow on a laptop, but it does not support scientific claims.

13.7.3 Guided Walkthrough C1: LSTM Remaining-Life Predictor

Learning objective: Train a small PyTorch LSTM to predict cycle life from the first 100 capacity observations and compare it with a linear baseline.

Create `track_c_health_ml/lstm_cycle_life.py`:

```
from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_percentage_error,
↳ mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

DATA_PATH = Path("data/severson_cell_summaries.csv")

def make_synthetic_dataset(seed: int = 42) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    records = []
    n_cells = 124
    for cell_index in range(n_cells):
```

```

        true_life = rng.integers(350, 2200)
        early_slope = -rng.lognormal(mean=-7.2, sigma=0.45) * (2100.0 /
↪ true_life)
        curvature = -rng.lognormal(mean=-10.0, sigma=0.50) * (2100.0 /
↪ true_life)
        initial_capacity = rng.normal(1.10, 0.025)
        for cycle in range(1, 101):
            capacity = (
                initial_capacity
                + early_slope * cycle
                + curvature * cycle**2
                + rng.normal(0.0, 0.0018)
            )
            records.append(
                {
                    "cell_id": f"synthetic_{cell_index:03d}",
                    "cycle_index": cycle,
                    "discharge_capacity_ah": capacity,
                    "cycle_life": true_life,
                }
            )
        return pd.DataFrame.from_records(records)

def load_or_create_data() -> pd.DataFrame:
    if DATA_PATH.exists():
        data = pd.read_csv(DATA_PATH)
        required = {"cell_id", "cycle_index", "discharge_capacity_ah",
↪ "cycle_life"}
        missing = required.difference(data.columns)
        if missing:
            raise ValueError(f"{DATA_PATH} is missing columns:
↪ {sorted(missing)}")
        print(f"Loaded real summary data from {DATA_PATH}")
        return data
    print("Real Severson/MATR summary file not found; using synthetic
↪ teaching data.")
    return make_synthetic_dataset()

def build_sequences(data: pd.DataFrame, n_cycles: int = 100) ->
↪ tuple[np.ndarray, np.ndarray, np.ndarray]:
    sequences = []
    targets = []
    cell_ids = []

```

```

for cell_id, group in data.groupby("cell_id"):
    ordered = group.sort_values("cycle_index")
    first_cycles = ordered[ordered["cycle_index"].between(1, n_cycles)]
    if len(first_cycles) != n_cycles:
        continue
    capacity =
    ↪ first_cycles["discharge_capacity_ah"].to_numpy(dtype=float)
        normalized_capacity = capacity / capacity[0]
        delta_capacity = normalized_capacity - normalized_capacity[0]
        cycle_axis = np.linspace(0.0, 1.0, n_cycles)
        sequence = np.column_stack([normalized_capacity, delta_capacity,
    ↪ cycle_axis])
        sequences.append(sequence)
        targets.append(float(first_cycles["cycle_life"].iloc[0]))
        cell_ids.append(cell_id)
    return np.stack(sequences), np.asarray(targets), np.asarray(cell_ids)

```

```

class CycleLifeLSTM(nn.Module):
    def __init__(self, n_features: int, hidden_size: int = 32) -> None:
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=n_features,
            hidden_size=hidden_size,
            num_layers=1,
            batch_first=True,
        )
        self.regressor = nn.Sequential(
            nn.Linear(hidden_size, 32),
            nn.ReLU(),
            nn.Linear(32, 1),
        )

    def forward(self, sequence: torch.Tensor) -> torch.Tensor:
        output, _hidden = self.lstm(sequence)
        final_hidden = output[:, -1, :]
        return self.regressor(final_hidden).squeeze(-1)

def train_lstm(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_val: np.ndarray,
    y_val: np.ndarray,
    seed: int = 3,

```

```

) -> tuple[CycleLifeLSTM, StandardScaler, StandardScaler]:
    torch.manual_seed(seed)
    feature_scaler = StandardScaler()
    target_scaler = StandardScaler()

    n_train, n_steps, n_features = x_train.shape
    x_train_2d = x_train.reshape(-1, n_features)
    feature_scaler.fit(x_train_2d)
    x_train_scaled = feature_scaler.transform(x_train_2d).reshape(n_train,
↪ n_steps, n_features)
    x_val_scaled = feature_scaler.transform(x_val.reshape(-1,
↪ n_features)).reshape(
        x_val.shape[0], n_steps, n_features
    )
    y_train_scaled = target_scaler.fit_transform(y_train.reshape(-1,
↪ 1)).ravel()
    y_val_scaled = target_scaler.transform(y_val.reshape(-1, 1)).ravel()

    train_dataset = TensorDataset(
        torch.tensor(x_train_scaled, dtype=torch.float32),
        torch.tensor(y_train_scaled, dtype=torch.float32),
    )
    train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)

    model = CycleLifeLSTM(n_features=n_features)
    optimizer = torch.optim.Adam(model.parameters(), lr=2.0e-3,
↪ weight_decay=1.0e-4)
    loss_function = nn.MSELoss()

    x_val_tensor = torch.tensor(x_val_scaled, dtype=torch.float32)
    y_val_tensor = torch.tensor(y_val_scaled, dtype=torch.float32)

    best_state = None
    best_val_loss = np.inf
    patience = 40
    stale_epochs = 0

    for epoch in range(1, 501):
        model.train()
        train_losses = []
        for batch_x, batch_y in train_loader:
            optimizer.zero_grad()
            prediction = model(batch_x)
            loss = loss_function(prediction, batch_y)
            loss.backward()

```

```

        optimizer.step()
        train_losses.append(loss.item())

    model.eval()
    with torch.no_grad():
        val_loss = loss_function(model(x_val_tensor),
↪ y_val_tensor).item()

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_state = {key: value.detach().clone() for key, value in
↪ model.state_dict().items()}
            stale_epochs = 0
        else:
            stale_epochs += 1

        if epoch % 50 == 0:
            print(f"Epoch {epoch:03d}: train MSE={np.mean(train_losses):.4f},
↪ val MSE={val_loss:.4f}")

            if stale_epochs >= patience:
                print(f"Early stopping at epoch {epoch}")
                break

    if best_state is not None:
        model.load_state_dict(best_state)

    return model, feature_scaler, target_scaler

def predict_lstm(
    model: CycleLifeLSTM,
    feature_scaler: StandardScaler,
    target_scaler: StandardScaler,
    x: np.ndarray,
) -> np.ndarray:
    n_samples, n_steps, n_features = x.shape
    x_scaled = feature_scaler.transform(x.reshape(-1,
↪ n_features)).reshape(n_samples, n_steps, n_features)
    model.eval()
    with torch.no_grad():
        prediction_scaled = model(torch.tensor(x_scaled,
↪ dtype=torch.float32)).numpy()
    return target_scaler.inverse_transform(prediction_scaled.reshape(-1,
↪ 1)).ravel()

```

```

def ridge_baseline(x_train: np.ndarray, y_train: np.ndarray, x_test:
↳ np.ndarray) -> np.ndarray:
    features_train = np.column_stack(
        [
            x_train[:, -1, 0],
            x_train[:, -1, 1],
            x_train[:, 9, 0] - x_train[:, 99, 0],
            np.var(x_train[:, :, 1], axis=1),
        ]
    )
    features_test = np.column_stack(
        [
            x_test[:, -1, 0],
            x_test[:, -1, 1],
            x_test[:, 9, 0] - x_test[:, 99, 0],
            np.var(x_test[:, :, 1], axis=1),
        ]
    )
    scaler = StandardScaler()
    features_train_scaled = scaler.fit_transform(features_train)
    features_test_scaled = scaler.transform(features_test)
    model = Ridge(alpha=1.0)
    model.fit(features_train_scaled, y_train)
    return model.predict(features_test_scaled)

def plot_predictions(y_test: np.ndarray, predictions: dict[str, np.ndarray])
↳ -> None:
    fig, ax = plt.subplots(figsize=(6.5, 6.0))
    min_value = min(y_test.min(), *(pred.min() for pred in
↳ predictions.values()))
    max_value = max(y_test.max(), *(pred.max() for pred in
↳ predictions.values()))
    ax.plot([min_value, max_value], [min_value, max_value], color="black",
↳ linewidth=1.0)
    for label, predicted in predictions.items():
        ax.scatter(y_test, predicted, s=42, alpha=0.75, label=label)
    ax.set_xlabel("Observed cycle life (cycles)")
    ax.set_ylabel("Predicted cycle life (cycles)")
    ax.set_title("Track C early-cycle life prediction")
    ax.grid(True, alpha=0.3)
    ax.legend(loc="best")
    fig.tight_layout()

```

```
plt.show()
```

```
def main() -> None:
    data = load_or_create_data()
    sequences, targets, cell_ids = build_sequences(data)
    x_train_val, x_test, y_train_val, y_test, ids_train_val, ids_test =
↳ train_test_split(
        sequences,
        targets,
        cell_ids,
        test_size=0.20,
        random_state=5,
    )
    x_train, x_val, y_train, y_val = train_test_split(
        x_train_val,
        y_train_val,
        test_size=0.25,
        random_state=6,
    )

    print(f"Cells: train={len(y_train)}, val={len(y_val)},
↳ test={len(y_test)}")
    model, feature_scaler, target_scaler = train_lstm(x_train, y_train,
↳ x_val, y_val)
    lstm_pred = predict_lstm(model, feature_scaler, target_scaler, x_test)
    ridge_pred = ridge_baseline(x_train_val, y_train_val, x_test)

    for label, prediction in [("LSTM", lstm_pred), ("Ridge baseline",
↳ ridge_pred)]:
        rmse = mean_squared_error(y_test, prediction, squared=False)
        mape = mean_absolute_percentage_error(y_test, prediction) * 100.0
        print(f"{label}: RMSE={rmse:.1f} cycles, MAPE={mape:.1f}%")

    plot_predictions(y_test, {"LSTM": lstm_pred, "Ridge baseline":
↳ ridge_pred})

if __name__ == "__main__":
    main()
```

Run:

```
python track_c_health_ml/lstm_cycle_life.py
```

The workflow is intentionally strict about splitting by cell. We never allow

cycles from the same cell to appear in both training and test sets. The feature scaler is fit on training data only. The target scaler is also fit on training data only. These habits matter more than the LSTM architecture. A fancy model with leaked normalization is not publishable.

Expected output on synthetic data:

```
Real Severson/MATR summary file not found; using synthetic teaching data.  
Cells: train=74, val=25, test=25  
Epoch 050: train MSE=0.8123, val MSE=0.9401  
Epoch 100: train MSE=0.4218, val MSE=0.6105  
LSTM: RMSE=roughly 150 to 350 cycles, MAPE=roughly 10 to 25%  
Ridge baseline: RMSE=roughly 120 to 300 cycles, MAPE=roughly 8 to 22%
```

Do not be surprised if the ridge baseline beats the LSTM on small data. That is a lesson, not a failure. The Severson paper’s core success came from physically meaningful voltage-curve features and careful model comparison, not from using the largest neural network available.

The prediction plot should show observed cycle life on the horizontal axis and predicted cycle life on the vertical axis. A perfect model lies on the diagonal black line. A common wrong result is a horizontal band, where the model predicts nearly the mean life for every cell. That means the model has not learned useful early-cycle signal.

What could go wrong

If PyTorch is not installed, run the CPU-only install command from the prerequisite box. If it still fails, run only the ridge baseline by commenting out the LSTM training block.

If training loss decreases but validation loss explodes, the model is overfitting. Reduce `hidden_size`, increase `weight_decay`, or stop earlier.

If the test error is unrealistically tiny on real data, check for leakage. Make sure `train_test_split` split cells, not individual rows.

If a real summary CSV fails, inspect column names. The required columns are `cell_id`, `cycle_index`, `discharge_capacity_ah`, and `cycle_life`.

Reflection

This track teaches the difference between a battery ML demo and a battery ML method. The method is not “train an LSTM.” The method is: define the prediction target, build cell-level splits, normalize correctly, compare to a simple model, and

interpret errors in the context of battery physics.

13.7.4 Track C Open-Ended Exercises

1. Replace capacity-only sequences with voltage-curve features from the MATR dataset. Hint: interpolate each discharge curve onto a common capacity grid.
2. Reproduce the qualitative idea of Severson et al. Figure 2 by plotting a voltage-curve difference feature against cycle life. Hint: use cycle 100 minus cycle 10 features.
3. Adapt the workflow to a sodium-ion dataset from Mendeley or Zenodo. Hint: keep the model architecture unchanged and focus on units, cycle labels, and train/test split quality.

13.8 Track D: Grid Storage Applications

13.8.1 Track D Opening

Track D moves from cell behavior to grid-service behavior. A grid battery is still made of cells, but the research question changes. Instead of asking whether a terminal voltage trace matches a lab test, we ask whether an energy-storage asset can reduce peak load, smooth renewables, provide arbitrage, or operate without excessive degradation. The natural model is an energy-balance model with power limits, efficiency, SOC constraints, and sometimes degradation cost.

This track operationalizes the systems view from Textbook Chapter 11. It is deliberately not a power-flow chapter. We will model a single battery connected to an aggregate load and solar profile. That is enough to learn dispatch logic and create a capstone seed.

13.8.2 Guided Walkthrough D1: Degradation-Aware Peak Shaving

Learning objective: Optimize a battery dispatch schedule that reduces grid peak demand while respecting SOC and power constraints.

Create `track_d_grid_storage/peak_shaving_dispatch.py`:

```
from __future__ import annotations

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```

from scipy.optimize import minimize

def make_grid_profile(n_hours: int = 48, seed: int = 21) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    time_h = np.arange(n_hours, dtype=float)
    daily = 1.8 + 0.55 * np.sin(2.0 * np.pi * (time_h - 7.0) / 24.0)
    evening_peak = 0.85 * np.exp(-0.5 * ((time_h % 24.0 - 19.0) / 2.2) ** 2)
    noise = rng.normal(0.0, 0.05, n_hours)
    load_mw = daily + evening_peak + noise
    solar_mw = 1.15 * np.maximum(0.0, np.sin(np.pi * (time_h % 24.0 - 6.0) /
    ↪ 12.0)) ** 1.7
    return pd.DataFrame({"time_h": time_h, "load_mw": load_mw, "solar_mw":
    ↪ solar_mw})

def simulate_soc(
    power_mw: np.ndarray,
    initial_soc: float,
    capacity_mwh: float,
    charge_efficiency: float,
    discharge_efficiency: float,
    dt_h: float,
) -> np.ndarray:
    soc = np.zeros(len(power_mw) + 1)
    soc[0] = initial_soc
    for k, power in enumerate(power_mw):
        if power >= 0.0:
            delta = -power * dt_h / (capacity_mwh * discharge_efficiency)
        else:
            delta = -power * dt_h * charge_efficiency / capacity_mwh
        soc[k + 1] = soc[k] + delta
    return soc

def objective(decision: np.ndarray, net_load_mw: np.ndarray,
    ↪ degradation_weight: float) -> float:
    battery_power_mw = decision[:-1]
    peak_mw = decision[-1]
    grid_power_mw = net_load_mw + battery_power_mw
    peak_penalty = 1.0e4 * np.sum(np.maximum(grid_power_mw - peak_mw, 0.0) **
    ↪ 2)
    cycling_penalty = degradation_weight * np.sum(np.abs(battery_power_mw))
    return peak_mw + peak_penalty + cycling_penalty

```

```

def optimize_dispatch(profile: pd.DataFrame) -> dict[str, np.ndarray]:
    dt_h = 1.0
    capacity_mwh = 4.0
    power_limit_mw = 1.2
    initial_soc = 0.55
    soc_min = 0.15
    soc_max = 0.90
    eta_ch = 0.96
    eta_dis = 0.96
    degradation_weight = 0.012

    net_load_mw = profile["load_mw"].to_numpy() -
    ↪ profile["solar_mw"].to_numpy()
    n = len(net_load_mw)
    initial_peak = np.max(net_load_mw)
    x0 = np.zeros(n + 1)
    x0[-1] = initial_peak

    bounds = [(-power_limit_mw, power_limit_mw)] * n + [(0.0, 5.0)]

    constraints = []
    for step in range(n + 1):
        constraints.append(
            {
                "type": "ineq",
                "fun": lambda x, step=step: simulate_soc(
                    x[:-1], initial_soc, capacity_mwh, eta_ch, eta_dis, dt_h
                )[step]
                - soc_min,
            }
        )
    constraints.append(
        {
            "type": "ineq",
            "fun": lambda x, step=step: soc_max
            - simulate_soc(x[:-1], initial_soc, capacity_mwh, eta_ch,
            ↪ eta_dis, dt_h)[step],
        }
    )

    result = minimize(
        objective,
        x0,
        args=(net_load_mw, degradation_weight),
    
```

```

        method="SLSQP",
        bounds=bounds,
        constraints=constraints,
        options={"maxiter": 800, "ftol": 1.0e-9, "disp": False},
    )
    if not result.success:
        raise RuntimeError(result.message)

    battery_power_mw = result.x[-1]
    soc = simulate_soc(battery_power_mw, initial_soc, capacity_mwh, eta_ch,
↳ eta_dis, dt_h)
    grid_power_mw = net_load_mw + battery_power_mw
    return {
        "battery_power_mw": battery_power_mw,
        "soc": soc,
        "grid_power_mw": grid_power_mw,
        "net_load_mw": net_load_mw,
        "optimized_peak_mw": result.x[-1],
        "baseline_peak_mw": initial_peak,
    }

def plot_dispatch(profile: pd.DataFrame, result: dict[str, np.ndarray]) ->
↳ None:
    time_h = profile["time_h"].to_numpy()
    fig, axes = plt.subplots(3, 1, figsize=(9, 8), sharex=True)
    axes[0].plot(time_h, profile["load_mw"], label="Load", linewidth=1.6)
    axes[0].plot(time_h, profile["solar_mw"], label="Solar", linewidth=1.6)
    axes[0].plot(time_h, result["grid_power_mw"], label="Grid after battery",
↳ linewidth=1.8)
    axes[0].set_ylabel("Power (MW)")
    axes[0].legend(loc="best")
    axes[0].grid(True, alpha=0.3)

    axes[1].step(time_h, result["battery_power_mw"], where="post",
↳ color="tab:red")
    axes[1].axhline(0.0, color="black", linewidth=0.8)
    axes[1].set_ylabel("Battery power (MW)")
    axes[1].grid(True, alpha=0.3)

    axes[2].step(np.arange(len(result["soc"])), result["soc"], where="post",
↳ color="tab:green")
    axes[2].set_xlabel("Time (h)")
    axes[2].set_ylabel("SOC (-)")
    axes[2].set_ylim(0.0, 1.0)

```

```

axes[2].grid(True, alpha=0.3)
fig.tight_layout()
plt.show()

def main() -> None:
    profile = make_grid_profile()
    result = optimize_dispatch(profile)
    print(f"Baseline peak: {result['baseline_peak_mw']:.3f} MW")
    print(f"Optimized peak: {np.max(result['grid_power_mw']):.3f} MW")
    print(f"Peak reduction: {result['baseline_peak_mw'] -
        ↳ np.max(result['grid_power_mw']):.3f} MW")
    print(f"Final SOC: {result['soc'][-1]:.3f}")
    plot_dispatch(profile, result)

if __name__ == "__main__":
    main()

```

Run:

```
python track_d_grid_storage/peak_shaving_dispatch.py
```

The optimization decision contains one battery power value per hour plus one scalar peak target. Positive battery power means discharge into the load, reducing stored energy. Negative power means charging. The objective minimizes the peak target while penalizing violations and adding a small throughput penalty that stands in for degradation cost.

Expected output:

```

Baseline peak: about 2.4 to 2.8 MW
Optimized peak: lower by about 0.2 to 0.7 MW
Peak reduction: positive
Final SOC: between 0.15 and 0.90

```

The plot should show the battery charging near solar surplus or low-load hours and discharging near evening peaks. The SOC panel should remain inside the allowed range. If SOC hits both bounds aggressively, the battery is being fully used. If SOC barely moves, the degradation penalty is too large or the peak is not binding.

What could go wrong

If SLSQP reports incompatible constraints, reduce the requested flexibility by increasing battery capacity or lowering the initial peak target indirectly by leaving the initial guess unchanged.

If the optimizer returns no battery use, set `degradation_weight = 0.0` temporarily. If dispatch appears then, your degradation penalty was dominating the peak objective.

If the grid power is larger after dispatch, inspect the sign convention. In this script positive `battery_power_mw` adds to `net_load_mw` as grid-facing demand because power is defined from grid to battery. If you prefer positive discharge, flip the sign consistently and rename the variable.

Reflection

Grid-storage modeling makes the battery part of an economic and operational system. The cell model still matters, but the publication figure is often dispatch, SOC, revenue, peak reduction, or degradation tradeoff. For sodium-ion, this is a natural application space because cost, safety, and stationary storage tolerance may matter more than gravimetric energy density.

13.8.3 Track D Open-Ended Exercises

1. Add time-of-use energy prices and minimize cost instead of peak. Hint: the energy cost is `sum(price * grid_power * dt)`.
2. Add a degradation cost proportional to squared C-rate. Hint: this penalizes high-power bursts more strongly than throughput.
3. Compare lithium-ion and sodium-ion assumptions by changing round-trip efficiency, usable SOC window, and degradation penalty.

13.9 Track E: EV Powertrain Integration

13.9.1 Track E Opening

Track E connects BMS-scale battery models to vehicle loads. In an EV study, current is not an arbitrary profile. It is produced by road load, vehicle mass, acceleration, drivetrain efficiency, auxiliary loads, and regenerative braking limits. This track gives you a laptop version of the workflow you would later move into Simulink or Simscape: drive cycle to wheel power, wheel power to battery power,

battery power to SOC and voltage.

The model is longitudinal and one-dimensional. That is enough to answer capstone questions about pack sizing, sodium-ion range penalties, low-temperature power limits, or estimator robustness under realistic drive-cycle excitation.

13.9.2 Guided Walkthrough E1: Drive Cycle to Battery Current

Learning objective: Convert a speed trace into battery power, current, voltage, and SOC using a vehicle model plus a Thevenin battery pack.

Create `track_e_ev_powertrain/drive_cycle_pack.py`:

```
from __future__ import annotations

from dataclasses import dataclass

import matplotlib.pyplot as plt
import numpy as np

@dataclass
class VehicleParameters:
    mass_kg: float = 1650.0
    frontal_area_m2: float = 2.25
    drag_coefficient: float = 0.28
    rolling_resistance: float = 0.010
    air_density_kg_m3: float = 1.20
    wheel_radius_m: float = 0.31
    drivetrain_efficiency: float = 0.91
    regen_efficiency: float = 0.65
    aux_power_w: float = 700.0

@dataclass
class PackParameters:
    capacity_ah: float = 95.0
    nominal_voltage_v: float = 360.0
    resistance_ohm: float = 0.085
    initial_soc: float = 0.82
    min_soc: float = 0.05
    max_regen_power_w: float = 55000.0

def make_drive_cycle(duration_s: int = 1800) -> tuple[np.ndarray,
    ↪ np.ndarray]:
```



```

time_s = np.arange(0, duration_s + 1, 1.0)
speed_m_s = np.zeros_like(time_s)
for k, t in enumerate(time_s):
    urban = 10.0 + 7.0 * np.sin(2.0 * np.pi * t / 220.0)
    stop_wave = np.maximum(0.0, np.sin(2.0 * np.pi * t / 95.0))
    highway_burst = 13.0 * np.exp(-0.5 * ((t - 1050.0) / 260.0) ** 2)
    speed_m_s[k] = max(0.0, urban * stop_wave + highway_burst)
return time_s, speed_m_s

def pack_ocv(soc: np.ndarray, chemistry: str) -> np.ndarray:
    soc_clipped = np.clip(soc, 0.0, 1.0)
    if chemistry == "sodium":
        return 285.0 + 55.0 * soc_clipped + 18.0 * np.tanh((soc_clipped -
↪ 0.88) / 0.05)
    return 310.0 + 85.0 * soc_clipped + 12.0 * np.tanh((soc_clipped - 0.12) /
↪ 0.04)

def compute_wheel_power(
    time_s: np.ndarray,
    speed_m_s: np.ndarray,
    vehicle: VehicleParameters,
) -> np.ndarray:
    dt_s = np.diff(time_s, prepend=time_s[0])
    dt_s[0] = 1.0
    acceleration_m_s2 = np.gradient(speed_m_s, time_s)
    aero_force_n = 0.5 * vehicle.air_density_kg_m3 * vehicle.drag_coefficient
↪ * vehicle.frontal_area_m2 * speed_m_s**2
    rolling_force_n = vehicle.mass_kg * 9.81 * vehicle.rolling_resistance *
↪ np.ones_like(speed_m_s)
    inertial_force_n = vehicle.mass_kg * acceleration_m_s2
    wheel_force_n = aero_force_n + rolling_force_n + inertial_force_n
    return wheel_force_n * speed_m_s

def simulate_pack(
    time_s: np.ndarray,
    wheel_power_w: np.ndarray,
    vehicle: VehicleParameters,
    pack: PackParameters,
    chemistry: str,
) -> dict[str, np.ndarray]:
    dt_s = np.diff(time_s, prepend=time_s[0])
    dt_s[0] = 1.0

```

```

soc = np.zeros_like(time_s, dtype=float)
voltage_v = np.zeros_like(time_s, dtype=float)
current_a = np.zeros_like(time_s, dtype=float)
battery_power_w = np.zeros_like(time_s, dtype=float)
soc[0] = pack.initial_soc

for k in range(len(time_s)):
    if wheel_power_w[k] >= 0.0:
        battery_power_w[k] = wheel_power_w[k] /
↳ vehicle.drivetrain_efficiency + vehicle.aux_power_w
    else:
        regen_power = max(wheel_power_w[k] * vehicle.regen_efficiency,
↳ -pack.max_regen_power_w)
        battery_power_w[k] = regen_power + vehicle.aux_power_w

    ocv = pack.ocv(np.array([soc[k]]), chemistry=chemistry)[0]
    discriminant = ocv**2 - 4.0 * pack.resistance_ohm *
↳ battery_power_w[k]
    if discriminant <= 0.0:
        current_a[k] = ocv / (2.0 * pack.resistance_ohm)
    else:
        current_a[k] = (ocv - np.sqrt(discriminant)) / (2.0 *
↳ pack.resistance_ohm)
        voltage_v[k] = ocv - pack.resistance_ohm * current_a[k]

    if k < len(time_s) - 1:
        soc[k + 1] = soc[k] - current_a[k] * dt_s[k + 1] / (3600.0 *
↳ pack.capacity_ah)
        soc[k + 1] = np.clip(soc[k + 1], pack.min_soc, 1.0)

return {
    "soc": soc,
    "voltage_v": voltage_v,
    "current_a": current_a,
    "battery_power_w": battery_power_w,
}

def plot_vehicle_results(time_s: np.ndarray, speed_m_s: np.ndarray, results:
↳ dict[str, dict[str, np.ndarray]]) -> None:
    time_min = time_s / 60.0
    fig, axes = plt.subplots(4, 1, figsize=(9, 9), sharex=True)
    axes[0].plot(time_min, speed_m_s * 3.6, color="black", linewidth=1.5)
    axes[0].set_ylabel("Speed (km/h)")
    axes[0].grid(True, alpha=0.3)

```

```

    for label, result in results.items():
        axes[1].plot(time_min, result["battery_power_w"] / 1000.0,
↪ linewidth=1.4, label=label)
        axes[2].plot(time_min, result["voltage_v"], linewidth=1.4,
↪ label=label)
        axes[3].plot(time_min, result["soc"], linewidth=1.4, label=label)
    axes[1].set_ylabel("Battery power (kW)")
    axes[2].set_ylabel("Pack voltage (V)")
    axes[3].set_ylabel("SOC (-)")
    axes[3].set_xlabel("Time (min)")
    for ax in axes[1:]:
        ax.legend(loc="best")
        ax.grid(True, alpha=0.3)
    fig.tight_layout()
    plt.show()

def main() -> None:
    vehicle = VehicleParameters()
    lithium_pack = PackParameters(capacity_ah=95.0, nominal_voltage_v=360.0,
↪ resistance_ohm=0.080)
    sodium_pack = PackParameters(capacity_ah=88.0, nominal_voltage_v=320.0,
↪ resistance_ohm=0.105)
    time_s, speed_m_s = make_drive_cycle()
    wheel_power_w = compute_wheel_power(time_s, speed_m_s, vehicle)
    results = {
        "lithium-ion pack": simulate_pack(time_s, wheel_power_w, vehicle,
↪ lithium_pack, "lithium"),
        "sodium-ion pack": simulate_pack(time_s, wheel_power_w, vehicle,
↪ sodium_pack, "sodium"),
    }
    for label, result in results.items():
        energy_kwh = np.trapz(result["battery_power_w"], time_s) / 3.6e6
        print(f"{label}")
        print(f"  Energy used: {energy_kwh:.2f} kWh")
        print(f"  Final SOC: {result['soc'][-1]:.3f}")
        print(f"  Minimum voltage: {result['voltage_v'].min():.1f} V")
    plot_vehicle_results(time_s, speed_m_s, results)

if __name__ == "__main__":
    main()

```

Run:

```
python track_e_ev_powertrain/drive_cycle_pack.py
```

The code converts speed into acceleration, aerodynamic drag, rolling resistance, and inertial force. Wheel power is then mapped to battery power using drivetrain efficiency and regenerative-braking efficiency. The pack model solves the quadratic relation $P = VI = (U - RI)I$ for current at each time step.

Expected output:

```
lithium-ion pack
  Energy used: a few kWh
  Final SOC: slightly below 0.82
  Minimum voltage: above 300 V
sodium-ion pack
  Energy used: similar kWh
  Final SOC: lower than lithium case if capacity is lower
  Minimum voltage: lower than lithium case
```

The plot should show speed in the top panel, battery power spikes during acceleration, voltage sag during high-power events, and SOC declining slowly over the 30-minute drive. A wrong result often shows negative SOC change during acceleration, which means the current sign convention or power sign has been flipped.

Reflection

This exercise gives your BMS algorithms a realistic load source. It also gives sodium-ion vehicle studies a sober framing: lower pack voltage, different resistance, and lower usable capacity can be studied quantitatively, not only discussed qualitatively.

13.9.3 Track E Open-Ended Exercises

1. Add road grade to the wheel-force equation. Hint: grade force is $mg \sin(\theta)$.
2. Impose a sodium-ion low-temperature power limit and quantify the drive-cycle speed segments where demanded power exceeds available power.
3. Export the current trace and feed it into the SOC estimator from Track A.

13.10 Track F: Fast Charging Optimization

13.10. Track F Opening

Track F is the most natural bridge into a sodium-ion simulation capstone. Fast charging is a constrained-control problem. The controller wants high current because high current reduces time. The cell objects because voltage, temperature, diffusion gradients, plating risk, and degradation all worsen when current is too aggressive. Lithium-ion fast-charge optimization is a mature field; sodium-ion fast-charge optimization is younger, which makes careful simulation studies valuable.

The reproduction anchor is Attia et al., “Closed-loop optimization of fast-charging protocols for batteries with machine learning,” published in *Nature* in 2020. That paper optimized six-step, ten-minute charging protocols using early cycle-life prediction and Bayesian optimization. We will not reproduce the full closed-loop campaign. Instead, we reproduce the protocol-design idea: represent a fast-charge policy as a small number of current steps, evaluate it with a cell model, and compare charging time, temperature rise, and constraint violation.

13.10.2 Guided Walkthrough F1: Step-Protocol Fast-Charge Optimization

Learning objective: Use nonlinear optimization to find a multi-step charging protocol that reaches a target SOC quickly without exceeding voltage and temperature limits.

Create `track_f_fast_charging/fast_charge_optimizer.py`:

```
from __future__ import annotations

from dataclasses import dataclass

import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import differential_evolution

@dataclass
class ChargeModel:
    capacity_ah: float = 2.5
    r0_ohm: float = 0.045
    thermal_mass_j_k: float = 520.0
    heat_transfer_w_k: float = 1.15
    ambient_c: float = 25.0
    initial_soc: float = 0.10
```

```

target_soc: float = 0.80
voltage_limit_v: float = 3.95
temperature_limit_c: float = 42.0
max_charge_current_a: float = 7.5

def sodium_ocv(soc: np.ndarray) -> np.ndarray:
    soc_clipped = np.clip(soc, 0.0, 1.0)
    return 2.55 + 0.62 * soc_clipped + 0.10 * np.tanh((soc_clipped - 0.12) /
        ↳ 0.035) + 0.20 * np.tanh((soc_clipped - 0.90) / 0.045)

def simulate_protocol(currents_a: np.ndarray, durations_s: np.ndarray, model:
    ↳ ChargeModel) -> dict[str, np.ndarray]:
    dt_s = 2.0
    time_values = [0.0]
    soc_values = [model.initial_soc]
    temp_values = [model.ambient_c]
    voltage_values = [sodium_ocv(np.array([model.initial_soc]))[0]]
    current_values = [0.0]

    time_s = 0.0
    soc = model.initial_soc
    temperature_c = model.ambient_c

    for current_a, duration_s in zip(currents_a, durations_s):
        n_steps = int(np.ceil(duration_s / dt_s))
        for _ in range(n_steps):
            if soc >= model.target_soc:
                break
            charge_current_a = max(0.0, current_a)
            soc += charge_current_a * dt_s / (3600.0 * model.capacity_ah)
            soc = min(soc, 1.0)
            ocv = sodium_ocv(np.array([soc]))[0]
            voltage_v = ocv + model.r0_ohm * charge_current_a
            heat_w = charge_current_a**2 * model.r0_ohm
            dtemp_dt = (heat_w - model.heat_transfer_w_k * (temperature_c -
    ↳ model.ambient_c)) / model.thermal_mass_j_k
            temperature_c += dtemp_dt * dt_s
            time_s += dt_s
            time_values.append(time_s)
            soc_values.append(soc)
            temp_values.append(temperature_c)
            voltage_values.append(voltage_v)
            current_values.append(-charge_current_a)

```

```

        if soc >= model.target_soc:
            break

    return {
        "time_s": np.asarray(time_values),
        "soc": np.asarray(soc_values),
        "temperature_c": np.asarray(temp_values),
        "voltage_v": np.asarray(voltage_values),
        "current_a": np.asarray(current_values),
    }

def protocol_objective(decision: np.ndarray, model: ChargeModel) -> float:
    currents_a = decision[:4]
    durations_s = decision[4:]
    result = simulate_protocol(currents_a, durations_s, model)
    final_soc = result["soc"][-1]
    charge_time_s = result["time_s"][-1]
    voltage_violation = np.maximum(result["voltage_v"] -
    ↪ model.voltage_limit_v, 0.0)
    temp_violation = np.maximum(result["temperature_c"] -
    ↪ model.temperature_limit_c, 0.0)
    soc_shortfall = max(model.target_soc - final_soc, 0.0)
    roughness = np.sum(np.diff(currents_a) ** 2)
    return (
        charge_time_s / 60.0
        + 5000.0 * soc_shortfall**2
        + 800.0 * np.max(voltage_violation) ** 2
        + 30.0 * np.max(temp_violation) ** 2
        + 0.02 * roughness
    )

def optimize_protocol(model: ChargeModel) -> tuple[np.ndarray, np.ndarray,
    ↪ dict[str, np.ndarray]]:
    bounds = [(0.5, model.max_charge_current_a)] * 4 + [(180.0, 900.0)] * 4
    result = differential_evolution(
        lambda x: protocol_objective(x, model),
        bounds=bounds,
        seed=12,
        maxiter=70,
        popsize=12,
        polish=True,
        tol=1.0e-5,
        updating="immediate",
    )

```

```

)
currents_a = result.x[:4]
durations_s = result.x[4:]
simulation = simulate_protocol(currents_a, durations_s, model)
return currents_a, durations_s, simulation

def make_baseline(model: ChargeModel) -> dict[str, np.ndarray]:
    currents_a = np.array([2.5, 2.5, 2.5, 2.5])
    durations_s = np.array([900.0, 900.0, 900.0, 900.0])
    return simulate_protocol(currents_a, durations_s, model)

def plot_fast_charge(baseline: dict[str, np.ndarray], optimized: dict[str,
↳ np.ndarray], model: ChargeModel) -> None:
    fig, axes = plt.subplots(4, 1, figsize=(9, 9), sharex=True)
    for label, result in [("baseline", baseline), ("optimized", optimized)]:
        time_min = result["time_s"] / 60.0
        axes[0].step(time_min, result["current_a"], where="post",
↳ linewidth=1.5, label=label)
        axes[1].plot(time_min, result["soc"], linewidth=1.5, label=label)
        axes[2].plot(time_min, result["voltage_v"], linewidth=1.5,
↳ label=label)
        axes[3].plot(time_min, result["temperature_c"], linewidth=1.5,
↳ label=label)
        axes[2].axhline(model.voltage_limit_v, color="black", linestyle="--",
↳ linewidth=1.0)
        axes[3].axhline(model.temperature_limit_c, color="black", linestyle="--",
↳ linewidth=1.0)
        axes[0].set_ylabel("Current (A)")
        axes[1].set_ylabel("SOC (-)")
        axes[2].set_ylabel("Voltage (V)")
        axes[3].set_ylabel("Temp. (deg C)")
        axes[3].set_xlabel("Time (min)")
        for ax in axes:
            ax.grid(True, alpha=0.3)
            ax.legend(loc="best")
    fig.tight_layout()
    plt.show()

def main() -> None:
    model = ChargeModel()
    currents_a, durations_s, optimized = optimize_protocol(model)
    baseline = make_baseline(model)

```



```

print("Optimized protocol:")
for index, (current, duration) in enumerate(zip(currents_a, durations_s),
↳ start=1):
    print(f" Step {index}: {current:.2f} A for {duration / 60.0:.1f}
↳ min")
for label, result in [("baseline", baseline), ("optimized", optimized)]:
    print(label)
    print(f" Charge time: {result['time_s'][-1] / 60.0:.1f} min")
    print(f" Final SOC: {result['soc'][-1]:.3f}")
    print(f" Peak voltage: {result['voltage_v'].max():.3f} V")
    print(f" Peak temperature: {result['temperature_c'].max():.2f} deg
↳ C")
plot_fast_charge(baseline, optimized, model)

if __name__ == "__main__":
    main()

```

Run:

```
python track_f_fast_charging/fast_charge_optimizer.py
```

The optimizer chooses four current levels and four step durations. In the plot, current is negative because the companion's canonical sign convention treats charge current as negative. Inside the optimizer, we use positive `charge_current_a` to keep the charging equations readable, then store the reported current as negative.

Expected output:

```

Optimized protocol:
  Step 1: high current for several minutes
  Step 2: moderate/high current
  Step 3: lower current as voltage rises
  Step 4: lower or unused step
baseline
  Charge time: longer
  Final SOC: near 0.80
  Peak voltage: below or near 3.95 V
  Peak temperature: below 42 C
optimized
  Charge time: shorter
  Final SOC: near 0.80
  Peak voltage: near the limit
  Peak temperature: near or below the limit

```

The correct optimized result should look like a constrained-control solution: aggressive early, gentler later. If it uses maximum current all the way through while violating voltage and temperature, the penalty weights are too small. If it barely charges, the SOC shortfall penalty is too small.

What could go wrong

If optimization is slow, lower `max_iter` to 25 while debugging. Restore it for final runs.

If the optimized protocol is not better than baseline, run again with a different seed. Differential evolution is stochastic.

If voltage exceeds the limit by a large amount, increase the `800.0` penalty multiplier.

If the protocol is physically strange, such as alternating high-low-high-low currents, increase the roughness penalty.

Reflection

This is the smallest useful fast-charge optimization loop: parameterize a protocol, simulate it, penalize constraint violations, and compare with a baseline. A publishable version would replace the simple ECM/thermal model with PyBaMM SPM or DFN, include plating or sodium-specific degradation constraints, and validate the assumptions against experimental data.

13.10.3 Track F Reproduction Exercise

Reproduce the qualitative structure of Attia et al. Figure 2 from the 2020 closed-loop fast-charging paper: a multi-step charging protocol represented by step currents over a fixed fast-charge window. Your reproduction does not need their proprietary experimental outcome model. It must show:

1. a baseline constant-current protocol,
2. at least three candidate multi-step protocols,
3. the resulting SOC, voltage, and temperature trajectories from your model,
4. a table ranking the protocols by charge time and constraint margin.

Where the paper is ambiguous for your simplified model, document your choice. For example, if you do not know the exact cell resistance, choose a plausible value, run a sensitivity case, and say so. “Close enough” means your reproduction captures the experimental-design logic of multi-step protocol search,

not the exact cycle-life results of the original campaign.

13.10.4 Track F Open-Ended Exercises

1. Replace the ECM with a PyBaMM SPM or SPM_e model. Hint: use `pybamm.Experiment` with current steps and read terminal voltage and temperature if thermal coupling is enabled.
2. Add a sodium-ion low-temperature case by setting `ambient_c = 0.0` and changing resistance. Quantify how the optimized protocol changes.
3. Convert the open-loop protocol optimizer into receding-horizon MPC. Hint: optimize only the next four steps, apply the first, update the state, and repeat.

What Changes for Sodium-Ion?

Sodium-ion specialization is not a find-and-replace operation on lithium-ion workflows. Each track changes in a different way.

In state estimation, hard-carbon OCV plateaus and hysteresis can reduce voltage-based SOC observability. A filter that looks excellent on an NMC/graphite OCV curve may become uncertain or biased on a sodium-ion curve. That does not mean the filter is wrong; it means the measurement is less informative.

In thermal and safety work, sodium-ion cells may offer different abuse behavior depending on cathode, anode, electrolyte, format, and state of charge. You should not claim sodium-ion safety superiority from a generic Arrhenius model. Use the model to structure questions, then cite chemistry-specific abuse data when making claims.

In data-driven health estimation, the largest public benchmark datasets are still lithium-ion. For sodium-ion, use lithium-ion data to validate the workflow and sodium-ion data to test chemistry-specific hypotheses. Keep those claims separate.

In grid storage, sodium-ion assumptions may be favorable: cost, material abundance, safety tolerance, and stationary volume constraints can matter more than gravimetric energy density. Dispatch models are a good place to quantify how much lower efficiency or lower energy density can be tolerated.

In EV integration, sodium-ion penalties show up directly as pack mass, pack volume, voltage sag, power limits, and range. A fair simulation should compare at the pack level, not merely at the cell nominal voltage level.

In fast charging, sodium-ion may benefit from different low-temperature kinetics and may suffer different degradation limits. Replace lithium-specific plating constraints with sodium-specific constraints only when you have a defensible source. If you do not, call the constraint a proxy.

Chapter Summary and Skill Checklist

- You learned how to choose a specialization track based on a capstone research direction.
- You implemented a particle filter for joint SOC and capacity estimation.
- You simulated thermal-runaway threshold behavior with an open finite-difference model.
- You trained a small LSTM and compared it against a simple health-prediction baseline.
- You optimized a grid battery dispatch schedule under SOC and power constraints.
- You converted a drive cycle into EV battery current, voltage, and SOC.
- You optimized a multi-step sodium-ion fast-charge protocol under voltage and temperature limits.
- You practiced separating workflow validation on lithium-ion data from sodium-ion case-study claims.

Commands, functions, and patterns that should now feel familiar:

- `systematic_resample`, weighted quantiles, and effective particle count
- Arrhenius heat-release terms and stiff `solve_ivp` calls with `method="BDF"`
- cell-level train/validation/test splits for battery ML
- `torch.utils.data.TensorDataset`, `DataLoader`, and early stopping
- constrained dispatch with `scipy.optimize.minimize`
- drive-cycle road-load equations
- protocol optimization with `scipy.optimize.differential_evolution`

You should now be able to:

- Explain why flat OCV regions make SOC estimation harder.
- Build a particle filter and interpret its uncertainty band.

- Write a simple thermal safety model without confusing it for a certified safety tool.
- Train a battery health model without leaking test data into training.
- Formulate a grid-storage dispatch problem with energy and power constraints.
- Generate a battery current profile from a vehicle speed trace.
- Define and optimize a multi-step charging protocol.
- State what must change before a lithium-ion workflow can support a sodium-ion claim.

Deliverable

The deliverable for this chapter is a completed track-specific project with documentation. Choose one track and create a folder named:

```
|
↳ SimulationCompanion/chapter13_specialization_tracks/results/track_<letter>_project
```

Your project folder should contain:

- a runnable script or notebook,
- at least one figure suitable for a technical report,
- a short `README.md` explaining the research question, assumptions, and how to run the code,
- a validation paragraph comparing your result with a baseline or published reference,
- a sodium-ion adaptation paragraph explaining what changed from the lithium-ion or generic workflow.

A good Track F deliverable, for example, would compare CCCV, constant-current, and optimized four-step sodium-ion charging under the same voltage and temperature limits. A good Track A deliverable would compare EKF and particle-filter SOC estimation under a sodium-like OCV plateau and current bias. A good Track C deliverable would reproduce an early-cycle feature plot from the MATR dataset and then test whether the same code can parse a sodium-ion cycling dataset.

13.11 Capstone Pre-Proposal Template

Before starting Chapter 14, write one page using this structure:

```
# Capstone Pre-Proposal

## Working Title

## Primary Track

## Research Question

## Model or Dataset

## Baseline

## Sodium-Ion Adaptation

## Validation Target

## Expected Figure 1

## Main Risk

## Definition of Done
```

The most important lines are **Research Question**, **Baseline**, and **Validation Target**. If those are vague, the capstone will drift. If those are sharp, the rest can evolve.

Further Practice and Reading

Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, and coauthors, “Closed-loop optimization of fast-charging protocols for batteries with machine learning,” *Nature* 578, 397-402, 2020, <https://doi.org/10.1038/s41586-020-1994-5>.

Kristen A. Severson, Peter M. Attia, Norman Jin, and coauthors, “Data-driven prediction of battery cycle life before capacity degradation,” *Nature Energy* 4, 383-391, 2019, <https://doi.org/10.1038/s41560-019-0356-8>. Dataset: <https://data.matr.io/1>.

PyTorch official previous-version installation guide for pinned CPU and CUDA wheels: <https://docs.pytorch.org/get-started/previous-versions/>.

COMSOL Battery Design Module overview, especially the sections on porous electrodes, battery packs, short-circuiting, and thermal runaway propagation: <https://www.comsol.com/battery-design-module>.

PyBaMM documentation and examples for replacing the simplified fast-charge model in Track F with SPM, SPMe, or DFN simulations: <https://docs.pybamm.org/>.

The next chapter is Lab Chapter 14: The Capstone Project.

The Capstone Project

This chapter is where the companion stops giving you isolated exercises and asks you to behave like a researcher.

Up to now, each lab chapter has protected you from one kind of chaos. PyBaMM chapters protected you from writing a battery model from scratch before you understood the tool. MATLAB chapters protected you from state-estimation abstraction by letting you build ECMs and filters one piece at a time. Dataset chapters protected you from public-data disorder by teaching file conventions, sign conventions, cycle detection, and reproducible loaders. The reproduction chapter protected you from the false confidence that comes from running only examples written for you.

The capstone removes some of that scaffolding. You will choose a modest sodium-ion research question, position it against the literature, design a simulation study, run an organized set of experiments, analyze the results, and produce four artifacts: a reproducible GitHub-style repository, an 8-12 page technical report in IEEE conference style, a one-page poster, and a research log that records decisions and pivots. The project is intentionally small. It is not supposed to become a dissertation in one chapter. It is supposed to be a complete dry run of the research process.

Keep Textbook Chapter 8 open for physics-based model assumptions, Textbook

Chapter 10 open for equivalent-circuit and BMS language, Textbook Chapter 11 open for thermal modeling, and the sodium-ion chapter open for chemistry-specific interpretation. This chapter operationalizes the whole theory textbook, especially the difference between a numerical result and a defensible research claim. A result says, “my script produced a curve.” A research claim says, “under these assumptions, with these parameter ranges, this performance metric changes in this direction, and here is why the conclusion survives reasonable checks.”

Because many readers will still not have access to sodium-ion laboratory data, the main guided project uses a transparent virtual sodium-ion cell rather than pretending that a public dataset solves every validation problem. We will build a first-order equivalent-circuit model with a hard-carbon-like OCV curve, temperature-dependent resistance, a lumped thermal state, and a protocol runner for discharge and pulse experiments. This model is simpler than a DFN, but it is honest, fast, and fully inspectable. You will use it to study a capstone question that is realistic enough to matter: how do hard-carbon OCV shape, internal resistance, and ambient temperature affect usable energy and voltage-limit behavior under pulsed loads?

That question is not the only valid capstone. It is the worked spine of this chapter. Once you can run it end to end, you can replace the virtual cell with a PyBaMM SPMe or DFN workflow, an ECM identified from CALCE or NASA lithium-ion data, a sodium-ion OCV curve digitized from a paper, or a fast-charge optimization study from Lab Chapter 13. The important thing is not this exact model. The important thing is the research workflow: scope, literature positioning, methodology, organized execution, analysis, figure production, writing, poster design, and self-review.

14.1 Prerequisites Check

- Required software: Python 3.11, numpy==1.26.4, scipy==1.13.1, pandas==2.2.2, matplotlib==3.9.0, seaborn==0.13.2, jinja2==3.1.4, and pyyaml==6.0.2
- Optional software: pybamm==26.3.1 if you want to extend the capstone with a physics-based simulation; a LaTeX distribution or Overleaf account if you want to compile the IEEE-style report template

- Install command: `python -m pip install numpy==1.26.4 scipy==1.13.1 pandas==2.2.2 matplotlib==3.9.0 seaborn==0.13.2 jinja2==3.1.4 pyyaml==6.0.2`
- Required textbook chapters: Textbook Chapters 8, 10, 11, and the sodium-ion chapter
- Required prior lab chapters: Lab Chapters 1, 2, 4, 6, 9, 11, and 12 are essential; Lab Chapters 3, 5, 7, 8, 10, and 13 are strongly recommended depending on your chosen topic
- Public data background: Lab Chapter 11 for CALCE, NASA, Oxford, Severson/MATR, Mendeley, Zenodo, and sodium-ion data limitations
- Estimated time: 20 to 35 hours for the guided capstone; 40 to 80 hours for a polished independent variant

If your Python ODE workflow feels rusty, revisit Lab Chapter 2 before starting. If equivalent-circuit language feels shaky, revisit Lab Chapter 6. If you are tempted to make the project enormous, reread the scoping section of this chapter twice. Ambition is useful only after the project can still be finished.

14.2 Environment Setup

The capstone environment is deliberately boring. We will use a small stack that is stable, easy to install, and adequate for a publishable-methods dry run. The main model uses `scipy.solve_ivp` rather than a battery-specific solver because the capstone is about research execution, not about hiding the model inside a package. If you extend the project with PyBaMM, use the environment from Lab Chapters 3-5 or 9 and record the version in your research log.

Create a fresh environment from the repository root:

```
cd /home/avirup/SodiumIonBatteryResearch
python3.11 -m venv .venv-chapter14
source .venv-chapter14/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==1.26.4 scipy==1.13.1 pandas==2.2.2
↪ matplotlib==3.9.0 seaborn==0.13.2 jinja2==3.1.4 pyyaml==6.0.2
```

On Windows PowerShell, use:

```
cd C:\path\to\SodiumIonBatteryResearch
py -3.11 -m venv .venv-chapter14
.\.venv-chapter14\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
python -m pip install numpy==1.26.4 scipy==1.13.1 pandas==2.2.2
↳ matplotlib==3.9.0 seaborn==0.13.2 jinja2==3.1.4 pyyaml==6.0.2
```

Create the capstone workspace:

```
mkdir -p SimulationCompanion/chapter14_capstone_workspace
cd SimulationCompanion/chapter14_capstone_workspace
mkdir -p config data/raw data/processed figures logs reports src tables
touch logs/research-log.md
```

Verify the installation:

```
import jinja2
import matplotlib
import numpy as np
import pandas as pd
import scipy
import seaborn as sns
import yaml

print("NumPy:", np.__version__)
print("SciPy:", scipy.__version__)
print("pandas:", pd.__version__)
print("Matplotlib:", matplotlib.__version__)
print("seaborn:", sns.__version__)
print("Jinja2:", jinja2.__version__)
print("PyYAML:", yaml.__version__)

time_s = np.linspace(0.0, 10.0, 6)
voltage_v = 3.1 - 0.02 * time_s
print(pd.DataFrame({"time_s": time_s, "voltage_v": voltage_v}))
```

Expected output:

```
NumPy: 1.26.4
SciPy: 1.13.1
pandas: 2.2.2
Matplotlib: 3.9.0
seaborn: 0.13.2
Jinja2: 3.1.4
PyYAML: 6.0.2
   time_s  voltage_v
0      0.0        3.10
1      2.0        3.06
2      4.0        3.02
```

| | | |
|---|------|------|
| 3 | 6.0 | 2.98 |
| 4 | 8.0 | 2.94 |
| 5 | 10.0 | 2.90 |

Common setup failures are ordinary here. `ModuleNotFoundError` means the notebook kernel is not the environment you installed into. Install a kernel with `python -m ipykernel install --user --name battery-chapter14 --display-name "Battery Chapter 14"` and select it in Jupyter. A plotting backend warning usually means you are running on a headless system; use `plt.savefig(...)` as the chapter code does. A LaTeX compile error is not a Python failure. The report template will still be generated as `.tex`, and you can compile it later in Overleaf.

14.3 Conceptual Bridge: From Labs to a Research Artifact

In the theory textbook, you learned to ask what a battery does. In the lab companion, you learned to ask how a tool represents what the battery does. In the capstone, the question changes again: what can you claim, and why should anyone believe you?

A publishable simulation project has five layers. The first layer is the physical question. For example, “Does a hard-carbon sodium-ion OCV plateau make SOC estimation harder?” or “How much usable energy is lost at low temperature for a pulsed load?” The second layer is the model. This may be an ECM, an SPMe, a DFN, a thermal model, a degradation model, or a hybrid. The third layer is the protocol: what current, voltage limits, temperature boundary conditions, and parameter variations you impose. The fourth layer is the analysis: what metrics you compute and what comparisons you make. The fifth layer is the artifact: code, data, figures, tables, report, poster, and log.

Most weak simulation projects fail by mixing these layers. They change a parameter without saying what physical question it answers. They run a protocol without recording why it was chosen. They show a beautiful voltage curve but never define a metric. They report a finding without checking whether it survives a reasonable perturbation. The capstone workflow keeps the layers separate enough that a reviewer can audit them.

The model in this chapter is a first-order Thevenin ECM with a lumped thermal state. Its states are SOC z , RC polarization voltage v_1 , and cell temperature T . We

use the discharge-positive current convention from earlier chapters. The continuous-time equations are

$$\frac{dz}{dt} = -\frac{\eta I(t)}{3600 Q_{\text{Ah}}}, \quad (14.1)$$

$$\frac{dv_1}{dt} = -\frac{v_1}{R_1 C_1} + \frac{I(t)}{C_1}, \quad (14.2)$$

$$C_{\text{th}} \frac{dT}{dt} = I(t)^2 (R_0(T, z) + R_1) - hA (T - T_{\infty}). \quad (14.3)$$

The terminal voltage is

$$V(t) = U_{\text{ocv}}(z, T) - I(t)R_0(T, z) - v_1. \quad (14.4)$$

Equation (14.1) is Coulomb counting. Equation (14.2) is the polarization branch you used in Lab Chapter 6. Equation (14.3) is the lumped thermal balance from Lab Chapter 9, simplified to irreversible heat. Equation (14.4) is the ECM voltage equation. This is not a full electrochemical sodium-ion model. It cannot predict electrolyte concentration, particle diffusion, plating, SEI growth, or electrode-specific potentials. That limitation is a feature for the capstone: every assumption is visible.

The sodium-ion content enters through three choices. First, the OCV curve is hard-carbon-like: it includes a sloping region and a low-voltage plateau, rather than borrowing a graphite lithium-ion OCV shape. Second, nominal voltage and voltage limits are sodium-ion-like, roughly 1.5-3.8 V for a generic full cell. Third, the low-temperature resistance penalty is treated as a parameter to sweep, because public SIB validation data are sparse and chemistry-dependent. We will be explicit that the model is a virtual cell calibrated for method development, not a validated commercial cell.

This is standard practice when done honestly. Researchers often use virtual cells to test algorithms, isolate mechanisms, or design experiments before high-quality data exist. The habit to internalize is not “synthetic data are as good as experiments.” They are not. The habit is “synthetic studies can support methodological claims when their assumptions, parameter ranges, and validation limits are stated plainly.” That distinction will keep your capstone from overclaiming.

Guided Walkthrough 1

Scope a Capstone as a Testable Research Question

Learning objective: Convert a broad sodium-ion interest into a small, testable capstone question with deliverables and decision rules.

Before writing code, we need a project definition that can survive contact with time. The most common capstone failure is starting with a theme instead of a question. “Sodium-ion fast charging” is a theme. “Under a fixed 10-minute pulsed-load profile, how do low-temperature resistance penalty and OCV plateau width change usable delivered energy before a 1.5 V cutoff?” is a question. The second one tells you what to simulate, what to vary, what to measure, and what figure might appear in the report.

Create config/capstone_scope.yaml:

```
project:
  title: "Pulsed-load usable energy in a virtual hard-carbon sodium-ion cell"
  short_name: "sib_pulsed_energy_capstone"
  research_question: >
    How do hard-carbon OCV plateau width, internal resistance, and ambient
    temperature affect usable energy and voltage-limit behavior under a
    repeated pulsed discharge load?
  claim_type: "simulation-method dry run with virtual sodium-ion parameters"
  primary_metric: "usable_energy_wh"
  secondary_metrics:
    - "minimum_voltage_v"
    - "peak_temperature_c"
    - "cutoff_time_s"
    - "voltage_sag_v"
  fixed_assumptions:
    cell_capacity_ah: 3.0
    lower_voltage_cutoff_v: 1.5
    upper_voltage_cutoff_v: 3.8
    initial_soc: 1.0
    initial_temperature_c: 25.0
    pulse_current_a: 6.0
    pulse_duration_s: 20.0
    rest_duration_s: 40.0
    number_of_pulses: 60
  factors:
    ambient_temperature_c: [0.0, 25.0, 45.0]
    resistance_scale: [0.8, 1.0, 1.3]
    plateau_width_scale: [0.75, 1.0, 1.25]
```

```

success_criteria:
  - "All simulations finish without numerical failure."
  - "Every figure and table is generated by a script."
  - "The report states that the sodium-ion cell is virtual, not
↪ experimentally validated."
  - "At least one sensitivity conclusion is supported by a table and a
↪ figure."

```

Now create `src/print_scope.py`:

```

from pathlib import Path

import yaml

def main() -> None:
    scope_path = Path("config/capstone_scope.yaml")
    with scope_path.open("r", encoding="utf-8") as file:
        scope = yaml.safe_load(file)

    project = scope["project"]
    print("Capstone title:")
    print(project["title"])
    print()
    print("Research question:")
    print(project["research_question"].strip())
    print()
    print("Primary metric:", project["primary_metric"])
    print("Factors:")
    for name, values in project["factors"].items():
        print(f"  - {name}: {values}")
    print()
    print("Success criteria:")
    for criterion in project["success_criteria"]:
        print(f"  - {criterion}")

if __name__ == "__main__":
    main()

```

Run:

```
python src/print_scope.py
```

Expected output:

```
Capstone title:
Pulsed-load usable energy in a virtual hard-carbon sodium-ion cell

Research question:
How do hard-carbon OCV plateau width, internal resistance, and ambient
temperature affect usable energy and voltage-limit behavior under a
repeated pulsed discharge load?

Primary metric: usable_energy_wh
Factors:
- ambient_temperature_c: [0.0, 25.0, 45.0]
- resistance_scale: [0.8, 1.0, 1.3]
- plateau_width_scale: [0.75, 1.0, 1.25]
```

The YAML file is not just configuration. It is a contract with yourself. If halfway through the capstone you want to add degradation, fast charging, pack imbalance, and a neural network estimator, this file reminds you what you promised to finish first. You can write a future-work paragraph later. You cannot write a paper from a project that never converged.

What could go wrong? If `yaml.safe_load` fails, check indentation. YAML is whitespace-sensitive, and tabs are a bad idea. If the script cannot find the file, run it from `SimulationCompanion/chapter14_capstone_workspace`, not from the repository root. If your research question still contains words like “optimize batteries” or “investigate sodium-ion performance” without a metric, rewrite it before coding.

This exercise taught the first capstone habit: a project starts as a falsifiable workflow, not as a folder full of scripts. We will reuse the scope file in later walkthroughs so that the code and report inherit the same assumptions.

Guided Walkthrough 2

Build the Virtual Sodium-Ion Cell Model

Learning objective: Implement a transparent ECM-thermal sodium-ion virtual cell that can be run under arbitrary current profiles.

We now build the model. The code is longer than a notebook toy because it has to become part of a capstone repository. We will use dataclasses for parameters, explicit functions for OCV and resistance, and a single simulation function that


```

plateau = 1.92 + 0.10 * np.tanh((z - 0.28) / (0.16 *
↳ plateau_width_scale))
high_soc_rise = 0.78 / (1.0 + np.exp(-(z - 0.78) / 0.060))
gentle_slope = 0.30 * z

return plateau + low_soc_rise + high_soc_rise + gentle_slope

def ohmic_resistance(
    soc: float,
    temperature_c: float,
    parameters: CellParameters,
) -> float:
    """Return SOC- and temperature-dependent ohmic resistance."""
    z = float(np.clip(soc, 0.0, 1.0))
    low_soc_penalty = 1.0 + parameters.resistance_soc_coeff * (1.0 - z) ** 2
    delta_t = parameters.reference_temperature_c - temperature_c
    temperature_penalty =
↳ np.exp(parameters.resistance_temperature_coeff_per_c * delta_t)
    return (
        parameters.r0_ohm
        * parameters.resistance_scale
        * low_soc_penalty
        * temperature_penalty
    )

def make_pulsed_current_profile(
    pulse_current_a: float,
    pulse_duration_s: float,
    rest_duration_s: float,
    number_of_pulses: int,
) -> tuple[np.ndarray, np.ndarray]:
    """Create a repeated discharge-pulse/rest current profile."""
    times = [0.0]
    currents = [0.0]
    current_time = 0.0

    for _ in range(number_of_pulses):
        times.extend([current_time, current_time + pulse_duration_s])
        currents.extend([pulse_current_a, pulse_current_a])
        current_time += pulse_duration_s

    times.extend([current_time, current_time + rest_duration_s])
    currents.extend([0.0, 0.0])

```

```

        current_time += rest_duration_s

    return np.asarray(times, dtype=float), np.asarray(currents, dtype=float)

def make_constant_current_profile(
    current_a: float,
    duration_s: float,
) -> tuple[np.ndarray, np.ndarray]:
    """Create a constant-current profile with discharge-positive
    ↪ convention."""
    return np.asarray([0.0, duration_s]), np.asarray([current_a, current_a])

def interpolate_current(
    profile_time_s: np.ndarray,
    profile_current_a: np.ndarray,
) -> Callable[[float], float]:
    """Build a zero-order-hold-like current interpolation function."""
    def current_function(time_s: float) -> float:
        return float(np.interp(time_s, profile_time_s, profile_current_a))

    return current_function

def simulate_cell(
    profile_time_s: np.ndarray,
    profile_current_a: np.ndarray,
    parameters: CellParameters,
    ambient_temperature_c: float,
    initial_soc: float = 1.0,
    initial_temperature_c: float = 25.0,
    sample_period_s: float = 1.0,
) -> pd.DataFrame:
    """Simulate the virtual cell and return a tidy time-series table."""
    current = interpolate_current(profile_time_s, profile_current_a)
    end_time_s = float(profile_time_s[-1])
    evaluation_time_s = np.arange(0.0, end_time_s + sample_period_s,
    ↪ sample_period_s)

    def rhs(time_s: float, state: np.ndarray) -> list[float]:
        soc, polarization_v, temperature_c = state
        applied_current_a = current(time_s)
        r0 = ohmic_resistance(soc, temperature_c, parameters)

```

```

dsoc_dt = (
    -parameters.coulombic_efficiency
    * applied_current_a
    / (3600.0 * parameters.capacity_ah)
)
dpolarization_dt = (
    -polarization_v / (parameters.r1_ohm * parameters.cl_f)
    + applied_current_a / parameters.cl_f
)
heat_generation_w = applied_current_a**2 * (r0 + parameters.r1_ohm)
heat_rejection_w = parameters.heat_transfer_w_per_k * (
    temperature_c - ambient_temperature_c
)
dtemperature_dt = (
    heat_generation_w - heat_rejection_w
) / parameters.thermal_capacity_j_per_k

return [dsoc_dt, dpolarization_dt, dtemperature_dt]

solution = solve_ivp(
    rhs,
    t_span=(0.0, end_time_s),
    y0=[initial_soc, 0.0, initial_temperature_c],
    t_eval=evaluation_time_s,
    method="BDF",
    rtol=1e-7,
    atol=1e-9,
)

if not solution.success:
    raise RuntimeError(f"Cell simulation failed: {solution.message}")

soc = np.clip(solution.y[0], 0.0, 1.0)
polarization_v = solution.y[1]
temperature_c = solution.y[2]
current_a = np.asarray([current(t) for t in solution.t])
r0_ohm = np.asarray(
    [
        ohmic_resistance(z, temp_c, parameters)
        for z, temp_c in zip(soc, temperature_c)
    ]
)
ocv_v = hard_carbon_ocv(soc, parameters.plateau_width_scale)
terminal_voltage_v = ocv_v - current_a * r0_ohm - polarization_v
power_w = terminal_voltage_v * current_a

```

```

frame = pd.DataFrame(
    {
        "time_s": solution.t,
        "current_a": current_a,
        "soc": soc,
        "ocv_v": ocv_v,
        "r0_ohm": r0_ohm,
        "polarization_v": polarization_v,
        "terminal_voltage_v": terminal_voltage_v,
        "temperature_c": temperature_c,
        "ambient_temperature_c": ambient_temperature_c,
        "power_w": power_w,
    }
)

frame["energy_wh"] = (
    np.cumsum(frame["power_w"].to_numpy()) * sample_period_s / 3600.0
)

frame["below_cutoff"] = (
    frame["terminal_voltage_v"] <= parameters.lower_voltage_cutoff_v
)

return frame

def summarize_run(frame: pd.DataFrame, parameters: CellParameters) ->
    dict[str, float]:
    """Compute capstone metrics for one simulation run."""
    below_cutoff = frame["below_cutoff"].to_numpy()
    if below_cutoff.any():
        cutoff_index = int(np.argmax(below_cutoff))
        usable = frame.iloc[: cutoff_index + 1]
        cutoff_time_s = float(frame.loc[cutoff_index, "time_s"])
    else:
        usable = frame
        cutoff_time_s = float(frame["time_s"].iloc[-1])

    loaded = usable[usable["current_a"] > 0.0]
    if loaded.empty:
        voltage_sag_v = 0.0
    else:
        voltage_sag_v = float(
            (loaded["ocv_v"] - loaded["terminal_voltage_v"]).max()
        )

```

```

return {
    "usable_energy_wh": float(usable["energy_wh"].iloc[-1]),
    "minimum_voltage_v": float(usable["terminal_voltage_v"].min()),
    "peak_temperature_c": float(usable["temperature_c"].max()),
    "cutoff_time_s": cutoff_time_s,
    "final_soc": float(usable["soc"].iloc[-1]),
    "voltage_sag_v": voltage_sag_v,
    "lower_voltage_cutoff_v": parameters.lower_voltage_cutoff_v,
}

```

Now create a quick sanity script, `src/run_single_case.py`:

```

from pathlib import Path

import matplotlib.pyplot as plt

from sib_virtual_cell import (
    CellParameters,
    make_pulsed_current_profile,
    simulate_cell,
    summarize_run,
)

def main() -> None:
    Path("data/processed").mkdir(parents=True, exist_ok=True)
    Path("figures").mkdir(parents=True, exist_ok=True)

    parameters = CellParameters()
    profile_time_s, profile_current_a = make_pulsed_current_profile(
        pulse_current_a=6.0,
        pulse_duration_s=20.0,
        rest_duration_s=40.0,
        number_of_pulses=60,
    )

    result = simulate_cell(
        profile_time_s=profile_time_s,
        profile_current_a=profile_current_a,
        parameters=parameters,
        ambient_temperature_c=25.0,
        initial_soc=1.0,
        initial_temperature_c=25.0,
        sample_period_s=1.0,
    )

```

```

result.to_csv("data/processed/single_case_timeseries.csv", index=False)

summary = summarize_run(result, parameters)
for key, value in summary.items():
    print(f"{key}: {value:.4f}")

fig, axes = plt.subplots(4, 1, figsize=(8.0, 8.0), sharex=True)
axes[0].plot(result["time_s"] / 60.0, result["current_a"],
↪ color="tab:blue")
axes[0].set_ylabel("Current (A)")
axes[0].grid(True, alpha=0.3)

axes[1].plot(result["time_s"] / 60.0, result["terminal_voltage_v"],
↪ color="tab:red")
axes[1].axhline(parameters.lower_voltage_cutoff_v, color="black",
↪ linestyle="--")
axes[1].set_ylabel("Voltage (V)")
axes[1].grid(True, alpha=0.3)

axes[2].plot(result["time_s"] / 60.0, result["soc"], color="tab:green")
axes[2].set_ylabel("SOC (-)")
axes[2].grid(True, alpha=0.3)

axes[3].plot(result["time_s"] / 60.0, result["temperature_c"],
↪ color="tab:orange")
axes[3].set_ylabel("Temp. (deg C)")
axes[3].set_xlabel("Time (min)")
axes[3].grid(True, alpha=0.3)

fig.tight_layout()
fig.savefig("figures/single_case_timeseries.png", dpi=200)
plt.close(fig)

if __name__ == "__main__":
    main()

```

Run:

```
python src/run_single_case.py
```

Expected output will look like this, with small numerical differences depending on SciPy:

```
usable_energy_wh: 5x.xxxx  
minimum_voltage_v: 2.xxxx  
peak_temperature_c: 2x.xxxx  
cutoff_time_s: 3600.0000  
final_soc: 0.33xx  
voltage_sag_v: 0.xxxx  
lower_voltage_cutoff_v: 1.5000
```

The plot figures/single_case_timeseries.png should have four stacked panels. The current panel should show repeated square pulses: 6 A for 20 s, then 0 A for 40 s. The voltage panel should sag during each pulse and partially recover during rest. The SOC panel should decrease only during pulses and flatten during rests. The temperature panel should rise during the pulse train and approach a modest steady value. A wrong result would show SOC increasing during discharge, voltage rising during load, or temperature decreasing below ambient while current is applied.

What could go wrong? If the voltage looks too high, remember that the OCV is a virtual full-cell curve and inspect `hard_carbon_ocv` over SOC. If temperature explodes, your heat-transfer or resistance parameters are too severe. If the solver is slow, increase `sample_period_s` to 2.0 for exploratory work and return to 1.0 for final figures. If the cutoff triggers immediately, check that current is discharge-positive and that the lower cutoff is not accidentally set near the initial voltage.

This exercise taught the second capstone habit: keep the model in a reusable module, run a one-case sanity check, and inspect physical signs before launching a parameter sweep.

Guided Walkthrough 3

Run an Organized Simulation Study

Learning objective: Replace ad-hoc scripting with a reproducible experiment sweep driven by the scope file.

Now we run the actual capstone experiment. The factors are ambient temperature, resistance scale, and plateau-width scale. With three values each, we have 27 runs. That is small enough for a laptop and large enough to support a sensitivity-style figure. The code will save every time series and one summary table. This matters

because you should never have to rerun simulations just to make a new plot.

Create `src/run_capstone_sweep.py`:

```
from typing import List
from typing import Dict
import sys
import os

from typing import Dict
import argparse

def main(
    data_dir: str = "data/processed/timeseries",
    tables_dir: str = "tables",
    save_figures: bool = True,
    save_tables: bool = True,
) -> None:
    """
    Main function to run the capstone sweep.
    """
    with open("config/capstone_scope.yaml", "r", encoding="utf-8") as file:
        scope = yaml.safe_load(file)["project"]

    assumptions = scope["fixed_assumptions"]
    factors = scope["factors"]

    profile_time_s, profile_current_a = make_pulsed_current_profile(
        pulse_current_a=assumptions["pulse_current_a"],
        pulse_duration_s=assumptions["pulse_duration_s"],
        rest_duration_s=assumptions["rest_duration_s"],
        number_of_pulses=assumptions["number_of_pulses"],
    )

    summary_rows = []
    run_index = 0

    for ambient_c, resistance_scale, plateau_scale in product(
        factors["ambient_temperature_c"],
        factors["resistance_scale"],
        factors["plateau_width_scale"],
    ):
        run_index += 1
        run_id = (
```

```

        f"run_{run_index:03d}_Tamb_{ambient_c:g}_"
        f"R_{resistance_scale:g}_P_{plateau_scale:g}"
    ).replace(".", "p")

parameters = CellParameters(
    capacity_ah=assumptions["cell_capacity_ah"],
    lower_voltage_cutoff_v=assumptions["lower_voltage_cutoff_v"],
    upper_voltage_cutoff_v=assumptions["upper_voltage_cutoff_v"],
    resistance_scale=float(resistance_scale),
    plateau_width_scale=float(plateau_scale),
)

frame = simulate_cell(
    profile_time_s=profile_time_s,
    profile_current_a=profile_current_a,
    parameters=parameters,
    ambient_temperature_c=float(ambient_c),
    initial_soc=assumptions["initial_soc"],
    initial_temperature_c=assumptions["initial_temperature_c"],
    sample_period_s=1.0,
)

frame["run_id"] = run_id
frame["resistance_scale"] = resistance_scale
frame["plateau_width_scale"] = plateau_scale

frame.to_csv(f"data/processed/timeseries/{run_id}.csv", index=False)

summary = summarize_run(frame, parameters)
summary.update(
    {
        "run_id": run_id,
        "ambient_temperature_c": ambient_c,
        "resistance_scale": resistance_scale,
        "plateau_width_scale": plateau_scale,
    }
)
summary_rows.append(summary)
print(
    f"{run_id}: energy={summary['usable_energy_wh']:.2f} Wh, "
    f"minV={summary['minimum_voltage_v']:.2f} V, "
    f"Tmax={summary['peak_temperature_c']:.1f} C"
)

summary_table = pd.DataFrame(summary_rows)
ordered_columns = [

```

```

        "run_id",
        "ambient_temperature_c",
        "resistance_scale",
        "plateau_width_scale",
        "usable_energy_wh",
        "minimum_voltage_v",
        "peak_temperature_c",
        "cutoff_time_s",
        "final_soc",
        "voltage_sag_v",
        "lower_voltage_cutoff_v",
    ]
    summary_table = summary_table[ordered_columns]
    summary_table.to_csv("tables/capstone_summary.csv", index=False)

    print()
    print("Wrote tables/capstone_summary.csv")
    print(summary_table.head())

if __name__ == "__main__":
    main()

```

Run:

```
python src/run_capstone_sweep.py
```

Expected output should print 27 lines, one per run. The cold, high-resistance cases should generally have lower minimum voltage, larger voltage sag, and higher resistive heating during pulses. The plateau-width effect may be subtler than resistance or ambient temperature; that itself is a finding. The summary table should contain 27 rows.

This code deserves a block-by-block reading. `itertools.product` is the experiment design: every combination of factor values is run once. The `run_id` encodes the factor values so a CSV file can be traced back to the table without opening metadata. The parameter object is rebuilt for each run rather than mutated in place, which avoids state leakage. The raw time series are saved separately from the summary metrics. That separation is standard practice: figures often need time series, while tables and statistics need summaries.

What could go wrong? If only one or two runs are produced, check YAML indentation under `factors`. If every case has the same result, verify that

`resistance_scale` and `plateau_width_scale` are passed into `CellParameters`. If file names contain awkward characters, keep the `.replace(".", "p")` step or use a cleaner run-index-only naming scheme with metadata in the table. If a reviewer later asks for an additional factor level, add it to YAML and rerun the sweep; do not hand-edit `capstone_summary.csv`.

This exercise taught the third capstone habit: organized experimentation is a data product. The sweep script, time-series files, and summary table together are the computational experiment.

Guided Walkthrough 4

Analyze Results and Produce Publication-Quality Figures

Learning objective: Turn simulation outputs into defensible figures, tables, and written findings.

A figure is not decoration. A figure is an argument. In a short technical report, each figure should answer one question. Here we will make three figures: an OCV family plot to show what the plateau-width parameter means, a representative time-series plot to show model behavior, and a heatmap of usable energy to show the main capstone result.

Create `src/make_capstone_figures.py`:

```
from matplotlib.figure import Figure
import matplotlib.pyplot as plt
import numpy as np
import sys

from mpl_toolkits.mplot3d import Axes3D

def plot_surface(x, y, z):
    fig = plt.figure(figsize=(10, 8))
    ax = fig.gca(projection='3d')
    ax.plot_surface(x, y, z, cmap=cm.coolwarm)

    fig.tight_layout()
    fig.savefig('surface.png', dpi=300)
    plt.close()

    return None

if __name__ == '__main__':
    x = np.linspace(0, 10, 100)
    y = np.linspace(0, 10, 100)
    z = np.zeros((100, 100))

    for i in range(100):
        for j in range(100):
            z[i, j] = 0.5 * x[i] * y[j]

    plot_surface(x, y, z)
```

```

        "axes.titlesize": 10,
        "legend.fontsize": 8,
        "xtick.labelsize": 8,
        "ytick.labelsize": 8,
    }
)

def make_ocv_figure() -> None:
    soc = np.linspace(0.0, 1.0, 400)
    fig, ax = plt.subplots(figsize=(5.4, 3.4))
    for scale in [0.75, 1.0, 1.25]:
        ax.plot(
            soc,
            hard_carbon_ocv(soc, plateau_width_scale=scale),
            linewidth=2.0,
            label=f"Plateau scale {scale:g}",
        )
    ax.set_xlabel("State of charge (-)")
    ax.set_ylabel("Open-circuit voltage (V)")
    ax.set_title("Virtual hard-carbon sodium-ion OCV family")
    ax.legend(frameon=False)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig("figures/fig1_ocv_family.png")
    plt.close(fig)

def make_representative_timeseries(summary: pd.DataFrame) -> None:
    nominal = summary[
        (summary["ambient_temperature_c"] == 25.0)
        & (summary["resistance_scale"] == 1.0)
        & (summary["plateau_width_scale"] == 1.0)
    ].iloc[0]
    frame = pd.read_csv(f"data/processed/timeseries/{nominal['run_id']}.csv")

    fig, axes = plt.subplots(3, 1, figsize=(6.4, 5.6), sharex=True)
    time_min = frame["time_s"] / 60.0

    axes[0].plot(time_min, frame["current_a"], color="#1f77b4",
    ↪ linewidth=1.3)
        0          "Current (A)"

    axes[1].plot(time_min, frame["terminal_voltage_v"], color="#d62728",
    ↪ linewidth=1.5)

```

```

axes[1].plot(time_min, frame["ocv_v"], color="#444444", linewidth=1.1,
↳ linestyle="--")
axes[1].axhline(1.5, color="black", linewidth=1.0, linestyle=":")
axes[1].set_ylabel("Voltage (V)")
axes[1].legend(["Terminal", "OCV", "Cutoff"], frameon=False, loc="best")

axes[2].plot(time_min, frame["temperature_c"], color="#ff7f0e",
↳ linewidth=1.5)
axes[2].set_ylabel("Temperature (deg C)")
axes[2].set_xlabel("Time (min)")

for ax in axes:
    ax.grid(True, alpha=0.3)

fig.tight_layout()
fig.savefig("figures/fig2_nominal_timeseries.png")
plt.close(fig)

def make_energy_heatmaps(summary: pd.DataFrame) -> None:
    plateau_values = sorted(summary["plateau_width_scale"].unique())
    fig, axes = plt.subplots(1, len(plateau_values), figsize=(9.2, 3.2),
↳ sharey=True)

    for ax, plateau_scale in zip(axes, plateau_values):
        subset = summary[summary["plateau_width_scale"] == plateau_scale]
        pivot = subset.pivot(
            index="ambient_temperature_c",
            columns="resistance_scale",
            values="usable_energy_wh",
        )
        sns.heatmap(
            pivot,
            annot=True,
            fmt=".1f",
            cmap="viridis",
            cbar=ax if ax is axes[-1],
            cbar_kws={"label": "Usable energy (Wh)"},
            ax=ax,
        )
        ax.set_title(f"Plateau scale {plateau_scale:g}")
        ax.set_xlabel("Resistance scale")
        ax.set_ylabel("Ambient temp. (deg C)" if ax is axes[0] else "")

    fig.tight_layout()

```

```

fig.savefig("figures/fig3_usable_energy_heatmaps.png")
plt.close(fig)

def write_ranked_table(summary: pd.DataFrame) -> None:
    ranked = summary.sort_values("usable_energy_wh", ascending=False).copy()
    ranked["usable_energy_wh"] = ranked["usable_energy_wh"].round(2)
    ranked["minimum_voltage_v"] = ranked["minimum_voltage_v"].round(3)
    ranked["peak_temperature_c"] = ranked["peak_temperature_c"].round(2)
    ranked["voltage_sag_v"] = ranked["voltage_sag_v"].round(3)
    ranked.head(10).to_csv("tables/top_10_cases.csv", index=False)
    ranked.tail(10).to_csv("tables/bottom_10_cases.csv", index=False)

def main() -> None:
    Path("figures").mkdir(parents=True, exist_ok=True)
    configure_plots()
    summary = pd.read_csv("tables/capstone_summary.csv")

    make_ocv_figure()
    make_representative_timeseries(summary)
    make_energy_heatmaps(summary)
    write_ranked_table(summary)

    print("Wrote:")
    print("  figures/fig1_ocv_family.png")
    print("  figures/fig2_nominal_timeseries.png")
    print("  figures/fig3_usable_energy_heatmaps.png")
    print("  tables/top_10_cases.csv")
    print("  tables/bottom_10_cases.csv")

if __name__ == "__main__":
    main()

```

Run:

```
python src/make_capstone_figures.py
```

Figure 1 should show three smooth OCV curves versus SOC. All curves should have a sodium-ion-like lower nominal voltage than graphite/NMC lithium-ion cells, and the plateau-scale parameter should visibly change the width of the flatter middle region. Figure 2 should show the nominal pulsed current, terminal voltage sagging below OCV during each pulse, and cell temperature rising modestly.

Figure 3 should show three heatmaps. The axes are ambient temperature and resistance scale; each panel is a plateau-width scale. Correct heatmaps should show a clear energy penalty as resistance increases, especially in colder conditions. If every heatmap cell has the same number, your sweep did not actually vary the parameters.

Now inspect the top and bottom tables:

```
python - <<'PY'
import pandas as pd

print("Top cases")
print(pd.read_csv("tables/top_10_cases.csv")[[
    "ambient_temperature_c",
    "resistance_scale",
    "plateau_width_scale",
    "usable_energy_wh",
    "minimum_voltage_v",
    "peak_temperature_c",
]])

print("\nBottom cases")
print(pd.read_csv("tables/bottom_10_cases.csv")[[
    "ambient_temperature_c",
    "resistance_scale",
    "plateau_width_scale",
    "usable_energy_wh",
    "minimum_voltage_v",
    "peak_temperature_c",
]])
PY
```

The best cases should cluster around warmer ambient temperature and lower resistance scale. The worst cases should cluster around cold ambient temperature and higher resistance scale. Plateau width may reorder cases within those clusters. That gives you the skeleton of a Results paragraph: resistance and temperature dominate usable energy under this protocol; OCV plateau shape changes voltage margin and cutoff timing but is secondary for the selected factor range.

What could go wrong? A heatmap can hide uncertainty by looking more authoritative than the model deserves. Label the model as virtual. If figure text is too small, increase `context` or figure size but do not cram more panels into one figure. If your report has more than one main conclusion per figure, split the figure

or simplify the claim. If a trend surprises you, inspect a representative time series before writing a mechanism story.

This exercise taught the fourth capstone habit: analysis is the bridge between simulation and claim. A good figure tells the reader what comparison you made and what changed.

Guided Walkthrough 5

Generate the Report, Poster Skeleton, and Research Log

Learning objective: Create paper-ready artifacts from the same project metadata and results used by the code.

The deliverable is not complete until the written artifacts exist. We will generate a lightweight IEEE-style report template and a poster outline. You will still write the prose yourself, but the script will insert the title, research question, assumptions, factor table, and key result placeholders. This prevents the common failure where the report and code quietly disagree.

Create `src/generate_artifacts.py`:

```
from          import
from          import

import        as
import
from          import

                = r"""
\documentclass[conference]{IEEEtran}
\usepackage{graphicx}
\usepackage{booktabs}
\usepackage{siunitx}
\usepackage{amsmath}

\title{ {{ title }} }
\author{\IEEEauthorblockN{Your Name}
\IEEEauthorblockA{Department of Electrical Engineering\
Your Institution\
email@example.com}}

\begin{document}
```

```

\maketitle

\begin{abstract}
This capstone studies {{ research_question }} The study uses a transparent
virtual sodium-ion equivalent-circuit and lumped-thermal model. The purpose
↪ is
to demonstrate a reproducible simulation research workflow, not to claim
validated performance for a commercial sodium-ion cell.
\end{abstract}

\section{Introduction}
Write one paragraph on why sodium-ion batteries are relevant for cost,
materials availability, safety, or low-temperature applications. Write one
paragraph narrowing the problem to pulsed-load usable energy. End with the
research question and the contribution of this capstone.

\section{Model and Assumptions}
The virtual cell uses a first-order Thevenin model with SOC, polarization
voltage, and lumped cell temperature as states. Current is positive during
discharge. The terminal voltage is
\begin{equation}
V = U_{\mathrm{ocv}}(z) - I R_0(z,T) - v_1 .
\end{equation}
State clearly that the OCV curve is hard-carbon-like but synthetic. Report
↪ the
fixed assumptions in Table~\ref{tab:assumptions}.

\begin{table}[h]
\centering
\caption{Fixed assumptions used in the capstone simulation.}
\label{tab:assumptions}
\begin{tabular}{ll}
\toprule
Quantity & Value \\
\midrule
Capacity & {{ capacity_ah }} Ah \\
Lower voltage cutoff & {{ lower_voltage_cutoff_v }} V \\
Initial SOC & {{ initial_soc }} \\
Pulse current & {{ pulse_current_a }} A \\
Pulse/rest duration & {{ pulse_duration_s }} s / {{ rest_duration_s }} s \\
Number of pulses & {{ number_of_pulses }} \\
\bottomrule
\end{tabular}
\end{table}

```

```

\section{Methodology}
Describe the full-factorial sweep over ambient temperature, resistance scale,
and OCV plateau-width scale. State the primary metric, usable delivered
↪ energy,
and the secondary metrics: minimum voltage, peak temperature, cutoff time,
↪ and
voltage sag.

\section{Results}
Insert Fig.~\ref{fig:ocv}, Fig.~\ref{fig:timeseries}, and
Fig.~\ref{fig:heatmap}. Write the results in comparison language:
↪ higher/lower
than, more/less sensitive than, dominated by, limited by.

\begin{figure}[h]
\centering
\includegraphics[width=0.95\linewidth]{../figures/fig1_ocv_family.png}
\caption{Virtual hard-carbon sodium-ion OCV family.}
\label{fig:ocv}
\end{figure}

\begin{figure}[h]
\centering
\l
↪ \includegraphics[width=0.95\linewidth]{../figures/fig2_nominal_timeseries.png}
\caption{Nominal pulsed-load simulation.}
\label{fig:timeseries}
\end{figure}

\begin{figure}[h]
\centering
\l
↪ \includegraphics[width=0.95\linewidth]{../figures/fig3_usable_energy_heatmaps.png}
\caption{Usable energy across the capstone factor sweep.}
\label{fig:heatmap}
\end{figure}

\section{Discussion}
Discuss what the virtual study can and cannot support. Explain why cold
temperature and resistance scaling affect voltage sag. Explain whether
↪ plateau
width produced a first-order or second-order effect under this protocol.

\section{Limitations and Future Work}
List the most important limitations: synthetic OCV, no electrode-resolved

```

physics, no aging, simplified heat generation, and no direct sodium-ion experimental validation. Then identify one concrete next step.

`\section{Conclusion}`

State the main finding in one careful sentence. Do not overclaim.

`\bibliographystyle{IEEEtran}`

`\begin{thebibliography}{1}`

`\bibitem{plett2015}`

G. L. Plett, *Battery Management Systems, Volume II: Equivalent-Circuit Methods*. Artech House, 2015.

`\bibitem{pybamm2021}`

V. Sulzer et al., ``Python Battery Mathematical Modelling (PyBaMM),''

Journal of Open Research Software, vol. 9, no. 1, 2021.

`\bibitem{hwang2017}`

J.-Y. Hwang, S.-T. Myung, and Y.-K. Sun, ``Sodium-ion batteries: present and future,'' *Chemical Society Reviews*, vol. 46, pp. 3529--3614, 2017.

`\end{thebibliography}`

`\end{document}`

"""

`POSTER_TEMPLATE = """# {{ title }}`

`## Research Question`

`{{ research_question }}`

`## Why It Matters`

Write three short sentences: sodium-ion motivation, pulsed-load relevance,
↪ and

why simulation helps before experimental data are available.

`## Method`

- Virtual hard-carbon-like sodium-ion ECM plus lumped thermal model
- Full-factorial sweep over ambient temperature, resistance scale, and OCV plateau-width scale
- Primary metric: `{{ primary_metric }}`

`## Main Figures`

1. OCV family
2. Nominal pulsed-load time series
3. Usable-energy heatmaps

`## Key Result`

Replace this paragraph with your strongest quantitative comparison from tables/top_10_cases.csv and tables/bottom_10_cases.csv.

Limitations

Synthetic OCV, no electrode-resolved validation, simplified thermal model,
↪ and
no direct sodium-ion experimental dataset.

Next Step

Replace the virtual OCV/resistance assumptions with a digitized or measured sodium-ion cell dataset.

"""

```
def append_log(scope: dict, summary: pd.DataFrame) -> None:
    log_path = Path("logs/research-log.md")
    best = summary.sort_values("usable_energy_wh", ascending=False).iloc[0]
    worst = summary.sort_values("usable_energy_wh", ascending=True).iloc[0]
    entry = f"""
```

```
## {date.today().isoformat()} - Capstone sweep completed
```

```
Research question: {scope["research_question"].strip()}
```

The full-factorial sweep produced {len(summary)} runs. The best usable-energy case was `{best["run\_id"]}` with {best["usable_energy_wh"]:.2f} Wh. The
↪ lowest

usable-energy case was `{worst["run\_id"]}` with
↪ {worst["usable_energy_wh"]:.2f}

Wh. The current interpretation is that resistance scaling and ambient temperature dominate the selected pulsed-load protocol. This needs to be
↪ stated

as a virtual-cell result, not as a validated sodium-ion performance claim.
"""

```
    with log_path.open("a", encoding="utf-8") as file:
        file.write(entry)
```

```
def main() -> None:
```

```
    Path("reports").mkdir(parents=True, exist_ok=True)
```

```
    with Path("config/capstone_scope.yaml").open("r", encoding="utf-8") as
        ↪ file:
        scope = yaml.safe_load(file)["project"]
```

```

assumptions = scope["fixed_assumptions"]
summary = pd.read_csv("tables/capstone_summary.csv")

report_context = {
    "title": scope["title"],
    "research_question": scope["research_question"].strip(),
    "primary_metric": scope["primary_metric"],
    **assumptions,
}
report_text = Template(REPORT_TEMPLATE).render(**report_context)
poster_text = Template(POSTER_TEMPLATE).render(
    title=scope["title"],
    research_question=scope["research_question"].strip(),
    primary_metric=scope["primary_metric"],
)

Path("reports/capstone_report.tex").write_text(report_text.strip() +
↳ "\n", encoding="utf-8")
Path("reports/capstone_poster_outline.md").write_text(poster_text,
↳ encoding="utf-8")
append_log(scope, summary)

print("Wrote reports/capstone_report.tex")
print("Wrote reports/capstone_poster_outline.md")
print("Appended logs/research-log.md")

if __name__ == "__main__":
    main()

```

Run:

```
python src/generate_artifacts.py
```

You should now have `reports/capstone_report.tex`, `reports/capstone_poster_outline.md`, and a `research-log` entry. The report will not be finished. It is a scaffold with the correct sections, figures, assumptions, and citations. Your job is to turn placeholders into argument. The poster outline is deliberately sparse because a poster is not a compressed paper. It needs one research question, one method block, one main result, and one honest limitation.

What could go wrong? If the report cannot find figures when compiled, remember

that LaTeX paths are relative to the report file. The template uses `../figures/...` because `capstone_report.tex` lives in `reports/`. If Jinja2 reports an undefined variable, check that the YAML assumption name matches the template name. If your research log remains empty, confirm that `tables/capstone_summary.csv` exists before running the artifact generator.

This exercise taught the fifth capstone habit: write artifacts from the same metadata that drives the code. Consistency is not a moral virtue; it is an engineering practice.

Dataset Integration

Using Public Data Without Pretending It Is Sodium-Ion Validation

For this capstone, public lithium-ion datasets are useful for workflow validation and scale checking, not for sodium-ion chemistry validation. CALCE HPPC data can help you compare whether your virtual resistance and polarization magnitudes are plausible. NASA aging data can help you practice extracting capacity and temperature traces. Oxford and Severson/MATR data can help you benchmark cycle-life feature workflows. None of those datasets proves a sodium-ion claim.

If you want to include a public-data appendix, use the loader from Lab Chapter 11 and add a short script that computes voltage sag during pulses from a CALCE HPPC file. The columns should be normalized to `time_s`, `current_a`, `voltage_v`, and `temperature_c`. Compute pulse sag as the difference between pre-pulse rest voltage and loaded voltage after a fixed delay. Then compare the order of magnitude with `voltage_sag_v` from this capstone. The comparison can say, “the virtual cell’s pulse sag is within the broad range seen in public 18650 lithium-ion HPPC data.” It cannot say, “the sodium-ion model is validated.”

For sodium-ion data, search Mendeley Data, Zenodo, and journal supplementary information using terms such as `sodium-ion battery OCV`, `hard carbon full cell cycling data`, `Na-ion pouch cell pulse`, and the specific cathode chemistry. Record the URL, license, file size, instrument if stated, temperature, voltage limits, and current sign convention. If the only available information is a plot in a PDF, use WebPlotDigitizer and cite the original paper. Treat digitized curves as approximate: report that they were digitized, include the digitization file, and avoid overfitting.

Reproduction Exercise

Reproduce a Published OCV Shape Qualitatively

For the Part VI reproduction requirement, use this chapter to reproduce a qualitative hard-carbon sodium-ion OCV feature rather than an entire paper. Read Hwang, Myung, and Sun, “Sodium-ion batteries: present and future,” *Chemical Society Reviews*, 2017, and one recent hard-carbon full-cell or half-cell paper relevant to your chosen chemistry. Identify a figure showing the sloping-plus-plateau voltage behavior of hard carbon or a sodium-ion full cell. Your task is not to copy the data exactly unless the source provides numerical data. Your task is to tune `hard_carbon_ocv` so that the simulated curve has the same qualitative regions: low-SOC rise, plateau or quasi-plateau, high-SOC rise, and realistic full-cell voltage range.

Document the reproduction in `logs/research-log.md`. Include the citation, figure number, whether data were available or digitized, what you changed in the OCV function, and what mismatch remains. “Close enough” means the curve supports the capstone mechanism being studied. If your claim depends on the exact derivative dU/dz , qualitative reproduction is not enough; you must digitize or obtain the original data.

This exercise is valuable because it forces you to separate visual mechanism matching from validation. A sodium-ion OCV curve borrowed from the literature can make a virtual study more realistic, but it does not validate resistance, thermal behavior, aging, or dynamic polarization.

What Changes for Sodium-Ion?

Sodium-ion changes the capstone in four practical ways. First, parameter availability is weaker. You may find OCV curves and cycling summaries, but full DFN parameter sets with transport, kinetic, and thermodynamic functions are much rarer than for common lithium-ion chemistries. Second, hard-carbon OCV shapes can have broad plateaus that reduce voltage sensitivity to SOC in some ranges, which matters for SOC estimation, pulse recovery interpretation, and voltage-limit control. Third, nominal voltage and energy density differ, so direct Wh comparisons to lithium-ion must be framed carefully. Fourth, validation strategy changes: you will often validate the workflow on lithium-ion benchmark

data, then use sodium-ion-specific literature only for the chemistry-sensitive pieces.

For a publishable sodium-ion simulation paper, be painfully explicit about which parameters are sodium-ion-specific, which are borrowed analogues, which are fitted, and which are varied in sensitivity analysis. Reviewers are usually tolerant of sparse data when the uncertainty is acknowledged. They are much less tolerant of lithium-ion assumptions silently relabeled as sodium-ion.

14.4 Open-Ended Exercises

14.4.1 Exercise 1: Add a constant-current discharge comparison

Modify the sweep so each parameter combination is simulated under both the pulsed profile and a constant-current discharge with the same average current. Hint: the pulsed profile uses 6 A for one-third of the time, so the average current is 2 A. Add a column named `protocol` to the summary table.

Worked solution outline: use

```
make_constant_current_profile(current_a=2.0,  
duration_s=3600.0) and wrap the existing sweep body in a loop over two  
protocol definitions. Save time series under file names that include protocol.
```

14.4.2 Exercise 2: Replace the synthetic OCV curve with digitized data

Find a sodium-ion OCV or low-rate voltage curve in a paper or dataset. Use WebPlotDigitizer to export SOC and voltage points as CSV. Replace `hard_carbon_ocv` with an interpolation function using `np.interp`. Hint: enforce monotonic SOC ordering and clip extrapolation at the endpoints.

Worked solution outline: read the CSV with `pandas`, sort by SOC, normalize SOC to 0-1 if necessary, and write `return np.interp(z, soc_points, voltage_points)`. Add the source citation and digitization notes to the research log.

14.4.3 Exercise 3: Add one uncertainty band

Choose one uncertain scalar parameter, such as `r0_ohm` or `heat_transfer_w_per_k`, and run five values around the nominal value. Plot usable energy as a band rather than a single heatmap value. Hint: keep the

original three-factor sweep fixed and add uncertainty only for the nominal plateau scale to avoid a runaway experiment count.

Worked solution outline: create a second sweep script named `run_uncertainty_sweep.py`, vary `r0_ohm` over `[0.032, 0.036, 0.040, 0.044, 0.048]`, group by ambient temperature and resistance scale, and plot minimum-to-maximum usable energy with `fill_between`.

14.4.4 Exercise 4: Write the self-review before polishing the report

Before editing prose, write a one-page reviewer report on your own capstone. Use three headings: “Major concerns,” “Minor concerns,” and “What would make the claim stronger.” Hint: if you cannot find any major concerns, you are being too friendly to yourself.

Worked solution outline: at least one major concern should mention the virtual nature of the sodium-ion parameters. Another should mention missing experimental validation. A strong minor concern might mention the simplified heat-generation model. A constructive strengthening step might be replacing the OCV curve with digitized data or adding a PyBaMM comparison.

Chapter Summary and Skill Checklist

- You scoped a capstone question tightly enough to finish.
- You wrote project assumptions and factors in a machine-readable YAML file.
- You implemented a virtual sodium-ion ECM plus thermal model in reusable Python code.
- You ran a full-factorial simulation sweep and saved both time series and summary metrics.
- You produced publication-style figures and ranked result tables.
- You generated report and poster scaffolds from project metadata.
- You practiced sodium-ion-specific caution: virtual parameters are useful, but they are not validation.

Commands and functions that should now be in muscle memory:

- `python src/print_scope.py`
- `python src/run_single_case.py`
- `python src/run_capstone_sweep.py`

- `python src/make_capstone_figures.py`
- `python src/generate_artifacts.py`
- `CellParameters`
- `simulate_cell`
- `summarize_run`
- `pd.read_csv`
- `DataFrame.to_csv`
- `plt.savefig`

You should now be able to:

- Define a simulation research question with explicit metrics and factors.
- Keep assumptions, code, figures, tables, and writing synchronized.
- Build a small but complete computational experiment repository.
- Explain what a virtual sodium-ion model can and cannot claim.
- Produce a short IEEE-style report skeleton from your results.
- Review your own work with enough skepticism to improve it before someone else does.

Deliverable

The deliverable is a complete capstone package:

- A GitHub-style repository folder with reproducible code
- A short technical report, 8-12 pages in IEEE conference structure
- A one-page poster or poster outline
- A research log documenting decisions, failures, and pivots

For the guided capstone, the repository is

`SimulationCompanion/chapter14_capstone_workspace`. A strong submission contains the `config`, `src`, `tables`, `figures`, `reports`, and `logs` folders; a `README.md` explaining how to rerun the project; and a report that states one careful finding supported by Figure 3 and the ranked tables. Do not bury the limitation that the sodium-ion cell is virtual. Put it in the abstract, methods, and discussion. That honesty makes the project stronger, not weaker.

Further Practice and Reading

Read Plett's *Battery Management Systems, Volume II* for equivalent-circuit modeling discipline, especially the habit of validating against held-out data. Bookmark the PyBaMM documentation and Sulzer et al.'s PyBaMM paper if you extend the capstone to SPMe or DFN simulations. Read Hwang, Myung, and Sun's 2017 *Chemical Society Reviews* article for sodium-ion context and chemistry-specific caution. For public data practice, return to the CALCE Battery Data Archive, NASA PCoE battery aging data, Oxford Battery Degradation Dataset, and the Severson/MATR fast-charge dataset from Lab Chapter 11. Community resources worth keeping close are the PyBaMM GitHub repository, the PyBaMM discussion forum or Discord, MATLAB File Exchange examples for battery ECMs, and reproducibility checklists from journals in energy storage and computational modeling.

The next chapter is beyond the planned companion: at this point you should either polish the capstone into a submission-quality internal report or branch it into a real research project with stronger sodium-ion data, a physics-based model, or an experimentally validated parameter set.